Curso de Python

Do zero ao primeiro sistema profissional

Curso de Python: do zero ao primeiro sistema profissional
- O que tem aqui
- Mapa do curso
- Como começar
- Como cada módulo funciona
- Ritmo sugerido
- Regras do jogo
- Depois do curso
- Para quem mantém o curso

Plano de estudos: a trilha semana a semana
- Como o curso ensina (e por que funciona)
- A rotina de cada dia (60 minutos)
- A rotina da semana
- A trilha
- Quando travar
- Diário de bordo
- Depois das 31 semanas

Módulo 00 — Preparação do ambiente
- O que você vai aprender
- Por que isso importa
- 1. O que é programar (e o que o Python faz com o seu código)
- 2. Instalando o Python no Windows
- 3. VS Code e a extensão Python
- 4. Git for Windows: instalar e configurar
- 5. O terminal (PowerShell) sem medo
- 6. O REPL: conversando com o Python
- 7. Seu primeiro script
- 8. Baixando o curso: clonar (ou atualizar) o repositório
- 9. Ambiente virtual (venv) e pip
- 10. O molde dos exercícios: `def` e `return`
- 11. Seu primeiro teste: o ciclo do curso
- 12. Git no dia a dia: status, add, commit, push, pull
- 13. Como estudar este curso
- Lendo mensagens de erro deste módulo
- Boas práticas
- No mundo real
- Resumo
- Você domina este módulo se...
- Exercícios
- Desafio do módulo

Quiz
Para ir além
Enunciados dos exercícios
Desafio 00 — Ambiente pronto e primeiro push
Quiz — Módulo 00: Preparação do ambiente

Módulo 01 — Fundamentos
O que você vai aprender
Por que isso importa
1. Dois jeitos de rodar Python: REPL e script
2. print: o programa falando com você
3. Comentários: notas para humanos
4. Variáveis e nomes
5. Tipos de dados: int, float, str, bool e None
6. Operadores aritméticos e precedência
7. Atribuição composta: atualizando valores
8. Strings básicas: trabalhando com texto
9. Conversões: int(), float() e str()
10. input(): perguntando ao usuário
11. f-strings e formatação de números
12. O problema do float e o round()
13. O módulo math
14. O molde dos exercícios: def e return
15. Pensando como programador
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 01 — Ficha de medição da bobina
Quiz do módulo 01 — Fundamentos

Módulo 02 — Decisões e repetições
O que você vai aprender
Por que isso importa
1. Comparações: perguntas de sim ou não
2. Combinando condições: and, or e not
3. if, elif e else: o programa decide

4. Expressão condicional: um if numa linha
5. Truthy e falsy: o que conta como verdadeiro
6. match-case: escolhendo entre vários casos
7. while: repetir enquanto for verdade
8. for e range: repetir para cada item
9. Percorrendo uma lista recebida
10. break e continue: controlando o laço por dentro
11. Contadores e acumuladores
12. Padrões clássicos: buscar, contar, maior e menor
13. Laços aninhados
14. Validação de dados: não confie na entrada
15. Depurando laços: o teste de mesa
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 02 — Inspetor de série no terminal
Quiz do módulo 02 — Decisões e repetições

Módulo 03 — Coleções: listas, tuplas, dicionários e conjuntos
O que você vai aprender
Por que isso importa
1. Listas: vários valores numa variável só
2. Fatiamento: recortando pedaços
3. Modificando listas: os métodos
4. Mutabilidade: cópia ou referência?
5. Tuplas e desempacotamento
6. Percorrendo melhor: `enumerate` e `zip`
7. Dicionários: buscar pelo nome, não pela posição
8. Dicionários aninhados: a tabela de tolerâncias
9. Conjuntos: sem repetição, com matemática
10. Compreensões: montar coleções numa linha
11. Funções embutidas que economizam laços
12. Lista de dicionários: sua primeira tabela
13. Qual coleção usar?
Lendo mensagens de erro deste módulo

Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 03 — Painel de conferência do lote
Quiz do módulo 03 — Coleções

Módulo 04 — Funções: organize, reutilize e decomponha problemas
O que você vai aprender
Por que isso importa
1. Funções: dar nome a uma ideia
2. Parâmetros e argumentos
3. Valores padrão (e a armadilha do padrão mutável)
4. Vários retornos e retorno antecipado
5. Escopo: onde cada nome vive
6. Docstrings e type hints
7. `*args` e `**kwargs`: quantos argumentos vierem
8. Funções são valores
9. `lambda` e o parâmetro `key=`
10. Funções puras e efeitos colaterais
11. Decomposição de problemas
12. Recursão (introdução)
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 04 — Fila de emissão de certificados
Quiz do módulo 04 — Funções

Módulo 05 — Texto e expressões regulares
O que você vai aprender
Por que isso importa

1. Strings por dentro
2. Procurando dentro do texto
3. Limpando e padronizando
4. Quebrando e juntando: split, partition e join
5. Formatação avançada com f-strings
6. Números no formato brasileiro
7. Unicode e acentos
8. Encodings: o mistério do "ExpediÃ§Ã£o"
9. Expressões regulares: a ideia
10. Construindo um padrão aos poucos
11. search, match, fullmatch, findall e finditer
12. Grupos e grupos nomeados
13. Substituindo e dividindo: re.sub e re.split
14. Flags e re.compile
15. Projeto guiado: lendo o e-mail da expedição
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 05 — Leitor de e-mails da expedição
Quiz — Módulo 05: Texto e expressões regulares

Módulo 06 — Arquivos e tratamento de erros
O que você vai aprender
Por que isso importa
1. Quando algo dá errado: exceções
2. try/except: tratando o erro
3. else e finally
4. raise: levantando seus próprios erros
5. Exceções personalizadas
6. Arquivos de texto: open e with
7. pathlib: caminhos sem dor
8. Encoding na prática: Excel, utf-8-sig e cp1252
9. CSV no padrão do Excel brasileiro
10. JSON: configurações e regras
11. Leitura e escrita segura

12. Validação defensiva
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 06 — Conferência de laudos em lote
Quiz — Módulo 06: Arquivos e tratamento de erros

Módulo 07 — Módulos, pacotes e biblioteca padrão
O que você vai aprender
Por que isso importa
1. Módulo: um arquivo `.py` que outro arquivo usa
2. As formas de importar
3. `if __name__ == "__main__":` — script e módulo ao mesmo tempo
4. Onde o Python procura módulos
5. Pacotes: pastas com `__init__.py`
6. Estrutura de projeto profissional: o src layout
7. Ambiente virtual e pip a fundo
8. Poetry: o gerenciador que já está no seu repositório
9. Datas e prazos com `datetime`
10. `collections`: Counter, defaultdict e namedtuple
11. `itertools`: chain e groupby
12. `statistics`: a estatística do laboratório
13. `math` e `random`
14. `os`, `sys` e `time`
15. `argparse`: programas de linha de comando
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios

Desafio do Módulo 07 — Agenda de validade de laudos
Quiz do Módulo 07 — Módulos, pacotes e biblioteca padrão

Módulo 08 — Orientação a objetos
O que você vai aprender
Por que isso importa
1. O problema: dados soltos
2. Classes e objetos
3. `__init__` e `self`
4. Métodos e atributos de classe
5. `__str__` e `__repr__`
6. Encapsulamento: o que é de dentro e o que é de fora
7. `@property`: atributo com validação
8. `@classmethod` e `@staticmethod`
9. Herança e `super()`
10. Polimorfismo e "duck typing"
11. Composição: "tem um" em vez de "é um"
12. `dataclasses`: classes de dados sem repetição
13. Igualdade: `is`, `==`, `__eq__` e `__hash__`
14. `Enum`: conjuntos fechados de valores
15. Outros métodos especiais
16. Quando NÃO usar classes
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do Módulo 08 — O Validador de Certificados orientado a objetos
Quiz do Módulo 08 — Orientação a objetos

Módulo 09 — Testes e qualidade de código
O que você vai aprender
Por que isso importa
1. Por que testar
2. `assert`: a peça mais simples
3. pytest: primeiros passos
4. Lendo uma falha do pytest
5. Anatomia de um bom teste

6. Floats e erros: `pytest.approx` e `pytest.raises`
7. Vários casos: `@pytest.mark.parametrize`
8. Fixtures: preparando o terreno
9. Arquivos de verdade em pasta temporária: `tmp_path`
10. O que foi impresso e o que foi registrado: `capsys` e `caplog`
11. Trocando peças: `monkeypatch`
12. Exemplo resolvido: testes para uma função que você não escreveu
13. TDD passo a passo: vermelho → verde → refatorar
14. Cobertura de testes (conceito)
15. ruff: lint e formatação automáticos
16. Type hints e mypy (conceito)
17. logging: os prints dos profissionais
18. Depuração: `breakpoint()` e o debugger do VS Code
19. Git no dia a dia
20. Código limpo e refatoração

Lendo mensagens de erro deste módulo
Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 09 — Do script legado ao código profissional
Quiz — Módulo 09: Testes e qualidade de código

Módulo 10 — Análise de dados com pandas, Excel e gráficos

O que você vai aprender
Por que isso importa
1. Preparando o terreno
2. Series e DataFrame
3. Lendo CSV brasileiro
4. Lendo Excel: uma aba, várias, todas
5. Inspecionando uma base nova
6. Selecionando linhas e colunas
7. Criando e alterando colunas
8. Tipos, texto, ausentes e duplicatas
9. Datas
10. Ordenando
11. Agrupando: groupby e agg

12. Juntando tabelas: merge e concat
13. Tabela dinâmica: pivot_table
14. Vetorização x apply x laço for
15. Exportando: CSV e Excel formatado
16. Gráficos com matplotlib
17. Gráfico de controle (média ± 3σ)
18. Laboratório guiado: o mercado de dados em 133 mil salários
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 10 — Relatório mensal da qualidade
Quiz — Módulo 10: pandas, Excel e gráficos

Módulo 11 — Bancos de dados e SQL (SQLite)
O que você vai aprender
Por que isso importa
1. O que é um banco de dados relacional
2. Primeira conexão com sqlite3
3. Criando tabelas: tipos e restrições
4. Inserindo dados com parâmetros
5. O banco do laboratório
6. SELECT: escolher, filtrar, ordenar
7. Agregação: GROUP BY e HAVING
8. JOINs: juntando tabelas
9. Subconsultas e CTEs
10. UPDATE e DELETE
11. Transações: commit, rollback e with
12. Linhas com nome: row_factory
13. SQL injection na prática
14. Índices
15. Modelando o banco de regras do projeto final
16. SQL e pandas: read_sql e to_sql
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real

Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 11 — Ferramenta do banco de regras
Quiz — Módulo 11: bancos de dados e SQL

Módulo 12 — Automação e integração
O que você vai aprender
Por que isso importa
1. O mapa da automação: API, arquivo ou tela?
2. HTTP em 10 minutos
3. requests: conversando com APIs
4. Falhas de rede: timeout, erros e novas tentativas
5. Testar sem internet: injeção de dependência e monkeypatch
6. Arquivos e pastas em lote (pathlib + shutil)
7. Relatórios Excel com openpyxl
8. E-mail: montar, salvar e ler mensagens
9. Outlook no Windows: clássico, novo e Microsoft Graph
10. Agendamento: o Agendador de Tarefas do Windows
11. Automação web com Playwright
12. O robô de certificados: modo ensaio, log, prints e idempotência
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 12 — Robô da expedição (modo ensaio)
Quiz — Módulo 12: Automação e integração

Módulo 13 — APIs web com FastAPI
O que você vai aprender
Por que isso importa
1. O que é uma API REST
2. A primeira API

3. Testando com TestClient
4. Parâmetros de caminho
5. Parâmetros de consulta
6. Corpo com Pydantic
7. Códigos de status e HTTPException
8. Organizando em módulos
9. Dependências
10. Persistência com SQLite
11. Documentação automática
12. Rodando de verdade: uvicorn e um cliente requests
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real
Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 13 — API de Certificados do laboratório
Quiz — Módulo 13: APIs com FastAPI

Módulo 14 — Projeto final: Validador de Certificados
O que você vai aprender
Por que isso importa
1. O problema, em uma página
2. Arquitetura em camadas
3. Decisões de projeto (e por quê)
4. Os dados de exemplo
5. Roteiro de construção
6. Domínio e regras (etapas 1 a 6)
7. Entradas (etapas 7 a 10)
8. Saídas e orquestração (etapas 11 a 14)
9. Montagem final: o projeto-completo
10. O robô do TOTVS, isolado
11. Do curso para o trabalho
12. Portfólio no GitHub
13. Extensões
Lendo mensagens de erro deste módulo
Boas práticas
No mundo real

Resumo
Você domina este módulo se...
Exercícios
Desafio do módulo
Quiz
Para ir além
Enunciados dos exercícios
Desafio do módulo 14 — A API do Validador (extensão FastAPI)
Quiz — Módulo 14: Projeto final

Guia de carreira: do laboratório de qualidade à programação

Resumo em 1 minuto
1. Python vale a pena ou devo mudar de linguagem?
2. As trilhas possíveis
3. O que estudar além deste curso
4. Portfólio com os projetos do curso
5. GitHub e LinkedIn
6. Como conseguir a primeira oportunidade
7. Como se preparar para entrevistas técnicas
8. Plano de 6 meses e plano de 12 meses
Fontes

Recursos complementares

Qual recurso usar em cada fase do curso
1. Documentação oficial
2. Livros
3. Cursos em vídeo gratuitos
4. Plataformas de exercícios
5. Comunidades brasileiras
6. Dados, BI, Excel e estatística
7. Mapas de carreira
Cuidados

Curso de Python: do zero ao primeiro sistema profissional

Um curso completo, prático e em português para aprender a programar de verdade com Python, pensado para quem trabalha na indústria e quer migrar para dados, automação ou desenvolvimento.

Você não vai só assistir: vai **escrever código todos os dias**. Cada exercício tem testes automáticos que dizem na hora se o seu código está certo. No fim, você terá construído o núcleo de um sistema real (o Validador de Certificados) e terá um portfólio para mostrar.

O que tem aqui

- **15 módulos** com aula completa, exemplos executáveis, exercícios corrigidos automaticamente, desafio e quiz.
- **191 exercícios** conferidos por **1.868 testes automáticos**, do mais fácil (★☆☆) ao desafiador (★★★), com exercícios de revisão que misturam assuntos de módulos anteriores.
- **Soluções oficiais comentadas** para cada exercício (abra só depois de tentar).
- **Projeto final** baseado num problema real de laboratório de qualidade.
- **Guia de carreira**: qual trilha seguir, o que estudar além do curso e como conseguir a primeira oportunidade.
- **Apostila em PDF** (versão A4 e versão para celular) com todas as aulas, enunciados, desafios e quizzes, na pasta `apostila/`.
- **Ferramenta** `curso.py` que testa seus exercícios e mostra seu progresso.

Mapa do curso

Módulo	Assunto	O que você passa a conseguir fazer
00	Preparação do ambiente	Instalar Python, VS Code e Git, e rodar o primeiro teste
01	Fundamentos	Variáveis, tipos, contas e textos formatados
02	Decisões e repetições	Programas que decidem e repetem: if, for, while
03	Coleções	Guardar e organizar dados: listas, dicionários, conjuntos

Módulo	Assunto	O que você passa a conseguir fazer
04	Funções	Quebrar problemas grandes em peças pequenas e reutilizáveis
05	Texto e expressões regulares	Ler e-mails, limpar textos, converter números brasileiros
06	Arquivos e erros	Ler e gravar CSV e JSON, tratar erros sem o programa quebrar
07	Módulos e biblioteca padrão	Organizar projetos, datas, estatística, programas de terminal
08	Orientação a objetos	Modelar o mundo real com classes e dataclasses
09	Testes e qualidade	Escrever testes, usar Git, depurar e manter código limpo
10	Dados com pandas	Analisar planilhas e CSV, fazer gráficos, gerar relatórios Excel
11	Banco de dados e SQL	Criar e consultar bancos de dados com SQL
12	Automação e integração	APIs, e-mail, Excel, pastas e robôs no navegador
13	APIs com FastAPI	Criar sua própria API web, com testes
14	Projeto final	Construir o Validador de Certificados de ponta a ponta

Os módulos 01 a 09 são a base de qualquer carreira com Python. Do 10 em diante você entra nas ferramentas que o mercado pede para dados e automação.

Como começar

1. Leia `PLANO-DE-ESTUDOS.md` (10 minutos): ele mostra o caminho e a rotina.
2. Abra `00-preparacao/README.md` e siga o passo a passo para preparar o computador.
3. Na pasta `curso-python`, com o ambiente virtual ativado, rode:

   ```
   python curso.py modulos
   python curso.py testar 00
   ```

4. Faça um módulo de cada vez, na ordem.

Como cada módulo funciona

```
NN-nome-do-modulo/
  README.md      a aula: leia com calma e digite os exemplos
  exemplos/      os códigos da aula, prontos para rodar
  exercicios/    ONDE VOCÊ TRABALHA: complete as funções
  testes/        os testes que corrigem seus exercícios (não precisa
mexer)
  solucoes/      as soluções oficiais comentadas (abra só depois de
tentar)
  desafio/       um mini projeto para juntar tudo
  quiz.md        perguntas de revisão com gabarito
```

O ciclo de cada exercício:

1. Abra `exercicios/exNN_....py` e leia o enunciado no topo do arquivo.
2. Apague a linha `raise NotImplementedError(...)` e escreva o seu código.
3. Rode `python curso.py testar NN exMM`.
4. Se falhar, leia a mensagem: ela mostra o que era esperado e o que seu código devolveu.
5. Passou? Compare com `solucoes/` para aprender outra forma de pensar.

Para ver o quanto você já avançou:

```
python curso.py progresso
python curso.py progresso --detalhes
```

Ritmo sugerido

Com **1 hora por dia, 5 dias por semana**, o curso leva cerca de **31 semanas (7 meses)**. Com 2 horas por dia, cerca de 4 meses. Mais importante que a velocidade é a constância: programar um pouco todo dia ensina mais do que uma maratona por semana.

Fase	Módulos	Semanas (1h/dia)	Marco de portfólio
Base da linguagem	00 a 04	1 a 9	Primeiro programa útil
Programas de verdade	05 a 08	10 a 17	Ferramenta de laboratório
Profissional	09 a 11	18 a 24	Análise de dados com SQL e pandas

Fase	Módulos	Semanas (1h/dia)	Marco de portfólio
Mundo real	12 a 14	25 a 31	Validador de Certificados

O passo a passo de cada semana, a rotina diária e o método de estudo estão em `PLANO-DE-ESTUDOS.md`. Leia antes de começar.

Regras do jogo

- **Digite, não copie.** A memória dos dedos é parte do aprendizado.
- **Tente de verdade antes de abrir a solução.** Travou por mais de 30 minutos? Releia a aula, teste pedaços no terminal do Python e só então olhe a solução.
- **Erro é informação.** Toda mensagem de erro diz o que aconteceu e em qual linha.
- **Use IA como professor, não como cola.** Peça explicações e dicas, não a resposta pronta.
- **Faça commit do seu progresso** no GitHub. Ele vira o seu portfólio.

Depois do curso

Leia `carreira/README.md` para escolher a trilha (dados, automação ou backend) e montar o plano dos próximos meses, e `carreira/recursos.md` para livros, vídeos e comunidades.

Para quem mantém o curso

```
python curso.py solucoes             # todas as soluções oficiais passam nos testes
ruff check . && ruff format --check .
python ferramentas/gerar_apostila.py # gera os PDFs (precisa de pandoc e Chrome/Edge)
```

Plano de estudos: a trilha semana a semana

Este é o caminho completo, na ordem certa, com o que fazer em cada semana. Ele foi montado para **1 hora por dia, 5 dias por semana**. Com mais tempo, avance mais rápido, mas não pule as revisões.

Duração total: **cerca de 31 semanas (7 meses)**. No fim você terá 4 projetos de portfólio e o núcleo de um sistema real do seu trabalho.

Como o curso ensina (e por que funciona)

O curso usa métodos que a pesquisa em ensino de programação mostra que funcionam melhor do que só assistir ou só ler:

Método	Como aparece no curso	Por que funciona
PRIMM (Preveja, Rode, Investigue, Modifique, Crie)	Blocos "🔮 Preveja antes de rodar" e "🔧 Modifique" nas aulas	Prever obriga você a pensar como o computador; errar a previsão mostra exatamente o que você ainda não entendeu
Exemplos resolvidos	Antes de cada assunto novo, um problema resolvido passo a passo	Iniciante aprende mais vendo o raciocínio completo antes de tentar sozinho
Prática com correção imediata	Testes automáticos em todo exercício	Você descobre o erro na hora, não dias depois
Revisão espaçada e intercalada	Exercícios `_revisao_` e as sextas-feiras de revisão	Revisar com intervalos e misturando assuntos fixa o conteúdo na memória de longo prazo
Recuperação ativa	Quiz sem consultar e o "resumo de memória" diário	Lembrar sem olhar fortalece a memória mais do que reler
Projeto contínuo e real	O Validador de Certificados cresce ao longo do curso	Aprender com um problema que importa para você mantém a motivação

Para saber mais: Sentance, Waite e Kallia, "Teaching computer programming with PRIMM" (2019); Sweller, sobre exemplos resolvidos e carga cognitiva; Dunlosky e colegas, "Improving Students' Learning With Effective Learning Techniques" (2013), sobre prática espaçada e recuperação ativa.

A rotina de cada dia (60 minutos)

Tempo	O que fazer
5 min	**Resumo de memória:** sem olhar nada, escreva no seu diário 3 coisas que lembra da última aula
20 min	**Aula:** leia a próxima seção, digite os exemplos, faça os "Preveja antes de rodar"
30 min	**Exercícios:** um ou dois, rodando `python curso.py testar NN exMM`
5 min	**Diário de bordo:** o que aprendeu, onde travou, e o commit do dia

Sexta-feira é dia de **revisão** (veja abaixo). Se travar mais de 30 minutos num exercício, siga o roteiro da seção "Quando travar".

A rotina da semana

- **Segunda a quinta:** aula e exercícios do módulo atual.
- **Sexta (revisão espaçada):**
  1. Refaça o quiz do módulo **anterior** sem consultar.
  2. Escolha 1 exercício de **2 a 3 módulos atrás**, apague sua solução e refaça do zero.
  3. Rode `python curso.py progresso` e faça o commit da semana no GitHub.

A trilha

Fase 1 — A base da linguagem (semanas 1 a 9)

Semana	Módulo	Meta da semana
1	00 Preparação	Ambiente pronto, primeiro teste passando, primeiro commit no GitHub
2	01 Fundamentos	Aula até formatação de números; metade dos exercícios
3	01 Fundamentos	Exercícios restantes, quiz e desafio

Semana	Módulo	Meta da semana
4	02 Decisões e repetições	Aula e exercícios de if e for
5	02 Decisões e repetições	while, exercícios de revisão, quiz e desafio
6	03 Coleções	Listas, tuplas e dicionários
7	03 Coleções	Conjuntos, compreensões, "tabela" de dicionários, desafio
8	04 Funções	Aula completa e exercícios
9	04 Funções	Exercícios de revisão e **Marco 1**

> 🏁 **Marco 1 — Seu primeiro programa útil.** O desafio do módulo 04 vira o primeiro projeto do portfólio. Critério: roda no terminal, tem README explicando o que faz e está no GitHub.

Fase 2 — Programas de verdade (semanas 10 a 17)

Semana	Módulo	Meta da semana
10	05 Texto e regex	Strings a fundo, números no formato brasileiro, acentos
11	05 Texto e regex	Expressões regulares e leitura dos e-mails da expedição
12	06 Arquivos e erros	Exceções, with/open, pathlib
13	06 Arquivos e erros	CSV padrão Excel, JSON, desafio
14	07 Módulos e biblioteca padrão	Módulos, pacotes, datas, estatística
15	07 Módulos e biblioteca padrão	argparse (programas de terminal), desafio
16	08 Orientação a objetos	Classes, property, herança e composição
17	08 Orientação a objetos	dataclasses, Enum e **Marco 2**

> 🏁 **Marco 2 — Ferramenta de laboratório.** O desafio do módulo 08 vira o segundo projeto do portfólio.

> 💼 **No trabalho:** a partir da semana 13 você já consegue automatizar uma tarefa pequena e real (ex.: juntar CSVs do laboratório num só). Faça isso. É a primeira prova concreta do seu valor com programação.

Fase 3 — Nível profissional (semanas 18 a 24)

Semana	Módulo	Meta da semana
18	09 Testes e qualidade	pytest a fundo e TDD
19	09 Testes e qualidade	ruff, logging, depuração, Git com branches
20	10 Dados com pandas	DataFrame, leitura de CSV e Excel, filtros
21	10 Dados com pandas	groupby, merge, pivot, limpeza de dados
22	10 Dados com pandas	Gráficos com matplotlib e laboratório de salários
23	11 Banco de dados e SQL	SQL do SELECT ao JOIN
24	11 Banco de dados e SQL	sqlite3 no Python, banco de regras e **Marco 3**

> 🏁 **Marco 3 — Análise de dados com SQL e pandas.** O desafio do módulo 11 vira o terceiro projeto do portfólio. Junto com o laboratório de salários do módulo 10, é o que um recrutador de vaga de dados quer ver.

Fase 4 — Mundo real (semanas 25 a 31)

Semana	Módulo	Meta da semana
25	12 Automação e integração	APIs com requests, arquivos em lote, Excel
26	12 Automação e integração	E-mail, agendamento e robô no navegador
27	13 APIs com FastAPI	Criar e testar sua própria API
28	14 Projeto final	Etapas 1 a 4: modelos, regras e tolerâncias
29	14 Projeto final	Etapas 5 a 8: planilha, especificações e e-mails
30	14 Projeto final	Etapas finais: fila, relatório e programa de terminal
31	14 Projeto final	Montagem do projeto completo e **Marco 4**

> 🏁 **Marco 4 — Validador de Certificados.** Seu projeto principal: repositório próprio, README com prints, testes rodando no GitHub Actions. É o projeto que você apresenta na sua empresa e em entrevistas.

Quando travar

1. Leia a mensagem de erro de baixo para cima: a última linha diz o tipo do erro, as linhas acima dizem onde.
2. Releia a seção da aula sobre o assunto do exercício.
3. Teste pedaços do seu código no terminal do Python (`python` e depois os comandos).
4. Coloque `print()` nos pontos onde você não sabe o que está acontecendo.
5. Explique o problema em voz alta, linha por linha (funciona de verdade).
6. Pergunte a uma IA pedindo **uma dica**, não a resposta. Exemplo: "Não me dê o código. Meu teste falha com esta mensagem: ... O que devo investigar?"
7. Só então abra a solução. Depois, feche, espere um dia e refaça sem olhar.

Diário de bordo

Crie um arquivo `DIARIO.md` na raiz do seu repositório e escreva todo dia, em 3 linhas:

```
2026-11-03 — Módulo 02, ex05
Aprendi: break sai do laço mais próximo, não de todos.
Travei em: contador fora do laço zerava a cada volta. Resolvido movendo a linha.
```

Em poucas semanas esse diário mostra o quanto você evoluiu, e vira material para contar sua história em entrevistas.

Depois das 31 semanas

O curso te deixa no nível de júnior em Python. Para a primeira vaga, siga o plano de 6 e 12 meses do `carreira/README.md`. Em resumo:

- **Vaga de dados:** SQL todos os dias (exercícios online), Power BI e estatística básica.
- **Vaga de automação ou backend:** Docker, publicar uma API na nuvem e mais projetos.

- **Em qualquer caminho:** coloque o Validador em uso real na sua empresa. Um sistema funcionando em produção vale mais que qualquer certificado de curso.

Módulo 00 — Preparação do ambiente

> **Tempo estimado:** 4 a 6 horas (divida em 3 ou 4 dias) · **Pré-requisitos:** nenhum · **Nível:** iniciante

Este módulo é diferente dos outros. Quase não tem teoria de programação. Ele deixa o seu computador pronto, ensina as ferramentas que todo profissional usa todo dia (terminal, ambiente virtual, Git) e mostra como estudar o curso do jeito certo.

Não pule. Ambiente mal configurado é a causa número um de gente que desiste no começo.

O que você vai aprender

- Explicar, com suas palavras, o que é um programa e o que o Python faz com o seu código.
- Instalar o Python 3.12+ no Windows com "Add python.exe to PATH" e conferir no terminal.
- Instalar o VS Code com a extensão Python e o Git for Windows.
- Andar pelas pastas no PowerShell com `cd`, `ls` e caminhos relativos e absolutos.
- Usar o REPL (`>>>`) para testar uma ideia em segundos.
- Criar e rodar um script `.py`.
- Clonar o repositório `estudos`, criar e ativar um ambiente virtual e instalar as bibliotecas com `pip`.
- Resolver o erro "a execução de scripts foi desabilitada neste sistema".
- Rodar os testes de um exercício com `python curso.py testar 00 ex01` e ver seu progresso com `python curso.py progresso`.
- Ler um traceback de baixo para cima e achar a linha do erro.
- Salvar e enviar seu trabalho com `git add`, `git commit`, `git push` e `git pull`.
- Usar IA como professor particular, e não como cola.

Por que isso importa

Todo programador profissional, de analista de dados a desenvolvedor backend, faz isto todos os dias: abre o terminal, ativa um ambiente virtual, roda testes e envia código para o Git. Você vai aprender isso no primeiro dia, não no último.

Tem mais. Você já viu (ou vai ver) o problema "o script só funciona no computador do fulano". Ambiente virtual + `requirements.txt` + Git resolvem exatamente isso. É o que separa um script solto de um sistema que outra pessoa consegue rodar.

E o seu repositório no GitHub começa a virar portfólio hoje. O primeiro commit deste módulo é o primeiro tijolo do projeto final (o Validador de Certificados).

---

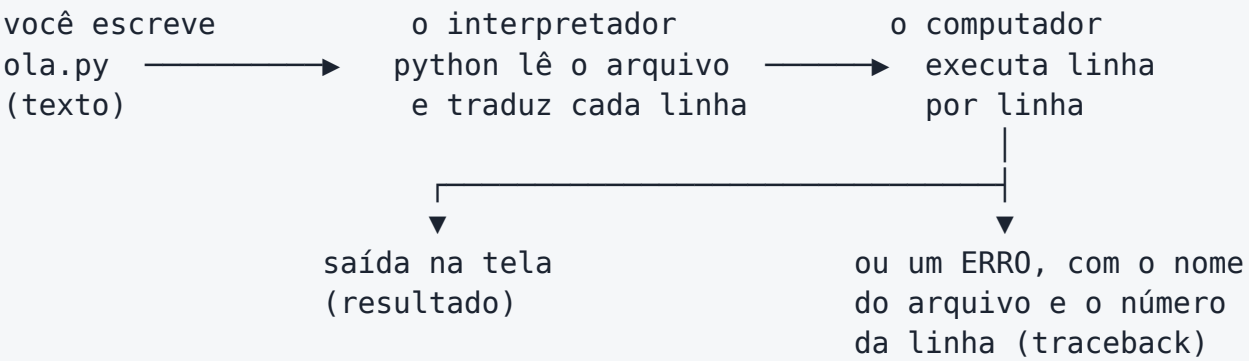
1. O que é programar (e o que o Python faz com o seu código)

Programar é escrever instruções exatas para o computador seguir.

Pense num POP (procedimento operacional padrão) de ensaio de tração. Um técnico segue os passos na ordem: corta o corpo de prova, mede a espessura, prende nas garras, roda o ensaio, anota o resultado. Se um passo estiver ambíguo, ele usa o bom senso.

O computador também segue os passos na ordem. Mas ele **não tem bom senso nenhum**. Ele faz exatamente o que está escrito, nem mais, nem menos. Se algo não estiver claro, ele para e reclama. Isso parece chato, mas é uma vantagem: o erro aparece na hora, com a linha exata.

O caminho do seu código até o resultado:



- **Código-fonte** é o texto que você escreve, num arquivo terminado em `.py`.
- **Interpretador** é o programa `python`. Ele lê o seu arquivo e executa as instruções, de cima para baixo.
- **Saída** é o que o programa mostra ou produz.

`print(...)` mostra na tela o que estiver entre os parênteses. Texto vai entre aspas.

> 🔮 **Preveja antes de rodar:** o que este programa mostra, e em que ordem?

```
print("Pesando a bobina...")
print(4 * 250)
print("Pronto")
```

<details>
<summary>Resposta</summary>

```
Pesando a bobina...
1000
Pronto
```

As linhas rodam de cima para baixo. Na segunda, `4 * 250` não está entre aspas: é uma conta, e o Python mostra o resultado (1000), não o texto "4 * 250".
</details>

Ainda não tem Python no seu computador? Vamos instalar.

---

2. Instalando o Python no Windows

2.1 Primeiro, confira se já existe

Abra o **PowerShell**: aperte a tecla Windows, digite `PowerShell` e Enter. Digite:

```
python --version
```

Três resultados possíveis:

O que aparece	O que significa
`Python 3.12.x`, `3.13.x` ou `3.14.x`	Já está instalado. Pule para a seção 3.
A Microsoft Store abre, ou nada acontece	Não está instalado (é só um atalho do Windows).
`O termo 'python' não é reconhecido...`	Não está instalado ou não está no PATH.

2.2 Qual versão instalar

Use **Python 3.14**: já tem um ano de correções e roda todas as bibliotecas do curso. As versões 3.12 e 3.13 também servem. O curso exige no mínimo 3.10, mas recomenda 3.12+.

A 3.15 saiu em outubro de 2026. Pode usar, mas saiba que em versões recém-lançadas alguma biblioteca às vezes demora a publicar o pacote pronto. Se a instalação das bibliotecas falhar (seção 9), volte para a 3.14.

2.3 Passo a passo com o instalador tradicional

1. Entre em `https://www.python.org/downloads/windows/`.
2. Procure a última versão **3.14.x** e baixe **Windows installer (64-bit)**.
3. Abra o instalador. Na **primeira tela**, lá embaixo, marque **"Add python.exe to PATH"**. Este é o passo mais importante.
4. Clique em **Install Now** e aguarde.
5. Se no final aparecer **"Disable path length limit"**, clique nele (pede permissão de administrador). Isso evita erros com caminhos de pasta muito longos. É opcional.
6. **Feche o PowerShell e abra de novo.** O terminal só enxerga o PATH novo depois disso.
7. Confira:

```
python --version
```

Deve aparecer algo como `Python 3.14.x`.

O que é **PATH**? É a lista de pastas onde o Windows procura um programa quando você digita um comando. É como a lista de ramais da empresa: se o ramal do Python não está na lista, o Windows responde "não conheço ninguém com esse nome".

> ⚠️ **Erro comum:** esquecer de marcar "Add python.exe to PATH". Solução: abra o instalador de novo, escolha **Modify**, avance e marque **"Add Python to environment variables"**. Depois feche e abra o terminal.

> ⚠️ **Erro comum:** digitar `python` e abrir a Microsoft Store, mesmo depois de instalar. São os "aliases de execução" do Windows. Abra **Configurações**, pesquise por **"aliases de execução de aplicativo"** e desligue os itens `python.exe` e `python3.exe` do "Instalador de Aplicativo".

2.4 Outras formas de instalar

O python.org agora recomenda, para Windows, o novo **Python install manager** (comando `py install 3.14`; aceite quando ele perguntar se deve adicionar a pasta de comandos ao PATH). O instalador da seção 2.3 continua disponível, mas foi marcado como obsoleto e deixará de ser publicado a partir do Python 3.16. Outra opção é o `winget`, que já vem no Windows 10/11: `winget install Python.Python.3.14`. Qualquer caminho serve: no fim, `python --version` precisa responder.

> 💡 **Dica:** em computador da empresa, a TI pode bloquear instalações. Use o seu computador pessoal para o curso ou peça a instalação à TI.

Linux e macOS: no Linux o Python costuma vir instalado (comando `python3`; no Ubuntu/Debian instale também `python3-venv`). No macOS use o instalador do python.org ou o Homebrew. Nesses sistemas, onde o curso escrever `python`, use `python3`.

---

3. VS Code e a extensão Python

O **VS Code** (Visual Studio Code) é o editor onde você vai escrever código. Ele é para o programador o que o Excel é para quem trabalha com planilhas: a ferramenta do dia a dia.

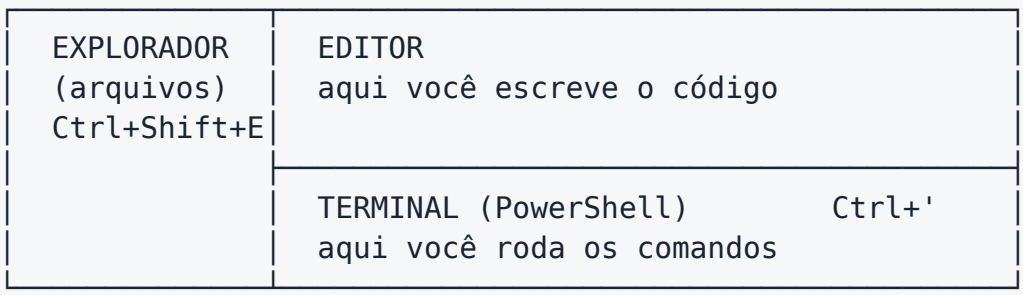
3.1 Instalar

1. Entre em `https://code.visualstudio.com` e baixe a versão para Windows.
2. Na tela "Selecionar tarefas adicionais", deixe marcado **"Adicionar ao PATH"** e marque **"Adicionar a ação 'Abrir com Code'"** (os dois menus de contexto).
3. Abra o VS Code.

3.2 Instalar a extensão Python

1. Aperte `Ctrl+Shift+X` (painel de extensões).
2. Pesquise **Python** e instale a da **Microsoft** (identificador `ms-python.python`). Ela já traz junto o Pylance (autocompletar) e o Python Debugger.

3.3 As partes que você vai usar




```
barra de status: mostra qual Python está em uso (canto inferior direito)
Paleta de comandos: Ctrl+Shift+P (procura qualquer função do VS Code)
```

> ⚠️ **Erro comum:** rodar o teste com o arquivo **não salvo**. O teste lê o arquivo do disco, não o que está na tela. Uma bolinha branca na aba significa "não salvo". Use `Ctrl+S` sempre, ou ligue **Arquivo → Salvamento Automático**.

> 💡 **Dica:** abra sempre a **pasta** do projeto (Arquivo → Abrir Pasta), não um arquivo solto. Assim o VS Code encontra o ambiente virtual e os testes.

---

4. Git for Windows: instalar e configurar

Git guarda o histórico do seu código. Cada **commit** é uma fotografia do projeto num momento, com data, autor e uma mensagem. Pense nas revisões de um procedimento controlado: rev. 00, rev. 01, rev. 02. Você sempre pode ver o que mudou e voltar atrás.

GitHub é um site que guarda uma cópia do seu repositório na nuvem. Serve de backup, permite trabalhar em mais de um computador e vira o seu portfólio.

4.1 Instalar

1. Entre em `https://git-scm.com/downloads/win` e baixe o **Git for Windows** (64-bit).
2. Avance com as opções padrão, com três atenções:
   - **Editor padrão:** escolha "Use Visual Studio Code as Git's default editor".
   - **Nome do branch inicial:** escolha "Override the default branch name" e deixe `main`.
   - **PATH:** deixe a opção recomendada, "Git from the command line and also from 3rd-party software".

3. Feche e abra o terminal. Confira:

```
git --version
```

Deve aparecer algo como `git version 2.x.x.windows.1`.

4.2 Dizer ao Git quem é você (uma vez por computador)

Todo commit leva nome e e-mail do autor. Use o **mesmo e-mail da sua conta do GitHub**:

```
git config --global user.name "Carlos Eduardo"
git config --global user.email "o-email-da-sua-conta@exemplo.com"
git config --global init.defaultBranch main
```

Para conferir: `git config --global --list`.

O login no GitHub acontece no primeiro `git push`: o Git for Windows abre uma janela do navegador para você autorizar. Você não digita senha no terminal.

---

5. O terminal (PowerShell) sem medo

O terminal é uma janela onde você dá ordens ao computador escrevendo. Parece antigo, mas servidores, robôs e ferramentas de dados são controlados assim. No VS Code: menu **Terminal → Novo Terminal** (ou `Ctrl+'`). No Windows ele abre o PowerShell.

O texto antes do cursor é o **prompt**. Ele mostra em que pasta você está:

```
PS C:\Users\carlos>
```

5.1 Os comandos que você vai usar

Comando	O que faz	Exemplo
`pwd`	mostra a pasta atual	`pwd`
`ls` (ou `dir`)	lista o que tem na pasta	`ls`
`cd pasta`	entra numa pasta	`cd projetos`
`cd ..`	volta uma pasta (sobe um nível)	`cd ..`
`cd ~`	vai para a sua pasta de usuário	`cd ~`

Comando	O que faz	Exemplo
`mkdir nome`	cria uma pasta	`mkdir projetos`
`cls` (ou `clear`)	limpa a tela	`cls`
`code .`	abre a pasta atual no VS Code	`code .`
`explorer .`	abre a pasta atual no Explorador de Arquivos	`explorer .`

Três atalhos que economizam horas:

- **Tab** completa nomes de pastas e arquivos. Digite `cd proj` e aperte Tab.
- **Seta para cima** traz os comandos anteriores.
- **Ctrl+C** interrompe um programa que travou.

5.2 Caminhos absolutos e relativos

- **Absoluto:** o endereço completo, desde o disco. `C:\Users\carlos\projetos\estudos`.
- **Relativo:** a partir de onde você está. Se você está em `C:\Users\carlos`, então `projetos\estudos` leva ao mesmo lugar.
- `.` é "a pasta atual". `..` é "a pasta de cima".

```
C:\Users\carlos
└── projetos
    └── estudos
        └── curso-python
            ├── 00-preparacao
            └── curso.py
```

> 🔮 **Preveja antes de rodar:** você está em `C:\Users\carlos\projetos\estudos\curso-python` e digita `cd ..\..`. Em que pasta você vai parar?

<details>
<summary>Resposta</summary>

Em `C:\Users\carlos\projetos`. Cada `..` sobe um nível: o primeiro sai de `curso-python` para `estudos`, o segundo sai de `estudos` para `projetos`.

</details>

> ⚠️ **Erro comum:** caminho com espaço sem aspas. `cd C:\Meus Projetos` falha. Use `cd "C:\Meus Projetos"`. Melhor ainda: crie pastas sem espaço e sem acento.

> ⚠️ **Erro comum:** guardar projetos em pastas sincronizadas pelo OneDrive (em muitos PCs, "Documentos" e "Área de Trabalho" são). O ambiente virtual tem milhares de arquivos e a sincronização atrapalha. Use `C:\Users\<seu-usuario>\projetos`.

No Linux e no macOS os comandos são quase os mesmos (`pwd`, `ls`, `cd`, `mkdir`), e as barras dos caminhos são `/` em vez de `\`.

---

6. O REPL: conversando com o Python

Digite `python` no terminal e aperte Enter. Aparece algo assim (a primeira linha muda conforme a sua versão):

```
Python 3.14.x (...) on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

O `>>>` é o **REPL** (Read-Eval-Print Loop): o Python lê o que você digita, calcula, mostra o resultado e espera a próxima instrução. É a calculadora de bolso do programador.

```
>>> 2 + 3
5
>>> 312.5 * 3
937.5
>>> 7 / 2
3.5
```

Para sair: digite `exit()` e Enter. (Atalho: no Windows, `Ctrl+Z` e Enter; no Linux e no macOS, `Ctrl+D`.)

> 🔮 **Preveja antes de rodar:** qual a diferença entre as duas saídas?

```
>>> "Olá"
>>> print("Olá")
```

<details>
<summary>Resposta</summary>

```
>>> "Olá"
'Olá'
```
</details>

```
>>> print("Olá")
Olá
```

Na primeira, o REPL mostra o **valor** como o Python o enxerga: um texto, por isso as aspas. Na segunda, o `print` mostra o texto **para humanos**, sem aspas. Guarde essa diferença: ela volta na seção 10 (print × return).

O REPL é o seu laboratório de hipóteses. "Será que `7 / 2` dá 3?" Não discuta com você mesmo: pergunte ao Python. Ele responde `3.5`, e a hipótese cai em 5 segundos.

> 💡 **Dica:** sempre que tiver dúvida sobre "o que acontece se...", abra o REPL e teste. É mais rápido que pesquisar e você lembra melhor do que descobriu sozinho.

---

7. Seu primeiro script

O REPL esquece tudo quando você sai. Um **script** é um arquivo `.py` que fica salvo e pode rodar quantas vezes quiser.

1. No terminal, crie uma pasta de rascunho e abra no VS Code:

```
cd ~
mkdir projetos
cd projetos
mkdir rascunho
cd rascunho
code .
```

2. No VS Code, crie o arquivo `ola.py` (ícone de novo arquivo no Explorador) e **digite**:

```
print("Olá, mundo!")
print("Meu primeiro script")
print(250 * 4)
```

3. Salve (`Ctrl+S`). No terminal do VS Code, rode:

```
python ola.py
```

Saída:

```
Olá, mundo!
Meu primeiro script
1000
```

Também dá para rodar pelo botão ▷ ("Run Python File") no canto superior direito do editor. Ele faz a mesma coisa: chama `python` com o caminho do arquivo.

> 🔧 **Modifique:** acrescente uma linha que mostre o seu nome e outra que calcule o peso de 3 bobinas de 312,5 kg (lembre: no código, número com vírgula se escreve com ponto, `312.5`). Rode de novo e confira.

> ⚠️ **Erro comum:** digitar `python ola.py` estando na pasta errada. Resultado: `can't open file ... [Errno 2] No such file or directory`. Confira com `ls` se o arquivo está na pasta atual.

---

8. Baixando o curso: clonar (ou atualizar) o repositório

O curso fica dentro do seu repositório `estudos`, no GitHub: `https://github.com/carlos-real/estudos`. **Clonar** é baixar o repositório inteiro, com todo o histórico, e deixá-lo ligado ao GitHub.

8.1 Se este computador ainda não tem o repositório

```
cd ~\projetos
git clone https://github.com/carlos-real/estudos.git
cd estudos\curso-python
ls
```

O `git clone` mostra algo como `Cloning into 'estudos'...` e alguns contadores de progresso. No fim você tem a pasta `estudos` com tudo dentro.

8.2 Se você já tinha clonado antes

Entre na pasta e traga as novidades:

```
cd ~\projetos\estudos
git pull
```

8.3 O que tem no repositório

```
estudos/
├── pyproject.toml     configuração do Poetry (dos seus estudos
anteriores)
├── src/               seu script antigo de salários
├── arquivos/          salaries.csv (será usado no módulo 10)
└── curso-python/      O CURSO: é aqui que você vai trabalhar
    ├── curso.py       a ferramenta que testa seus exercícios
    ├── requirements.txt
    ├── 00-preparacao/
    ├── 01-.../
    └── carreira/
```

Sobre o `pyproject.toml` na raiz: ele não atrapalha. Neste curso vamos usar `venv` + `pip` dentro de `curso-python`, que é o caminho mais simples para começar. O Poetry volta, a fundo, no módulo 07.

Agora abra a pasta do curso no VS Code:

```
cd ~\projetos\estudos\curso-python
code .
```

---

9. Ambiente virtual (venv) e pip

9.1 O problema

Python tem milhares de bibliotecas prontas (pandas, openpyxl, pytest...), instaladas com o **pip**. Se você instalar tudo "no Python do computador", um dia um projeto precisa do pandas 2 e outro do pandas 3: atualizou para um, quebrou o outro.

Pense nas bancadas do laboratório: cada linha tem o seu próprio jogo de calibradores, e trocar um calibrador da linha A não mexe na linha B. Um **ambiente virtual** é isso: uma pasta (`.venv`) com um Python e bibliotecas só daquele projeto.

9.2 Criar o ambiente virtual

No terminal, **dentro de `curso-python`**:

```
python -m venv .venv
```

Leva alguns segundos e cria a pasta `.venv`. O `-m` significa "rode o módulo": aqui, "Python, rode o módulo venv e crie um ambiente na pasta `.venv`". Isso se faz **uma vez** por computador.

9.3 Ativar

```
.venv\Scripts\Activate.ps1
```

Funcionou se o prompt ganhar `(.venv)` na frente:

```
(.venv) PS C:\Users\carlos\projetos\estudos\curso-python>
```

A partir daí, `python` e `pip` são os do ambiente virtual.

No Prompt de Comando (cmd), o comando é `.venv\Scripts\activate.bat`. No Linux e no macOS: `source .venv/bin/activate`.

9.4 O erro da política de execução (quase todo mundo vê)

Na primeira ativação, o PowerShell costuma recusar:

```
.venv\Scripts\Activate.ps1 : O arquivo C:\Users\carlos\...\Activate.ps1
não pode ser
carregado porque a execução de scripts foi desabilitada neste sistema.
Para obter mais
informações, consulte about_Execution_Policies em
https://go.microsoft.com/fwlink/?LinkID=135170.
```

(Em inglês: `cannot be loaded because running scripts is disabled on this system`.)

Por quê? No Windows 10 e 11 a política padrão do PowerShell é `Restricted`: ele não roda nenhum script `.ps1`, nem os criados no seu próprio computador. O `Activate.ps1` é um script, então é bloqueado.

Solução (vale só para o seu usuário e não precisa de administrador):

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Se pedir confirmação, digite `S` (Sim) e Enter. Depois rode o `Activate.ps1` de novo.

O que isso faz: `RemoteSigned` libera scripts criados no seu computador e exige assinatura digital dos baixados da internet. É o que a documentação da Microsoft recomenda para quem não assina scripts. Para ver a política atual: `Get-ExecutionPolicy -List`.

> ⚠️ **Erro comum:** em computador da empresa, a TI pode impor a política (Group Policy) e o comando não muda nada. Não tente burlar: estude no computador pessoal ou fale com a TI. (Se a TI liberar o Prompt de Comando, o `cmd`, a ativação nele é `.venv\Scripts\activate.bat`.)

9.5 Instalar as bibliotecas do curso

Com o `(.venv)` aparecendo no prompt:

```
python -m pip install -r requirements.txt
```

O `-r` diz "leia a lista deste arquivo". O `requirements.txt` do curso lista pytest, ruff, pandas, openpyxl, matplotlib, requests, fastapi, httpx e uvicorn. A instalação leva alguns minutos e termina com uma linha `Successfully installed ...`.

Para ver o que está instalado: `python -m pip list`.

> 💡 **Dica:** prefira `python -m pip` a só `pip`. Assim você garante que está usando o pip do mesmo Python que vai rodar o código (o do `.venv`).

9.6 Ensinar o VS Code a usar o `.venv`

1. `Ctrl+Shift+P` → digite **Python: Select Interpreter** → escolha o que tem `.venv`.
2. Abra um terminal novo (`Ctrl+Shift+'`). Normalmente o VS Code já ativa o `.venv` sozinho e o `(.venv)` aparece.

9.7 Conferir tudo de uma vez

O curso tem um verificador. Na pasta `curso-python`, com o `.venv` ativo:

```
python 00-preparacao/exemplos/04_verificar_ambiente.py
```

Saída quando está tudo certo (o número da versão será o do seu Python):

```
Verificando o ambiente do curso...

✔ Python 3.13.16
✔ Ambiente virtual ativo
✔ Bibliotecas do curso instaladas
✔ Git encontrado
✔ Terminal na pasta curso-python
```

```
Tudo pronto! Pode rodar:  python curso.py testar 00
```

🔮 **Preveja antes de rodar:** você abre um PowerShell novo, fora do VS Code, vai até `curso-python` e roda o verificador **sem ativar** o `.venv`. Que linha vai aparecer com ✘?

<details>
<summary>▼ Resposta</summary>

`✘ Ambiente virtual ativo`, com a dica de como ativar. Se as bibliotecas não estiverem instaladas no Python do sistema, também aparece `✘ Bibliotecas do curso instaladas`. Cada terminal novo começa **sem** ambiente ativo; é preciso ativar de novo (o VS Code faz isso por você quando o interpretador está selecionado).
</details>

Para sair do ambiente virtual: `deactivate`. Deu tudo errado? Apague a pasta `.venv` e crie de novo; não se perde nada.

⚠️ **Erro comum:** copiar a pasta `.venv` para outro computador ou mandar para o GitHub. Ela não funciona em outro lugar. O certo é recriar com os comandos acima. O curso já tem um `.gitignore` que impede o `.venv` de ir para o GitHub.

---

10. O molde dos exercícios: `def` e `return`

10.1 Rodando os exemplos da aula

Cada aula tem uma pasta `exemplos/` com scripts prontos. Rode sempre da pasta `curso-python`:

```
python 00-preparacao/exemplos/02_calculadora.py
```

🔮 **Preveja antes de rodar:** abra `00-preparacao/exemplos/02_calculadora.py`, leia o código e escreva num papel as 7 linhas que você acha que vão aparecer. Depois rode.

```
print("Peso de 4 bobinas de 250 kg:")
print(4 * 250)
```

```
print("Média de três medições de espessura (µm):")
print((50.2 + 49.8 + 51.0) / 3)

print("Precedência: multiplicação antes da soma")
print(2 + 3 * 4)
print((2 + 3) * 4)
```

▼ **Resposta**

```
Peso de 4 bobinas de 250 kg:
1000
Média de três medições de espessura (µm):
50.333333333333336
Precedência: multiplicação antes da soma
14
20
```

O `50.333333333333336` costuma surpreender: o computador guarda números com vírgula em binário e às vezes sobra um "resto" minúsculo no fim. Não é erro seu. O módulo 01 explica e ensina a arredondar. E o 14 e o 20 seguem a matemática da escola: multiplicação antes da soma, e parênteses mudam a ordem.

10.2 Função: uma máquina que recebe e devolve

Quase todo exercício do curso pede para você completar uma **função**. Pense num equipamento de ensaio: você coloca a amostra (entrada), ele faz o trabalho e devolve um resultado (saída). Você não precisa saber o que acontece lá dentro para usar o resultado.

```
def peso_liquido(peso_bruto_kg: float, tara_kg: float) -> float:
    """Devolve o peso líquido (kg): peso bruto menos a tara (tubete e
embalagem)."""
    return peso_bruto_kg - tara_kg
```

Peça por peça:

```
def peso_liquido(peso_bruto_kg: float, tara_kg: float) -> float:
|   |            |                                        |       |
|   |            └── parâmetros: as entradas              |       └── dois-
pontos
|   └── nome da função                                    └── tipo do
resultado (anotação)
└── "vou definir uma função"
```

```
    """Devolve o peso líquido..."""  ← docstring: explica o que a função
faz
    return peso_bruto_kg - tara_kg    ← return: DEVOLVE o resultado para
quem chamou
^^^^
4 espaços de recuo: é assim que o Python sabe o que está "dentro" da
função
```

Definir não executa nada. A função só trabalha quando é **chamada**, com valores entre os parênteses (um para cada parâmetro, na mesma ordem):

```
>>> peso_liquido(1050.0, 50.0)
1000.0
>>> peso_liquido(937.5, 37.5)
900.0
```

Os `: float` e `-> float` são **anotações de tipo** (type hints): avisos para você e para o VS Code sobre o que entra e o que sai. O Python não confere. Detalhes nos módulos 01 e 04.

> 💡 **Dica:** o recuo de 4 espaços não é enfeite. No VS Code, aperte Tab para recuar e Shift+Tab para voltar. Ele converte o Tab em 4 espaços.

10.3 `print` mostra, `return` devolve

Esta é a confusão número um de quem começa. Abra `00-preparacao/exemplos/03_molde_da_funcao.py`:

```
def dobro(numero: float) -> float:
    """Devolve o dobro de um número."""
    return numero * 2


def mensagem_liberacao(ensaio: str) -> str:
    """Devolve a observação de liberação de um ensaio, ex.: 'COF:
liberado'."""
    return ensaio + ": liberado"


def ola_com_print() -> None:
    """Só MOSTRA um texto na tela. Não devolve nada (devolve None)."""
    print("Olá (mostrado com print)")
```

```python
# Chamando as funções e mostrando o que elas DEVOLVEM:
print(dobro(21))
print(dobro(2.5))
print(mensagem_liberacao("COF"))
print(mensagem_liberacao("Selagem"))

# A diferença entre print e return:
resultado = ola_com_print()
print(resultado)
```

🔮 **Preveja antes de rodar:** as quatro primeiras linhas da saída são fáceis. E as duas últimas? O que fica guardado em `resultado`?

▼ **Resposta**

```
42
5.0
COF: liberado
Selagem: liberado
Olá (mostrado com print)
None
```

`ola_com_print()` **mostra** o texto na tela, mas não tem `return`. Toda função sem `return` devolve `None`, que é o jeito do Python dizer "nada". Então `resultado` guarda `None`, e o último `print` mostra `None`.

É por isso que os testes do curso reclamam quando você usa `print` no lugar de `return`: o teste recebe `None`, não o texto.

Duas coisas novas no exemplo, só o suficiente por agora:

- `resultado = ...` guarda um valor com um nome (uma **variável**). Módulo 01 a fundo.
- `ensaio + ": liberado"`: o `+` entre textos **junta** os textos. `"COF" + ": liberado"` vira `"COF: liberado"`. Outra forma, que você vai usar muito, é a f-string: `f"{ensaio}: liberado"` (módulo 01).

🔧 **Modifique:** no exemplo, crie uma função `triplo(numero)` que devolve o número vezes 3 e mostre `print(triplo(7))`. Depois troque o `return` de `dobro` por `print` e veja o que acontece com `print(dobro(21))`. (Desfaça depois.)

10.4 Como é um exercício do curso

Abra `00-preparacao/exercicios/ex01_ola_mundo.py`. Todo exercício tem este formato:

```
"""
Exercício 00.01 — Olá, mundo!
Nível: ★☆☆
(... contexto, tarefa, exemplos e dica ...)
Teste com:  python curso.py testar 00 ex01
"""


def ola_mundo() -> str:
    """Devolve o texto 'Olá, mundo!'."""
    # Escreva seu código aqui
    raise NotImplementedError("Implemente esta função")
```

- O texto entre `"""` no topo é o **enunciado**. Leia inteiro, inclusive os exemplos.
- `raise NotImplementedError(...)` é uma placa de "em construção": se alguém chamar a função, ela avisa que ainda não foi feita. **Apague essa linha** e escreva o seu código no lugar.
- Não mude o nome da função nem os parâmetros: os testes chamam exatamente esse nome.

10.5 Exemplo resolvido: a mensagem de liberação

Antes de você fazer os exercícios, veja como eu penso um problema parecido, passo a passo.

Enunciado: escreva `liberar(ensaio)`, que devolve a observação que vai na planilha, no formato `"<ensaio>: liberado"`. Exemplo: `liberar("COF")` devolve `"COF: liberado"`.

Passo 1 — entrada e saída. Primeiro eu pensei: o que entra? Um texto (o nome do ensaio). O que sai? Outro texto. Então a assinatura é:

```
def liberar(ensaio: str) -> str:
```

Passo 2 — resolver um caso concreto no REPL. Antes de pensar em "qualquer ensaio", resolvo para um ensaio só:

```
>>> "COF" + ": liberado"
'COF: liberado'
```

Passo 3 — generalizar. Agora troco o valor fixo `"COF"` pelo parâmetro `ensaio` (sem aspas, porque quero o conteúdo da caixa, não a palavra "ensaio"):

```python
def liberar(ensaio: str) -> str:
    """Devolve a observação de liberação do ensaio."""
    return ensaio + ": liberado"
```

Passo 4 — testar com mais de um caso. Inclusive um que não está no enunciado:

```
>>> liberar("COF")
'COF: liberado'
>>> liberar("Tração")
'Tração: liberado'
```

Passo 5 — conferir os detalhes. O enunciado pede dois-pontos e **um** espaço. Está certo. Pronto.

Esse roteiro (entrada/saída → caso concreto → generalizar → testar → conferir detalhes) resolve os três exercícios deste módulo e boa parte dos próximos.

---

11. Seu primeiro teste: o ciclo do curso

11.1 Como cada módulo é organizado

```
00-preparacao/
├── README.md      a aula (este arquivo)
├── exemplos/      scripts da aula, prontos para rodar
├── exercicios/    ONDE VOCÊ TRABALHA: complete as funções
├── testes/        os testes que corrigem seus exercícios (não mexa)
├── solucoes/      soluções oficiais comentadas (só depois de tentar!)
├── desafio/       um mini projeto
└── quiz.md        perguntas de revisão com gabarito
```

11.2 A ferramenta `curso.py`

Rode sempre de dentro da pasta `curso-python`, com o `.venv` ativo:

Comando	O que faz
`python curso.py modulos`	lista os módulos e quantos exercícios cada um tem
`python curso.py testar 00`	testa todos os exercícios do módulo 00

Comando	O que faz
`python curso.py testar 00 ex01`	testa só o exercício 01 do módulo 00
`python curso.py progresso`	mostra quantos exercícios você já resolveu
`python curso.py progresso -d`	o mesmo, com o detalhe de cada exercício

O exercício pode ser escrito como `ex01`, `ex1`, `01` ou `1`. O `-d` também pode ser escrito `--detalhes`.

11.3 Passo a passo do exercício 01

1. Rode o teste ANTES de mexer. Sim, ele vai falhar. Isso confirma que tudo está ligado:

```
python curso.py testar 00 ex01
```

Saída (trecho):

```
collecting ... collected 5 items

00-preparacao/testes/test_m00_ex01_ola_mundo.py::test_devolve_um_valor
FAILED [ 20%]
(...)
00-
preparacao/testes/test_m00_ex01_ola_mundo.py::test_sem_espacos_sobrando
FAILED [100%]

=================================== FAILURES
===================================
_____________________________ test_devolve_um_valor
_____________________________
00-preparacao/testes/test_m00_ex01_ola_mundo.py:7: in
test_devolve_um_valor
    resultado = ex.ola_mundo()
                ^^^^^^^^^^^^^^
00-preparacao/exercicios/ex01_ola_mundo.py:26: in ola_mundo
    raise NotImplementedError("Implemente esta função")
E   NotImplementedError: Implemente esta função
(...)
============================== 5 failed in 0.03s
==============================

✘ Ainda há testes falhando. Leia a mensagem acima: ela diz o que era
  esperado e o que seu código devolveu. Corrija e rode de novo.
```

Como ler: são 5 testes, todos `FAILED`. A linha que começa com `E` diz o motivo: `NotImplementedError`, ou seja, a função ainda está com a placa de "em construção". (No Windows, as barras dos caminhos podem aparecer como `\`.)

2. Escreva o código. Apague o `raise NotImplementedError(...)` e escreva o `return`.

> 🔮 **Preveja antes de rodar:** suponha que, em vez de `return "Olá, mundo!"`, você escreva `print("Olá, mundo!")`. Quantos dos 5 testes passam?

<details>
<summary>▼ Resposta</summary>

Nenhum. A função mostra o texto, mas devolve `None`. Trecho da saída real:

```
_____________________________ test_devolve_um_valor
_____________________________
00-preparacao/testes/test_m00_ex01_ola_mundo.py:8: in
test_devolve_um_valor
    assert resultado is not None, (
E   AssertionError: Sua função não devolveu nada (devolveu None). Você
usou print() em vez de return?
E   assert None is not None
---------------------------- Captured stdout call ------------------
----------
Olá, mundo!
```

Repare no "Captured stdout call": o pytest mostra o que o seu `print` escreveu na tela. O texto apareceu, mas não foi devolvido.
</details>

3. Se errar um detalhe, leia a diferença. Por exemplo, sem o acento (`return "Ola, mundo!"`), dois testes falham e um deles mostra:

```
E   AssertionError: assert 'Ola, mundo!' == 'Olá, mundo!'
E
E     - Olá, mundo!
E     ?   ^
E     + Ola, mundo!
E     ?   ^
```

A linha com `-` é o **esperado**, a linha com `+` é o que o **seu código devolveu**, e o `^` aponta exatamente a letra diferente.

4. Tudo certo. Com o código correto e o arquivo salvo, os 5 testes aparecem como `PASSED` e o fim da saída é:

```
============================== 5 passed in 0.02s
==============================

✔ Tudo certo! Pode seguir para o próximo exercício.
```

5. Compare com a solução oficial. Só agora abra `solucoes/ex01_ola_mundo.py`. Ela traz comentários sobre armadilhas e alternativas. Mesmo acertando, sempre dá para aprender algo.

6. Veja o seu progresso:

```
python curso.py progresso -d
```

```
Conferindo seus exercícios, aguarde...

  00 Preparação do ambiente                          ██████░░░░░░░░░░
1/3
   ex01✔  ex02·  ex03·
  01 Fundamentos                                     ░░░░░░░░░░░░░░░░
0/14
   ex01·  ex02·  ex03·  ...
  ...

Total: 1/191 exercícios (1%)
```

(Cada módulo aparece numa linha; o resumo acima foi encurtado.)

> ⚠️ **Erro comum:** rodar `python curso.py ...` fora da pasta `curso-python`. Resultado: `can't open file '...\curso.py': [Errno 2] No such file or directory`. Volte com `cd ~\projetos\estudos\curso-python`.

---

12. Git no dia a dia: status, add, commit, push, pull

12.1 O caminho de uma alteração

```
 sua pasta            área de preparação         histórico local
GitHub
 (arquivos editados)  (o que vai no commit)      (commits)
(nuvem)
        │   git add          │    git commit           │    git push
│
        └──────────────────►  └──────────────────────►
```



Analogia do laboratório:

- `git add`: colocar as amostras na bandeja do que vai ser registrado.
- `git commit`: emitir o laudo, com data, responsável e descrição.
- `git push`: enviar o laudo para o arquivo central (GitHub).
- `git pull`: buscar no arquivo central o que outros (ou você, em outro computador) registraram.

12.2 Salvando o exercício 01

Depois de passar no teste, na pasta `curso-python`:

```
git status
```

O Git mostra o que mudou (a mensagem pode aparecer em português, conforme a instalação):

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   00-preparacao/exercicios/ex01_ola_mundo.py

no changes added to commit (use "git add" and/or "git commit -a")
```

Coloque o arquivo na bandeja e confira de novo:

```
git add 00-preparacao/exercicios/ex01_ola_mundo.py
git status
```

Agora o arquivo aparece em `Changes to be committed` (vai no próximo commit). Registre e envie:

```
git commit -m "Resolve exercício 00.01 (olá, mundo)"
git push
```

No primeiro `push`, o navegador abre para você autorizar o acesso ao GitHub. Depois disso, a última linha do `push` mostra algo como `main -> main`. Abra `https://github.com/carlos-real/estudos` e veja o seu commit lá.

Para ver o histórico: `git log --oneline` (aperte `q` para sair, se a lista for longa).

> 🔮 **Preveja antes de rodar:** você resolveu o ex02, salvou e rodou direto `git commit -m "Resolve ex02"`, **sem** `git add`. O que acontece?

<details>
<summary>▼ Resposta</summary>

Nada é registrado. O Git responde:

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
    modified:   00-preparacao/exercicios/ex02_saudacao.py

no changes added to commit (use "git add" and/or "git commit -a")
```

O commit só leva o que está na área de preparação. Faltou o `git add`.
</details>

12.3 Boas mensagens de commit

Ruim	Boa
`ajustes`	`Resolve exercício 00.02 (saudação)`
`teste`	`Corrige acento na saudação do ex01`
`aaaa`	`Adiciona script de apresentação do desafio 00`

Regra prática: complete a frase "Este commit...". Verbo no presente, curto, específico.

12.4 `git pull` no começo, `git push` no fim

Comece cada sessão com `git pull` (traz novidades do curso ou do outro computador) e termine com `git add`, `git commit` e `git push`. O atalho `git add .` coloca na bandeja **tudo** o que mudou na pasta atual: rode `git status` antes para conferir o que vai junto.

> ⚠️ **Erro comum:** colocar no GitHub planilhas da empresa, laudos com nome de cliente, senhas ou arquivos `.env`. Se o repositório for público, qualquer pessoa vê. Use dados fictícios nos seus projetos.

---

13. Como estudar este curso

13.1 O método, em sete passos

1. **Leia a aula com o VS Code aberto.** Não só no celular. Ler sem testar dá a falsa sensação de que entendeu.
2. **Digite os exemplos, não copie e cole.** Digitar obriga a reparar em cada parêntese, aspas e recuo. É assim que os dedos e os olhos aprendem a sintaxe.
3. **Nos blocos 🔮, preveja antes de rodar.** Escreva a previsão num papel. Depois rode e compare.
4. **Faça os exercícios em ordem.** Eles sobem de dificuldade de propósito.
5. **Só abra a solução depois de tentar de verdade.** Travou por mais de 30 minutos, seguindo o roteiro da seção 13.4? Aí sim. E depois de ver a solução, apague o seu código e refaça sem olhar.
6. **Faça o quiz sem consultar nada.** Só depois confira o gabarito.
7. **Faça o desafio e dê commit/push.** O desafio junta tudo num programa de verdade.

13.2 Por que esses métodos funcionam

O curso usa técnicas com boa base em pesquisa sobre aprendizagem e ensino de programação. Saber o porquê ajuda a não pular etapas.

PRIMM: Preveja, Rode, Investigue, Modifique, Crie. Método criado por pesquisadores de ensino de computação no Reino Unido (Sue Sentance e colegas). Primeiro você **lê** e **prevê** o que um código faz; depois roda e confere; investiga o que não entendeu; modifica um detalhe; e só então cria algo do zero. Prever obriga a ler com atenção e mostra o que você *acha* que sabe. Errar a previsão é exatamente o momento em que você mais aprende. Nas aulas, isso aparece como 🔮 **Preveja antes de rodar** e 🔧 **Modifique**.

Exemplos resolvidos. Para quem está começando, ver um problema parecido resolvido passo a passo, com o raciocínio explicado, ensina mais do que se debater sozinho desde o início. A memória de trabalho de um iniciante enche rápido (é a ideia central da teoria da carga cognitiva, de John Sweller); o exemplo resolvido libera espaço para aprender a *estratégia*. Por isso cada aula mostra um "Exemplo resolvido" antes dos exercícios, como o da seção 10.5.

Prática deliberada com feedback imediato. Repetir o que já sabe não faz evoluir. O que faz é praticar no limite do que você consegue, com foco num ponto e com retorno rápido sobre o erro (ideia estudada por Anders Ericsson). Os testes são

esse retorno: em segundos você sabe o que estava esperado e o que o seu código devolveu. É como ter o resultado do ensaio na hora, em vez de esperar a reclamação do cliente.

Revisão espaçada e intercalada. A partir do módulo 02, os dois últimos exercícios de cada módulo se chamam `exNN_revisao_<tema>.py`. Eles misturam o assunto novo com conceitos de módulos anteriores. Dois efeitos estão em jogo:

- **Espaçamento:** revisar depois de alguns dias, quando já esqueceu um pouco, fixa muito mais do que estudar tudo de uma vez.
- **Intercalação:** misturar tipos de problema treina a parte mais difícil: decidir *qual* ferramenta usar, e não só *como* usá-la.

Por isso esses exercícios parecem mais difíceis. É de propósito.

Recuperação ativa. Tentar lembrar fortalece a memória mais do que reler (o "efeito de teste", estudado por Henry Roediger e Jeffrey Karpicke). Por isso o quiz é para ser feito **sem consultar**. Errou? Ótimo diagnóstico: volte àquela seção da aula. Outra forma simples: no começo de cada sessão, escreva de memória, em 3 linhas, o que aprendeu na anterior.

Projeto contínuo. Tudo no curso converge para o Validador de Certificados (módulo 14), um problema real do seu trabalho: planilhas das linhas A a D, tolerâncias, e-mails da expedição, certificados. Aprender com um objetivo concreto dá contexto ("isso serve para aquilo") e motivação, e no fim vira portfólio.

13.3 Rotina sugerida: 1 hora por dia

Com 1 hora por dia, 5 dias por semana, o curso leva cerca de 31 semanas (7 meses). Constância vence intensidade: 5 horas espalhadas na semana ensinam mais que 5 horas num sábado.

Tempo	O que fazer
5 min	Resumo de memória: sem olhar nada, escreva 3 coisas da última aula. `git pull`.
20 min	Aula: uma ou duas seções, digitando os exemplos e fazendo os 🔮.
30 min	Exercícios: um ou dois, com o ciclo testar → corrigir → passar.
5 min	Diário de bordo (o que aprendeu, onde travou) e `git add` / `commit` / `push`.

Sexta-feira é dia de revisão espaçada: refaça sem consultar o quiz do módulo anterior e refaça do zero um exercício de 2 ou 3 módulos atrás. O calendário semana a semana, com os marcos de portfólio, está em `PLANO-DE-ESTUDOS.md` (na

pasta `curso-python`).

> 💡 **Dica:** o celular é ótimo para **reler** a aula e fazer o quiz no ônibus ou no intervalo. Mas exercício se faz no computador.

13.4 Quando travar

Siga esta ordem antes de abrir a solução:

1. Releia o enunciado inteiro, devagar. Muitas respostas estão nos exemplos dele.
2. Leia a mensagem do teste: o que era esperado e o que veio.
3. Teste um pedaço da sua ideia no REPL, com valores concretos.
4. Releia a seção da aula sobre o assunto e o exemplo resolvido.
5. Explique o problema em voz alta, linha por linha, como se fosse para um colega (ou para um pato de borracha: a técnica tem esse nome, "rubber duck debugging"). Muitas vezes a resposta aparece no meio da explicação.
6. Peça uma **dica** (não a resposta) para a IA ou para alguém.
7. Só então abra a solução. Entenda, apague, refaça sem olhar.

13.5 IA como professor, não como cola

Ferramentas de IA (ChatGPT, Claude, Copilot, Gemini) são excelentes professores particulares. E péssimas colas: se a IA escreve o código, quem aprende é ela.

Use assim:

- "Sou iniciante, estou no módulo 00 de um curso de Python. **Não me dê a resposta.** Meu teste falha com `assert None == 'Olá, mundo!'`. Me dê uma dica do que pode ser."
- "Explique este traceback linha por linha, como para um iniciante: (cole o erro)."
- "Faça 5 perguntas sobre ambiente virtual, sem as respostas. Depois corrija as minhas."
- "Meu código já passa nos testes. O que um programador experiente faria diferente? Explique o porquê."
- "Explique a diferença entre `print` e `return` com uma analogia de laboratório."

Não use assim:

- Colar o enunciado e pedir o código pronto.
- Aceitar código que você não sabe explicar linha por linha. Regra: **se você não consegue explicar, o código não é seu.**
- Colar dados da empresa (planilhas reais, nomes de clientes, NFs, laudos). Além da política interna, existe a LGPD. Use dados inventados.

E lembre: a IA erra com toda a confiança do mundo. Confira sempre rodando o código.

13.6 Como pesquisar

1. Copie a **última linha** do erro (sem os seus caminhos de pasta) e pesquise junto com a palavra `python`. Ex.: `python NameError: name 'pritn' is not defined`. Tente também em inglês: há muito mais conteúdo.
2. Prefira fontes confiáveis: a documentação oficial (`docs.python.org/pt-br/3/`), o Stack Overflow, a documentação da biblioteca.
3. Confira a data e a versão: resposta de 2012 pode ser de Python 2.

13.7 Como pedir ajuda (e ser respondido)

Quem pergunta bem recebe resposta rápida. Use este modelo, em comunidades ou com colegas:

```
O que estou tentando fazer: resolver o exercício 00.02 (saudação).
O que eu esperava: saudar("Ana") devolver "Olá, Ana!".
O que aconteceu: o teste diz  assert 'Olá, nome!' == 'Olá, Ana!'
Meu código:
    def saudar(nome):
        return "Olá, nome!"
O que já tentei: reli o enunciado e testei no REPL.
Ambiente: Windows 11, Python 3.14, VS Code.
```

Cole o erro **como texto**, não como foto da tela. E mostre o **menor** pedaço de código que reproduz o problema. Muitas vezes, ao montar a pergunta, você mesmo acha a resposta.

Onde perguntar: veja as comunidades em `carreira/recursos.md`.

Lendo mensagens de erro deste módulo

Como ler um traceback

Quando um programa quebra, o Python mostra um **traceback**: o rastro do que estava acontecendo. **Leia de baixo para cima.**

```
Traceback (most recent call last):
  File "C:\Users\carlos\projetos\rascunho\nome.py", line 2, in <module>
← ONDE: arquivo e linha
```

```
    pritn("Olá, mundo!")
← a linha culpada
    ^^^^^
← o pedaço exato
NameError: name 'pritn' is not defined. Did you mean: 'print'?
← O QUÊ: tipo e motivo
```

1. **Última linha:** o tipo do erro (`NameError`) e o motivo.
2. **Acima dela:** o arquivo, o número da linha e o trecho de código.
3. Corrija, salve, rode de novo.

As mensagens abaixo são reais (Python 3.13). Em outras versões os detalhes mudam um pouco, mas a estrutura é a mesma.

Erros de Python

`SyntaxError`: o Python nem conseguiu ler o código. Ele aponta onde se perdeu:

```
    print("Olá, mundo!)
          ^
SyntaxError: unterminated string literal (detected at line 1)
```

Faltou fechar as aspas.

Outras mensagens de `SyntaxError` que você vai ver:

Mensagem	Causa provável
`'(' was never closed`	faltou fechar um parêntese
`expected ':'`	faltaram os dois-pontos no fim da linha do `def` (ou `if`, `for`…)
`invalid syntax`	algo fora do lugar; olhe a linha apontada **e a de cima**

`IndentationError`: recuo errado.

```
    return "Olá, mundo!"
    ^^^^^^
IndentationError: expected an indented block after function definition on
line 1
```

O `return` precisava de 4 espaços de recuo para ficar "dentro" da função.

`NameError`: nome que o Python não conhece. Erro de digitação ou variável que não existe:

```
    print(peso_total)
          ^^^^^^^^^^
NameError: name 'peso_total' is not defined
```

`NotImplementedError: Implemente esta função`: você rodou o teste de um exercício que ainda está com a linha `raise NotImplementedError(...)`. Apague-a e escreva o código.

`AssertionError` no teste: seu código rodou, mas o resultado está diferente do esperado. Leia as linhas `-` (esperado) e `+` (seu resultado), como na seção 11.3.

`ModuleNotFoundError: No module named 'pytest'` (ou `pandas` etc.): a biblioteca não está instalada **neste** Python. Quase sempre é o `.venv` desativado. Ative e, se precisar, rode `python -m pip install -r requirements.txt`. (O `curso.py` mostra uma mensagem amigável quando falta o pytest.)

Erros do terminal

Mensagem (resumida)	Causa	Solução
`O termo 'python' não é reconhecido...`	Python não instalado ou fora do PATH	Seção 2.3
A Microsoft Store abre	Aliases de execução do Windows	Seção 2.3
`...a execução de scripts foi desabilitada neste sistema`	Política do PowerShell	Seção 9.4
`can't open file '...\curso.py': [Errno 2]`	Pasta errada	`cd` para `curso-python`
`fatal: not a git repository`	Fora da pasta do repositório	`cd` para dentro de `estudos`
`Author identity unknown`	Git sem nome/e-mail	Seção 4.2
`! [rejected] main -> main (fetch first)`	O GitHub tem commits que você não tem	`git pull`, depois `git push`

O `push` recusado é comum quando você estuda em dois computadores. A mensagem completa termina com uma dica do próprio Git: `use 'git pull' before pushing again`.

Boas práticas

- **Um ambiente virtual por projeto**, criado dentro da pasta do projeto, com o nome `.venv`.
- **Nunca coloque o `.venv` no Git.** Ele é recriado com `requirements.txt`.
- `python -m pip` em vez de só `pip`.
- **Projetos fora do OneDrive** e em pastas sem espaço nem acento.
- **Abra a pasta do projeto no VS Code**, não arquivos soltos.
- **Salve antes de testar** (ou ligue o salvamento automático).
- **Commits pequenos e frequentes**, com mensagens que completam "Este commit...".
- `git pull` **no começo,** `git push` **no fim** de cada sessão.
- **Leia a mensagem de erro inteira** antes de pesquisar ou perguntar.
- **Nada de dados reais da empresa** em repositórios ou ferramentas de IA.

No mundo real

A planilha que só funcionava no computador do estagiário.

Uma cena comum na indústria: alguém escreve um script em Python que lê a planilha de resultados do laboratório e gera um resumo para a reunião. Funciona bem por meses. Aí a pessoa sai da empresa, ou a TI troca o computador, ou alguém atualiza o pandas "para ver se melhora". O script para de funcionar e ninguém sabe por quê, nem qual versão de cada biblioteca ele usava.

Como um profissional evita isso: o script mora num repositório Git, com um `requirements.txt` listando as bibliotecas. Qualquer colega reproduz o ambiente com cinco comandos, os mesmos que você usou neste módulo:

```
git clone https://github.com/<empresa>/resumo-laboratorio.git
cd resumo-laboratorio
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

E se alguém mudar o código e quebrar algo, o histórico do Git mostra **o quê** mudou, **quando** e **quem** mudou, e permite voltar atrás. Os testes automáticos (como os do curso) avisam do problema antes de o resumo errado chegar na reunião.

Em entrevista de emprego, contar que você organiza seus scripts assim (repositório, ambiente virtual, requirements, testes) já te coloca à frente de muita gente que só "sabe Python".

Resumo

Tarefa	Comando (Windows / PowerShell)
Versão do Python	`python --version`
Abrir o REPL / sair	`python` / `exit()`
Rodar um script	`python arquivo.py`
Pasta atual / listar / entrar / voltar	`pwd` / `ls` / `cd pasta` / `cd ..`
Abrir a pasta no VS Code	`code .`
Clonar o repositório	`git clone https://github.com/carlos-real/estudos.git`
Criar o ambiente virtual	`python -m venv .venv`
Ativar o ambiente virtual	`.venv\Scripts\Activate.ps1`
Liberar scripts no PowerShell	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
Instalar as bibliotecas	`python -m pip install -r requirements.txt`
Conferir o ambiente	`python 00-preparacao/exemplos/04_verificar_ambiente.py`
Testar um exercício	`python curso.py testar 00 ex01`
Ver o progresso	`python curso.py progresso -d`
Ver o que mudou	`git status`
Registrar e enviar	`git add ...` → `git commit -m "..."` → `git push`
Trazer novidades	`git pull`

Linux/macOS: `python3`, `source .venv/bin/activate`, caminhos com `/`.

O molde de todo exercício:

```python
def nome_da_funcao(parametro: tipo) -> tipo_do_resultado:
    """O que a função faz."""
    return resultado  # return DEVOLVE; print só mostra
```

Você domina este módulo se...

- ☐ Consigo abrir o PowerShell, ir até `curso-python` com `cd` e explicar a diferença entre caminho absoluto e relativo.
- ☐ Sei explicar o que é o PATH e o que fazer quando aparece "o termo 'python' não é reconhecido".
- ☐ Consigo criar e ativar o `.venv` sozinho, e sei por que cada projeto deve ter o seu.
- ☐ Sei resolver o erro "a execução de scripts foi desabilitada" e explicar o que o `RemoteSigned` permite.
- ☐ Consigo explicar, sem consultar, a diferença entre `print` e `return`.
- ☐ Leio um traceback de baixo para cima e acho a linha e o tipo do erro.
- ☐ Faço o ciclo completo: testar → corrigir → passar → `git add` → `git commit` → `git push`.
- ☐ Sei pedir uma **dica** à IA sem pedir a resposta.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_ola_mundo.py` — Olá, mundo!	★☆☆	O ciclo testar → editar → passar; `return` de um texto
02	`ex02_saudacao.py` — Saudação personalizada	★☆☆	Usar um parâmetro; juntar textos com `+`
03	`ex03_peso_total.py` — Peso total da carga	★☆☆	Dois parâmetros; conta com `*`; testes com `approx`

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError`, salve e rode na pasta `curso-python`:

```
python curso.py testar 00 ex01
python curso.py testar 00 ex02
python curso.py testar 00 ex03
python curso.py testar 00          # os três de uma vez
```

São só três, e fáceis de propósito: aqui o objetivo é validar o ambiente e fixar o ciclo. A partir do módulo 01 vêm de 10 a 14 exercícios por módulo.

Desafio do módulo

Ambiente pronto e primeiro push. Você vai conferir o ambiente com o verificador, escrever um pequeno script de apresentação (`meu_cartao.py`) usando o molde `def` + `return`, e enviar tudo para o GitHub com um commit bem feito.

Enunciado completo, critérios de aceite e dicas: desafio/README.md.

Quiz

Teste o que ficou, **sem consultar**: quiz.md.

Para ir além

- Documentação oficial, em português:
  - Utilizando o interpretador Python
  - Ambientes virtuais e pacotes
  - Utilizando Python no Windows
- Configurar o ambiente de desenvolvimento Python no Windows (Microsoft Learn)
- about_Execution_Policies (o porquê do erro da seção 9.4, na documentação da Microsoft)
- Pro Git, capítulos 1 e 2 (livro oficial do Git, gratuito e em português)
- Python no VS Code (tutorial oficial, em inglês)
- Python Tutor: cole um código curto e veja, passo a passo, o que o Python faz em cada linha.

Enunciados dos exercícios

Exercício 00.01 — Olá, mundo!

Arquivo: `exercicios/ex01_ola_mundo.py`

```
Nível: ★☆☆

Contexto: Todo programador começa por aqui. O objetivo não é o texto em
si: é provar
que o seu ambiente funciona e que você entendeu o ciclo "editar → testar
→ passar".

Tarefa: implemente `ola_mundo()`, que DEVOLVE (com `return`) exatamente o
texto
```

```
"Olá, mundo!": O maiúsculo, acento no "á", vírgula, espaço e ponto de
exclamação.
Nada de espaço sobrando no começo nem no fim.

Exemplos:
    >>> ola_mundo()
    'Olá, mundo!'

Dica: use `return`, não `print`. O print só mostra o texto na tela; o
teste precisa
receber o texto de volta.

Teste com:  python curso.py testar 00 ex01
```

Exercício 00.02 — Saudação personalizada

Arquivo: `exercicios/ex02_saudacao.py`

```
Nível: ★☆☆

Contexto: O sistema do laboratório vai cumprimentar quem fizer login.
Para isso,
precisa de uma função que recebe um nome e monta a saudação.

Tarefa: implemente `saudar(nome)`, que DEVOLVE o texto "Olá, <nome>!".
- Use o nome exatamente como recebeu (não mude maiúsculas nem
minúsculas).
- Formato fixo: "Olá, " + nome + "!" (vírgula e espaço depois de "Olá").

Exemplos:
    >>> saudar("Carlos")
    'Olá, Carlos!'
    >>> saudar("Maria Clara")
    'Olá, Maria Clara!'

Dica: o sinal `+` junta textos: "Bom " + "dia" vira "Bom dia". Escreva o
nome do
parâmetro (`nome`), sem aspas, para usar o valor que a função recebeu.

Teste com:  python curso.py testar 00 ex02
```

Exercício 00.03 — Peso total da carga

Arquivo: `exercicios/ex03_peso_total.py`

```
Nível: ★☆☆
```

```
Contexto: A expedição vai carregar várias bobinas do mesmo filme, todas
com o mesmo
peso. A nota fiscal (NF) precisa do peso total da carga, em kg.

Tarefa: implemente `peso_total(peso_bobina_kg, quantidade)`, que DEVOLVE
o peso
total: o peso de uma bobina multiplicado pela quantidade de bobinas.
- Quantidade 0 dá peso total 0.
- Não arredonde: devolva o resultado da conta.

Exemplos:
    >>> peso_total(250.0, 4)
    1000.0
    >>> peso_total(312.5, 3)
    937.5
    >>> peso_total(180.0, 0)
    0.0

Dica: em Python, multiplicação é `*`. Experimente `312.5 * 3` no REPL
antes.

Teste com:  python curso.py testar 00 ex03
```

Desafio 00 — Ambiente pronto e primeiro push

Contexto

Na indústria, um script só tem valor se outra pessoa (ou você, em outro computador) consegue rodar de novo. Neste desafio você prova que o seu ambiente está completo, escreve um pequeno programa no molde do curso e publica tudo no GitHub, do jeito que um profissional faz.

É também o primeiro commit do seu portfólio.

Requisitos obrigatórios

1. **Ambiente completo.** Python 3.12+, VS Code com a extensão Python, Git configurado com seu nome e e-mail, repositório `estudos` clonado, `.venv` criado em `curso-python` e bibliotecas instaladas com `python -m pip install -r requirements.txt`.
2. **Verificador sem ✘.** Na pasta `curso-python`, com o `.venv` ativo: `python 00-preparacao/exemplos/04_verificar_ambiente.py` mostra todas as linhas com ✔ (um ⚠ na versão do Python é aceitável se você estiver na 3.10 ou 3.11).
3. **Exercícios do módulo resolvidos.** `python curso.py testar 00` termina com `19 passed` e a mensagem `✔ Tudo certo!`.

4. **Seu cartão de apresentação.** Crie o arquivo `00-preparacao/desafio/meu_cartao.py` que, ao rodar, mostra pelo menos:
   - seu nome;
   - sua função atual;
   - o seu objetivo com o curso;
   - a versão do Python em uso.
5. **Pelo menos uma função no molde `def` + `return`** dentro do `meu_cartao.py`, usada para montar alguma linha do cartão (ex.: `montar_linha(rotulo, valor)`).
6. **Commit e push.** Os exercícios resolvidos e o `meu_cartao.py` estão no GitHub, num ou mais commits com mensagens claras (nada de "teste" ou "ajustes").

Extras (opcionais)

- Mostrar no cartão se o ambiente virtual está ativo (veja a dica 4).
- Ligar o salvamento automático do VS Code e instalar a extensão **Ruff** (você vai usá-la no módulo 09).
- Criar o arquivo `DIARIO.md` na raiz do repositório (veja o `PLANO-DE-ESTUDOS.md`) com a primeira entrada do seu diário de bordo.
- Rodar o mesmo repositório em um segundo computador (clone, `.venv`, `pip install`) e conferir que tudo funciona igual.

Exemplo de execução

```
python 00-preparacao/desafio/meu_cartao.py
```

```
--------------------------------------------------
CARTÃO DE APRESENTAÇÃO
--------------------------------------------------
Nome: Carlos Eduardo
Função atual: Laboratório de Qualidade (embalagens plásticas flexíveis)
Objetivo: Trabalhar com dados e automação usando Python
Python: 3.14.0
Ambiente virtual ativo: sim
--------------------------------------------------
```

O seu não precisa ficar igual: o formato é livre. A versão do Python será a sua.

Critérios de aceite

- [ ] O verificador de ambiente mostra todas as linhas com ✔.
- [ ] `python curso.py testar 00` passa nos 19 testes.
- [ ] `python curso.py progresso` mostra `✔ 00 Preparação do ambiente ... 3/3`.

- [ ] `python 00-preparacao/desafio/meu_cartao.py` roda sem erro e mostra as 4 informações pedidas.
- [ ] O `meu_cartao.py` tem pelo menos uma função com `return` que é chamada no script.
- [ ] `git status` mostra "nothing to commit, working tree clean" (tudo commitado).
- [ ] No site `https://github.com/carlos-real/estudos` aparecem o `meu_cartao.py` e os exercícios resolvidos.
- [ ] Nenhum dado real da empresa foi para o GitHub.

Dicas de como começar

1. **Faça na ordem dos requisitos.** Não adianta escrever o cartão se o `.venv` não ativa. Siga as seções 2 a 9 da aula e rode o verificador.
2. **Comece pelo mínimo que roda.** Crie o `meu_cartao.py` com um único `print("Nome: Carlos Eduardo")`, rode, e só depois vá acrescentando linhas.
3. **A versão do Python** se pega assim (copie estas duas linhas; `import` é assunto do módulo 07):

   ```
   import platform

   print("Python: " + platform.python_version())
   ```

4. **Ambiente virtual ativo?** O verificador usa a comparação `sys.prefix != sys.base_prefix` (precisa de `import sys`). Se quiser mostrar isso no cartão, `print(sys.prefix != sys.base_prefix)` mostra `True` ou `False`.
5. **Commit em partes.** Um commit para os exercícios ("Resolve exercícios do módulo 00") e outro para o cartão ("Adiciona cartão de apresentação do desafio 00") deixa o histórico mais fácil de ler.
6. **Antes do `git add .`, rode `git status`** e confira se não está indo nada que não devia.

Só depois de terminar, compare com `solucao_referencia.py`. Ela mostra uma forma de organizar o cartão em funções pequenas.

Quiz — Módulo 00: Preparação do ambiente

Responda **sem consultar** a aula nem o computador. Anote suas respostas num papel e só depois confira o gabarito no final. Errou? Volte à seção indicada no gabarito.

---

1. O que o interpretador Python faz com o seu arquivo `.py`?

- a) Transforma o arquivo num `.exe` e apaga o original.
- b) Lê o código e executa as instruções, de cima para baixo.
- c) Corrige automaticamente os erros de digitação.
- d) Envia o código para o GitHub.

2. Você instalou o Python, digitou `python --version` e apareceu `O termo 'python' não é reconhecido como nome de cmdlet...`. Cite as duas causas mais prováveis.

3. Para que serve um ambiente virtual (`.venv`)?

- a) Para o Python rodar mais rápido.
- b) Para cada projeto ter o seu próprio Python e as suas bibliotecas, sem interferir nos outros.
- c) Para guardar uma cópia do código na nuvem.
- d) Para esconder o código de outras pessoas.

4. Qual comando **ativa** o ambiente virtual no PowerShell?

- a) `python -m venv .venv`
- b) `.venv\Scripts\Activate.ps1`
- c) `pip install venv`
- d) `cd .venv`

5. Ao ativar o `.venv`, o PowerShell diz que "a execução de scripts foi desabilitada neste sistema". Qual comando resolve no seu computador pessoal, sem precisar de administrador? E o que esse comando passa a permitir?

6. Por que o curso recomenda `python -m pip install ...` em vez de só `pip install ...`?

7. O que este código imprime?

```python
def dobro(n):
    return n * 2


print(dobro(5))
print(dobro(1.5))
```

8. O que este código imprime?

```python
def ola():
    print("Oi")
```

```
x = ola()
print(x)
```

9. Onde está o erro? Qual mensagem o Python mostra?

```
def saudar(nome)
    return "Olá, " + nome + "!"
```

10. Este código roda sem erro, mas o teste do exercício 00.02 falha. Por quê?

```
def saudar(nome):
    return "Olá, nome!"


print(saudar("Ana"))
```

11. Leia o traceback. Em que linha está o problema, qual o tipo do erro e como corrigir?

```
Traceback (most recent call last):
  File "C:\Users\carlos\projetos\rascunho\relatorio.py", line 4, in
<module>
    print(peso_totl)
          ^^^^^^^^^
NameError: name 'peso_totl' is not defined. Did you mean: 'peso_total'?
```

12. Qual a ordem certa para enviar um exercício resolvido ao GitHub?

- a) `git push` → `git commit` → `git add`
- b) `git add` → `git commit` → `git push`
- c) `git commit` → `git add` → `git push`
- d) `git pull` → `git push` → `git add`

13. O `git push` foi recusado com `! [rejected] main -> main (fetch first)`. O que aconteceu e o que fazer?

14. Qual destes é o melhor uso de uma IA enquanto você faz os exercícios?

- a) Colar o enunciado e pedir o código pronto, para ganhar tempo.
- b) Colar a planilha real do laboratório para ela gerar os dados de teste.
- c) Mostrar a mensagem de erro do teste e pedir uma dica, sem a resposta.
- d) Pedir para ela resolver e depois só ler a resposta com atenção.

15. Você editou o exercício, rodou `python curso.py testar 00 ex01` e o resultado não mudou nada, como se você não tivesse mexido. Qual a causa mais comum?

Gabarito

1. b. O interpretador (o programa `python`) lê o código-fonte e executa as instruções na ordem. Ele não corrige nada: quando encontra algo que não entende, para e mostra um traceback. (Seção 1.)

2. (1) O Python não foi instalado, ou (2) foi instalado sem marcar "Add python.exe to PATH". Uma terceira possibilidade comum: o terminal ficou aberto durante a instalação e não enxerga o PATH novo. Feche e abra de novo. (Seção 2.3.)

3. b. Cada projeto tem as suas bibliotecas, nas versões de que precisa. Atualizar o pandas de um projeto não quebra outro. Guardar na nuvem é papel do Git/GitHub. (Seção 9.1.)

4. b. A alternativa a **cria** o ambiente (uma vez só); a b **ativa** (em todo terminal novo). Funcionou se aparecer `(.venv)` no começo do prompt. (Seção 9.3.)

5. `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`. Vale só para o seu usuário. Passa a permitir scripts criados no seu computador (como o `Activate.ps1`) e continua exigindo assinatura digital de scripts baixados da internet. Em computador da empresa a TI pode impor outra política: não tente burlar. (Seção 9.4.)

6. Porque `python -m pip` usa o pip **do mesmo Python** que vai rodar o código. Com o `.venv` ativo, as bibliotecas vão para o `.venv`, e não para outro Python qualquer do computador. (Seção 9.5.)

7.

```
10
3.0
```

`dobro(1.5)` dá `3.0`, e não `3`: número com ponto decimal continua com ponto decimal depois da conta. (Seção 10.3.)

8.

```
Oi
None
```

A função **mostra** "Oi" com `print`, mas não tem `return`, então devolve `None`. O `x` guarda `None`, e o último `print` mostra isso. (Seção 10.3.)

9. Faltam os dois-pontos no fim da linha do `def`. O Python mostra `SyntaxError: expected ':'` apontando o fim da linha 1. (Seção "Lendo mensagens de erro".)

10. Dentro das aspas, `nome` é texto fixo, não o parâmetro. A função devolve sempre `'Olá, nome!'` (o `print` mostra `Olá, nome!`). O certo é `"Olá, " + nome + "!"`, com `nome` **fora** das aspas. (Seção 10.5 e exercício 00.02.)

11. Linha 4. É um `NameError`: o nome `peso_totl` não existe. É erro de digitação, e o próprio Python sugere `peso_total`. Corrija para `print(peso_total)`. Leia sempre de baixo para cima: a última linha diz o quê, as de cima dizem onde. (Seção "Lendo mensagens de erro".)

12. b. `add` coloca na área de preparação, `commit` registra no histórico local, `push` envia para o GitHub. (Seção 12.)

13. O GitHub tem commits que o seu computador ainda não tem (por exemplo, você enviou algo de outro computador). Rode `git pull` para trazer as novidades e depois `git push` de novo. (Seção "Lendo mensagens de erro".)

14. c. Pedir uma dica a partir da mensagem de erro mantém o raciocínio com você. A alternativa a e a d entregam a solução (você não aprende a resolver). A b expõe dados da empresa: nunca cole dados reais em ferramentas de IA. (Seção 13.5.)

15. O arquivo não foi salvo. O teste lê o arquivo do disco, não o que está na tela do VS Code. Use `Ctrl+S` ou ligue o salvamento automático. Outra causa possível: você editou o arquivo de `solucoes/` em vez do de `exercicios/`. (Seções 3.3 e 11.)

Módulo 01 — Fundamentos

> **Tempo estimado:** 8 a 10 horas · **Pré-requisitos:** módulo 00 (Python, VS Code e terminal funcionando) · **Nível:** iniciante

O que você vai aprender

Ao final deste módulo, você vai conseguir:

- mostrar informações na tela com `print()`, controlando o separador (`sep`) e o final da linha (`end`);
- criar variáveis com nomes claros e explicar o que acontece quando uma variável recebe outro valor;
- dizer o tipo de qualquer valor (`int`, `float`, `str`, `bool`, `None`) e conferir com `type()`;
- calcular com `+ - * / // % **` e prever o resultado de uma expressão sem parênteses;
- atualizar totais e contadores com `+=`, `-=`, `*=` e `/=`;
- limpar e examinar textos com `len`, `upper`, `lower`, `strip`, `replace` e `in`;
- converter texto em número (e o contrário) com `int()`, `float()` e `str()`, sabendo quando isso dá erro;
- ler o que o usuário digita com `input()`;
- montar textos com f-strings e formatar números com `:.2f`, `:,.2f`, `:.1%`, `:03d` e alinhamento;
- explicar por que `0.1 + 0.2` não dá `0.3` e decidir quando e como arredondar;
- usar `math.sqrt`, `math.pi`, `math.ceil` e `math.floor`;
- escrever uma função com `def` e `return`, o molde de todos os exercícios do curso;
- ler e corrigir os erros mais comuns: `NameError`, `TypeError`, `ValueError`, `SyntaxError`, `IndentationError` e `ZeroDivisionError`.

Por que isso importa

Todo programa, do script de 10 linhas ao sistema de uma multinacional, é feito das mesmas peças: valores, nomes para esses valores, contas e textos. Este módulo é o alfabeto. Sem ele, nada do resto faz sentido. Com ele firme, todo o resto fica mais fácil.

Pense no projeto final, o validador de certificados. Ele vai precisar:

- ler `"25,4"` de uma célula da planilha e transformar em número → **conversões** (seção 9);
- calcular os limites de "25 µm ± 10%" → **operadores** (seção 6);
- transformar `" op-12345 "` em `"OP12345"` para cruzar com o e-mail da expedição → **strings** (seção 8);
- escrever "Espessura média: 25.07 µm" no relatório → **f-strings** (seção 11);
- não se enganar com a 17ª casa decimal ao somar pesos → **float e round** (seção 12).

No mercado é igual. Os primeiros testes de código em processos seletivos para estágio e júnior cobram exatamente isto: manipular texto, fazer contas e formatar a saída sem tropeçar.

> 💡 **Dica:** estude com o VS Code aberto. Apareceu código? Digite e rode. Ler código sem rodar é como ler sobre natação.

Como estudar este módulo (método PRIMM)

Você vai ver quatro tipos de caixa ao longo da aula:

- 🔮 **Preveja antes de rodar:** leia o código e **escreva** (no papel ou num comentário) o que acha que vai sair. Só depois rode e compare. Errar a previsão é ótimo: mostra exatamente o que você ainda não entendeu.
- 🔧 **Modifique:** mude um detalhe do exemplo e veja o efeito. É assim que o conhecimento "gruda".
- **Exemplo resolvido:** um problema resolvido passo a passo, com o raciocínio. Leia antes de tentar os parecidos.
- **Tente você:** um mini exercício com a resposta escondida. Tente antes de abrir.

1. Dois jeitos de rodar Python: REPL e script

No módulo 00 você conheceu os dois. Recapitulando, porque esta aula usa os dois o tempo todo.

REPL é o modo conversa. No PowerShell, digite `python` (no Linux/macOS, `python3`). Aparece o sinal `>>>`. Você digita uma linha, aperta Enter e o Python responde na hora:

```
>>> 2 + 3
5
```

```
>>> "laudo".upper()
'LAUDO'
```

Repare: no REPL você não precisa de `print` para ver o resultado. Ele mostra o valor de cada expressão (textos aparecem entre aspas). Para sair, digite `exit()`.

Script é um arquivo `.py` com várias linhas, executado de uma vez:

```
PS C:\curso-python> python 01-fundamentos\exemplos\01_print.py
```

No script, só aparece na tela o que você mandar imprimir com `print`.

Nesta aula, blocos que começam com `>>>` são sessões de REPL. Os outros blocos são scripts, e a saída vem logo abaixo, marcada como **Saída**.

> 💡 **Dica:** o REPL é o seu laboratório de bancada. Ficou em dúvida sobre o que uma linha faz? Não fique adivinhando: abra o REPL e teste. Leva 5 segundos.

2. print: o programa falando com você

`print()` mostra valores na tela. Pense nele como o visor de um equipamento: é por onde o programa conta o que está acontecendo.

```
print("Olá, laboratório!")
print("OP", 12345, "liberada")
print("Espessura:", 25.4, "µm")
```

Saída:

```
Olá, laboratório!
OP 12345 liberada
Espessura: 25.4 µm
```

Você pode passar vários valores separados por vírgula. O `print` coloca um espaço entre eles e quebra a linha no final.

Dois controles úteis:

- `sep=` troca o separador entre os valores;
- `end=` troca o que vai no fim (o padrão é uma quebra de linha).

```
print("A", "B", "C", "D", sep=" | ")
print("Medindo...", end=" ")
```

```
print("pronto!")
```

Saída:

```
A | B | C | D
Medindo... pronto!
```

Dentro de um texto, alguns "códigos de barra invertida" têm significado especial: `\n` quebra a linha e `\t` insere uma tabulação.

```
print("Linha 1\nLinha 2")
print("Amostra\tResultado")
```

Saída:

```
Linha 1
Linha 2
Amostra Resultado
```

Textos podem usar aspas simples ou duplas. Use uma para "proteger" a outra:

```
print('Ela disse: "COF liberado"')
print("Copo d'água")
```

Saída:

```
Ela disse: "COF liberado"
Copo d'água
```

⚠️ **Erro comum:** escrever `Print` com P maiúsculo. Python diferencia maiúsculas de minúsculas: `Print` não existe e você recebe `NameError: name 'Print' is not defined. Did you mean: 'print'?`. Leia a sugestão no fim da mensagem: ela costuma acertar.

🔮 **Preveja antes de rodar:** o que este código imprime? Escreva antes de testar.

```
print("Lote", 7, sep="-", end="!")
print("OK")
```

▼ **Resposta**

Sai `Lote-7!OK` numa linha só. O `sep="-"` colocou um hífen entre `"Lote"` e `7`, e o `end="!"` trocou a quebra de linha por um `!`. Como o primeiro `print` não quebrou a linha, o `OK` do segundo ficou colado.

O arquivo `exemplos/01_print.py` tem todos esses casos. Rode:

```
python 01-fundamentos\exemplos\01_print.py
```

3. Comentários: notas para humanos

Tudo depois de `#` numa linha é ignorado pelo Python. Comentários são para quem lê o código, inclusive você daqui a três meses.

```
# Converte a espessura de micrômetros para milímetros
espessura_mm = 25.4 / 1000  # 1 mm = 1000 µm
print(espessura_mm)
```

Saída:

```
0.0254
```

Bons comentários explicam o **porquê**, não o óbvio:

```
# Ruim: repete o que o código já diz
total = total + 1  # soma 1 no total

# Bom: explica uma decisão
tolerancia = 0.08  # 8%: linha C é coextrusão, a espec. do cliente
permite mais
```

Um texto entre três aspas logo no começo de um arquivo ou de uma função é uma **docstring**: a documentação oficial daquele trecho. Todos os exercícios do curso começam com uma docstring, que é o enunciado.

> 💡 **Dica:** no VS Code, `Ctrl+;` (ou `Ctrl+/`, conforme o teclado) comenta e descomenta as linhas selecionadas. Ótimo para "desligar" um trecho enquanto testa.

4. Variáveis e nomes

Uma variável é um **nome que aponta para um valor**. Pense na etiqueta que você cola numa amostra: a etiqueta "OP12345 - bobina 3" não é a amostra, mas permite encontrá-la depois.

```
op = "OP12345"
quantidade_bobinas = 8
espessura_um = 25.4
print(op, quantidade_bobinas, espessura_um)
```

Saída:

```
OP12345 8 25.4
```

O `=` em Python **não** é o "igual" da matemática. Ele é uma ordem: "calcule o que está à direita e cole esta etiqueta no resultado". Por isso, isto é válido:

```
>>> bobinas = 8
>>> bobinas = bobinas + 2
>>> bobinas
10
```

Na matemática, `x = x + 2` é absurdo. Em Python é rotina: pegue o valor atual (8), some 2 e cole a etiqueta `bobinas` no novo valor (10).

Regras e convenções de nomes

Regras (se quebrar, dá erro):

- só letras, números e `_`;
- não pode começar com número (`1amostra` é inválido);
- não pode ser uma palavra reservada (`if`, `for`, `def`, `class`, `True`, `None`...);
- maiúsculas e minúsculas são diferentes: `peso` e `Peso` são duas variáveis.

Convenções (o Python aceita quebrar, mas os outros programadores não):

- **snake_case**: palavras minúsculas separadas por `_` → `peso_liquido_kg`;
- **sem acento**: `espessura_media`, não `espessura_média` (funciona, mas dá problema com teclados, ferramentas e colegas);
- **nomes que dizem o que é**: `tolerancia_pct` em vez de `t`; `peso_kg` em vez de `p`;

- **unidade no nome** quando houver risco de confusão: `espessura_um`, `largura_mm`, `peso_kg`;
- **MAIÚSCULAS** para constantes, valores que não devem mudar: `TOLERANCIA_PADRAO_PCT = 10`.

> ⚠️ **Erro comum:** usar uma variável antes de criá-la, ou digitar o nome errado. `print(espesura)` (faltou um "s") dá `NameError: name 'espesura' is not defined. Did you mean: 'espessura'?`.

> 💡 **Dica:** nome bom economiza comentário. Compare `x = a * b / 1000` com `peso_kg = gramatura * area_m2 / 1000`. O segundo dispensa explicação.

Tente você: quais destes nomes são inválidos? `peso2`, `2peso`, `peso-liquido`, `peso_liquido`, `class`, `Classe`.

<details>
<summary>Resposta</summary>

Inválidos: `2peso` (começa com número), `peso-liquido` (o hífen seria lido como subtração) e `class` (palavra reservada). `Classe` é válido, porque maiúsculas contam como letra diferente, mas foge da convenção snake_case.
</details>

5. Tipos de dados: int, float, str, bool e None

Todo valor tem um **tipo**. O tipo diz o que dá para fazer com o valor, como a unidade de medida diz o que dá para somar: você soma µm com µm, não µm com kg.

Tipo	Para quê	Exemplos
`int`	números inteiros	`8`, `-3`, `12345`
`float`	números com casas decimais	`25.4`, `0.1`, `-2.5`, `1e3`
`str`	texto (string)	`"OP12345"`, `'linha B'`, `""`
`bool`	verdadeiro ou falso	`True`, `False`
`None`	"nada", ausência de valor	`None`

A função `type()` diz o tipo de qualquer valor:

```
>>> type(8)
<class 'int'>
>>> type(25.4)
```

```
<class 'float'>
>>> type("25.4")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type(None)
<class 'NoneType'>
```

Repare em `"25.4"`: entre aspas, é **texto**, mesmo parecendo número. Essa diferença vai aparecer o tempo todo quando você ler planilhas e e-mails.

Detalhes que valem ouro:

- `True` e `False` começam com maiúscula. `true` dá `NameError`.
- `None` é o "campo vazio": a célula da planilha que ainda não foi preenchida, a observação que não existe. Não é zero nem texto vazio; é "não tem valor".
- O Python usa **ponto** como separador decimal: `25.4`, nunca `25,4`. A vírgula separa valores. `1e3` é notação científica: $1 \times 10^3 = 1000.0$.

```
>>> 25,4
(25, 4)
```

Viu? Com vírgula, o Python entende **dois números** (25 e 4) agrupados. Isso é uma tupla, que você verá no módulo 03. Por enquanto: decimal é com ponto.

O exemplo `exemplos/02_variaveis_e_tipos.py` cria uma variável de cada tipo e mostra o `type()` de cada uma.

> 🔮 **Preveja antes de rodar:** qual o tipo de cada valor? `10`, `10.0`, `"10"`, `10 / 2`, `"True"`.

<details>
<summary>Resposta</summary>

`int`, `float`, `str`, `float` e `str`. Duas pegadinhas: `10 / 2` dá `5.0` (a divisão `/` **sempre** devolve float, mesmo quando a conta é exata) e `"True"` entre aspas é texto, não booleano. Confira no REPL com `type(10 / 2)`.

</details>

6. Operadores aritméticos e precedência

Operador	Operação	Exemplo	Resultado
`+`	soma	`7 + 2`	`9`

Operador	Operação	Exemplo	Resultado
`-`	subtração	`7 - 2`	`5`
`*`	multiplicação	`7 * 2`	`14`
`/`	divisão	`7 / 2`	`3.5`
`//`	divisão inteira	`7 // 2`	`3`
`%`	resto da divisão	`7 % 2`	`1`
`**`	potência	`7 ** 2`	`49`

Os dois "estranhos" são `//` e `%`. Pense em embalar: `//` responde "quantas caixas completas?" e `%` responde "quanto sobra fora das caixas?".

Exemplo resolvido: dividir uma produção em bobinas

Problema: a extrusora produziu 12.300 m de filme. Cada bobina leva 1.000 m. Quantas bobinas completas saem, e quanto filme sobra?

Raciocínio: "quantas vezes 1.000 cabe inteiro em 12.300" é divisão inteira (`//`). "O que sobra" é o resto (`%`). Vou guardar cada informação numa variável com nome claro e imprimir no fim.

```
metros_totais = 12300
metros_por_bobina = 1000
bobinas = metros_totais // metros_por_bobina
sobra = metros_totais % metros_por_bobina
print(bobinas, "bobinas completas e sobra de", sobra, "m")
```

Saída:

```
12 bobinas completas e sobra de 300 m
```

Conferindo: 12 × 1.000 + 300 = 12.300. Sempre que usar `//` e `%`, essa prova real vale: `quociente * divisor + resto == dividendo`.

> 🔧 **Modifique:** troque `metros_por_bobina` para `1500`. Antes de rodar, calcule de cabeça. Depois tente `12300 / 1000` (com uma barra só) e veja a diferença.

Outro uso clássico do `%` é saber se um número é par: `n % 2` dá `0` para pares e `1` para ímpares.

Precedência: quem é calculado primeiro

Como na matemática da escola, o Python não faz as contas simplesmente da esquerda para a direita. Ele segue uma ordem:

```
1º  ( )             parênteses
2º  **              potência
3º  * / // %        multiplicação, divisões e resto (da esquerda para a
direita)
4º  + -             soma e subtração (da esquerda para a direita)
```

```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
>>> 2 ** 3 * 2
16
>>> 10 - 4 - 3
3
```

🔮 **Preveja antes de rodar:** um aluno tirou 7, 8 e 9. O que este código imprime? É a média dele?

```
print(7 + 8 + 9 / 3)
```

▼ **Resposta**

Imprime `18.0`, e não é a média. A divisão vem antes da soma, então o Python calculou `9 / 3 = 3.0` e depois `7 + 8 + 3.0 = 18.0`. A média correta é `(7 + 8 + 9) / 3`, que dá `8.0`. Esse erro aparece em código profissional também; o exercício 03 cobra exatamente isso.

💡 **Dica:** na dúvida, use parênteses. Eles nunca atrapalham e deixam a intenção clara para quem lê: `(nominal * tolerancia_pct) / 100`.

⚠️ **Erro comum:** dividir por zero. `100 / 0` (ou uma variável que vale 0) dá `ZeroDivisionError: division by zero`. O Python não devolve "infinito": ele para o programa. No módulo 02 você vai aprender a verificar antes de dividir.

Uma curiosidade com números negativos: `//` arredonda **para baixo** (em direção ao menos infinito), não "corta" as casas.

```
>>> 7 // 2
3
>>> -7 // 2
-4
```

Tente você: quantos minutos e segundos são 200 segundos? Use `//` e `%`.

<details>
<summary>Resposta</summary>

```
>>> 200 // 60
3
>>> 200 % 60
20
```

3 minutos e 20 segundos. Prova real: 3 × 60 + 20 = 200.
</details>

7. Atribuição composta: atualizando valores

Somar algo ao valor atual de uma variável é tão comum que existe um atalho:

```
total_kg = 0
total_kg += 125.5  # o mesmo que: total_kg = total_kg + 125.5
total_kg += 130.2
total_kg -= 5.7
print("Peso acumulado:", total_kg)
```

Saída:

```
Peso acumulado: 250.0
```

Atalho	Equivale a
`x += 5`	`x = x + 5`
`x -= 5`	`x = x - 5`
`x *= 2`	`x = x * 2`
`x /= 2`	`x = x / 2`

Isso vai ser essencial no módulo 02, quando você somar centenas de medições dentro de um laço: `soma += medicao`.

> ⚠️ **Erro comum:** usar `+=` numa variável que ainda não existe. `contador += 1` sem antes ter `contador = 0` dá `NameError`. O Python precisa de um valor inicial para somar.

8. Strings básicas: trabalhando com texto

Uma string (`str`) é uma sequência de caracteres. Em sistemas de laboratório, texto está em todo lugar: OP, cliente, observação, nome do ensaio, linha do e-mail.

Juntar, repetir e medir

```
>>> "Linha " + "B"
'Linha B'
>>> "-" * 20
'--------------------'
>>> len("filme bopp")
10
```

- `+` entre textos **concatena** (cola um no outro);
- `*` entre texto e inteiro **repete**;
- `len()` conta os caracteres, incluindo espaços.

Métodos: ferramentas que o texto já traz

Um **método** é uma função que pertence ao valor. Você chama com um ponto: `texto.metodo()`.

```
>>> "filme bopp".upper()
'FILME BOPP'
>>> "COF LIBERADO".lower()
'cof liberado'
>>> "   Embalagens Sul  ".strip()
'Embalagens Sul'
>>> "25,4".replace(",", ".")
'25.4'
```

Método	O que faz
`upper()`	tudo em maiúsculas (inclusive acentos: `"seleção"` → `"SELEÇÃO"`)

Método	O que faz
`lower()`	tudo em minúsculas
`strip()`	remove espaços (e quebras de linha) do começo e do fim
`replace(velho, novo)`	troca **todas** as ocorrências de `velho` por `novo`

Procurar com `in`

O operador `in` pergunta "este pedaço está dentro daquele texto?" e responde com um `bool`:

```
>>> "COF" in "COF: liberado"
True
>>> "cof" in "COF: liberado"
False
>>> "cof" in "COF: liberado".lower()
True
```

A busca diferencia maiúsculas. O truque da última linha (passar tudo para minúsculas antes) é o jeito padrão de ignorar essa diferença. É exatamente o exercício 05.

Strings não mudam

Strings são **imutáveis**: nenhum método altera o texto original. Eles devolvem um texto **novo**.

🔮 **Preveja antes de rodar:** o que é impresso?

```
nome = "laudo"
nome.upper()
print(nome)
```

▼ **Resposta**

Imprime `laudo`, em minúsculas. O `nome.upper()` criou o texto `"LAUDO"`, mas ninguém guardou esse resultado, e ele se perdeu. Para "mudar", guarde o resultado: `nome = nome.upper()`. No REPL você veria `'LAUDO'` aparecer ao rodar a segunda linha, mas a variável continuaria igual.

Exemplo resolvido: limpar o código de uma OP

Problema: a OP chegou digitada como `" op-12345 "`. O sistema precisa de `"OP12345"`.

Raciocínio: quebro em etapas pequenas, testando cada uma no REPL.

1. Tirar os espaços das pontas → `strip()`.
2. Deixar tudo maiúsculo → `upper()`.
3. Remover o hífen → `replace("-", "")` (trocar por "nada").

Como cada método devolve uma string nova, posso **encadear**: o resultado de um vira a entrada do próximo.

```
>>> " op-12345 ".strip()
'op-12345'
>>> " op-12345 ".strip().upper()
'OP-12345'
>>> " op-12345 ".strip().upper().replace("-", "")
'OP12345'
```

Repare no método de trabalho: construí a expressão aos poucos, conferindo cada passo. Isso é muito mais rápido do que escrever tudo de uma vez e caçar o erro depois.

> 🔧 **Modifique:** e se a OP vier como `"OP 12.345"`? Que `replace` você acrescentaria? Teste no REPL. (O exercício 08 generaliza isso.)

> ⚠️ **Erro comum:** somar texto com número. `"Bobinas: " + 8` dá `TypeError: can only concatenate str (not "int") to str`. O Python se recusa a adivinhar se você queria somar números ou juntar textos. Converta com `str(8)` ou, melhor, use uma f-string (seção 11).

Tente você: a observação veio como `"  cof: LIBERADO  "`. Escreva uma linha que deixe `"COF: LIBERADO"`.

<details>
<summary>Resposta</summary>

```
>>> "  cof: LIBERADO  ".strip().upper()
'COF: LIBERADO'
```

</details>

Rode `exemplos/04_strings.py` para ver tudo isso junto.

9. Conversões: int(), float() e str()

Aqui está o motivo número 1 de bugs em scripts de iniciante: **o que vem de fora do programa chega como texto**. O que a pessoa digita, o que vem de um arquivo, de um e-mail, de um formulário. E texto não faz conta.

> 🔮 **Preveja antes de rodar:**
>
> ```
> print("25" + "5")
> print(25 + 5)
> ```

<details>
<summary>▼ Resposta</summary>

```
255
30
```

Com aspas, são textos, e `+` cola um no outro. Sem aspas, são números, e `+` soma. Se um programa lê dois pesos de uma planilha e "soma" `"125"` com `"130"`, o resultado é `"125130"`, e nenhum erro avisa. Por isso, converta na entrada.

</details>

As funções de conversão têm o nome do tipo de destino:

```
>>> int("42")
42
>>> float("25.4")
25.4
>>> str(12345)
'12345'
>>> int(25.9)
25
>>> float(7)
7.0
>>> int("  42  ")
42
```

Detalhes importantes:

- `int(25.9)` **corta** as casas decimais (não arredonda): dá `25`, não `26`.

- `int()` e `float()` ignoram espaços nas pontas.
- `str()` transforma qualquer valor em texto.

Quando a conversão falha: ValueError

Se o texto não representa um número válido, o Python levanta `ValueError`:

```
>>> int("abc")
Traceback (most recent call last):
  ...
ValueError: invalid literal for int() with base 10: 'abc'
>>> int("25.4")
Traceback (most recent call last):
  ...
ValueError: invalid literal for int() with base 10: '25.4'
>>> float("25,4")
Traceback (most recent call last):
  ...
ValueError: could not convert string to float: '25,4'
```

(Nesta aula, as linhas do meio dos tracebacks foram resumidas com `...`. A seção "Lendo mensagens de erro" mostra um completo.)

Leia a última linha de cada erro: ela diz o tipo do erro e mostra, entre aspas, **o texto exato** que causou o problema. Dois casos que pegam muita gente:

- `int("25.4")` falha: `int()` não aceita texto com casas decimais. Use `float("25.4")` (e, se precisar de inteiro, `int(float("25.4"))`).
- `float("25,4")` falha: o padrão brasileiro usa vírgula, e o Python só aceita ponto. Troque antes: `float("25,4".replace(",", "."))`.

> ⚠️ **Erro comum:** achar que o programa "travou" num `ValueError`. Ele fez a coisa certa: preferiu parar a inventar um número. Inventar seria muito pior num certificado de qualidade. No módulo 06 você vai aprender a tratar esses erros com elegância.

Exemplo resolvido: somar dois pesos vindos de uma planilha

Problema: duas células trazem `"125,5"` e `"130,2"`. Quero o total.

Raciocínio: são textos com vírgula. Para cada um: trocar vírgula por ponto, converter com `float()`. Só então somar.

```
peso_1 = float("125,5".replace(",", "."))
peso_2 = float("130,2".replace(",", "."))
total = peso_1 + peso_2
print("Total:", total, "kg")
```

Saída:

```
Total: 255.7 kg
```

> 🔧 **Modifique:** tire o `.replace(",", ".")` de um dos pesos e rode. Leia a mensagem de erro com calma: ela diz qual texto o Python não conseguiu converter.

10. input(): perguntando ao usuário

`input()` mostra uma mensagem, espera a pessoa digitar e apertar Enter, e devolve o que foi digitado. **Sempre como texto**, mesmo que a pessoa digite um número.

```
nome = input("Seu nome: ")
idade_texto = input("Sua idade: ")
idade = int(idade_texto)
print("Olá,", nome, "- daqui a 10 anos você terá", idade + 10, "anos.")
```

Uma execução no terminal (o que a pessoa digitou está depois dos dois-pontos):

```
Seu nome: Ana
Sua idade: 28
Olá, Ana - daqui a 10 anos você terá 38 anos.
```

Sem o `int(...)`, a linha `idade + 10` daria `TypeError`, porque `idade_texto` é `"28"`, um texto.

Duas observações práticas:

- Os **exercícios do curso não usam `input()`**. Eles recebem os valores como parâmetros de função (seção 14). Assim, os testes automáticos conseguem conferir seu código sem ninguém digitar nada.
- O exemplo `exemplos/05_conversoes_e_input.py` **simula** o que seria digitado, para rodar sem parar. Abra o arquivo, troque a linha marcada por `input(...)` e teste a versão interativa.

> 💡 **Dica:** um programa que espera `input()` parece "travado" no terminal. Não está: está esperando você digitar. Se quiser interromper qualquer programa, use `Ctrl+C`.

11. f-strings e formatação de números

Montar textos com `+` e `str()` funciona, mas é trabalhoso e cheio de armadilhas:

```
op = "OP12345"
media = 25.0666
print("A OP " + op + " teve média " + str(media) + " µm")
```

Saída:

```
A OP OP12345 teve média 25.0666 µm
```

A **f-string** é o jeito moderno. Coloque um `f` antes das aspas e escreva as variáveis entre chaves `{}`:

```
op = "OP12345"
media = 25.0666
print(f"A OP {op} teve média {media} µm")
```

Saída:

```
A OP OP12345 teve média 25.0666 µm
```

Dentro das chaves vale qualquer expressão: `{peso / 8}`, `{op.lower()}`, `{len(op)}`. E a conversão para texto é automática.

Formatando números

Depois do valor, um `:` abre a "especificação de formato". As que você mais vai usar:

```
>>> media = 25.0666
>>> f"{media:.2f}"
'25.07'
>>> f"{1250.5:,.2f}"
'1,250.50'
>>> f"{0.956:.1%}"
'95.6%'
```

```
>>> f"{7:03d}"
'007'
>>> f"{-0.35:+.2f} e {0.35:+.2f}"
'-0.35 e +0.35'
```

Formato	Significado	Exemplo
`:.2f`	2 casas decimais (arredonda na exibição)	`25.0666` → `25.07`
`:,.2f`	separador de milhar + 2 casas	`1250.5` → `1,250.50`
`:.1%`	multiplica por 100 e põe `%`	`0.956` → `95.6%`
`:03d`	inteiro com zeros à esquerda até 3 dígitos	`7` → `007`
`:+.2f`	sempre mostra o sinal	`0.35` → `+0.35`

Repare que `:,.2f` usa o **padrão americano** (vírgula no milhar, ponto no decimal). Para o padrão brasileiro (`1.250,50`) existe um truque com `replace`, que você vai montar no exercício 12. No módulo 05 você verá o assunto a fundo.

Alinhando colunas

Um número antes do formato define a **largura**; `<`, `>` e `^` alinham à esquerda, à direita e ao centro:

```
print(f"[{'COF':<10}]")
print(f"[{'COF':>10}]")
print(f"[{'COF':^10}]")
print(f"{'Ensaio':<12}{'Resultado':>10}")
print(f"{'Espessura':<12}{25.07:>10.2f}")
print(f"{'Gramatura':<12}{52.3:>10.2f}")
```

Saída:

```
[COF       ]
[       COF]
[   COF    ]
Ensaio       Resultado
Espessura        25.07
Gramatura        52.30
```

Com isso você já monta relatórios de terminal bem apresentáveis. Note a combinação `>10.2f`: largura 10, alinhado à direita, 2 casas.

Exemplo resolvido: uma linha de laudo

Problema: imprimir `Peso líquido: 1,217.70 kg (8 bobinas, lote 007)` a partir de `peso = 1217.7`, `bobinas = 8` e `lote = 7`.

Raciocínio: separo o texto em partes fixas e partes variáveis. As variáveis vão entre chaves, cada uma com o formato que pede: o peso com milhar e 2 casas (`:,.2f`), o lote com 3 dígitos (`:03d`), as bobinas sem formato.

```
peso = 1217.7
bobinas = 8
lote = 7
print(f"Peso líquido: {peso:,.2f} kg ({bobinas} bobinas, lote
{lote:03d})")
```

Saída:

```
Peso líquido: 1,217.70 kg (8 bobinas, lote 007)
```

🔧 **Modifique:** mude `lote` para `123` e para `1234`. O que acontece com o `:03d` quando o número já tem 3 dígitos ou mais?

💡 **Dica:** para depurar, coloque `=` depois do nome: `print(f"{media=}")` mostra `media=25.0666`. Você vê o nome e o valor de uma vez.

⚠️ **Erro comum:** esquecer o `f`. `print("Média: {media}")` imprime literalmente `Média: {media}`, com as chaves. Sem erro, sem aviso, só o texto errado no relatório.

🔮 **Preveja antes de rodar:**

```
valor = 2.5
print(f"{valor:.0f} | {3.5:.0f} | {0.5:.0%}")
```

▼ **Resposta**

Sai `2 | 4 | 50%`. O `:.0f` arredonda para zero casas, e o empate `.5` vai para o número **par** mais próximo (2.5 → 2, 3.5 → 4). Isso não é bug; é o arredondamento "do banqueiro", explicado na próxima seção. E `0.5` em

porcentagem é `50%`.

12. O problema do float e o round()

Rode isto no REPL:

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 1.1 * 3
3.3000000000000003
```

Não é defeito do Python. Acontece em JavaScript, Java, C e em praticamente toda linguagem que usa float. Até no Excel: ele só esconde boa parte disso porque arredonda o que mostra na célula. O motivo: o computador guarda números em **binário** (base 2), e alguns decimais simples não têm representação exata em binário.

Analogia: tente escrever 1/3 em decimal. Dá `0.3333...` e nunca termina; em algum momento você corta, e o valor fica um pouquinho errado. Com `0.1` em binário acontece o mesmo. O erro é minúsculo (na 17ª casa), mas aparece quando você imprime ou compara.

round(): arredondar na hora certa

```
>>> round(0.1 + 0.2, 2)
0.3
>>> round(25.67, 1)
25.7
>>> round(25.67)
26
```

- `round(x, n)` arredonda para `n` casas e devolve `float`.
- `round(x)` sem casas devolve `int`.

Duas surpresas que você **precisa** conhecer:

```
>>> round(2.5)
2
>>> round(3.5)
4
>>> round(2.675, 2)
2.67
```

1. **Empate vai para o par** ("arredondamento do banqueiro"): `2.5` → `2`, `3.5` → `4`. Isso evita que, somando muitos arredondamentos, o total "puxe" sempre para cima. É o padrão IEEE 754, o mesmo de muitos sistemas financeiros.
2. `round(2.675, 2)` dá `2.67`, não `2.68`, porque `2.675` está guardado como `2.67499999...`. De novo o binário.

Regras práticas:

- **Calcule com todas as casas e arredonde só no fim**, para exibir ou gravar. Arredondar no meio da conta acumula erro.
- Para **exibir**, prefira formatar (`f"{x:.2f}"`) a arredondar o valor.
- Para **comparar** floats, nunca confie na igualdade exata. Use `math.isclose` (próxima seção) ou compare com uma tolerância.
- Para **dinheiro** de verdade (sistemas financeiros), existe o tipo `Decimal`, do módulo `decimal`. Fica para depois; no laboratório, float com arredondamento no fim resolve.

> ⚠️ **Erro comum:** concluir que "o Python calcula errado" e sair arredondando tudo em todo lugar. O cálculo está certo dentro da precisão do float. O que você controla é **quando** arredonda e **como** exibe.

13. O módulo math

O Python vem com uma "biblioteca padrão" enorme, organizada em módulos. Para usar um módulo, você o **importa** uma vez, no topo do arquivo:

```
>>> import math
>>> math.pi
3.141592653589793
>>> math.sqrt(144)
12.0
>>> math.ceil(83.1)
84
>>> math.floor(83.9)
83
>>> math.isclose(0.1 + 0.2, 0.3)
True
```

Função	O que faz
`math.sqrt(x)`	raiz quadrada (devolve float)
`math.pi`	o número π (não é função: sem parênteses)

Função	O que faz
`math.ceil(x)`	arredonda **para cima** até o inteiro (teto)
`math.floor(x)`	arredonda **para baixo** até o inteiro (piso)
`math.isclose(a, b)`	`True` se `a` e `b` são "praticamente iguais"

Potência não precisa de `math`: use `**`. E raiz também pode ser potência: `144 ** 0.5` dá `12.0`.

Exemplo resolvido: quantas caixas?

Problema: 47 bobinas, 6 por caixa. Quantas caixas a expedição precisa?

Raciocínio: `47 / 6 = 7.83`. Não existe 0,83 de caixa, e as bobinas que sobram também precisam de caixa. Logo, arredondo **para cima**, sempre, mesmo que fosse 7,01. Isso é `math.ceil`. Se eu usasse `round`, daria 8 aqui, mas para 7,2 daria 7, e uma bobina ficaria sem caixa.

```
import math

bobinas = 47
por_caixa = 6
print("Caixas:", math.ceil(bobinas / por_caixa))
```

Saída:

```
Caixas: 8
```

> 🔧 **Modifique:** troque para 48 bobinas e depois para 49. Compare com `round(bobinas / por_caixa)` e `bobinas // por_caixa`. Qual dos três sempre dá a resposta certa para "quantas caixas"?

> 💡 **Dica:** como descobrir o que existe em `math`? No REPL: `import math` e depois `help(math)` (aperte `q` para sair) ou `dir(math)`. A documentação oficial em português também é ótima: procure "math" em docs.python.org/pt-br/3/. Aprender a consultar a documentação é uma habilidade profissional, não um sinal de que você "não sabe".

14. O molde dos exercícios: def e return

Todos os exercícios do curso pedem para você completar **funções**. O assunto é aprofundado no módulo 04; aqui está só o necessário para trabalhar.

Uma função é uma **máquina com entrada e saída**, como um equipamento do laboratório: você coloca o corpo de prova (a entrada), a máquina faz o ensaio e devolve um resultado (a saída). Quem usa a máquina não precisa saber como ela funciona por dentro.

```
def area_retangulo(largura: float, comprimento: float) -> float:
    """Devolve a área de um retângulo."""
    return largura * comprimento


area = area_retangulo(0.1, 0.1)
print("Área:", round(area, 4), "m²")
print("Outra:", area_retangulo(2, 3))
```

Saída:

```
Área: 0.01 m²
Outra: 6
```

Peça por peça:

```
def area_retangulo(largura: float, comprimento: float) -> float:
|   |              |                                      |
|   |              |                                      └─ tipo do
resultado (dica)
|   |              └─ parâmetros: os valores de entrada
|   └─ nome da função (snake_case, como variável)
└─ "vou definir uma função"

    """Devolve a área de um retângulo."""   ← docstring: o que ela faz
    return largura * comprimento           ← devolve o resultado e
encerra
```

- `def` cria a função, mas **não executa nada**. O código de dentro só roda quando alguém **chama** a função: `area_retangulo(0.1, 0.1)`.
- Na chamada, os valores entre parênteses (os **argumentos**) ocupam os parâmetros na ordem: `largura` recebe `0.1`, `comprimento` recebe `0.1`.
- `return` devolve o resultado para quem chamou. Quem chamou pode guardar numa variável, imprimir ou usar em outra conta.

- O corpo da função é tudo que está **indentado** (recuado 4 espaços) abaixo do `def`. A indentação não é enfeite: é ela que diz ao Python onde a função começa e termina.

Sobre `largura: float` e `-> float`: são **dicas de tipo** (type hints). Elas dizem o que a função espera receber e o que devolve; o Python não obriga, mas o editor usa para avisar erros, e elas documentam o código. Detalhes no módulo 04.

print não é return

Esta é a confusão número 1 com funções. `print` **mostra** um valor na tela. `return` **devolve** o valor para o programa continuar usando.

> 🔮 **Preveja antes de rodar:** o que aparece na tela?
>
> ```python
> def dobro(numero):
>     return numero * 2
>
>
> def mostrar_dobro(numero):
>     print(numero * 2)
>
>
> a = dobro(21)
> b = mostrar_dobro(21)
> print(a, b)
> ```

<details>
<summary>Resposta</summary>

```
42
42 None
```

A primeira linha (`42`) vem do `print` **dentro** de `mostrar_dobro`. A segunda mostra `a` e `b`: `a` recebeu `42` porque `dobro` usa `return`; `b` recebeu `None`, porque `mostrar_dobro` não tem `return`, e toda função sem `return` devolve `None`. Moral: se o resultado vai ser usado depois (e nos exercícios sempre vai, pelos testes), use `return`.
</details>

Exemplo resolvido: do enunciado à função

Enunciado: "Escreva `calcular_gramatura(massa_g, area_m2)` que devolve a gramatura em g/m², arredondada em 1 casa."

Raciocínio:

1. O que entra? `massa_g` e `area_m2` (já estão nos parâmetros).
2. O que sai? Um número: gramatura = massa ÷ área, arredondada em 1 casa.
3. Testo a conta no REPL com um caso que eu sei a resposta: 0,523 g em 0,01 m² devem dar 52,3 g/m².

```
>>> 0.523 / 0.01
52.300000000000004
>>> round(0.523 / 0.01, 1)
52.3
```

4. Agora escrevo a função, trocando os números pelos parâmetros:

```
def calcular_gramatura(massa_g: float, area_m2: float) -> float:
    """Devolve a gramatura em g/m², com 1 casa decimal."""
    gramatura = massa_g / area_m2
    return round(gramatura, 1)


print(calcular_gramatura(0.523, 0.01))
```

Saída:

```
52.3
```

Como é um exercício do curso

Abra `exercicios/ex01_apresentacao.py`. Você verá:

```
def apresentar(nome: str, idade: int, cidade: str) -> str:
    """Devolve a frase de apresentação com nome, idade e cidade."""
    # Escreva seu código aqui
    raise NotImplementedError("Implemente esta função")
```

A linha `raise NotImplementedError(...)` é um "aviso de obra": ela faz a função dar erro de propósito, para os testes mostrarem que o exercício ainda não foi feito. Seu trabalho:

1. ler o enunciado (a docstring no topo do arquivo);
2. **apagar** a linha do `raise` e escrever seu código no lugar, terminando com `return`;
3. rodar, na pasta `curso-python`: `python curso.py testar 01 ex01`;
4. ler a mensagem dos testes, corrigir e repetir até passar.

> ⚠️ **Erro comum:** esquecer de recuar o código dentro do `def`. Sem os 4 espaços, aparece `IndentationError: expected an indented block after function definition on line 1`. No VS Code, a tecla Tab já insere 4 espaços.

15. Pensando como programador

Saber a sintaxe é metade do caminho. A outra metade é saber **o que fazer quando você não sabe o que fazer**. Quatro hábitos que separam quem trava de quem avança:

1. Decomponha. Problema grande é a soma de problemas pequenos. "Gerar a ficha de uma amostra" vira: (a) limpar a OP; (b) converter as medições; (c) calcular a média; (d) calcular a faixa; (e) imprimir. Cada item é uma ou duas linhas que você já sabe escrever.

2. Teste hipóteses no REPL. Não sabe se `strip()` tira espaços do meio? Não discuta com a dúvida, pergunte ao Python:

```
>>> "  OP 123  ".strip()
'OP 123'
```

Resposta em 5 segundos: não tira. Para tirar todos, `replace(" ", "")`.

3. Leia a documentação. `help(str.replace)` no REPL mostra como usar um método. A documentação oficial em português (docs.python.org/pt-br/3/) tem tutorial e referência completos. Comece pelo tutorial.

4. Depure com print. Resultado estranho? Imprima os valores intermediários e veja onde a conta saiu do trilho:

```
massa_g = 0.523
largura_cm = 10
comprimento_cm = 10
area = largura_cm * comprimento_cm  # esqueci de converter para m²!
print(f"{area=}")
print(f"{massa_g / area=}")
```

Saída:

```
area=100
massa_g / area=0.00523
```

Gramatura de 0,005 g/m² é absurda (filmes têm dezenas de g/m²). O `print` mostra que a área está em cm², não em m². Achou o erro sem adivinhar.

> 💡 **Dica:** ao usar IA para estudar, peça "me explique por que este código dá TypeError" ou "me dê uma dica sem a resposta", e não "resolva o exercício". O objetivo é você conseguir resolver sozinho na entrevista e no trabalho.

Lendo mensagens de erro deste módulo

Mensagem de erro (traceback) não é bronca: é um **laudo** do problema. Leia **de baixo para cima**:

1. **última linha**: tipo do erro e descrição;
2. linhas de cima: **arquivo e número da linha** onde aconteceu, com o trecho de código e marcas (`^` ou `~`) apontando o lugar exato.

Um traceback completo, de um script `teste.py`:

```
Traceback (most recent call last):
  File "C:\curso-python\teste.py", line 2, in <module>
    print("Bobinas: " + quantidade)
          ~~~~~~~~~~~~^~~~~~~~~~~~
TypeError: can only concatenate str (not "int") to str
```

Tradução: "na linha 2 de teste.py, você tentou juntar (`+`) um texto com um inteiro; só sei concatenar texto com texto". As marcas `~~~^~~~` apontam o `+` problemático. Correção: `"Bobinas: " + str(quantidade)` ou `f"Bobinas: {quantidade}"`.

Os erros mais comuns deste módulo:

Erro	Exemplo que causa	O que significa	Como corrigir
`NameError: name 'espesura' is not defined. Did you mean: 'espessura'?`	`print(espesura)`	nome não existe (erro de digitação ou variável não criada)	confira a grafia; crie a variável antes de usar

Erro	Exemplo que causa	O que significa	Como corrigir
`NameError: name 'Print' is not defined. Did you mean: 'print'?`	`Print("Olá")`	maiúscula/minúscula errada	`print` é minúsculo
`TypeError: can only concatenate str (not "int") to str`	`"Lote " + 7`	juntou texto com número	`str(7)` ou f-string
`ValueError: invalid literal for int() with base 10: 'abc'`	`int("abc")`	o texto não é um inteiro válido	confira o dado; para decimais use `float`
`ValueError: could not convert string to float: '12,5'`	`float("12,5")`	vírgula decimal	`.replace(",", ".")` antes
`ZeroDivisionError: division by zero`	`100 / pessoas` com `pessoas = 0`	divisão por zero	confira o divisor (módulo 02 ensina a testar antes)
`AttributeError: 'int' object has no attribute 'upper'`	`lote = 7` e `lote.upper()`	método de texto num número	converta, ou reveja o tipo da variável

Erros de **sintaxe** são diferentes: o Python nem começa a rodar, porque não entendeu o texto do programa. Não aparece a linha `Traceback`, só o local:

```
  File "C:\curso-python\teste.py", line 1
    print("Olá)
          ^
SyntaxError: unterminated string literal (detected at line 1)
```

"String não terminada": abriu aspas e não fechou. Outro clássico:

```
  File "C:\curso-python\teste.py", line 1
    print("Média:", (7 + 8 + 9 / 3)
         ^
SyntaxError: '(' was never closed
```

Parêntese aberto e nunca fechado. Conte os parênteses: cada `(` precisa do seu `)`.

E os de **indentação**:

```
  File "C:\curso-python\teste.py", line 2
    media = total / 2
IndentationError: unexpected indent
```

"Recuo inesperado": a linha 2 começa com espaços sem motivo (não está dentro de nenhuma função). Tire os espaços.

```
  File "C:\curso-python\teste.py", line 2
    return n * 2
    ^^^^^^
IndentationError: expected an indented block after function definition on
line 1
```

"Esperava um bloco recuado depois da definição da função": o corpo do `def` precisa estar 4 espaços para dentro.

> 💡 **Dica:** às vezes o erro de sintaxe aponta para a linha **seguinte** à que tem o problema (um parêntese não fechado na linha 5 pode estourar na linha 6). Se a linha apontada parece perfeita, olhe a anterior.

Boas práticas

- **Nomes que explicam**: `peso_liquido_kg`, não `p` ou `x1`. Inclua a unidade quando houver dúvida.
- **snake_case sem acento** para variáveis e funções; MAIÚSCULAS para constantes.
- **Converta na entrada**: transforme texto em número assim que o dado chega, não no meio das contas.
- **Arredonde só no fim**, e prefira formatar na exibição (`:.2f`) a arredondar o valor.
- **f-strings** em vez de `+` com `str()`.
- **Uma ideia por linha**: quebrar uma fórmula em variáveis intermediárias (`area_m2`, `peso_g`) facilita conferir e depurar.
- **Comentários explicam o porquê**, não repetem o código.
- **Teste no REPL** antes de escrever no arquivo.
- **Formate o código** com `ruff format` (o curso já vem configurado). Código bem formatado é mais fácil de ler e é o que as empresas esperam.

No mundo real

A ficha abaixo junta tudo do módulo num caso de laboratório: os dados chegam como texto (como viriam de um formulário ou planilha), são limpos e convertidos, os cálculos são feitos com nomes claros e o resultado sai numa ficha alinhada. É o arquivo `exemplos/10_ficha_amostra.py`.

```
import math

# --- Dados de entrada (do jeito que chegam) ---
op_digitada = " op-48213 "
linha = "c"
espessura_1 = "24,8"
espessura_2 = "25,3"
espessura_3 = "25,1"
massa_corpo_prova_g = "0,4710"  # disco de 100 cm²
comprimento_bobina_m = 6000
largura_bobina_mm = 640
espessura_nominal_um = 25.0
tolerancia_pct = 8.0

# --- Limpeza e conversão ---
op = "OP" + op_digitada.upper().replace("OP", "").replace("-",
"").strip()
linha = linha.strip().upper()
e1 = float(espessura_1.replace(",", "."))
e2 = float(espessura_2.replace(",", "."))
e3 = float(espessura_3.replace(",", "."))
massa_g = float(massa_corpo_prova_g.replace(",", "."))

# --- Cálculos ---
media_um = (e1 + e2 + e3) / 3
minimo_um = espessura_nominal_um - espessura_nominal_um * tolerancia_pct
/ 100
maximo_um = espessura_nominal_um + espessura_nominal_um * tolerancia_pct
/ 100
desvio = (media_um - espessura_nominal_um) / espessura_nominal_um
gramatura = massa_g / 0.01  # 100 cm² = 0,01 m²
peso_kg = gramatura * (largura_bobina_mm / 1000) * comprimento_bobina_m /
1000
diametro_mm = math.sqrt(76**2 + 4 * comprimento_bobina_m * 1000 *
(media_um / 1000) / math.pi)

# --- Saída ---
faixa = f"{minimo_um:.2f} a {maximo_um:.2f}"
print("=" * 40)
print(f"{'FICHA DE AMOSTRA':^40}")
print("=" * 40)
```

```
print(f"{'OP':<22}{op:>18}")
print(f"{'Linha':<22}{linha:>18}")
print(f"{'Espessura média (µm)':<22}{media_um:>18.2f}")
print(f"{'Faixa (µm)':<22}{faixa:>18}")
print(f"{'Desvio do nominal':<22}{desvio:>+18.2%}")
print(f"{'Gramatura (g/m²)':<22}{gramatura:>18.1f}")
print(f"{'Peso estimado (kg)':<22}{peso_kg:>18,.2f}")
print(f"{'Diâmetro estimado (mm)':<22}{diametro_mm:>18.0f}")
print("-" * 40)
print("Veredito: aprovado ou reprovado? No módulo 02!")
```

Saída:

```
========================================
            FICHA DE AMOSTRA
========================================
OP                                OP48213
Linha                                   C
Espessura média (µm)                25.07
Faixa (µm)                  23.00 a 27.00
Desvio do nominal                  +0.27%
Gramatura (g/m²)                     47.1
Peso estimado (kg)                 180.86
Diâmetro estimado (mm)                444
----------------------------------------
Veredito: aprovado ou reprovado? No módulo 02!
```

Repare na organização em três blocos (entrada, cálculo, saída). Esse padrão vale para scripts de qualquer tamanho e é a semente do projeto final. O que falta? **Decidir**: a média está dentro da faixa? A ficha deve dizer "APROVADO" ou "REPROVADO"? Isso exige `if`, assunto do módulo 02.

Resumo

Assunto	Sintaxe essencial
Mostrar	`print("a", 1, sep=" - ", end="\n")`
Comentário	`# texto ignorado pelo Python`
Variável	`peso_kg = 125.5`
Tipos	`int` `8` · `float` `25.4` · `str` `"OP1"` · `bool` `True` · `None`
Tipo de um valor	`type(valor)`
Contas	`+ - * / // % **` (precedência: `()` → `**` → `* / // %` → `+ -`)

Assunto	Sintaxe essencial
Atualizar	`total += x` · `total -= x` · `total *= x` · `total /= x`
Texto	`+` concatena · `*` repete · `len(t)` · `t.upper()` · `t.lower()` · `t.strip()` · `t.replace(a, b)` · `"x" in t`
Conversão	`int("42")` · `float("25.4")` · `str(42)` · vírgula: `float(t.replace(",", "."))`
Entrada	`texto = input("Pergunta: ")` (sempre devolve `str`)
f-string	`f"{valor}"` · `:.2f` · `:,.2f` · `:.1%` · `:03d` · `:+.2f` · `:<10` `:>10` `:^10` · `{x=}`
Arredondar	`round(x, 2)` (float) · `round(x)` (int) · empate vai para o par
math	`import math` · `math.sqrt` · `math.pi` · `math.ceil` · `math.floor` · `math.isclose`
Função	`def nome(param: tipo) -> tipo:` + corpo recuado + `return resultado`

Você domina este módulo se...

- ▢ explico, sem consultar, a diferença entre `/`, `//` e `%`, com um exemplo de bobinas ou caixas;
- ▢ prevejo o resultado de `2 + 3 * 4 ** 2` antes de rodar (e acerto);
- ▢ sei por que `"25" + "5"` dá `"255"` e como converter para somar de verdade;
- ▢ transformo `" 12,5 kg "` no número `12.5` usando só métodos de string e `float()`;
- ▢ formato `1250.5` como `1,250.50` e `0.956` como `95.6%` com f-strings, sem consultar;
- ▢ explico por que `0.1 + 0.2` não dá `0.3` e quando usar `round` ou `:.2f`;
- ▢ explico a diferença entre `print` e `return` numa função e por que os exercícios usam `return`;
- ▢ leio a última linha de um traceback e digo o tipo de erro, a linha e a causa provável.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_apresentacao`	★☆☆	f-string, return
02	`ex02_cabecalho`	★☆☆	print, repetição de string, upper

Nº	Exercício	Nível	O que pratica
03	`ex03_media_notas`	★☆☆	precedência, parênteses
04	`ex04_celsius_fahrenheit`	★☆☆	fórmulas, ordem das operações
05	`ex05_contem_termo`	★☆☆	`in`, lower, strip, bool
06	`ex06_peso_liquido`	★★☆	contas encadeadas, round
07	`ex07_dividir_conta`	★★☆	porcentagem, round, f-string com `:.2f`, ZeroDivisionError
08	`ex08_normalizar_op`	★★☆	upper, replace encadeado
09	`ex09_gramatura`	★★☆	conversão de unidades, round
10	`ex10_faixa_tolerancia`	★★☆	tolerância ± %, função chamando função, `:.2f`
11	`ex11_texto_para_numero`	★★☆	limpeza de texto, float(), ValueError, problema do float
12	`ex12_formatar_reais`	★★★	`:,.2f` + troca de separadores
13	`ex13_diametro_bobina`	★★★	`math.sqrt`, `math.pi`, unidades, fórmula inversa
14	`ex14_tempo_de_producao`	★★★	`math.ceil`, `//`, `%`, `:02d`

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError`, e rode na pasta `curso-python`:

```
python curso.py testar 01 ex01
```

Para testar o módulo inteiro: `python curso.py testar 01`. Faça na ordem: os primeiros preparam os seguintes. Depois de passar, compare com `solucoes/`: muitas vezes há uma alternativa comentada.

Desafio do módulo

Ficha de medição da bobina: um script que recebe os dados brutos de uma amostra (OP mal digitada, medições com vírgula, massa do corpo de prova, dimensões da bobina) e imprime uma ficha completa e alinhada, com média, faixa de tolerância, desvio, gramatura, peso e diâmetro estimados e os valores no formato brasileiro. Enunciado, requisitos e critérios de aceite em desafio/README.md.

Quiz

Teste o que ficou: quiz.md (15 perguntas, gabarito comentado no final).

Para ir além

- Tutorial oficial em português, capítulo 3, "Uma introdução informal ao Python": https://docs.python.org/pt-br/3/tutorial/introduction.html
- Formatação de strings (f-strings e a "mini-linguagem" de formato): https://docs.python.org/pt-br/3/tutorial/inputoutput.html e https://docs.python.org/pt-br/3/library/string.html#formatspec
- Aritmética de ponto flutuante, problemas e limitações: https://docs.python.org/pt-br/3/tutorial/floatingpoint.html
- Módulo `math`: https://docs.python.org/pt-br/3/library/math.html
- Métodos de string: https://docs.python.org/pt-br/3/library/stdtypes.html#string-methods
- Leitura extra: PEP 8, o guia de estilo do Python (seções "Naming Conventions" e "Comments"): https://peps.python.org/pep-0008/

Enunciados dos exercícios

Exercício 01.01 — Apresentação

Arquivo: `exercicios/ex01_apresentacao.py`

```
Nível: ★☆☆

Contexto: Todo programa começa conversando com alguém. Aqui você monta
uma frase de
apresentação a partir de três informações.

Tarefa: implemente `apresentar(nome, idade, cidade)`, que DEVOLVE (com
return, sem
print) exatamente a frase:
    Meu nome é <nome>, tenho <idade> anos e moro em <cidade>.
- Use uma f-string.
- Não mude nada nos valores recebidos (nem maiúsculas, nem espaços).

Exemplos:
    >>> apresentar("Ana", 28, "Joinville")
    'Meu nome é Ana, tenho 28 anos e moro em Joinville.'
    >>> apresentar("Carlos Souza", 41, "São Bento do Sul")
    'Meu nome é Carlos Souza, tenho 41 anos e moro em São Bento do Sul.'
```

```
Dica: o REPL mostra o texto devolvido entre aspas; as aspas não fazem
parte da frase.

Teste com:  python curso.py testar 01 ex01
```

Exercício 01.02 — Cabeçalho de laudo

Arquivo: `exercicios/ex02_cabecalho.py`

```
Nível: ★☆☆

Contexto: Os laudos impressos pelo laboratório começam com um cabeçalho:
uma linha de
sinais de igual, o título em MAIÚSCULAS e outra linha de sinais de igual.

Tarefa: implemente `imprimir_cabecalho(titulo, largura)`, que IMPRIME
(com print) três
linhas:
    1. o caractere "=" repetido `largura` vezes;
    2. o título em maiúsculas;
    3. de novo o "=" repetido `largura` vezes.
A função não devolve nada (não precisa de return).

Exemplos:
    >>> imprimir_cabecalho("laudo de espessura", 20)
    ====================
    LAUDO DE ESPESSURA
    ====================

Dica: em Python, "ab" * 3 resulta em "abababab".

Teste com:  python curso.py testar 01 ex02
```

Exercício 01.03 — Média de notas

Arquivo: `exercicios/ex03_media_notas.py`

```
Nível: ★☆☆

Contexto: Clássico de todo curso: calcular a média de um aluno. É o lugar
perfeito para
cair (ou não) na armadilha da precedência dos operadores.

Tarefa:
1. `calcular_media(nota1, nota2, nota3)` devolve a média aritmética das
três notas.
```

```
2. `calcular_media_ponderada(nota1, peso1, nota2, peso2)` devolve a média
ponderada:
   (nota1 * peso1 + nota2 * peso2) / (peso1 + peso2).
Não arredonde os resultados.

Exemplos:
    >>> calcular_media(7, 8, 9)
    8.0
    >>> calcular_media_ponderada(8, 2, 5, 1)
    7.0

Dica: o que o Python calcula primeiro em 7 + 8 + 9 / 3? Teste no REPL.

Teste com:  python curso.py testar 01 ex03
```

Exercício 01.04 — Celsius e Fahrenheit

Arquivo: `exercicios/ex04_celsius_fahrenheit.py`

```
Nível: ★☆☆

Contexto: O manual da seladora importada traz as temperaturas em °F, mas
o
procedimento do laboratório usa °C. Você vai escrever as duas conversões.

Tarefa:
1. `celsius_para_fahrenheit(celsius)` devolve a temperatura em °F:
   F = C * 9 / 5 + 32
2. `fahrenheit_para_celsius(fahrenheit)` faz o caminho inverso:
   C = (F - 32) * 5 / 9
Não arredonde os resultados.

Exemplos:
    >>> celsius_para_fahrenheit(100)
    212.0
    >>> fahrenheit_para_celsius(32)
    0.0

Dica: na fórmula inversa, os parênteses em (F - 32) são obrigatórios. Por
quê?

Teste com:  python curso.py testar 01 ex04
```

Exercício 01.05 — Procurar um termo na observação

Arquivo: `exercicios/ex05_contem_termo.py`

```
Nível: ★☆☆

Contexto: Na planilha de resultados, a coluna de observação traz textos
como
"COF: liberado" ou "Tração: OK". O sistema do projeto final precisa saber
se um termo
aparece ali, sem se importar com maiúsculas e minúsculas.

Tarefa: implemente `contem_termo(texto, termo)`, que devolve True se
`termo` aparece
dentro de `texto` e False caso contrário.
- Ignore a diferença entre maiúsculas e minúsculas ("COF" e "cof" são
iguais).
- Ignore espaços no começo e no fim de `termo` ("  liberado " vale como
"liberado").

Exemplos:
    >>> contem_termo("COF: liberado", "cof")
    True
    >>> contem_termo("Tração: OK", "COF")
    False
    >>> contem_termo("cof: LIBERADO", "  liberado ")
    True

Dica: o operador `in` já devolve True ou False. Você não precisa de if.

Teste com:  python curso.py testar 01 ex05
```

Exercício 01.06 — Peso líquido do palete

Arquivo: `exercicios/ex06_peso_liquido.py`

```
Nível: ★★☆

Contexto: A expedição pesa o palete inteiro na balança (peso bruto). Para
a nota
fiscal, o que vale é o peso líquido do filme: tire o peso do palete de
madeira e o peso
de cada tubete (o tubo de papelão no centro de cada bobina).

Tarefa: implemente `calcular_peso_liquido(peso_bruto_kg, tara_palete_kg,
qtd_bobinas,
peso_tubete_kg)`, que devolve:
    peso bruto - tara do palete - (quantidade de bobinas x peso de um
tubete)
arredondado para 2 casas decimais com round().

Exemplos:
```

```
    >>> calcular_peso_liquido(1250.5, 22.0, 8, 1.35)
    1217.7
    >>> calcular_peso_liquido(500.0, 18.7, 4, 1.237)
    476.35

Dica: confira no REPL quanto dá 500.0 - 18.7 - 4 * 1.237 sem arredondar.

Teste com:  python curso.py testar 01 ex06
```

Exercício 01.07 — Dividir a conta

Arquivo: `exercicios/ex07_dividir_conta.py`

```
Nível: ★★☆

Contexto: Fim do mês, a equipe do laboratório sai para almoçar. A conta
vem sem a taxa
de serviço e alguém precisa calcular quanto cada um paga.

Tarefa:
1. `valor_por_pessoa(total, pessoas, servico_pct)` devolve quanto cada
pessoa paga:
   o total acrescido de `servico_pct` por cento, dividido pelo número de
pessoas,
   arredondado com round() para 2 casas.
   Se `pessoas` for 0, deixe o próprio Python levantar ZeroDivisionError
(não trate).
2. `resumo_conta(total, pessoas, servico_pct)` devolve o texto:
   Total com <servico_pct>% de serviço: R$ <total com serviço> | Por
pessoa: R$ <valor>
   com os dois valores em 2 casas decimais (formato :.2f).

Exemplos:
    >>> valor_por_pessoa(150.0, 4, 10)
    41.25
    >>> resumo_conta(100.0, 3, 10)
    'Total com 10% de serviço: R$ 110.00 | Por pessoa: R$ 36.67'

Dica: acrescentar 10% é multiplicar por (1 + 10 / 100).

Teste com:  python curso.py testar 01 ex07
```

Exercício 01.08 — Normalizar o número da OP

Arquivo: `exercicios/ex08_normalizar_op.py`

```
Nível: ★★☆

Contexto: Cada pessoa escreve a ordem de produção (OP) de um jeito: " op-
12345 ",
"OP 12.345", "Op: 12345" ou só "12345". Para cruzar a planilha com o e-
mail da
expedição, o sistema precisa de um formato único: "OP12345".

Tarefa: implemente `normalizar_op(texto)`, que:
1. passa o texto para maiúsculas;
2. remove as letras "OP" e os caracteres "-", ".", ":" e espaços (em
qualquer posição);
3. devolve "OP" seguido do que sobrou.

Exemplos:
    >>> normalizar_op(" op-12345 ")
    'OP12345'
    >>> normalizar_op("OP 12.345")
    'OP12345'
    >>> normalizar_op("12345")
    'OP12345'

Dica: replace() pode ser encadeado: texto.replace("a", "").replace("b",
"").

Teste com:  python curso.py testar 01 ex08
```

Exercício 01.09 — Gramatura e peso da bobina

Arquivo: `exercicios/ex09_gramatura.py`

```
Nível: ★★☆

Contexto: Para medir a gramatura, o laboratório corta um corpo de prova
retangular do
filme e pesa na balança analítica. Gramatura é massa por área: gramas por
metro
quadrado (g/m²). Com a gramatura, dá para estimar o peso de uma bobina
inteira.

Tarefa:
1. `calcular_gramatura(massa_g, largura_cm, comprimento_cm)` devolve a
gramatura em
   g/m², arredondada para 1 casa decimal. Lembre: 1 cm = 0,01 m.
2. `peso_bobina_kg(gramatura_g_m2, largura_mm, comprimento_m)` devolve o
peso da
   bobina em kg, arredondado para 2 casas:
   gramatura (g/m²) x largura (m) x comprimento (m) = gramas; divida por
```

```
1000 para kg.
   Lembre: 1 mm = 0,001 m.

Exemplos:
    >>> calcular_gramatura(0.523, 10, 10)
    52.3
    >>> peso_bobina_kg(52.3, 600, 4000)
    125.52

Dica: calcule a área em m² numa variável separada antes de dividir.

Teste com:  python curso.py testar 01 ex09
```

Exercício 01.10 — Faixa de tolerância

Arquivo: `exercicios/ex10_faixa_tolerancia.py`

```
Nível: ★★☆

Contexto: A especificação de um filme diz "espessura 25 µm ± 10%". Antes
de aprovar
ou reprovar qualquer coisa, o sistema precisa saber os limites: o mínimo
e o máximo
aceitos.

Tarefa:
1. `limite_minimo(nominal, tolerancia_pct)` devolve nominal -
tolerancia_pct% do nominal.
2. `limite_maximo(nominal, tolerancia_pct)` devolve nominal +
tolerancia_pct% do nominal.
3. `descrever_faixa(nominal, tolerancia_pct, unidade)` devolve o texto
   "<mínimo> a <máximo> <unidade>", com os limites em 2 casas decimais.
Não arredonde os limites nas funções 1 e 2 (só o texto da função 3 é
formatado).

Exemplos:
    >>> limite_minimo(25, 10)
    22.5
    >>> descrever_faixa(25, 10, "µm")
    '22.50 a 27.50 µm'

Dica: 10% de 25 é 25 * 10 / 100.

Teste com:  python curso.py testar 01 ex10
```

Exercício 01.11 — Texto para número

Arquivo: `exercicios/ex11_texto_para_numero.py`

```
Nível: ★★☆

Contexto: Valores que vêm de planilhas, e-mails e formulários chegam como
TEXTO:
" 25,4 ", "1250.5", "12,5 kg". Antes de fazer qualquer conta, é preciso
converter.

Tarefa:
1. `texto_para_numero(texto)` devolve o número (float) contido no texto:
   - ignore maiúsculas/minúsculas e remova a unidade "kg", se houver;
   - remova os espaços do começo e do fim;
   - troque a vírgula decimal por ponto;
   - converta com float().
   Se o texto não for um número (ex.: "abc" ou ""), deixe o float()
levantar
   ValueError (não trate o erro).
   Não precisa tratar separador de milhar como "1.250,5" (isso fica para
o módulo 05).
2. `somar_pesos(peso1, peso2)` recebe dois textos, converte os dois com a
função acima
   e devolve a soma arredondada com round() para 2 casas.

Exemplos:
    >>> texto_para_numero(" 25,4 ")
    25.4
    >>> texto_para_numero("12,5 KG")
    12.5
    >>> somar_pesos("0,1", "0,2")
    0.3

Dica: sem o round(), quanto dá 0.1 + 0.2? Teste no REPL.

Teste com:  python curso.py testar 01 ex11
```

Exercício 01.12 — Formatar em reais

Arquivo: `exercicios/ex12_formatar_reais.py`

```
Nível: ★★★

Contexto: O Python formata números no padrão americano: f"{1234.5:,.2f}"
vira
"1,234.50". No Brasil, o certo é "1.234,50". Relatórios para a diretoria
e para
```

```
clientes precisam do formato brasileiro.

Tarefa: implemente `formatar_reais(valor)`, que devolve o valor no
formato
"R$ 1.234,56":
- sempre 2 casas decimais;
- ponto separando os milhares e vírgula separando os centavos;
- "R$ " (com espaço) na frente.
Considere apenas valores maiores ou iguais a zero.

Exemplos:
    >>> formatar_reais(1234.56)
    'R$ 1.234,56'
    >>> formatar_reais(0.5)
    'R$ 0,50'
    >>> formatar_reais(1234567.891)
    'R$ 1.234.567,89'

Dica: formate no padrão americano primeiro e depois troque os
separadores. Se trocar
"," por "." e logo em seguida "." por ",", tudo vira vírgula. Use um
caractere
provisório no meio do caminho.

Teste com:  python curso.py testar 01 ex12
```

Exercício 01.13 — Diâmetro da bobina

Arquivo: `exercicios/ex13_diametro_bobina.py`

```
Nível: ★★★

Contexto: Na rebobinadeira, ninguém mede o comprimento de uma bobina
desenrolando o
filme. Usa-se geometria: o filme enrolado ocupa a área de uma "rosca"
entre o tubete e
o diâmetro externo. Com a espessura do filme, dá para ir do comprimento
ao diâmetro e
do diâmetro ao comprimento.

Fórmulas (todas as medidas em MILÍMETROS):
    diâmetro externo  D = raiz quadrada de ( d² + 4 x L x e / pi )
    comprimento       L = pi x (D² - d²) / (4 x e)
onde d = diâmetro do tubete, L = comprimento do filme e e = espessura do
filme.

Tarefa:
1. `diametro_bobina_mm(comprimento_m, espessura_um, diametro_tubete_mm)`
devolve o
```

```
   diâmetro externo em mm, arredondado para 1 casa decimal.
2. `comprimento_bobina_m(diametro_externo_mm, espessura_um,
diametro_tubete_mm)`
   devolve o comprimento em METROS, arredondado para o metro inteiro com
round(x)
   (sem o segundo argumento, round devolve um int).
Converta as unidades antes de usar as fórmulas: 1 m = 1000 mm e 1 µm =
0,001 mm.

Exemplos:
    >>> diametro_bobina_mm(4000, 25, 76)
    364.8
    >>> comprimento_bobina_m(500, 50, 152)
    3564

Dica: comece o arquivo com `import math` (logo abaixo desta docstring)
para usar
math.sqrt e math.pi.

Teste com:  python curso.py testar 01 ex13
```

Exercício 01.14 — Tempo de produção

Arquivo: `exercicios/ex14_tempo_de_producao.py`

```
Nível: ★★★

Contexto: O PCP pergunta ao laboratório quando a OP vai terminar para
agendar a
coleta de amostras. Você sabe quantos metros faltam produzir e a
velocidade da
máquina (metros por minuto).

Tarefa:
1. `minutos_de_producao(metros, velocidade_m_min)` devolve quantos
minutos INTEIROS
   a máquina precisa. Qualquer fração de minuto conta como um minuto a
mais
   (83,3 minutos viram 84). Use math.ceil.
2. `tempo_de_producao(metros, velocidade_m_min)` devolve esse tempo no
formato
   "<horas>h<minutos>min", com os minutos SEMPRE em 2 dígitos: "1h24min",
"0h05min".
3. `horario_termino(hora_inicio, minuto_inicio, metros,
velocidade_m_min)` devolve o
   horário de término no formato "HH:MM" (2 dígitos cada). Se passar da
meia-noite,
   volte a contar do zero: 22:50 + 84 minutos = "00:14".
```

```
Exemplos:
    >>> minutos_de_producao(12500, 150)
    84
    >>> tempo_de_producao(12500, 150)
    '1h24min'
    >>> horario_termino(22, 50, 12500, 150)
    '00:14'

Dica: converta tudo para minutos desde a meia-noite, some, e use // e %
para voltar
a horas e minutos. Para "dar a volta" no relógio, % 24 nas horas resolve.
Lembre do `import math` no topo do arquivo.

Teste com:  python curso.py testar 01 ex14
```

Desafio do módulo 01 — Ficha de medição da bobina

Contexto

Toda bobina amostrada no laboratório gera uma ficha: OP, linha, cliente, as medições de espessura, a gramatura e algumas estimativas que o PCP e a expedição pedem (peso e diâmetro da bobina). Hoje essa ficha é preenchida à mão numa planilha, com contas feitas na calculadora.

Seu desafio é escrever um script que recebe os dados **brutos** (do jeito que chegam: OP mal digitada, números com vírgula, espaços sobrando) e imprime a ficha completa, calculada e alinhada. Ainda sem decidir "aprovado ou reprovado": isso exige `if`, assunto do módulo 02. Lá você vai voltar a este script e acrescentar o veredito.

Requisitos obrigatórios

1. Crie o arquivo `ficha_bobina.py` (pode ser dentro de `desafio/`).
2. **Entrada no topo do arquivo**, em variáveis, com dados brutos como texto quando for realista:
   - OP digitada de qualquer jeito (ex.: `"  op-48.213 "`);
   - linha (ex.: `" c "`) e cliente (ex.: `"  embalagens sul ltda "`);
   - **cinco** medições de espessura como texto com vírgula (ex.: `"24,8"`);
   - espessura nominal (µm) e tolerância (%);
   - massa do corpo de prova (texto com vírgula, em g) e as dimensões dele (cm);
   - largura (mm) e comprimento (m) da bobina, e diâmetro do tubete (mm).

3. **Limpeza e conversão:** OP no formato `OP48213`; linha e cliente sem espaços nas pontas e em maiúsculas; medições e massa convertidas para `float`.
4. **Cálculos:**
   - média das cinco medições;
   - limites mínimo e máximo da faixa (nominal ± tolerância %);
   - desvio da média em relação ao nominal, em porcentagem e com sinal (`+0.16%`);
   - gramatura (g/m²) do corpo de prova;
   - peso estimado da bobina (kg);
   - diâmetro estimado da bobina (mm), com a fórmula do exercício 13 (use a espessura média).
5. **Saída:** uma ficha com título centralizado, rótulos alinhados à esquerda e valores alinhados à direita, linhas de separação, espessuras com 2 casas, gramatura com 1 casa e diâmetro sem casas.
6. **Pelo menos duas funções** com `def` e `return` (ex.: `texto_para_float(texto)` e `linha_da_ficha(rotulo, valor)`).
7. Código organizado em blocos comentados: entrada, limpeza, cálculos, saída.
8. Roda sem erro com `python ficha_bobina.py` e passa no `ruff check` sem avisos.

Extras opcionais

- **Formato brasileiro:** mostre todos os números com vírgula decimal e ponto de milhar (`1.234,56`), reaproveitando a técnica do exercício 12.
- **Bobinas da OP:** dada a produção total da OP em metros, mostre quantas bobinas completas saem e quanto sobra (`//` e `%`).
- **Tempo de produção:** dada a velocidade da máquina, mostre quanto tempo falta e o horário previsto de término (exercício 14).
- **Versão interativa:** troque as variáveis de entrada por `input()`. Lembre que `input()` sempre devolve texto.

Exemplo de execução

Com os dados de exemplo da solução de referência (que implementa também os extras de formato brasileiro e de bobinas da OP):

```
PS C:\curso-python> python 01-fundamentos\desafio\solucao_referencia.py
============================================
         FICHA DE MEDIÇÃO DA BOBINA
============================================
OP                                    OP48213
Linha                                       C
Cliente                    EMBALAGENS SUL LTDA
--------------------------------------------
```

```
Medições (µm):
            24,8 | 25,3 | 25,1 | 24,6 | 25,4
Espessura média (µm)                    25,04
Nominal (µm)                         25,0 ± 8%
Faixa aceita (µm)                23,00 a 27,00
Desvio do nominal                       +0,16%
-----------------------------------------
Gramatura (g/m²)                          47,1
Peso estimado (kg)                      180,86
Diâmetro estimado (mm)                     444
Bobinas na OP                 7 + sobra 3500 m
=========================================
Veredito automático: fica para o módulo 02 (if/else).
```

Sua ficha não precisa ser idêntica. O importante é estar correta, legível e alinhada.

Critérios de aceite

- [ ] O script roda do começo ao fim sem erro e sem pedir nada ao usuário (exceto na versão interativa).
- [ ] A OP sai no formato `OP` + número, sem espaços, hífens ou pontos, qualquer que seja a forma digitada.
- [ ] As cinco medições com vírgula são convertidas corretamente (confira a média na calculadora).
- [ ] A faixa de 25 µm ± 8% sai como 23,00 a 27,00 (ou 23.00 a 27.00).
- [ ] A gramatura de 0,4710 g em 10 cm × 10 cm sai como 47,1 g/m².
- [ ] Os valores estão alinhados à direita numa mesma coluna.
- [ ] Nenhum arredondamento no meio das contas: só na exibição.
- [ ] Existem pelo menos duas funções com `return`, e elas são usadas.
- [ ] Os nomes de variáveis dizem o que guardam e incluem a unidade quando há dúvida.
- [ ] `ruff check` e `ruff format --check` não reclamam do arquivo.

Dicas de como começar

1. **Comece pela entrada e pela saída**, sem cálculo nenhum: crie as variáveis e imprima a ficha com valores fixos (`"???"`). Rode. Ver o "esqueleto" da ficha na tela dá ânimo e mostra o que falta.
2. **Uma conta por vez.** Faça a limpeza da OP e rode. Depois a conversão das medições e rode. Depois a média e rode. Nunca escreva 30 linhas sem rodar.
3. **Confira com um caso que você sabe de cabeça.** Troque as cinco medições por `"25,0"`: a média tem que dar 25 e o desvio, 0%.
4. **Teste fórmulas no REPL** antes de colocar no arquivo, como na seção 15 da aula.

5. Travou no alinhamento? Volte à seção 11 (f-strings) e teste `f"[{'texto':>18}]"` no REPL com larguras diferentes.
6. Só depois de tudo funcionando, pense nos extras.

A solução de referência está em `solucao_referencia.py`. Tente o seu antes de abrir.

Quiz do módulo 01 — Fundamentos

Responda sem rodar o código. Anote suas respostas e só depois confira o gabarito no final (e rode os trechos no REPL para ver com seus olhos).

---

1. O que este código imprime?

```
print("A", "B", sep="", end="-")
print("C")
```

a) `A B-C` b) `AB-C` c) `AB` numa linha e `-C` na outra d) `A B C`

2. Qual é o tipo do resultado de `7 / 7`?

a) `int` b) `float` c) `str` d) `bool`

3. O que este código imprime?

```
print(17 // 5, 17 % 5)
```

4. Qual expressão calcula corretamente a média de `a`, `b` e `c`?

a) `a + b + c / 3` b) `(a + b + c) / 3` c) `a + b + (c / 3)` d) `(a + b) + c / 3`

5. O que este código imprime?

```
x = 10
x += 5
x *= 2
print(x)
```

6. Onde está o erro? Qual mensagem aparece e como corrigir?

```
quantidade = 8
print("Bobinas: " + quantidade)
```

7. O que `print("25" * 2)` imprime?

a) `50` b) `2525` c) `25 25` d) dá erro

8. Qual destas conversões dá `ValueError`?

a) `int("42")` b) `float("25.4")` c) `int("25.4")` d) `float("7")`

9. O que este código imprime?

```
codigo = "  lote-07 "
codigo.strip().upper()
print(codigo)
```

a) `LOTE-07` b) `lote-07` c) o texto original, sem nenhuma mudança (com os espaços e em minúsculas) d) dá erro

10. O que este código imprime?

```
print(f"{1234.5:,.2f} | {0.075:.1%} | {5:03d}")
```

11. O que `round(2.5)` e `round(3.5)` devolvem, nessa ordem?

a) `3` e `4` b) `2` e `4` c) `2` e `3` d) `3` e `3`

12. Qual é o jeito certo de verificar se `0.1 + 0.2` é "igual" a `0.3`?

a) imprimir `0.1 + 0.2` e conferir no olho b) `math.isclose(0.1 + 0.2, 0.3)` c) `int(0.1 + 0.2)` d) `str(0.1 + 0.2)`

13. A expedição tem 43 bobinas e cabem 6 por caixa. Qual expressão dá o número de caixas necessárias?

a) `43 // 6` b) `round(43 / 6)` c) `math.ceil(43 / 6)` d) `43 % 6`

14. O que acontece ao rodar este código?

```
def area(largura, comprimento):
    print(largura * comprimento)


total = area(2, 3) + 1
```

15. Qual nome de variável segue as convenções do Python?

a) `PesoLiquido` b) `peso-liquido` c) `peso_liquido_kg` d) `pesoLíquido`

Gabarito

1. **b)** **`AB-C`.** `sep=""` junta os valores sem espaço, e `end="-"` troca a quebra de linha por um hífen. Por isso o `C` do segundo `print` fica colado na mesma linha.

2. **b)** **`float`.** A divisão `/` sempre devolve float, mesmo quando a conta é exata: `7 / 7` dá `1.0`. Para um inteiro, use `//`.

3. **`3 2`.** O 5 cabe 3 vezes inteiro no 17 (`17 // 5 = 3`) e sobram 2 (`17 % 5 = 2`). Prova real: 3 × 5 + 2 = 17.

4. **b)** **`(a + b + c) / 3`.** Sem os parênteses, a divisão é feita antes da soma, e só o `c` é dividido por 3. As opções a, c e d calculam a mesma coisa errada.

5. **`30`.** `x` começa em 10, `x += 5` faz virar 15 e `x *= 2` faz virar 30.

6. **Linha 2:** **`TypeError: can only concatenate str (not "int") to str`.** O `+` não junta texto com número. Corrija com `"Bobinas: " + str(quantidade)` ou, melhor, `f"Bobinas: {quantidade}"`.

7. **b)** **`2525`.** `"25"` é texto, e texto multiplicado por inteiro é repetido. Se fosse o número `25 * 2`, daria `50`.

8. **c)** **`int("25.4")`.** `int()` não aceita texto com casas decimais: dá `ValueError: invalid literal for int() with base 10: '25.4'`. Use `float("25.4")`.

9. **c) o texto original, sem mudança.** Strings são imutáveis: `strip()` e `upper()` criaram um texto novo (`"LOTE-07"`), mas ninguém guardou o resultado. O certo seria `codigo = codigo.strip().upper()`.

10. **`1,234.50 | 7.5% | 005`.** `:,.2f` põe separador de milhar (padrão americano) e 2 casas; `:.1%` multiplica por 100 e mostra 1 casa; `:03d` completa com zeros até 3 dígitos.

11. **b)** **`2` e `4`.** No empate (.5), o `round` vai para o número **par** mais próximo (arredondamento do banqueiro).

12. **b)** **`math.isclose(0.1 + 0.2, 0.3)`.** `0.1 + 0.2` dá `0.30000000000000004` por causa da representação binária do float. `math.isclose` considera "praticamente iguais" e devolve `True`. Olhar no olho (a) não serve para um programa; c e d nem comparam nada.

13. **c)** **`math.ceil(43 / 6)`.** `43 / 6` dá 7,17. Sete caixas não bastam (sobra 1 bobina), então é preciso arredondar **sempre para cima**: 8. `43 // 6` e `round(43 / 6)` dão 7, e `43 % 6` dá 1, que é a sobra, não o total de caixas.

14. **Imprime `6` e depois dá `TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'`.** A função imprime o resultado, mas não tem `return`, então devolve `None`. E `None + 1` não existe. Se o resultado vai ser usado em outra conta, a função precisa de `return largura * comprimento`.

15. **c) `peso_liquido_kg`.** snake_case, sem acento e com a unidade no nome. `PesoLiquido` é o estilo de nomes de classe (módulo 08); `peso-liquido` é lido como `peso - liquido` (uma subtração); `pesoLíquido` mistura estilo de outra linguagem com acento.

Módulo 02 — Decisões e repetições

> **Tempo estimado:** 10 a 12 horas · **Pré-requisitos:** módulos 00 e 01 · **Nível:** iniciante

O que você vai aprender

Ao final deste módulo, você vai conseguir:

- comparar valores com `==`, `!=`, `<`, `>`, `<=` e `>=` e prever o resultado, inclusive com textos e floats;
- combinar condições com `and`, `or` e `not`, sem cair no erro `linha == "A" or "B"`;
- escrever `if` / `elif` / `else` na ordem certa para classificar um valor em faixas;
- usar a expressão condicional (`"OK" if dentro else "FORA"`);
- dizer quais valores são "falsos" (`0`, `0.0`, `""`, `None`, lista vazia) e quando usar `is None`;
- escrever um `match-case` com `|` e `_`, e saber quando ele é melhor que uma fila de `elif`;
- escrever um `while` com saída garantida, e reconhecer e interromper um loop infinito;
- percorrer uma string, um `range` e uma lista com `for`, sem errar o "um a mais, um a menos";
- usar `break` para parar uma busca e `continue` para pular dados ruins;
- implementar à mão os padrões contador, acumulador, busca e máximo/mínimo;
- escrever laços aninhados e calcular quantas vezes o laço de dentro roda;
- validar um dado digitado antes de convertê-lo, sem deixar o programa quebrar;
- fazer um teste de mesa para descobrir por que um laço não faz o que você queria.

Por que isso importa

Até aqui, seus programas eram uma receita: linha 1, linha 2, linha 3, fim. A partir de agora eles **decidem** e **repetem**. É isso que faz um programa parecer inteligente.

Repare no projeto final, o validador de certificados. Ele é praticamente feito disto:

- **para cada** linha da planilha (repetição),
- **para cada** ensaio daquela amostra (repetição dentro de repetição),
- **se** o valor está fora da tolerância (decisão), marque como reprovado;

- **se** a OP tem especificação própria, use-a; **senão**, use o padrão da linha (decisão).

No mercado, lógica de programação é o primeiro filtro. Entrevistas técnicas para júnior quase sempre começam com "percorra esta lista e conte/encontre/some alguma coisa". Este módulo treina exatamente isso, à mão, sem atalhos. Atalhos como `sum()` e `max()` vêm no módulo 03, e você vai entendê-los muito melhor por ter feito o trabalho antes.

Como estudar este módulo

O método é o mesmo do módulo 01 (PRIMM):

- 🔮 **Preveja antes de rodar:** escreva sua previsão **antes** de abrir a resposta.
- 🔧 **Modifique:** altere o exemplo e observe.
- **Exemplo resolvido:** leia o raciocínio passo a passo antes de tentar os parecidos.
- **Tente você:** mini exercício com resposta escondida.

Com laços, há uma técnica extra que vale ouro: o **teste de mesa** (seção 15). Quando um laço "não faz sentido", pegue papel e anote o valor de cada variável a cada volta.

1. Comparações: perguntas de sim ou não

Uma comparação é uma pergunta que o Python responde com `True` ou `False`, o tipo `bool` que você viu no módulo 01.

Operador	Pergunta	Exemplo	Resultado
`==`	é igual?	`25 == 25.0`	`True`
`!=`	é diferente?	`"B" != "b"`	`True`
`<`	é menor?	`22.4 < 22.5`	`True`
`>`	é maior?	`27.6 > 27.5`	`True`
`<=`	é menor ou igual?	`22.5 <= 22.5`	`True`
`>=`	é maior ou igual?	`7 >= 7.5`	`False`

```
>>> espessura = 25.4
>>> espessura > 27.0
False
>>> espessura != 25
```

```
True
>>> resultado = espessura >= 23.0
>>> resultado
True
>>> type(resultado)
<class 'bool'>
```

O resultado de uma comparação é um valor como outro qualquer: dá para guardar numa variável, imprimir ou devolver com `return`.

> ⚠️ **Erro comum:** confundir `=` com `==`. Um `=` **guarda** um valor (atribuição). Dois `==` **perguntam** se são iguais. Escrever `if valor = 27:` dá `SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?`. O próprio Python sugere a correção.

Comparação encadeada: igual à matemática

Para saber se um valor está numa faixa, o Python aceita a forma que você escreveria no caderno:

```
>>> minimo = 23.0
>>> maximo = 27.0
>>> minimo <= 25.4 <= maximo
True
>>> minimo <= 27.0 <= maximo
True
>>> minimo <= 27.1 <= maximo
False
```

`23.0 <= valor <= 27.0` significa "valor maior ou igual a 23 **e** menor ou igual a 27". Repare que os limites contam como dentro, porque usamos `<=`. Na especificação de um cliente, isso faz diferença: confirme sempre se os limites são inclusivos.

Comparando textos

Textos são comparados caractere por caractere, diferenciando maiúsculas e minúsculas:

```
>>> "COF" == "cof"
False
>>> "COF" == "cof".upper()
True
>>> "abacaxi" < "banana"
True
```

🔮 **Preveja antes de rodar:** o que cada linha mostra?

```
print("10" < "9")
print(10 < 9)
print("25" == 25)
```

▼ **Resposta**

```
True
False
False
```

Na primeira linha os dois são **textos**: o Python compara o primeiro caractere de cada um, `"1"` com `"9"`, e `"1"` vem antes. Por isso `"10" < "9"` é `True`. É o mesmo motivo de alguns sistemas listarem `lote10` antes de `lote9`. Na segunda, números de verdade. Na terceira, texto nunca é igual a número, então dá `False` sem erro. Moral: converta antes de comparar (`float(texto)`).

E comparar texto com número usando `<` ou `>` nem funciona:

```
>>> "25" < 30
Traceback (most recent call last):
  ...
TypeError: '<' not supported between instances of 'str' and 'int'
```

(Como no módulo 01, as linhas do meio dos tracebacks estão resumidas com `...`.)

Comparando floats

Lembra do `0.1 + 0.2`? Ele volta aqui:

```
>>> 0.1 + 0.2 == 0.3
False
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)
True
```

Para conferir se dois floats são "iguais", use `math.isclose`. Para faixas de tolerância (`<=` e `>=`), o problema quase nunca aparece, porque um erro na 17ª casa não muda o resultado de uma medição real.

Tente você: escreva uma comparação que diga se `peso_kg` está entre 980 e 1020, inclusive.

▼ **Resposta**

```
>>> peso_kg = 1003.5
>>> 980 <= peso_kg <= 1020
True
```

2. Combinando condições: and, or e not

Operador	Resultado	Exemplo do laboratório
`a and b`	`True` só se **os dois** forem verdadeiros	espessura OK **e** COF OK
`a or b`	`True` se **pelo menos um** for verdadeiro	linha A **ou** linha B
`not a`	inverte: `True` vira `False` e vice-versa	**não** liberado

A tabela completa (a "tabela-verdade"):

```
  a      b     | a and b | a or b
 True   True   |  True   |  True
 True   False  |  False  |  True
 False  True   |  False  |  True
 False  False  |  False  |  False
```

```
>>> espessura_ok = True
>>> cof_ok = False
>>> espessura_ok and cof_ok
False
>>> espessura_ok or cof_ok
True
>>> not cof_ok
True
>>> linha = "B"
>>> linha == "A" or linha == "B"
True
```

Precedência: `not` é resolvido primeiro, depois `and`, por último `or`. Igual ao módulo 01: na dúvida, parênteses. `(a and not b) or c` é muito mais legível que `a and not b or c`, mesmo que o resultado seja o mesmo.

Curto-circuito: o Python para de olhar quando já sabe

Em `a and b`, se `a` é falso, o resultado já é falso, e o Python **nem avalia** `b`. Em `a or b`, se `a` é verdadeiro, ele nem olha `b`. Isso é útil para proteger contas perigosas:

```
>>> pessoas = 0
>>> pessoas != 0 and 150 / pessoas > 30
False
```

Como `pessoas != 0` é falso, a divisão por zero nunca acontece. Se você inverter a ordem (`150 / pessoas > 30 and pessoas != 0`), vem `ZeroDivisionError`.

🔮 **Preveja antes de rodar:** a linha é `"C"`. O que este código imprime?

```
linha = "C"
if linha == "A" or "B":
    print("Tolerância de 5%")
else:
    print("Outra tolerância")
```

▼ **Resposta**

Imprime `Tolerância de 5%`, que está **errado**. O Python não lê como "linha é A ou B". Ele lê `(linha == "A") or ("B")`. A primeira parte é `False`, mas a segunda é o texto `"B"`, que conta como verdadeiro (seção 5). Resultado: a condição é **sempre** verdadeira. O certo é repetir a comparação: `linha == "A" or linha == "B"`. Esse erro é tão comum que vale escrever num post-it.

💡 **Dica:** cuidado com o atalho `linha in "AB"`. Ele aceita `"A"` e `"B"`, mas também `""` (texto vazio) e `"AB"`, porque todos estão "dentro" de `"AB"`. No módulo 03 você vai ver o jeito certo com tuplas: `linha in ("A", "B")`.

3. if, elif e else: o programa decide

Pense num procedimento do laboratório escrito como fluxograma:

```
          [COF medido]
               |
      COF < 0,20 ? ──sim──> "baixo: filme escorrega na envasadora"
               | não
```

```
        COF <= 0,40 ? —sim——> "normal"
                | não
                └——————————> "alto: filme trava na máquina"
```

Em Python:

```python
cof = 0.47

if cof < 0.20:
    print("COF baixo: filme escorrega na envasadora")
elif cof <= 0.40:
    print("COF normal")
else:
    print("COF alto: filme trava na máquina")

print("Fim da avaliação")
```

Saída:

```
COF alto: filme trava na máquina
Fim da avaliação
```

As regras:

- `if` testa a primeira condição. `elif` ("senão, se") testa a próxima. `else` ("senão") pega tudo que sobrou.
- Cada linha de condição termina com **dois-pontos** `:`.
- O bloco de cada caso é **recuado** (4 espaços). A indentação é que diz ao Python o que pertence a cada caso. O `print("Fim da avaliação")`, sem recuo, roda sempre.
- O Python testa **de cima para baixo** e executa **só o primeiro** bloco verdadeiro. Os outros são ignorados, mesmo que também fossem verdadeiros.
- `elif` e `else` são opcionais. Pode haver quantos `elif` você quiser e no máximo um `else`.

Exemplo resolvido: a ordem dos elif importa

Problema: classificar a umidade relativa da sala de ensaios. Até 40%: "seca". Até 60%: "ideal". Acima de 60%: "úmida" (atrapalha o ensaio de COF).

Primeira tentativa (errada):

```python
umidade = 35
if umidade <= 60:
    print("ideal")
```

```
elif umidade <= 40:
    print("seca")
else:
    print("úmida")
```

Saída:

```
ideal
```

35% deveria ser "seca". O que houve? `35 <= 60` é verdadeiro, então o primeiro bloco ganhou e o `elif` nem foi testado.

Raciocínio da correção: quando as faixas são "até X", testo da **menor** para a maior. Cada `elif` só é alcançado se os anteriores falharam, então nele eu já sei que o valor passou dos limites anteriores.

```
umidade = 35
if umidade <= 40:
    print("seca")
elif umidade <= 60:  # aqui já sei que umidade > 40
    print("ideal")
else:  # aqui já sei que umidade > 60
    print("úmida")
```

Saída:

```
seca
```

> 🔧 **Modifique:** teste com `40`, `41`, `60` e `61`. Os valores de fronteira são onde os bugs moram. Depois, reescreva testando da **maior** faixa para a menor (`if umidade > 60:` ...). As duas ordens funcionam, desde que você seja coerente.

if dentro de if

Um bloco pode conter outro `if`. Use quando uma decisão só faz sentido depois de outra:

```
cof = 0.32
liberado_pelo_cliente = True

if cof > 0.30:
    if liberado_pelo_cliente:
        print("COF acima do padrão, mas liberado pelo cliente.")
    else:
```

```
        print("COF acima do padrão: bloquear lote.")
else:
    print("COF dentro do padrão.")
```

Saída:

```
COF acima do padrão, mas liberado pelo cliente.
```

> 💡 **Dica:** mais de dois ou três níveis de `if` aninhado fica difícil de ler. Muitas vezes `and` resolve: `if cof > 0.30 and not liberado_pelo_cliente:`. Outra saída é o **retorno antecipado** dentro de funções (seção 14).

> ⚠️ **Erro comum:** escrever `if aprovado == True:`. Funciona, mas é redundante: `aprovado` já é `True` ou `False`. Escreva `if aprovado:` e `if not aprovado:`.

Tente você: escreva um `if` que imprima `"par"` ou `"ímpar"` para um inteiro `n`.

<details>
<summary>Resposta</summary>

```
n = 7
if n % 2 == 0:
    print("par")
else:
    print("ímpar")
```

O resto da divisão por 2 é 0 para os pares (módulo 01, seção 6).
</details>

4. Expressão condicional: um if numa linha

Quando a decisão serve só para **escolher entre dois valores**, existe uma forma curta:

```
valor = 22.9
status = "OK" if 23.0 <= valor <= 27.0 else "FORA"
print(status)
print(f"Lote {'liberado' if status == 'OK' else 'retido'}")
```

Saída:

```
FORA
Lote retido
```

Leia como em português: "status é `OK` **se** a condição for verdadeira, **senão** `FORA`". A forma geral é `valor_se_verdadeiro if condição else valor_se_falso`.

Use para escolhas simples. Se precisar de `elif` ou de mais de uma ação, volte ao `if` normal: legibilidade vem primeiro.

5. Truthy e falsy: o que conta como verdadeiro

O `if` aceita qualquer valor, não só `True` e `False`. O Python decide assim:

- **falsy** (contam como falso): `False`, `None`, `0`, `0.0`, `""` (texto vazio) e coleções vazias, como a lista vazia `[]`;
- **truthy** (contam como verdadeiro): todo o resto, inclusive `"0"`, `" "` (um espaço) e `-1`.

A função `bool()` mostra como o Python enxerga cada valor:

```
>>> bool(0)
False
>>> bool("")
False
>>> bool(None)
False
>>> bool("0")
True
>>> bool(" ")
True
>>> bool(-1)
True
```

Isso permite testes curtos e muito usados:

```
observacao = ""
if not observacao:
    print("Sem observação: usar o texto padrão da linha.")
```

Saída:

```
Sem observação: usar o texto padrão da linha.
```

None e o operador is

`None` significa "sem valor". Para perguntar se algo é `None`, use `is None` (ou `is not None`), e não `== None`. O `is` pergunta "é exatamente este objeto?", e só existe um `None`, então é a forma mais clara e segura:

```
>>> especificacao = None
>>> especificacao is None
True
>>> especificacao is not None
False
```

🔮 **Preveja antes de rodar:** a balança mediu exatamente zero de variação de peso (a amostra não absorveu umidade nenhuma, ótimo resultado). O que é impresso?

```
variacao = 0.0
if variacao:
    print("Variação registrada:", variacao)
else:
    print("Sem medição!")
```

▼ **Resposta**

Imprime `Sem medição!`, que está **errado**. `0.0` é falsy, então o `if variacao:` trata uma medição válida de zero como se não existisse. Quando "zero" é um valor legítimo e "ausente" é `None`, teste explicitamente: `if variacao is not None:`. Essa distinção vai importar muito ao ler planilhas (célula vazia vira `None`; célula com 0 vira `0`).

6. match-case: escolhendo entre vários casos

Quando você compara **a mesma variável** com uma lista de valores fixos, uma fila de `elif` fica repetitiva. O Python 3.10 trouxe o `match`:

```
ensaio = " Gramatura "

match ensaio.strip().lower():
    case "espessura":
        unidade = "µm"
    case "gramatura":
```

```
        unidade = "g/m²"
    case "tração" | "tracao":
        unidade = "MPa"
    case "cof":
        unidade = "(adimensional)"
    case _:
        unidade = None

print("Unidade do ensaio:", unidade)
```

Saída:

```
Unidade do ensaio: g/m²
```

- `match valor:` avalia o valor uma vez; cada `case` é um valor possível.
- `|` junta vários valores no mesmo caso (`"tração" | "tracao"` aceita com e sem acento).
- `case _:` é o "qualquer outra coisa", como o `else`. Se nenhum caso casar e não houver `_`, nada acontece.
- Só o primeiro caso que casar é executado (não existe o "vazamento" para o próximo, como em outras linguagens).

A mesma lógica com `if`:

```
codigo = ensaio.strip().lower()
if codigo == "espessura":
    unidade = "µm"
elif codigo == "gramatura":
    unidade = "g/m²"
elif codigo == "tração" or codigo == "tracao":
    unidade = "MPa"
# ...e assim por diante
```

Quando usar cada um? `match` para "esta variável é qual destes valores fixos?" (menu de comandos, código de linha, tipo de ensaio). `if/elif` para faixas e condições variadas (`valor < minimo`, `cof > 0.30 and not liberado`).

> ⚠️ **Erro comum:** usar `match` no Python 3.9 ou anterior: dá `SyntaxError`. O curso pede Python 3.12+ (módulo 00), então você está coberto. Mas lembre disso se for rodar seu script num servidor antigo.

O match-case tem recursos bem mais poderosos (casar estruturas, capturar partes), que aparecem quando você conhecer listas e dicionários. Por ora, esta introdução cobre 90% do uso do dia a dia.

7. while: repetir enquanto for verdade

O `while` repete um bloco **enquanto** a condição for verdadeira. Pense na expedição: "**enquanto** houver bobina no palete, pegue a próxima e confira a etiqueta". Você não precisa saber de antemão quantas são.

Todo `while` saudável tem três partes:

```
contador = 3                1) estado inicial
while contador > 0:         2) condição: testada ANTES de cada volta
    print("Faltam", contador)
    contador -= 1           3) atualização: aproxima a condição de ficar
falsa
print("Partiu!")
```

```
contador = 3
while contador > 0:
    print("Faltam", contador)
    contador -= 1
print("Partiu!")
```

Saída:

```
Faltam 3
Faltam 2
Faltam 1
Partiu!
```

Exemplo resolvido: quando comprar resina?

Problema: o estoque tem 5.000 kg de resina; a extrusora consome 380 kg por dia. O estoque de segurança é 1.000 kg. Em quantos dias o estoque cai abaixo do mínimo?

Raciocínio: é uma repetição "enquanto o estoque estiver acima ou igual ao mínimo, passe um dia". Preciso de um contador de dias (começa em 0) e de atualizar o estoque a cada volta. Quando o `while` terminar, o contador terá a resposta.

```
estoque_kg = 5000.0
consumo_diario_kg = 380.0
estoque_seguranca_kg = 1000.0
dias = 0
while estoque_kg >= estoque_seguranca_kg:
```

```
    estoque_kg -= consumo_diario_kg
    dias += 1
print(f"Em {dias} dias o estoque cai para {estoque_kg:.0f} kg: hora de
comprar.")
```

Saída:

```
Em 11 dias o estoque cai para 820 kg: hora de comprar.
```

Conferindo: 5.000 − 11 × 380 = 820, abaixo de 1.000. Com 10 dias seriam 1.200, ainda acima. Bate.

> 🔧 **Modifique:** troque `consumo_diario_kg` para `0` e rode. O que acontece? (Use `Ctrl+C` para parar.) Depois, acrescente antes do `while` um `if` que avisa "consumo zero: o estoque nunca acaba" e evita o laço.

Loop infinito

Se a condição nunca ficar falsa, o `while` roda para sempre. O programa parece travado, a ventoinha do notebook acelera. Causas típicas: esquecer a atualização (`contador -= 1`), atualizar na direção errada (`contador += 1`), ou um valor que torna a saída impossível (consumo zero, taxa zero).

Para interromper, aperte `Ctrl+C` no terminal. Com o exemplo do estoque e `consumo_diario_kg = 0.0`, o Python para e mostra:

```
Traceback (most recent call last):
  File "C:\curso-python\teste.py", line 5, in <module>
    while estoque_kg >= estoque_seguranca_kg:
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt
```

`KeyboardInterrupt` não é bug do Python: é você interrompendo. A linha mostrada é onde o programa estava naquele instante, uma boa pista de qual laço não terminava.

> 💡 **Dica:** antes de escrever um `while`, responda: "o que, dentro do laço, faz a condição ficar falsa um dia?". Se não souber responder, há risco de loop infinito.

while True com break: repetir até acontecer algo

Às vezes a condição de parada fica mais natural no meio do laço. O padrão é `while True:` (repetir sempre) com `break` (seção 10) para sair:

```
while True:
    texto = input("Peso da bobina (ou 'fim'): ")
    if texto == "fim":
        break
    print("Registrado:", texto)
print("Encerrado.")
```

Uma execução:

```
Peso da bobina (ou 'fim'): 125,5
Registrado: 125,5
Peso da bobina (ou 'fim'): 130
Registrado: 130
Peso da bobina (ou 'fim'): fim
Encerrado.
```

A palavra `"fim"` é uma **sentinela**: um valor especial que sinaliza "acabou". É o padrão de todo programa de terminal que lê dados até o usuário cansar, como o desafio deste módulo.

O exemplo `exemplos/04_while.py` tem ainda um "jogo de adivinhação" em que o computador acha um número de 1 a 100 em poucas tentativas, cortando a faixa pela metade a cada palpite (busca binária). Rode e veja quantas tentativas ele precisa.

8. for e range: repetir para cada item

O `for` percorre uma sequência, **um item por volta**. É o laço mais usado em Python. Com uma string, cada volta recebe um caractere:

```
for letra in "COF":
    print("Letra:", letra)
```

Saída:

```
Letra: C
Letra: O
Letra: F
```

A variável (`letra`) é criada pelo próprio `for` e recebe o próximo item a cada volta. Escolha um nome que diga o que ela é: `for letra in texto`, `for medicao in medicoes`.

range: uma sequência de números

`range` gera números inteiros sob demanda. Ele tem três formas:

Forma	Gera	Exemplo
`range(fim)`	0, 1, ..., fim-1	`range(3)` → 0, 1, 2
`range(início, fim)`	início, ..., fim-1	`range(1, 4)` → 1, 2, 3
`range(início, fim, passo)`	de passo em passo	`range(20, 61, 10)` → 20, 30, 40, 50, 60

O **fim nunca entra**. É a fonte nº 1 de erros "um a mais, um a menos" (em inglês, *off-by-one*). Para ir de 1 a 10, use `range(1, 11)`.

```
for bobina in range(1, 4):
    print(f"Bobina {bobina} de 3")
print("Contagem regressiva:", end=" ")
for n in range(5, 0, -1):
    print(n, end=" ")
print()
```

Saída:

```
Bobina 1 de 3
Bobina 2 de 3
Bobina 3 de 3
Contagem regressiva: 5 4 3 2 1
```

🔮 **Preveja antes de rodar:** quais números são impressos?

```
for i in range(2, 11, 4):
    print(i, end=" ")
```

▼ **Resposta**

`2 6 10`. Começa em 2 e soma 4 a cada volta: 2, 6, 10. O próximo seria 14, que passa do fim (11). Repare que o 10 entrou porque é **menor** que 11; com `range(2, 10, 4)` ele não entraria.

> ⚠️ **Erro comum:** `range` só aceita inteiros. `range(2.5)` dá `TypeError: 'float' object cannot be interpreted as an integer`. E `for i in 10:` dá `TypeError: 'int' object is not iterable`: um número sozinho não é uma sequência; o certo é `for i in range(10):`.

for ou while?

- **for**: quando você vai percorrer algo que já existe (os caracteres de um texto, os números de 1 a N, as medições de uma lista). O fim é garantido.
- **while**: quando você não sabe quantas voltas serão (até o estoque acabar, até o usuário digitar "fim", até o valor dobrar). Exige cuidado com a saída.

Na dúvida, prefira `for`: ele não tem como virar loop infinito.

9. Percorrendo uma lista recebida

Listas são o assunto do módulo 03. Para este módulo, bastam três linhas:

- `[24.8, 25.3, 25.1]` cria uma lista: vários valores, em ordem, entre colchetes;
- `for valor in valores:` percorre a lista, um valor por volta, do primeiro ao último;
- `len(valores)` diz quantos itens ela tem.

Nos exercícios, a lista chega pronta como **parâmetro da função**: você só percorre.

```
pesos = [125.5, 130.2, 128.9]
total = 0.0
for peso in pesos:
    total += peso
print(f"{len(pesos)} bobinas, total {total:.1f} kg")
```

Saída:

```
3 bobinas, total 384.6 kg
```

Precisa saber a **posição** de cada item (1ª, 2ª, 3ª)? Use um contador que aumenta a cada volta:

```
medicoes = [24.8, 25.3, 25.1]
posicao = 0
for medicao in medicoes:
    posicao += 1
    print(f"Medição {posicao}: {medicao} µm")
```

Saída:

```
Medição 1: 24.8 µm
Medição 2: 25.3 µm
Medição 3: 25.1 µm
```

(No módulo 03 você vai conhecer o `enumerate`, que faz isso sozinho.)

10. break e continue: controlando o laço por dentro

- `break` **encerra** o laço imediatamente. O programa segue na primeira linha depois do laço.
- `continue` **pula o resto desta volta** e vai direto para a próxima.

```
medicoes = [25.1, 0.0, 24.8, 31.2, 25.0, -1.0, 26.9]

print("Medições válidas:", end=" ")
for valor in medicoes:
    if valor <= 0:
        continue
    print(valor, end=" ")
print()

posicao = 0
for valor in medicoes:
    posicao += 1
    if valor > 27:
        print(f"Primeira acima de 27 µm: {valor} (posição {posicao}).")
        break
print("Fim da inspeção.")
```

Saída:

```
Medições válidas: 25.1 24.8 31.2 25.0 26.9
Primeira acima de 27 µm: 31.2 (posição 4).
Fim da inspeção.
```

No primeiro laço, `continue` pula as falhas de leitura (zero e negativo) sem imprimir. No segundo, `break` encerra a busca assim que acha o primeiro valor acima de 27: não faz sentido continuar olhando.

> 🔮 **Preveja antes de rodar:** o que é impresso?

```python
codigo = ""
for caractere in "OP-12.3":
    if caractere == "-" or caractere == ".":
        continue
    if caractere == "3":
        break
    codigo += caractere
print(codigo)
```

▼ **Resposta**

`OP12`. O `-` e o `.` são pulados pelo `continue`. As letras `O`, `P`, `1` e `2` são acumuladas. Quando chega o `3`, o `break` encerra o laço **antes** de acumular o `3`.

💡 **Dica:** dentro de uma função, `return` também encerra o laço (e a função inteira). Em buscas, `return posicao` no momento em que acha é muitas vezes mais claro do que `break` mais uma variável auxiliar.

11. Contadores e acumuladores

Dois padrões aparecem em quase todo laço:

- **contador**: começa em `0` e soma `1` quando algo acontece (`quantidade += 1`);
- **acumulador**: começa num valor neutro e incorpora cada item (`soma += valor`).

O valor inicial do acumulador é o "neutro" da operação: `0` para somar, `1` para multiplicar, `""` (texto vazio) para juntar textos.

Exemplo resolvido: média das bobinas acima de 100 kg

Problema: da lista de pesos, quero a média só das bobinas com mais de 100 kg (as menores são pontas de bobina e não entram).

Raciocínio:

1. Média = soma ÷ quantidade. Preciso de um acumulador (`soma`) e de um contador (`quantidade`).
2. Os dois começam **antes** do laço; senão seriam zerados a cada volta.
3. Dentro do laço: se o peso passa de 100, somo e conto.
4. Depois do laço: se ninguém foi contado, não dá para dividir.

```python
pesos = [125.5, 48.0, 130.2, 128.9, 75.3]
soma = 0.0
quantidade = 0
for peso in pesos:
    if peso > 100:
        soma += peso
        quantidade += 1
if quantidade > 0:
    print(f"Média de {quantidade} bobinas: {soma / quantidade:.2f} kg")
else:
    print("Nenhuma bobina acima de 100 kg.")
```

Saída:

```
Média de 3 bobinas: 128.20 kg
```

> 🔧 **Modifique:** mova as linhas `soma = 0.0` e `quantidade = 0` para **dentro** do `for` (logo abaixo dele) e rode. O resultado muda para quê? Por quê? Esse é um dos bugs mais comuns de iniciante, e o exercício 11 pede que você o reconheça.

> ⚠️ **Erro comum:** calcular a média de uma lista vazia. Com zero itens, `soma / quantidade` vira `0 / 0` e dá `ZeroDivisionError: division by zero`. Sempre trate o caso "nada para calcular" antes de dividir.

Acumulador de texto também é muito útil para montar relatórios:

```python
relatorio = ""
for linha in "ABC":
    relatorio += f"[Linha {linha}] "
print(relatorio)
```

Saída:

```
[Linha A] [Linha B] [Linha C]
```

12. Padrões clássicos: buscar, contar, maior e menor

Quase todo problema com laços é uma variação de poucos padrões. Reconhecê-los é metade da solução.

Contar com condição

"Quantas medições ficaram fora da faixa?" → contador que só aumenta quando a condição é verdadeira:

```
medicoes = [24.8, 25.3, 22.7, 25.4, 27.6]
fora = 0
for valor in medicoes:
    if valor < 23 or valor > 27:
        fora += 1
print(f"{fora} de {len(medicoes)} fora ({fora / len(medicoes):.0%})")
```

Saída:

```
2 de 5 fora (40%)
```

Buscar

"Existe alguma bobina acima de 140 kg? Em que posição?" → percorrer até achar e parar. Antes do laço, um valor que significa "não achei":

```
pesos = [125.5, 130.2, 142.8, 128.9, 151.0]
achou_em = 0  # 0 = não achou
posicao = 0
for peso in pesos:
    posicao += 1
    if peso > 140:
        achou_em = posicao
        break
print("Primeira acima de 140 kg na posição:", achou_em)
```

Saída:

```
Primeira acima de 140 kg na posição: 3
```

Sem o `break`, o laço continuaria e `achou_em` terminaria com a posição da **última** bobina acima de 140 (a 5ª). Às vezes é isso que se quer; leia o enunciado com atenção.

Maior e menor

"Qual a bobina mais pesada?" → guardar o "campeão até agora" e trocar sempre que aparecer alguém melhor. O detalhe é o valor inicial: comece com `None` ("ainda não vi ninguém"):

```python
pesos = [125.5, 130.2, 142.8, 128.9]
mais_pesada = None
for peso in pesos:
    if mais_pesada is None or peso > mais_pesada:
        mais_pesada = peso
print("Mais pesada:", mais_pesada, "kg")
```

Saída:

```
Mais pesada: 142.8 kg
```

Na primeira volta, `mais_pesada is None` é verdadeiro e o primeiro peso vira o campeão. O `or` faz curto-circuito: o Python nem tenta `peso > None`, que daria `TypeError`.

🔮 **Preveja antes de rodar:** e se o "campeão" começasse em `0`?

```python
temperaturas = [-18.5, -21.0, -19.2]  # câmara fria do ensaio de impacto
maior = 0
for t in temperaturas:
    if t > maior:
        maior = t
print(maior)
```

<details>
<summary>Resposta</summary>

Imprime `0`, um valor que **nem está na lista**. Como todas as temperaturas são negativas, nenhuma é maior que 0, e o valor inicial nunca é trocado. Começar com `None` (ou com o primeiro item da lista, o que você vai aprender no módulo 03) evita esse bug.
</details>

Tente você: adapte o código da bobina mais pesada para achar a **mais leve**.

<details>
<summary>Resposta</summary>

```python
pesos = [125.5, 130.2, 142.8, 128.9]
mais_leve = None
for peso in pesos:
    if mais_leve is None or peso < mais_leve:
        mais_leve = peso
print("Mais leve:", mais_leve, "kg")
```
</details>

Só mudam o nome e o sinal: `<` no lugar de `>`.

13. Laços aninhados

Um laço dentro de outro: para **cada** volta do laço de fora, o de dentro roda **inteiro**. Pense num plano de amostragem: para cada linha de produção, coletar em cada turno.

```
coletas = 0
for linha in "ABCD":
    for turno in range(1, 4):
        coletas += 1
        print(f"{linha}{turno}", end=" ")
    print()  # quebra a linha depois dos 3 turnos
print("Total de coletas:", coletas)
```

Saída:

```
A1 A2 A3
B1 B2 B3
C1 C2 C3
D1 D2 D3
Total de coletas: 12
```

O total de voltas do laço de dentro é **(voltas do laço de fora) × (voltas do laço de dentro)**: 4 × 3 = 12. Com 4 linhas × 3 turnos × 20 medições, seriam 240 voltas no laço mais interno. Laços aninhados multiplicam o trabalho: com dados grandes, isso começa a pesar.

🔮 **Preveja antes de rodar:** quantas vezes `"*"` é impresso, e em que formato?

```
for i in range(1, 4):
    for j in range(i):
        print("*", end="")
    print()
```

▼ **Resposta**

Seis asteriscos, em forma de escada:

```
*
**
***
```

O laço de dentro depende do de fora: na volta `i = 1`, `range(1)` roda 1 vez; com `i = 2`, 2 vezes; com `i = 3`, 3 vezes. Total: 1 + 2 + 3 = 6.

Um clássico: a tabuada, montando cada linha com um acumulador de texto (`exemplos/08_lacos_aninhados.py`):

```python
for n in range(1, 4):
    linha = ""
    for i in range(1, 5):
        linha += f"{n}x{i}={n * i:<4}"
    print(linha)
```

Saída:

```
1x1=1   1x2=2   1x3=3   1x4=4
2x1=2   2x2=4   2x3=6   2x4=8
3x1=3   3x2=6   3x3=9   3x4=12
```

> 🔧 **Modifique:** faça a tabuada ir até 10 nos dois laços. Quantas contas o programa faz?

14. Validação de dados: não confie na entrada

Dado digitado, lido de planilha ou de e-mail **vai** vir errado um dia: vazio, com letra, negativo, absurdo. Um programa profissional confere antes de usar. Três ideias:

1. Verifique o formato antes de converter. Sem `try/except` (que é do módulo 06), você pode olhar caractere por caractere. Exemplo: aceitar só inteiros positivos para a quantidade de bobinas.

```python
def eh_inteiro_positivo(texto: str) -> bool:
    """True se o texto (sem espaços nas pontas) tem só dígitos e não é zero."""
    limpo = texto.strip()
    if not limpo:  # texto vazio é falsy
        return False
    for caractere in limpo:
        if caractere not in "0123456789":
            return False  # basta UM caractere ruim para reprovar
```

```
    return int(limpo) > 0


print(eh_inteiro_positivo(" 12 "), eh_inteiro_positivo("7.5"),
eh_inteiro_positivo("0"))
```

Saída:

```
True False False
```

Repare na lógica de "reprovar no primeiro defeito": o laço procura um caractere ruim e devolve `False` assim que acha. Se chegar ao fim sem achar, o texto passou na inspeção.

2. Verifique se o valor faz sentido. Um texto pode ser um número válido e ainda assim absurdo: espessura negativa, palete com 50 bobinas, umidade de 130%.

3. Retorno antecipado (cláusulas de guarda). Dentro de funções, trate os casos ruins primeiro e saia logo. O caminho "feliz" fica no fim, sem aninhamento:

```
def validar_quantidade(texto: str) -> str:
    if not eh_inteiro_positivo(texto):
        return "digite um inteiro maior que zero"
    quantidade = int(texto)  # seguro: já sabemos que só tem dígitos
    if quantidade > 12:
        return "um palete leva no máximo 12 bobinas"
    return f"OK, {quantidade} bobinas"
```

O exemplo `exemplos/09_validacao.py` roda essas funções com várias entradas, boas e ruins.

Laço de validação: perguntar até vir certo

Combinando `while True`, validação e `break`, você obtém o padrão de todo formulário de terminal:

```
while True:
    texto = input("Quantidade de bobinas: ")
    if eh_inteiro_positivo(texto) and int(texto) <= 12:
        quantidade = int(texto)
        break
    print("  Valor inválido: digite um inteiro de 1 a 12.")
print("Palete com", quantidade, "bobinas.")
```

Uma execução:

```
Quantidade de bobinas: doze
  Valor inválido: digite um inteiro de 1 a 12.
Quantidade de bobinas: 15
  Valor inválido: digite um inteiro de 1 a 12.
Quantidade de bobinas: 8
Palete com 8 bobinas.
```

> 💡 **Dica:** escreva mensagens de erro que digam **o que fazer**, não só o que deu errado. "Digite um inteiro de 1 a 12" ajuda o operador; "Erro: entrada inválida" não.

15. Depurando laços: o teste de mesa

Laço com resultado estranho? Antes de mudar código no chute, **simule à mão**. Faça uma tabela com uma coluna por variável e uma linha por volta. Exemplo para este trecho:

```python
total = 0
n = 1
while n < 20:
    total += n
    n *= 3
print(total, n)
```

Saída:

```
13 27
```

O teste de mesa que explica a saída:

```
volta |  n  | n < 20? | total depois
------+-----+---------+-------------
  1   |   1 |   sim   |   0 + 1 = 1
  2   |   3 |   sim   |   1 + 3 = 4
  3   |   9 |   sim   |   4 + 9 = 13
  -   |  27 |   não   |   (sai do laço)
```

Parece trabalho manual demais? É o que os programadores experientes fazem de cabeça. Fazer no papel agora é o que vai permitir fazer de cabeça depois.

Ferramentas que ajudam:

- **print dentro do laço:** `print(f"{n=} {total=}")` mostra cada volta. Apague depois.
- **Python Tutor** (pythontutor.com): cole o código e veja, passo a passo, as variáveis mudando. Excelente para laços.
- **Depurador do VS Code:** clique à esquerda do número da linha para criar um ponto de parada (bolinha vermelha) e rode com F5. O módulo 09 ensina a fundo.

Lendo mensagens de erro deste módulo

Os erros típicos de decisões e laços, como aparecem rodando um script `teste.py`:

Esqueceu os dois-pontos:

```
  File "C:\curso-python\teste.py", line 2
    if valor > 27
                ^
SyntaxError: expected ':'
```

Todo `if`, `elif`, `else`, `while`, `for`, `match`, `case` e `def` termina com `:`. O mesmo erro aparece se você escrever `else if` (que não existe em Python) em vez de `elif`.

Usou `=` no lugar de `==`:

```
  File "C:\curso-python\teste.py", line 2
    if valor = 27:
       ^^^^^^^^^^
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

Esqueceu de recuar o bloco:

```
  File "C:\curso-python\teste.py", line 3
    print("Acima")
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line
2
```

O Python esperava um bloco recuado depois do `if` da linha 2. Coloque 4 espaços antes do `print`.

Comparou texto com número:

```
Traceback (most recent call last):
  File "C:\curso-python\teste.py", line 2, in <module>
    if leitura < 30:
```

```
       ^^^^^^^^^^^^
TypeError: '<' not supported between instances of 'str' and 'int'
```

`leitura` é um texto (veio de `input()` ou de uma planilha). Converta com `float()` antes de comparar.

Tentou percorrer um número:

```
Traceback (most recent call last):
  File "C:\curso-python\teste.py", line 2, in <module>
    for i in 10:
             ^^
TypeError: 'int' object is not iterable
```

"Iterable" é "algo que dá para percorrer". Um número não é; use `range(10)`.

Variável criada só dentro de um `if` que não rodou:

```
Traceback (most recent call last):
  File "C:\curso-python\teste.py", line 4, in <module>
    print(status)
          ^^^^^^
NameError: name 'status' is not defined
```

O código era `if valor > 27: status = "ACIMA"` e o valor era 25: o bloco não rodou e `status` nunca foi criada. Dentro de uma função, o mesmo problema aparece com outro nome. Com uma função `rotulo(valor)` que tem esse mesmo `if` e termina com `return status`, chamada como `print(rotulo(25))`:

```
Traceback (most recent call last):
  File "C:\curso-python\teste.py", line 7, in <module>
    print(rotulo(25))
          ~~~~~~^^^^
  File "C:\curso-python\teste.py", line 4, in rotulo
    return status
           ^^^^^^
UnboundLocalError: cannot access local variable 'status' where it is not
associated with a value
```

Repare que há dois blocos `File`: o erro aconteceu dentro de `rotulo` (linha 4), chamada pela linha 7. O traceback mostra o caminho inteiro, do mais externo para o mais interno.

Correção nos dois casos: garanta um valor em **todos** os caminhos, com um `else` ou com um valor inicial antes do `if`.

Média de lista vazia:

```
Traceback (most recent call last):
  File "C:\curso-python\teste.py", line 7, in <module>
    print(soma / quantidade)
          ~~~~~^~~~~~~~~~~~
ZeroDivisionError: division by zero
```

O laço não rodou nenhuma vez e `quantidade` ficou 0. Teste `if quantidade == 0:` antes de dividir.

Interrompeu com Ctrl+C: `KeyboardInterrupt` (seção 7). Se aconteceu sem querer, procure o `while` cuja condição nunca fica falsa.

Boas práticas

- **Condições legíveis:** prefira `if aprovado:` a `if aprovado == True:`, e `if not texto:` a `if len(texto) == 0:`.
- **Nomeie condições complexas:** `dentro_da_faixa = minimo <= valor <= maximo` e depois `if dentro_da_faixa and cof_ok:`. O código vira frase.
- **Trate os casos ruins primeiro** (retorno antecipado). Menos aninhamento, menos bug.
- **Ordem dos `elif`:** coerente (da menor faixa para a maior, ou o contrário) e teste os valores de fronteira.
- **Acumuladores e contadores antes do laço**, nunca dentro.
- **Todo `while` com saída garantida**, e com proteção para os casos em que a saída seria impossível.
- **`for` sempre que possível;** `while` quando o número de voltas é desconhecido.
- **`is None`** para testar ausência de valor; não confie em truthy/falsy quando `0` é um valor válido.
- **Não use `max()`, `min()` e `sum()` ainda** nos exercícios: o objetivo é dominar o padrão. No dia a dia (e a partir do módulo 03), use-os.

No mundo real

Um caso completo de linha de produção: as leituras do micrômetro chegam (com falhas de leitura registradas como 0 ou negativo), o programa classifica cada uma, diz ao operador o que fazer, aplica a regra "3 seguidas fora = pare a linha" e emite o veredito. É o `exemplos/10_inspecao_serie.py`.

```python
def acao_do_operador(situacao: str) -> str:
    """O que o operador da extrusora deve fazer para cada situação."""
    match situacao:
        case "ABAIXO":
            return "abrir a matriz"
        case "ACIMA":
            return "fechar a matriz"
        case _:
            return ""


linha = "C"
nominal = 25.0
tolerancia = 8.0  # especificação da OP (ou o padrão da linha)
leituras = [24.9, 25.3, 0.0, 26.1, -1.0, 27.4, 25.0, 22.8, 22.5, 22.9,
25.1]

if tolerancia <= 0:
    print("Tolerância inválida na especificação. Nada a fazer.")
else:
    minimo = nominal - nominal * tolerancia / 100
    maximo = nominal + nominal * tolerancia / 100
    print(f"Linha {linha} | {nominal} µm ± {tolerancia:.0f}% ->
{minimo:.2f} a {maximo:.2f}")

    posicao = 0
    validas = 0
    soma = 0.0
    fora = 0
    seguidas = 0
    parar_linha = False
    for valor in leituras:
        posicao += 1
        if valor <= 0:
            print(f"  #{posicao:<2} falha de leitura, ignorada")
            continue
        validas += 1
        soma += valor
        if valor < minimo:
            situacao = "ABAIXO"
        elif valor > maximo:
            situacao = "ACIMA"
        else:
            situacao = "ok"
        if situacao == "ok":
            seguidas = 0
        else:
            fora += 1
            seguidas += 1
```

```
        print(f"  #{posicao:<2} {valor:>6.2f} µm  {situacao:<7}
{acao_do_operador(situacao)}")
        if seguidas == 3:
            parar_linha = True
            print("  >>> 3 leituras seguidas fora: PARE A LINHA e chame o
líder.")
            break

    print("-" * 50)
    if validas == 0:
        print("Nenhuma leitura válida.")
    else:
        media = soma / validas
        if parar_linha:
            veredito = "REPROVADO (processo fora de controle)"
        elif fora > 0:
            veredito = "CONDICIONAL (pontos isolados fora)"
        else:
            veredito = "APROVADO"
        print(f"Válidas: {validas} | média {media:.2f} µm | fora:
{fora}")
        print(f"Veredito: {veredito}")
```

Saída:

```
Linha C | 25.0 µm ± 8% -> 23.00 a 27.00
  #1   24.90 µm  ok
  #2   25.30 µm  ok
  #3  falha de leitura, ignorada
  #4   26.10 µm  ok
  #5  falha de leitura, ignorada
  #6   27.40 µm  ACIMA  fechar a matriz
  #7   25.00 µm  ok
  #8   22.80 µm  ABAIXO abrir a matriz
  #9   22.50 µm  ABAIXO abrir a matriz
  #10  22.90 µm  ABAIXO abrir a matriz
  >>> 3 leituras seguidas fora: PARE A LINHA e chame o líder.
--------------------------------------------------
Válidas: 8 | média 24.61 µm | fora: 4
Veredito: REPROVADO (processo fora de controle)
```

Repare em quantos padrões trabalham juntos: `continue` para falhas de leitura, contador de posição, acumulador de soma, contador de fora, contador de **seguidas** que zera quando aparece uma leitura boa, `break` quando a regra dispara, `if/elif/else` para classificar, `match` para a ação e uma cascata de `if` para o veredito. Nenhum é difícil sozinho. Juntá-los é o que você vai treinar nos exercícios e no desafio.

Resumo

Assunto	Sintaxe essencial
Comparar	`==` `!=` `<` `>` `<=` `>=` · faixa: `minimo <= x <= maximo` · floats: `math.isclose(a, b)`
Combinar	`a and b` · `a or b` · `not a` (precedência: `not` → `and` → `or`)
Decidir	`if cond:` · `elif cond:` · `else:` (só o 1º bloco verdadeiro roda)
Escolher valor	`x = "OK" if cond else "FORA"`
Falsy	`False` `None` `0` `0.0` `""` `[]` · ausência: `x is None` / `x is not None`
match	`match valor:` · `case "C":` · vários valores no mesmo caso com a barra vertical · `case _:`
while	estado inicial → `while cond:` → atualização · `while True:` + `break` · `Ctrl+C` interrompe
for	`for letra in texto:` · `for valor in valores:` · `len(valores)`
range	`range(fim)` · `range(início, fim)` · `range(início, fim, passo)` (fim exclusivo)
Controle	`break` encerra o laço · `continue` pula para a próxima volta
Padrões	contador `n += 1` · acumulador `soma += x` · busca com `break` · maior: `if maior is None or x > maior:`
Aninhados	voltas internas = voltas de fora × voltas de dentro
Validação	conferir formato e faixa antes de usar · retorno antecipado · laço "pergunte até vir certo"

Você domina este módulo se...

- [ ] explico, sem consultar, por que `linha == "A" or "B"` é sempre verdadeiro e escrevo a forma correta;
- [ ] escrevo um `if/elif/else` de faixas e testo os valores de fronteira por conta própria;
- [ ] digo quais destes são falsy sem rodar: `0`, `"0"`, `""`, `" "`, `None`, `0.0`, `[]`, e sei quando `is None` é obrigatório;
- [ ] escrevo de memória um laço que calcula soma, quantidade, média, maior e menor de uma lista, tratando a lista vazia;
- [ ] prevejo quais números um `range(a, b, passo)` gera, inclusive com passo negativo;

- [ ] explico a diferença entre `break`, `continue` e `return` dentro de um laço;
- [ ] olho para um `while` e digo se ele pode virar loop infinito, e como evitar;
- [ ] faço o teste de mesa de um laço de 4 ou 5 voltas e acerto o valor final das variáveis.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_situacao_aluno`	★☆☆	`if/elif/else`, ordem das faixas, validação
02	`ex02_dentro_da_faixa`	★☆☆	comparação encadeada, expressão condicional
03	`ex03_ano_bissexto`	★☆☆	`and`, `or`, `not`, `%`
04	`ex04_tolerancia_da_linha`	★☆☆	`match-case`, vários valores por caso, `case _`, `None`
05	`ex05_contar_vogais`	★☆☆	`for` em string, contador, `in`
06	`ex06_fatorial`	★☆☆	`for` com `range`, acumuladores de soma e produto
07	`ex07_classificar_medicao`	★★☆	limites de tolerância, `elif`, cláusula de guarda
08	`ex08_media_das_medicoes`	★★☆	lista recebida, `continue`, média, `None`
09	`ex09_amplitude`	★★☆	padrão maior/menor com `None`, função chamando função
10	`ex10_anos_para_dobrar`	★★☆	`while`, proteção contra loop infinito
11	`ex11_leitura_de_lacos`	★★☆	**leitura de código:** prever saídas e consertar um bug
12	`ex12_sequencia_fora`	★★★	busca com dois contadores, regra de "N seguidas"
13	`ex13_revisao_tabela_limites`	★★★	**revisão** (módulos 00 a 02): laços aninhados, `match`, f-strings, tolerância
14	`ex14_revisao_resumo_serie`	★★★	**revisão** (módulos 00 a 02): validação de texto, conversão, contadores, máximo/mínimo, formatação

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError`, e rode na pasta `curso-python`:

```
python curso.py testar 02 ex01
```

Para testar o módulo inteiro: `python curso.py testar 02`. Os exercícios 13 e 14 são de **revisão**: misturam este módulo com o módulo 01 (f-strings, conversões, limpeza de texto). Se travar neles, releia as seções 8, 9 e 11 do módulo 01. Depois de passar em cada exercício, compare com `solucoes/`.

Desafio do módulo

Inspetor de série no terminal: um programa interativo que pergunta a linha e o nominal (validando as respostas), lê as medições até o operador digitar `fim`, classifica cada uma na hora, manda parar a linha se 3 seguidas saírem da faixa e emite o veredito com média, mínimo, máximo e porcentagem fora. Tem um modo demonstração que roda sem digitação. Enunciado completo em desafio/README.md.

Quiz

Teste o que ficou: quiz.md (15 perguntas, gabarito comentado no final).

Para ir além

- Tutorial oficial, "Mais ferramentas de controle de fluxo" (`if`, `for`, `range`, `break`, `continue`, `match`): https://docs.python.org/pt-br/3/tutorial/controlflow.html
- Valores verdade e operadores booleanos: https://docs.python.org/pt-br/3/library/stdtypes.html#truth-value-testing
- Comparações: https://docs.python.org/pt-br/3/library/stdtypes.html#comparisons
- PEP 636, tutorial do match-case (em inglês, com muitos exemplos): https://peps.python.org/pep-0636/
- Python Tutor, para visualizar laços passo a passo: https://pythontutor.com/
- Leitura extra: os problemas iniciais do site Exercism (trilha de Python) treinam exatamente os padrões deste módulo: https://exercism.org/tracks/python

Enunciados dos exercícios

Exercício 02.01 — Situação do aluno

Arquivo: `exercicios/ex01_situacao_aluno.py`

```
Nível: ★☆☆

Contexto: O clássico dos clássicos: a escola precisa dizer se o aluno
passou, ficou
de recuperação ou reprovou, a partir da média final (de 0 a 10).

Tarefa: implemente `situacao(media)`, que devolve:
- "Nota inválida" se a média for menor que 0 ou maior que 10;
- "Aprovado" se a média for 7 ou mais;
- "Recuperação" se a média for 5 ou mais (e menor que 7);
- "Reprovado" nos demais casos.

Exemplos:
    >>> situacao(7.0)
    'Aprovado'
    >>> situacao(6.9)
    'Recuperação'
    >>> situacao(11)
    'Nota inválida'

Dica: valide primeiro (nota inválida) e depois teste da maior faixa para
a menor.
A ordem dos elif importa.

Teste com:  python curso.py testar 02 ex01
```

Exercício 02.02 — Dentro da faixa?

Arquivo: `exercicios/ex02_dentro_da_faixa.py`

```
Nível: ★☆☆

Contexto: A pergunta mais feita no laboratório: "este resultado está
dentro da
especificação?". Os limites são inclusivos: um valor exatamente igual ao
mínimo ou ao
máximo está aprovado.

Tarefa:
1. `dentro_da_faixa(valor, minimo, maximo)` devolve True se minimo <=
valor <= maximo,
   e False caso contrário.
2. `rotular(valor, minimo, maximo)` devolve "OK" se o valor estiver
dentro da faixa e
   "FORA" se não estiver. Use uma expressão condicional (x if condição
else y).

Exemplos:
```

```
    >>> dentro_da_faixa(25.4, 23.0, 27.0)
    True
    >>> dentro_da_faixa(27.0, 23.0, 27.0)
    True
    >>> rotular(22.9, 23.0, 27.0)
    'FORA'

Dica: Python aceita comparação encadeada, como na matemática: a <= b <=
c.

Teste com:  python curso.py testar 02 ex02
```

Exercício 02.03 — Ano bissexto

Arquivo: `exercicios/ex03_ano_bissexto.py`

```
Nível: ★☆☆

Contexto: Calendários, validade de laudos, prazos de garantia: todo
sistema que mexe
com datas precisa saber se um ano tem 365 ou 366 dias.

Tarefa:
1. `eh_bissexto(ano)` devolve True se o ano for bissexto. A regra é:
   - divisível por 4 e NÃO divisível por 100; OU
   - divisível por 400.
   (2024 é bissexto; 1900 não é; 2000 é.)
2. `dias_no_ano(ano)` devolve 366 para anos bissextos e 365 para os
demais.

Exemplos:
    >>> eh_bissexto(2024)
    True
    >>> eh_bissexto(1900)
    False
    >>> dias_no_ano(2000)
    366

Dica: "divisível por 4" é ano % 4 == 0. Monte a regra com and, or e not.

Teste com:  python curso.py testar 02 ex03
```

Exercício 02.04 — Tolerância da linha

Arquivo: `exercicios/ex04_tolerancia_da_linha.py`

```
Nível: ★☆☆

Contexto: No projeto final, cada linha de produção tem uma tolerância
padrão de
espessura, usada quando a OP não tem especificação própria:
    linha A: 5%    linha B: 5%    linha C: 8%    linha D: 10%

Tarefa: implemente `tolerancia_da_linha(linha)`, que devolve a tolerância
(float) da
linha informada, ou None se a linha não existir.
- Ignore espaços nas pontas e maiúsculas/minúsculas (" b " vale como
"B").
- Use match-case.

Exemplos:
    >>> tolerancia_da_linha("A")
    5.0
    >>> tolerancia_da_linha(" d ")
    10.0
    >>> print(tolerancia_da_linha("E"))
    None

Dica: case "A" | "B": junta dois casos com a mesma resposta. E case _:
pega "o resto".
O tipo de retorno `float | None` significa "um float ou None".

Teste com:  python curso.py testar 02 ex04
```

Exercício 02.05 — Contar vogais

Arquivo: `exercicios/ex05_contar_vogais.py`

```
Nível: ★☆☆

Contexto: Exercício clássico para praticar o for sobre uma string e o
padrão contador.

Tarefa: implemente `contar_vogais(texto)`, que devolve quantas vogais o
texto tem.
- Conte maiúsculas e minúsculas.
- Conte também as vogais acentuadas: á à â ã é ê í ó ô õ ú.
- "y" não é vogal.

Exemplos:
    >>> contar_vogais("Python")
    1
    >>> contar_vogais("Seleção")
    4
```

```
    >>> contar_vogais("")
    0

Dica: guarde todas as vogais numa string, como "aeiouáàâãéêíóôõú", e use
`in` para
perguntar se cada letra está lá.

Teste com:  python curso.py testar 02 ex05
```

Exercício 02.06 — Fatorial e soma acumulada

Arquivo: `exercicios/ex06_fatorial.py`

```
Nível: ★☆☆

Contexto: Dois clássicos para praticar for, range e acumuladores. O
fatorial aparece em
combinatória e estatística: de quantas formas dá para ordenar n amostras?

Tarefa:
1. `fatorial(n)` devolve n! = 1 x 2 x 3 x ... x n. Por definição, 0! = 1.
   Se n for negativo, devolva None.
2. `soma_de_1_ate(n)` devolve 1 + 2 + ... + n. Se n for menor que 1,
devolva 0.
Use laços for com range (não use fórmulas prontas nem math.factorial).

Exemplos:
    >>> fatorial(5)
    120
    >>> fatorial(0)
    1
    >>> soma_de_1_ate(100)
    5050

Dica: para multiplicar, o acumulador começa em 1 (não em 0!). Por quê?

Teste com:  python curso.py testar 02 ex06
```

Exercício 02.07 — Classificar uma medição

Arquivo: `exercicios/ex07_classificar_medicao.py`

```
Nível: ★★☆

Contexto: Não basta saber se a medição está fora: o operador da extrusora
precisa saber
para que LADO ela saiu, para ajustar a máquina (abrir ou fechar a
```

```
matriz).

Tarefa: implemente `classificar_medicao(valor, nominal, tolerancia_pct)`,
que:
1. devolve "INVÁLIDA" se o valor for menor ou igual a zero (falha do
micrômetro);
2. calcula os limites: nominal ± tolerancia_pct% do nominal;
3. devolve "ABAIXO" se o valor for menor que o mínimo;
4. devolve "ACIMA" se o valor for maior que o máximo;
5. devolve "DENTRO" nos demais casos (os limites estão dentro).

Exemplos:
    >>> classificar_medicao(25.4, 25, 10)
    'DENTRO'
    >>> classificar_medicao(22.4, 25, 10)
    'ABAIXO'
    >>> classificar_medicao(0, 25, 10)
    'INVÁLIDA'

Dica: calcule minimo e maximo em variáveis ANTES dos ifs. Fica mais
legível e você
confere os limites com um print se algo der errado.

Teste com:  python curso.py testar 02 ex07
```

Exercício 02.08 — Média das medições válidas

Arquivo: `exercicios/ex08_media_das_medicoes.py`

```
Nível: ★★☆

Contexto: O micrômetro digital às vezes "pisca" e registra 0 ou um valor
negativo.
Esses valores são falhas de leitura e não podem entrar na média, senão
puxam o
resultado para baixo e reprovam um lote bom.

Tarefa:
1. `media_valida(valores)` recebe uma lista de medições e devolve a média
só dos
   valores maiores que zero, arredondada com round() para 2 casas.
   - Use continue para pular os valores inválidos.
   - Se não houver nenhum valor válido (ou a lista estiver vazia),
devolva None.
2. `contar_invalidas(valores)` devolve quantos valores são menores ou
iguais a zero.
Não use sum(), len() de listas filtradas nem compreensões: o objetivo é
praticar
acumuladores e contadores.
```

```
Exemplos:
    >>> media_valida([24.8, 25.1, 25.3])
    25.07
    >>> media_valida([25.0, 0, -1, 25.4])
    25.2
    >>> print(media_valida([0, 0]))
    None
    >>> contar_invalidas([25.0, 0, -1, 25.4])
    2

Dica: você vai precisar de dois acumuladores: a soma e a quantidade de
válidos.

Teste com:  python curso.py testar 02 ex08
```

Exercício 02.09 — Maior, menor e amplitude

Arquivo: `exercicios/ex09_amplitude.py`

```
Nível: ★★☆

Contexto: A amplitude (maior valor menos o menor) mostra a uniformidade
da bobina: uma
espessura média perfeita com amplitude alta indica filme irregular, com
pontos finos
que podem rasgar na máquina de envase do cliente.

Tarefa (SEM usar as funções prontas max() e min(): o objetivo é praticar
o padrão):
1. `maior_valor(valores)` devolve o maior valor da lista, ou None se
estiver vazia.
2. `menor_valor(valores)` devolve o menor valor da lista, ou None se
estiver vazia.
3. `amplitude(valores)` devolve maior - menor, arredondado para 2 casas.
Para lista
   vazia, devolva 0.0.

Exemplos:
    >>> maior_valor([24.8, 25.3, 25.1])
    25.3
    >>> menor_valor([24.8, 25.3, 25.1])
    24.8
    >>> amplitude([24.8, 25.3, 25.1])
    0.5

Dica: comece com `maior = None`. No laço, se maior ainda é None OU o
valor é maior que
ele, atualize. Começar com maior = 0 parece funcionar, mas falha com
```

```
números
negativos. Teste com [-3, -1, -7].

Teste com:  python curso.py testar 02 ex09
```

Exercício 02.10 — Quanto tempo até a meta?

Arquivo: `exercicios/ex10_anos_para_dobrar.py`

```
Nível: ★★☆

Contexto: Você decidiu guardar dinheiro para um notebook melhor e um
curso de dados.
Perguntas do tipo "quantas vezes até chegar lá?" são o território do
while: você não
sabe de antemão quantas repetições serão necessárias.

Tarefa:
1. `anos_para_dobrar(taxa_anual_pct)` devolve quantos anos inteiros um
capital leva
   para dobrar (ficar maior ou igual ao dobro) com juros compostos de
   `taxa_anual_pct`% ao ano. A cada ano, o capital é multiplicado por
   (1 + taxa_anual_pct / 100).
   Se a taxa for menor ou igual a zero, o capital nunca dobra: devolva -1
(sem isso,
   seu while vira um loop infinito!).
2. `meses_para_meta(saldo_inicial, deposito_mensal, meta)` devolve
quantos depósitos
   mensais são necessários para o saldo ficar maior ou igual à meta (sem
juros).
   - Se o saldo inicial já atinge a meta, devolva 0.
   - Se o depósito for menor ou igual a zero (e a meta ainda não foi
atingida),
     devolva -1.

Exemplos:
    >>> anos_para_dobrar(10)
    8
    >>> anos_para_dobrar(0)
    -1
    >>> meses_para_meta(0, 500, 3000)
    6
    >>> meses_para_meta(5000, 100, 3000)
    0

Dica: comece com capital = 1.0 (o valor inicial não importa para
"dobrar") e conte
as voltas do while.
```

```
Teste com:  python curso.py testar 02 ex10
```

Exercício 02.11 — Leitura de código: preveja e conserte

Arquivo: `exercicios/ex11_leitura_de_lacos.py`

```
Nível: ★★☆

Contexto: Programador passa mais tempo LENDO código do que escrevendo.
Este exercício
treina isso: primeiro você prevê o que trechos prontos fazem; depois
conserta uma
função com defeito.

PARTE 1 — Preveja. Para cada trecho abaixo, PRIMEIRO escreva sua previsão
num papel
(faça um teste de mesa: anote o valor das variáveis a cada volta do
laço). DEPOIS rode
o trecho no REPL para conferir. Se errou, descubra por quê. Por fim, faça
cada função
`previsao_x()` devolver o valor final da variável pedida.

    Trecho A — valor final de `total`:
        total = 0
        for i in range(1, 10, 3):
            total += i

    Trecho B — valor final de `contador`:
        contador = 0
        n = 100
        while n > 1:
            n = n // 3
            contador += 1

    Trecho C — valor final de `texto`:
        texto = ""
        for letra in "bobina":
            if letra in "aeiou":
                continue
            texto += letra.upper()

    Trecho D — valor final de `achou`:
        posicao = 0
        achou = 0
        for valor in [25.1, 24.9, 22.0, 21.5, 20.0]:
            posicao += 1
            if valor < 23:
                achou = posicao
```

```
                break

PARTE 2 — Conserte. Um colega escreveu a função abaixo para calcular a
média só das
notas maiores ou iguais a 7. Ela tem TRÊS defeitos. Encontre-os (rode,
use print, faça
teste de mesa) e escreva a versão correta em `media_aprovadas`:

        def media_aprovadas(notas):
            for nota in notas:
                soma = 0
                quantidade = 0
                if nota >= 7:
                    soma += nota
                    quantidade + 1
            return soma / quantidade

    Regras da versão correta: média das notas >= 7; se nenhuma nota for
>= 7 (ou a
    lista estiver vazia), devolva 0.0.

Exemplos:
    >>> media_aprovadas([8, 6, 9])
    8.5
    >>> media_aprovadas([5, 6])
    0.0

Dica: um dos defeitos só aparece com a lista vazia ou sem aprovados;
outro faz a
função "esquecer" o que já somou.

Teste com:  python curso.py testar 02 ex11
```

Exercício 02.12 — Três seguidas fora: pare a linha

Arquivo: `exercicios/ex12_sequencia_fora.py`

```
Nível: ★★★

Contexto: Uma medição fora da faixa pode ser um ponto isolado. Mas várias
medições
SEGUIDAS fora indicam que o processo saiu de controle, e a regra do
laboratório manda
parar a linha. Você vai encontrar onde começa a primeira sequência
problemática.

Tarefa: implemente `inicio_sequencia_fora(valores, minimo, maximo,
tamanho)`, que
devolve a POSIÇÃO (contando a partir de 1) da medição que inicia a
```

```
primeira sequência
de `tamanho` medições consecutivas fora da faixa [minimo, maximo].
- Os limites são inclusivos: valor igual a minimo ou maximo está DENTRO.
- Se não existir tal sequência (ou a lista estiver vazia), devolva 0.
- `tamanho` é sempre 1 ou mais.
- Sequências mais longas que `tamanho` contam (devolva o início delas).

Exemplos:
    >>> inicio_sequencia_fora([25, 22, 21, 20, 25], 23, 27, 3)
    2
    >>> inicio_sequencia_fora([22, 25, 21, 20, 25], 23, 27, 3)
    0
    >>> inicio_sequencia_fora([20, 20, 25, 20, 20, 20], 23, 27, 3)
    4

Dica: mantenha dois contadores: a posição atual (aumenta sempre) e
quantas fora
SEGUIDAS você viu (aumenta quando está fora, volta a zero quando está
dentro).
Quando o segundo chegar em `tamanho`, onde começou a sequência?

Teste com:  python curso.py testar 02 ex12
```

Exercício 02.13 — Revisão: tabela de limites por linha

Arquivo: `exercicios/ex13_revisao_tabela_limites.py`

```
Nível: ★★★

Revisão: este exercício mistura o módulo 02 (laços aninhados, match-case,
continue)
com o módulo 01 (cálculo de tolerância, f-strings com :.2f, upper) e com
o molde de
função do módulo 00.

Contexto: O supervisor quer uma "cola" impressa com os limites de
espessura de cada
linha para os nominais mais produzidos, para pendurar ao lado das
extrusoras.

Tolerâncias por linha: A: 5%   B: 5%   C: 8%   D: 10%

Tarefa: implemente `tabela_limites(linhas, nominais)`:
- `linhas` é uma string em que cada caractere é uma linha (ex.: "ACD").
Ignore espaços
  e aceite minúsculas ("a c" equivale a "AC").
- `nominais` é uma lista de espessuras nominais em µm.
- Para cada linha conhecida, gere um cabeçalho e, para cada nominal, uma
linha
```

```
  recuada com 2 espaços:
      Linha A (±5%)
        20.00 µm: 19.00 a 21.00
        25.00 µm: 23.75 a 26.25
- Para uma linha desconhecida (ex.: "X"), gere só: Linha X: sem
tolerância cadastrada
- TODA linha do texto termina com "\n" (inclusive a última).
- Valores com 2 casas (:.2f); tolerância sem casas (:.0f).

Exemplos:
    >>> print(tabela_limites("a", [20, 25]), end="")
    Linha A (±5%)
      20.00 µm: 19.00 a 21.00
      25.00 µm: 23.75 a 26.25
    >>> tabela_limites("X", [20])
    'Linha X: sem tolerância cadastrada\n'

Dica: o laço de fora percorre as linhas; o de dentro, os nominais. Use um
texto
acumulador (resultado = "" e resultado += ...). Uma função auxiliar com
match-case
para a tolerância deixa tudo mais limpo.

Teste com:  python curso.py testar 02 ex13
```

Exercício 02.14 — Revisão: resumo de uma série vinda da planilha

Arquivo: `exercicios/ex14_revisao_resumo_serie.py`

```
Nível: ★★★

Revisão: este exercício mistura o módulo 02 (laços, validação,
contadores, máximo e
mínimo, if/elif) com o módulo 01 (strip, replace, float(), f-strings com
:.2f).

Contexto: As leituras de espessura de uma série chegam da planilha como
TEXTO, do jeito
que o operador digitou: "24,8", " 25.1 ", células vazias e, às vezes,
lixo como "erro"
ou "25,4,1". O sistema precisa resumir a série sem quebrar.

Tarefa:
1. `eh_numero(texto)` devolve True se o texto representa um número
decimal
   não negativo. Regras, depois de tirar os espaços das pontas e trocar
"," por ".":
   - não pode ficar vazio;
   - só pode ter dígitos (0 a 9) e no máximo UM ponto;
```

```
   - precisa ter pelo menos um dígito.
   Não use try/except (módulo 06): verifique caractere por caractere.
2. `resumo_serie(leituras, minimo, maximo)` percorre as leituras e:
   - ignora as vazias (só espaços também conta como vazia);
   - conta como inválidas as que não passam em eh_numero;
   - converte as válidas para float e calcula: quantidade, média, mínimo,
máximo e
     quantas estão fora da faixa [minimo, maximo] (limites inclusivos).
   Devolve o texto (valores com 2 casas):
       válidas=3 | inválidas=1 | média=25.97 | mín=24.80 | máx=28.00 |
fora=1 | REPROVADO
   Status: "REPROVADO" se houver alguma fora; senão "REVISAR" se houver
alguma
   inválida; senão "APROVADO".
   Se não houver nenhuma válida, devolva: Nenhuma leitura válida
(inválidas=N)

Exemplos:
    >>> eh_numero(" 25,4 ")
    True
    >>> eh_numero("25,4,1")
    False
    >>> resumo_serie(["24,8", "25,3", "25,1"], 23, 27)
    'válidas=3 | inválidas=0 | média=25.07 | mín=24.80 | máx=25.30 |
fora=0 | APROVADO'

Dica: resolva eh_numero primeiro e teste no REPL com vários textos. Em
resumo_serie,
um único laço pode atualizar todos os contadores e acumuladores.

Teste com:  python curso.py testar 02 ex14
```

Desafio do módulo 02 — Inspetor de série no terminal

Contexto

No fim de cada bobina, o operador mede a espessura em vários pontos e anota numa folha. O laboratório quer trocar a folha por um programa simples de terminal: o operador informa a linha e o nominal, digita as leituras uma a uma e o programa responde na hora se cada leitura está boa. Se o processo sair de controle, o programa manda parar a linha. No fim, emite o veredito da série.

É o primeiro programa **interativo e que decide** do curso: junta validação, `while`, `match-case`, contadores, acumuladores, máximo e mínimo, `break` e `continue`.

Requisitos obrigatórios

1. Crie o arquivo `inspetor.py` (pode ser dentro de `desafio/`).
2. **Linha de produção:** pergunte a linha (A, B, C ou D), aceitando minúsculas e espaços. Use `match-case` para obter a tolerância (A e B: 5%, C: 8%, D: 10%). Linha desconhecida: avise e **pergunte de novo** (laço de validação).
3. **Nominal:** pergunte a espessura nominal em µm, aceitando vírgula. Se não for um número maior que zero, avise e pergunte de novo. Não use `try/except` (é do módulo 06): valide o texto antes de converter.
4. Mostre a faixa aceita (nominal ± tolerância), com 2 casas.
5. **Leituras:** leia as medições uma a uma até o operador digitar `fim`:
   - Enter vazio é ignorado sem aviso;
   - texto que não é número gera um aviso e não conta;
   - cada leitura válida é classificada na hora: `OK`, `FORA (abaixo)` ou `FORA (acima)` (limites inclusivos);
   - **regra de parada:** 3 leituras válidas **seguidas** fora da faixa encerram a leitura com o alerta `PARE A LINHA`.
6. **Relatório final:** quantidade de leituras válidas, média, mínima, máxima, amplitude (máxima - mínima), quantidade e porcentagem fora da faixa. Sem `sum()`, `min()` ou `max()`: use os padrões da aula.
7. **Veredito:**
   - `REPROVADO (processo fora de controle)` se a regra de parada disparou;
   - senão, `REPROVADO (muitas leituras fora)` se mais de 20% das leituras ficaram fora;
   - senão, `CONDICIONAL` se houve alguma fora;
   - senão, `APROVADO`. Sem nenhuma leitura válida, diga isso em vez de dividir por zero.
8. **Modo demonstração:** o programa precisa conseguir rodar **sem ninguém digitar nada**, usando respostas de exemplo, para você (e o professor) testarem rápido. Uma constante `MODO_DEMO = True` no topo liga esse modo. Dica de implementação na seção "Dicas".
9. Roda sem erro e passa no `ruff check`.

Extras opcionais

- **Limite de segurança:** pare depois de 50 leituras válidas, mesmo sem `fim`.
- **Maior sequência:** informe no relatório a maior sequência de leituras seguidas fora.
- **Gramatura também:** pergunte no início qual ensaio será feito (espessura ou gramatura, com `match-case`) e ajuste a unidade das mensagens.
- **Formato brasileiro:** mostre os números com vírgula decimal no relatório (técnica do exercício 01.12).

- **Volte ao desafio do módulo 01:** acrescente à ficha da bobina o veredito automático (dentro ou fora da faixa).

Exemplo de execução

Modo demonstração (as respostas aparecem depois de cada pergunta, como se alguém tivesse digitado):

```
PS C:\curso-python> python 02-decisoes-e-
repeticoes\desafio\solucao_referencia.py
================================================
        INSPETOR DE SÉRIE — ESPESSURA
================================================
Linha (A, B, C ou D): E
  Linha desconhecida. Tente de novo.
Linha (A, B, C ou D): b
Espessura nominal (µm): abc
  Digite um número maior que zero, como 25 ou 25,5.
Espessura nominal (µm): 0
  Digite um número maior que zero, como 25 ou 25,5.
Espessura nominal (µm): 25
Faixa da linha B (±5%): 23.75 a 26.25 µm
Digite as leituras (Enter vazio é ignorado; 'fim' encerra).
  Leitura 1: 25,3
    25.30 µm -> OK
  Leitura 2:
  Leitura 2: 24,9
    24.90 µm -> OK
  Leitura 3: xx
    Leitura inválida, não contada.
  Leitura 3: 26,8
    26.80 µm -> FORA (acima)
  Leitura 4: 25,1
    25.10 µm -> OK
  Leitura 5: 24,6
    24.60 µm -> OK
  Leitura 6: 25,0
    25.00 µm -> OK
  Leitura 7: 25,2
    25.20 µm -> OK
  Leitura 8: fim
------------------------------------------------
Leituras válidas                              7
Média (µm)                                25.27
Mínima / máxima (µm)             24.60 / 26.80
Amplitude (µm)                             2.20
Fora da faixa                        1 (14%)
```

```
Veredito: CONDICIONAL (avaliar com o líder)
================================================
```

Modo interativo, disparando a regra de parada (linha D, nominal 20 µm, faixa 18 a 22):

```
  Leitura 1: 18,1
    18.10 µm -> OK
  Leitura 2: 17,5
    17.50 µm -> FORA (abaixo)
  Leitura 3: 22,9
    22.90 µm -> FORA (acima)
  Leitura 4: 17
    17.00 µm -> FORA (abaixo)
    !!! 3 seguidas fora: PARE A LINHA.
------------------------------------------------
Leituras válidas                                4
Média (µm)                                  18.88
Mínima / máxima (µm)            17.00 / 22.90
Amplitude (µm)                               5.90
Fora da faixa                           3 (75%)
Veredito: REPROVADO (processo fora de controle)
================================================
```

Critérios de aceite

- [ ] Com `MODO_DEMO = True`, o programa roda do começo ao fim sem pedir nada.
- [ ] Com `MODO_DEMO = False`, ele pergunta de verdade e funciona com digitação.
- [ ] Linha inválida e nominal inválido (texto, zero, vazio) fazem o programa perguntar de novo, sem quebrar.
- [ ] Nenhuma entrada digitada derruba o programa com `ValueError` (teste `abc`, `25,4,1`, `-3`, `.`).
- [ ] Os limites da faixa contam como dentro (teste uma leitura exatamente igual ao mínimo).
- [ ] Uma leitura boa no meio "zera" a contagem de seguidas fora.
- [ ] Três seguidas fora encerram as leituras com o alerta.
- [ ] O relatório não divide por zero quando não há leituras válidas (digite `fim` logo de cara).
- [ ] O veredito segue a ordem de prioridade dos requisitos.
- [ ] Não há `while` sem uma saída garantida (`break` ou condição que fica falsa).
- [ ] `ruff check` não reclama.

Dicas de como começar

1. **Comece sem interação.** Escreva primeiro o "miolo" com as leituras numa lista fixa (`for texto in ["25,3", "xx", ...]`), como o exemplo `exemplos/10_inspecao_serie.py`. Só depois troque a lista por perguntas.
2. **Um laço por pergunta.** A linha e o nominal usam o mesmo molde: `while True:` → pergunta → se válido, `break` → senão, avisa. Escreva um, teste, copie o molde para o outro.
3. **Modo demonstração sem índice de lista:** guarde as respostas de exemplo numa lista e escreva uma função `ler(passo, pergunta)`. O programa principal conta as perguntas (`passo += 1` antes de cada uma) e a função percorre a lista com um contador até chegar à resposta número `passo` (é o padrão "buscar posição" da aula). Se `MODO_DEMO` for `False`, a função simplesmente devolve `input(pergunta)`.
4. **Valide o número antes de converter:** reaproveite a ideia do exercício 14 (`eh_numero`).
5. **Teste de mesa** com a sequência `ok, fora, fora, ok, fora, fora, fora`: em que leitura o alerta deve disparar? Confira se o seu programa concorda.
6. Para sair de um programa travado num loop infinito: `Ctrl+C`.

A solução de referência está em `solucao_referencia.py`. Tente o seu antes de abrir.

Quiz do módulo 02 — Decisões e repetições

Responda sem rodar o código. Para os laços, faça o teste de mesa no papel. Só depois confira o gabarito no final (e rode os trechos no REPL).

---

1. O que este código imprime?

```
print(7 >= 7, 7 > 7, "a" == "A")
```

2. Qual condição verifica corretamente se `linha` é `"A"` ou `"B"`?

a) `linha == "A" or "B"` b) `linha == "A" or linha == "B"` c) `linha == ("A" or "B")`
d) `linha in "AB"`

3. A média é 8,5. O que este código imprime?

```
nota = 8.5
if nota >= 5:
    print("Recuperação")
elif nota >= 7:
```

```
    print("Aprovado")
else:
    print("Reprovado")
```

4. Quais destes valores são falsy (contam como falso num `if`)?

`0` `"0"` `""` `" "` `None` `[]` `-1`

5. O que este código imprime?

```
status = "OK" if 0 else "FORA"
print(status)
```

6. Quais números `range(10, 0, -3)` gera?

a) 10, 7, 4, 1 b) 10, 7, 4, 1, 0 c) 10, 7, 4 d) nenhum: `range` não aceita passo negativo

7. Quantas vezes o `print` é executado?

```
for linha in "AB":
    for turno in range(3):
        print(linha, turno)
```

8. O que este código imprime?

```
total = 0
for n in range(1, 6):
    if n == 4:
        break
    if n % 2 == 0:
        continue
    total += n
print(total)
```

9. Onde está o defeito? O que acontece ao rodar?

```
contador = 10
while contador > 0:
    print(contador)
    contador += 1
```

10. O que este código imprime?

```
soma = 0
for valor in [10, 20, 30]:
```

```
    soma = 0
    soma += valor
print(soma)
```

11. O código abaixo deveria mostrar o maior valor da lista. O que ele imprime, e qual é o defeito?

```
valores = [-5, -2, -9]
maior = 0
for v in valores:
    if v > maior:
        maior = v
print(maior)
```

12. O código abaixo dá erro quando o usuário digita `25`. Qual erro e por quê?

```
leitura = input("Espessura: ")
if leitura < 30:
    print("Abaixo de 30")
```

13. O que este código imprime?

```
codigo = "b"
match codigo.upper():
    case "A" | "B":
        print("5%")
    case "C":
        print("8%")
    case _:
        print("?")
```

14. Uma variável `variacao` pode valer `None` (não medido) ou um número, inclusive `0.0`. Qual teste diz corretamente se ela **foi medida**?

a) `if variacao:` b) `if variacao != 0:` c) `if variacao is not None:` d) `if variacao == "":`

15. O que este código imprime?

```
x = 0
while x < 10:
    x += 3
print(x)
```

Gabarito

1. **`True False False`.** 7 é maior **ou igual** a 7 (verdade), mas não é estritamente maior. E texto diferencia maiúsculas de minúsculas.

2. **b) `linha == "A" or linha == "B"`.** A opção a) é sempre verdadeira, porque `"B"` sozinho é truthy. A c) calcula `"A" or "B"` primeiro (dá `"A"`), então só aceita `"A"`. A d) aceita `"A"` e `"B"`, mas também `""` e `"AB"`, porque ambos estão "dentro" de `"AB"`.

3. **`Recuperação`.** O Python testa de cima para baixo e executa só o primeiro bloco verdadeiro. Como `8.5 >= 5` é verdade, o `elif nota >= 7` nem é testado. A ordem correta testa `>= 7` antes de `>= 5`.

4. **`0`, `""`, `None` e `[]`.** `"0"` e `" "` são textos **não vazios** (truthy), e `-1` é um número diferente de zero (truthy).

5. **`FORA`.** `0` é falsy, então a expressão condicional escolhe o valor do `else`.

6. **a) 10, 7, 4, 1.** Começa em 10 e soma -3 a cada passo. O próximo seria -2, que já passou do fim (0). O fim nunca entra, e passo negativo é permitido.

7. **6 vezes.** O laço de dentro roda inteiro (3 voltas) para cada volta do de fora (2 linhas): 2 × 3 = 6.

8. **`4`.** Teste de mesa: n=1 é ímpar, total = 1; n=2 é par, `continue` pula; n=3 é ímpar, total = 4; n=4 dispara o `break` antes de qualquer outra coisa. O 5 nunca é visto.

9. **Loop infinito.** O contador começa em 10 e **aumenta** (`+= 1`), então `contador > 0` nunca fica falso. O programa imprime números sem parar até você apertar `Ctrl+C`. O certo é `contador -= 1`.

10. **`30`.** O acumulador é zerado **dentro** do laço, a cada volta. No fim, só sobra o último valor. `soma = 0` deve ficar antes do `for` (e aí o resultado seria 60).

11. **Imprime `0`**, um valor que nem está na lista. Como todos os valores são negativos, nenhum é maior que o valor inicial 0. O certo é começar com `maior = None` e testar `if maior is None or v > maior:`. Resposta correta: -2.

12. **`TypeError: '<' not supported between instances of 'str' and 'int'`.** `input()` sempre devolve texto (`"25"`), e o Python não compara texto com número usando `<`. Converta antes: `leitura = float(input("Espessura: "))` (e, num programa de verdade, valide antes de converter).

13. **`5%`.** `"b".upper()` vira `"B"`, que casa com o caso `"A" | "B"`.

14. **c) `if variacao is not None:`.** A opção a) trata `0.0` (uma medição válida) como "não medido", porque `0.0` é falsy. A b) também rejeita o zero. A d) compara com texto vazio, que não tem nada a ver com `None`.

15. **`12`.** x passa por 3, 6, 9 e 12. Com 9 a condição `9 < 10` ainda é verdadeira, então soma mais 3. Só com 12 o laço termina. O `while` testa a condição **antes** de cada volta, não depois de cada soma.

Módulo 03 — Coleções: listas, tuplas, dicionários e conjuntos

> **Tempo estimado:** 8 a 12 horas · **Pré-requisitos:** módulos 00, 01 e 02 · **Nível:** iniciante

O que você vai aprender

- Criar listas, acessar itens por índice (inclusive negativo) e recortar trechos com fatiamento.
- Modificar listas com `append`, `insert`, `remove`, `pop` e `sort`, sabendo quais métodos devolvem `None`.
- Explicar com um desenho a diferença entre **cópia e referência**, e copiar uma lista (ou um dicionário de listas) sem efeitos surpresa.
- Usar tuplas para registros fixos e desempacotar valores (`op, peso = pedido`).
- Criar e consultar dicionários com `[]` e `get`, percorrer com `items()` e montar contagens e agrupamentos.
- Navegar num dicionário aninhado, como a tabela de tolerâncias por linha.
- Usar conjuntos para tirar repetições e cruzar listas (o que está nas duas, o que falta em uma).
- Percorrer com `enumerate` e `zip`, e usar `sum`, `min`, `max`, `any`, `all` e `sorted` com `key`.
- Escrever compreensões de lista, dicionário e conjunto, e saber quando um `for` comum é melhor.
- Tratar uma **lista de dicionários como tabela**: filtrar, escolher colunas, ordenar, agrupar e juntar. É a mesma lógica do pandas e do SQL.

Por que isso importa

Até agora, cada variável guardava um valor. Uma espessura. Uma nota. Um peso.

Seu trabalho nunca é um valor só. É a série de medições de uma bobina. É a planilha com as amostras do dia. É a tabela de tolerâncias de cada linha. É a lista de OPs que a expedição pediu por e-mail.

Coleções são o jeito de o Python guardar muitos valores organizados. Sem elas não existe análise de dados, automação nem API:

- um DataFrame do pandas (módulo 10) é uma tabela, e você vai montá-la a partir de listas e dicionários;
- uma consulta SQL (módulo 11) devolve uma lista de linhas;
- o JSON de uma API (módulos 12 e 13) é feito de dicionários e listas.

No projeto final, as tolerâncias por linha viram um dicionário aninhado, os valores da planilha viram uma lista de dicionários e o cruzamento "OPs do e-mail × OPs com resultado" é uma operação de conjuntos. Você vai escrever versões pequenas de tudo isso neste módulo.

1. Listas: vários valores numa variável só

Pense numa coluna do Excel com as espessuras das bobinas da linha A. Cada valor ocupa uma posição. Em Python, isso é uma **lista**: valores entre colchetes, separados por vírgula. (Abra o REPL com `python` no PowerShell e digite junto: ver a saída na sua tela fixa muito mais do que ler.)

```
>>> espessuras = [50.2, 49.8, 51.0, 50.5, 48.9]
>>> espessuras
[50.2, 49.8, 51.0, 50.5, 48.9]
>>> len(espessuras)
5
```

Cada item tem um **índice**: a posição dele. A diferença do Excel é que o Python começa a contar do **zero**.

```
 índice:       0      1      2      3      4
             +------+------+------+------+------+
 espessuras  | 50.2 | 49.8 | 51.0 | 50.5 | 48.9 |
             +------+------+------+------+------+
 negativo:    -5     -4     -3     -2     -1
```

```
>>> espessuras[0]
50.2
>>> espessuras[2]
51.0
>>> espessuras[-1]
48.9
>>> espessuras[-2]
50.5
```

O índice negativo conta do fim para o começo. `espessuras[-1]` é sempre o último, não importa o tamanho da lista. Isso é muito mais seguro do que `espessuras[len(espessuras) - 1]`.

Você também troca um item pelo índice:

```
>>> espessuras[1] = 49.9
>>> espessuras
[50.2, 49.9, 51.0, 50.5, 48.9]
```

Lista vazia é `[]`, e `list(range(5))` vira `[0, 1, 2, 3, 4]`. Uma lista pode misturar tipos (`["OP-4512", "A", 1250.5]`), mas na prática prefira listas de **uma coisa só**: só espessuras, só OPs. Para um registro com campos diferentes, você vai ver ferramentas melhores (tuplas e dicionários) daqui a pouco.

> ⚠️ **Erro comum:** pedir um índice que não existe. A lista acima tem 5 itens, então `espessuras[5]` dá `IndexError: list index out of range`. Com `n` itens, os índices vão de `0` a `n - 1`.

O exemplo completo está em `exemplos/01_listas.py`. No PowerShell, dentro da pasta `03-colecoes`:

```
python exemplos\01_listas.py
```

(No Linux/macOS, use `python3 exemplos/01_listas.py`.)

Tente você: dada `notas = [7.0, 8.5, 6.0, 9.5]`, pegue o `9.5` de dois jeitos diferentes.

<details>
<summary>Resposta</summary>

`notas[3]` e `notas[-1]`. Qual usar? Se a pergunta é "qual a última nota?", use `notas[-1]`: continua certo mesmo se a lista crescer. `notas[3]` só faz sentido quando a posição 3 tem um significado fixo (por exemplo, "a 4ª prova").

</details>

2. Fatiamento: recortando pedaços

Às vezes você não quer um item, quer um **trecho**: as 3 primeiras medições, as 2 últimas. Para isso existe o **fatiamento** (*slicing*): `lista[inicio:fim]`.

A melhor forma de entender é imaginar **cortes entre os itens**, numerados assim:

```
cortes:   0      1      2      3      4      5
          |      |      |      |      |      |
          +------+------+------+------+------+
          | 50.2 | 49.8 | 51.0 | 50.5 | 48.9 |
          +------+------+------+------+------+

medicoes[1:4]  ->  do corte 1 ao corte 4  ->  [49.8, 51.0, 50.5]
```

O item do `fim` **não entra**. `[1:4]` pega os índices 1, 2 e 3. Quantos itens? `4 - 1 = 3`.

```
>>> medicoes = [50.2, 49.8, 51.0, 50.5, 48.9, 50.1]
>>> medicoes[1:4]
[49.8, 51.0, 50.5]
>>> medicoes[:3]
[50.2, 49.8, 51.0]
>>> medicoes[3:]
[50.5, 48.9, 50.1]
>>> medicoes[-2:]
[48.9, 50.1]
```

- Sem `inicio`: começa do começo. Sem `fim`: vai até o fim.
- `[:3]` = "os 3 primeiros". `[-2:]` = "os 2 últimos".

Existe um terceiro número, o **passo**: `lista[inicio:fim:passo]`.

```
>>> medicoes[::2]
[50.2, 51.0, 48.9]
>>> medicoes[::-1]
[50.1, 48.9, 50.5, 51.0, 49.8, 50.2]
```

Duas propriedades importantes:

1. **Fatiar nunca dá `IndexError`.** Se o fim passa do tamanho, ele simplesmente para no fim: `medicoes[:100]` devolve a lista toda.
2. **Fatiar sempre cria uma lista nova.** `medicoes[:]` é uma cópia da lista inteira (isso vai ser importante na seção 4).

Strings também fatiam, do mesmo jeito: `"OP-4512-A"[3:7]` é `'4512'` e `"OP-4512-A"[-1]` é `'A'`.

> 🔮 **Preveja antes de rodar:** você quer as `n` últimas medições e escreve `medicoes[-n:]`. O que acontece quando `n = 0`?

```
n = 0
print(medicoes[-n:])
```

▼ **Resposta**

Imprime a lista **inteira**: `[50.2, 49.8, 51.0, 50.5, 48.9, 50.1]`.

Por quê? `-0` é igual a `0`. Então `medicoes[-0:]` vira `medicoes[0:]`, que é "do começo até o fim". Quando `n` pode ser zero, trate esse caso antes com um `if n <= 0`. O exercício 01 cobra exatamente isso.

🔧 **Modifique:** em `exemplos/02_fatiamento.py`, troque `medicoes[1:4]` por `medicoes[4:1]` e por `medicoes[4:1:-1]`. Explique por que um devolve `[]` e o outro não.

3. Modificando listas: os métodos

Listas têm **métodos**: funções que você chama com um ponto depois do nome (`lista.append(x)`). Os mais usados:

Método	O que faz	Devolve
`lista.append(x)`	acrescenta `x` no fim	`None`
`lista.extend(outra)`	acrescenta todos os itens de `outra` no fim	`None`
`lista.insert(i, x)`	insere `x` na posição `i`	`None`
`lista.remove(x)`	remove a **primeira** ocorrência de `x`	`None`
`lista.pop()` / `lista.pop(i)`	remove o último (ou o da posição `i`)	o item removido
`lista.index(x)`	posição da primeira ocorrência de `x`	um `int`
`lista.count(x)`	quantas vezes `x` aparece	um `int`
`lista.sort()`	ordena a própria lista	`None`
`lista.copy()`	cria uma cópia rasa	uma lista nova

Veja alguns em ação:

```
>>> ops = ["OP-4512", "OP-4513"]
>>> ops.append("OP-4514")
>>> ops.insert(0, "OP-4500")
>>> ops
['OP-4500', 'OP-4512', 'OP-4513', 'OP-4514']
>>> ops.extend(["OP-4515", "OP-4516"])
>>> ops.remove("OP-4513")
>>> ultima = ops.pop()
>>> ultima
'OP-4516'
>>> ops
['OP-4500', 'OP-4512', 'OP-4514', 'OP-4515']
```

Operadores também funcionam: `[1, 2] + [3]` dá `[1, 2, 3]`, `[0] * 4` dá `[0, 0, 0, 0]` e `"OP-4512" in ops` dá `True`.

`sort()` ou `sorted()`?

- `lista.sort()` **ordena a própria lista** e devolve `None`.
- `sorted(lista)` **devolve uma lista nova** ordenada e não mexe na original.

```
>>> notas = [7.5, 9.0, 6.0]
>>> sorted(notas)
[6.0, 7.5, 9.0]
>>> notas
[7.5, 9.0, 6.0]
>>> notas.sort(reverse=True)
>>> notas
[9.0, 7.5, 6.0]
```

🔮 **Preveja antes de rodar:** o que este código imprime?

```
medicoes = [50.2, 49.8, 51.0]
medicoes = medicoes.sort()
print(medicoes)
```

▼ **Resposta**

Imprime `None`. O `sort()` ordenou a lista e devolveu `None`, e a linha `medicoes = ...` guardou esse `None` na variável. A lista ordenada se perdeu.

> Regra prática: **métodos que modificam a lista no lugar devolvem `None`**. Use `medicoes.sort()` sozinho na linha, ou `medicoes = sorted(medicoes)`.

Como descobrir sozinho o que um método faz

Você não precisa decorar a tabela. No REPL, `help(list.remove)` mostra a documentação do método (aperte `q` para sair). Ou teste uma hipótese. "Será que `remove` apaga **todas** as ocorrências?" Não adivinhe, pergunte ao Python:

```
>>> teste = [1, 2, 1, 3]
>>> teste.remove(1)
>>> teste
[2, 1, 3]
```

Só a primeira. Esse hábito (criar um exemplo mínimo no REPL para tirar a dúvida) vai economizar horas da sua vida de programador. A documentação oficial das listas está em docs.python.org/pt-br/3/tutorial/datastructures.html.

4. Mutabilidade: cópia ou referência?

Esta é a seção mais importante do módulo. Os bugs mais estranhos de iniciante (e de muito profissional) vêm daqui.

4.1 Variáveis são etiquetas, não caixas

No módulo 01, dissemos que uma variável "guarda" um valor. Agora vamos ser mais precisos: **uma variável é uma etiqueta presa num objeto**. O objeto (a lista) mora na memória; o nome só aponta para ele.

```
>>> a = [50.2, 49.8]
>>> b = a
>>> b.append(51.0)
>>> a
[50.2, 49.8, 51.0]
```

Mexemos em `b` e `a` mudou. Por quê? Porque `b = a` **não copiou a lista**. Só colou uma segunda etiqueta no mesmo objeto:

```
 b = a            (NÃO copia: só cola mais uma etiqueta)

     a ---------> +------+------+
                  | 50.2 | 49.8 |
     b ---------> +------+------+
```

```
b.append(51.0)   (muda o objeto; os dois nomes "veem")

    a ---------> +------+------+------+
                 | 50.2 | 49.8 | 51.0 |
    b ---------> +------+------+------+
```

Uma analogia do seu dia a dia: `b = a` é como mandar o **link** de uma planilha do OneDrive. Quem edita pelo link muda a planilha de todo mundo. Copiar de verdade é mandar a planilha **em anexo**: cada um fica com a sua.

Como provar que são o mesmo objeto? Com o operador `is`:

```
>>> a is b
True
```

- `==` pergunta: "têm o **mesmo conteúdo**?"
- `is` pergunta: "são o **mesmo objeto**?" (o mesmo que comparar `id(a) == id(b)`; `id()` é o "número de série" do objeto na memória).

4.2 Como copiar de verdade

Três formas equivalentes de copiar uma lista: `original.copy()`, `original[:]` e `list(original)`.

```
>>> original = [50.2, 49.8]
>>> copia = original.copy()
>>> original == copia, original is copia
(True, False)
>>> copia.append(51.0)
>>> original
[50.2, 49.8]
```

Agora são dois objetos, cada um com sua etiqueta. Dicionários e conjuntos também têm `.copy()`.

4.3 Mutáveis e imutáveis

Alguns tipos **podem mudar** depois de criados; outros **não**.

Imutáveis (não mudam)	Mutáveis (mudam no lugar)
`int`, `float`, `bool`, `None`	`list`
`str`	`dict`

Imutáveis (não mudam)	Mutáveis (mudam no lugar)
`tuple`	`set`

Com imutáveis, "mudar" sempre cria um objeto novo e move a etiqueta. Por isso este código é tranquilo:

```
>>> x = 10
>>> y = x
>>> x += 1
>>> x, y
(11, 10)
```

O `x += 1` criou o objeto `11` e moveu só a etiqueta `x`; `y` continua no `10`. Com strings é igual: `texto.upper()` não muda `texto`, devolve uma string nova. Você já viu isso no módulo 01.

🔮 **Preveja antes de rodar:** o que aparece nas duas últimas linhas?

```
a = [1, 2]
b = a
a += [3]
print(b)
a = a + [4]
print(a, b)
```

▼ **Resposta**

```
[1, 2, 3]
[1, 2, 3, 4] [1, 2, 3]
```

Com listas, `a += [3]` **modifica a lista no lugar** (como um `extend`), então `b` vê o 3. Já `a = a + [4]` **cria uma lista nova** e move só a etiqueta `a`. Daí em diante, `a` e `b` apontam para objetos diferentes.

4.4 Cópia rasa × cópia profunda

`.copy()` copia só **a caixa de fora**. Se dentro dela houver outras listas, as de dentro continuam compartilhadas. Isso se chama **cópia rasa** (*shallow copy*).

```
>>> por_linha = {"A": [50.2, 49.8], "B": [20.1]}
>>> rasa = por_linha.copy()
```

```
>>> rasa["C"] = [35.0]
>>> "C" in por_linha
False
>>> rasa["A"].append(99.9)
>>> por_linha["A"]
[50.2, 49.8, 99.9]
```

Acrescentar a chave `"C"` em `rasa` não afetou o original (o dicionário é novo). Mas o `append` em `rasa["A"]` afetou! Veja por quê:

```
 rasa = por_linha.copy()

  por_linha --> { "A": * , "B": * }   dicionário original
                        |       |
                        v       v
                  [50.2, 49.8] [20.1]  as listas existem UMA vez só
                        ^       ^
                        |       |
  rasa -------> { "A": * , "B": * }   dicionário NOVO, mesmas listas
```

É como copiar uma pasta cheia de **atalhos**: a pasta é nova, mas os atalhos abrem os mesmos arquivos.

Para copiar "até o fundo", use `copy.deepcopy` (o módulo `copy` vem com o Python; você importa como importou o `math` no módulo 01):

```
>>> import copy
>>> por_linha = {"A": [50.2, 49.8], "B": [20.1]}
>>> profunda = copy.deepcopy(por_linha)
>>> profunda["A"].append(99.9)
>>> por_linha["A"]
[50.2, 49.8]
```

⚠️ **Erro comum:** criar uma "grade" com `[[0] * 3] * 3`. O `* 3` de fora repete **a mesma lista interna** três vezes:

```
>>> grade = [[0] * 3] * 3
>>> grade[0][0] = 1
>>> grade
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

A forma certa usa uma compreensão (seção 10), que cria uma lista nova a cada volta: `[[0] * 3 for _ in range(3)]`. O `_` é o nome convencional para "variável que eu não vou usar".

4.5 Funções recebem o mesmo objeto

Quando você passa uma lista para uma função, a função recebe **uma etiqueta para a mesma lista**, não uma cópia. Se a função modificar a lista, quem chamou vê a mudança.

Exemplo resolvido: a função que estragava os dados brutos. Um analista escreveu esta função para descontar o *offset* do micrômetro:

```
>>> def corrigir_offset(medicoes, offset):
...     for i in range(len(medicoes)):
...         medicoes[i] = round(medicoes[i] - offset, 2)
...     return medicoes
...
>>> brutas = [50.4, 49.9, 51.2]
>>> corrigidas = corrigir_offset(brutas, 0.2)
>>> corrigidas
[50.2, 49.7, 51.0]
>>> brutas
[50.2, 49.7, 51.0]
```

O resultado está certo, mas os dados **brutos sumiram**. Se o laudo precisar mostrar o valor medido, ele não existe mais.

Como eu penso para consertar:

1. **Quem é dono da lista?** Quem chamou a função. A função só deveria **ler**.
2. **O que a função precisa entregar?** Uma lista com os valores corrigidos.
3. **Então:** monto uma lista **nova** e devolvo ela. A recebida fica intacta.

```
>>> def corrigir_offset(medicoes, offset):
...     corrigidas = []
...     for valor in medicoes:
...         corrigidas.append(round(valor - offset, 2))
...     return corrigidas
...
>>> brutas = [50.4, 49.9, 51.2]
>>> corrigir_offset(brutas, 0.2)
[50.2, 49.7, 51.0]
>>> brutas
[50.4, 49.9, 51.2]
```

Regra que vale para o curso inteiro: **uma função não deve modificar a coleção que recebeu**, a menos que esse seja o objetivo declarado dela (como o `sort()`). No módulo 04 você vai ver que isso tem nome: função **pura**.

🔮 **Preveja antes de rodar:** você quer tirar os valores acima de 50 de uma lista. O que sobra?

```
valores = [10, 55, 60, 20]
for v in valores:
    if v > 50:
        valores.remove(v)
print(valores)
```

▼ **Resposta**

`[10, 60, 20]`. O 60 escapou!

O `for` anda por **posições**. Na posição 1 está o 55: ele é removido e todo mundo anda uma casa para a esquerda (o 60 vai para a posição 1). O `for` segue para a posição 2, que agora é o 20. O 60 nunca foi olhado.

Nunca remova itens da lista que você está percorrendo. Monte uma lista nova com o que fica: `[v for v in valores if v <= 50]`.

🔧 **Modifique:** rode `exemplos/03_mutabilidade.py`, depois cole o código em pythontutor.com (escolha Python 3) e avance passo a passo. Você vai **ver** as setas dos diagramas acima.

5. Tuplas e desempacotamento

Uma **tupla** é como uma lista que não pode mudar. Usa parênteses:

```
>>> bobina = ("OP-4512", "A", 1250.5)
>>> bobina[0]
'OP-4512'
>>> len(bobina)
3
>>> bobina[2] = 1300.0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

Para que serve uma "lista que não muda"?

- **Registros fixos:** (OP, linha, peso); (mínimo, máximo); (latitude, longitude). A posição tem significado e não deve mudar.
- **Devolver vários valores** de uma função (módulo 04).
- **Chaves de dicionário:** tuplas podem ser chaves, listas não (seção 7).
- **Segurança:** quem recebe sua tupla não consegue alterá-la sem querer.

Desempacotamento

O poder real das tuplas está no **desempacotamento**: cada posição vai para uma variável. Funciona com qualquer sequência (listas também) e deixa o código muito mais legível. Compare `peso` com `bobina[2]`: qual dos dois explica melhor o que é?

```
>>> op, linha, peso = bobina
>>> peso
1250.5
>>> minimo, maximo = 55.0, 45.0
>>> minimo, maximo = maximo, minimo
>>> minimo, maximo
(45.0, 55.0)
>>> medicoes = [50.2, 49.8, 51.0, 50.5]
>>> primeira, *meio, ultima = medicoes
>>> meio
[49.8, 51.0]
>>> pedidos = [("OP-4512", 500.0), ("OP-4513", 320.5)]
>>> for op, peso in pedidos:
...     print(f"{op}: {peso} kg")
...
OP-4512: 500.0 kg
OP-4513: 320.5 kg
```

- `minimo, maximo = maximo, minimo` troca dois valores sem variável auxiliar.
- `*meio` recolhe "o resto" (sempre vira lista).
- `for op, peso in pedidos` desempacota cada tupla da lista, a cada volta.

> ⚠️ **Erro comum:** tupla de um item só. `(50.2)` é só um número entre parênteses. A vírgula é que faz a tupla: `(50.2,)`.

> ⚠️ **Erro comum:** número de variáveis diferente do número de itens. `op, peso = ("OP-4512", "A", 1250.5)` dá `ValueError: too many values to unpack (expected 2)`: são 3 itens para 2 variáveis.

Exemplo completo: `exemplos/04_tuplas.py`.

6. Percorrendo melhor: `enumerate` e `zip`

`enumerate`: posição e valor juntos

No módulo 02, para saber a posição você faria `for i in range(len(medicoes))`. Funciona, mas é feio e fácil de errar. O `enumerate` entrega a dupla (posição, valor):

```
>>> medicoes = [50.2, 44.1, 51.0]
>>> for ponto, valor in enumerate(medicoes, start=1):
...     print(f"Ponto {ponto}: {valor} µm")
...
Ponto 1: 50.2 µm
Ponto 2: 44.1 µm
Ponto 3: 51.0 µm
```

`start=1` faz a contagem começar em 1, como o analista conta os pontos. Sem ele, começa em 0.

`zip`: andar por várias listas ao mesmo tempo

Duas colunas de uma planilha (OPs e pesos) viram duas listas. O `zip` junta o 1º com o 1º, o 2º com o 2º...

```
>>> ops = ["OP-4512", "OP-4513", "OP-4514"]
>>> pesos = [500.0, 320.5, 210.0]
>>> for op, peso in zip(ops, pesos, strict=True):
...     print(op, peso)
...
OP-4512 500.0
OP-4513 320.5
OP-4514 210.0
>>> dict(zip(ops, pesos, strict=True))
{'OP-4512': 500.0, 'OP-4513': 320.5, 'OP-4514': 210.0}
```

> ⚠️ **Erro comum:** listas de tamanhos diferentes. Sem `strict=True`, o `zip` **para na mais curta sem avisar**, e você perde dados em silêncio:
>
> ```
> >>> list(zip(["OP-1", "OP-2"], [500.0, 320.5, 210.0]))
> [('OP-1', 500.0), ('OP-2', 320.5)]
> ```
>
> Com `strict=True` (Python 3.10+), o Python reclama com `ValueError`. Prefira sempre ele; o ruff, que você vai usar no curso, até avisa quando falta. Se você não quer que o programa pare, confira `len()` das listas antes.

Exemplo completo: `exemplos/05_enumerate_zip.py`.

7. Dicionários: buscar pelo nome, não pela posição

Numa lista, você acha as coisas pela **posição**. Mas qual é o peso da OP-4513? Você teria que procurar item por item.

Um **dicionário** guarda pares **chave → valor**. Você acha o valor pela chave, na hora. É o **PROCV** do Python: você dá a chave, ele devolve o valor.

```
>>> peso_da_op = {"OP-4512": 500.0, "OP-4513": 320.5}
>>> peso_da_op["OP-4513"]
320.5
>>> len(peso_da_op)
2
```

Acrescentar, alterar e remover

A mesma sintaxe acrescenta (se a chave é nova) ou altera (se já existe):

```
>>> peso_da_op["OP-4514"] = 210.0
>>> peso_da_op["OP-4512"] = 510.0
>>> peso_da_op
{'OP-4512': 510.0, 'OP-4513': 320.5, 'OP-4514': 210.0}
>>> peso_da_op.pop("OP-4513")
320.5
>>> del peso_da_op["OP-4514"]
>>> peso_da_op
{'OP-4512': 510.0}
```

Desde o Python 3.7, o dicionário **lembra a ordem em que as chaves entraram**.

`[]` ou `get`?

Chave que não existe com `[]` dá erro. Com `get`, devolve `None` (ou um padrão que você escolher):

```
>>> peso_da_op["OP-9999"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'OP-9999'
>>> print(peso_da_op.get("OP-9999"))
None
>>> peso_da_op.get("OP-9999", 0.0)
```

```
0.0
>>> "OP-4512" in peso_da_op
True
```

Quando usar cada um:

- `dic[chave]` quando a chave **tem que existir**. Se não existir, é bug, e é bom o programa parar e avisar.
- `dic.get(chave)` quando faltar é **normal** (uma OP que ainda não tem resultado).
- `chave in dic` para perguntar se existe. O `in` olha as **chaves**, não os valores.

Percorrendo

```
>>> peso_da_op = {"OP-4512": 500.0, "OP-4513": 320.5}
>>> for op, peso in peso_da_op.items():
...     print(f"{op} -> {peso} kg")
...
OP-4512 -> 500.0 kg
OP-4513 -> 320.5 kg
>>> sum(peso_da_op.values())
820.5
```

`items()` entrega pares (chave, valor); desempacote direto no `for`. `keys()` e `values()` entregam só um dos lados. Um `for` direto no dicionário (`for op in peso_da_op`) passa pelas chaves.

O que pode ser chave?

Chaves precisam ser **imutáveis**: `str`, `int`, `float`, `tuple`. Uma tupla como `("A", 1)` (linha, ponto) é uma ótima chave. Uma lista não serve: `{["A", 1]: 50.2}` dá `TypeError: unhashable type: 'list'`, o jeito do Python dizer "isso pode mudar, então não serve de chave".

Os dois padrões que você vai usar a vida toda

Exemplo resolvido: contar defeitos por tipo. O inspetor anotou os defeitos de um lote, um por linha:

```
>>> defeitos = ["bolha", "risco", "bolha", "gel", "bolha", "risco"]
```

Quero saber quantos de cada. Como eu penso:

1. A resposta tem cara de "para cada defeito, um número". **"Para cada X, um Y" = dicionário.**

2. Começo com um dicionário vazio e passo por cada defeito.
3. Para cada um: se já existe no dicionário, somo 1; se não, começa em 1. O `get(defeito, 0)` faz as duas coisas: devolve o valor atual, ou 0 se ainda não existe.

```
>>> contagem = {}
>>> for defeito in defeitos:
...     contagem[defeito] = contagem.get(defeito, 0) + 1
...
>>> contagem
{'bolha': 3, 'risco': 2, 'gel': 1}
```

Padrão de agrupamento: em vez de contar, junto os valores de cada grupo numa lista.

```
>>> bobinas = [("A", 50.2), ("B", 20.1), ("A", 49.8), ("B", 19.9), ("A",
51.0)]
>>> por_linha = {}
>>> for linha, espessura in bobinas:
...     if linha not in por_linha:
...         por_linha[linha] = []
...     por_linha[linha].append(espessura)
...
>>> por_linha
{'A': [50.2, 49.8, 51.0], 'B': [20.1, 19.9]}
```

O `if ... not in` cria a lista vazia na primeira vez. Existe um atalho que faz o mesmo numa linha: `por_linha.setdefault(linha, []).append(espessura)`. O `setdefault` devolve a lista que já está lá, ou cria uma vazia e devolve.

> 🔧 **Modifique:** troque o padrão de agrupamento para **somar** as espessuras de cada linha em vez de guardá-las numa lista. (Dica: é parecido com a contagem.)

Juntando dicionários

No seu projeto, a OP pode ter uma especificação própria que **substitui** parte do padrão da linha. O operador `|` (Python 3.9+) junta dois dicionários; nas chaves repetidas, **vence o da direita**:

```
>>> padrao_da_linha = {"nominal": 50.0, "tol_pct": 10.0, "cof_max": 0.30}
>>> especificacao_da_op = {"tol_pct": 5.0}
>>> padrao_da_linha | especificacao_da_op
{'nominal': 50.0, 'tol_pct': 5.0, 'cof_max': 0.3}
```

Essa linha é, literalmente, a regra "use a especificação da OP; o que ela não define, vem do padrão da linha". Outra forma que você vai ver por aí: `{**padrao_da_linha, **especificacao_da_op}`.

> 🔮 **Preveja antes de rodar:** quantas chaves tem `estoque` no final, e qual o valor de `"L-101"`?
>
> ```
> estoque = {"L-101": 3}
> estoque["L-102"] = 5
> estoque["L-101"] = estoque.get("L-101", 0) + 2
> estoque["l-101"] = 1
> print(len(estoque), estoque["L-101"])
> ```

▼ **Resposta**

Imprime `3 5`. São 3 chaves: `"L-101"`, `"L-102"` e `"l-101"`. Para o Python, `"l-101"` (minúsculo) é outra chave. Por isso, ao usar textos digitados como chave, normalize antes (`.strip().upper()`). E `"L-101"` vale `3 + 2 = 5`.

Exemplo completo: `exemplos/06_dicionarios.py`.

8. Dicionários aninhados: a tabela de tolerâncias

O valor de um dicionário pode ser **qualquer coisa**, inclusive outro dicionário. É assim que você representa a tabela de tolerâncias: para cada linha, para cada propriedade, a regra.

```
>>> TOLERANCIAS = {
...     "A": {
...         "espessura": {"nominal": 50.0, "tol_pct": 10.0},
...         "gramatura": {"nominal": 46.0, "tol_pct": 5.0},
...         "cof": {"min": 0.15, "max": 0.30},
...     },
...     "B": {
...         "espessura": {"nominal": 20.0, "tol_pct": 8.0},
...         "cof": {"min": 0.20, "max": 0.40},
...     },
... }
```

Desenhando como árvore:

```
TOLERANCIAS
|
+-- "A"
|    +-- "espessura" --> {"nominal": 50.0, "tol_pct": 10.0}
|    +-- "gramatura" --> {"nominal": 46.0, "tol_pct": 5.0}
|    +-- "cof"       --> {"min": 0.15, "max": 0.30}
|
+-- "B"  (mesma ideia: espessura e cof)
```

Cada `[]` desce um nível:

```
>>> TOLERANCIAS["A"]["espessura"]["nominal"]
50.0
>>> regra = TOLERANCIAS["A"]["espessura"]
>>> minimo = round(regra["nominal"] * (1 - regra["tol_pct"] / 100), 2)
>>> maximo = round(regra["nominal"] * (1 + regra["tol_pct"] / 100), 2)
>>> minimo, maximo
(45.0, 55.0)
```

Guardar um nível numa variável (`regra`) evita linhas enormes como `TOLERANCIAS["A"]["espessura"]["tol_pct"]` repetidas.

Para acessar com segurança um caminho que pode não existir, encadeie `get` usando `{}` como padrão:

```
>>> print(TOLERANCIAS.get("E", {}).get("espessura"))
None
>>> print(TOLERANCIAS.get("B", {}).get("gramatura"))
None
```

Se a linha `"E"` não existe, o primeiro `get` devolve `{}`, e o segundo `get` num dicionário vazio devolve `None`. Sem erro.

Para percorrer dois níveis, use um `for` dentro de outro: `for linha, propriedades in TOLERANCIAS.items():` e, dentro dele, `for nome, regra in propriedades.items():`. O exemplo `07` mostra isso rodando.

> 💡 **Dica:** isso que você acabou de escrever é praticamente um arquivo **JSON**. Quando, no módulo 06, você salvar as regras num arquivo `regras.json`, e no módulo 12 receber dados de uma API, a estrutura vai ser exatamente esta: dicionários e listas aninhados.

Exemplo completo: `exemplos/07_dicionarios_aninhados.py`.

9. Conjuntos: sem repetição, com matemática

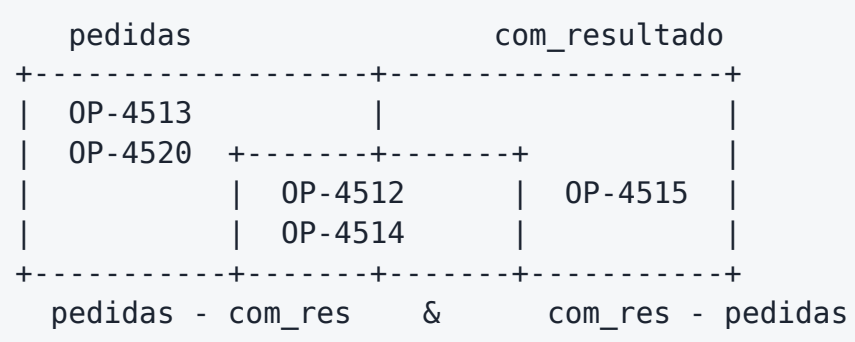
Um **conjunto** (`set`) é uma coleção **sem itens repetidos** e **sem ordem**. Ele responde muito rápido à pergunta "isto está aqui?".

```
>>> lidos = ["L-101", "L-102", "L-101", "L-103", "L-102"]
>>> unicos = set(lidos)
>>> len(unicos)
3
>>> sorted(unicos)
['L-101', 'L-102', 'L-103']
>>> "L-102" in unicos
True
```

> ⚠️ **Erro comum:** `{}` cria um **dicionário** vazio, não um conjunto. Conjunto vazio é `set()`.

A grande vantagem são as operações entre conjuntos. Imagine as OPs que a expedição pediu e as OPs que já têm resultado no laboratório:

```
>>> pedidas = {"OP-4512", "OP-4513", "OP-4514", "OP-4520"}
>>> com_resultado = {"OP-4512", "OP-4514", "OP-4515"}
>>> sorted(pedidas & com_resultado)
['OP-4512', 'OP-4514']
>>> sorted(pedidas - com_resultado)
['OP-4513', 'OP-4520']
>>> sorted(com_resultado - pedidas)
['OP-4515']
>>> sorted(pedidas | com_resultado)
['OP-4512', 'OP-4513', 'OP-4514', 'OP-4515', 'OP-4520']
```



Operador	Nome	Pergunta que responde
`a & b`	interseção	o que está nos dois?
`a | b`	união	o que está em pelo menos um?

Operador	Nome	Pergunta que responde
`a - b`	diferença	o que está em `a` e não em `b`?
`a ^ b`	diferença simétrica	o que está em só um dos dois?

Para modificar: `conjunto.add(x)`, `conjunto.remove(x)` (dá `KeyError` se não existir) e `conjunto.discard(x)` (não reclama).

> 💡 **Dica:** um conjunto não tem ordem garantida. A ordem em que ele aparece no `print` pode mudar de uma execução para outra. Sempre que for **mostrar** um conjunto, use `sorted(...)`.

> 🔮 **Preveja antes de rodar:** o que cada linha imprime?
>
> ```
> print(len(set("bobina")))
> print(sorted({"A", "b", "a", "B "}))
> ```

▼ **Resposta**

```
5
['A', 'B ', 'a', 'b']
```

- `set("bobina")` guarda as letras sem repetir: `b`, `o`, `i`, `n`, `a`. O segundo `b` não entra. São 5.
- Para o conjunto, `"A"` e `"a"` são diferentes, e `"B "` (com espaço) também é diferente de `"B"`. Nada foi eliminado. E o `sorted` põe maiúsculas antes de minúsculas. Se você queria as linhas A e B, normalize antes: `{x.strip().upper() for x in ...}`.

Exemplo completo: `exemplos/08_conjuntos.py`.

10. Compreensões: montar coleções numa linha

Você vai escrever muito este padrão: "crie uma lista nova a partir de outra, transformando e/ou filtrando".

```
>>> espessuras_um = [50.2, 44.1, 51.0, 56.3, 49.8]
>>> em_mm = []
>>> for valor in espessuras_um:
```

```
...     em_mm.append(round(valor / 1000, 4))
...
>>> em_mm
[0.0502, 0.0441, 0.051, 0.0563, 0.0498]
```

A **compreensão de lista** faz o mesmo numa linha:

```
>>> [round(valor / 1000, 4) for valor in espessuras_um]
[0.0502, 0.0441, 0.051, 0.0563, 0.0498]
```

Leia assim: "**o que guardar**, para cada item, de onde vem, (se passar no filtro)".

```
 [ round(v / 1000, 4)   for v in espessuras_um   if v > 45 ]
   \_________________/    \____________________/   \_______/
     o que guardar          de onde vem           filtro
                                                  (opcional)
```

Com filtro:

```
>>> [valor for valor in espessuras_um if valor < 45 or valor > 55]
[44.1, 56.3]
```

Com expressão condicional (módulo 02) na parte do "o que guardar". Aqui **todos** entram, transformados:

```
>>> ["ok" if 45 <= valor <= 55 else "fora" for valor in espessuras_um]
['ok', 'fora', 'ok', 'fora', 'ok']
```

> ⚠️ **Erro comum:** confundir os dois `if`. O `if` no **fim** filtra (alguns itens não entram). O `if ... else` no **começo** transforma (todos entram). O `if` do fim não aceita `else`.

De dicionário e de conjunto

Trocando os colchetes por chaves, você monta dicionários e conjuntos:

```
>>> ops = ["OP-4512", "OP-4513", "OP-4514"]
>>> pesos = [500.0, 320.5, 210.0]
>>> peso_da_op = {op: peso for op, peso in zip(ops, pesos, strict=True)}
>>> {op: peso for op, peso in peso_da_op.items() if peso > 300}
{'OP-4512': 500.0, 'OP-4513': 320.5}
>>> linhas = {linha.strip().upper() for linha in ["a", "A ", " b", "B",
"c"]}
```

```
>>> sorted(linhas)
['A', 'B', 'C']
```

Quando NÃO usar compreensão

- Quando a lógica tem **vários passos** ou mais de uma condição complicada. Um `for` com nomes claros é melhor.
- Quando o objetivo é um **efeito** (como `print`), não montar uma lista. `[print(x) for x in lista]` funciona, mas cria uma lista inútil de `None`. Use `for`.
- Quando ela não cabe numa linha legível. Compreensão é para deixar o código **mais claro**, não mais curto.

> 🔧 **Modifique:** em `exemplos/09_compreensoes.py`, crie a lista `fora_com_posicao` com tuplas (posição, valor) das espessuras fora de 45–55, usando `enumerate(..., start=1)` dentro da compreensão. (É o exercício 06 quase pronto.)

11. Funções embutidas que economizam laços

Muitos laços do módulo 02 viram uma chamada de função:

```
>>> medicoes = [50.2, 49.8, 51.0, 50.5, 48.9]
>>> round(sum(medicoes) / len(medicoes), 2)
50.08
>>> min(medicoes), max(medicoes)
(48.9, 51.0)
>>> any(valor < 49 for valor in medicoes)
True
>>> all(45 <= valor <= 55 for valor in medicoes)
True
```

- `any(...)`: **algum** item satisfaz? (para no primeiro `True`)
- `all(...)`: **todos** satisfazem? (para no primeiro `False`)

O que vai dentro do `any` e do `all` é uma **expressão geradora**: parece uma compreensão sem os colchetes. Ela produz os valores um por um, sem montar uma lista inteira na memória. Também funciona com `sum`: `sum(peso for peso in pesos if peso > 300)`.

> ⚠️ **Erro comum:** `max([])` e `min([])` dão `ValueError`, porque não existe maior item numa lista vazia. Use `max(lista, default=None)` ou confira antes com `if`

> `lista:`.

`sorted`, `min` e `max` com `key`

E se você quer ordenar bobinas pelo **peso**, e não pela OP? O parâmetro `key` recebe uma função que diz **o que comparar** em cada item.

A forma mais curta de escrever essa função é uma **lambda**: `lambda bobina: bobina["peso_kg"]` é uma função de uma linha, sem nome, que recebe `bobina` e devolve `bobina["peso_kg"]`. O módulo 04 explica lambdas a fundo.

```
>>> bobinas = [
...     {"op": "OP-4513", "peso_kg": 320.5},
...     {"op": "OP-4512", "peso_kg": 500.0},
...     {"op": "OP-4514", "peso_kg": 210.0},
... ]
>>> por_peso = sorted(bobinas, key=lambda bobina: bobina["peso_kg"])
>>> [b["op"] for b in por_peso]
['OP-4514', 'OP-4513', 'OP-4512']
>>> max(bobinas, key=lambda bobina: bobina["peso_kg"])["op"]
'OP-4512'
>>> min([50.2, 49.8, 51.0, 48.9], key=lambda valor: abs(valor - 49.0))
48.9
```

O último exemplo pergunta: "qual medição está **mais perto** de 49.0?". A key transforma cada valor na distância até 49.0; o `min` escolhe a menor distância, mas devolve o **valor original**.

Exemplo resolvido: ranking com desempate. Quero as bobinas da maior espessura para a menor; se empatar, pela OP em ordem crescente.

```
>>> amostras = [("OP-4513", 50.4), ("OP-4512", 51.0), ("OP-4511", 50.4)]
```

Como eu penso:

1. `reverse=True` inverte **tudo**, inclusive o desempate (as OPs sairiam de trás para frente). Não serve.
2. A key pode devolver uma **tupla**. Tuplas comparam o 1º item; se empatar, o 2º.
3. Para a espessura ficar decrescente e a OP crescente, uso o **negativo** da espessura: o maior valor vira o menor número.

```
>>> sorted(amostras, key=lambda par: (-par[1], par[0]))
[('OP-4512', 51.0), ('OP-4511', 50.4), ('OP-4513', 50.4)]
```

O exercício 09 usa exatamente esse raciocínio com as notas de uma turma.

Juntando textos de uma lista

Um último método, de string, que você vai querer para montar relatórios: `separador.join(lista_de_textos)` junta os textos colocando o separador entre eles. O módulo 05 aprofunda.

```
>>> ", ".join(["espessura", "cof"])
'espessura, cof'
```

Exemplo completo: `exemplos/10_funcoes_embutidas.py`.

12. Lista de dicionários: sua primeira tabela

Agora vem a ideia que liga este módulo ao resto da sua carreira com dados.

Uma planilha é um monte de **linhas**, e cada linha tem as mesmas **colunas**. Em Python, a forma mais natural de representar isso é uma **lista de dicionários**: cada dicionário é uma linha; as chaves são os nomes das colunas.

```
>>> amostras = [
...     {"op": "OP-4512", "linha": "A", "espessura": 50.4, "cof": 0.22},
...     {"op": "OP-4513", "linha": "B", "espessura": 20.9, "cof": 0.31},
...     {"op": "OP-4514", "linha": "A", "espessura": 56.1, "cof": 0.25},
...     {"op": "OP-4515", "linha": "C", "espessura": 35.2, "cof": 0.18},
...     {"op": "OP-4516", "linha": "A", "espessura": 49.7, "cof": 0.27},
... ]
```

```
                colunas (as chaves) -->
                "op"       "linha"  "espessura"  "cof"
amostras[0] --> OP-4512    A        50.4         0.22
amostras[1] --> OP-4513    B        20.9         0.31
amostras[2] --> OP-4514    A        56.1         0.25
amostras[3] --> OP-4515    C        35.2         0.18
amostras[4] --> OP-4516    A        49.7         0.27
```

Pegar um valor é "linha, depois coluna": `amostras[2]["espessura"]` é `56.1`.

Com o que você já sabe, dá para fazer **tudo** que um analista faz numa tabela. E repare na coluna da direita: é o mesmo raciocínio, com outra sintaxe, que você vai usar no SQL (módulo 11) e no pandas (módulo 10).

Operação	Python puro (este módulo)	SQL	pandas
Filtrar linhas	`[a for a in amostras if a["linha"] == "A"]`	`WHERE linha = 'A'`	`df[df["linha"] == "A"]`
Pegar uma coluna	`[a["espessura"] for a in amostras]`	`SELECT espessura`	`df["espessura"]`
Ordenar	`sorted(amostras, key=lambda a: a["espessura"])`	`ORDER BY espessura`	`df.sort_values("espessura")`
Agregar	`sum(...) / len(...)`	`AVG(espessura)`	`df["espessura"].mean()`
Agrupar	dicionário de listas (seção 7)	`GROUP BY linha`	`df.groupby("linha")`
Nova coluna	`{**a, "status": ...}`	`SELECT *, ... AS status`	`df["status"] = ...`
Juntar tabelas	índice com dicionário	`JOIN`	`pd.merge(...)`

Filtrar e escolher colunas

```
>>> linha_a = [a for a in amostras if a["linha"] == "A"]
>>> [a["op"] for a in linha_a]
['OP-4512', 'OP-4514', 'OP-4516']
>>> [a["espessura"] for a in amostras]
[50.4, 20.9, 56.1, 35.2, 49.7]
>>> [(a["op"], a["cof"]) for a in amostras if a["cof"] > 0.30]
[('OP-4513', 0.31)]
```

Ordenar e mostrar como tabela

```
>>> ordenadas = sorted(amostras, key=lambda a: a["espessura"],
reverse=True)
>>> for a in ordenadas[:3]:
...     print(f"{a['op']:<9}{a['linha']:<3}{a['espessura']:>6.1f}")
...
OP-4514  A    56.1
OP-4512  A    50.4
OP-4516  A    49.7
```

Na f-string, `:<9` alinha à esquerda ocupando 9 caracteres e `:>6.1f` alinha à direita em 6 caracteres com 1 casa decimal. É o que deixa as colunas retinhas.

Exemplo resolvido: média de espessura por linha (GROUP BY)

Problema: "qual a espessura média de cada linha de produção?"

Como eu penso, passo a passo:

1. **Que forma tem a resposta?** "Para cada linha, um número" → um dicionário `{"A": ..., "B": ...}`.
2. **Dá para fazer em um passo só?** Não fácil. Divido em dois: primeiro **agrupo** as espessuras de cada linha; depois **calculo** a média de cada grupo.
3. **Passo 1, agrupar:** é o padrão de agrupamento da seção 7.
4. **Passo 2, resumir:** uma compreensão de dicionário que transforma cada lista em média.
5. **Testo** com os dados que tenho e confiro uma conta de cabeça: linha A tem 50.4, 56.1 e 49.7; média ≈ 52.07.

```
>>> grupos = {}
>>> for a in amostras:
...     grupos.setdefault(a["linha"], []).append(a["espessura"])
...
>>> grupos
{'A': [50.4, 56.1, 49.7], 'B': [20.9], 'C': [35.2]}
>>> {linha: round(sum(v) / len(v), 2) for linha, v in grupos.items()}
{'A': 52.07, 'B': 20.9, 'C': 35.2}
```

Confere com a conta de cabeça. Esse "agrupar, depois resumir" é exatamente o que o `GROUP BY` do SQL e o `groupby` do pandas fazem por baixo. O exercício 11 pede a versão completa (quantidade, média, mínimo e máximo, ignorando células vazias).

> 🔧 **Modifique:** mude o código para calcular a média do **COF** por linha. Depois, para contar **quantas amostras** cada linha tem.

Nova coluna sem estragar a tabela original

Para acrescentar uma coluna `status`, crie dicionários **novos**. `{**a, "status": ...}` copia todas as chaves de `a` e acrescenta (ou substitui) `status`:

```
>>> com_status = [
...     {**a, "status": "ok" if a["cof"] <= 0.30 else "COF alto"} for a
in amostras
... ]
>>> [(a["op"], a["status"]) for a in com_status][:2]
[('OP-4512', 'ok'), ('OP-4513', 'COF alto')]
```

```
>>> "status" in amostras[0]
False
```

Se você fizesse `a["status"] = ...` dentro de um `for`, estaria alterando a tabela original (seção 4). Às vezes é isso que você quer; na dúvida, crie uma nova.

Exemplo resolvido: juntar duas tabelas (JOIN)

A expedição mandou os pedidos; o laboratório tem os resultados. Quero, para cada pedido, a linha e o COF da amostra.

```
>>> pedidos = [
...     {"op": "OP-4514", "nf": "000123", "peso_kg": 500.0},
...     {"op": "OP-4599", "nf": "000124", "peso_kg": 80.5},
... ]
```

Como eu penso:

1. Primeira ideia: para cada pedido, percorrer **todas** as amostras procurando a OP. Funciona, mas é um laço dentro de outro. Com 1.000 pedidos e 1.000 amostras, são 1.000.000 comparações.
2. Ideia melhor: montar **uma vez** um dicionário `OP → amostra` (um "PROCV pronto") e depois só consultar com `get`. Cada consulta é instantânea.
3. Pedido sem amostra? `get` devolve `None`, e eu trato esse caso.

```
>>> indice = {a["op"]: a for a in amostras}
>>> for pedido in pedidos:
...     amostra = indice.get(pedido["op"])
...     if amostra is None:
...         print(pedido["op"], "-> sem resultado no laboratório")
...     else:
...         print(pedido["op"], "-> linha", amostra["linha"], "| COF",
amostra["cof"])
...
OP-4514 -> linha A | COF 0.25
OP-4599 -> sem resultado no laboratório
```

Isso é um **LEFT JOIN**: todos os pedidos aparecem, com ou sem resultado. O exercício 13 pede a versão completa, com OPs digitadas de jeitos diferentes.

Linhas ou colunas?

Existe outro formato para a mesma tabela: um **dicionário de listas**, uma lista por coluna, como `colunas = {"op": ["OP-4512", "OP-4513"], "espessura": [50.4, 20.9]}`. O pandas aceita os dois formatos: `pd.DataFrame(amostras)` e `pd.DataFrame(colunas)` geram a mesma tabela. Lista de dicionários é mais natural para **ler e processar linha a linha** (é o que o `csv.DictReader` do módulo 06 entrega). Dicionário de listas é mais natural para **cálculos por coluna**.

Exemplo completo: `exemplos/11_tabela_lista_de_dicts.py`.

13. Qual coleção usar?

	Lista `[]`	**Tupla `()`**	**Dicionário `{k: v}`**	**Conjunto `{}` / `set()`**
Tem ordem?	sim	sim	sim (de inserção)	não
Pode mudar?	sim	não	sim	sim
Aceita repetidos?	sim	sim	chaves não	não
Você acessa por...	posição	posição	chave	só pergunta "está aqui?"
Use para...	sequência de itens do mesmo tipo	registro fixo, vários retornos	buscar por nome, contar, agrupar	tirar repetidos, cruzar grupos
Exemplo	medições de uma bobina	`(mínimo, máximo)`	peso de cada OP	OPs pedidas

Na dúvida, faça a pergunta que você vai fazer aos dados:

- "Qual é o 3º?" → lista.
- "Qual é o valor de X?" → dicionário.
- "X está aqui? O que tem em A e não tem em B?" → conjunto.
- "Estes valores andam sempre juntos e não mudam?" → tupla.

Lendo mensagens de erro deste módulo

Um traceback se lê **de baixo para cima**: a última linha diz **o que** deu errado; as de cima dizem **onde**. Desde o Python 3.11, os `~~~^^^` apontam o pedaço exato da linha.

1. `KeyError`: chave que não existe.

```
Traceback (most recent call last):
  File "C:\curso-python\conferencia.py", line 5, in <module>
    minimo, maximo = tolerancias[amostra["linha"]]
                     ~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
KeyError: 'E'
```

Leitura: na linha 5, o dicionário `tolerancias` não tem a chave `'E'`. Pergunte-se: a chave deveria existir (então o dado está errado ou o dicionário está incompleto) ou faltar é normal (então use `get` e trate o `None`)? Repare também se não é diferença de maiúscula ou espaço (`"e"`, `"E "`).

2. `TypeError: 'NoneType' object is not subscriptable`.

```
Traceback (most recent call last):
  File "C:\curso-python\medias.py", line 3, in <module>
    print(medicoes[0])
          ~~~~~~~~^^^
TypeError: 'NoneType' object is not subscriptable
```

Leitura: `medicoes` vale `None`, e `None` não tem índice. Quase sempre é um `medicoes = medicoes.sort()` (ou `.append`, `.reverse`) mais acima. Procure a linha que **atribuiu** a variável, não a linha do erro.

Outros que você vai encontrar:

Mensagem	Causa provável	Correção
`IndexError: list index out of range`	índice passou do último (`len(lista) - 1`), comum em `for i in range(...)`	percorra os itens direto, ou use `enumerate`
`AttributeError: 'NoneType' object has no attribute 'append'`	a variável recebeu o `None` de um método como `sort()`	não atribua o resultado de métodos que modificam no lugar
`TypeError: unhashable type: 'list'`	lista usada como chave de dicionário ou item de conjunto	use uma tupla

Mensagem	Causa provável	Correção
`TypeError: 'tuple' object does not support item assignment`	tentou mudar uma tupla	crie uma tupla nova, ou use lista
`ValueError: list.remove(x): x not in list`	`remove` de valor que não está na lista	confira com `in` antes
`ValueError: too many values to unpack (expected 2)`	mais itens do que variáveis no desempacotamento	confira o formato do dado (ou use `*resto`)
`ValueError: max() iterable argument is empty`	`max` / `min` de lista vazia	`max(lista, default=None)`
`RuntimeError: dictionary changed size during iteration`	acrescentou/removeu chaves dentro de um `for` sobre o dicionário	percorra `list(dic)` ou monte um dicionário novo
`TypeError: '<' not supported between instances of 'str' and 'float'`	`sort` / `sorted` / `max` numa lista com textos e números misturados	converta tudo para o mesmo tipo antes
`TypeError: 'list' object is not callable`	você criou uma variável chamada `list` (ou `dict`, `sum`...) e depois tentou usar a função	renomeie a variável

> 💡 **Dica:** as mensagens podem mudar um pouco entre versões do Python. O tipo do erro (a palavra antes dos dois-pontos) não muda, e é por ele que você começa.

Boas práticas

- **Nomes no plural para coleções, singular para o item:** `for medicao in medicoes`, `for amostra in amostras`. O código se lê como português.
- **Nunca remova itens da lista que você está percorrendo.** Monte uma lista nova com o que fica.
- **Não altere a coleção que você recebeu**, a não ser que esse seja o objetivo declarado. Devolva uma nova.
- **`dic[chave]` quando faltar é bug; `dic.get(chave)` quando faltar é normal.**

- **Normalize textos antes de usar como chave** (`.strip().upper()`): `"a "` e `"A"` são chaves diferentes.
- **Use `zip(..., strict=True)`** para não perder dados em silêncio.
- **Compreensões curtas.** Se não cabe numa linha legível, vire um `for`.
- **Mantenha as "linhas" da tabela com as mesmas chaves.** Se uma coluna pode faltar, decida o padrão (`None`) e use `get`.
- **Não use nomes de funções embutidas como variável:** `list`, `dict`, `set`, `sum`, `max`, `min`, `id`, `type`. O Python deixa, e depois a função "some".

No mundo real

Fim de turno. A planilha do dia tem as amostras das linhas A a D. A expedição mandou por e-mail as OPs que saem amanhã. Você precisa: conferir cada amostra contra as tolerâncias da linha, respeitar a observação "COF: liberado" e montar a lista de certificados a emitir.

O script `exemplos/12_mundo_real_conferencia.py` faz isso só com o que você aprendeu neste módulo. No começo do arquivo ficam os dados:

- `TOLERANCIAS`: dicionário aninhado. Para cada linha (A a D), cada propriedade tem uma tupla: `(nominal, %)` para espessura e gramatura, `(mín, máx)` para COF. Exemplo: `"A": {"espessura": (50.0, 10.0), "gramatura": (46.0, 5.0), "cof": (0.15, 0.30)}`.
- `PERCENTUAIS = {"espessura", "gramatura"}`: o conjunto das propriedades que usam nominal ± %.
- `AMOSTRAS`: a planilha do dia como lista de dicionários, montada com `dict(zip(COLUNAS, linha, strict=True))` a partir de tuplas, como se viesse de um CSV. Cada amostra tem `op`, `linha`, `espessura`, `gramatura`, `cof` e `obs` (a observação, que às vezes diz "COF: liberado").
- `OPS_PEDIDAS`: as OPs do e-mail da expedição, com uma repetida.

O miolo do programa:

```
def faixa(propriedade, regra):
    """Converte a regra em (mínimo, máximo)."""
    if propriedade in PERCENTUAIS:
        nominal, pct = regra
        return round(nominal * (1 - pct / 100), 2), round(nominal * (1 +
pct / 100), 2)
    return regra  # já é (mínimo, máximo)


def conferir(amostra):
    """Devolve uma cópia da amostra com as colunas 'falhas' e
```

```
'status'."""
    regras = TOLERANCIAS[amostra["linha"]]
    falhas = []
    for propriedade, regra in regras.items():
        minimo, maximo = faixa(propriedade, regra)
        if not minimo <= amostra[propriedade] <= maximo:
            falhas.append(propriedade)

    if not falhas:
        status = "APROVADA"
    elif falhas == ["cof"] and "cof: liberado" in amostra["obs"].lower():
        status = "LIBERADA"  # COF fora, mas liberado pela qualidade
    else:
        status = "REPROVADA"
    return {**amostra, "falhas": falhas, "status": status}


conferidas = [conferir(amostra) for amostra in AMOSTRAS]

print("== Conferência do dia ==")
for c in conferidas:
    detalhe = ", ".join(c["falhas"]) if c["falhas"] else "-"
    print(f"{c['op']}  linha {c['linha']}  {c['status']:<9}  fora:
{detalhe}")

print("\n== Resumo por linha ==")
for linha in sorted(TOLERANCIAS):
    da_linha = [c for c in conferidas if c["linha"] == linha]
    ok = sum(c["status"] != "REPROVADA" for c in da_linha)
    print(f"Linha {linha}: {ok}/{len(da_linha)} amostras liberadas")

liberadas = {c["op"] for c in conferidas if c["status"] != "REPROVADA"}
reprovadas = {c["op"] for c in conferidas if c["status"] == "REPROVADA"}
pedidas = set(OPS_PEDIDAS)
com_resultado = liberadas | reprovadas

print("\n== Expedição ==")
print("Emitir certificado:", sorted(pedidas & liberadas))
print("Bloqueadas:        ", sorted(pedidas & reprovadas))
print("Sem resultado:     ", sorted(pedidas - com_resultado))
```

Saída:

```
== Conferência do dia ==
OP-4512  linha A  APROVADA   fora: -
OP-4513  linha B  REPROVADA  fora: espessura
OP-4514  linha A  LIBERADA   fora: cof
OP-4515  linha C  APROVADA   fora: -
OP-4516  linha D  REPROVADA  fora: espessura, gramatura
```

```
OP-4517  linha D  APROVADA   fora: -

== Resumo por linha ==
Linha A: 2/2 amostras liberadas
Linha B: 0/1 amostras liberadas
Linha C: 1/1 amostras liberadas
Linha D: 1/2 amostras liberadas

== Expedição ==
Emitir certificado: ['OP-4512', 'OP-4514']
Bloqueadas:         ['OP-4516']
Sem resultado:      ['OP-4520']
```

Repare no que cada coleção está fazendo:

- **dicionário aninhado** para as regras (busca pela linha, depois pela propriedade);
- **tuplas** para registros fixos (`(nominal, %)`, `(mín, máx)`, as linhas da planilha);
- **lista de dicionários** para a tabela de amostras, e uma **nova** tabela com as colunas `falhas` e `status`;
- **conjuntos** para cruzar o que a expedição pediu com o que o laboratório liberou.

Nos próximos módulos, isso cresce: as funções ficam melhores (04), a observação vira texto analisado com regex (05), os dados vêm de arquivos de verdade (06 e 10) e as regras vão para um banco (11). A lógica, você já tem.

Resumo

Quero...	Sintaxe
criar uma lista	`medicoes = [50.2, 49.8]`
primeiro / último item	`lista[0]` / `lista[-1]`
trecho	`lista[ini:fim]` (fim não entra), `lista[-3:]`, `lista[::-1]`
acrescentar / inserir	`lista.append(x)` / `lista.insert(i, x)`
remover por valor / por posição	`lista.remove(x)` / `lista.pop(i)`
ordenar no lugar / ordenada nova	`lista.sort()` / `sorted(lista)`
copiar (rasa / profunda)	`lista.copy()` ou `lista[:]` / `copy.deepcopy(obj)`
mesmo objeto? mesmo conteúdo?	`a is b` / `a == b`

Quero...	Sintaxe
tupla e desempacotamento	`t = ("OP-1", 500.0)`; `op, peso = t`; `a, *resto = lista`
posição + valor	`for i, v in enumerate(lista, start=1)`
andar em várias listas	`for a, b in zip(lista1, lista2, strict=True)`
criar dicionário	`d = {"OP-1": 500.0}`; `dict(zip(chaves, valores, strict=True))`
ler com segurança	`d.get(chave)`, `d.get(chave, padrao)`
percorrer dicionário	`for chave, valor in d.items()`
contar	`d[k] = d.get(k, 0) + 1`
agrupar	`d.setdefault(k, []).append(v)`
juntar dicionários	`padrao | especifico`
conjunto	`set(lista)`; `a & b`, `a | b`, `a - b`; vazio: `set()`
compreensões	`[x for x in xs if ...]`, `{k: v for ...}`, `{x for ...}`
agregar	`sum`, `min`, `max`, `len`, `any`, `all`
ordenar por critério	`sorted(xs, key=lambda x: x["peso"], reverse=True)`
filtrar tabela	`[r for r in tabela if r["linha"] == "A"]`
juntar tabelas	`indice = {r["op"]: r for r in tabela}`; `indice.get(op)`

Você domina este módulo se...

- [ ] Consigo explicar, desenhando etiquetas e caixas, por que `b = a` seguido de `b.append(1)` muda `a`, e como evitar isso.
- [ ] Sei a diferença entre cópia rasa e profunda e consigo dar um exemplo em que a rasa não basta.
- [ ] Prevejo sem rodar o resultado de `lista[-3:]`, `lista[::-1]` e `lista[-0:]`.
- [ ] Escolho entre lista, tupla, dicionário e conjunto justificando pela pergunta que vou fazer aos dados.
- [ ] Escrevo de memória os padrões de contagem e de agrupamento com dicionário.
- [ ] Transformo um `for` com `append` numa compreensão (e sei quando não vale a pena).
- [ ] Filtro, ordeno, agrupo e junto uma lista de dicionários, e sei o nome dessas operações em SQL.
- [ ] Leio um `KeyError` ou um `'NoneType' object is not subscriptable` e sei onde procurar a causa.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_ultimas_medicoes`	★☆☆	fatiamento, índice negativo, caso `n = 0`
02	`ex02_resumo_espessura`	★☆☆	`sum`, `min`, `max`, `len`, montar dicionário
03	`ex03_dentro_da_tolerancia`	★☆☆	`dict.get`, desempacotar tupla, normalizar chave
04	`ex04_contar_votos`	★☆☆	padrão de contagem, `max` com `key`
05	`ex05_remover_duplicadas`	★☆☆	conjunto como "já vi", manter a ordem
06	`ex06_fora_da_faixa`	★★☆	`enumerate`, compreensão com filtro, percentual
07	`ex07_pesos_por_op`	★★☆	`zip`, acumular em dicionário, validar tamanhos
08	`ex08_leitura_copia_referencia`	★★☆	**leitura de código:** prever saídas e consertar um bug de referência
09	`ex09_ranking_alunos`	★★☆	`sorted` com `key` de tupla, desempate
10	`ex10_conferir_ops`	★★☆	operações de conjunto
11	`ex11_resumo_por_linha`	★★★	lista de dicionários, agrupar e resumir (GROUP BY)
12	`ex12_revisao_tolerancias`	★★★	**revisão** 01+02+03: dicionário aninhado, f-strings, laços
13	`ex13_revisao_juntar_tabelas`	★★★	**revisão** 01+02+03: JOIN com índice, normalizar textos

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError` e rode, na pasta `curso-python`:

```
python curso.py testar 03 ex01
```

Para testar o módulo todo: `python curso.py testar 03`. Para ver seu progresso: `python curso.py progresso`.

> 💡 **Dica:** antes de olhar a solução (`solucoes/`), rode os exemplos do enunciado no REPL e leia a mensagem do teste que falhou. Ela diz o que era esperado e o que seu código devolveu.

Desafio do módulo

Painel de conferência do lote. Você recebe a "planilha" do dia (lista de dicionários), as tolerâncias por linha (dicionário aninhado) e as OPs pedidas pela expedição. Seu programa imprime a tabela conferida, o resumo por linha, as maiores não conformidades e a fila de certificados.

Enunciado completo, requisitos e critérios de aceite: desafio/README.md.

Quiz

Teste o que ficou: quiz.md (15 perguntas, gabarito no final).

Para ir além

- Tutorial oficial, estruturas de dados: docs.python.org/pt-br/3/tutorial/datastructures.html
- Tipos embutidos (listas, tuplas, dicionários, conjuntos): docs.python.org/pt-br/3/library/stdtypes.html
- Ordenação, com `key` e desempates: docs.python.org/pt-br/3/howto/sorting.html
- Módulo `copy` (cópia rasa e profunda): docs.python.org/pt-br/3/library/copy.html
- Leitura extra: *Facts and myths about Python names and values*, de Ned Batchelder (em inglês, a melhor explicação de "etiquetas"): nedbatchelder.com/text/names.html
- Ferramenta: pythontutor.com desenha as setas de referência enquanto o código roda. Use sempre que a seção 4 parecer mágica.

Enunciados dos exercícios

Exercício 03.01 — Últimas medições

Arquivo: `exercicios/ex01_ultimas_medicoes.py`

```
Nível: ★☆☆

Contexto: O painel do laboratório mostra só as últimas medições de
espessura de
cada linha. Você recebe a lista completa, da mais antiga para a mais
recente.

Tarefa: implemente `ultimas_medicoes(medicoes, n)`, que devolve uma lista
```

```
NOVA
com as `n` últimas medições, na ordem original.
- Se `n` for 0 ou negativo, devolva uma lista vazia.
- Se `n` for maior que o tamanho da lista, devolva todas as medições.
- Não modifique a lista recebida.

Exemplos:
    >>> ultimas_medicoes([50.2, 49.8, 51.0, 50.5], 2)
    [51.0, 50.5]
    >>> ultimas_medicoes([50.2, 49.8], 5)
    [50.2, 49.8]
    >>> ultimas_medicoes([50.2, 49.8], 0)
    []

Dica: fatiamento com índice negativo resolve quase tudo. Mas teste no
REPL o que
`[1, 2, 3][-0:]` devolve antes de confiar nele.

Teste com:  python curso.py testar 03 ex01
```

Exercício 03.02 — Resumo de espessura

Arquivo: `exercicios/ex02_resumo_espessura.py`

```
Nível: ★☆☆

Contexto: No fim de cada turno, o laboratório anota um resumo das
medições de
espessura (µm) de uma bobina: menor valor, maior valor, média e
amplitude.

Tarefa: implemente `resumo_espessura(medicoes)`, que devolve um
dicionário com
as chaves exatamente nesta forma:
    {"minimo": ..., "maximo": ..., "media": ..., "amplitude": ...}
- "media" e "amplitude" (máximo − mínimo) arredondadas com round(..., 2).
- Lista vazia: devolva um dicionário vazio {}.

Exemplos:
    >>> resumo_espessura([50.2, 49.8, 51.0, 50.6])
    {'minimo': 49.8, 'maximo': 51.0, 'media': 50.4, 'amplitude': 1.2}
    >>> resumo_espessura([])
    {}

Dica: sum, min, max e len fazem quase todo o trabalho. Sem round, a
amplitude
acima sairia 1.2000000000000028 (lembra do problema do float no módulo
01?).
```

```
Teste com:  python curso.py testar 03 ex02
```

Exercício 03.03 — Dentro da tolerância?

Arquivo: `exercicios/ex03_dentro_da_tolerancia.py`

```
Nível: ★☆☆

Contexto: Cada linha de produção tem uma faixa de espessura aceitável
(mín, máx),
guardada num dicionário. Na planilha, o nome da linha às vezes vem
digitado em
minúscula ou com espaços (" a ").

Tarefa: implemente `dentro_da_tolerancia(valor, linha, tolerancias)`.
- `tolerancias` é um dicionário  linha -> (mínimo, máximo), por exemplo
  {"A": (45.0, 55.0), "B": (18.0, 22.0)}.
- Limpe o nome da linha (tire espaços e passe para maiúscula) antes de
procurar.
- Devolva True se mínimo <= valor <= máximo (os limites contam como
dentro).
- Devolva False se o valor estiver fora da faixa.
- Devolva None se a linha não existir no dicionário.

Exemplos:
    >>> tolerancias = {"A": (45.0, 55.0), "B": (18.0, 22.0)}
    >>> dentro_da_tolerancia(50.2, "A", tolerancias)
    True
    >>> dentro_da_tolerancia(23.5, " b ", tolerancias)
    False
    >>> print(dentro_da_tolerancia(30.0, "Z", tolerancias))
    None

Dica: `tolerancias.get(chave)` devolve None quando a chave não existe,
sem erro.
Depois, desempacote a tupla em duas variáveis.

Teste com:  python curso.py testar 03 ex03
```

Exercício 03.04 — Enquete do refeitório

Arquivo: `exercicios/ex04_contar_votos.py`

```
Nível: ★☆☆

Contexto: O RH fez uma enquete para escolher o prato da sexta-feira. Cada
```

```
pessoa
digitou o voto à mão, então vieram votos como "Feijoada", "feijoada " e
"FEIJOADA".

Tarefa: implemente duas funções.
1. `contar_votos(votos)` devolve um dicionário  prato -> quantidade de
votos.
   - Normalize cada voto: tire espaços das pontas e passe para minúscula.
   - Ignore votos vazios (que ficam "" depois de limpar).
   - As chaves aparecem na ordem do primeiro voto de cada prato.
2. `vencedor(contagem)` recebe o dicionário de contagem e devolve o prato
com
   mais votos. Em caso de empate, vale o que aparece primeiro no
dicionário.
   Dicionário vazio: devolva None.

Exemplos:
    >>> contagem = contar_votos(["Feijoada", "lasanha ", "FEIJOADA", "",
"strogonoff"])
    >>> contagem
    {'feijoada': 2, 'lasanha': 1, 'strogonoff': 1}
    >>> vencedor(contagem)
    'feijoada'
    >>> print(vencedor({}))
    None

Dica: o padrão de contagem é  contagem[chave] = contagem.get(chave, 0) +
1.
Para o vencedor, max() aceita key= (seção 11 da aula).

Teste com:  python curso.py testar 03 ex04
```

Exercício 03.05 — Lotes sem repetição

Arquivo: `exercicios/ex05_remover_duplicadas.py`

```
Nível: ★☆☆

Contexto: O leitor de código de barras da expedição às vezes lê o mesmo
lote
duas vezes. Você precisa da lista de lotes sem repetição, mantendo a
ordem em
que cada lote foi lido pela primeira vez.

Tarefa: implemente `remover_duplicadas(itens)`, que devolve uma lista
NOVA:
- sem itens repetidos;
- na ordem da primeira ocorrência de cada item;
- sem modificar a lista recebida.
```

```
Exemplos:
    >>> remover_duplicadas(["L-101", "L-102", "L-101", "L-103", "L-102"])
    ['L-101', 'L-102', 'L-103']
    >>> remover_duplicadas([])
    []

Dica: `list(set(itens))` remove repetidos, mas BAGUNÇA a ordem. Use um
conjunto
só para lembrar "o que eu já vi" e uma lista para o resultado.

Teste com:  python curso.py testar 03 ex05
```

Exercício 03.06 — Medições fora da faixa

Arquivo: `exercicios/ex06_fora_da_faixa.py`

```
Nível: ★★☆

Contexto: Ao medir a espessura em vários pontos de uma bobina, o analista
precisa
saber QUAIS pontos ficaram fora da faixa (e não só quantos), para anotar
no laudo.

Tarefa: implemente duas funções.
1. `fora_da_faixa(medicoes, minimo, maximo)` devolve uma lista de tuplas
   (posição, valor) das medições fora da faixa [minimo, maximo].
   - A posição começa em 1 (é assim que o analista conta os pontos).
   - Os limites contam como dentro da faixa.
2. `percentual_conforme(medicoes, minimo, maximo)` devolve o percentual
de
   medições DENTRO da faixa, arredondado com round(..., 1).
   Lista vazia: devolva 0.0.

Exemplos:
    >>> fora_da_faixa([50.2, 44.1, 51.0, 56.3], 45.0, 55.0)
    [(2, 44.1), (4, 56.3)]
    >>> percentual_conforme([50.2, 44.1, 51.0, 56.3], 45.0, 55.0)
    50.0
    >>> percentual_conforme([50.0, 50.1, 44.0], 45.0, 55.0)
    66.7

Dica: enumerate(medicoes, start=1) entrega (posição, valor). Tente
escrever a
primeira função com uma compreensão de lista.

Teste com:  python curso.py testar 03 ex06
```

Exercício 03.07 — Peso total por OP

Arquivo: `exercicios/ex07_pesos_por_op.py`

```
Nível: ★★☆

Contexto: A planilha da expedição tem uma coluna com a OP de cada bobina
e outra
com o peso (kg). Uma mesma OP pode ter várias bobinas. Para o
certificado, você
precisa do peso total de cada OP.

Tarefa: implemente `pesos_por_op(ops, pesos)`.
- `ops` e `pesos` são "colunas": o item i de uma corresponde ao item i da
outra.
- Devolva um dicionário  OP -> peso total, com cada total arredondado com
  round(..., 2). As OPs aparecem na ordem da primeira vez que surgem.
- Se as listas tiverem tamanhos diferentes, há erro na planilha: devolva
{}.

Exemplos:
    >>> pesos_por_op(["OP-4512", "OP-4513", "OP-4512"], [500.0, 320.5,
250.25])
    {'OP-4512': 750.25, 'OP-4513': 320.5}
    >>> pesos_por_op(["OP-4512"], [500.0, 320.5])
    {}

Dica: zip(ops, pesos) caminha pelas duas listas ao mesmo tempo. Mas zip
para em
silêncio na lista mais curta, então confira os tamanhos antes.

Teste com:  python curso.py testar 03 ex07
```

Exercício 03.08 — Leitura de código: cópia ou referência?

Arquivo: `exercicios/ex08_leitura_copia_referencia.py`

```
Nível: ★★☆

Contexto: Um colega escreveu um script de correção de medições e jura que
"o
Python mudou a lista sozinho". Antes de culpar o Python, leia o código.

Parte 1 — Preveja. Para cada trecho, faça a função correspondente
devolver o
valor que o print mostraria. Preveja no papel PRIMEIRO; depois rode no
REPL para
conferir. Se errou, descubra o porquê (seção 4 da aula).
```

```
    # Trecho A
    a = [50.2, 49.8]
    b = a
    b.append(51.0)
    print(a)

    # Trecho B
    original = [50.2, 49.8]
    copia = original.copy()
    copia[0] = 99.9
    print(original)

    # Trecho C
    linhas = {"A": [50.2], "B": [20.1]}
    backup = linhas.copy()
    backup["A"].append(49.9)
    print(linhas["A"])

    # Trecho D
    valores = [10, 55, 60, 20]
    for v in valores:
        if v > 50:
            valores.remove(v)
    print(valores)

Parte 2 — Conserte. A função abaixo deveria devolver as medições
corrigidas
(cada uma menos o offset do aparelho, arredondada em 2 casas) SEM alterar
a
lista original. Mas ela altera:

    def corrigir_offset(medicoes, offset):
        for i in range(len(medicoes)):
            medicoes[i] = round(medicoes[i] - offset, 2)
        return medicoes

Escreva `corrigir_offset` corrigida: mesma resposta, lista original
intacta.

Exemplos:
    >>> medicoes = [50.4, 49.9]
    >>> corrigir_offset(medicoes, 0.2)
    [50.2, 49.7]
    >>> medicoes
    [50.4, 49.9]

Dica: para a Parte 1, desenhe as etiquetas e as caixas como nos diagramas
da
aula. Para a Parte 2, monte uma lista NOVA em vez de escrever na
```

```
recebida.

Teste com:  python curso.py testar 03 ex08
```

Exercício 03.09 — Ranking da turma

Arquivo: `exercicios/ex09_ranking_alunos.py`

```
Nível: ★★☆

Contexto: Um professor de curso técnico guarda as notas finais num
dicionário
nome -> nota e quer publicar o ranking e a lista de aprovados.

Tarefa: implemente duas funções.
1. `ranking(notas)` devolve uma lista de tuplas (nome, nota):
   - da maior nota para a menor;
   - em caso de empate na nota, por nome em ordem alfabética.
2. `aprovados(notas, media_minima)` devolve a lista de nomes com nota
maior ou
   igual à média mínima, em ordem alfabética.

Exemplos:
    >>> notas = {"Carla": 8.5, "Bruno": 9.0, "Ana": 8.5, "Davi": 4.0}
    >>> ranking(notas)
    [('Bruno', 9.0), ('Ana', 8.5), ('Carla', 8.5), ('Davi', 4.0)]
    >>> aprovados(notas, 6.0)
    ['Ana', 'Bruno', 'Carla']

Dica: notas.items() dá pares (nome, nota). sorted aceita key= com uma
lambda que
devolve uma TUPLA de critérios. Para inverter só a nota, use o sinal de
menos.

Teste com:  python curso.py testar 03 ex09
```

Exercício 03.10 — OPs pedidas x OPs com resultado

Arquivo: `exercicios/ex10_conferir_ops.py`

```
Nível: ★★☆

Contexto: A expedição mandou por e-mail as OPs que vão sair hoje. A
planilha do
laboratório tem as OPs que já têm resultado de ensaio. Antes de emitir os
certificados, você precisa cruzar as duas listas.
```

```
Tarefa: implemente `conferir_ops(ops_pedidas, ops_com_resultado)`, que
devolve
um dicionário com três listas, cada uma SEM repetidos e em ordem
alfabética:
- "prontas": OPs pedidas que têm resultado (podem ganhar certificado);
- "sem_resultado": OPs pedidas que ainda não têm resultado (cobrar o
lab);
- "nao_pedidas": OPs com resultado que a expedição não pediu.

Exemplos:
    >>> conferir_ops(["OP-3", "OP-1", "OP-2", "OP-1"], ["OP-2", "OP-4",
"OP-1"])
    {'prontas': ['OP-1', 'OP-2'], 'sem_resultado': ['OP-3'],
'nao_pedidas': ['OP-4']}

Dica: converta as listas em conjuntos e use as operações & (interseção) e
- (diferença). Conjuntos não têm ordem: ordene com sorted() no final.

Teste com:  python curso.py testar 03 ex10
```

Exercício 03.11 — Resumo por linha (o seu primeiro GROUP BY)

Arquivo: `exercicios/ex11_resumo_por_linha.py`

```
Nível: ★★★

Contexto: As amostras do dia estão numa "tabela": uma lista de
dicionários, um
por amostra. Algumas células vieram vazias (a chave falta ou o valor é
None).
O supervisor quer, para cada linha de produção, quantas amostras têm
valor e a
média, o mínimo e o máximo de uma propriedade (espessura, gramatura...).

Tarefa: implemente duas funções (a segunda usa a primeira).
1. `agrupar_valores(amostras, campo)` devolve  linha -> lista de valores
do
   campo, na ordem em que aparecem. Ignore amostras em que o campo falta
ou é
   None. Linhas sem nenhum valor válido não aparecem.
2. `resumo_por_linha(amostras, campo)` devolve  linha -> {"qtd": ...,
   "media": ..., "minimo": ..., "maximo": ...}, com "media" arredondada
com
   round(..., 2) e as linhas em ORDEM ALFABÉTICA.

Exemplos:
    >>> amostras = [
    ...     {"op": "OP-4512", "linha": "B", "espessura": 20.9},
    ...     {"op": "OP-4513", "linha": "A", "espessura": 50.4},
```

```
    ...     {"op": "OP-4514", "linha": "A", "espessura": 49.7},
    ...     {"op": "OP-4515", "linha": "A", "espessura": None},
    ...     {"op": "OP-4516", "linha": "C"},
    ... ]
    >>> agrupar_valores(amostras, "espessura")
    {'B': [20.9], 'A': [50.4, 49.7]}
    >>> resumo_por_linha(amostras, "espessura")["A"]
    {'qtd': 2, 'media': 50.05, 'minimo': 49.7, 'maximo': 50.4}
    >>> list(resumo_por_linha(amostras, "espessura"))
    ['A', 'B']

Dica: amostra.get(campo) devolve None tanto se a chave falta quanto se o
valor
é None. Para a ordem alfabética, percorra sorted(grupos).

Teste com:  python curso.py testar 03 ex11
```

Exercício 03.12 — Revisão: relatório de tolerâncias

Arquivo: `exercicios/ex12_revisao_tolerancias.py`

```
Nível: ★★★

Contexto: Este exercício mistura o módulo 03 (dicionário aninhado, lista
de
dicionários) com o 01 (strip, upper, f-strings, round) e o 02 (if, for,
acumuladores). As tolerâncias de cada linha ficam num dicionário
aninhado:

    TOLERANCIAS = {
        "A": {"espessura": {"nominal": 50.0, "tol_pct": 10.0},
              "gramatura": {"nominal": 46.0, "tol_pct": 5.0}},
        "B": {"espessura": {"nominal": 20.0, "tol_pct": 8.0}},
    }

Tarefa: implemente quatro funções pequenas, cada uma usando a anterior.
1. `limites(nominal, tol_pct)` devolve a tupla (mínimo, máximo), com
   mínimo = nominal × (1 − tol_pct/100) e máximo = nominal × (1 +
tol_pct/100),
   ambos arredondados com round(..., 2).
2. `falhas_da_amostra(amostra, tolerancias)` devolve a lista de falhas.
   - Normalize a linha da amostra (strip + upper).
   - Se a linha não tem tolerância cadastrada, devolva None.
   - Para cada propriedade da linha (na ordem do dicionário), se a
amostra
     tiver valor (não None) e ele estiver fora de [mínimo, máximo],
acrescente
     o texto  f"{propriedade} {valor:.1f} fora de {minimo:.1f} a
{maximo:.1f}".
```

```
3. `linha_do_relatorio(amostra, tolerancias)` devolve um texto:
   - "OP-4512 | linha A | APROVADA"  (sem falhas)
   - "OP-4512 | linha A | REPROVADA: falha 1; falha 2"  (falhas unidas
por "; ")
   - "OP-4512 | linha E | PENDENTE: sem tolerância cadastrada"  (None)
4. `relatorio(amostras, tolerancias)` devolve a lista com uma linha de
   relatório por amostra e, no fim, o total:
   "Total: 3 amostras, 1 aprovadas (33.3%)". Sem amostras: 0.0%.
Não altere as amostras nem as tolerâncias.

Exemplos:
    >>> limites(46.0, 5.0)
    (43.7, 48.3)
    >>> amostra = {"op": "OP-4512", "linha": " a", "espessura": 56.3,
"gramatura": 46.2}
    >>> falhas_da_amostra(amostra, TOLERANCIAS)
    ['espessura 56.3 fora de 45.0 a 55.0']
    >>> linha_do_relatorio(amostra, TOLERANCIAS)
    'OP-4512 | linha A | REPROVADA: espessura 56.3 fora de 45.0 a 55.0'

Dica: "; ".join(lista) junta textos de uma lista separando por "; " (o
join é
visto a fundo no módulo 05). Um laço que concatena strings também
resolve.

Teste com:  python curso.py testar 03 ex12
```

Exercício 03.13 — Revisão: juntar pedidos e resultados (o seu primeiro JOIN)

Arquivo: `exercicios/ex13_revisao_juntar_tabelas.py`

```
Nível: ★★★

Contexto: Este exercício mistura o módulo 03 (lista de dicionários,
índice com
dicionário) com o 01 (strip, upper, replace, round) e o 02 (if/elif/else,
acumuladores). A expedição digita a OP de vários jeitos ("op-4512",
" OP 4512 "); a planilha do laboratório tem o status de cada OP.

    pedidos = [{"op": "op-4512", "nf": "123", "peso_kg": 500.0}, ...]
    resultados = [{"op": "OP-4512", "linha": "A", "status": "Aprovado"},
...]

Tarefa: implemente quatro funções.
1. `normalizar_op(texto)`: tira espaços das pontas, passa para maiúscula
e
   troca espaços internos por "-".  " op 4512 " -> "OP-4512".
```

```
2. `indexar_por_op(resultados)`: devolve um dicionário  OP normalizada ->
   resultado (o próprio dicionário). Se a OP se repetir, vale o ÚLTIMO.
3. `juntar(pedidos, resultados)`: para cada pedido, na ordem, devolve um
   dicionário NOVO com todas as chaves do pedido (com "op" normalizada)
mais:
   - "linha": a linha do resultado (ou None, se não houver resultado);
   - "status": o status do resultado em minúsculas e sem espaços nas
pontas
     (ou "sem resultado").
   Não altere os pedidos nem os resultados.
4. `resumo_expedicao(juntados)`: recebe a saída de `juntar` e devolve
   {"emitir": [...], "bloqueadas": [...], "sem_resultado": [...],
    "peso_emitir_kg": ...}
   - "emitir": OPs com status "aprovado"; "sem_resultado": OPs com status
     "sem resultado"; "bloqueadas": as demais (reprovado, em análise...).
   - As listas seguem a ordem dos pedidos.
   - "peso_emitir_kg": soma dos pesos das OPs a emitir, round(..., 1).

Exemplos:
    >>> normalizar_op(" op 4512 ")
    'OP-4512'
    >>> pedidos = [{"op": "op-4512", "nf": "123", "peso_kg": 500.0},
    ...            {"op": "OP-4599", "nf": "124", "peso_kg": 80.5}]
    >>> resultados = [{"op": "OP-4512", "linha": "A", "status": "
Aprovado"}]
    >>> juntados = juntar(pedidos, resultados)
    >>> juntados[0]
    {'op': 'OP-4512', 'nf': '123', 'peso_kg': 500.0, 'linha': 'A',
'status': 'aprovado'}
    >>> juntados[1]["status"]
    'sem resultado'
    >>> resumo = resumo_expedicao(juntados)
    >>> resumo["emitir"], resumo["sem_resultado"],
resumo["peso_emitir_kg"]
    (['OP-4512'], ['OP-4599'], 500.0)

Dica: procurar cada pedido percorrendo TODOS os resultados funciona, mas
é
lento (é um laço dentro de outro). Monte o índice uma vez e use .get(op).

Teste com:  python curso.py testar 03 ex13
```

Desafio do módulo 03 — Painel de conferência do lote

Contexto

Fim de turno no laboratório. A planilha do dia tem 10 amostras das linhas A a D. A expedição mandou por e-mail os pedidos de amanhã (OP, NF e peso). Antes de emitir os certificados, o supervisor quer um painel no terminal com:

- cada amostra conferida contra as tolerâncias da linha;
- um resumo por linha;
- as maiores não conformidades de espessura;
- a fila de certificados, com o que está bloqueado e o que nem tem resultado.

Você vai montar esse painel usando só coleções (e funções simples com `def` e `return`, como no "molde" dos exercícios).

Dados de entrada

Crie o arquivo `desafio/painel.py` e cole estes dados no topo. No projeto final, eles virão da planilha (openpyxl) e do e-mail; por enquanto, estão no código.

```
TOLERANCIAS = {
    "A": {
        "espessura": {"nominal": 50.0, "tol_pct": 10.0},
        "gramatura": {"nominal": 46.0, "tol_pct": 5.0},
        "cof": {"min": 0.15, "max": 0.30},
    },
    "B": {
        "espessura": {"nominal": 20.0, "tol_pct": 8.0},
        "gramatura": {"nominal": 18.5, "tol_pct": 5.0},
        "cof": {"min": 0.20, "max": 0.40},
    },
    "C": {
        "espessura": {"nominal": 35.0, "tol_pct": 10.0},
        "gramatura": {"nominal": 32.0, "tol_pct": 6.0},
        "cof": {"min": 0.15, "max": 0.30},
    },
    "D": {
        "espessura": {"nominal": 70.0, "tol_pct": 7.0},
        "gramatura": {"nominal": 65.0, "tol_pct": 5.0},
        "cof": {"min": 0.10, "max": 0.25},
    },
}

COLUNAS = ("op", "linha", "espessura", "gramatura", "cof", "obs")
PLANILHA = [
```

```
    ("OP-5101", "A", 50.4, 46.3, 0.22, ""),
    ("OP-5102", "A", 55.8, 46.9, 0.24, ""),
    ("OP-5103", "B", 20.6, 18.9, 0.42, "COF: liberado"),
    ("OP-5104", "B", 19.1, None, 0.30, ""),
    ("OP-5105", "C", 35.9, 32.4, 0.19, ""),
    ("OP-5106", "C", 31.0, 29.5, 0.33, ""),
    ("OP-5107", "D", 71.2, 66.0, 0.12, ""),
    ("OP-5108", "D", 70.4, 64.1, 0.26, "cof: liberado pela supervisão"),
    ("OP-5109", "A", 47.9, 44.0, 0.16, ""),
    ("OP-5110", "D", 64.9, 61.5, 0.11, ""),
]
AMOSTRAS = [dict(zip(COLUNAS, linha, strict=True)) for linha in PLANILHA]

PEDIDOS = [
    {"op": "OP-5101", "nf": "004501", "peso_kg": 1250.5},
    {"op": "OP-5103", "nf": "004502", "peso_kg": 830.0},
    {"op": "OP-5106", "nf": "004503", "peso_kg": 640.2},
    {"op": "OP-5108", "nf": "004504", "peso_kg": 1105.8},
    {"op": "OP-5104", "nf": "004505", "peso_kg": 412.0},
    {"op": "OP-5120", "nf": "004506", "peso_kg": 300.0},
]
```

Regras de tolerância:

- **espessura** e **gramatura**: nominal ± `tol_pct` %. Limites arredondados em 2 casas, e os limites contam como dentro.
- **cof**: faixa fixa `min` a `max`.

Requisitos obrigatórios

1. **Conferir cada amostra** e produzir uma tabela **nova** (sem alterar `AMOSTRAS`) com duas colunas a mais:
   - `falhas`: lista das propriedades fora da faixa;
   - `status`:
     - `INCOMPLETA` se alguma propriedade da linha está vazia (`None`);
     - `APROVADA` se não há falhas;
     - `LIBERADA` se a **única** falha é o COF e a observação contém "cof: liberado" (sem diferenciar maiúsculas);
     - `REPROVADA` nos demais casos.
2. **Imprimir a tabela** alinhada: OP, linha, espessura, gramatura, COF, status e falhas. Célula vazia aparece como `-`.
3. **Resumo por linha** (em ordem alfabética): quantidade de amostras, quantas podem ser emitidas (APROVADA ou LIBERADA), percentual e espessura média.
4. **Os 3 maiores desvios de espessura**, em módulo, em relação ao nominal da linha, mostrando o sinal (`+11.6%`, `-7.3%`).

5. **Expedição**:
   - pedidos que podem ganhar certificado, com NF, peso e status, e o peso total;
   - pedidos bloqueados (com o motivo);
   - OPs pedidas que não têm resultado no laboratório (use conjuntos).

Extras opcionais

- Perguntar ao usuário (com `input()`) uma linha (A–D) e mostrar só as amostras dela. Deixe também um jeito de rodar sem perguntar.
- Mostrar os números no formato brasileiro (`1.250,5 kg`). Dica: formate com `:,.1f` e troque os separadores com `replace`.
- Avisar se a mesma OP aparece duas vezes na planilha.
- Ordenar a tabela pela coluna que o usuário escolher.
- Mostrar, para cada falha, o valor medido e a faixa (`espessura 55.8 fora de 45.0 a 55.0`).

Exemplo de execução

```
python desafio\painel.py
```

(No Linux/macOS: `python3 desafio/painel.py`.) Saída esperada, para os dados acima:

```
============================================================
PAINEL DE CONFERÊNCIA DO LOTE
============================================================
OP       Linha   Esp.  Gram.   COF  Status     Fora
OP-5101  A       50.4   46.3  0.22  APROVADA   -
OP-5102  A       55.8   46.9  0.24  REPROVADA  espessura
OP-5103  B       20.6   18.9  0.42  LIBERADA   cof
OP-5104  B       19.1      -  0.30  INCOMPLETA gramatura
OP-5105  C       35.9   32.4  0.19  APROVADA   -
OP-5106  C       31.0   29.5  0.33  REPROVADA  espessura, gramatura, cof
OP-5107  D       71.2   66.0  0.12  APROVADA   -
OP-5108  D       70.4   64.1  0.26  LIBERADA   cof
OP-5109  A       47.9   44.0  0.16  APROVADA   -
OP-5110  D       64.9   61.5  0.11  REPROVADA  espessura, gramatura

--- Resumo por linha ---
Linha  Amostras Liberadas      %  Esp. média
A             3         2  66.7%       51.37
B             2         1  50.0%       19.85
C             2         1  50.0%       33.45
D             3         2  66.7%       68.83
```

```
--- 3 maiores desvios de espessura ---
OP-5102 (linha A): +11.6% do nominal
OP-5106 (linha C): -11.4% do nominal
OP-5110 (linha D): -7.3% do nominal

--- Expedição ---
Emitir certificado (3 OPs, 3186.3 kg):
  OP-5101  NF 004501  1250.5 kg  [APROVADA]
  OP-5103  NF 004502  830.0 kg  [LIBERADA]
  OP-5108  NF 004504  1105.8 kg  [LIBERADA]
Bloqueadas:
  OP-5106  REPROVADA: espessura, gramatura, cof
  OP-5104  INCOMPLETA: gramatura
Sem resultado no laboratório: ['OP-5120']
```

O layout pode ser diferente; os números e os status, não.

Critérios de aceite

- [ ] O programa roda do começo ao fim sem erro e sem pedir nada ao usuário (nos requisitos obrigatórios).
- [ ] Os status das 10 amostras batem com a saída de exemplo (inclusive `OP-5108`, cuja observação está em minúsculas).
- [ ] `AMOSTRAS` e `PEDIDOS` não são alterados (teste: no fim do programa, `"status" not in AMOSTRAS[0]`).
- [ ] O resumo por linha sai em ordem alfabética, com os percentuais corretos.
- [ ] Os 3 maiores desvios são `OP-5102`, `OP-5106` e `OP-5110`, nessa ordem.
- [ ] O peso total a emitir é 3186.3 kg e `OP-5120` aparece como sem resultado.
- [ ] Nenhuma lista é modificada enquanto é percorrida.
- [ ] O código passa no `ruff check` (rode `ruff check desafio` na pasta do módulo).

Dicas de como começar

1. **Comece pelos dados, não pelo `print`.** Primeiro escreva `faixa(regra)`, que transforma uma regra em `(mínimo, máximo)`. Teste no REPL com `faixa(TOLERANCIAS["A"]["gramatura"])`: deve dar `(43.7, 48.3)`.
2. Depois escreva `conferir(amostra)`, que devolve um dicionário **novo** (`{**amostra, "falhas": ..., "status": ...}`). Teste com uma amostra de cada status.
3. Monte a tabela conferida com uma compreensão: `[conferir(a) for a in AMOSTRAS]`.
4. Só então faça as partes do painel, uma de cada vez, conferindo com a saída de exemplo.

5. Para o resumo por linha, use o padrão "agrupar, depois resumir" da seção 12 da aula.
6. Para a expedição, monte um índice `{op: amostra_conferida}` (o "PROCV" da seção 12).

Travou? A solução de referência está em solucao_referencia.py. Tente por pelo menos 30 minutos antes de abrir; depois compare as duas versões e anote o que você faria diferente.

Quiz do módulo 03 — Coleções

Responda no papel **antes** de rodar qualquer código. Depois confira no REPL e no gabarito do final.

Perguntas

1. O que este código imprime?

```
m = [10, 20, 30, 40]
print(m[1:3], m[-1])
```

2. Qual destes remove um item de uma lista **e devolve o item removido**?

- a) `lista.remove(x)`
- b) `lista.pop()`
- c) `del lista[0]`
- d) `lista.clear()`

3. O que este código imprime?

```
a = [1, 2]
b = a
b.append(3)
print(len(a))
```

4. O que este código imprime?

```
x = [[1], [2]]
y = x.copy()
y[0].append(9)
y.append([3])
print(x)
```

5. Onde está o erro? O que acontece ao rodar e como corrigir?

```
notas = [8, 6, 9]
notas = notas.sort()
print("Menor nota:", notas[0])
```

6. O que este código imprime?

```
op, *resto = ("OP-1", "A", 500.0)
print(resto)
```

7. O que `{}` cria?

- a) um conjunto vazio
- b) um dicionário vazio
- c) uma tupla vazia
- d) um erro de sintaxe

8. O que este código imprime?

```
d = {"A": 1}
print(d.get("B", 0) + d.get("A", 0))
```

9. O que este código imprime?

```
print(sorted({"OP-3", "OP-1"} - {"OP-1", "OP-2"}))
```

10. O que este código imprime?

```
print([v * 2 for v in range(5) if v % 2 == 0])
```

11. Este código roda sem erro e imprime `2`. Qual é o problema, e como evitá-lo?

```
ops = ["OP-1", "OP-2", "OP-3"]
pesos = [500.0, 320.5]
total = {op: p for op, p in zip(ops, pesos)}
print(len(total))
```

12. O que este código imprime?

```
bobinas = [("OP-2", 300), ("OP-1", 500), ("OP-3", 300)]
print(sorted(bobinas, key=lambda b: (b[1], b[0]))[0])
```

13. Qual destes pode ser chave de dicionário?

- a) `[1, 2]`
- b) `{"a": 1}`
- c) `("A", 1)`
- d) `{1, 2}`

14. O que este código imprime?

```
amostras = [
    {"linha": "A", "esp": 50},
    {"linha": "B", "esp": 20},
    {"linha": "A", "esp": 52},
]
g = {}
for a in amostras:
    g.setdefault(a["linha"], []).append(a["esp"])
print(g)
```

15. O que este código imprime?

```
print(all([]), any([]))
```

Gabarito

▼ **Ver respostas**

1. `[20, 30] 40`. O fatiamento `[1:3]` pega os índices 1 e 2 (o 3 não entra). `[-1]` é o último.

2. b) `pop()` devolve o item removido. `remove` e `clear` devolvem `None`; `del` é um comando, não devolve nada.

3. `3`. `b = a` não copia: as duas variáveis apontam para a mesma lista.

4. `[[1, 9], [2]]`. `copy()` é cópia **rasa**: a lista de fora é nova (por isso o `[3]` não aparece em `x`), mas as listas de dentro são as mesmas (por isso o `9` aparece).

5. `notas.sort()` ordena no lugar e devolve `None`; a atribuição guarda `None` em `notas`. A linha do `print` dá `TypeError: 'NoneType' object is not subscriptable`. Correção: `notas.sort()` sozinho na linha, ou `notas = sorted(notas)`.

6. `['A', 500.0]`. O `*resto` recolhe o que sobrou, sempre como **lista**.

7. b) um dicionário vazio. Conjunto vazio é `set()`.

8. `1`. `get("B", 0)` devolve o padrão `0` (a chave não existe) e `get("A", 0)` devolve `1`.

9. `['OP-3']`. A diferença `-` mantém o que está no primeiro conjunto e não está no segundo.

10. `[0, 4, 8]`. O filtro deixa passar 0, 2 e 4; cada um é multiplicado por 2.

11. O `zip` parou na lista mais curta e a `OP-3` sumiu **sem aviso**. Use `zip(ops, pesos, strict=True)` (dá `ValueError` se os tamanhos forem diferentes) ou confira `len(ops) == len(pesos)` antes.

12. `('OP-2', 300)`. A key ordena pelo peso (crescente) e, no empate de 300, pela OP: `OP-2` vem antes de `OP-3`.

13. c) `("A", 1)`. Chaves precisam ser imutáveis. Listas, dicionários e conjuntos mudam, então dão `TypeError: unhashable type`.

14. `{'A': [50, 52], 'B': [20]}`. É o padrão de agrupamento: `setdefault` cria a lista na primeira vez e devolve a lista existente nas seguintes.

15. `True False`. `all` de nada é `True` ("nenhum item falhou") e `any` de nada é `False` ("nenhum item passou"). Cuidado com isso ao conferir listas que podem vir vazias: `all(dentro_da_faixa(v) for v in [])` diz "aprovado" para um lote sem medição nenhuma!

Módulo 04 — Funções: organize, reutilize e decomponha problemas

> **Tempo estimado:** 10 a 14 horas · **Pré-requisitos:** módulos 00 a 03 · **Nível:** iniciante

O que você vai aprender

- Escrever funções com `def`, parâmetros e `return`, e explicar a diferença entre **devolver** e **mostrar** um valor.
- Chamar funções com argumentos posicionais e nomeados, e definir valores padrão.
- Reconhecer e corrigir a armadilha do **valor padrão mutável**.
- Devolver vários valores de uma vez e usar **retorno antecipado** para acabar com `if` dentro de `if`.
- Prever o valor de uma variável sabendo onde ela vive (escopo local e global).
- Documentar funções com docstring e type hints.
- Receber quantos argumentos vierem com `*args` e `**kwargs`.
- Passar funções como argumento, guardá-las em dicionários e criar funções dentro de funções.
- Usar `lambda` com `key=`.
- Separar **funções puras** de **efeitos colaterais**.
- **Decompor** um problema grande em funções pequenas, testáveis e fáceis de ler.
- Escrever uma função recursiva simples, com caso base, e saber quando preferir um laço.

Por que isso importa

Até agora seus programas eram uma lista de comandos de cima para baixo. Funciona para 30 linhas. Com 300 linhas, vira um labirinto: a mesma conta de tolerância aparece em cinco lugares, uma correção num lugar esquece os outros, e ninguém (nem você, daqui a um mês) entende o que o código faz.

Funções resolvem isso. Elas deixam você:

- **dar nome a uma ideia** (`calcular_limites`, `conferir_amostra`) e usar o nome no lugar dos detalhes;
- **escrever uma vez e usar em vários lugares**;

- **testar cada pedaço sozinho**, que é exatamente o que o `python curso.py testar` faz com os seus exercícios.

No mercado, "saber programar" é em grande parte saber **quebrar um problema em funções bem nomeadas**. É o que um entrevistador observa num teste técnico, e é o que separa um script que só o autor entende de um sistema que uma equipe mantém. O seu projeto final (ler a planilha, conferir tolerâncias, ler e-mails, gerar certificados) só fica de pé se cada parte for uma função pequena que você consegue testar.

1. Funções: dar nome a uma ideia

Você já escreveu funções no "molde" dos exercícios. Agora vamos entender cada peça.

Imagine a conta de limites de tolerância repetida em três lugares do seu script:

```
minimo_a = round(50.0 * (1 - 10.0 / 100), 2)
maximo_a = round(50.0 * (1 + 10.0 / 100), 2)
minimo_b = round(20.0 * (1 - 8.0 / 100), 2)
maximo_b = round(20.0 * (1 + 8.0 / 100), 2)
# ... e de novo para a gramatura, para a linha C ...
```

Se a regra mudar (arredondar em 1 casa, por exemplo), você precisa caçar todas as cópias. Com uma função, a regra mora num lugar só:

```
>>> def calcular_limites(nominal, tol_pct):
...     """Devolve (mínimo, máximo) para nominal ± tol_pct %."""
...     fator = tol_pct / 100
...     return round(nominal * (1 - fator), 2), round(nominal * (1 +
fator), 2)
...
>>> calcular_limites(50.0, 10.0)
(45.0, 55.0)
>>> calcular_limites(20.0, 8.0)
(18.4, 21.6)
```

Anatomia

```
 def calcular_limites(nominal, tol_pct):          <- cabeçalho
 |   |                \______ ______/
 |   |                       v
 |   nome (verbo, snake_case) parâmetros: as "entradas"
 palavra-chave
```

```
    """Devolve (mínimo, máximo) ..."""          <- docstring (o manual)
    fator = tol_pct / 100                       <- corpo (indentado)
    return round(...), round(...)               <- return: a "saída"
```

E o que acontece quando você **chama** a função:

```
limites = calcular_limites(50.0, 10.0)
                |
                |  1. o Python pula para dentro da função
                v     com nominal = 50.0 e tol_pct = 10.0
        fator = 0.1
        return (45.0, 55.0)
                |
                |  2. volta para onde foi chamada,
                v     trazendo o valor do return
limites = (45.0, 55.0)
```

O `def` só **cria** a função; nada roda nessa hora. O corpo só executa quando alguém **chama**: `calcular_limites(50.0, 10.0)`.

Devolver não é mostrar

Esta é a confusão número 1 de quem começa.

- `print` **mostra** um valor na tela. Quem chamou a função não recebe nada.
- `return` **devolve** o valor para quem chamou, que pode guardar, somar, comparar, passar adiante.

🔮 **Preveja antes de rodar:** o que aparece na tela?

```python
def media_mostrando(valores):
    print(sum(valores) / len(valores))

resultado = media_mostrando([50.0, 52.0])
print(resultado)
```

▼ **Resposta**

```
51.0
None
```

A primeira linha vem do `print` **dentro** da função. A segunda mostra o que a função **devolveu**: como não há `return`, ela devolve `None`. Toda função sem `return` (ou com `return` sozinho) devolve `None`.

Se você tentasse `resultado * 2`, teria `TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'`. Regra prática: **funções de cálculo devolvem; só a parte do programa que conversa com o usuário imprime** (seção 10).

O exemplo completo está em `exemplos/01_def_e_return.py`. No PowerShell, na pasta `04-funcoes`:

```
python exemplos\01_def_e_return.py
```

(No Linux/macOS: `python3 exemplos/01_def_e_return.py`.)

Tente você: escreva `peso_liquido(peso_bruto, tara)` que devolve o peso bruto menos a tara, arredondado em 1 casa.

▼ **Resposta**

```
>>> def peso_liquido(peso_bruto, tara):
...     return round(peso_bruto - tara, 1)
...
>>> peso_liquido(1265.3, 14.8)
1250.5
```

2. Parâmetros e argumentos

Dois nomes que você vai ver na documentação:

- **parâmetro** é o nome na definição (`nominal`, `tol_pct`);
- **argumento** é o valor que você passa na chamada (`50.0`, `10.0`).

Você pode passar argumentos de dois jeitos:

```
>>> calcular_limites(46.0, 5.0)
(43.7, 48.3)
>>> calcular_limites(nominal=46.0, tol_pct=5.0)
(43.7, 48.3)
>>> calcular_limites(tol_pct=5.0, nominal=46.0)
(43.7, 48.3)
```

```
>>> calcular_limites(46.0, tol_pct=5.0)
(43.7, 48.3)
```

- **Posicionais:** casam pela ordem. Curtos, mas você precisa lembrar a ordem.
- **Nomeados:** casam pelo nome, em qualquer ordem. Mais legíveis, principalmente com números soltos: `calcular_limites(46.0, 5.0)` obriga o leitor a abrir a função; `calcular_limites(46.0, tol_pct=5.0)` não.
- Pode misturar, mas **posicionais primeiro**.

Os erros de chamada mais comuns, com a mensagem que o Python mostra:

```
>>> calcular_limites(46.0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: calcular_limites() missing 1 required positional argument:
'tol_pct'
>>> calcular_limites(46.0, 5.0, 2)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: calcular_limites() takes 2 positional arguments but 3 were
given
>>> calcular_limites(46.0, tolerancia=5.0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: calcular_limites() got an unexpected keyword argument
'tolerancia'
```

Leia a mensagem: ela diz **qual função**, **qual parâmetro** e **o que estava errado** (faltou, sobrou, nome desconhecido).

3. Valores padrão (e a armadilha do padrão mutável)

Um parâmetro pode ter um **valor padrão**, usado quando o argumento não é passado:

```
>>> def formatar_medida(valor, unidade="µm", casas=1):
...     return f"{valor:.{casas}f} {unidade}"
...
>>> formatar_medida(50.24)
'50.2 µm'
>>> formatar_medida(46, "g/m²")
'46.0 g/m²'
>>> formatar_medida(0.2234, unidade="(COF)", casas=3)
'0.223 (COF)'
```

Repare no `{valor:.{casas}f}`: dá para colocar uma variável **dentro** da especificação de formato. O Python troca `{casas}` pelo número (3) e depois aplica `.3f`. O exercício 02 usa isso.

Regras:

- Parâmetros **com** padrão vêm **depois** dos sem padrão. `def f(a=1, b)` dá `SyntaxError: parameter without a default follows parameter with a default`.
- Use padrão para o **caso comum**. Quem precisa do caso especial passa o argumento, de preferência nomeado.

Argumentos só por nome

Um `*` sozinho na lista de parâmetros diz: "daqui para frente, só por nome".

```
>>> def formatar(valor, *, casas=1, unidade="µm"):
...     return f"{valor:.{casas}f} {unidade}"
...
>>> formatar(0.2234, casas=3, unidade="")
'0.223 '
>>> formatar(0.2234, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: formatar() takes 1 positional argument but 2 were given
```

Isso evita chamadas misteriosas como `formatar(0.2234, 3, "")`. Bibliotecas profissionais (pandas, por exemplo) usam muito.

A armadilha do padrão mutável

🔮 **Preveja antes de rodar:** o que as duas chamadas devolvem?

```
def registrar(defeito, defeitos=[]):
    defeitos.append(defeito)
    return defeitos

print(registrar("bolha"))
print(registrar("risco"))
```

▼ **Resposta**

```
['bolha']
['bolha', 'risco']
```

A segunda chamada "lembrou" da primeira. Ninguém passou a lista, então as duas usaram o padrão. E o padrão é **a mesma lista**, criada uma vez só.

Por quê? O valor padrão é criado **quando o `def` roda**, uma única vez, e fica guardado dentro da função. Toda chamada sem o argumento usa **esse mesmo objeto**. Com números e strings (imutáveis) não há problema. Com uma lista (mutável), o `append` modifica o objeto guardado, e a próxima chamada já o encontra modificado.

```
 Quando o def roda (UMA vez):
     registrar  ---- padrão de "defeitos" ---->  [ ]

 registrar("bolha"):   defeitos --+
                                  +-->  ["bolha"]           (a mesma
lista)
 registrar("risco"):   defeitos --+
                                  +-->  ["bolha", "risco"]  (ainda a
mesma!)
```

Não precisa acreditar em mim. Teste a hipótese no REPL: a função guarda os padrões em `__defaults__`.

```
>>> def registrar(defeito, defeitos=[]):
...     defeitos.append(defeito)
...     return defeitos
...
>>> registrar.__defaults__
([],)
>>> registrar("bolha")
['bolha']
>>> registrar.__defaults__
(['bolha'],)
```

Ali está: a lista padrão mudou dentro da própria função.

A correção é um idioma que você vai ver em todo código Python profissional: use `None` como padrão e crie a lista **dentro** da função.

```
>>> def registrar(defeito, defeitos=None):
...     if defeitos is None:
...         defeitos = []
...     defeitos.append(defeito)
...     return defeitos
...
>>> registrar("bolha")
['bolha']
```

```
>>> registrar("risco")
['risco']
```

Vale para qualquer padrão mutável: lista (`[]`), dicionário (`{}`), conjunto (`set()`). O ruff, que você vai usar no módulo 09, aponta esse erro sozinho (regra B006).

> ⚠️ **Erro comum:** escrever `if not defeitos:` em vez de `if defeitos is None:`. Uma lista vazia que o chamador passou **de propósito** também é falsy, e seria trocada por outra. Use `is None`.

Exemplos completos: `exemplos/02_parametros.py` e `exemplos/03_padrao_mutavel.py`.

4. Vários retornos e retorno antecipado

Devolvendo mais de um valor

Uma função devolve **um** objeto. Mas esse objeto pode ser uma tupla, e o desempacotamento do módulo 03 faz o resto:

```
>>> def dividir_em_bobinas(peso_total_kg, peso_bobina_kg):
...     bobinas = int(peso_total_kg // peso_bobina_kg)
...     sobra = round(peso_total_kg % peso_bobina_kg, 2)
...     return bobinas, sobra
...
>>> dividir_em_bobinas(1250.5, 300)
(4, 50.5)
>>> bobinas, sobra = dividir_em_bobinas(1250.5, 300)
>>> sobra
50.5
```

Prefira tupla quando são **2 ou 3 valores** com ordem óbvia. Mais do que isso, devolva um dicionário (`{"bobinas": 4, "sobra": 50.5}`): ninguém precisa lembrar qual posição é o quê.

Retorno antecipado (guard clauses)

`return` encerra a função **na hora**. Isso permite tratar os casos especiais logo no começo e sair, deixando o caminho principal limpo.

Exemplo resolvido: a "flecha" de ifs. Um colega escreveu:

```
def status_amostra(amostra, tolerancias):
    if amostra is not None:
```

```
        if amostra["linha"] in tolerancias:
            if amostra["espessura"] is not None:
                minimo, maximo = tolerancias[amostra["linha"]]
                if minimo <= amostra["espessura"] <= maximo:
                    return "aprovada"
                else:
                    return "reprovada"
            else:
                return "sem medição"
        else:
            return "linha sem tolerância"
    else:
        return "sem amostra"
```

O código anda para a direita como uma flecha, e o `else` de cada caso fica longe do `if` dele. Como eu penso para arrumar:

1. **Quais são os casos que impedem o trabalho?** Sem amostra, linha sem tolerância, sem medição.
2. **Trato cada um logo no começo, com `return`.** Se passou por eles, os dados são válidos.
3. **O caminho feliz fica por último**, sem indentação extra.

```
>>> def status_amostra(amostra, tolerancias):
...     if amostra is None:
...         return "sem amostra"
...     if amostra["linha"] not in tolerancias:
...         return "linha sem tolerância"
...     if amostra["espessura"] is None:
...         return "sem medição"
...     minimo, maximo = tolerancias[amostra["linha"]]
...     if minimo <= amostra["espessura"] <= maximo:
...         return "aprovada"
...     return "reprovada"
...
>>> TOL = {"A": (45.0, 55.0)}
>>> status_amostra({"linha": "A", "espessura": 50.2}, TOL)
'aprovada'
>>> status_amostra({"linha": "E", "espessura": 50.2}, TOL)
'linha sem tolerância'
>>> status_amostra(None, TOL)
'sem amostra'
```

Mesmo comportamento, metade da indentação, e cada regra numa linha. Repare que **a ordem dos `if` é a regra de negócio**: o exercício 03 cobra exatamente isso ("falta vence tudo").

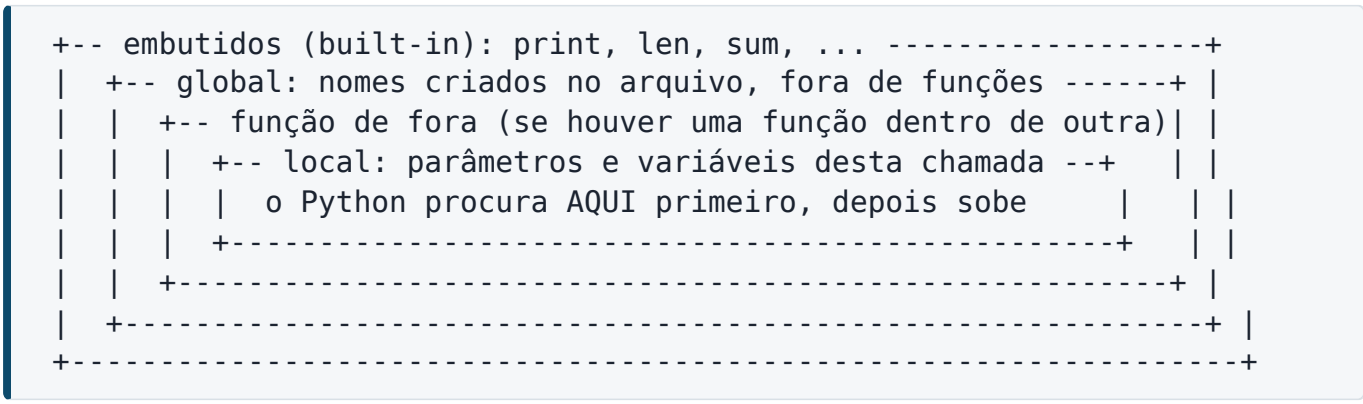
🔧 **Modifique:** acrescente a regra "espessura negativa → `'medição inválida'`" na versão nova. Onde ela entra? Agora tente acrescentar a mesma regra na versão "flecha" e compare o esforço.

5. Escopo: onde cada nome vive

Cada chamada de função ganha um "rascunho" próprio. Os nomes criados lá dentro (parâmetros e variáveis) são **locais**: nascem na chamada e somem quando ela termina.

```
>>> def conferir(valor):
...     resultado_local = 45 <= valor <= 55
...     return resultado_local
...
>>> conferir(50)
True
>>> resultado_local
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'resultado_local' is not defined
```

Para achar um nome, o Python procura de dentro para fora:



Uma função pode **ler** uma variável global. Mas **atribuir** a um nome dentro da função cria uma variável **local** com esse nome.

🔮 **Preveja antes de rodar:** o que o último `print` mostra?

```
total = 0

def somar(valor):
    total = valor + 1
    return total
```

```
somar(10)
print(total)
```

▼ **Resposta**

`0`. O `total = valor + 1` criou um `total` **local**, que sumiu no fim da chamada. O global nunca foi tocado. (E o 11 que a função devolveu foi jogado fora, porque ninguém guardou.)

E se você tentar **alterar** a global com `+=`?

```
>>> reprovadas = 0
>>> def registrar_reprovacao():
...     reprovadas += 1
...
>>> registrar_reprovacao()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in registrar_reprovacao
UnboundLocalError: cannot access local variable 'reprovadas' where it is
not associated with a value
```

Como existe uma atribuição (`+=`) a `reprovadas` dentro da função, o Python decide que ela é local **na função inteira**. Aí, para somar 1, ele tenta ler a local, que ainda não tem valor. (No Python 3.10, a mensagem é `local variable 'reprovadas' referenced before assignment`.)

Existe a palavra `global reprovadas` para forçar o uso da global, mas **evite**. Uma função que mexe em variáveis de fora é difícil de testar e de entender. O jeito limpo é receber por parâmetro e devolver com `return`:

```
>>> def registrar_reprovacao(reprovadas):
...     return reprovadas + 1
...
>>> reprovadas = 0
>>> reprovadas = registrar_reprovacao(reprovadas)
>>> reprovadas
1
```

💡 **Dica:** variáveis globais que **nunca mudam** são bem-vindas, como configurações: `TOLERANCIA_PADRAO = 10.0`. Pela convenção, escreva o nome em

> MAIÚSCULAS para avisar "isto é uma constante".

Parâmetros e objetos mutáveis

No módulo 03 você viu que variáveis são etiquetas. Um parâmetro é só **mais uma etiqueta para o objeto que veio na chamada**. Daí duas situações bem diferentes:

> 🔮 **Preveja antes de rodar:** o que aparece no `print`?
>
> ```
> def reatribuir(valores):
>     valores = [0, 0]
>
> def modificar(valores):
>     valores.append(0)
>
> a = [1, 2]
> b = [1, 2]
> reatribuir(a)
> modificar(b)
> print(a, b)
> ```

<details>
<summary>Resposta</summary>

`[1, 2] [1, 2, 0]`.

```
reatribuir(a):                          modificar(b):

 a ---------> [1, 2]                     b ---------> [1, 2, 0]
 valores --X                             valores ---^  (mesmo objeto)
 valores ---> [0, 0]  (nova; some
                       no fim)
```

- `valores = [0, 0]` só **move a etiqueta local** para uma lista nova. A lista de `a` não é tocada.
- `valores.append(0)` **mexe no objeto**, que é o mesmo de `b`.

</details>

Por isso a regra do módulo 03 continua valendo: **não modifique a coleção que você recebeu**, a não ser que esse seja o objetivo declarado da função. Exemplo completo: `exemplos/04_escopo.py`.

6. Docstrings e type hints

Docstring: o manual da função

A string logo abaixo do `def` é a **docstring**. Ela vira a ajuda da função:

```
def calcular_limites(nominal: float, tol_pct: float) -> tuple[float,
float]:
    """Devolve (mínimo, máximo) para nominal ± tol_pct %.

    Os limites são arredondados em 2 casas, para evitar o ruído do float
    (46 * 0.95 daria 43.699999999999996).
    """
```

```
>>> help(calcular_limites)
Help on function calcular_limites in module __main__:

calcular_limites(nominal: float, tol_pct: float) -> tuple[float, float]
    Devolve (mínimo, máximo) para nominal ± tol_pct %.
    ...
```

No VS Code, passar o mouse sobre o nome da função mostra a mesma coisa. Boas docstrings:

- primeira linha curta dizendo **o que a função devolve ou faz** ("Devolve...", "Conta...");
- depois, se precisar, regras e casos especiais ("lista vazia devolve None");
- não repita o código; explique o que o código não diz.

Type hints: os tipos esperados

As anotações depois dos parâmetros (`nominal: float`) e depois da seta (`-> tuple[float, float]`) são **type hints**: dizem que tipo cada coisa deveria ter.

Anotação	Significa
`x: int`, `x: float`, `x: str`, `x: bool`	tipos simples
`medicoes: list[float]`	lista de floats
`tolerancias: dict[str, float]`	dicionário de str para float
`-> tuple[int, float]`	devolve uma tupla (int, float)
`-> float | None`	devolve um float ou None
`-> None`	não devolve nada útil (só faz efeito)

O Python **não confere** os hints na hora de rodar. Eles servem para você, para o leitor e para as ferramentas (o VS Code avisa quando você passa o tipo errado; o mypy, visto no módulo 09, confere o projeto inteiro).

```
>>> def dobro(x: int) -> int:
...     return x * 2
...
>>> dobro("ab")
'abab'
```

O hint dizia `int`, mas o Python rodou com string mesmo. Hint é documentação verificável por ferramenta, não trava.

> 💡 **Dica:** nos exercícios, os stubs já vêm com hints. Leia-os como parte do enunciado: `-> list[str] | None` já diz "devolva uma lista de textos, ou None".

7. `*args` e `**kwargs`: quantos argumentos vierem

Às vezes você não sabe quantos argumentos virão. Quer escrever `media(50.2, 49.8, 51.0)` sem montar lista.

- `*nome` recolhe os argumentos **posicionais** que sobraram numa **tupla**.
- `**nome` recolhe os argumentos **nomeados** que sobraram num **dicionário**.

(Os nomes `args` e `kwargs` são só convenção; o que importa são os asteriscos.)

```
>>> def media(*valores):
...     if not valores:
...         return None
...     return sum(valores) / len(valores)
...
>>> media(50.0, 52.0, 51.0)
51.0
>>> print(media())
None
>>> def criar_amostra(op, linha, **medidas):
...     return {"op": op, "linha": linha, **medidas}
...
>>> criar_amostra("OP-4512", "A", espessura=50.2, cof=0.22)
{'op': 'OP-4512', 'linha': 'A', 'espessura': 50.2, 'cof': 0.22}
```

Os asteriscos também funcionam **na chamada**, no sentido contrário: espalham uma lista em argumentos posicionais, ou um dicionário em argumentos nomeados.

```
>>> medicoes = [49.2, 51.5, 50.1]
>>> media(*medicoes)
50.26666666666667
>>> dados = {"op": "OP-4513", "linha": "B", "gramatura": 18.4}
>>> criar_amostra(**dados)
{'op': 'OP-4513', 'linha': 'B', 'gramatura': 18.4}
```

Ordem dos parâmetros numa definição, quando aparecem juntos:

```
def f(normais, com_padrao=1, *args, so_por_nome=2, **kwargs):
```

Um uso que você vai adorar: um filtro do tipo `WHERE` em que qualquer coluna vira critério.

```
>>> def filtrar(registros, **criterios):
...     return [r for r in registros
...             if all(r.get(coluna) == valor for coluna, valor in
criterios.items())]
...
>>> tabela = [{"op": "OP-1", "linha": "A"}, {"op": "OP-2", "linha": "B"}]
>>> filtrar(tabela, linha="B")
[{'op': 'OP-2', 'linha': 'B'}]
```

> ⚠️ **Erro comum:** abusar de `**kwargs`. Se a função sempre recebe `op`, `nf` e `peso_kg`, declare os três parâmetros. Eles aparecem na ajuda, no autocompletar, e o Python reclama se faltar um. Use `*args` / `**kwargs` quando a quantidade ou os nomes **realmente** variam.

Exemplo completo: `exemplos/05_args_kwargs.py`.

8. Funções são valores

Em Python, uma função é um objeto como um número ou uma lista. Você pode guardá-la numa variável, passá-la como argumento e devolvê-la de outra função. Isso parece estranho no começo e é muito poderoso depois.

```
>>> conferir_limites = calcular_limites
>>> conferir_limites(50.0, 10.0)
(45.0, 55.0)
```

Sem parênteses, `calcular_limites` é **a função**. Com parênteses, é **a chamada** (o resultado).

Passando função como argumento

Pense no `CONT.SE` do Excel: você passa um intervalo e um **critério**. Em Python, o critério pode ser uma função:

```
>>> def cont_se(valores, condicao):
...     return sum(1 for valor in valores if condicao(valor))
...
>>> def acima_de_50(valor):
...     return valor > 50
...
>>> cont_se([50.2, 44.1, 56.3], acima_de_50)
2
```

`cont_se` não sabe qual é o critério. Só sabe que pode chamá-lo. Você troca o critério sem mexer em `cont_se`. Você já usou essa ideia no módulo 03: o `key=` do `sorted` recebe uma função.

Nos type hints, uma função é anotada como `Callable`. Para usar, importe no topo do arquivo: `from collections.abc import Callable`. `Callable[[float], bool]` quer dizer "função que recebe um float e devolve um bool". Os exercícios 06 e 13 usam.

Exemplo resolvido: uma regra de conferência para cada tipo

No projeto, algumas propriedades usam **nominal ± %** (espessura, gramatura) e outras **faixa fixa** (COF). Como eu penso:

1. Escrevo **uma função para cada tipo de regra**, com a mesma "cara": recebem `(valor, regra)` e devolvem `True` / `False`.
2. Guardo as funções num **dicionário**: tipo → função.
3. Na hora de conferir, pego a função pelo tipo e chamo. Sem `if tipo == ... elif ...`.

```
>>> def faixa_fixa(valor, regra):
...     return regra["min"] <= valor <= regra["max"]
...
>>> def nominal_percentual(valor, regra):
...     margem = regra["nominal"] * regra["tol_pct"] / 100
...     return regra["nominal"] - margem <= valor <= regra["nominal"] +
margem
...
>>> VERIFICADORES = {"faixa": faixa_fixa, "nominal": nominal_percentual}
>>> regra_cof = {"tipo": "faixa", "min": 0.15, "max": 0.30}
>>> verificar = VERIFICADORES[regra_cof["tipo"]]
```

```
>>> verificar(0.33, regra_cof)
False
```

Amanhã surge um tipo novo ("só mínimo", para tração)? Escreva a função e acrescente uma linha no dicionário. Nada mais muda. Esse padrão (tabela de despacho) aparece no projeto final.

Funções que fabricam funções

Uma função pode criar outra lá dentro e devolvê-la. A função de dentro **lembra** as variáveis de onde nasceu, mesmo depois que a de fora terminou. Isso se chama **closure** (fechamento).

```
>>> def criar_verificador(minimo, maximo):
...     def verificar(valor):
...         return minimo <= valor <= maximo
...     return verificar
...
>>> linha_a = criar_verificador(45.0, 55.0)
>>> linha_b = criar_verificador(18.0, 22.0)
>>> linha_a(50.0), linha_b(50.0)
(True, False)
>>> cont_se([50.2, 44.1, 56.3], linha_a)
1
```

`linha_a` e `linha_b` são funções "pré-configuradas", cada uma com sua faixa.

Por fim, **métodos também são funções**: `str.upper` é a função por trás de `"op".upper()`. Então `str.upper("op-4512")` devolve `'OP-4512'`, e você pode passar `str.strip` ou `str.upper` como argumento para outra função, como no exercício 13.

> 🔧 **Modifique:** em `exemplos/06_funcoes_como_valores.py`, acrescente um tipo de regra `"minimo"` (só valor mínimo, como a tração) com sua função, e uma propriedade `"tracao"` que o use.

9. `lambda` e o parâmetro `key=`

Muitas vezes a função que você passa é minúscula e usada uma vez só. Para isso existe a **lambda**: uma função de uma expressão, sem nome.

```
 lambda bobina: bobina["peso_kg"]
        \____/  \_______________/
     parâmetros  o que devolve (uma expressão, sem "return")
```

É equivalente a:

```
def peso(bobina):
    return bobina["peso_kg"]
```

O lugar natural da lambda é o `key=` de `sorted`, `min` e `max`:

```
>>> bobinas = [
...     {"op": "OP-4513", "peso_kg": 320.5},
...     {"op": "OP-4512", "peso_kg": 500.0},
...     {"op": "OP-4514", "peso_kg": 210.0},
... ]
>>> [b["op"] for b in sorted(bobinas, key=lambda b: b["peso_kg"])]
['OP-4514', 'OP-4513', 'OP-4512']
>>> min([50.6, 49.8, 51.0], key=lambda v: abs(v - 50.0))
49.8
>>> sorted(["Selagem", "cof", "Gel"], key=lambda p: (len(p), p.lower()))
['cof', 'Gel', 'Selagem']
```

Quando **não** usar lambda:

- Se a lógica tem mais de uma expressão, ou precisa de um nome para ser entendida: use `def`.
- **Não guarde lambda em variável** (`peso = lambda b: b["peso_kg"]`). Se precisa de nome, use `def`: a mensagem de erro fica melhor e o ruff deixa de reclamar (regra E731).

Exemplo completo: `exemplos/07_lambda_key.py`.

10. Funções puras e efeitos colaterais

Uma função é **pura** quando:

1. o resultado depende **só dos argumentos** (mesma entrada → mesma saída, sempre);
2. ela **não muda nada fora dela**.

Qualquer coisa que a função faz "no mundo", além de devolver um valor, é um **efeito colateral**:

- imprimir na tela (`print`);
- alterar uma lista ou dicionário recebido;
- alterar uma variável global;
- ler ou gravar arquivo, banco de dados, planilha;

- mandar e-mail, clicar no TOTVS com um robô.

Efeitos colaterais não são "errados". Sem eles, o programa não serve para nada: alguém tem que ler a planilha e alguém tem que gerar o certificado. O problema é **misturar** efeitos com cálculo. Compare:

```
reprovadas_do_dia = 0


def conferir_e_mostrar(medicoes, minimo, maximo):
    global reprovadas_do_dia
    medicoes.sort()  # altera a lista de quem chamou
    fora = [v for v in medicoes if not minimo <= v <= maximo]
    if fora:
        reprovadas_do_dia += 1  # altera o mundo lá fora
    print(f"{len(fora)} fora da faixa: {fora}")  # mostra em vez de
devolver


def valores_fora(medicoes, minimo, maximo):
    """Pura: só recebe, calcula e devolve."""
    return [v for v in medicoes if not minimo <= v <= maximo]
```

Como você testaria `conferir_e_mostrar`? Teria que capturar a tela, conferir a global, conferir se a lista foi reordenada... E se chamar duas vezes, a global já mudou. Já a pura se testa com uma linha:

```
>>> def valores_fora(medicoes, minimo, maximo):
...     return [v for v in medicoes if not minimo <= v <= maximo]
...
>>> valores_fora([50.2, 56.3, 44.1], 45.0, 55.0) == [56.3, 44.1]
True
```

Núcleo puro, casca com efeitos

A arquitetura que você vai usar no projeto final (e que times profissionais usam) é esta:

```
 +------------------------- casca: efeitos --------------------------+
 |                                                                    |
 |  ler planilha  -->  +------------ núcleo puro ------------+  -->  |
 |  ler e-mail         |  calcular limites                    |       |
 |                     |  conferir amostras                   |  imprimir
 |                     |  decidir status                      |  salvar
 |                     |  montar o texto do certificado       |  robô no
TOTVS
```

```
|                    +-----------------------------------+       |
+-------------------------------------------------------------------+
```

- O **núcleo** recebe dados prontos (listas, dicionários) e devolve resultados. É onde mora a lógica que pode dar errado, e é 100% testável.
- A **casca** é fina: lê, chama o núcleo, mostra ou grava. Quase não tem `if`.

Quando a planilha mudar de formato, só a casca muda. Quando a regra de tolerância mudar, só o núcleo muda, e os testes avisam se algo quebrou.

> 🔮 **Preveja antes de rodar:** esta função é pura? Por quê?
>
> ```python
> def proxima_op(ops_emitidas):
>     ops_emitidas.sort()
>     return ops_emitidas[-1]
> ```

<details>
<summary>▼ Resposta</summary>

Não. O resultado depende só do argumento, mas o `sort()` **altera a lista de quem chamou**: depois da chamada, `ops_emitidas` está ordenada. Versão pura: `return max(ops_emitidas)`, ou `return sorted(ops_emitidas)[-1]` (o `sorted` cria uma lista nova).
</details>

Exemplo completo, com a versão "faz tudo" e a versão pura lado a lado: `exemplos/08_puras_vs_efeitos.py`.

11. Decomposição de problemas

Esta é a habilidade mais importante do módulo. Linguagem você aprende em semanas; **quebrar um problema** é o que leva tempo, e é o que faz você ser contratado.

O método em 6 passos

1. **Escreva o problema em uma frase.** Se não cabe numa frase, ainda não está claro.
2. **Liste os passos em português**, sem pensar em código. "Para cada amostra: achar os limites da linha, contar quantas medições estão fora, ..."
3. **Cada passo vira uma função** com nome de verbo. Se um passo ainda é grande, quebre de novo.

4. **Escreva as assinaturas antes do corpo:** nome, parâmetros, o que devolve, docstring. Escreva a função de cima **como se as de baixo já existissem**. (Isso tem nome: *programação por desejo*. Você "deseja" a função e depois a escreve.)
5. **Implemente de baixo para cima, testando cada uma** no REPL ou com `assert` antes de ir para a próxima.
6. **Componha:** a função de cima vira uma "receita" que só chama as outras.

Exemplo resolvido: o relatório do turno

Passo 1. O problema em uma frase: "Para cada amostra do turno, dizer a média da espessura, quantas medições ficaram fora da tolerância da linha e se a OP foi aprovada; no fim, o total de aprovadas."

Os dados:

```
>>> NOMINAIS = {"A": (50.0, 10.0), "B": (20.0, 8.0)}
>>> AMOSTRAS = [
...     {"op": "OP-4512", "linha": "A", "medicoes": [50.2, 49.8, 51.0]},
...     {"op": "OP-4513", "linha": "B", "medicoes": [20.1, 22.3, 19.0]},
...     {"op": "OP-4514", "linha": "A", "medicoes": [44.0, 50.3]},
... ]
```

A saída desejada:

```
OP-4512 (A): média 50.33 µm, 0 fora -> APROVADA
OP-4513 (B): média 20.47 µm, 1 fora -> REPROVADA
OP-4514 (A): média 47.15 µm, 1 fora -> REPROVADA
Turno: 1/3 OPs aprovadas
```

O primeiro impulso é escrever tudo num `for` só, com ifs e contas misturados. Funciona (está em `exemplos/09_decomposicao.py`, na função `relatorio_monolitico`), mas tem 23 linhas, quatro níveis de indentação, e nenhum pedaço pode ser testado sozinho.

Passo 2. Os passos em português:

- para cada amostra:
  - achar os limites da linha (nominal ± %);
  - contar as medições fora dos limites;
  - calcular a média;
  - decidir o status;
  - escrever a linha do relatório;
- no fim, escrever a linha do total.

Passo 3. A árvore de funções:

```
gerar_relatorio(amostras, nominais)  -> list[str]
+-- conferir_amostra(amostra, nominais)  -> dict
|   +-- calcular_limites(nominal, tol_pct)  -> (mín, máx)
|   +-- contar_fora(medicoes, limites)      -> int
|   +-- media(valores)                      -> float
+-- formatar_linha(conferencia)          -> str
+-- formatar_rodape(conferencias)        -> str
```

Repare: o **cálculo** (`conferir_amostra`) está separado da **apresentação** (`formatar_...`). Se amanhã o relatório virar uma planilha Excel, só as funções de formatar mudam.

Passo 4. As assinaturas primeiro. Começo pela função de cima, "desejando" as outras:

```
def gerar_relatorio(amostras, nominais):
    """A receita: só junta as funções de baixo."""
    conferencias = [conferir_amostra(a, nominais) for a in amostras]
    return [formatar_linha(c) for c in conferencias] +
[formatar_rodape(conferencias)]
```

Ela ainda não roda (as outras não existem), mas já mostra se o desenho faz sentido. Fez? Desço um nível.

Passo 5. De baixo para cima, testando cada uma. Primeiro as folhas da árvore, que não dependem de ninguém:

```
>>> def calcular_limites(nominal, tol_pct):
...     fator = tol_pct / 100
...     return round(nominal * (1 - fator), 2), round(nominal * (1 +
fator), 2)
...
>>> def contar_fora(medicoes, limites):
...     minimo, maximo = limites
...     return sum(1 for valor in medicoes if not minimo <= valor <=
maximo)
...
>>> def media(valores):
...     return sum(valores) / len(valores)
...
>>> calcular_limites(50.0, 10.0)
(45.0, 55.0)
>>> contar_fora([44.0, 50.3], (45.0, 55.0))
1
```

Testei cada uma **antes** de usar. Se `contar_fora` tivesse um erro, eu descobriria agora, numa função de 3 linhas, e não depois, no meio do relatório. Agora o nível do meio:

```
>>> def conferir_amostra(amostra, nominais):
...     limites = calcular_limites(*nominais[amostra["linha"]])
...     fora = contar_fora(amostra["medicoes"], limites)
...     return {
...         "op": amostra["op"],
...         "linha": amostra["linha"],
...         "media": media(amostra["medicoes"]),
...         "fora": fora,
...         "status": "APROVADA" if fora == 0 else "REPROVADA",
...     }
...
>>> conferir_amostra(AMOSTRAS[2], NOMINAIS)["status"]
'REPROVADA'
```

O `*nominais[...]` espalha a tupla `(50.0, 10.0)` nos dois parâmetros de `calcular_limites` (seção 7). E as funções de apresentação:

```
>>> def formatar_linha(c):
...     return (f"{c['op']} ({c['linha']}): média {c['media']:.2f} µm, "
...             f"{c['fora']} fora -> {c['status']}")
...
>>> def formatar_rodape(conferencias):
...     aprovadas = sum(1 for c in conferencias if c["status"] ==
"APROVADA")
...     return f"Turno: {aprovadas}/{len(conferencias)} OPs aprovadas"
...
```

Passo 6. Compor:

```
>>> def gerar_relatorio(amostras, nominais):
...     conferencias = [conferir_amostra(a, nominais) for a in amostras]
...     return [formatar_linha(c) for c in conferencias] +
[formatar_rodape(conferencias)]
...
>>> for linha in gerar_relatorio(AMOSTRAS, NOMINAIS):
...     print(linha)
...
OP-4512 (A): média 50.33 µm, 0 fora -> APROVADA
OP-4513 (B): média 20.47 µm, 1 fora -> REPROVADA
OP-4514 (A): média 47.15 µm, 1 fora -> REPROVADA
Turno: 1/3 OPs aprovadas
```

Mais linhas no total do que a versão monolítica? Um pouco. Mas cada função cabe na tela, tem um nome que explica o que faz e pode ser testada sozinha. E o arquivo de exemplo termina com um `assert` provando que as duas versões dão **exatamente** o mesmo resultado: é assim que se refatora com segurança.

> 🔧 **Modifique:** o supervisor quer ver também o **maior desvio** de cada OP em relação ao nominal. Em `exemplos/09_decomposicao.py`, quais funções você muda? (Resposta: uma função nova `maior_desvio`, uma chave a mais em `conferir_amostra` e um trecho em `formatar_linha`. Nenhuma outra função precisa mudar.)

Sinais de que uma função precisa ser quebrada

- O nome precisa de um "**e**": `conferir_e_imprimir`, `ler_e_validar`. São duas funções.
- Ela tem mais de ~20 linhas, ou mais de 2 níveis de indentação.
- Você escreveu um comentário do tipo `# agora calcula a média` no meio dela: esse trecho quer ser uma função chamada `media`.
- Ela mistura **calcular** com **mostrar** ou **gravar**.
- Para testá-la, você precisa preparar o mundo inteiro.

Mais um exemplo resolvido: regras de senha

Decomposição não é só para laboratório. Um sistema exige senha com 8+ caracteres, pelo menos um dígito e pelo menos uma maiúscula, e quer dizer **todos** os problemas de uma vez.

Três métodos de string ajudam aqui (o módulo 05 aprofunda): `c.isdigit()` diz se o caractere `c` é um dígito; `c.isupper()`, se é letra maiúscula; e `texto.startswith("OP-")` diz se o texto começa com `"OP-"`. Os dois primeiros funcionam também com um texto inteiro: `"4512".isdigit()` é `True`.

Como eu penso: cada regra é uma pergunta de sim/não → uma função pequena. A lista de problemas é "as regras que falharam" → um dicionário descrição → função, e uma compreensão.

```
>>> def tem_tamanho_minimo(senha):
...     return len(senha) >= 8
...
>>> def tem_digito(senha):
...     return any(c.isdigit() for c in senha)
...
>>> def tem_maiuscula(senha):
```

```
...     return any(c.isupper() for c in senha)
...
>>> REGRAS = {
...     "pelo menos 8 caracteres": tem_tamanho_minimo,
...     "pelo menos um dígito": tem_digito,
...     "pelo menos uma maiúscula": tem_maiuscula,
... }
>>> def problemas_da_senha(senha):
...     return [descricao for descricao, regra in REGRAS.items() if not
regra(senha)]
...
>>> problemas_da_senha("bobina")
['pelo menos 8 caracteres', 'pelo menos um dígito', 'pelo menos uma
maiúscula']
>>> problemas_da_senha("Bobina2026")
[]
```

Uma regra nova é uma função nova e uma linha no dicionário. O exercício 12 usa a mesma ideia para validar os pedidos da expedição (OP, NF e peso).

12. Recursão (introdução)

Uma função **recursiva** chama a si mesma para resolver um pedaço **menor** do mesmo problema. Toda recursão tem duas partes:

- **caso base:** o problema é tão pequeno que a resposta é direta. Aqui a função **para** de se chamar;
- **caso recursivo:** a função resolve um pedaço e chama a si mesma com o resto, que é menor.

O exemplo clássico é o fatorial: `n! = n × (n−1)!`, e `0! = 1`.

```
>>> def fatorial(n):
...     if n == 0:
...         return 1
...     return n * fatorial(n - 1)
...
>>> fatorial(5)
120
```

O `exemplos/10_recursao.py` imprime o "rastro" das chamadas. Cada chamada fica esperando a de baixo responder:

```
fatorial(3) chamado
  fatorial(2) chamado
    fatorial(1) chamado
```

```
      fatorial(0) chamado
      fatorial(0) devolve 1
    fatorial(1) devolve 1
  fatorial(2) devolve 2
fatorial(3) devolve 6
```

🔮 **Preveja antes de rodar:** em que ordem os números aparecem?

```python
def contagem(n):
    if n == 0:
        return
    print("ida", n)
    contagem(n - 1)
    print("volta", n)

contagem(2)
```

▼ **Resposta**

```
ida 2
ida 1
volta 1
volta 2
```

O `print("volta", n)` de cada chamada só roda **depois** que a chamada de baixo termina. Por isso as "voltas" saem em ordem inversa: a última chamada a começar é a primeira a terminar (como uma pilha de pratos).

Onde a recursão brilha: estruturas aninhadas

Para somar uma lista "reta", um `for` é mais simples. A recursão vale a pena quando a estrutura tem **níveis que você não conhece de antemão**: listas dentro de listas, pastas dentro de pastas, categorias dentro de categorias.

Para saber se um item é lista, use `isinstance(item, list)`: devolve `True` se `item` for uma lista. É parecido com `type(item) == list`, que você viu no módulo 01, mas é a forma recomendada.

Exemplo resolvido: o peso de uma carga. Um carregamento tem paletes; paletes têm bobinas ou caixas; caixas têm bobinas... Quero o peso total.

```
>>> carga = [[100.0, 120.5], [80.0, [40.0, 40.0]], 10.0]
```

Como eu penso:

1. Uma carga é "uma lista de coisas, e cada coisa é um **número** ou **outra carga**". A definição já é recursiva!
2. Caso base: o item é um número → o peso é ele mesmo.
3. Caso recursivo: o item é uma lista → o peso é a soma dessa lista, que é **o mesmo problema, menor**.
4. Por que termina? Cada chamada recebe uma lista que está **dentro** da anterior. Uma hora só sobram números.

```
>>> def soma_aninhada(itens):
...     total = 0
...     for item in itens:
...         if isinstance(item, list):
...             total += soma_aninhada(item)
...         else:
...             total += item
...     return total
...
>>> soma_aninhada(carga)
390.5
```

Quando a recursão dá errado

Sem caso base (ou com um caso recursivo que não diminui o problema), a função se chama para sempre. O Python corta depois de cerca de 1000 chamadas:

```
Traceback (most recent call last):
  File "C:\curso-python\rec.py", line 6, in <module>
    contagem(3)
  File "C:\curso-python\rec.py", line 3, in contagem
    return contagem(n - 1)
  File "C:\curso-python\rec.py", line 3, in contagem
    return contagem(n - 1)
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

Use recursão **com moderação**: para dados aninhados de profundidade desconhecida, ela é a ferramenta certa. Para "repetir N vezes" ou percorrer uma lista, use laço.

Lendo mensagens de erro deste módulo

Com funções, o traceback ganha **várias camadas**: uma para cada chamada em andamento. Leia de baixo para cima: a última linha diz **o que**; a última camada diz **onde** estourou; as camadas de cima dizem **quem chamou quem**.

1. Argumento faltando, duas camadas abaixo.

```
Traceback (most recent call last):
  File "C:\curso-python\laudo.py", line 11, in <module>
    print(conferir(50.2, 50.0))
          ~~~~~~~~^^^^^^^^^^^^
  File "C:\curso-python\laudo.py", line 7, in conferir
    minimo, maximo = calcular_limites(nominal)
                     ~~~~~~~~~~~~~~~~^^^^^^^^^
TypeError: calcular_limites() missing 1 required positional argument:
'tol_pct'
```

Leitura: o programa chamou `conferir` (linha 11), que chamou `calcular_limites` (linha 7) sem o `tol_pct`. O defeito está na **linha 7**, na chamada, não dentro de `calcular_limites`.

2. `UnboundLocalError`.

```
Traceback (most recent call last):
  File "C:\curso-python\contador.py", line 8, in <module>
    registrar_reprovacao()
    ~~~~~~~~~~~~~~~~~~~^^
  File "C:\curso-python\contador.py", line 5, in registrar_reprovacao
    reprovadas += 1
    ^^^^^^^^^^
UnboundLocalError: cannot access local variable 'reprovadas' where it is
not associated with a value
```

Leitura: a função tenta alterar uma variável global. Correção: receba o valor por parâmetro e devolva o novo valor (seção 5).

3. Função sem `return` usada num cálculo.

```
Traceback (most recent call last):
  File "C:\curso-python\media.py", line 6, in <module>
    print(round(m, 1))
          ~~~~~^^^^^^
TypeError: type NoneType doesn't define __round__ method
```

Leitura: `m` é `None`. Ele veio de `m = media([...])`, e a função `media` calcula mas **esqueceu o `return`**. Toda mensagem com `NoneType` pede a mesma pergunta: "de qual função veio esse valor, e ela tem `return` em todos os caminhos?"

Outros que você vai encontrar:

Mensagem	Causa provável	Correção
`TypeError: f() takes 2 positional arguments but 3 were given`	argumento a mais (ou parâmetro esquecido no `def`)	confira a assinatura
`TypeError: f() got an unexpected keyword argument 'x'`	nome de argumento errado (erro de digitação?)	use o nome exato do parâmetro
`TypeError: f() got multiple values for argument 'x'`	o mesmo parâmetro passado por posição e por nome	passe uma vez só
`SyntaxError: parameter without a default follows parameter with a default`	parâmetro sem padrão depois de um com padrão	parâmetros sem padrão primeiro
`NameError: name 'x' is not defined`	usou fora da função uma variável local dela	devolva o valor com `return`
`TypeError: 'NoneType' object is not subscriptable`	função sem `return` (ou com `return` só em alguns caminhos)	garanta `return` em todos os caminhos
`TypeError: 'float' object is not callable`	você criou uma variável com o nome de uma função (`media = media(...)`) e depois chamou a "função"	use nomes diferentes para a função e para o resultado
`RecursionError: maximum recursion depth exceeded`	recursão sem caso base, ou que não diminui o problema	confira o caso base

> 💡 **Dica:** quando o traceback é longo, procure a **última camada que é código seu**. As camadas de bibliotecas (pandas, openpyxl) raramente são onde está o defeito.

Boas práticas

- **Nome de função é verbo** (`calcular_limites`, `conferir_amostra`, `formatar_linha`). Função que devolve bool pode ser pergunta: `esta_na_faixa`, `tem_digito`.

- **Uma função, uma tarefa.** Se o nome precisa de "e", quebre.
- **Funções pequenas:** cabem na tela, idealmente com menos de 20 linhas.
- **Calcule em funções puras; imprima e grave nas bordas** (núcleo puro, casca fina).
- **Devolva, não imprima,** em funções de cálculo.
- **Não modifique argumentos mutáveis** a menos que esse seja o objetivo declarado.
- **Nunca use lista, dicionário ou conjunto como valor padrão.** Use `None`.
- **Retorno antecipado** para casos inválidos; caminho feliz no fim, sem indentação extra.
- **Argumentos nomeados** para números soltos: `calcular_limites(46.0, tol_pct=5.0)`.
- **Docstring em toda função** que outra pessoa (ou você daqui a um mês) vai usar.
- **Evite `global`.** Constantes em MAIÚSCULAS no topo do arquivo são a exceção bem-vinda.
- **Teste cada função assim que escrevê-la**, com um `assert` ou no REPL.

No mundo real

O coração do seu projeto final é: dado um pedido da expedição e o resultado do laboratório, montar o certificado. A regra: **se a OP tem especificação própria, ela vale; o que ela não define vem do padrão da linha**. E COF fora da faixa pode ser liberado pela qualidade com a observação "COF: liberado".

O `exemplos/11_mundo_real_certificado.py` resolve isso com funções puras e uma casca mínima. Os dados (`PADROES_DA_LINHA`, `ESPECIFICACOES` e `UNIDADES`) ficam no topo do arquivo; o núcleo é este:

```
def regras_da_op(op, linha):
    """Devolve (título, regras): especificação da OP sobre o padrão da
linha."""
    padrao = PADROES_DA_LINHA[linha]
    especificacao = ESPECIFICACOES.get(op)
    if especificacao is None:
        return f"Padrão da linha {linha}", padrao
    return especificacao["titulo"], padrao | especificacao["regras"]


def faixa(regra):
    """Converte {nominal, tol_pct} ou {min, max} em (mínimo, máximo)."""
    if "nominal" in regra:
        fator = regra["tol_pct"] / 100
        return round(regra["nominal"] * (1 - fator), 2),
```

```
    round(regra["nominal"] * (1 + fator), 2)
    return regra["min"], regra["max"]


def conforme(valor, regra):
    minimo, maximo = faixa(regra)
    return minimo <= valor <= maximo


def parecer(falhas, observacao):
    """APROVADO, APROVADO COM RESSALVA (só COF fora e liberado) ou
REPROVADO."""
    if not falhas:
        return "APROVADO"
    if falhas == ["cof"] and "cof: liberado" in observacao.lower():
        return "APROVADO COM RESSALVA (COF: liberado)"
    return "REPROVADO"


def montar_certificado(pedido, resultado):
    """Texto completo do certificado. Não imprime nada: devolve uma
string."""
    titulo, regras = regras_da_op(pedido["op"], resultado["linha"])
    medidas = resultado["medidas"]
    falhas = [p for p, regra in regras.items() if not
conforme(medidas[p], regra)]
    cabecalho = [
        "CERTIFICADO DE QUALIDADE",
        f"OP: {pedido['op']}   NF: {pedido['nf']}   Peso:
{pedido['peso_kg']} kg",
        f"Especificação: {titulo}",
        "-" * 50,
    ]
    tabela = [linha_da_tabela(p, medidas[p], regra) for p, regra in
regras.items()]
    rodape = ["-" * 50, f"Parecer: {parecer(falhas, resultado['obs'])}"]
    return "\n".join(cabecalho + tabela + rodape)  # "\n" = quebra de
linha
```

(`linha_da_tabela` formata uma linha da tabela com as casas decimais de cada propriedade; veja no arquivo.) A casca (`main`) só monta os dados de exemplo e imprime:

```
CERTIFICADO DE QUALIDADE
OP: OP-4512   NF: 000123456   Peso: 1250.5 kg
Especificação: Filme PE 50 µm alta selagem
--------------------------------------------------
espessura  51.2 µm     47.5 a 52.5    conforme
gramatura  46.3 g/m²   43.7 a 48.3    conforme
```

```
cof        0.22        0.15 a 0.30    conforme
-------------------------------------------------
Parecer: APROVADO

CERTIFICADO DE QUALIDADE
OP: OP-4514   NF: 000123457   Peso: 830.0 kg
Especificação: Padrão da linha A
-------------------------------------------------
espessura  49.1 µm     45.0 a 55.0    conforme
gramatura  45.8 g/m²   43.7 a 48.3    conforme
cof        0.33        0.15 a 0.30    NÃO conforme
-------------------------------------------------
Parecer: APROVADO COM RESSALVA (COF: liberado)
```

Repare em três decisões de projeto:

- **`padrao | especificacao["regras"]`** : a regra de negócio "especificação da OP sobre o padrão" é uma linha, e mora numa função só (`regras_da_op`).
- **`montar_certificado` devolve texto**: amanhã o mesmo texto pode ir para o `print` , para um arquivo, para o corpo de um e-mail ou para o robô que preenche o TOTVS. O núcleo não muda.
- **Cada função se testa com um `assert`** : `assert parecer([], "") == "APROVADO"` , `assert faixa({"min": 0.15, "max": 0.30}) == (0.15, 0.30)` . No módulo 09, esses asserts viram uma suíte de testes com pytest.

Resumo

Quero...	Sintaxe
definir e devolver	`def nome(a, b):` ... `return resultado`
devolver vários valores	`return minimo, maximo` → `mi, ma = f(...)`
valor padrão	`def f(valor, casas=1):`
padrão mutável (certo)	`def f(x, itens=None):` + `if itens is None: itens = []`
argumento nomeado	`f(46.0, tol_pct=5.0)`
só por nome	`def f(valor, *, casas=1):`
quantos posicionais vierem	`def f(*valores):` (tupla)
quantos nomeados vierem	`def f(**campos):` (dicionário)
espalhar na chamada	`f(*lista)` , `f(**dicionario)`
retorno antecipado	`if invalido: return None` no começo
docstring	`"""Devolve ..."""` logo abaixo do `def`

Quero...	Sintaxe
type hints	`def f(x: float, ys: list[str]) -> dict[str, int] | None:`
função como argumento	`cont_se(valores, acima_de_50)`
fábrica de funções (closure)	`def criar(m): def f(x): ... return f`
lambda	`sorted(itens, key=lambda i: i["peso"])`
recursão	caso base com `return` + chamada com problema menor
ver ajuda	`help(funcao)`

Você domina este módulo se...

- ☐ Explico, sem consultar, a diferença entre `print` e `return`, e por que uma função sem `return` devolve `None`.
- ☐ Prevejo o que acontece com uma lista usada como valor padrão, e escrevo a correção com `None` de memória.
- ☐ Sei dizer se uma variável é local ou global olhando o código, e explico um `UnboundLocalError`.
- ☐ Reescrevo uma "flecha" de `if` aninhados com retorno antecipado.
- ☐ Passo uma função como argumento, guardo funções num dicionário e escrevo uma função que devolve outra.
- ☐ Classifico uma função como pura ou não, e aponto os efeitos colaterais dela.
- ☐ Dado um problema de uma frase, desenho a árvore de funções, escrevo as assinaturas e implemento de baixo para cima, testando cada uma.
- ☐ Escrevo uma função recursiva com caso base e explico por que ela termina.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_dividir_lote`	★☆☆	vários retornos (tupla), retorno antecipado, uma função usando outra
02	`ex02_formatar_medida`	★☆☆	valores padrão, argumentos nomeados, formato com variável
03	`ex03_conceito_nota`	★☆☆	retorno antecipado, ordem das regras
04	`ex04_media_args`	★☆☆	`*args`, espalhar lista na chamada
05	`ex05_padrao_mutavel`	★☆☆	a armadilha do padrão mutável e o idioma `None`

Nº	Exercício	Nível	O que pratica
06	`ex06_funcoes_como_valores`	★★☆	função como argumento (CONT.SE/SOMASE), fábrica de funções
07	`ex07_ordenar_com_key`	★★☆	`lambda`, `key=` em `min` e `sorted`, critério com tupla
08	`ex08_filtrar_kwargs`	★★☆	`**kwargs`, filtro tipo WHERE
09	`ex09_leitura_escopo`	★★☆	**leitura de código:** prever escopo, padrão mutável, print × return; consertar uma função
10	`ex10_soma_recursiva`	★★☆	recursão em listas aninhadas
11	`ex11_laudo_decomposto`	★★★	**decomposição:** compor cinco funções pequenas num resumo de lote
12	`ex12_revisao_validar_pedido`	★★★	**revisão** 01+02+03: validadores pequenos, strings, `enumerate`, dicionários
13	`ex13_revisao_pipeline_limpeza`	★★★	**revisão** 01+02+03: `compor(*funcoes)`, métodos de string, conjunto para repetidos

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError` e rode, na pasta `curso-python`:

```
python curso.py testar 04 ex01
```

Para testar o módulo todo: `python curso.py testar 04`. Para ver seu progresso: `python curso.py progresso`.

> 💡 **Dica:** no exercício 11, siga os 6 passos da seção 11. Faça uma função por vez, rode o teste, e só depois passe para a próxima. Os testes de cada função são separados justamente para isso.

Desafio do módulo

Fila de emissão de certificados. Você recebe os pedidos do dia (alguns com erro de digitação), os resultados do laboratório, as especificações próprias de algumas OPs e os padrões de cada linha. Seu programa valida os pedidos, decide quais certificados podem ser emitidos (respeitando "especificação da OP sobre o padrão da linha" e "COF: liberado"), gera o texto de cada certificado e lista as pendências. Tudo com núcleo puro e casca fina.

Enunciado completo, requisitos e critérios de aceite: desafio/README.md.

Quiz

Teste o que ficou: quiz.md (14 perguntas, gabarito no final).

Para ir além

- Tutorial oficial, definindo funções: docs.python.org/pt-br/3/tutorial/controlflow.html#defining-functions
- Mais sobre funções (padrões, nomeados, `*args`, `**kwargs`, lambda, docstrings): docs.python.org/pt-br/3/tutorial/controlflow.html#more-on-defining-functions
- FAQ oficial: por que valores padrão são compartilhados entre chamadas: docs.python.org/pt-br/3/faq/programming.html
- Type hints, guia rápido do mypy (em inglês): mypy.readthedocs.io/en/stable/cheat_sheet_py3.html
- Leitura extra: capítulo "Funções" de *Código Limpo*, de Robert C. Martin. Use com bom senso: a ideia de funções pequenas e com um único propósito vale ouro; as regras radicais, nem tanto.
- Leitura extra: *Composing Programs*, capítulo 1 (em inglês, gratuito, baseado no clássico SICP): composingprograms.com

Enunciados dos exercícios

Exercício 04.01 — Dividir o lote em bobinas

Arquivo: `exercicios/ex01_dividir_lote.py`

```
Nível: ★☆☆

Contexto: A produção precisa saber em quantas bobinas cheias um lote vai
ser
dividido e quanto sobra. Às vezes alguém digita o peso da bobina como 0
na
planilha, e o programa não pode quebrar por isso.

Tarefa: implemente duas funções.
1. `dividir_em_bobinas(peso_total_kg, peso_bobina_kg)` devolve uma TUPLA
   (bobinas_cheias, sobra_kg), com a sobra arredondada com round(..., 2).
   Se peso_bobina_kg for 0 ou negativo, devolva None (retorno
antecipado).
2. `descrever_divisao(peso_total_kg, peso_bobina_kg)` usa a primeira
```

```
função e
   devolve um texto:
   - "4 bobinas de 300.0 kg + sobra de 50.5 kg"
   - "peso da bobina inválido"  (quando a primeira devolve None)
   Os pesos no texto aparecem com 1 casa decimal.

Exemplos:
    >>> dividir_em_bobinas(1250.5, 300)
    (4, 50.5)
    >>> dividir_em_bobinas(900, 300)
    (3, 0.0)
    >>> print(dividir_em_bobinas(500, 0))
    None
    >>> descrever_divisao(1250.5, 300)
    '4 bobinas de 300.0 kg + sobra de 50.5 kg'

Dica: // dá a parte inteira da divisão e % dá o resto (módulo 01). Na
segunda
função, desempacote a tupla: bobinas, sobra = resultado.

Teste com:  python curso.py testar 04 ex01
```

Exercício 04.02 — Formatar medidas para o laudo

Arquivo: `exercicios/ex02_formatar_medida.py`

```
Nível: ★☆☆

Contexto: O laudo mostra medidas de vários tipos: espessura em µm (1
casa),
gramatura em g/m², COF sem unidade (2 ou 3 casas), peso em kg. Às vezes o
cliente quer vírgula decimal. Em vez de uma função para cada caso, você
vai
escrever UMA função com valores padrão.

Tarefa: implemente `formatar_medida(valor, unidade="µm", casas=1,
decimal=".")`.
- Formate o valor com `casas` casas decimais.
- Se `decimal` for ",", troque o ponto decimal por vírgula.
- Se `unidade` não for vazia, acrescente um espaço e a unidade.
  Se for vazia (""), devolva só o número, sem espaço no fim.

Exemplos:
    >>> formatar_medida(50.24)
    '50.2 µm'
    >>> formatar_medida(46, "g/m²")
    '46.0 g/m²'
    >>> formatar_medida(0.2234, unidade="", casas=3)
    '0.223'
```

```
    >>> formatar_medida(1250.5, unidade="kg", decimal=",")
    '1250,5 kg'

Dica: o número de casas pode vir de uma variável dentro da f-string:
f"{valor:.{casas}f}" (seção 3 da aula).

Teste com:  python curso.py testar 04 ex02
```

Exercício 04.03 — Conceito e situação do aluno

Arquivo: `exercicios/ex03_conceito_nota.py`

```
Nível: ★☆☆

Contexto: Uma escola técnica converte a nota (0 a 10) em conceito e
decide a
situação final do aluno, que também depende da frequência.

Tarefa: implemente duas funções usando RETORNO ANTECIPADO (um return por
caso,
sem if aninhado).
1. `conceito(nota)` devolve:
   - "inválida" se a nota for menor que 0 ou maior que 10;
   - "A" se nota >= 9;  "B" se nota >= 7;  "C" se nota >= 5;  senão "D".
2. `situacao(nota, frequencia_pct)` devolve, nesta ordem de prioridade:
   - "reprovado por falta" se a frequência for menor que 75;
   - "nota inválida" se o conceito for "inválida";
   - "aprovado" se o conceito for A, B ou C;
   - "reprovado por nota" nos demais casos.
   Use a função conceito dentro de situacao.

Exemplos:
    >>> conceito(9.5)
    'A'
    >>> conceito(7)
    'B'
    >>> conceito(11)
    'inválida'
    >>> situacao(8.0, 60)
    'reprovado por falta'
    >>> situacao(4.9, 90)
    'reprovado por nota'

Dica: com retorno antecipado, a ordem dos ifs é a regra de negócio.
Escreva o
caso mais restritivo primeiro.

Teste com:  python curso.py testar 04 ex03
```

Exercício 04.04 — Média com quantos valores vierem

Arquivo: `exercicios/ex04_media_args.py`

```
Nível: ★☆☆

Contexto: O analista quer chamar a média direto no REPL, sem montar
lista:
media(50.2, 49.8, 51.0). E quer saber o maior desvio de um conjunto de
medições em relação ao nominal.

Tarefa: implemente duas funções com *args.
1. `media(*valores)` devolve a média dos valores recebidos.
   Sem nenhum valor, devolva None.
2. `maior_desvio(nominal, *medicoes)` devolve o maior desvio, em módulo,
entre
   cada medição e o nominal, arredondado com round(..., 2).
   Sem medições, devolva 0.0.

Exemplos:
    >>> media(50.0, 52.0)
    51.0
    >>> print(media())
    None
    >>> medicoes = [49.2, 51.5, 50.1]
    >>> media(*medicoes)
    50.26666666666667
    >>> maior_desvio(50.0, 49.2, 51.5, 50.1)
    1.5

Dica: dentro da função, `valores` é uma tupla comum. E abs(x) dá o
módulo.

Teste com:  python curso.py testar 04 ex04
```

Exercício 04.05 — A armadilha do padrão mutável

Arquivo: `exercicios/ex05_padrao_mutavel.py`

```
Nível: ★☆☆

Contexto: Um colega escreveu esta função para anotar defeitos de
inspeção:

    def adicionar_defeito(defeito, defeitos=[]):
        defeitos.append(defeito.strip().lower())
        return defeitos
```

```
Na primeira inspeção do dia, adicionar_defeito("Bolha") devolveu
["bolha"].
Na segunda, numa bobina sem relação nenhuma com a primeira,
adicionar_defeito("Risco") devolveu ["bolha", "risco"]. A "bolha" veio de
onde?
(Resposta na seção 3 da aula.)

Tarefa: implemente `adicionar_defeito(defeito, defeitos=None)` do jeito
certo.
- Normalize o defeito (strip + lower) antes de acrescentar.
- Se `defeitos` não for informado, comece uma lista NOVA a cada chamada.
- Se `defeitos` for informado, acrescente nessa lista (é o objetivo
declarado
  da função) e devolva essa mesma lista.

Exemplos:
    >>> adicionar_defeito("Bolha")
    ['bolha']
    >>> adicionar_defeito("Risco")
    ['risco']
    >>> anotados = ["gel"]
    >>> adicionar_defeito(" Bolha ", anotados)
    ['gel', 'bolha']
    >>> anotados
    ['gel', 'bolha']

Dica: o idioma é "if defeitos is None: defeitos = []".

Teste com:  python curso.py testar 04 ex05
```

Exercício 04.06 — CONT.SE e SOMASE em Python

Arquivo: `exercicios/ex06_funcoes_como_valores.py`

```
Nível: ★★☆

Contexto: No Excel, CONT.SE e SOMASE recebem um intervalo e um CRITÉRIO.
Em
Python, o critério pode ser uma FUNÇÃO que você passa como argumento. E
uma
função também pode FABRICAR outra função, já configurada.

Tarefa: implemente três funções.
1. `cont_se(valores, condicao)`: quantos valores fazem condicao(valor)
ser True.
2. `soma_se(valores, condicao)`: soma dos valores que fazem
condicao(valor)
   ser True (0.0 se nenhum), arredondada com round(..., 2).
3. `criar_verificador_faixa(minimo, maximo)`: devolve uma FUNÇÃO que
```

```
recebe
   um valor e diz (True/False) se ele está entre mínimo e máximo,
inclusive.

Exemplos:
    >>> cont_se([50.2, 44.1, 56.3], lambda v: v > 50)
    2
    >>> soma_se([500.0, 320.5, 80.0], lambda p: p >= 100)
    820.5
    >>> dentro = criar_verificador_faixa(45.0, 55.0)
    >>> dentro(50.2), dentro(56.3)
    (True, False)
    >>> cont_se([50.2, 44.1, 56.3], dentro)
    1

Dica: dentro de cont_se, `condicao` é uma variável que guarda uma função;
para
usá-la, chame: condicao(valor). Para a fábrica, defina uma função dentro
da
outra e devolva-a SEM parênteses (seção 8 da aula).

Teste com:  python curso.py testar 04 ex06
```

Exercício 04.07 — Ordenar com key e lambda

Arquivo: `exercicios/ex07_ordenar_com_key.py`

```
Nível: ★★☆

Contexto: O parâmetro key= de sorted, min e max recebe uma função que diz
O QUE
comparar. Com ela você responde perguntas como "qual medição está mais
perto do
nominal?" sem escrever nenhum laço.

Tarefa: implemente três funções, cada uma com UMA chamada a min ou
sorted.
1. `mais_proxima_do_nominal(medicoes, nominal)`: a medição com a menor
   distância até o nominal. Em empate, a que aparece primeiro.
   Lista vazia: None.
2. `ordenar_por_desvio(medicoes, nominal)`: lista NOVA, da medição mais
   próxima do nominal para a mais distante. Empates mantêm a ordem
original.
3. `ordenar_palavras(palavras)`: lista nova ordenada pelo tamanho da
palavra
   e, no empate, em ordem alfabética SEM diferenciar maiúsculas.

Exemplos:
    >>> mais_proxima_do_nominal([50.6, 49.8, 51.0], 50.0)
```

```
    49.8
    >>> ordenar_por_desvio([53.0, 50.5, 47.0, 49.0], 50.0)
    [50.5, 49.0, 53.0, 47.0]
    >>> ordenar_palavras(["Selagem", "cof", "tração", "Gel"])
    ['cof', 'Gel', 'tração', 'Selagem']

Dica: a key de ordenar_palavras devolve uma tupla: (tamanho, palavra em
minúsculas).

Teste com:  python curso.py testar 04 ex07
```

Exercício 04.08 — Filtros com **kwargs

Arquivo: `exercicios/ex08_filtrar_kwargs.py`

```
Nível: ★★☆

Contexto: Você quer uma função de filtro flexível, como um WHERE do SQL:
filtrar_registros(amostras, linha="A", status="aprovado"). Os nomes dos
critérios não são fixos: qualquer coluna pode virar filtro.

Tarefa: implemente duas funções.
1. `criar_amostra(op, linha, **medicoes)` devolve um dicionário com "op",
   "linha" (em maiúscula e sem espaços nas pontas) e cada medição
recebida
   por nome, nessa ordem.
2. `filtrar_registros(registros, **criterios)` devolve uma lista NOVA com
os
   registros em que, para TODOS os critérios, registro.get(coluna) ==
valor.
   Sem critérios, devolve todos os registros (numa lista nova).

Exemplos:
    >>> criar_amostra("OP-4512", " a", espessura=50.2, cof=0.22)
    {'op': 'OP-4512', 'linha': 'A', 'espessura': 50.2, 'cof': 0.22}
    >>> tabela = [
    ...     {"op": "OP-1", "linha": "A", "status": "aprovado"},
    ...     {"op": "OP-2", "linha": "B", "status": "aprovado"},
    ...     {"op": "OP-3", "linha": "A", "status": "reprovado"},
    ... ]
    >>> [r["op"] for r in filtrar_registros(tabela, linha="A")]
    ['OP-1', 'OP-3']
    >>> [r["op"] for r in filtrar_registros(tabela, linha="A",
status="aprovado")]
    ['OP-1']

Dica: dentro da função, `criterios` é um dicionário comum. all() com uma
expressão geradora (módulo 03) resolve o "para TODOS os critérios".
```

```
Teste com:  python curso.py testar 04 ex08
```

Exercício 04.09 — Leitura de código: escopo, padrão mutável e print × return

Arquivo: `exercicios/ex09_leitura_escopo.py`

```
Nível: ★★☆

Contexto: Antes de escrever funções, todo programador passa muito tempo
LENDO
funções dos outros. Aqui você lê, prevê e conserta.

Parte 1 — Preveja. Faça cada função devolver a sua previsão. Preveja no
papel
PRIMEIRO; depois rode no REPL para conferir (e entenda o porquê se
errou).

    # Trecho A: o que aparece na tela, linha por linha?
    def dobro(x):
        print(x * 2)

    resultado = dobro(5)
    print(resultado)

    # Trecho B: o que o último print mostra?
    total = 0

    def somar(valor):
        total = valor + 1
        return total

    somar(10)
    print(total)

    # Trecho C: o que o print mostra?
    def registrar(item, lista=[]):
        lista.append(item)
        return lista

    registrar("bolha")
    registrar("risco")
    print(registrar("gel"))

    # Trecho D: o que o print mostra?
    def zerar(valores):
        valores = []
```

```
        return valores

    def limpar(valores):
        valores.clear()

    a = [1, 2]
    b = [3, 4]
    zerar(a)
    limpar(b)
    print(a, b)

Parte 2 — Conserte. Esta função tem TRÊS defeitos:

    def resumo_lote(medicoes, minimo, maximo, fora=[]):
        for valor in medicoes:
            if valor < minimo or valor > maximo:
                fora.append(valor)
        pct = len(fora) / len(medicoes) * 100
        print(f"{pct:.1f}% fora da faixa")

Reescreva `resumo_lote(medicoes, minimo, maximo)` como função PURA:
devolve a
tupla (lista dos valores fora da faixa, percentual fora com round(...,
1)).
Lista vazia: ([], 0.0). Chamadas seguidas não podem interferir uma na
outra.

Exemplos:
    >>> resumo_lote([50.2, 44.1, 56.3, 49.9], 45.0, 55.0)
    ([44.1, 56.3], 50.0)
    >>> resumo_lote([], 45.0, 55.0)
    ([], 0.0)

Dica: para a Parte 2, pergunte de cada linha: "isso depende de algo de
fora da
função?" e "isso devolve o resultado ou só mostra?".

Teste com:  python curso.py testar 04 ex09
```

Exercício 04.10 — Recursão: pesos aninhados

Arquivo: `exercicios/ex10_soma_recursiva.py`

```
Nível: ★★☆

Contexto: Um carregamento é uma lista de paletes; cada palete é uma lista
de
caixas ou de bobinas; uma caixa pode ter outras caixas dentro... Não dá
para
```

```
saber quantos níveis existem. Esse é o tipo de estrutura em que a
RECURSÃO
brilha.

    carga = [[100.0, 120.5], [80.0, [40.0, 40.0]], 10.0]

Aqui há dois paletes e uma bobina avulsa (10.0). O 2º palete tem uma
bobina
de 80.0 e uma caixa com duas bobinas de 40.0.

Tarefa: implemente duas funções RECURSIVAS (que chamam a si mesmas).
1. `soma_aninhada(itens)` devolve a soma de todos os números, em qualquer
   profundidade.
2. `contar_valores(itens)` devolve quantos números existem, em qualquer
   profundidade (as listas em si não contam).
Listas vazias (em qualquer nível) somam 0 e têm 0 valores.

Exemplos:
    >>> carga = [[100.0, 120.5], [80.0, [40.0, 40.0]], 10.0]
    >>> soma_aninhada(carga)
    390.5
    >>> contar_valores(carga)
    6
    >>> soma_aninhada([[], [[]]])
    0

Dica: para cada item, pergunte isinstance(item, list). Se for lista,
chame a
própria função para ela (caso recursivo). Se for número, use o número
(caso
base).

Teste com:  python curso.py testar 04 ex10
```

Exercício 04.11 — Laudo decomposto em funções pequenas

Arquivo: `exercicios/ex11_laudo_decomposto.py`

```
Nível: ★★★

Contexto: O supervisor quer uma linha de resumo por OP no fim do turno:

    OP-4512 | 5/6 conformes (83.3%) | 1 abaixo, 0 acima | REPROVADO

Em vez de uma função gigante, você vai DECOMPOR o problema (seção 11 da
aula):
cada função faz uma coisa só, e a última apenas junta as outras.

Tarefa: implemente as cinco funções, nesta ordem (de baixo para cima).
```

```
1. `calcular_limites(nominal, tol_pct)` -> (mínimo, máximo), nominal ∓
tol_pct %,
   cada um com round(..., 2).
2. `classificar(valor, limites)` -> "abaixo", "ok" ou "acima". `limites`
é a
   tupla (mínimo, máximo); os limites contam como "ok".
3. `classificar_todas(medicoes, limites)` -> lista com a classe de cada
medição.
4. `contar_por_classe(classes)` -> dicionário SEMPRE com as três chaves,
na
   ordem {"abaixo": n, "ok": n, "acima": n} (zero quando não houver).
5. `resumo_lote(op, medicoes, nominal, tol_pct)` -> o texto do resumo:
   "{op} | {ok}/{total} conformes ({pct:.1f}%) | {abaixo} abaixo, {acima}
acima | {situação}"
   - situação: "APROVADO" se todas estiverem ok; senão "REPROVADO".
   - sem medições: "{op} | sem medições".
   resumo_lote deve USAR as outras quatro funções.
Nenhuma função imprime nada nem altera a lista recebida.

Exemplos:
    >>> calcular_limites(50.0, 10.0)
    (45.0, 55.0)
    >>> classificar(44.9, (45.0, 55.0))
    'abaixo'
    >>> classificar_todas([50.2, 44.1, 56.3], (45.0, 55.0))
    ['ok', 'abaixo', 'acima']
    >>> contar_por_classe(["ok", "ok", "acima"])
    {'abaixo': 0, 'ok': 2, 'acima': 1}
    >>> resumo_lote("OP-4512", [50.2, 49.8, 44.1, 51.0, 50.5, 49.9],
50.0, 10.0)
    'OP-4512 | 5/6 conformes (83.3%) | 1 abaixo, 0 acima | REPROVADO'
    >>> resumo_lote("OP-4513", [], 50.0, 10.0)
    'OP-4513 | sem medições'

Dica: teste cada função no REPL antes de escrever a próxima. Quando
chegar em
resumo_lote, ela deve ter poucas linhas: só chamadas e a montagem do
texto.

Teste com:  python curso.py testar 04 ex11
```

Exercício 04.12 — Revisão: validar os pedidos da expedição

Arquivo: `exercicios/ex12_revisao_validar_pedido.py`

```
Nível: ★★★

Contexto: Este exercício mistura o módulo 04 (decomposição, retorno
antecipado,
```

funções pequenas) com o 01 (métodos de string), o 02 (validação, laços) e o 03
(dicionários, listas, enumerate). Os pedidos chegam do e-mail da expedição
assim:

    {"op": " op-45120", "nf": "000123456", "peso_kg": 1250.5}

Tarefa: implemente cinco funções. Cada validador devolve None quando está tudo
certo, ou o TEXTO do problema.
1. `validar_op(op)`: limpe com strip + upper.
   - vazia -> "OP vazia"
   - precisa começar com "OP-" e ter de 4 a 6 dígitos depois do hífen;
     senão -> "OP inválida: 'OP-12'" (com o texto já limpo entre aspas simples)
2. `validar_nf(nf)`: limpe com strip.
   - vazia -> "NF vazia"
   - só dígitos e no máximo 9 deles; senão -> "NF inválida: '12A'"
3. `validar_peso(peso_kg)`:
   - None -> "peso ausente"
   - menor ou igual a 0 -> "peso deve ser maior que zero"
   - maior que 30000 -> "peso acima de 30000 kg (confira a unidade)"
4. `validar_pedido(pedido)`: devolve a LISTA de problemas, na ordem op, nf,
   peso (lista vazia = pedido válido). Chave ausente ou com None conta como
   vazia (op, nf) ou ausente (peso).
5. `separar_pedidos(pedidos)`: devolve a tupla (validos, mensagens):
   - validos: dicionários NOVOS dos pedidos sem problema, com "op" limpa
     (strip + upper) e "nf" limpa (strip); as demais chaves ficam iguais;
   - mensagens: um texto por pedido com problema, com a posição (a partir de
     1) e os problemas unidos por "; ":
     "pedido 2: OP inválida: 'OP-12'; peso deve ser maior que zero"

Exemplos:
    >>> validar_op(" op-45120")
    >>> validar_op("OP-12")
    "OP inválida: 'OP-12'"
    >>> validar_nf("12A")
    "NF inválida: '12A'"
    >>> validar_pedido({"op": "OP-4512", "nf": "", "peso_kg": None})
    ['NF vazia', 'peso ausente']
    >>> validos, mensagens = separar_pedidos([
    ...   {"op": " op-45120", "nf": "123456", "peso_kg": 1250.5},
    ...   {"op": "OP-12", "nf": "123", "peso_kg": 0},
    ...])
    >>> validos
    [{'op': 'OP-45120', 'nf': '123456', 'peso_kg': 1250.5}]

```
    >>> mensagens
    ["pedido 2: OP inválida: 'OP-12'; peso deve ser maior que zero"]

Dica: dois métodos de string ajudam (seção 11 da aula):
texto.startswith("OP-")
e texto.isdigit(). Para "chave ausente ou None", use pedido.get("op") or
"".

Teste com:  python curso.py testar 04 ex12
```

Exercício 04.13 — Revisão: pipeline de limpeza de códigos

Arquivo: `exercicios/ex13_revisao_pipeline_limpeza.py`

```
Nível: ★★★

Contexto: Este exercício mistura o módulo 04 (funções como valores,
*args,
closures) com o 01 (strip, upper, replace), o 02 (laços e decisões) e o
03
(conjunto para tirar repetidos). Os códigos de OP chegam sujos de vários
lugares: " op 4512", "OP-4512", "op-4513 ", "". Em vez de uma função
cheia de
ifs, você vai montar uma LINHA DE MONTAGEM de funções pequenas.

Tarefa: implemente três funções.
1. `compor(*funcoes)` devolve UMA função nova que recebe um valor e
aplica as
   funções em sequência, da esquerda para a direita:
   compor(f, g, h)(x) é o mesmo que h(g(f(x))).
   compor() sem funções devolve uma função que devolve o próprio valor.
2. `trocar_espacos_por_hifen(texto)` troca cada espaço por "-".
3. `limpar_codigos(codigos)`: monte, com compor, a limpeza
   strip -> upper -> trocar_espacos_por_hifen e aplique a cada código.
   Descarte os que ficarem vazios e tire os repetidos, mantendo a ordem
da
   primeira aparição. Devolva a lista nova.

Exemplos:
    >>> def dobrar(x):
    ...     return x * 2
    >>> def somar_um(x):
    ...     return x + 1
    >>> compor(dobrar, somar_um)(3)
    7
    >>> compor(somar_um, dobrar)(3)
    8
    >>> compor()(5)
    5
```

```
    >>> limpar = compor(str.strip, str.upper)
    >>> limpar("  op-1 ")
    'OP-1'
    >>> limpar_codigos([" op 4512", "OP-4512", "", "op-4513 "])
    ['OP-4512', 'OP-4513']

Dica: str.strip é a própria função do método: str.strip("  a ") devolve
"a".
Dentro de compor, defina uma função que percorre `funcoes` aplicando uma
de
cada vez (um acumulador, como no módulo 02) e devolva essa função.

Teste com:  python curso.py testar 04 ex13
```

Desafio do módulo 04 — Fila de emissão de certificados

Contexto

Fim do dia. A expedição mandou os pedidos de amanhã (OP, NF e peso), alguns com erro de digitação. O laboratório tem os resultados de quase todas as OPs. Algumas OPs têm **especificação própria** (título + regras); as outras seguem o **padrão da linha**. E a qualidade pode liberar um COF fora da faixa com a observação "COF: liberado".

Seu programa monta a **fila de emissão**: o texto de cada certificado que pode sair e a lista de pendências, com o motivo de cada uma. É o núcleo do projeto final, só que com os dados no código (planilha, banco e e-mail entram nos próximos módulos).

O objetivo principal não é só "funcionar": é **decompor** o problema em funções pequenas e puras, com uma casca fina que imprime.

Dados de entrada

Crie `desafio/fila.py` e cole os dados no topo:

```python
PADROES_DA_LINHA = {
    "A": {
        "espessura": {"nominal": 50.0, "tol_pct": 10.0},
        "gramatura": {"nominal": 46.0, "tol_pct": 5.0},
        "cof": {"min": 0.15, "max": 0.30},
    },
    "B": {
        "espessura": {"nominal": 20.0, "tol_pct": 8.0},
        "gramatura": {"nominal": 18.5, "tol_pct": 5.0},
```

```
        "cof": {"min": 0.20, "max": 0.40},
    },
}

ESPECIFICACOES = {
    "OP-6101": {
        "titulo": "Filme PE 50 µm alta selagem",
        "regras": {"espessura": {"nominal": 50.0, "tol_pct": 5.0}},
    },
    "OP-6104": {
        "titulo": "Filme BOPP 20 µm baixo atrito",
        "regras": {"cof": {"min": 0.10, "max": 0.25}},
    },
}

PEDIDOS = [
    {"op": "OP-6101", "nf": "000501", "peso_kg": 1250.5},
    {"op": " op-6102", "nf": "000502", "peso_kg": 830.0},
    {"op": "OP-6103", "nf": "000503", "peso_kg": 640.2},
    {"op": "OP-6104", "nf": "000504", "peso_kg": 910.0},
    {"op": "OP-61", "nf": "5O5", "peso_kg": 0},
    {"op": "OP-6106", "nf": "000506", "peso_kg": 300.0},
    {"op": "OP-6107", "nf": "000507", "peso_kg": 455.5},
]

RESULTADOS = {
    "OP-6101": {
        "linha": "A",
        "obs": "",
        "medidas": {"espessura": 51.2, "gramatura": 46.3, "cof": 0.22},
    },
    "OP-6102": {
        "linha": "A",
        "obs": "COF: liberado",
        "medidas": {"espessura": 49.1, "gramatura": 45.8, "cof": 0.33},
    },
    "OP-6103": {
        "linha": "B",
        "obs": "",
        "medidas": {"espessura": 22.4, "gramatura": 18.9, "cof": 0.31},
    },
    "OP-6104": {
        "linha": "B",
        "obs": "",
        "medidas": {"espessura": 20.3, "gramatura": 18.2, "cof": 0.18},
    },
    "OP-6107": {"linha": "A", "obs": "", "medidas": {"espessura": 50.6,
"cof": 0.19}},
}
```

Regras de negócio

1. **Pedido inválido** (não vai adiante): OP limpa (strip + upper) precisa ser `"OP-"` + 4 a 6 dígitos; NF limpa precisa ser só dígitos; peso precisa existir e ser maior que zero. Mostre **todos** os problemas do pedido, não só o primeiro.
2. **Aguardando**: pedido válido cuja OP não tem resultado no laboratório.
3. **Regras da OP**: `padrão da linha | regras da especificação` (a especificação vence nas propriedades que define). O título é o da especificação ou "Padrão da linha X".
4. **Conferência**: `nominal ± tol_pct %` (limites com 2 casas) ou faixa `min` – `max`, limites inclusive. Propriedade sem medição conta como falha.
5. **Parecer**: sem falhas → `APROVADO`; a única falha é o COF e a observação contém "cof: liberado" (sem diferenciar maiúsculas) → `APROVADO COM RESSALVA (COF: liberado)`; senão → `REPROVADO` (o pedido fica **bloqueado**).
6. **Certificado** (só para aprovados): OP, NF, peso, título da especificação, cada propriedade com resultado e faixa, e o parecer.

Requisitos obrigatórios

- [ ] Pelo menos **6 funções** no núcleo, cada uma com uma tarefa, docstring e no máximo ~20 linhas.
- [ ] **Nenhuma função do núcleo usa `print`** nem altera os dados recebidos. Só a `main` imprime.
- [ ] Nenhuma variável global é alterada (constantes em MAIÚSCULAS só são lidas).
- [ ] Nenhum valor padrão mutável.
- [ ] A função que decide o destino de **um** pedido devolve um dicionário (por exemplo `{"op": ..., "destino": "emitir", ...}`), e outra função monta a fila aplicando-a a todos.
- [ ] Ao menos **5 `assert`** no fim do arquivo testando funções do núcleo isoladamente.
- [ ] A saída mostra os certificados, as pendências com o motivo e uma linha de resumo com a contagem por destino e o peso total a emitir.

Extras opcionais

- Use um **dicionário de funções** para os tipos de regra (`"nominal"`, `"faixa"`) em vez de `if "nominal" in regra` (seção 8 da aula).
- Mostre o peso no formato brasileiro (`1.250,5 kg`).
- Acrescente um tipo de regra "só mínimo" para tração (`{"min": 25.0}`) e uma OP que o use.
- Ordene as pendências por destino e, dentro de cada destino, por OP.
- Escreva `compor(*funcoes)` (exercício 13) e monte a limpeza da OP com ele.

Exemplo de execução

```
python desafio\fila.py
```

(No Linux/macOS: `python3 desafio/fila.py`.) Saída esperada para os dados acima:

```
=== CERTIFICADOS A EMITIR ===

CERTIFICADO DE QUALIDADE - OP-6101
  NF 000501  |  1250.5 kg  |  Filme PE 50 µm alta selagem
  espessura  51.2 µm     47.5 a 52.5
  gramatura  46.3 g/m²   43.7 a 48.3
  cof        0.22        0.15 a 0.30
  Parecer: APROVADO

CERTIFICADO DE QUALIDADE - OP-6102
  NF 000502  |  830.0 kg  |  Padrão da linha A
  espessura  49.1 µm     45.0 a 55.0
  gramatura  45.8 g/m²   43.7 a 48.3
  cof        0.33        0.15 a 0.30
  Parecer: APROVADO COM RESSALVA (COF: liberado)

CERTIFICADO DE QUALIDADE - OP-6104
  NF 000504  |  910.0 kg  |  Filme BOPP 20 µm baixo atrito
  espessura  20.3 µm     18.4 a 21.6
  gramatura  18.2 g/m²   17.6 a 19.4
  cof        0.18        0.10 a 0.25
  Parecer: APROVADO

=== PENDÊNCIAS ===
  OP-6103   bloqueado   fora da especificação: espessura
  OP-61     inválido    OP inválida 'OP-61'; NF inválida '5O5'; peso deve
ser maior que zero
  OP-6106   aguardando  sem resultado do laboratório
  OP-6107   bloqueado   fora da especificação: gramatura (sem medição)

Fila: 3 emitir, 2 bloqueado, 1 aguardando, 1 inválido | peso a emitir:
2990.5 kg
```

O layout pode variar; os destinos, os motivos e os números não.

Critérios de aceite

- [ ] Roda sem erro e sem interação.
- [ ] Emite exatamente `OP-6101`, `OP-6102` (com ressalva) e `OP-6104`, com peso total 2990.5 kg.

- [ ] `OP-6104` usa a faixa de COF **da especificação** (0.10 a 0.25), e não a da linha B (0.20 a 0.40): por isso o COF 0.18 passa.
- [ ] `OP-6101` usa a espessura da especificação (47.5 a 52.5), mas a gramatura e o COF **do padrão** da linha A.
- [ ] O pedido `OP-61` aparece como inválido com os **três** problemas.
- [ ] `OP-6107` fica bloqueada por falta de medição de gramatura (e não quebra o programa).
- [ ] Os `assert` do fim do arquivo passam.
- [ ] `ruff check desafio` não aponta nada.

Dicas de como começar

1. **Não comece pela `main`.** Escreva o problema numa frase e desenhe a árvore de funções (seção 11 da aula). Uma sugestão:

   ```
   montar_fila
   +-- processar_pedido
       +-- problemas_do_pedido
       +-- regras_da_op
       +-- falhas_das_medidas
       |   +-- faixa
       +-- parecer
       +-- montar_certificado
   ```

2. Implemente **de baixo para cima**: `faixa` primeiro, testada com `assert faixa({"nominal": 46.0, "tol_pct": 5.0}) == (43.7, 48.3)`.

3. `regras_da_op` é uma linha com o operador `|` de dicionários (módulo 03). Teste com uma OP com especificação e uma sem.

4. `processar_pedido` usa **retorno antecipado**: inválido? devolve. Sem resultado? devolve. Reprovado? devolve. O caminho feliz (emitir) fica no fim.

5. Só depois escreva a `main`, que percorre a fila e imprime.

Travou? A solução de referência está em solucao_referencia.py. Tente por pelo menos 40 minutos antes de abrir. Depois, compare as árvores de funções: a sua e a de referência não precisam ser iguais, mas ambas devem ser fáceis de testar.

Quiz do módulo 04 — Funções

Responda no papel **antes** de rodar qualquer código. Depois confira no REPL e no gabarito do final.

Perguntas

1. O que este código imprime?

```
def f(x):
    x * 2

print(f(3))
```

2. Qual destes cabeçalhos dá erro de sintaxe?

- a) `def f(a, b=1):`
- b) `def f(a=1, b):`
- c) `def f(*args):`
- d) `def f(a, *, b=2):`

3. O que este código imprime?

```
def adicionar(x, lista=[]):
    lista.append(x)
    return lista

adicionar(1)
print(adicionar(2))
```

4. O que este código imprime?

```
x = 10

def f():
    x = 5
    return x

f()
print(x)
```

5. Onde está o erro? O que acontece ao rodar e como corrigir sem usar `global`?

```
contador = 0

def registrar():
    contador += 1

registrar()
```

6. O que este código imprime?

```python
def f(*args, **kwargs):
    return len(args), list(kwargs)

print(f(1, 2, a=3))
```

7. O que este código imprime?

```python
def limites(n, p=10):
    return n - n * p / 100, n + n * p / 100

mi, ma = limites(50, p=4)
print(mi, ma)
```

8. Qual destas funções é **pura**?

- a) `def f(lista): lista.append(1); return lista`
- b) `def f(lista): return lista + [1]`
- c) `def f(x): print(x); return x`
- d) `def f(x): global total; total += x; return total`

9. O que este código imprime?

```python
def criar(m):
    def f(x):
        return x * m
    return f

dobro = criar(2)
triplo = criar(3)
print(dobro(5), triplo(5))
```

10. O que este código imprime?

```python
print(sorted(["bb", "a", "ccc", "B"], key=lambda s: (len(s), s.lower())))
```

11. O que este código imprime?

```python
def g(lista):
    lista = lista + [9]
    return len(lista)

nums = [1, 2]
print(g(nums), nums)
```

12. Onde está o erro? O que acontece ao chamar `soma_ate(3)` e como corrigir?

```python
def soma_ate(n):
    """Soma 1 + 2 + ... + n."""
    return n + soma_ate(n - 1)
```

13. O que este código imprime?

```python
def f(a, b):
    return a - b

print(f(b=1, a=5))
```

14. Pensando em decomposição, qual destes nomes de função indica que ela deveria ser quebrada em duas?

- a) `calcular_media`
- b) `ler_planilha_e_enviar_email`
- c) `formatar_linha`
- d) `esta_na_faixa`

Gabarito

▼ **Ver respostas**

1. `None`. A função calcula `x * 2` mas não devolve: falta o `return`. Toda função sem `return` devolve `None`.

2. b) Um parâmetro sem valor padrão não pode vir depois de um com padrão: `SyntaxError: parameter without a default follows parameter with a default`.

3. `[1, 2]`. A lista padrão é criada uma vez só, no `def`, e as duas chamadas usam a mesma. Correção: `lista=None` e `if lista is None: lista = []`.

4. `10`. O `x = 5` dentro da função cria uma variável **local**. O `x` global não muda.

5. `UnboundLocalError: cannot access local variable 'contador' where it is not associated with a value`. O `+=` torna `contador` local na função inteira, e ela é lida antes de ter valor. Correção: `def registrar(contador): return contador + 1` e, fora, `contador = registrar(contador)`.

6. `(2, ['a'])`. `*args` recolhe os posicionais numa tupla `(1, 2)`; `**kwargs` recolhe os nomeados num dicionário `{"a": 3}`, e `list()` de um dicionário dá as chaves.

7. `48.0 52.0`. O `p=4` substitui o padrão 10; a função devolve uma tupla, desempacotada em `mi` e `ma`.

8. b) `lista + [1]` cria uma lista nova e não mexe em nada de fora. A a) altera a lista recebida, a c) imprime e a d) altera uma global: todas têm efeitos colaterais.

9. `10 15`. Cada chamada de `criar` gera uma função que "lembra" o seu próprio `m` (closure).

10. `['a', 'B', 'bb', 'ccc']`. A key ordena pelo tamanho; no empate (`"a"` e `"B"`, tamanho 1), compara em minúsculas: `"a"` antes de `"b"`.

11. `3 [1, 2]`. `lista + [9]` cria uma lista nova e move só a etiqueta local; `nums` não é tocada.

12. Não há caso base: a função se chama para sempre (`n` vai a 0, -1, -2...) até `RecursionError: maximum recursion depth exceeded`. Correção: começar com `if n == 0: return 0` (ou `if n <= 0`).

13. `4`. Argumentos nomeados casam pelo nome, não pela ordem: `a=5`, `b=1`.

14. b) O "e" no nome denuncia duas tarefas (ler e enviar), e uma delas nem é cálculo: são dois efeitos colaterais diferentes. Quebre em `ler_planilha` e `enviar_email`, com o cálculo numa função pura no meio.

Módulo 05 — Texto e expressões regulares

> **Tempo estimado:** 10 a 14 horas · **Pré-requisitos:** módulos 00 a 04 · **Nível:** iniciante → intermediário

O que você vai aprender

- Fatiar, procurar e limpar textos com os métodos certos de `str`.
- Quebrar e juntar textos com `split`, `partition` e `join`.
- Montar relatórios alinhados com f-strings (largura, alinhamento, milhar, casas).
- Converter `"1.250,50"` em `1250.5` (e o caminho de volta) sem cair nas armadilhas.
- Remover acentos e comparar nomes de forma confiável com `unicodedata`.
- Explicar a diferença entre `str` e `bytes` e consertar o famoso `ExpediÃ§Ã£o`.
- Escrever uma expressão regular do zero, testando cada pedaço no REPL.
- Usar `re.search`, `re.match`, `re.fullmatch`, `re.findall`, `re.finditer` e `re.sub`.
- Extrair dados com grupos e grupos nomeados, e usar as flags `IGNORECASE`, `MULTILINE` e `VERBOSE`.
- Transformar o e-mail da expedição numa lista de pedidos com OP, NF e peso.

Por que isso importa

Quase todo dado do mundo real chega como **texto**. O e-mail da expedição, a célula da planilha com `25,4`, o código `op-004512`, o nome do cliente digitado de três jeitos. Antes de calcular qualquer coisa, alguém precisa ler e limpar esse texto.

No mercado isso tem nome: *parsing* e *limpeza de dados*. É uma parte enorme do trabalho de quem faz automação e análise de dados. E é exatamente o que separa um script que "funciona no meu exemplo" de um que aguenta o e-mail real da segunda-feira.

No projeto final, este módulo é a peça que lê os pedidos da expedição (OP, NF e peso), entende números brasileiros e reconhece a observação "COF: liberado".

> 💡 **Dica:** deixe o REPL aberto (`python` no PowerShell) e digite os exemplos junto com a aula. Texto se aprende testando. As versões completas estão em `exemplos/`, na ordem das seções (`python 05-texto-e-`

> `regex/exemplos/01_fatiar_e_limpar.py`).

1. Strings por dentro

Uma string é uma **sequência de caracteres**. Cada caractere tem uma posição (o índice), que começa em 0. Índices negativos contam a partir do fim.

```
texto:     O   P   -   0   0   4   5   1   2
índice:    0   1   2   3   4   5   6   7   8
negativo: -9  -8  -7  -6  -5  -4  -3  -2  -1
```

```
>>> op = "OP-004512"
>>> op[0]
'O'
>>> op[-1]
'2'
>>> op[3:]
'004512'
>>> len(op)
9
```

O fatiamento é `texto[início:fim:passo]`. O `início` entra, o `fim` não entra. Sem `início`, começa do zero; sem `fim`, vai até o final.

```
>>> lote = "2025-A-0153"
>>> lote[:4]      # ano
'2025'
>>> lote[5]       # linha de produção
'A'
>>> lote[-4:]     # sequencial
'0153'
>>> lote[::-1]    # passo -1: de trás para frente
'3510-A-5202'
>>> lote[7:100]   # fatiar além do fim não dá erro
'0153'
```

Já um índice isolado fora da faixa (`lote[50]`) dá `IndexError`.

Strings são imutáveis

Você não troca uma letra "dentro" de uma string. Todo método devolve uma string **nova** e deixa a original como estava.

```
>>> codigo = "op-004512"
>>> codigo[0] = "O"
Traceback (most recent call last):
  ...
TypeError: 'str' object does not support item assignment
>>> codigo.upper()
'OP-004512'
>>> codigo
'op-004512'
>>> codigo = codigo.upper()   # para guardar o resultado, reatribua
>>> codigo
'OP-004512'
```

⚠️ **Erro comum:** escrever `texto.strip()` sozinho numa linha e achar que `texto` mudou. Não mudou. Escreva `texto = texto.strip()`.

🔮 **Preveja antes de rodar:** anote o que cada `print` mostra. Depois rode.

```
nf = "000012345"
print(nf[-5:])
print(nf[::2])
print(nf[20:])
```

▼ **Resposta**

`12345`, depois `00135` e por fim uma linha vazia.

`nf[::2]` pega as posições 0, 2, 4, 6 e 8 (`0`, `0`, `1`, `3`, `5`). `nf[20:]` começa depois do fim: o resultado é a string vazia `''`, sem erro.

2. Procurando dentro do texto

Antes de pensar em expressão regular, conheça as ferramentas simples. Elas resolvem a maioria dos casos.

```
>>> assunto = "Liberação de carga - OP 004512 - Cliente Plastifort"
>>> "OP" in assunto
True
>>> assunto.find("OP")      # posição da primeira ocorrência
21
>>> assunto.find("NF")      # -1 quando não encontra
```

```
-1
>>> assunto.count("a")
4
>>> assunto.startswith("Liberação")
True
>>> "LAUDO_0153.PDF".lower().endswith((".pdf", ".xlsx"))
True
```

`startswith` e `endswith` aceitam uma **tupla** de opções. Muito útil para extensões de arquivo.

> 💡 **Dica:** use `in` quando só quer saber "tem ou não tem". Use `find` quando precisa da posição. Existe também `index`, que faz o mesmo que `find`, mas dá `ValueError: substring not found` quando não acha.

3. Limpando e padronizando

Texto digitado por gente tem espaço sobrando, maiúscula aleatória e prefixo repetido. Estes são os métodos de limpeza do dia a dia:

```
>>> bruto = "   embalagens   DO sul  ltda \n"
>>> bruto.strip()                  # tira espaços e quebras de linha das pontas
'embalagens   DO sul  ltda'
>>> "***ATENÇÃO***".strip("*")     # tira os caracteres indicados
'ATENÇÃO'
>>> "NF-e 12345".removeprefix("NF-e ")
'12345'
>>> "laudo_0153.pdf".removesuffix(".pdf")
'laudo_0153'
>>> "12345".zfill(9)               # completa com zeros à esquerda
'000012345'
>>> "cof ESTÁTICO".capitalize()    # só a primeira letra maiúscula
'Cof estático'
>>> "tração na ruptura".title()    # primeira letra de cada palavra
'Tração Na Ruptura'
>>> "Peso: 1.250,5 kg".replace(" kg", "")
'Peso: 1.250,5'
```

Para checar o conteúdo, existem os métodos `is...`: `"123456".isdigit()` dá `True`, `"25,4".isdigit()` dá `False` (a vírgula não é dígito), `"OP".isalpha()` dá `True`.

> 🔮 **Preveja antes de rodar:** você quer tirar a extensão `.csv` do nome do arquivo.

```
print("laudos.csv".rstrip(".csv"))
print("laudos.csv".removesuffix(".csv"))
```

▼ **Resposta**

A primeira linha mostra `laudo` (sumiu o `s`!). A segunda mostra `laudos`.

`strip`, `lstrip` e `rstrip` recebem um **conjunto de caracteres**, não uma palavra. `rstrip(".csv")` vai arrancando do fim qualquer `.`, `c`, `s` ou `v` que encontrar, inclusive o `s` de "laudos". Para tirar um prefixo ou sufixo exato, use `removeprefix` e `removesuffix` (Python 3.9+).

4. Quebrando e juntando: split, partition e join

`split` quebra a string numa **lista** de pedaços. Você escolhe o separador.

```
>>> linha = "OP: 123456 | NF: 98765 | Peso: 1.250,5 kg"
>>> linha.split("|")
['OP: 123456 ', ' NF: 98765 ', ' Peso: 1.250,5 kg']
>>> [parte.strip() for parte in linha.split("|")]
['OP: 123456', 'NF: 98765', 'Peso: 1.250,5 kg']
>>> "Peso: 1.250,5 kg".split(":", maxsplit=1)
['Peso', ' 1.250,5 kg']
```

Sem argumento, `split()` é especial: quebra em **qualquer** espaço em branco (espaço, tab, quebra de linha) e ignora repetições. Com `split(" ")`, cada espaço conta.

```
>>> "  embalagens   DO sul\tltda ".split()
['embalagens', 'DO', 'sul', 'ltda']
>>> "  embalagens   DO".split(" ")
['', '', 'embalagens', '', '', 'DO']
```

`partition` quebra só na **primeira** ocorrência e sempre devolve 3 partes: antes, separador, depois. Ótimo para "rótulo: valor", porque o valor pode ter `:` dentro.

```
>>> "Horário: 14:30".partition(":")
('Horário', ':', ' 14:30')
>>> "sem separador".partition(":")
('sem separador', '', '')
```

`splitlines` quebra um texto em linhas, entendendo `\n` (Linux) e `\r\n` (Windows):

```
>>> email = "Bom dia!\r\nSeguem as OPs.\nAtt."
>>> email.splitlines()
['Bom dia!', 'Seguem as OPs.', 'Att.']
```

`join` é o contrário de `split`: junta uma lista de strings usando um "cola".

```
>>> "-".join(["2025", "03", "05"])
'2025-03-05'
>>> " ".join("  embalagens   DO sul\tltda ".split())   # truque: espaços normalizados
'embalagens DO sul ltda'
```

> ⚠️ **Erro comum:** `join` só aceita strings; com números dá `TypeError`. Converta antes: `", ".join(str(x) for x in valores)`. E o separador vem **antes** do ponto: `", ".join(lista)`, e não `lista.join(", ")`.

Exemplo resolvido: padronizar o código de lote

Problema: o operador digita o lote de vários jeitos (`" 2025 a 153 "`, `"2025-a-153"`). O padrão da empresa é `2025-A-0153` (ano, linha, sequencial com 4 dígitos).

Primeiro eu penso nos **pedaços**: ano, linha, sequencial. Se eu trocar o hífen por espaço, consigo usar `split()` para separar os três, não importa quantos espaços tenham. Depois eu formato cada pedaço e junto com `-`.

```
>>> def padronizar_lote(texto):
...     ano, linha, seq = texto.replace("-", " ").split()
...     return "-".join([ano, linha.upper(), seq.zfill(4)])
...
>>> padronizar_lote(" 2025 a 153 ")
'2025-A-0153'
>>> padronizar_lote("2025-b-7")
'2025-B-0007'
```

Repare no desempacotamento `ano, linha, seq = ...` (módulo 03). Se o texto tiver mais ou menos de 3 pedaços, ele dá `ValueError`. Por enquanto, isso é aceitável: dado errado não deve passar em silêncio. No módulo 06 você aprende a tratar esse erro.

> 🔧 **Modifique:** faça a função aceitar também `/` como separador

(`"2025/a/153"`). Uma linha a mais resolve.

🔮 **Preveja antes de rodar:**

```
print("a,,b".split(","))
print(len("OP 1 | OP 2 | OP 3".split("|")))
print("-".join("ABC"))
```

▼ **Resposta**

`['a', '', 'b']`, depois `3` e por fim `A-B-C`.

Duas vírgulas seguidas geram um pedaço vazio entre elas. E `join` aceita qualquer coisa que dê para percorrer com `for`, inclusive uma string: ela é percorrida letra por letra.

5. Formatação avançada com f-strings

No módulo 01 você viu `f"{valor:.2f}"`. O que vem depois dos dois-pontos é uma pequena linguagem de formatação. Ela controla largura, alinhamento, preenchimento, sinal, milhar e casas decimais.

```
f"{valor:*>12,.2f}"
         || | | |
         || | | └── .2f = número com 2 casas decimais
         || | └──── ,   = separador de milhar
         || └────── 12  = largura mínima (em caracteres)
         |└───────── >   = alinhar à direita  (< esquerda, ^ centro)
         └────────── *   = caractere de preenchimento (opcional)
```

```
>>> valor = 1250.5
>>> f"{valor:,.2f}"
'1,250.50'
>>> f"[{valor:12.1f}]"
'[      1250.5]'
>>> f"[{valor:*>12,.2f}]"
'[****1,250.50]'
>>> f"[{'OP':<6}]", f"[{'OP':>6}]", f"[{'OP':^6}]"
('[OP    ]', '[    OP]', '[  OP  ]')
>>> f"{42:06d}"           # inteiro com zeros à esquerda
'000042'
```

```
>>> f"{0.987:.1%}"        # porcentagem: multiplica por 100
'98.7%'
>>> f"{3.5:+.1f} / {-3.5:+.1f}"   # sinal sempre visível
'+3.5 / -3.5'
```

Números se alinham à direita por padrão; textos, à esquerda. Dois extras muito úteis para depurar:

```
>>> espessura = 25.4
>>> f"{espessura=}"       # mostra nome e valor
'espessura=25.4'
>>> cliente = "Plastifort "
>>> print(f"{cliente!r}")     # !r mostra as aspas: revela espaços escondidos
'Plastifort '
```

A largura também pode vir de uma variável. Coloque-a entre chaves dentro da especificação:

```
>>> largura = 10
>>> f"[{'COF':>{largura}}]"
'[       COF]'
>>> casas = 3
>>> f"{0.21:.{casas}f}"
'0.210'
```

Exemplo resolvido: tabela alinhada de medições

Problema: imprimir as medições de um lote em colunas, para colar no e-mail.

Primeiro eu decido as larguras: o nome mais longo ("COF estático") tem 12 letras, então uso 14 para sobrar espaço. Valores à direita, com 2 casas, em 8 caracteres. Depois eu escrevo o cabeçalho com **as mesmas larguras** das linhas, senão nada alinha.

```
medicoes = [
    ("Espessura", 25.4, "µm"),
    ("Gramatura", 23.1, "g/m²"),
    ("COF estático", 0.21, "-"),
]
print(f"{'Ensaio':<14}{'Valor':>8}  Unidade")
print("-" * 31)
for ensaio, valor, unidade in medicoes:
    print(f"{ensaio:<14}{valor:>8.2f}  {unidade}")
```

Saída:

```
Ensaio           Valor  Unidade
------------------------------
Espessura        25.40  µm
Gramatura        23.10  g/m²
COF estático      0.21  -
```

O exemplo completo está em `exemplos/02_formatacao.py`.

> 🔧 **Modifique:** acrescente uma coluna "Status" com `OK` centralizado em 8 caracteres (`:^8`). E troque o `"-" * 31` por um cálculo que use as larguras.

6. Números no formato brasileiro

O Python só entende ponto como separador decimal. O texto brasileiro usa ponto para milhar e vírgula para decimal.

```
>>> float("1250.5")
1250.5
>>> float("1.250,5")
Traceback (most recent call last):
  ...
ValueError: could not convert string to float: '1.250,5'
```

Pior que o erro é o caso que **não** dá erro:

```
>>> float("1.250")     # na expedição, isso é mil duzentos e cinquenta quilos!
1.25
```

Para o Python, `"1.250"` é um vírgula vinte e cinco. Uma carga de 1.250 kg viraria 1,25 kg sem nenhum aviso. Por isso, texto brasileiro **sempre** passa pela conversão.

A receita tem duas trocas, nesta ordem: primeiro some com os pontos de milhar, depois troque a vírgula por ponto.

```
>>> texto = " 1.250,5 "
>>> texto.strip().replace(".", "").replace(",", ".")
'1250.5'
>>> float(texto.strip().replace(".", "").replace(",", "."))
1250.5
```

🔮 **Preveja antes de rodar:** e se eu inverter a ordem das trocas?

```
texto = "1.250,5"
print(texto.replace(",", ".").replace(".", ""))
print(texto.replace(".", "").replace(",", "."))
```

▼ **Resposta**

`12505` e `1250.5`. Na primeira linha, a vírgula vira ponto e logo depois **todos** os pontos somem, inclusive o decimal que você acabou de criar. A ordem das operações importa.

O caminho de volta (número → texto brasileiro) usa a formatação americana e depois troca os símbolos. O truque é não trocar vírgula por ponto e ponto por vírgula "ao mesmo tempo" com dois `replace` (um desfaria o outro). Use um caractere provisório:

```
>>> americano = f"{1250.5:,.2f}"
>>> americano
'1,250.50'
>>> americano.replace(",", "_").replace(".", ",").replace("_", ".")
'1.250,50'
```

Também existe `str.maketrans` com `translate`, que troca vários caracteres de uma vez:

```
>>> trocas = str.maketrans(",.", ".,")    # ',' vira '.', e '.' vira ','
>>> "1,250.50".translate(trocas)
'1.250,50'
```

💡 **Dica:** o módulo `locale` também formata no padrão do país, mas depende da configuração do computador. Para dados e automação, prefira a conversão explícita.

⚠️ **Erro comum:** converter o peso para BR **antes** de fazer as contas. Regra: texto brasileiro entra, vira `float` logo na leitura, todas as contas são feitas em `float`, e só na hora de mostrar ele volta a ser texto brasileiro.

```
 "1.250,5"  ──br_para_float──▶  1250.5  ──contas──▶  2230.5
──float_para_br──▶ "2.230,50"
  (entrada)                      (Python trabalha aqui)
(saída para gente)
```

7. Unicode e acentos

Cada caractere tem um número único no padrão **Unicode** (o *code point*). `ord("ç")` dá `231`; `chr(231)` faz o caminho contrário e dá `'ç'`.

Mesma aparência, strings diferentes

Um "ã" pode ser guardado de dois jeitos: como **um** caractere pronto, ou como "a" + "til que se encaixa na letra anterior" (um caractere *combinante*). Com `print`, os dois aparecem como "ã". Para o Python, são diferentes.

```
>>> composto = "ã"
>>> decomposto = "a\u0303"     # 'a' + til combinante (U+0303)
>>> composto == decomposto
False
>>> len(composto), len(decomposto)
(1, 2)
>>> import unicodedata
>>> unicodedata.normalize("NFC", decomposto) == composto   #
normalizando, ficam iguais
True
```

Isso aparece de verdade: nomes de arquivo vindos de Mac, texto copiado de PDF ou de sites. A solução é **normalizar** as duas strings para a mesma forma antes de comparar.

```
NFC  = forma composta    "ã" é 1 caractere          (padrão do Windows e
da web)
NFD  = forma decomposta  "ã" é "a" + "◌̃"             (útil para remover
acentos)
NFKC/NFKD = como acima, e ainda simplifica "parecidos": "²" vira "2", "ﬁ"
vira "fi"
```

Exemplo resolvido: remover acentos

Problema: comparar "Tração" com "TRACAO" e achar que são iguais.

Primeiro eu pensei em fazer um `replace` para cada letra acentuada. Seriam dezenas de `replace`, e eu ia esquecer algum (e o "ü"? e o "ñ"?). Melhor usar a NFD: ela separa cada letra do seu acento. Aí basta jogar fora os caracteres combinantes. Vamos ver o que a NFD faz com "ção":

```
>>> for c in unicodedata.normalize("NFD", "ção"):
...     print(f"U+{ord(c):04X}", unicodedata.name(c),
unicodedata.combining(c))
...
U+0063 LATIN SMALL LETTER C 0
U+0327 COMBINING CEDILLA 202
U+0061 LATIN SMALL LETTER A 0
U+0303 COMBINING TILDE 230
U+006F LATIN SMALL LETTER O 0
```

`unicodedata.combining(c)` devolve 0 para letras normais e outro número para os acentos. Então a regra é: fique só com os caracteres em que `combining` dá 0.

```
>>> def remover_acentos(texto):
...     decomposto = unicodedata.normalize("NFD", texto)
...     return "".join([c for c in decomposto if not
unicodedata.combining(c)])
...
>>> remover_acentos("Tração na ruptura - São Paulo")
'Tracao na ruptura - Sao Paulo'
>>> remover_acentos("Tração").upper() == "TRACAO"
True
```

NFKD: quando "parecido" deve virar igual

Cabeçalho de planilha tem `g/m²`. Para criar um nome de coluna "limpo", você quer `g/m2`. A NFD não mexe no "²"; a NFKD troca pelo "2" comum.

```
>>> unicodedata.normalize("NFD", "g/m²")
'g/m²'
>>> unicodedata.normalize("NFKD", "g/m²")
'g/m2'
```

> ⚠️ **Erro comum:** existem **dois** "µ" no Unicode: o "sinal de micro" (U+00B5, o do teclado) e a letra grega "mi" (U+03BC). A NFKD troca o primeiro pelo segundo. Se o seu código procura `"µm"` depois de uma NFKD, não acha. Na dúvida, troque o `µ` por `u` **antes** de normalizar.

Para comparar ignorando maiúsculas, prefira `casefold()`: é um `lower()` feito para comparação (trata casos especiais como o "ß" alemão, que vira "ss").

```
>>> remover_acentos("JOÃO da Silva").casefold() == remover_acentos("joao
DA SILVA").casefold()
True
```

🔮 **Preveja antes de rodar:**

```
import unicodedata
x = "café"
y = unicodedata.normalize("NFD", x)
print(x == y, len(x), len(y))
print(unicodedata.normalize("NFC", y) == x)
```

▼ **Resposta**

`False 4 5` e depois `True`.

A forma NFD guarda o "é" como "e" + acento agudo combinante: 5 caracteres. Na tela as duas aparecem iguais. Normalizando de volta para NFC, ficam idênticas.

8. Encodings: o mistério do "ExpediÃ§Ã£o"

Até agora você trabalhou com `str`: texto "na cabeça" do Python. Mas no disco, no e-mail e na rede só existem **bytes** (números de 0 a 255). O **encoding** é a tabela que diz como cada caractere vira bytes.

Analogia: o texto é a mensagem; o encoding é o idioma em que ela foi escrita. Se quem escreveu usou um idioma e quem leu usou outro, sai bobagem.

```
>>> texto = "Expedição"
>>> texto.encode("utf-8")
b'Expedi\xc3\xa7\xc3\xa3o'
>>> texto.encode("cp1252")
b'Expedi\xe7\xe3o'
>>> len(texto), len(texto.encode("utf-8")), len(texto.encode("cp1252"))
(9, 11, 9)
```

`encode` vai de `str` para `bytes`; `decode` faz o contrário. O `b'...'` indica um objeto `bytes`, e `\xe7` é um byte escrito em hexadecimal. Os dois encodings do seu dia a dia:

```
caractere   UTF-8       cp1252 (Windows-1252, "ANSI" no Bloco de Notas)
A           41          41
ç           C3 A7       E7
ã           C3 A3       E3
µ           C2 B5       B5
€           E2 82 AC    80
```

UTF-8 é o padrão da web, do Python e dos arquivos modernos: qualquer caractere do mundo, de 1 a 4 bytes. **cp1252** é o padrão antigo do Windows em português (parente do `latin-1`): um byte por caractere. Sem acento, os dois coincidem. Por isso o problema só aparece no primeiro "ç". E aparece de dois jeitos.

Problema 1: bytes UTF-8 lidos como cp1252

```
>>> dados = "Expedição".encode("utf-8")
>>> dados.decode("cp1252")
'ExpediÃ§Ã£o'
```

Cada letra acentuada vira **duas** letras estranhas (`Ã§`, `Ã£`, `Ã©`). Isso se chama *mojibake*. Se o estrago já foi feito, desfaça o caminho:

```
>>> "ExpediÃ§Ã£o".encode("cp1252").decode("utf-8")
'Expedição'
```

Problema 2: bytes cp1252 lidos como UTF-8

```
>>> dados = "Expedição".encode("cp1252")
>>> dados.decode("utf-8")
Traceback (most recent call last):
  ...
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe7 in position 6:
invalid continuation byte
>>> dados.decode("utf-8", errors="replace")
'Expedi��o'
```

Leia a mensagem: o byte `0xe7` (o "ç" no cp1252) na posição 6 não é UTF-8 válido. Com `errors="replace"`, o Python troca o que não entende pelo losango `�` (U+FFFD) em vez de dar erro.

Onde isso morde no laboratório

- Excel, **"CSV (separado por vírgulas)"**, no Windows em português: grava em **cp1252**. Lido como UTF-8, dá o Problema 2.
- Excel, **"CSV UTF-8"**: grava em UTF-8 **com BOM** (veja abaixo). Lido como cp1252 por um programa antigo, dá o Problema 1.
- **Python no Windows**: sem `encoding=`, usa o padrão regional (normalmente cp1252). O Python 3.15 muda esse padrão para UTF-8 (PEP 686), mas você não controla em que versão seu código vai rodar. Regra de ouro: **sempre diga o encoding**.
- **Exportações do ERP e e-mails salvos como `.txt`**: podem vir em qualquer um dos dois.

> 💡 **Dica:** no VS Code, a barra inferior mostra o encoding do arquivo aberto ("UTF-8" ou "Windows 1252"). Clique nela e use "Reopen with Encoding" para testar. No Bloco de Notas, "Salvar como" mostra a codificação embaixo.

O BOM do "CSV UTF-8"

O Excel coloca 3 bytes invisíveis no começo do arquivo UTF-8 (`EF BB BF`), o **BOM**. Lido com `"utf-8"`, ele vira o caractere `\ufeff` grudado na primeira palavra. Lido com `"utf-8-sig"`, ele some.

```
>>> dados = "lote;espessura".encode("utf-8-sig")
>>> dados
b'\xef\xbb\xbflote;espessura'
>>> dados.decode("utf-8")
'\ufefflote;espessura'
>>> dados.decode("utf-8-sig")
'lote;espessura'
```

Com o BOM grudado, a primeira coluna se chama `'\ufefflote'`, e procurar por `'lote'` falha. Você vai ver muito isso no módulo 06, com CSV.

Exemplo resolvido: decodificar sem saber o encoding

Problema: os e-mails salvos pela expedição às vezes estão em UTF-8 e às vezes em cp1252.

Primeiro eu pensei: "UTF-8 é mais exigente". Bytes cp1252 com acento quase nunca formam UTF-8 válido. Então eu tento UTF-8 com `errors="replace"`. Se aparecer o losango, não era UTF-8, e eu decodifico como cp1252. Uso `"utf-8-sig"` para já

sumir com o BOM.

```
>>> def decodificar(dados):
...     texto = dados.decode("utf-8-sig", errors="replace")
...     if "\ufffd" in texto:    # apareceu o losango: não era UTF-8
...         texto = dados.decode("cp1252")
...     return texto
...
>>> decodificar("Expedição".encode("utf-8"))
'Expedição'
>>> decodificar("Expedição".encode("cp1252"))
'Expedição'
>>> decodificar("Expedição".encode("utf-8-sig"))
'Expedição'
```

No módulo 06 você vai ver um jeito mais direto com `try/except`.

Para ler um arquivo de verdade, uma prévia do módulo 06: `Path` (do módulo `pathlib`) representa um caminho de arquivo. `Path("email.txt").read_bytes()` devolve os bytes crus, e `Path("email.txt").read_text(encoding="utf-8")` já devolve a `str`. O arquivo `exemplos/05_encodings.py` usa os dois com os e-mails da pasta `dados/emails/`.

🔮 **Preveja antes de rodar:**

```
s = "µm"
print(len(s), len(s.encode("utf-8")), len(s.encode("cp1252")))
print(s.encode("utf-8").decode("cp1252"))
```

▼ **Resposta**

`2 3 2` e depois `Âµm`.

O "µ" ocupa 2 bytes em UTF-8 (`C2 B5`) e 1 em cp1252 (`B5`). Lendo os bytes UTF-8 como cp1252, o `C2` vira "Â" e o `B5` vira "µ". Por isso tanto relatório mostra "Âµm".

9. Expressões regulares: a ideia

Os métodos de string resolvem padrões **fixos**: "começa com OP", "tem um `|`". Mas o e-mail real traz `OP 123456`, `op-123456`, `OP:123456`. Você precisa descrever a **forma** do texto: "as letras OP, talvez um separador, e seis dígitos".

Uma **expressão regular** (regex) é uma minilinguagem para descrever formas de texto. O módulo `re` da biblioteca padrão faz a busca.

```
>>> import re
>>> re.search(r"\d+", "Peso: 980 kg")
<re.Match object; span=(6, 9), match='980'>
>>> print(re.search(r"\d+", "Sem números aqui"))
None
```

`re.search(padrão, texto)` procura a primeira ocorrência. Se acha, devolve um objeto `Match` (com o trecho encontrado e a posição). Se não acha, devolve `None`.

Por que o `r` antes das aspas?

Dentro de uma string normal, a barra invertida é especial: `"\n"` é **um** caractere (quebra de linha). Regex usa muitas barras (`\d`, `\s`, `\b`). O `r` cria uma *raw string*: a barra fica sendo só uma barra.

```
>>> len("\n"), len(r"\n")
(1, 2)
```

> ⚠️ **Erro comum:** esquecer o `r`. `"\d"` até funciona por sorte, mas o Python 3.12+ avisa `SyntaxWarning: invalid escape sequence '\d'`. E `"\b"` (que em regex significa "fronteira de palavra") vira o caractere de *backspace* e quebra o padrão sem avisar. **Regex sempre com `r"..."`.**

As peças básicas

```
Peça        Significa                                    Exemplo
Casa com
---------   ------------------------------------------   ------------
-----------
abc         o texto literal "abc"                        OP
OP
.           qualquer caractere (menos quebra de linha)     O.
OP, O-, O1
\d          um dígito (0 a 9)                            \d\d
45
\s          um espaço em branco (espaço, tab, \n)        OP\s1
OP 1
\w          letra, dígito ou _ (inclui letras com acento) \w+
Tração
[abc]       um caractere da lista                        [-:]
- ou :
```

```
[^abc]      um caractere FORA da lista                    [^|]+
tudo até o |
[0-9a-f]    faixa de caracteres                           [A-D]
A, B, C, D
?           o anterior 0 ou 1 vez (opcional)              kg?
k, kg
*           o anterior 0 ou mais vezes                    \s*
"", " ", "   "
+           o anterior 1 ou mais vezes                    \d+
7, 980, 1250
{n}         exatamente n vezes                            \d{6}
123456
{n,m}       de n a m vezes                                \d{1,3}
7, 98, 980
^   $       começo / fim do texto                         ^OP
OP no início
\b          fronteira de palavra                          \bOP\b
OP (não TOP)
a|b         a OU b                                        kg|t
kg, t
( )         grupo: junta peças e captura o trecho         (\d+)
(ver seção 12)
\. \| \(    o caractere literal (escapado)                \d+\.\d+
25.4
```

Não precisa decorar: volte nesta tabela sempre que precisar.

10. Construindo um padrão aos poucos

Ninguém escreve uma regex grande de uma vez. O método é:

1. Junte **exemplos reais**: os que devem casar e os que **não** devem.
2. Comece com o padrão mais simples possível.
3. Teste. Veja o que falhou. Ajuste **uma** peça. Teste de novo.

Para testar rápido, crie uma função de apoio no REPL. `re.fullmatch` responde: "o texto **inteiro** tem essa forma?".

```
>>> def testar(padrao, exemplos, flags=0):
...     for texto in exemplos:
...         casou = re.fullmatch(padrao, texto, flags)
...         print("✔" if casou else "✘", repr(texto))
...
>>> exemplos = ["OP 123456", "OP-123456", "OP123456", "op 123456",
...             "OP 12345", "OP 1234567", "OS 123456"]
```

Os quatro primeiros devem casar. Os três últimos, não (5 dígitos, 7 dígitos, "OS").

Passo 1. O mais simples: "OP", espaço, dígitos.

```
>>> testar(r"OP \d+", exemplos)
✔ 'OP 123456'
✘ 'OP-123456'
✘ 'OP123456'
✘ 'op 123456'
✔ 'OP 12345'
✔ 'OP 1234567'
✘ 'OS 123456'
```

Dois problemas: casou números com 5 e 7 dígitos, e não aceitou hífen nem "op".

Passo 2. Exatamente 6 dígitos: `\d{6}`.

```
>>> testar(r"OP \d{6}", exemplos)
✔ 'OP 123456'
✘ 'OP-123456'
✘ 'OP123456'
✘ 'op 123456'
✘ 'OP 12345'
✘ 'OP 1234567'
✘ 'OS 123456'
```

Os negativos agora estão certos. Faltam as variações.

Passo 3. O separador é "espaço ou hífen, ou nada": `[ -]?`.

```
>>> testar(r"OP[ -]?\d{6}", exemplos)
✔ 'OP 123456'
✔ 'OP-123456'
✔ 'OP123456'
✘ 'op 123456'
✘ 'OP 12345'
✘ 'OP 1234567'
✘ 'OS 123456'
```

Passo 4. Ignorar maiúsculas e minúsculas: a flag `re.IGNORECASE`.

```
>>> testar(r"OP[ -]?\d{6}", exemplos, re.IGNORECASE)
✔ 'OP 123456'
✔ 'OP-123456'
✔ 'OP123456'
✔ 'op 123456'
✘ 'OP 12345'
```

```
✘ 'OP 1234567'
✘ 'OS 123456'
```

Pronto: todos os ✔ e ✘ estão onde deveriam. Repare no processo: a cada passo, **um** ajuste, e a lista inteira de exemplos é testada de novo. Assim você nunca conserta um caso e quebra outro sem perceber.

> 💡 **Dica:** o site regex101.com (escolha "Python" no menu *Flavor*) mostra, peça por peça, o que o seu padrão está fazendo. Ótimo para depurar. Só não cole lá dados sigilosos da empresa.

> 🔧 **Modifique:** acrescente `"OP: 123456"` e `"OP - 123456"` aos exemplos e ajuste o padrão para aceitar dois-pontos e espaços em volta do hífen. Dica: `[\s:-]*`.

11. search, match, fullmatch, findall e finditer

Todas recebem `(padrão, texto)`. A diferença é **onde** procuram e **o que** devolvem.

```
Função         Procura...                          Devolve
------------   ---------------------------------   -----------------------
------
re.search      em qualquer lugar (1ª ocorrência)   Match ou None
re.match       só no começo do texto               Match ou None
re.fullmatch   o texto inteiro                     Match ou None
re.findall     todas as ocorrências                lista de str (ou de
tuplas)
re.finditer    todas as ocorrências                um Match de cada vez
(for)
```

```
>>> texto = "Liberar OP 123456 hoje"
>>> re.search(r"OP \d{6}", texto)
<re.Match object; span=(8, 17), match='OP 123456'>
>>> print(re.match(r"OP \d{6}", texto))
None
>>> print(re.fullmatch(r"OP \d{6}", texto))
None
>>> re.findall(r"\d{6}", "OPs 123456, 123457 e 123458")
['123456', '123457', '123458']
```

O objeto `Match` guarda o que foi encontrado:

```
>>> achado = re.search(r"\d+", "Peso: 980 kg")
>>> achado.group()
'980'
>>> achado.start(), achado.end()
(6, 9)
```

`finditer` é o `findall` que entrega um `Match` de cada vez, num `for`. Use quando precisa de mais do que o texto (posição, grupos). Você vai usá-lo na seção 15.

> ⚠️ **Erro comum:** encadear `.group()` direto no `search`. Se não achar, `search` devolve `None`, e `None.group()` quebra com `AttributeError: 'NoneType' object has no attribute 'group'`. Guarde numa variável e teste: `if achado:`.

> 🔮 **Preveja antes de rodar:**
>
> ```
> import re
> texto = "OP 123456, OP 7654321, op 111111"
> print(re.findall(r"OP \d{6}", texto))
> ```

▼ **Resposta**

`['OP 123456', 'OP 765432']`.

O "OP 7654321" tem 7 dígitos, mas `\d{6}` casa com os 6 primeiros e pronto: a regex não sabe que tinha mais um dígito depois. E "op 111111" ficou de fora porque não usamos `re.IGNORECASE`. A correção é exigir fronteira de palavra: `r"\bOP \d{6}\b"`. O `\b` depois dos dígitos garante que não vem outro dígito colado.

12. Grupos e grupos nomeados

Achar o trecho é metade do trabalho. Quase sempre você quer só **uma parte** dele: o número da OP, sem o "OP". Para isso servem os parênteses: eles **capturam** um pedaço.

```
>>> achado = re.search(r"OP[ -]?(\d{6})", "Liberar OP-004512 hoje")
>>> achado.group(0)     # o trecho inteiro
'OP-004512'
```

```
>>> achado.group(1)     # só o que estava no 1º par de parênteses
'004512'
```

Com vários grupos, `groups()` devolve todos numa tupla:

```
>>> linha = "Espessura: 25,4 µm"
>>> achado = re.fullmatch(r"(\w+): ([\d.,]+) (\S+)", linha)
>>> achado.groups()
('Espessura', '25,4', 'µm')
```

Contar parênteses fica confuso rápido. Dê **nomes** aos grupos com `(?P<nome>...)`:

```
>>> padrao = r"(?P<ensaio>\w+): (?P<valor>[\d.,]+) (?P<unidade>\S+)"
>>> achado = re.fullmatch(padrao, linha)
>>> achado["valor"]
'25,4'
>>> achado.groupdict()
{'ensaio': 'Espessura', 'valor': '25,4', 'unidade': 'µm'}
```

`groupdict()` já devolve um dicionário, pronto para a sua "tabela" (lista de dicts).

Grupos no findall

Quando o padrão tem **um** grupo, `findall` devolve só o conteúdo do grupo. Com **vários**, devolve tuplas:

```
>>> re.findall(r"OP (\d{6})", "OP 123456 e OP 654321")
['123456', '654321']
>>> re.findall(r"(OP|NF) (\d+)", "OP 123456 NF 98765")
[('OP', '123456'), ('NF', '98765')]
```

Às vezes você precisa de parênteses só para agrupar (por exemplo, numa alternativa), sem capturar. Use `(?:...)`:

```
>>> re.findall(r"(?:OP|NF) (\d+)", "OP 123456 NF 98765")
['123456', '98765']
```

Guloso e preguiçoso

Os quantificadores `*` e `+` são **gulosos**: pegam o máximo que conseguirem. Colocando `?` depois (`*?`, `+?`), viram **preguiçosos**: pegam o mínimo.

> 🔮 **Preveja antes de rodar:**

```
import re
obs = 'Obs: "COF liberado" e "Selagem ok"'
print(re.findall(r'"(.*)"', obs))
print(re.findall(r'"(.*?)"', obs))
```

▼ Resposta

`['COF liberado" e "Selagem ok']` e depois `['COF liberado', 'Selagem ok']`.

O `.*` guloso vai da primeira aspa até a **última** aspa do texto. O `.*?` preguiçoso para na primeira aspa que fecha. Uma alternativa sem `?` é dizer "qualquer coisa que não seja aspa": `r'"([^"]*)"'`.

🔧 **Modifique:** o padrão com `\w+` para o ensaio falha em `"COF estático: 0,21"` (tem espaço no nome) e em `"Tração: 32 MPa"`? Teste os dois. Depois troque `\w+` por `[^:]+` ("tudo que não for dois-pontos") e veja o que muda.

13. Substituindo e dividindo: re.sub e re.split

`re.sub(padrão, troca, texto)` substitui **todas** as ocorrências.

```
>>> re.sub(r"\s+", " ", "Embalagens    do   Sul\t Ltda")
'Embalagens do Sul Ltda'
>>> re.sub(r"\d", "#", "CPF 123.456.789-00")
'CPF ###.###.###-##'
```

Na troca, `\1`, `\2`... reaproveitam os grupos capturados. Com grupos nomeados, use `\g<nome>`. Exemplo clássico: data brasileira para o formato ISO (ano-mês-dia), que ordena certo no Excel e em qualquer sistema.

```
>>> re.sub(r"(\d{2})/(\d{2})/(\d{4})", r"\3-\2-\1", "Produzido em
05/03/2025")
'Produzido em 2025-03-05'
>>> re.sub(r"(?P<d>\d{2})/(?P<m>\d{2})/(?P<a>\d{4})", r"\g<a>-\g<m>-
\g<d>", "05/03/2025")
'2025-03-05'
```

E quando a troca depende de uma conta? Passe uma **função** no lugar do texto de troca. O `re.sub` chama a função para cada ocorrência, entregando o `Match`, e usa o que ela devolver (funções como valores: módulo 04).

```
>>> def br_para_ponto(achado):
...     return achado.group().replace(",", ".")
...
>>> re.sub(r"\d+,\d+", br_para_ponto, "Espessura 25,4 e gramatura 23,1")
'Espessura 25.4 e gramatura 23.1'
>>> re.sub(r"\d+", lambda m: str(int(m.group()) * 2), "3 bobinas e 12
caixas")
'6 bobinas e 24 caixas'
```

`re.split` quebra o texto onde o padrão casar. Bom quando há **vários** separadores possíveis:

```
>>> re.split(r"\s*[|;]\s*", "OP: 123456 | NF: 98765; Peso: 980 kg")
['OP: 123456', 'NF: 98765', 'Peso: 980 kg']
```

▼ Tente você: troque cada "kg" (com ou sem espaço antes) por " quilos". Ex.: "980kg e 1.250 kg" → "980 quilos e 1.250 quilos".

```
>>> re.sub(r"\s*kg\b", " quilos", "980kg e 1.250 kg")
'980 quilos e 1.250 quilos'
```

O `\s*` engole os espaços que existirem antes (zero ou mais), e a troca sempre coloca exatamente um.

14. Flags e re.compile

Flags mudam o comportamento do padrão. As três que você mais vai usar:

- `re.IGNORECASE` (ou `re.I`): maiúsculas e minúsculas tanto faz.
- `re.MULTILINE` (ou `re.M`): `^` e `$` passam a valer para **cada linha**, e não só para o começo e o fim do texto todo.
- `re.VERBOSE` (ou `re.X`): espaços e quebras de linha no padrão são ignorados, e `#` começa um comentário. Permite escrever a regex "em pé", explicada.

```
>>> re.findall(r"op \d{6}", "OP 123456 e op 654321", flags=re.IGNORECASE)
['OP 123456', 'op 654321']
>>> email = "Assunto: carga\nOP 123456\nobs: a OP 999999 saiu\nOP 654321"
>>> re.findall(r"^OP (\d{6})", email)
[]
>>> re.findall(r"^OP (\d{6})", email, flags=re.MULTILINE)
['123456', '654321']
```

Sem `MULTILINE`, `^` só casa no começo do texto inteiro (que começa com "Assunto"). Com ela, casa no começo de cada linha. A OP 999999 não está no começo da linha.

Para combinar flags, use `|`: `re.IGNORECASE | re.MULTILINE`.

Quando um padrão é usado muitas vezes, **compile** uma vez e guarde numa constante. Junto com `VERBOSE`, fica legível como código comum:

```
>>> PADRAO_OP = re.compile(
...     r"""
...     \b OP        # a sigla, como palavra inteira
...     [\s:-]*      # separador opcional: espaços, dois-pontos ou hífen
...     (\d{6})      # o número: exatamente 6 dígitos (grupo 1)
...     \b           # nada de dígito colado depois
...     """,
...     re.VERBOSE | re.IGNORECASE,
... )
>>> PADRAO_OP.findall("Liberar OP: 123456, op-654321 e a TOP 111111")
['123456', '654321']
```

> ⚠️ **Erro comum:** no modo `VERBOSE`, o espaço literal do padrão é **ignorado**. Para casar um espaço de verdade, use `\s`, `[ ]` ou `\ `.

15. Projeto guiado: lendo o e-mail da expedição

Hora de juntar tudo. Este é o e-mail padrão que a expedição manda ao laboratório:

```
De: expedicao@plastifort.com.br
Assunto: Certificados - carga de 28/10

Bom dia, laboratório!

Favor emitir os certificados dos pedidos abaixo:

OP: 123456 | NF: 98765 | Peso: 1.250,5 kg
OP: 123457 | NF: 98766 | Peso: 980,0 kg
OP: 123458 | NF: 98767 | Peso: 2.040,75 kg

A OP 123459 vai na próxima carga.

Obrigado,
Expedição
```

Objetivo: uma lista de dicionários, um por pedido, com a OP, a NF e o peso em `float`.

Passo 1: decompor o problema

```
e-mail (str)
  └─► achar as linhas de pedido          (ignorar saudação, assinatura,
"A OP 123459...")
        └─► em cada uma, extrair OP, NF, peso   (regex com grupos
nomeados)
              └─► converter o peso "1.250,5" → 1250.5   (seção 6)
                    └─► juntar num dicionário e acumular numa lista
```

Repare: a linha "A OP 123459 vai na próxima carga" **tem** uma OP, mas não é um pedido. Por isso o padrão não pode ser só "ache OPs": tem que descrever a **linha inteira** do pedido.

Passo 2: uma peça de cada vez, numa linha só

```
>>> linha = "OP: 123456 | NF: 98765 | Peso: 1.250,5 kg"
>>> re.search(r"OP:\s*(\d{6})", linha).group(1)
'123456'
>>> re.search(r"NF:\s*(\d+)", linha).group(1)
'98765'
>>> re.search(r"Peso:\s*([\d.]+(?:,\d+)?)\s*kg", linha).group(1)
'1.250,5'
```

O peso é a peça mais difícil. Leia assim: `[\d.]+` são dígitos e pontos de milhar; `(?:,\d+)?` é "uma vírgula com dígitos", opcional (o peso pode vir sem decimais, como `980 kg`). Aqui eu uso `.group(1)` direto porque **acabei** de ver que a linha casa; no código de verdade, teste o `None` antes.

Passo 3: juntar as peças num padrão só

As peças já funcionam. Agora eu junto, com `VERBOSE` para comentar cada parte e grupos nomeados para não contar parênteses.

```
>>> PEDIDO = re.compile(
...     r"""
...     OP:   \s* (?P<op>\d{6})              # OP com 6 dígitos
...     \s* \| \s*                           # separador | (escapado!)
...     NF:   \s* (?P<nf>\d+)                # NF: só dígitos
...     \s* \| \s*
...     Peso: \s* (?P<peso>[\d.]+(?:,\d+)?)  # peso no formato brasileiro
```

```
...     \s* kg
...     """,
...     re.VERBOSE | re.IGNORECASE,
... )
>>> PEDIDO.search(linha).groupdict()
{'op': '123456', 'nf': '98765', 'peso': '1.250,5'}
```

⚠️ **Erro comum:** esquecer de escapar o `|`. Sem a barra, `|` significa "OU" e o padrão vira "OP... OU NF... OU Peso...". Ele passa a casar com pedaços soltos e os grupos vêm `None`. Todo caractere especial que você quer literal (`| . ( ) [ ] ? * +`) precisa de `\` na frente.

Passo 4: o e-mail inteiro

`finditer` percorre todas as ocorrências do padrão no texto. Para cada uma, eu converto o peso e monto o dicionário.

```
>>> def ler_pedidos(email):
...     pedidos = []
...     for achado in PEDIDO.finditer(email):
...         peso = float(achado["peso"].replace(".", "").replace(",",
"."))
...         pedidos.append({"op": achado["op"], "nf": achado["nf"],
"peso_kg": peso})
...     return pedidos
...
>>> email = """Bom dia, laboratório!
... OP: 123456 | NF: 98765 | Peso: 1.250,5 kg
... OP: 123457 | NF: 98766 | Peso: 980,0 kg
... OP: 123458 | NF: 98767 | Peso: 2.040,75 kg
... A OP 123459 vai na próxima carga."""
>>> for pedido in ler_pedidos(email):
...     print(pedido)
...
{'op': '123456', 'nf': '98765', 'peso_kg': 1250.5}
{'op': '123457', 'nf': '98766', 'peso_kg': 980.0}
{'op': '123458', 'nf': '98767', 'peso_kg': 2040.75}
>>> sum(p["peso_kg"] for p in ler_pedidos(email))
4271.25
```

A linha "A OP 123459..." ficou de fora, como queríamos. O programa completo, com o e-mail inteiro e um resumo formatado, está em `exemplos/08_email_expedicao.py`.

Passo 5: quando não casa, como depurar

Na semana seguinte chega esta linha, e `ler_pedidos` devolve lista vazia para ela:

```
OP: 123460 | NF: 98.768 | Peso: 500 kg
```

Não fique olhando para o padrão inteiro. **Corte ao meio** e teste os pedaços, do começo para o fim, até achar o primeiro que falha:

```
>>> problema = "OP: 123460 | NF: 98.768 | Peso: 500 kg"
>>> re.search(r"OP:\s*(\d{6})", problema) is not None
True
>>> re.search(r"OP:\s*(\d{6})\s*\|\s*NF:\s*(\d+)\s*\|", problema) is not
None
False
>>> re.search(r"NF:\s*(\d+)", problema).group(1)
'98'
```

Achado: a NF veio com ponto de milhar (`98.768`), e `\d+` para no ponto. Agora é só decidir a regra (aceitar `[\d.]+` e tirar os pontos depois) e testar de novo **todos** os exemplos antigos. É exatamente isso que o exercício 12 pede, com mais variações.

Lendo mensagens de erro deste módulo

- **`AttributeError: 'NoneType' object has no attribute 'group'`**: o `search` (ou `match`, `fullmatch`) não achou nada e devolveu `None`. Teste antes (`if achado:`) e depois descubra **por que** não casou (seção 15, passo 5).
- **`ValueError: could not convert string to float: '1.250,5'`**: texto brasileiro entrou direto no `float`. Converta antes (seção 6). Se aparece `'25,4 kg'` ou `''`, sobrou unidade ou a célula estava vazia.
- **`TypeError: 'str' object does not support item assignment`**: tentou mudar uma letra (`texto[0] = "X"`). Monte uma string nova: `"X" + texto[1:]`.
- **`TypeError: sequence item 1: expected str instance, int found`**: o `join` recebeu algo que não é string na posição 1. Converta com `str()` antes.
- **`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe7 in position 6`**: bytes cp1252 lidos como UTF-8. O `0xe7` é o "ç" do cp1252 (`0xe3` é "ã", `0xe9` é "é"). Use `encoding="cp1252"` ou a estratégia da seção 8.
- **Texto com `Ã§`, `Ã£`, `Âµ`, sem erro nenhum**: UTF-8 lido como cp1252. Leia com `encoding="utf-8"`; se já veio estragado, `texto.encode("cp1252").decode("utf-8")`.
- **`SyntaxWarning: invalid escape sequence '\d'`**: faltou o `r` antes das aspas da regex. Escreva `r"\d+"`.

- **`re.error: missing ), unterminated subpattern at position 0`**: abriu parêntese no padrão e não fechou. Se queria um `(` literal, escape: `\(`. Parecidas: `unterminated character set` (`[` sem fechar) e `nothing to repeat` (um `*`, `+` ou `?` sem nada antes, como em `r"+55"`).

```
>>> re.search(r"(OP \d+", "OP 123")
Traceback (most recent call last):
  ...
re.error: missing ), unterminated subpattern at position 0
```

No Python 3.13+, o nome aparece como `re.PatternError`. É o mesmo erro.

Boas práticas

- **Métodos de string primeiro, regex depois.** Se `startswith`, `split` ou `in` resolvem, use. Regex é para quando a forma varia.
- **Regex sempre com `r"..."`.** Sem exceção.
- **Ancore quando validar.** Para "este texto É um código de OP", use `fullmatch` (ou `^...$`). Com `search`, "XYZOP 1234567" passaria.
- **Use `\b` em volta de números e siglas** para não casar pedaço de algo maior.
- **Padrões grandes: `re.compile` + `re.VERBOSE` + grupos nomeados**, numa constante com nome claro (`PADRAO_PEDIDO`). Seu "eu" de daqui a seis meses agradece.
- **Guarde exemplos reais** (positivos e negativos) e teste o padrão contra todos a cada mudança. No módulo 09 isso vira teste automatizado com `pytest.mark.parametrize`.
- **Converta na entrada, formate na saída.** Texto brasileiro vira `float` na leitura; volta a ser texto só na hora de mostrar.
- **Sempre diga o encoding** ao ler e gravar. Na dúvida sobre arquivos que vêm do Excel, tente `utf-8-sig` e caia para `cp1252`.
- **Normalize antes de comparar**: `strip`, `casefold`, remover acentos, espaços repetidos. Compare a versão normalizada, mas guarde a original para mostrar.

No mundo real

Na planilha de resultados, a coluna "Observação" diz quais ensaios foram liberados. Cada analista escreve de um jeito: `COF: liberado`, `cof liberado`, `Selagem - PENDENTE`, `Tração: Liberada; COF: reprovado`. O robô dos certificados precisa transformar isso num dicionário confiável e só emitir quando tudo estiver liberado.

O raciocínio:

1. Uma observação pode ter vários ensaios: quebro em `;` ou `,` (`re.split`).
2. Em cada pedaço, o formato é "ensaio, separador opcional (`:` ou `-`), status".
3. Normalizo tudo (sem acento, minúsculas) para comparar sem surpresas.
4. "liberado" e "liberada" contam igual: comparo só o começo da palavra (`liberad`).

```
import re
import unicodedata

PADRAO_ITEM = re.compile(
    r"""
    (?P<ensaio>[a-z ]+?)   # nome do ensaio (já sem acento e minúsculo)
    \s* [:-]? \s*          # separador opcional
    (?P<status>[a-z]+)     # a palavra do status
    $
    """,
    re.VERBOSE,
)


def normalizar(texto):
    decomposto = unicodedata.normalize("NFD", texto)
    sem_acento = "".join([c for c in decomposto if not
unicodedata.combining(c)])
    return " ".join(sem_acento.casefold().split())


def ler_liberacoes(observacao):
    resultado = {}
    for pedaco in re.split(r"[;,]", observacao):
        pedaco = normalizar(pedaco)
        achado = PADRAO_ITEM.match(pedaco)
        if not achado:
            continue
        status = "liberado" if achado["status"].startswith("liberad")
else achado["status"]
        resultado[achado["ensaio"].strip()] = status
    return resultado


observacoes = [
    "COF: liberado",
    "cof LIBERADO",
    "Tração: Liberada; COF: reprovado",
    "Selagem - PENDENTE, cof: liberado",
]
for obs in observacoes:
    liberacoes = ler_liberacoes(obs)
    pode_emitir = all(s == "liberado" for s in liberacoes.values())
```

```
    print(f"{obs:<36} -> {liberacoes} | emitir? {'sim' if pode_emitir
else 'não'}")
```

Saída:

```
COF: liberado                        -> {'cof': 'liberado'} | emitir? sim
cof LIBERADO                         -> {'cof': 'liberado'} | emitir? sim
Tração: Liberada; COF: reprovado     -> {'tracao': 'liberado', 'cof':
'reprovado'} | emitir? não
Selagem - PENDENTE, cof: liberado    -> {'selagem': 'pendente', 'cof':
'liberado'} | emitir? não
```

Três escolhas de profissional: o padrão roda **depois** de normalizar (por isso basta `[a-z ]`); pedaços que não casam são ignorados em vez de quebrar o programa; e "liberada" vira "liberado". O arquivo está em `exemplos/09_liberacoes.py`.

Resumo

Tarefa	Como
Fatiar	`s[ini:fim]`, `s[-4:]`, `s[::-1]`
Procurar	`"x" in s`, `s.find("x")` (-1), `s.count("x")`, `s.startswith(("a", "b"))`
Limpar	`strip()`, `strip("*")`, `removeprefix`, `removesuffix`, `zfill(9)`
Caixa	`upper`, `lower`, `title`, `capitalize`, `casefold` (para comparar)
Quebrar e juntar	`split()`, `split(";", maxsplit=1)`, `partition(":")`, `splitlines()`, `"-".join(lista)`
Espaços repetidos	`" ".join(s.split())`
Formatar	`f"{v:>10,.2f}"`, `f"{t:<12}"`, `f"{n:06d}"`, `f"{p:.1%}"`, `f"{v=}"`, `f"{t!r}"`
BR → float	`float(s.strip().replace(".", "").replace(",", "."))`
float → BR	`f"{v:,.2f}".translate(str.maketrans(",.", ".,"))`
Remover acento	`normalize("NFD", s)` + descartar `unicodedata.combining(c)`
"²" → "2"	`unicodedata.normalize("NFKD", s)`
Encoding	`s.encode("utf-8")`, `b.decode("cp1252")`, `"utf-8-sig"` tira o BOM
Consertar "Ã§"	`s.encode("cp1252").decode("utf-8")`
Regex: achar 1	`re.search(p, s)` → `Match` ou `None`

Tarefa	Como
Regex: validar	`re.fullmatch(p, s)`
Regex: achar todos	`re.findall(p, s)` (lista), `re.finditer(p, s)` (Matches)
Grupos	`(...)`, `(?P<nome>...)`, `(?:...)`; `m.group(1)`, `m["nome"]`, `m.groupdict()`
Trocar	`re.sub(p, r"\2-\1", s)`, `re.sub(p, funcao, s)`
Dividir	`re.split(r"\s*[
Flags	`re.IGNORECASE`, `re.MULTILINE`, `re.VERBOSE`, combine com `|`
Reusar	`PADRAO = re.compile(r"...", flags)`

Você domina este módulo se...

- [ ] Explico sem consultar a diferença entre `strip(".csv")` e `removesuffix(".csv")`.
- [ ] Converto `"1.250,5"` em `1250.5` e `1250.5` em `"1.250,50"`, e sei por que `float("1.250")` é perigoso.
- [ ] Monto uma tabela alinhada com f-strings sem tentativa e erro.
- [ ] Removo acentos com `unicodedata` e explico o que é NFD.
- [ ] Olho para `ExpediÃ§Ã£o` ou para um `UnicodeDecodeError` e digo qual encoding foi usado errado e como consertar.
- [ ] Explico a diferença entre `search`, `match`, `fullmatch` e `findall`.
- [ ] Construo uma regex aos poucos, testando contra exemplos que devem e que não devem casar.
- [ ] Extraio OP, NF e peso de um e-mail com grupos nomeados e converto o peso.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_limpar_nome`	★☆☆	`split`, `join`, `capitalize`, regras com `if`
02	`ex02_formatar_nf`	★☆☆	filtrar dígitos, `zfill`, fatiamento
03	`ex03_separar_campo`	★☆☆	`partition`, `strip`, tuplas
04	`ex04_br_para_float`	★☆☆	número brasileiro → `float`
05	`ex05_validar_op`	★☆☆	primeira regex: `fullmatch` e `IGNORECASE`
06	`ex06_float_para_br`	★★☆	formatação dinâmica, troca de símbolos

Nº	Exercício	Nível	O que pratica
07	`ex07_normalizar_chave`	★★☆	`unicodedata` (NFKD), `re.sub`
08	`ex08_extrair_ops`	★★☆	`findall` com grupo, `\b`, sem repetir
09	`ex09_ler_medicao`	★★☆	grupos nomeados, `fullmatch`, `None`
10	`ex10_padronizar_datas`	★★☆	`re.sub` com função
11	`ex11_depurar_peso`	★★☆	leitura de código: prever e consertar
12	`ex12_ler_email_expedicao`	★★★	o parser da expedição, com variações reais
13	`ex13_revisao_tolerancias`	★★★	revisão: regex + dicionários + decisões (módulos 02 a 04)
14	`ex14_revisao_resumo`	★★★	revisão: formatação + `sorted` + acumuladores (módulos 01 a 03)

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError` e rode, na pasta `curso-python`:

```
python curso.py testar 05 ex01
```

Para testar o módulo inteiro: `python curso.py testar 05`.

Desafio do módulo

Leitor de e-mails da expedição. Um programa que lê todos os e-mails salvos em `dados/emails/` (alguns em UTF-8, outros em cp1252), extrai os pedidos, aponta as linhas suspeitas, padroniza o nome do cliente e imprime um resumo com o peso total no formato brasileiro. Enunciado completo em desafio/README.md.

Quiz

Teste o que ficou: quiz.md.

Para ir além

- Métodos de string: https://docs.python.org/pt-br/3/library/stdtypes.html#string-methods
- Formatação: https://docs.python.org/pt-br/3/library/string.html#formatspec
- HOWTO de expressões regulares (excelente): https://docs.python.org/pt-br/3/howto/regex.html

- Módulo `re`: https://docs.python.org/pt-br/3/library/re.html
- HOWTO de Unicode: https://docs.python.org/pt-br/3/howto/unicode.html
- Encodings disponíveis: https://docs.python.org/pt-br/3/library/codecs.html#standard-encodings
- Leitura extra: capítulo 7 (regex) do livro gratuito *Automate the Boring Stuff with Python*: https://automatetheboringstuff.com/2e/chapter7/
- Leitura extra: Joel Spolsky, "The Absolute Minimum Every Software Developer Must Know About Unicode": https://www.joelonsoftware.com/2003/10/08/the-absolute-minimum-every-software-developer-absolutely-positively-must-know-about-unicode-and-character-sets-no-excuses/
- Praticar regex com explicação visual: https://regex101.com (*Flavor*: Python).

Enunciados dos exercícios

Exercício 05.01 — Padronizar nome de cliente

Arquivo: `exercicios/ex01_limpar_nome.py`

```
Nível: ★☆☆

Contexto: O nome do cliente chega do ERP e dos e-mails de muitos jeitos:
"  embalagens   DO sul  ltda ", "EMBALAGENS DO SUL LTDA". O certificado
de
qualidade precisa mostrar sempre o mesmo formato.

Tarefa: implemente `limpar_nome(nome)`, que devolve o nome padronizado:
- sem espaços nas pontas e com UM espaço entre as palavras (tabs e
quebras de
  linha também contam como espaço);
- cada palavra com a primeira letra maiúscula e o resto minúsculo;
- exceção: as palavras "da", "das", "de", "do", "dos" e "e" ficam
minúsculas,
  a não ser que sejam a PRIMEIRA palavra do nome;
- nome vazio (ou só com espaços) devolve "".

Exemplos:
    >>> limpar_nome("  embalagens   DO sul  ltda ")
    'Embalagens do Sul Ltda'
    >>> limpar_nome("JOÃO DA SILVA E FILHOS")
    'João da Silva e Filhos'
    >>> limpar_nome("de souza plásticos")
    'De Souza Plásticos'

Dica: `split()` sem argumento já resolve os espaços repetidos. Percorra
as
```

```
palavras com `enumerate` para saber qual é a primeira.

Teste com:  python curso.py testar 05 ex01
```

Exercício 05.02 — Formatar número de NF

Arquivo: `exercicios/ex02_formatar_nf.py`

```
Nível: ★☆☆

Contexto: O número da nota fiscal eletrônica tem até 9 dígitos. No e-mail
ele
aparece de todo jeito ("12345", "NF-e 98.765"), mas no certificado
precisa
aparecer sempre igual: 9 dígitos em grupos de 3, como "000.012.345".

Tarefa: implemente `formatar_nf(numero)`:
- fique só com os dígitos do texto (ignore letras, espaços, pontos,
hífens);
- complete com zeros à esquerda até ter 9 dígitos;
- devolva no formato "DDD.DDD.DDD";
- se o texto não tiver nenhum dígito, devolva "".
Você pode supor que nunca haverá mais de 9 dígitos.

Exemplos:
    >>> formatar_nf("12345")
    '000.012.345'
    >>> formatar_nf("NF-e 98.765")
    '000.098.765'
    >>> formatar_nf("123456789")
    '123.456.789'
    >>> formatar_nf("sem número")
    ''

Dica: `"".join([c for c in texto if c.isdigit()])` fica só com os
dígitos.
Depois, `zfill` e fatiamento.

Teste com:  python curso.py testar 05 ex02
```

Exercício 05.03 — Separar rótulo e valor

Arquivo: `exercicios/ex03_separar_campo.py`

```
Nível: ★☆☆

Contexto: Os laudos exportados em texto têm uma informação por linha, no
```

```
formato
"rótulo: valor". O valor às vezes tem ":" dentro (um horário, por
exemplo).

Tarefa: implemente `separar_campo(linha)`, que devolve uma tupla
(rótulo, valor):
- divida no PRIMEIRO ":" da linha;
- tire os espaços das pontas do rótulo e do valor;
- se a linha não tiver ":", devolva (linha sem espaços nas pontas, "").

Exemplos:
    >>> separar_campo("Espessura (µm): 25,0")
    ('Espessura (µm)', '25,0')
    >>> separar_campo("  Horário da coleta :  14:30 ")
    ('Horário da coleta', '14:30')
    >>> separar_campo("Observações gerais")
    ('Observações gerais', '')

Dica: `partition` foi feito para isso.

Teste com:  python curso.py testar 05 ex03
```

Exercício 05.04 — Converter número brasileiro

Arquivo: `exercicios/ex04_br_para_float.py`

```
Nível: ★☆☆

Contexto: Na planilha e no e-mail, os números vêm no formato brasileiro:
"1.250,50", "25,4". O Python só entende ponto decimal, e "1.250" (mil
duzentos
e cinquenta) viraria 1.25 sem nenhum aviso.

Tarefa: implemente `br_para_float(texto)`:
- aceite espaços nas pontas;
- "." é separador de milhar e "," é separador decimal;
- aceite sinal de menos na frente;
- se o texto não for um número válido, deixe o próprio `float()` levantar
  ValueError (não trate o erro: isso é assunto do módulo 06).

Exemplos:
    >>> br_para_float("1.250,50")
    1250.5
    >>> br_para_float(" 25,4 ")
    25.4
    >>> br_para_float("1.250")
    1250.0
    >>> br_para_float("-0,5")
    -0.5
```

```
Dica: a ordem das trocas importa (seção 6 da aula).

Teste com:  python curso.py testar 05 ex04
```

Exercício 05.05 — Validar código de OP

Arquivo: `exercicios/ex05_validar_op.py`

```
Nível: ★☆☆

Contexto: Antes de procurar a OP no ERP, o robô confere se o que foi
digitado
tem cara de código de OP. Isso evita buscas inúteis e mensagens confusas.

Tarefa: implemente `validar_op(codigo)`, que devolve True ou False.
É válido o texto que, depois de tirar os espaços das pontas, é formado
EXATAMENTE por:
- as letras "OP" (maiúsculas ou minúsculas);
- opcionalmente, UM espaço ou UM hífen;
- exatamente 6 dígitos.
Nada antes, nada depois.

Exemplos:
    >>> validar_op("OP 123456")
    True
    >>> validar_op(" op-004512 ")
    True
    >>> validar_op("OP123456")
    True
    >>> validar_op("OP 12345")
    False
    >>> validar_op("OP 123456 cancelada")
    False

Dica: é o padrão construído na seção 10 da aula. `re.fullmatch` devolve
um
Match ou None; transforme em bool. Lembre-se do `import re` no topo do
arquivo.

Teste com:  python curso.py testar 05 ex05
```

Exercício 05.06 — Número no formato brasileiro

Arquivo: `exercicios/ex06_float_para_br.py`

```
Nível: ★★☆

Contexto: O certificado e a resposta para a expedição mostram números no
padrão
brasileiro: "1.250,50 kg", "0,215". As contas são feitas em float; a
conversão
para texto só acontece na hora de mostrar.

Tarefa: implemente `float_para_br(valor, casas=2)`:
- ponto como separador de milhar e vírgula como separador decimal;
- exatamente `casas` casas decimais (arredondadas como faz a f-string);
- com `casas=0`, sem vírgula nenhuma;
- números negativos mantêm o sinal de menos.

Exemplos:
    >>> float_para_br(1250.5)
    '1.250,50'
    >>> float_para_br(0.215, 3)
    '0,215'
    >>> float_para_br(-1234567.891, 1)
    '-1.234.567,9'
    >>> float_para_br(980, 0)
    '980'

Dica: f"{valor:,.{casas}f}" gera o formato americano com a quantidade de
casas
que está na variável. Depois é só trocar os símbolos sem que uma troca
desfaça
a outra (seção 6 da aula).

Teste com:  python curso.py testar 05 ex06
```

Exercício 05.07 — Normalizar nome de coluna

Arquivo: `exercicios/ex07_normalizar_chave.py`

```
Nível: ★★☆

Contexto: Cada setor batiza as colunas da planilha de um jeito:
"Gramatura (g/m²)", "COEFICIENTE DE ATRITO – Estático". Para juntar
planilhas,
você transforma cada nome numa "chave" simples, como um nome de variável:
gramatura_g_m2.

Tarefa: implemente `normalizar_chave(texto)`, seguindo NESTA ORDEM:
1. troque o símbolo de micro "µ" por "u";
2. normalize com NFKD e remova os caracteres combinantes (os acentos);
   assim "ç" vira "c" e "²" vira "2";
```

```
3. passe tudo para minúsculas;
4. troque cada sequência de caracteres que NÃO seja letra de a a z nem
dígito
   de 0 a 9 por um único "_";
5. tire os "_" das pontas.

Exemplos:
    >>> normalizar_chave("Gramatura (g/m²)")
    'gramatura_g_m2'
    >>> normalizar_chave("  Coeficiente de Atrito — Estático ")
    'coeficiente_de_atrito_estatico'
    >>> normalizar_chave("Espessura (µm)")
    'espessura_um'
    >>> normalizar_chave("Tração MD (N/15mm)")
    'tracao_md_n_15mm'

Dica: o passo 4 é um `re.sub` com a classe `[^a-z0-9]+`. Você vai
precisar de
`import re` e `import unicodedata` no topo do arquivo.

Teste com:  python curso.py testar 05 ex07
```

Exercício 05.08 — Encontrar OPs num texto

Arquivo: `exercicios/ex08_extrair_ops.py`

```
Nível: ★★☆

Contexto: Nem todo e-mail segue o padrão. Às vezes alguém escreve
"liberar a
OP 123456 e a op-123457 hoje". O robô precisa achar todas as OPs citadas.

Tarefa: implemente `extrair_ops(texto)`, que devolve a lista dos números
de OP
(como texto, 6 dígitos) na ordem em que aparecem, SEM repetir.
Uma OP no texto é:
- a sigla "OP", maiúscula ou minúscula, sem letra ou dígito colado ANTES
dela
  ("TOP 111111" não conta);
- seguida de separadores opcionais, em qualquer quantidade: espaços, ":"
ou "-";
- seguida de exatamente 6 dígitos, que não podem ter letra nem dígito
colado
  depois ("OP 1234567" não conta).

Exemplos:
    >>> extrair_ops("Liberar OP 123456 e op-123457. A OP: 123456 já
foi.")
    ['123456', '123457']
```

```
    >>> extrair_ops("OP 1234567 tem dígito demais; TOP 111111 não é OP")
    []
    >>> extrair_ops("Sem OPs hoje")
    []

Dica: com UM grupo de captura no padrão, `re.findall` devolve só o
conteúdo do
grupo. Para não repetir, confira se o número já está na lista antes de
pôr.

Teste com:  python curso.py testar 05 ex08
```

Exercício 05.09 — Ler uma linha de medição

Arquivo: `exercicios/ex09_ler_medicao.py`

```
Nível: ★★☆

Contexto: O equipamento de ensaio exporta um texto com uma medição por
linha:
"Espessura: 25,4 µm", "Gramatura = 23,10 g/m²", "COF estático: 0,21".
Você
quer cada linha como um dicionário.

Tarefa: implemente `ler_medicao(linha)`. O formato da linha é:
    <ensaio> <separador> <valor> <unidade opcional>
- ensaio: o texto antes do primeiro ":" ou "=", sem espaços nas pontas;
não
  pode ser vazio;
- separador: ":" ou "=";
- valor: número no formato brasileiro (dígitos, pontos de milhar
opcionais,
  vírgula decimal opcional), podendo ter "-" na frente;
- unidade: tudo o que vem depois do número, sem espaços nas pontas (pode
ser "").
Devolva {"ensaio": str, "valor": float, "unidade": str}, ou None se a
linha
não estiver nesse formato.

Exemplos:
    >>> ler_medicao("Espessura: 25,4 µm")
    {'ensaio': 'Espessura', 'valor': 25.4, 'unidade': 'µm'}
    >>> ler_medicao("  Selagem = 1.250,5 N/15 mm ")
    {'ensaio': 'Selagem', 'valor': 1250.5, 'unidade': 'N/15 mm'}
    >>> ler_medicao("COF estático: 0,21")
    {'ensaio': 'COF estático', 'valor': 0.21, 'unidade': ''}
    >>> print(ler_medicao("Observação: COF liberado"))
    None
```

```
Dica: `re.fullmatch` com grupos nomeados. Para o ensaio, pense em "tudo
que não
é : nem =". Teste cada grupo separado no REPL antes de juntar.

Teste com:  python curso.py testar 05 ex09
```

Exercício 05.10 — Datas no padrão ISO

Arquivo: `exercicios/ex10_padronizar_datas.py`

```
Nível: ★★☆

Contexto: Os laudos antigos têm datas como "05/03/2025" e "7/3/2025" no
meio do
texto. Para ordenar e filtrar, o sistema novo quer o formato ISO "2025-
03-05"
(ano-mês-dia), que ordena certo em qualquer lugar.

Tarefa: implemente `padronizar_datas(texto)`, que troca cada data
dia/mês/ano
por ano-mês-dia e devolve o texto novo:
- na entrada, dia e mês têm 1 ou 2 dígitos ("5/3/2025" ou "05/03/2025");
- na saída, dia e mês têm SEMPRE 2 dígitos ("2025-03-05");
- o ano tem 4 dígitos;
- a data não pode estar grudada em outros dígitos: "105/03/2025" e
  "01/02/20251" não são datas;
- não precisa validar se a data existe; o resto do texto não muda.

Exemplos:
    >>> padronizar_datas("Produzido em 05/03/2025, liberado em
7/3/2025.")
    'Produzido em 2025-03-05, liberado em 2025-03-07.'
    >>> padronizar_datas("NF 123/2025 sem data")
    'NF 123/2025 sem data'

Dica: `re.sub` aceita uma FUNÇÃO no lugar do texto de troca (seção 13 da
aula).
A função recebe o Match e devolve o texto novo; `zfill` completa os
zeros.

Teste com:  python curso.py testar 05 ex10
```

Exercício 05.11 — Depure a função do estagiário

Arquivo: `exercicios/ex11_depurar_peso.py`

Nível: ★★☆

Contexto: Um estagiário escreveu a função abaixo para pegar o peso de uma linha
do e-mail da expedição. "Funcionou no exemplo dele", mas a expedição reclamou
de pesos errados e de o programa quebrar.

```
    import re

    def extrair_peso_original(linha):
        achado = re.search("(\d+,\d+) kg", linha)
        return float(achado.group(1).replace(",", "."))
```

Tarefa, parte 1 (LEIA antes de rodar): sem executar o código, preveja o que
`extrair_peso_original` faz em cada caso e complete `previsoes()`, que devolve
uma tupla com 3 respostas:
  a) o valor devolvido para "Peso: 25,0 kg";
  b) o valor devolvido para "Peso: 1.250,5 kg";
  c) o NOME do erro (por exemplo, "ValueError") levantado para "Peso: 980 kg".
Só depois copie a função para o REPL e confira suas respostas.

Tarefa, parte 2: escreva `extrair_peso(linha)`, a versão corrigida:
- o número que interessa é o que vem depois da palavra "Peso" (maiúscula ou
  minúscula, com ou sem ":"); outros números da linha, como a tara, não contam;
- aceite ponto de milhar e decimais opcionais ("1.250,5", "980", "2.000,00");
- "kg" depois do número é opcional;
- devolva o peso como float, ou None se a linha não tiver peso.

Exemplos:

```
    >>> extrair_peso("OP: 123456 | NF: 98765 | Peso: 1.250,5 kg")
    1250.5
    >>> extrair_peso("peso 980")
    980.0
    >>> print(extrair_peso("Bom dia, laboratório!"))
    None
```

Dica: a função original tem 5 defeitos. Procure: o "r" que falta, o rótulo
"Peso" que ninguém procura, o ponto de milhar, os decimais obrigatórios e o
None que não é tratado.

```
Teste com:  python curso.py testar 05 ex11
```

Exercício 05.12 — Ler o e-mail da expedição

Arquivo: `exercicios/ex12_ler_email_expedicao.py`

```
Nível: ★★★

Contexto: A expedição manda um e-mail por carga, com uma linha por
pedido. O
formato padrão é "OP: 123456 | NF: 98765 | Peso: 1.250,5 kg", mas cada
pessoa
digita de um jeito: "op 123457; nf 98.766; peso 980". O robô dos
certificados
precisa entender todas as variações abaixo.

Tarefa: implemente `ler_email_expedicao(texto)`, que devolve uma lista
com um
dicionário por pedido, na ordem do e-mail:
    {"op": "123456", "nf": "98765", "peso_kg": 1250.5}
Uma linha de pedido tem os campos OP, NF e Peso, NESTA ORDEM, separados
por
"|" ou ";":
- os rótulos OP, NF e Peso podem vir em maiúsculas ou minúsculas, com ou
sem
  ":" depois; entre "OP" e o número também pode haver um "-" ("OP-
123456");
- OP: exatamente 6 dígitos;
- NF: dígitos, com ou sem pontos de milhar ("98.765"); devolva só os
dígitos;
- Peso: número no formato brasileiro, com "kg" opcional depois;
- espaços extras em volta de qualquer parte são permitidos.
Linhas fora desse formato são ignoradas. E-mail sem pedidos devolve [].

Exemplos:
    >>> email = '''Bom dia!
    ... OP: 123456 | NF: 98765 | Peso: 1.250,5 kg
    ... op 123457; nf 98.766; peso 980
    ... A OP 123459 vai na próxima carga.'''
    >>> for pedido in ler_email_expedicao(email):
    ...     print(pedido)
    {'op': '123456', 'nf': '98765', 'peso_kg': 1250.5}
    {'op': '123457', 'nf': '98766', 'peso_kg': 980.0}

Dica: parta do padrão da seção 15 da aula e acrescente UMA variação por
vez,
testando todos os exemplos a cada mudança. Tanto `re.MULTILINE` quanto um
`for`
```

```
nas linhas (`splitlines`) funcionam.

Teste com:  python curso.py testar 05 ex12
```

Exercício 05.13 — Revisão: ler a especificação da OP

Arquivo: `exercicios/ex13_revisao_tolerancias.py`

```
Nível: ★★★

Contexto: Quando a OP tem especificação especial, ela chega como texto:
um
título e uma linha por ensaio. Espessura e gramatura usam tolerância
sobre o
valor nominal (± %); os outros ensaios usam faixa de mínimo a máximo.

    ESPECIFICAÇÃO ESPECIAL - CLIENTE PLASTIFORT
    Espessura: 25 µm ± 10%
    Gramatura: 23,0 g/m² +/- 5%
    COF estático: 0,15 a 0,30
    Tração MD: 30 a 45 MPa

Este exercício mistura regex (módulo 05) com dicionários e tuplas (módulo
03),
decisões (módulo 02) e funções auxiliares (módulo 04).

Tarefa 1: implemente `ler_tolerancias(texto)`, que devolve um dicionário
{chave_do_ensaio: (mínimo, máximo)}.
- Formato A (nominal): "<ensaio>: <nominal> [unidade] ± <pct>%"; aceite
também
  "+/-" no lugar de "±". mínimo = nominal * (1 - pct/100) e
  máximo = nominal * (1 + pct/100).
- Formato B (faixa): "<ensaio>: <mínimo> a <máximo> [unidade]".
- Os números vêm no formato brasileiro ("23,0", "0,15").
- Chave do ensaio: minúsculas, sem acento, espaços trocados por "_"
  ("COF estático" vira "cof_estatico").
- Arredonde mínimo e máximo com round(valor, 4).
- Linhas em outro formato (como o título) são ignoradas.

Tarefa 2: implemente `conferir(texto_especificacao, medicoes)`.
`medicoes` é um
dicionário {chave: valor medido}. Para cada ensaio DA ESPECIFICAÇÃO,
devolva
"OK" (dentro da faixa, limites inclusive), "FORA" ou "SEM MEDIÇÃO"
(quando a
chave não está em `medicoes`). Medições de ensaios que não estão na
especificação são ignoradas.

Exemplos:
```

```
    >>> spec = "Espessura: 25 µm ± 10%\nCOF estático: 0,15 a 0,30"
    >>> ler_tolerancias(spec)
    {'espessura': (22.5, 27.5), 'cof_estatico': (0.15, 0.3)}
    >>> conferir(spec, {"espessura": 26.1, "tracao_md": 40.0})
    {'espessura': 'OK', 'cof_estatico': 'SEM MEDIÇÃO'}

Dica: duas regex (uma para cada formato) são mais fáceis de testar do que
uma
gigante. Escreva funções auxiliares pequenas: uma converte o número
brasileiro,
outra cria a chave do ensaio (lembra da aula sobre remover acentos?).

Teste com:  python curso.py testar 05 ex13
```

Exercício 05.14 — Revisão: resumo para responder a expedição

Arquivo: `exercicios/ex14_revisao_resumo.py`

```
Nível: ★★★

Contexto: Depois de ler o e-mail, o laboratório responde à expedição com
um
resumo alinhado, para conferir os pedidos de relance (o Outlook mostra em
fonte de largura fixa quando você cola como texto sem formatação).

Este exercício mistura formatação de texto (módulo 05) com f-strings e
alinhamento (módulo 01), decisões (módulo 02) e listas, sorted e
acumuladores
(módulo 03).

Tarefa: implemente `montar_resumo(pedidos)`. `pedidos` é uma lista de
dicionários como os do ex12: {"op": "123456", "nf": "98765", "peso_kg":
1250.5}.
Regras:
- uma linha por pedido, em ordem CRESCENTE de OP, neste formato:
      OP 123456 | NF 000.098.765 |   1.250,50 kg
  (NF com 9 dígitos e pontos, como no ex02; peso no formato brasileiro
com
  2 casas, alinhado à direita em 10 caracteres, como no ex06);
- no fim, uma linha de total:
      Total: 2 pedidos           |   2.230,50 kg
  ("Total: N pedidos", ou "Total: 1 pedido" no singular, alinhado à
esquerda
  em 26 caracteres; depois " | ", o peso total no mesmo formato e " kg");
- as linhas são separadas por quebra de linha, sem quebra no final;
- lista vazia: devolva "Nenhum pedido encontrado.".

Exemplos:
    >>> pedidos = [
```

```
    ...     {"op": "123457", "nf": "98766", "peso_kg": 980.0},
    ...     {"op": "123456", "nf": "98765", "peso_kg": 1250.5},
    ... ]
    >>> print(montar_resumo(pedidos))
    OP 123456 | NF 000.098.765 |   1.250,50 kg
    OP 123457 | NF 000.098.766 |     980,00 kg
    Total: 2 pedidos           |   2.230,50 kg

Dica: copie para este arquivo as funções que você escreveu nos ex02 e
ex06 (no
módulo 07 você aprende a importá-las em vez de copiar). Monte uma lista
de
linhas e junte no final com "\n".join(linhas).

Teste com:  python curso.py testar 05 ex14
```

Desafio do módulo 05 — Leitor de e-mails da expedição

Contexto

Toda carga que sai da fábrica gera um e-mail da expedição para o laboratório, com as OPs, as NFs e os pesos. Hoje alguém lê cada e-mail, copia os números para uma planilha e confere tudo à mão. Erros de digitação passam, pedidos repetidos passam, e "1.250" kg às vezes vira "1,25".

Os e-mails foram salvos como `.txt` na pasta `dados/emails/` deste módulo. E, como na vida real, cada um foi salvo de um jeito:

```
dados/emails/
  2025-10-28_carga_embalagens_do_sul.txt         UTF-8, formato padrão
  2025-10-29_carga_flexpack.txt                  cp1252 (Bloco de Notas
antigo), digitação variada
  2025-10-30_complemento_embalagens_do_sul.txt   UTF-8 com BOM, quebras
de linha do Windows
```

Sua missão: escrever `leitor_emails.py`, um programa que lê todos esses arquivos e imprime um resumo confiável.

Requisitos obrigatórios

1. Ler **todos** os arquivos `.txt` da pasta `dados/emails/`, em ordem alfabética.
2. Decodificar cada arquivo corretamente, seja UTF-8, UTF-8 com BOM ou cp1252, e mostrar qual encoding foi usado. Nenhum `Ã§` e nenhum `�` na saída.

3. Extrair do cabeçalho o remetente (`De:`), o assunto (`Assunto:`) e o cliente (`Cliente:`), com o nome do cliente padronizado (como no ex01).
4. Extrair os pedidos aceitando as mesmas variações do ex12 (rótulos em maiúsculas ou minúsculas, `|` ou `;`, NF com pontos, `kg` opcional).
5. Para cada e-mail, imprimir os pedidos ordenados por OP no formato do ex14 (NF com 9 dígitos e pontos, peso brasileiro alinhado) e a linha de total.
6. Apontar as **linhas para conferir**: linhas que falam de OP e também de NF ou peso, mas que não estão no formato de pedido (OP com 5 dígitos, peso "a confirmar"...).
7. No fim, um **resumo geral**: quantidade de e-mails, clientes (sem repetir, mesmo que escritos de jeitos diferentes), total de pedidos, peso total no formato brasileiro, lista das linhas para conferir e as **OPs repetidas** em mais de um e-mail.

Extras opcionais

- Aceitar o caminho da pasta como argumento na linha de comando (`sys.argv`, uma prévia do módulo 07).
- Gerar o texto pronto de resposta à expedição: "Recebemos N pedidos (X kg). As linhas abaixo precisam de correção: ...".
- Avisar quando o mesmo pedido (OP e NF iguais) aparece duas vezes **no mesmo** e-mail.
- Destacar pesos muito acima da média (por exemplo, mais que o dobro), que podem ser erro de digitação.
- Salvar o resumo num arquivo `.txt` em UTF-8 (prévia do módulo 06).

Exemplo de execução

Na pasta `curso-python`:

```
python 05-texto-e-regex/desafio/leitor_emails.py
```

Saída esperada (a da solução de referência):

```
============================================================
E-mail : 2025-10-28_carga_embalagens_do_sul.txt  (UTF-8)
De     : expedicao@plastifort.com.br
Assunto: Certificados - carga de 28/10
Cliente: Embalagens do Sul Ltda
------------------------------------------------------------
OP 123456 | NF 000.098.765 |   1.250,50 kg
OP 123457 | NF 000.098.766 |     980,00 kg
OP 123458 | NF 000.098.767 |   2.040,75 kg
```

```
Total: 3 pedidos           |   4.271,25 kg
============================================================
E-mail : 2025-10-29_carga_flexpack.txt  (cp1252)
De     : Márcia (Expedição) <expedicao@plastifort.com.br>
Assunto: Liberação urgente - caminhão às 14h
Cliente: Flexpack Indústria e Comércio de Embalagens
------------------------------------------------------------
OP 223344 | NF 000.012.345 |   2.000,00 kg
OP 223345 | NF 000.012.346 |     750,25 kg
Total: 2 pedidos           |   2.750,25 kg
  ! conferir: OP: 22335 | NF: 12347 | Peso: 500 kg
============================================================
E-mail : 2025-10-30_complemento_embalagens_do_sul.txt  (UTF-8 com BOM)
De     : expedicao@plastifort.com.br
Assunto: Complemento da carga de 28/10
Cliente: Embalagens do Sul Ltda
------------------------------------------------------------
OP 123457 | NF 000.098.766 |     980,00 kg
OP 334455 | NF 000.099.001 |   1.100,00 kg
Total: 2 pedidos           |   2.080,00 kg
  ! conferir: OP 334456 | NF 99002 | Peso: a confirmar
============================================================
RESUMO GERAL
E-mails lidos  : 3
Clientes       : Embalagens do Sul Ltda, Flexpack Indústria e Comércio de
Embalagens
Pedidos        : 7
Peso total     : 9.101,50 kg
Para conferir  : 2 linha(s)
  - 2025-10-29_carga_flexpack.txt: OP: 22335 | NF: 12347 | Peso: 500 kg
  - 2025-10-30_complemento_embalagens_do_sul.txt: OP 334456 | NF 99002 |
Peso: a confirmar
OPs repetidas  :
  - OP 123457 em: 2025-10-28_carga_embalagens_do_sul.txt, 2025-10-
30_complemento_embalagens_do_sul.txt
```

O seu layout pode ser diferente. O que importa são os números e os avisos.

Critérios de aceite

- [ ] Roda sem erro com `python 05-texto-e-regex/desafio/leitor_emails.py`, sem pedir nada ao usuário.
- [ ] Os três e-mails aparecem com acentos corretos ("Márcia", "Expedição").
- [ ] São encontrados exatamente 7 pedidos, somando 9.101,50 kg.
- [ ] As 2 linhas com problema aparecem para conferência, e a frase "A OP 123459 vai na próxima carga." **não** aparece (ela não é um pedido).
- [ ] A OP 123457 é apontada como repetida.

- [ ] "Embalagens do Sul Ltda" aparece uma vez só na lista de clientes.
- [ ] Nenhuma conta é feita com texto: os pesos viram `float` na leitura e voltam a ser texto brasileiro só na impressão.
- [ ] Cada tarefa está numa função pequena com nome claro (decodificar, ler cabeçalho, ler pedidos, montar resumo...).
- [ ] Todas as regex usam `r"..."`, e as grandes estão compiladas em constantes.

Dicas de como começar

1. **Comece pelos bytes.** Para chegar na pasta dos e-mails a partir do seu script, use `Path(__file__).resolve().parent.parent / "dados" / "emails"`: `Path(__file__)` é o caminho do próprio script, `.parent` sobe uma pasta (detalhes no módulo 06). Depois, `sorted(pasta.glob("*.txt"))` lista os arquivos e `arquivo.read_bytes()` lê os bytes.
2. **Decodifique antes de tudo.** Reaproveite a função `decodificar` da seção 8 da aula. Imprima as primeiras linhas de cada e-mail e confira os acentos antes de seguir.
3. **Reaproveite seus exercícios.** `limpar_nome` (ex01), `formatar_nf` (ex02), `float_para_br` (ex06), `ler_email_expedicao` (ex12) e `montar_resumo` (ex14) já fazem metade do trabalho. Copie-os para o seu arquivo.
4. **Um e-mail de cada vez.** Faça funcionar para o primeiro arquivo, depois rode nos três. O segundo e o terceiro existem justamente para quebrar o que funcionou no primeiro.
5. **Linhas para conferir.** Percorra as linhas: se casou com o padrão do pedido, é pedido; se não casou, mas tem "OP" e também "NF" ou "Peso", é suspeita.
6. **OPs repetidas.** Monte um dicionário `{op: [arquivos onde apareceu]}` e mostre só as OPs com mais de um arquivo.

A solução de referência está em `solucao_referencia.py`. Abra só depois de tentar.

Quiz — Módulo 05: Texto e expressões regulares

Responda sem rodar o código. Depois confira no gabarito e, na dúvida, teste no REPL.

1. O que este código imprime?

```
print("lotes.csv".rstrip(".csv"))
```

- a) `lotes`
- b) `lote`
- c) `lotes.`

- d) Dá erro.

2. O que este código imprime?

```
print(float("1.250"))
```

- a) `1250.0`
- b) `1.25`
- c) `1250`
- d) `ValueError`

3. Qual é o resultado de `" ".join("  embalagens   do  sul ".split())`?

- a) `' embalagens do sul '`
- b) `'embalagens   do  sul'`
- c) `'embalagens do sul'`
- d) `['embalagens', 'do', 'sul']`

4. Você quer saber se o texto **inteiro** digitado pelo operador é um código de OP. Qual função usar?

- a) `re.search`
- b) `re.match`
- c) `re.fullmatch`
- d) `re.findall`

5. O que este código imprime?

```
import re
print(re.findall(r"(\d+),(\d+)", "25,4 e 0,21"))
```

6. Onde está o erro? O que acontece ao rodar?

```
import re
achado = re.search(r"NF:\s*(\d+)", "Nota ainda sem número")
print(achado.group(1))
```

7. O que este código imprime?

```
print(f"[{'OK':^6}]")
```

8. Um relatório mostra `ExpediÃ§Ã£o`. O que provavelmente aconteceu?

- a) O arquivo estava em cp1252 e foi lido como UTF-8.

- b) O arquivo estava em UTF-8 e foi lido como cp1252.
- c) O texto tinha acentos decompostos (NFD).
- d) Faltou o `r` antes das aspas.

9. Quanto vale `len("ação".encode("utf-8"))`?

- a) 4
- b) 5
- c) 6
- d) 8

10. O que este código imprime?

```
import re
texto = "OP-123456, OP 1234567, OP 654321"
print(re.findall(r"\bOP[ -]?(\d{6})\b", texto))
```

11. O que este código imprime?

```
import re
print(re.sub(r"(\d{2})/(\d{2})", r"\2/\1", "05/03"))
```

12. Onde está o erro? Por que o primeiro `search` não acha nada?

```
import re
padrao = re.compile(r"OP \d{6}", re.VERBOSE)
print(padrao.search("OP 123456"))   # None
print(padrao.search("OP123456"))    # acha!
```

13. Qual expressão transforma `1234.5` em `'1.234,50'`?

- a) `f"{1234.5:.2f}".replace(".", ",")`
- b) `f"{1234.5:,.2f}".replace(",", ".").replace(".", ",")`
- c) `f"{1234.5:,.2f}".translate(str.maketrans(",.", ".,"))`
- d) `str(1234.5).replace(".", ",")`

14. Para transformar o cabeçalho `"Gramatura (g/m²)"` em `"gramatura_g_m2"`, qual normalização faz o `²` virar `2`?

- a) NFC
- b) NFD
- c) NFKD
- d) Nenhuma: precisa de `replace("²", "2")`.

Gabarito

1. **b)** **`lote`.** `rstrip` recebe um **conjunto de caracteres**, não um sufixo. Ele tira do fim todos os `.`, `c`, `s`, `v` que encontrar, inclusive o `s` de "lotes". Para sufixo exato: `removesuffix(".csv")`.
2. **b)** **`1.25`.** Para o Python, o ponto é decimal. Um peso brasileiro de 1.250 kg vira 1,25 kg sem erro nenhum. Por isso texto brasileiro sempre passa pela conversão.
3. **c)** **`'embalagens do sul'`.** `split()` sem argumento quebra em qualquer espaço e ignora repetições e pontas; `join` cola com um espaço só.
4. **c)** **`re.fullmatch`.** `search` acha o padrão em qualquer lugar; `match` só exige o começo. `fullmatch` exige que o texto inteiro case (o mesmo que `^...$`).
5. **`[('25', '4'), ('0', '21')]`.** Com dois grupos, `findall` devolve uma tupla por ocorrência, com o conteúdo de cada grupo.
6. **`AttributeError: 'NoneType' object has no attribute 'group'`.** O texto não tem "NF:", então `search` devolve `None`. Teste antes: `if achado: ...`.
7. **`[  OK  ]`.** `^6` centraliza "OK" em 6 caracteres (2 espaços de cada lado).
8. **b).** Cada letra acentuada virou duas letras estranhas: é UTF-8 lido como cp1252. O caso a) costuma dar `UnicodeDecodeError` ou o losango `�`. Conserto: `texto.encode("cp1252").decode("utf-8")`.
9. **c) 6.** "a" e "o" ocupam 1 byte cada; "ç" e "ã" ocupam 2 bytes cada em UTF-8.
10. **`['123456', '654321']`.** O `\b` depois dos dígitos impede casar os 6 primeiros dígitos de "1234567". O grupo faz o `findall` devolver só o número.
11. **`03/05`.** `\2` e `\1` reaproveitam os grupos na ordem trocada.
12. **No modo `VERBOSE`, espaços no padrão são ignorados.** O padrão vira `OP\d{6}`, que não aceita espaço. Para um espaço literal, use `\s`, `[ ]` ou `\ `.
13. **c).** A b) troca vírgula por ponto e depois **todos** os pontos por vírgula, dando `'1,234,50'`. A a) não tem separador de milhar (`'1234,50'`). A d) dá `'1234,5'`.
14. **c) NFKD.** A forma de compatibilidade simplifica caracteres "parecidos": `²` vira `2`, `ﬁ` vira `fi`. A NFD só separa letras e acentos.

Módulo 06 — Arquivos e tratamento de erros

> **Tempo estimado:** 10 a 14 horas · **Pré-requisitos:** módulos 00 a 05 · **Nível:** iniciante → intermediário

O que você vai aprender

- Ler um traceback de baixo para cima e reconhecer as exceções mais comuns.
- Tratar erros com `try`, `except`, `else` e `finally`, capturando só o que deve.
- Levantar erros com `raise`, com mensagens que dizem **onde** está o problema.
- Criar exceções próprias (`ValorForaDaTolerancia`, `ArquivoInvalido`) e organizá-las numa família.
- Abrir, ler e gravar arquivos de texto com `with open(...)`, escolhendo modo e encoding.
- Trabalhar com caminhos usando `pathlib` (`Path`, `/`, `glob`, `mkdir`, `read_text`).
- Ler e gravar CSV no padrão do Excel brasileiro: `;`, vírgula decimal, `utf-8-sig` e `cp1252`.
- Ler e gravar JSON (configurações e regras).
- Gravar arquivos de forma segura (sem corromper o que já existia) e validar dados na entrada.

Por que isso importa

Programa de verdade conversa com arquivos: a planilha exportada do Excel, o JSON de configuração, o log do robô. E arquivo de verdade vem com problema: célula vazia, "#N/D", acento quebrado, coluna faltando, arquivo aberto no Excel por outra pessoa.

Iniciante escreve código que funciona com o arquivo perfeito. Profissional escreve código que, diante do arquivo ruim, **para no lugar certo, com uma mensagem clara**, ou segue em frente registrando o problema. Essa diferença aparece em qualquer entrevista de dados, automação ou backend.

No projeto final, este módulo é a base de tudo: ler as medições, carregar as regras de tolerância, gravar o resultado para o Excel e **não emitir certificado** de lote fora da especificação.

> 💡 **Dica:** os exemplos completos estão em `exemplos/`, na ordem das seções. Os arquivos de dados ficam em `dados/`. Rode a partir da pasta `curso-python`, por exemplo: `python 06-arquivos-e-erros/exemplos/01_excecoes.py`.

1. Quando algo dá errado: exceções

Quando o Python encontra algo que não consegue fazer, ele **levanta uma exceção**: um objeto que descreve o problema. Se ninguém trata a exceção, o programa para e mostra o *traceback*.

```
>>> int("25,4")
Traceback (most recent call last):
  ...
ValueError: invalid literal for int() with base 10: '25,4'
```

Lendo um traceback

Leia **de baixo para cima**. A última linha diz o quê; as de cima dizem onde.

```
Traceback (most recent call last):
  File "C:\curso-python\conferir.py", line 12, in <module>      ← quem
chamou
    media = calcular_media(valores)
  File "C:\curso-python\conferir.py", line 5, in calcular_media ← onde
quebrou
    return sum(valores) / len(valores)
ZeroDivisionError: division by zero                              ← o QUE
aconteceu
```

1. Última linha: o **tipo** (`ZeroDivisionError`) e a **mensagem**.
2. Linha logo acima: o arquivo, a linha e o código exato que quebrou.
3. Mais acima: o caminho de chamadas até ali. Procure a primeira linha que é **seu** código (e não de uma biblioteca).

As exceções que você mais vai ver

```
Exceção              Quando acontece                          Exemplo
-------------------  ---------------------------------------  -----------
----------------
ValueError           tipo certo, valor impossível             float("25,4
kg")
TypeError            tipo errado para a operação              "25,4" + 1
```

```
KeyError             chave que não existe no dicionário
linha["lote"] sem coluna lote
IndexError           posição que não existe na lista          valores[10]
numa lista de 3
ZeroDivisionError    divisão por zero                         sum([]) /
len([])
AttributeError       atributo/método que não existe
None.group(1)
FileNotFoundError    arquivo ou pasta não existe
open("laudo.csv")
PermissionError      sem permissão (ou arquivo aberto no Excel) gravar em
resultado.csv aberto
UnicodeDecodeError   bytes num encoding diferente do pedido   cp1252 lido
como UTF-8
```

As exceções formam uma **família** (uma hierarquia). Capturar a "mãe" captura também as filhas. Isso vai ser importante daqui a pouco.

```
Exception
 ├── ArithmeticError
 │    └── ZeroDivisionError
 ├── LookupError
 │    ├── IndexError
 │    └── KeyError
 ├── OSError
 │    ├── FileNotFoundError
 │    ├── FileExistsError
 │    └── PermissionError
 ├── TypeError
 └── ValueError
      ├── UnicodeDecodeError     (via UnicodeError)
      └── json.JSONDecodeError
```

🔮 **Preveja antes de rodar:** qual exceção (tipo e mensagem) cada linha levanta? Rode uma de cada vez no REPL.

```python
medicoes = {"espessura": 25.4}
valores = [25.4, 26.1]
medicoes["cof"]
valores[5]
"25,4" + 1
10 / 0
```

▼ **Resposta**

- `medicoes["cof"]` → `KeyError: 'cof'` (a mensagem é só a chave que faltou).
- `valores[5]` → `IndexError: list index out of range`.
- `"25,4" + 1` → `TypeError: can only concatenate str (not "int") to str`.
- `10 / 0` → `ZeroDivisionError: division by zero`.

2. try/except: tratando o erro

Quando você **sabe** que algo pode dar errado e sabe o que fazer nesse caso, use `try/except`. O código "arriscado" vai no `try`; o plano B, no `except`.

```
>>> texto = "25,4"
>>> try:
...     valor = float(texto)
... except ValueError:
...     print(f"Não consegui converter {texto!r}")
...
Não consegui converter '25,4'
```

Se nada der errado, o `except` é pulado. Se der um erro **de outro tipo**, ele não é capturado e continua subindo. Com `as`, você pega o objeto da exceção e pode usar a mensagem:

```
>>> try:
...     float("abc")
... except ValueError as erro:
...     print("Problema:", erro)
...
Problema: could not convert string to float: 'abc'
```

Exemplo resolvido: rendimento do filme

Problema: calcular o rendimento (m² por kg) a partir de dois textos da planilha. Podem acontecer dois problemas diferentes, e cada um merece uma mensagem diferente.

Primeiro eu listei o que pode dar errado: texto que não é número (`ValueError`) e peso zero (`ZeroDivisionError`). Depois escrevi um `except` para cada um. Deixei no `try` só as linhas que podem falhar.

```
>>> def rendimento(area_texto, peso_texto):
...     try:
...         area = float(area_texto.replace(",", "."))
```

```
...         peso = float(peso_texto.replace(",", "."))
...         return area / peso
...     except ValueError:
...         print(f"Número inválido: área={area_texto!r}, peso=
{peso_texto!r}")
...     except ZeroDivisionError:
...         print("Peso zero: não dá para calcular o rendimento")
...     return None
...
>>> rendimento("1250", "25,5")
49.01960784313726
>>> rendimento("1250", "")
Número inválido: área='1250', peso=''
>>> rendimento("1250", "0")
Peso zero: não dá para calcular o rendimento
```

Quando o tratamento é igual para vários tipos, junte-os numa tupla: `except (ValueError, ZeroDivisionError):`.

> ⚠️ **Erro comum:** o `except` "pega-tudo". Um `except:` sozinho (ou `except Exception:`) captura **qualquer** erro, inclusive os seus bugs:
>
> ```
> def media(valores):
>     try:
>         return sum(valores) / len(valroes)   # erro de digitação!
>     except:
>         return 0
> ```
>
> `media([25.4, 26.1])` devolve `0`. O `NameError` do `valroes` foi engolido, e você vai passar horas procurando por que a média é zero. **Capture só o tipo que você sabe tratar.**

Pedir perdão ou pedir permissão?

Existem dois estilos para lidar com problemas previsíveis:

```
# Pedir permissão (verificar antes): "Look Before You Leap"
if "lote" in linha:
    lote = linha["lote"]

# Pedir perdão (tentar e tratar): "Easier to Ask Forgiveness than
Permission"
try:
    lote = linha["lote"]
```

```
except KeyError:
    lote = None
```

Em Python, o segundo estilo é muito comum, principalmente com arquivos: entre o `if caminho.exists()` e o `open`, outra pessoa pode apagar ou abrir o arquivo. Tentar e tratar o erro é mais seguro. Para coisas simples, como uma chave de dicionário, use o que ficar mais legível (aqui, `linha.get("lote")` seria ainda melhor).

3. else e finally

O `try` tem mais duas partes opcionais:

```
try:       o código que pode dar erro
except:    roda SE deu erro (do tipo indicado)
else:      roda SE NÃO deu erro
finally:   roda SEMPRE: deu erro ou não, com return ou sem
```

```
>>> def converter(texto):
...     try:
...         valor = float(texto)
...     except ValueError:
...         print("  except: inválido")
...         return None
...     else:
...         print("  else: convertido")
...         return valor
...     finally:
...         print("  finally: sempre")
...
>>> converter("25.4")
  else: convertido
  finally: sempre
25.4
>>> converter("x")
  except: inválido
  finally: sempre
```

Repare: o `finally` roda **antes** de a função de fato devolver o valor, mesmo com `return` no meio. Por que usar `else` em vez de pôr o código no fim do `try`? Para deixar no `try` só a linha que pode dar o erro tratado. Assim um erro no código do `else` não é capturado por engano.

O `finally` serve para "arrumar a casa" sempre: fechar uma conexão, apagar um arquivo temporário, registrar no log que o processo terminou. Para fechar arquivos, porém, existe um jeito melhor: o `with` (seção 6).

🔮 **Preveja antes de rodar:** o que aparece na tela, e em que ordem?

```
for texto in ["10", "0", "x"]:
    try:
        resultado = 100 / int(texto)
    except ZeroDivisionError:
        print(texto, "-> divisão por zero")
    except ValueError:
        print(texto, "-> não é número")
    else:
        print(texto, "->", resultado)
    finally:
        print("fim de", texto)
```

▼ **Resposta**

```
10 -> 10.0
fim de 10
0 -> divisão por zero
fim de 0
x -> não é número
fim de x
```

Cada volta do `for` passa por exatamente um entre `except` e `else`, e sempre pelo `finally`. O resultado de `100 / 10` é `10.0` (a barra `/` sempre devolve `float`).

4. raise: levantando seus próprios erros

Até agora o Python levantou os erros. Você também pode: `raise TipoDoErro("mensagem")`. Use quando a sua função recebe algo com que **não dá para continuar**.

```
>>> def checar_faixa(minimo, maximo):
...     if minimo > maximo:
...         raise ValueError(f"faixa inválida: mínimo {minimo} maior que
máximo {maximo}")
...     return (minimo, maximo)
...
>>> checar_faixa(20, 30)
(20, 30)
>>> checar_faixa(30, 20)
```

```
Traceback (most recent call last):
  ...
ValueError: faixa inválida: mínimo 30 maior que máximo 20
```

Uma boa mensagem de erro diz **o que** está errado e **com qual valor**. Compare "erro na leitura" com "linha 7: valor inválido em espessura: '2O,1'". A segunda poupa meia hora de quem vai corrigir a planilha.

Trocar o erro por outro mais claro: raise ... from

Às vezes você captura um erro técnico e quer levantar um erro com mais contexto. Use `from erro` para **encadear**: o traceback mostra os dois.

```
>>> def ler_peso(texto):
...     try:
...         return float(texto.replace(".", "").replace(",", "."))
...     except ValueError as erro:
...         raise ValueError(f"peso inválido no e-mail da expedição:
{texto!r}") from erro
...
>>> ler_peso("1.250,5")
1250.5
```

Com `ler_peso("1.250,5 kg")`, o traceback fica assim (resumido):

```
ValueError: could not convert string to float: '1250.5 kg'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  ...
ValueError: peso inválido no e-mail da expedição: '1.250,5 kg'
```

Repare na primeira mensagem: ela mostra o texto **já transformado** (`'1250.5 kg'`), que não existe em lugar nenhum do e-mail. A mensagem nova mostra o texto original, que a pessoa consegue procurar e corrigir.

E se você só quer registrar algo e deixar o **mesmo** erro seguir? Use `raise` sozinho dentro do `except`:

```
try:
    processar(arquivo)
except PermissionError:
    print(f"Feche o arquivo {arquivo} no Excel e rode de novo.")
    raise   # relança o mesmo PermissionError
```

▼ **Tente você: escreva `validar_linha(texto)`, que devolve a linha de produção em maiúscula ("a" → "A") e levanta ValueError com uma mensagem clara se ela não for A, B, C ou D.**

```python
def validar_linha(texto):
    linha = texto.strip().upper()
    if linha not in {"A", "B", "C", "D"}:
        raise ValueError(f"linha de produção inválida: {texto!r} (use
A, B, C ou D)")
    return linha
```

`validar_linha(" b ")` devolve `'B'`; `validar_linha("E")` levanta `ValueError: linha de produção inválida: 'E' (use A, B, C ou D)`. Repare no `!r`: as aspas mostram espaços escondidos no valor digitado.

Devolver None ou levantar erro?

```
Situação                                                  Faça
--------------------------------------------------------  ------------------
----------------
"Não achei" é normal e esperado (OP sem e-mail hoje)      devolva None (ou
lista vazia)
Dado de entrada impossível (faixa invertida)              raise ValueError
Arquivo fora do formato (falta coluna)                    raise
ArquivoInvalido (seção 5)
Resultado de negócio esperado (lote reprovado)            devolva o status
("REPROVADO")
Ação que NÃO pode acontecer (certificado de reprovado)    raise
ValorForaDaTolerancia
```

5. Exceções personalizadas

`ValueError` serve para muita coisa. Quando o problema é do **seu domínio** (qualidade, certificados), crie exceções com nomes que falam a língua do negócio. Quem chama pode tratar cada uma de um jeito.

Criar uma exceção é criar uma **classe** filha de `Exception`. Classes são o assunto do módulo 08; por enquanto, basta saber que `class Nome(Mae):` cria um tipo novo que "é um" `Mae`, e que o corpo pode ter só uma docstring.

```
>>> class ErroDeQualidade(Exception):
...     """Base de todos os erros do sistema de qualidade."""
...
```

```
>>> class ValorForaDaTolerancia(ErroDeQualidade):
...     """Uma medição saiu da faixa permitida."""
...
>>> class ArquivoInvalido(ErroDeQualidade):
...     """O arquivo existe, mas não está no formato esperado."""
...
```

```
Exception
 └── ErroDeQualidade
      ├── ValorForaDaTolerancia
      └── ArquivoInvalido
```

Você usa do mesmo jeito que as exceções do Python: `raise ArquivoInvalido("...")` e `except ArquivoInvalido:`. E `except ErroDeQualidade:` captura as duas filhas de uma vez.

Exemplo resolvido: certificado só de lote aprovado

Problema: a função que emite o certificado **nunca** pode emitir para lote fora da faixa, mesmo que quem a chamou esqueça de conferir.

Primeiro pensei em devolver `None` quando o lote estiver fora. Mas `None` é fácil de ignorar: o robô poderia seguir e mandar um certificado vazio. Um erro não dá para ignorar sem querer: ou alguém trata, ou o programa para. Por isso a função levanta `ValorForaDaTolerancia`, e o laço que emite os certificados decide o que fazer.

```
>>> def emitir_certificado(lote, espessura, minimo, maximo):
...     if not minimo <= espessura <= maximo:
...         raise ValorForaDaTolerancia(
...             f"lote {lote}: espessura {espessura} fora de [{minimo},
{maximo}]"
...         )
...     return f"Certificado do lote {lote}: espessura {espessura} µm
(OK)"
...
>>> for lote, espessura in [("L001", 25.1), ("L002", 28.3), ("L003",
24.0)]:
...     try:
...         print(emitir_certificado(lote, espessura, 22.5, 27.5))
...     except ValorForaDaTolerancia as erro:
...         print("NÃO EMITIDO:", erro)
...
Certificado do lote L001: espessura 25.1 µm (OK)
NÃO EMITIDO: lote L002: espessura 28.3 fora de [22.5, 27.5]
Certificado do lote L003: espessura 24.0 µm (OK)
```

O programa não parou no L002: o erro foi tratado no lugar certo, e os outros lotes seguiram. Repare na divisão de tarefas: `emitir_certificado` **detecta** o problema; o laço **decide** o que fazer com ele.

> 🔧 **Modifique:** troque `except ValorForaDaTolerancia` por `except ErroDeQualidade` e rode de novo. Continua funcionando? Por quê? Depois troque por `except ArquivoInvalido` e veja o que acontece no L002.

> 🔮 **Preveja antes de rodar:** o que é impresso?
>
> ```
> class ErroDeQualidade(Exception):
>     """Base."""
>
> class ArquivoInvalido(ErroDeQualidade):
>     """Arquivo fora do formato."""
>
> try:
>     raise ArquivoInvalido("cabeçalho sem a coluna lote")
> except ValueError:
>     print("A")
> except ErroDeQualidade as erro:
>     print("B", erro)
> except ArquivoInvalido:
>     print("C")
> ```

▼ **Resposta**

`B cabeçalho sem a coluna lote`.

`ArquivoInvalido` não é filha de `ValueError`, então o primeiro `except` não serve. O segundo serve (`ArquivoInvalido` é uma `ErroDeQualidade`) e, como só **um** `except` roda, o terceiro nunca é alcançado. Lição: coloque os `except` mais específicos **antes** dos mais genéricos.

6. Arquivos de texto: open e with

Para ler ou gravar um arquivo, você o **abre** com `open(caminho, modo, encoding=...)` e recebe um objeto arquivo. O jeito certo é sempre dentro de um `with`:

```
>>> with open("laudo.txt", "w", encoding="utf-8") as arquivo:
...     arquivo.write("Lote: L001\n")
```

```
...     arquivo.write("Espessura: 25,4 µm\n")
...
11
19
```

O `with` **fecha o arquivo sozinho** quando o bloco termina, mesmo que aconteça um erro no meio. É como uma porta com mola: você não precisa lembrar de fechar. (Os números `11` e `19` são o REPL mostrando o que `write` devolve: quantos caracteres gravou.)

> ⚠️ **Erro comum:** esperar que `write` pule de linha. Ele grava exatamente o texto que você passou. A quebra de linha é o `"\n"`, e é você quem coloca.

Para ler, há três jeitos principais:

```
>>> with open("laudo.txt", encoding="utf-8") as arquivo:
...     conteudo = arquivo.read()          # o arquivo inteiro numa str
...
>>> conteudo
'Lote: L001\nEspessura: 25,4 µm\n'
>>> with open("laudo.txt", encoding="utf-8") as arquivo:
...     for linha in arquivo:              # uma linha por vez, com o
"\n"
...         print(repr(linha))
...
'Lote: L001\n'
'Espessura: 25,4 µm\n'
>>> with open("laudo.txt", encoding="utf-8") as arquivo:
...     linhas = [linha.strip() for linha in arquivo]
...
>>> linhas
['Lote: L001', 'Espessura: 25,4 µm']
```

O `for linha in arquivo` lê aos poucos: serve até para arquivos de vários gigabytes. O `read()` carrega tudo na memória de uma vez, o que é ótimo para arquivos pequenos.

Modos de abertura

```
Modo   Faz                                                  Se o arquivo
não existe
----   ---------------------------------------------------  ------------
------------
"r"    ler (é o padrão; pode omitir)
FileNotFoundError
```

```
"w"    gravar do zero: APAGA o conteúdo anterior              cria
"a"    acrescentar no fim (append)                            cria
"x"    criar: dá erro se já existir (não sobrescreve nada)    cria
"b"    junto com outro ("rb", "wb"): bytes em vez de texto    -
```

🔮 **Preveja antes de rodar:** o que o último `print` mostra?

```python
with open("teste.txt", "w", encoding="utf-8") as f:
    f.write("A")
    f.write("B")
with open("teste.txt", "a", encoding="utf-8") as f:
    f.write("C\n")
with open("teste.txt", "w", encoding="utf-8") as f:
    f.write("D")
with open("teste.txt", encoding="utf-8") as f:
    print(f.read())
```

▼ **Resposta**

Só `D`.

O primeiro bloco grava `AB`; o segundo acrescenta `C` e uma quebra de linha (`ABC\n`). Mas o terceiro abre em `"w"`, que **apaga tudo** antes de gravar. Esse é o erro que mais destrói arquivos de log: abrir em `"w"` quando a intenção era `"a"`.

Sempre diga o encoding

Você viu no módulo 05: sem `encoding=`, o Python usa o padrão do sistema, que no Windows costuma ser cp1252. O mesmo programa lê certo no seu computador e quebra no do colega. Regra: **todo `open` de texto leva `encoding=`**. Para arquivos seus, use `"utf-8"`. Para arquivos do Excel, veja a seção 8.

7. pathlib: caminhos sem dor

Caminho de arquivo como texto (`"C:\\Laudos\\2025\\linha_A.csv"`) dá trabalho: barras diferentes no Windows e no Linux, `\n` virando quebra de linha, juntar pedaços com `+`. O módulo `pathlib` resolve isso com o objeto `Path`. O operador `/` junta os pedaços.

```python
>>> from pathlib import Path
>>> laudo = Path("laudos") / "2025" / "linha_A.csv"
```

```
>>> laudo.name, laudo.stem, laudo.suffix
('linha_A.csv', 'linha_A', '.csv')
>>> laudo.parent.name
'2025'
>>> laudo.with_suffix(".xlsx").name
'linha_A.xlsx'
>>> laudo.exists()
False
>>> laudo.parent.mkdir(parents=True, exist_ok=True)
>>> laudo.write_text("lote;espessura\nL001;25,4\n", encoding="utf-8")
25
>>> laudo.exists(), laudo.is_file(), laudo.parent.is_dir()
(True, True, True)
>>> laudo.read_text(encoding="utf-8").splitlines()
['lote;espessura', 'L001;25,4']
```

- `mkdir(parents=True, exist_ok=True)`: cria as pastas que faltam e não reclama se já existirem.
- `read_text` e `write_text` abrem, leem (ou gravam) e fecham numa linha só. Ótimos para arquivos pequenos. Para ler bytes crus: `read_bytes`.

Ao imprimir, o `Path` usa a barra do sistema:

```
>>> print(laudo)
laudos\2025\linha_A.csv      (Windows)
laudos/2025/linha_A.csv      (Linux e macOS)
```

Percorrendo uma pasta

```
>>> pasta = Path("laudos") / "2025"
>>> for nome in ["linha_B.csv", "linha_C.CSV", "notas.txt",
"~$linha_D.xlsx"]:
...     (pasta / nome).touch()        # touch cria um arquivo vazio
...
>>> sorted(p.name for p in pasta.iterdir())
['linha_A.csv', 'linha_B.csv', 'linha_C.CSV', 'notas.txt',
'~$linha_D.xlsx']
>>> sorted(p.name for p in pasta.glob("*.txt"))
['notas.txt']
>>> sorted(p.name for p in pasta.iterdir() if p.suffix.lower() == ".csv")
['linha_A.csv', 'linha_B.csv', 'linha_C.CSV']
```

`iterdir()` entrega tudo o que está na pasta; `glob("*.txt")` filtra por um padrão (`rglob` faz o mesmo entrando nas subpastas). Cuidado: no Windows o `glob` ignora maiúsculas e minúsculas, no Linux não. Para o mesmo resultado em qualquer

sistema, filtre com `p.suffix.lower()`. E os arquivos `~$...` são os temporários que o Excel cria enquanto uma planilha está aberta: quase sempre você quer ignorá-los.

Caminhos do Windows e a pasta do script

```
Path(r"C:\Laudos\2025")      # raw string: as barras ficam como estão
Path("C:/Laudos/2025")       # barra normal também funciona no Windows
Path.home() / "Documents"    # a pasta do usuário (C:\Users\voce)
```

> ⚠️ **Erro comum:** escrever `"C:\novos\testes"` sem o `r`. O `\n` vira quebra de linha e o `\t` vira tab, e o caminho fica irreconhecível.

Outra armadilha: caminho relativo (`Path("dados/regras.json")`) é relativo à pasta de onde você **rodou** o comando, e não à pasta do script. Rodou de outro lugar, levou um `FileNotFoundError`. A solução é partir da pasta do próprio script:

```
PASTA_DO_SCRIPT = Path(__file__).resolve().parent     # onde este .py
está
REGRAS = PASTA_DO_SCRIPT.parent / "dados" / "regras_tolerancia.json"
```

`__file__` é o caminho do arquivo `.py` que está rodando; `resolve()` transforma em caminho completo. Todos os exemplos deste módulo usam esse truque.

Exemplo resolvido: quantas medições tem cada CSV?

Problema: na pasta de laudos, contar as linhas de dados de cada CSV (sem o cabeçalho) para conferir se a exportação veio completa.

Primeiro eu separei em passos: listar os CSVs (filtro pelo `suffix`), ler cada um (`read_text`), contar as linhas não vazias e descontar o cabeçalho. O arquivo pode vir do Excel, então leio com `utf-8-sig` (seção 8).

```
from pathlib import Path

pasta = Path("laudos") / "2025"
for arquivo in sorted(pasta.iterdir()):
    if arquivo.suffix.lower() != ".csv":
        continue
    linhas = [l for l in arquivo.read_text(encoding="utf-8-
sig").splitlines() if l.strip()]
    print(f"{arquivo.name:<14} {max(len(linhas) - 1, 0):>3} medições")
```

Saída:

```
linha_A.csv      1 medições
linha_B.csv      0 medições
linha_C.CSV      0 medições
```

(Os arquivos B e C foram criados vazios com `touch`; por isso o `max(..., 0)`.)

> 🔧 **Modifique:** mostre também o tamanho de cada arquivo em bytes, com `arquivo.stat().st_size`.

8. Encoding na prática: Excel, utf-8-sig e cp1252

No módulo 05 você viu os dois encodings do dia a dia e o BOM. Agora, com `try/except`, a estratégia para ler um arquivo vindo do Excel fica direta:

```python
def ler_texto_excel(caminho):
    try:
        return caminho.read_text(encoding="utf-8-sig")   # UTF-8, com ou sem BOM
    except UnicodeDecodeError:
        return caminho.read_text(encoding="cp1252")      # o "ANSI" do Windows
```

Por que tentar UTF-8 primeiro? Porque ele é exigente: bytes cp1252 com acento quase nunca formam UTF-8 válido, então o erro aparece e o plano B entra. O contrário não funciona: cp1252 aceita quase qualquer byte e "lê" um arquivo UTF-8 sem erro nenhum, só que cheio de `Ã§`.

E para **gravar** um CSV que o Excel em português abre certo com duplo clique, use `encoding="utf-8-sig"`. O BOM avisa ao Excel que o arquivo é UTF-8. Sem ele, o Excel supõe cp1252 e mostra `LiberaÃ§Ã£o`.

```
Situação                                  Encoding
----------------------------------------  ------------------------------
----------
Ler arquivo seu (JSON, log, txt)          "utf-8"
Ler CSV/TXT vindo do Excel ou do ERP      tente "utf-8-sig"; se falhar,
"cp1252"
Gravar CSV para alguém abrir no Excel     "utf-8-sig"
Gravar JSON, log, txt                     "utf-8"
```

9. CSV no padrão do Excel brasileiro

CSV quer dizer "valores separados por vírgula". Mas no Brasil a vírgula é o separador decimal, então o Excel em português usa **ponto e vírgula** (`;`) para separar as colunas. Um CSV de medições exportado no laboratório fica assim:

```
lote;linha;espessura;observacao
L001;A;25,4;COF: liberado
L002;A;27,9;"Selagem; refazer"
```

Repare na última linha: a observação tem um `;` dentro, então o Excel a colocou entre aspas. É por isso que `split(";")` não serve para ler CSV:

```
>>> 'L002;A;27,9;"Selagem; refazer"'.split(";")
['L002', 'A', '27,9', '"Selagem', ' refazer"']
```

O módulo `csv` da biblioteca padrão entende aspas, quebras de linha dentro de células e outras regras do formato. Vamos criar o arquivo e ler com `csv.reader`:

```
>>> import csv
>>> conteudo = 'lote;linha;espessura;observacao\nL001;A;25,4;COF:
liberado\nL002;A;27,9;"Selagem; refazer"\n'
>>> Path("medicoes.csv").write_text(conteudo, encoding="utf-8-sig")
89
>>> with open("medicoes.csv", encoding="utf-8-sig", newline="") as
arquivo:
...     for linha in csv.reader(arquivo, delimiter=";"):
...         print(linha)
...
['lote', 'linha', 'espessura', 'observacao']
['L001', 'A', '25,4', 'COF: liberado']
['L002', 'A', '27,9', 'Selagem; refazer']
```

Dois detalhes que não podem faltar: `delimiter=";"` (o padrão do módulo é vírgula) e `newline=""` no `open` (a documentação do `csv` pede isso: o próprio módulo cuida das quebras de linha).

DictReader: cada linha vira um dicionário

```
>>> with open("medicoes.csv", encoding="utf-8-sig", newline="") as
arquivo:
...     linhas = list(csv.DictReader(arquivo, delimiter=";"))
...
>>> linhas[0]
{'lote': 'L001', 'linha': 'A', 'espessura': '25,4', 'observacao': 'COF:
```

```
liberado'}
>>> [float(l["espessura"].replace(",", ".")) for l in linhas]
[25.4, 27.9]
```

A primeira linha do arquivo vira as chaves. Tudo chega como **texto**: a conversão do número brasileiro é com você (módulo 05).

🔮 **Preveja antes de rodar:** o que acontece se você esquecer o `utf-8-sig` (usando `"utf-8"`) ou esquecer o `delimiter=";"`?

```
import csv
with open("medicoes.csv", encoding="utf-8", newline="") as arquivo:
    print(list(csv.DictReader(arquivo, delimiter=";"))[0]["lote"])
```

▼ **Resposta**

`KeyError: 'lote'`. Lido com `"utf-8"`, o BOM fica grudado no primeiro nome de coluna, que passa a ser `'\ufefflote'`. A chave `'lote'` não existe.

E sem o `delimiter=";"`, o `csv` procura vírgulas: a linha inteira do cabeçalho vira **uma** chave só, `'lote;linha;espessura;observacao'`, e o `25,4` é partido no meio. Os dois erros são clássicos com arquivos do Excel brasileiro.

Gravando com DictWriter

Para gravar, diga as colunas (`fieldnames`), escreva o cabeçalho e depois as linhas. Os números você formata com vírgula antes de gravar:

```
import csv

resultado = [
    {"lote": "L001", "espessura": 25.4, "status": "APROVADO"},
    {"lote": "L002", "espessura": 27.9, "status": "REPROVADO"},
]
with open("resultado.csv", "w", encoding="utf-8-sig", newline="") as
arquivo:
    escritor = csv.DictWriter(arquivo, fieldnames=["lote", "espessura",
"status"], delimiter=";")
    escritor.writeheader()
    for item in resultado:
        linha = dict(item)  # cópia, para não alterar a lista original
        linha["espessura"] = f"{item['espessura']:.2f}".replace(".", ",")
        escritor.writerow(linha)
```

```
print(open("resultado.csv", "rb").read())
```

Saída:

```
b'\xef\xbb\xbflote;espessura;status\r\nL001;25,40;APROVADO\r\nL002;27,90;
REPROVADO\r\n'
```

Os bytes mostram tudo o que importa: o BOM no começo, o `;`, a vírgula decimal e o fim de linha `\r\n` (padrão do `csv` e do Windows). Com duplo clique, o Excel abre em colunas e com os números à direita, prontos para somar.

> ⚠️ **Erro comum:** esquecer o `newline=""` ao gravar. No Windows, cada linha ganha um `\r` extra e o Excel mostra uma linha em branco entre cada registro.

Exemplo resolvido: média de espessura por linha de produção

Problema: ler o CSV de medições e gravar um resumo com a média de cada linha (A, B...).

Primeiro eu pensei na estrutura: um dicionário `{linha: [valores]}` (módulo 03). Leio com `DictReader`, converto o número, acumulo na lista da linha. No fim, calculo as médias e gravo com `DictWriter`.

```
import csv

conteudo = "lote;linha;espessura\nA-1;A;25,4\nA-2;A;26,0\nB-1;B;50,2\nB-
2;B;49,4\n"
with open("medicoes2.csv", "w", encoding="utf-8-sig") as arquivo:
    arquivo.write(conteudo)

por_linha = {}
with open("medicoes2.csv", encoding="utf-8-sig", newline="") as arquivo:
    for registro in csv.DictReader(arquivo, delimiter=";"):
        valor = float(registro["espessura"].replace(".", "").replace(",",
"."))
        por_linha.setdefault(registro["linha"], []).append(valor)

with open("medias.csv", "w", encoding="utf-8-sig", newline="") as
arquivo:
    escritor = csv.writer(arquivo, delimiter=";")
    escritor.writerow(["linha", "medicoes", "media"])
    for linha, valores in sorted(por_linha.items()):
        media = sum(valores) / len(valores)
        escritor.writerow([linha, len(valores), f"
```

```
{media:.2f}".replace(".", ",")])

with open("medias.csv", encoding="utf-8-sig") as arquivo:
    print(arquivo.read(), end="")
```

Saída:

```
linha;medicoes;media
A;2;25,70
B;2;49,80
```

`setdefault(chave, [])` devolve a lista da chave, criando uma vazia na primeira vez. É o jeito curto de "agrupar" valores num dicionário. E `csv.writer` (sem o `Dict`) grava listas em vez de dicionários.

▼ **Tente você: grave o arquivo `cof.csv` com as colunas lote e cof, para os lotes A-1 (0.215) e A-2 (0.3), com 3 casas e vírgula decimal, pronto para o Excel.**

```
import csv

dados = [("A-1", 0.215), ("A-2", 0.3)]
with open("cof.csv", "w", encoding="utf-8-sig", newline="") as
arquivo:
    escritor = csv.writer(arquivo, delimiter=";")
    escritor.writerow(["lote", "cof"])
    for lote, cof in dados:
        escritor.writerow([lote, f"{cof:.3f}".replace(".", ",")])
```

O arquivo fica com as linhas `lote;cof`, `A-1;0,215` e `A-2;0,300`.

CSV a partir de um texto já lido

Às vezes você já tem o conteúdo numa `str` (por exemplo, depois de decidir o encoding com `try/except`). O `csv.reader` aceita qualquer lista de linhas:

```
>>> texto = "lote;espessura\nA-1;25,4\n"
>>> list(csv.reader(texto.splitlines(), delimiter=";"))
[['lote', 'espessura'], ['A-1', '25,4']]
```

Só não funciona se alguma célula tiver quebra de linha dentro das aspas (raro em planilhas de medição).

10. JSON: configurações e regras

JSON é o formato de texto mais usado para trocar dados entre programas (APIs web, configurações, regras). Ele parece muito com dicionários e listas do Python:

```
{
  "linha": "A",
  "espessura": {"nominal": 25.0, "tolerancia_pct": 10},
  "cof": [0.15, 0.30],
  "liberacao": "COF: liberado",
  "ativo": true,
  "observacao": null
}
```

```
Python            JSON
---------------   ----------------------------
dict              objeto  {"chave": valor}
list, tuple       array   [1, 2, 3]
str               string  "texto" (só aspas DUPLAS)
int, float        number  25.4
True / False      true / false
None              null
```

O módulo `json` converte nos dois sentidos. `dumps` / `loads` trabalham com `str`; `dump` / `load` (sem o "s") trabalham direto com arquivos.

```
>>> import json
>>> regras = {"linha": "A", "cof": [0.15, 0.30], "liberacao": "COF:
liberado", "ativo": True}
>>> texto = json.dumps(regras, ensure_ascii=False)
>>> texto
'{"linha": "A", "cof": [0.15, 0.3], "liberacao": "COF: liberado",
"ativo": true}'
>>> json.loads(texto) == regras
True
>>> json.dumps({"obs": "Liberação"})
'{"obs": "Libera\\u00e7\\u00e3o"}'
>>> json.dumps({"obs": "Liberação"}, ensure_ascii=False)
'{"obs": "Liberação"}'
```

Por padrão, o `json` troca cada acento por um código (`\u00e7`). Funciona, mas fica ilegível para quem abre o arquivo no Bloco de Notas. Use `ensure_ascii=False` (e grave em UTF-8). Com `indent=2`, o arquivo fica indentado e fácil de editar à mão:

```
>>> with open("regras.json", "w", encoding="utf-8") as arquivo:
...     json.dump(regras, arquivo, ensure_ascii=False, indent=2)
```

```
...
>>> print(Path("regras.json").read_text(encoding="utf-8"))
{
  "linha": "A",
  "cof": [
    0.15,
    0.3
  ],
  "liberacao": "COF: liberado",
  "ativo": true
}
>>> with open("regras.json", encoding="utf-8") as arquivo:
...     lidas = json.load(arquivo)
...
>>> lidas["cof"][1], lidas["ativo"]
(0.3, True)
```

Como o JSON pode trazer qualquer coisa, confira o tipo antes de usar: `isinstance(dados, dict)` responde se `dados` é um dicionário (você verá `isinstance` a fundo no módulo 08).

🔮 **Preveja antes de rodar:**

```
import json
dados = {"faixa": (0.15, 0.30), "linhas": {"A", "B"}}
print(json.dumps(dados["faixa"]))
print(json.dumps(dados))
```

▼ **Resposta**

A primeira linha imprime `[0.15, 0.3]`: a tupla vira lista (JSON não tem tupla; ao ler de volta, você recebe uma lista). A segunda dá `TypeError: Object of type set is not JSON serializable`: conjuntos não existem em JSON. Converta antes: `sorted(dados["linhas"])`.

Quando o JSON está quebrado

Arquivo editado à mão quebra: aspas simples, vírgula esquecida. O `json` levanta `json.JSONDecodeError` (uma filha de `ValueError`) dizendo linha e coluna:

```
>>> json.loads("{'linha': 'A'}")
Traceback (most recent call last):
  ...
```

```
json.decoder.JSONDecodeError: Expecting property name enclosed in double
quotes: line 1 column 2 (char 1)
>>> try:
...     json.loads('{"casas": 3\n "pasta": "laudos"}')
... except json.JSONDecodeError as erro:
...     print("Erro na linha", erro.lineno, "->", erro.msg)
...
Erro na linha 2 -> Expecting ',' delimiter
```

Em programa de verdade, você captura esse erro e levanta um `ArquivoInvalido` com o nome do arquivo e a linha, encadeado com `from erro` (é o exercício 10).

11. Leitura e escrita segura

Gravar arquivo parece simples até dar errado no meio. Três situações reais:

1. **Abrir em `"w"` apaga o arquivo na hora.** Se o erro acontece depois do `open` e antes do fim da gravação, o arquivo antigo já era e o novo ficou pela metade.
2. **O Excel tranca o arquivo.** No Windows, se `resultado.csv` está aberto no Excel, gravar nele dá `PermissionError: [Errno 13] Permission denied: 'resultado.csv'`.
3. **O programa cai** (energia, Ctrl+C, erro) no meio da gravação de um JSON de regras. No dia seguinte, o robô não consegue nem ler a configuração.

A técnica profissional é a **gravação atômica**: grave num arquivo temporário na mesma pasta e, só quando tudo deu certo, troque pelo definitivo com `Path.replace`. A troca é uma operação única: quem abrir o arquivo vê o antigo inteiro ou o novo inteiro, nunca um pedaço.

```
from pathlib import Path


def gravar_texto_seguro(caminho, texto):
    temporario = caminho.with_name(caminho.name + ".tmp")
    temporario.write_text(texto, encoding="utf-8")   # se falhar, o
original está intacto
    temporario.replace(caminho)                      # troca de uma vez


destino = Path("relatorio.txt")
gravar_texto_seguro(destino, "versão 1\n")
gravar_texto_seguro(destino, "versão 2\n")
print(destino.read_text(encoding="utf-8"), end="")
print(sorted(p.name for p in Path(".").glob("relatorio*")))
```

Saída:

```
versão 2
['relatorio.txt']
```

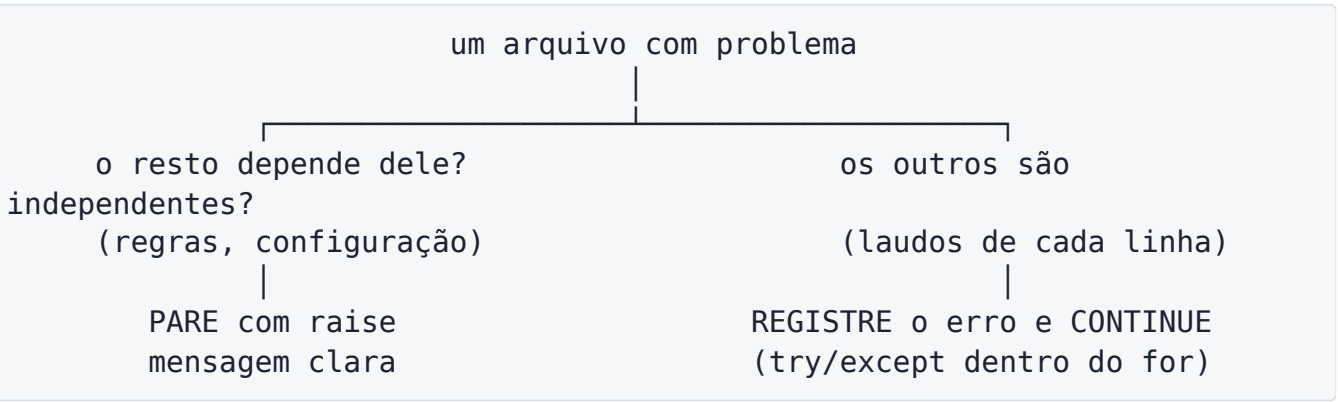
Outras proteções baratas:

- **Valide e prepare tudo antes de abrir para gravar.** Gere o texto, a lista de linhas, o JSON; só depois abra o arquivo.
- **Modo `"x"`** quando o arquivo não pode ser sobrescrito (por exemplo, um certificado já emitido): se ele existir, dá `FileExistsError` em vez de apagar.
- **Mensagem útil para o `PermissionError`**: capture e diga "feche o arquivo no Excel", em vez de mostrar o traceback para o usuário.

12. Validação defensiva

Dados ruins devem ser barrados **na entrada**, perto de onde chegam, e não descobertos três funções depois como um `TypeError` misterioso. Três regras práticas:

1. **Valide cedo, com cláusulas de guarda.** No começo da função, confira o que recebeu e saia com `raise` se não der para continuar (módulo 04: retorno antecipado).
2. **Mensagens com contexto.** Arquivo, linha, coluna e valor: `"linha 7: valor inválido em espessura: '20,1'"`.
3. **Decida: parar ou continuar?** Se o problema invalida tudo (regras de tolerância corrompidas), pare. Se afeta só uma parte (um entre quatro arquivos de laudo), registre o problema e siga com o resto.



Exemplo resolvido: processar a pasta sem parar no primeiro erro

Problema: somar as espessuras de todos os CSVs da pasta. Se um arquivo estiver ruim, ele fica de fora, o erro é anotado, e os outros são processados.

Primeiro eu separei o trabalho de **um** arquivo numa função que pode levantar erro. Depois, no laço da pasta, coloquei o `try/except` em volta da chamada: o erro de um arquivo fica preso naquela volta do `for`. E capturei só os tipos que eu sei explicar (`KeyError` para coluna que falta, `ValueError` para número inválido).

```
import csv
from pathlib import Path


def somar_espessuras(caminho):
    with open(caminho, encoding="utf-8-sig", newline="") as arquivo:
        return sum(
            float(linha["espessura"].replace(",", "."))
            for linha in csv.DictReader(arquivo, delimiter=";")
        )


pasta = Path("lote_do_dia")
pasta.mkdir(exist_ok=True)
(pasta / "a.csv").write_text("lote;espessura\nA-1;25,4\nA-2;26,0\n",
encoding="utf-8")
(pasta / "b.csv").write_text("lote;espessura\nB-1;abc\n", encoding="utf-
8")
(pasta / "c.csv").write_text("lote;gramatura\nC-1;18,4\n", encoding="utf-
8")

erros = {}
for caminho in sorted(pasta.glob("*.csv")):
    try:
        total = somar_espessuras(caminho)
    except KeyError as erro:
        erros[caminho.name] = f"falta a coluna {erro}"
    except ValueError as erro:
        erros[caminho.name] = f"valor inválido ({erro})"
    else:
        print(f"{caminho.name}: soma {total:.1f}")
print("Com problema:", erros)
```

Saída:

```
a.csv: soma 51.4
Com problema: {'b.csv': "valor inválido (could not convert string to
float: 'abc')", 'c.csv': "falta a coluna 'espessura'"}
```

O exercício 14 usa essa mesma ideia, com um detalhe a mais: o arquivo com erro precisa ficar de fora **inteiro**, mesmo que as primeiras linhas estivessem certas.

Lendo mensagens de erro deste módulo

- **`FileNotFoundError: [Errno 2] No such file or directory: 'dados/regras.json'`**: o caminho não existe **a partir da pasta onde você rodou o comando**. Confira com `Path("dados/regras.json").resolve()` onde o Python está procurando. Solução definitiva: `Path(__file__).resolve().parent` (seção 7).
- **`PermissionError: [Errno 13] Permission denied: 'resultado.csv'`**: no Windows, quase sempre o arquivo está aberto no Excel. Feche e rode de novo. Também acontece ao tentar abrir uma **pasta** como se fosse arquivo.
- **`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe7 in position 31`**: arquivo cp1252 lido como UTF-8. Use a estratégia da seção 8.
- **`KeyError: 'lote'`** ao ler um CSV: confira o BOM (use `utf-8-sig`), o `delimiter=";"` e espaços no nome da coluna (`" lote"`). Imprima `list(linha.keys())` para ver as chaves de verdade.
- **`json.decoder.JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)`**: JSON com aspas simples ou com vírgula sobrando depois do último item. JSON só aceita aspas duplas e não aceita vírgula no fim.
- **`TypeError: Object of type set is not JSON serializable`**: você tentou gravar algo que o JSON não conhece (set, Path, objetos). Converta antes (`sorted(conjunto)`, `str(caminho)`).
- **`ValueError: I/O operation on closed file.`**: usou o arquivo **depois** do bloco `with`, quando ele já foi fechado. Faça a leitura dentro do bloco.
- **`io.UnsupportedOperation: not writable`**: abriu em modo leitura (o padrão) e tentou gravar. Passe `"w"` ou `"a"`.
- **`FileExistsError: [Errno 17] File exists: 'certificado_L001.pdf'`**: o modo `"x"` (ou `mkdir` sem `exist_ok=True`) encontrou o arquivo ou a pasta já criados.
- **Traceback com "During handling of the above exception, another exception occurred"**: um erro aconteceu **dentro** do seu `except`. Confira o código do tratamento. (Se a frase for "was the direct cause of", foi um `raise ... from` proposital.)

Boas práticas

- **Capture só o que sabe tratar**, com o tipo exato. Nada de `except:` sozinho.
- **`try` curto**: dentro dele, só as linhas que podem levantar o erro tratado.
- **Erros do domínio com nome do domínio**: `ArquivoInvalido`, `ValorForaDaTolerancia`, numa família com uma mãe (`ErroDeQualidade`).
- **Mensagem de erro com contexto**: arquivo, linha, coluna e valor.
- **`raise ... from erro`** ao trocar um erro técnico por um erro com mais contexto.
- **Sempre `with open(...)`** e **sempre `encoding=`** em arquivos de texto.

- `pathlib` **em vez de juntar texto** para montar caminhos; `Path(__file__)` para achar os arquivos do projeto.
- **CSV com o módulo** `csv`, nunca com `split(";")`; para o Excel brasileiro: `delimiter=";"`, vírgula decimal, `utf-8-sig` e `newline=""`.
- **JSON legível**: `ensure_ascii=False` e `indent=2`.
- **Grave de forma atômica** arquivos importantes (temporário + `replace`).
- **Em lote, um arquivo ruim não derruba os outros**: `try/except` dentro do `for`, erros registrados num dicionário ou num log.

No mundo real

Fim do turno na linha A. O analista exporta as medições do Excel e precisa saber, antes de liberar a expedição, quais lotes estão dentro da especificação. As regras de tolerância ficam num JSON (`dados/regras_tolerancia.json`), uma entrada por linha de produção: espessura e gramatura com tolerância sobre o nominal, COF com mínimo e máximo fixos.

```
"A": {
  "espessura": {"nominal": 25.0, "tolerancia_pct": 10},
  "gramatura": {"nominal": 23.0, "tolerancia_pct": 5},
  "cof": {"min": 0.15, "max": 0.3}
}
```

O programa abaixo junta o módulo inteiro: lê o CSV com o encoding certo, valida o cabeçalho e os números (com mensagens que dizem a linha), carrega as regras do JSON, confere cada lote e grava o resultado para o Excel.

```
import csv
import json
from pathlib import Path

DADOS = Path(__file__).resolve().parent.parent / "dados"
SAIDA = Path(__file__).resolve().parent / "saida"
ENSAIOS = ("espessura", "gramatura", "cof")


class ErroDeQualidade(Exception):
    """Base dos erros do sistema de qualidade."""


class ArquivoInvalido(ErroDeQualidade):
    """O arquivo existe, mas não está no formato esperado."""
```

```
def numero_br(texto):
    return float(texto.strip().replace(".", "").replace(",", "."))


def ler_texto_excel(caminho):
    try:
        return caminho.read_text(encoding="utf-8-sig")
    except UnicodeDecodeError:
        return caminho.read_text(encoding="cp1252")


def carregar_regras(caminho):
    try:
        with open(caminho, encoding="utf-8") as arquivo:
            return json.load(arquivo)
    except json.JSONDecodeError as erro:
        raise ArquivoInvalido(f"{caminho.name}: JSON inválido (linha
{erro.lineno})") from erro


def faixa(regra):
    """Converte a regra do JSON em (mínimo, máximo): nominal ± % ou
mín/máx fixos."""
    if "nominal" in regra:
        variacao = regra["nominal"] * regra["tolerancia_pct"] / 100
        return regra["nominal"] - variacao, regra["nominal"] + variacao
    return regra["min"], regra["max"]


def ler_medicoes(caminho):
    linhas = list(csv.reader(ler_texto_excel(caminho).splitlines(),
delimiter=";"))
    if not linhas:
        raise ArquivoInvalido(f"{caminho.name}: arquivo vazio")
    cabecalho = [nome.strip().lower() for nome in linhas[0]]
    faltando = {"lote", "linha", *ENSAIOS} - set(cabecalho)
    if faltando:
        raise ArquivoInvalido(f"{caminho.name}: faltam as colunas
{sorted(faltando)}")
    medicoes = []
    for numero, valores in enumerate(linhas[1:], start=2):
        registro = dict(zip(cabecalho, valores, strict=False))
        try:
            for ensaio in ENSAIOS:
                registro[ensaio] = numero_br(registro[ensaio])
        except ValueError as erro:
            raise ArquivoInvalido(f"{caminho.name}, linha {numero}:
{erro}") from erro
        medicoes.append(registro)
    return medicoes
```

```
def conferir(medicao, regras_da_linha):
    problemas = []
    for ensaio in ENSAIOS:
        minimo, maximo = faixa(regras_da_linha[ensaio])
        if not minimo <= medicao[ensaio] <= maximo:
            problemas.append(
                f"{ensaio} {medicao[ensaio]} fora de [{minimo:.2f},
{maximo:.2f}]"
            )
    return problemas


def main():
    regras = carregar_regras(DADOS / "regras_tolerancia.json")
    medicoes = ler_medicoes(DADOS / "medicoes_linha_a.csv")
    SAIDA.mkdir(exist_ok=True)
    destino = SAIDA / "conferencia_linha_a.csv"
    with open(destino, "w", encoding="utf-8-sig", newline="") as arquivo:
        escritor = csv.writer(arquivo, delimiter=";")
        escritor.writerow(["lote", "status", "motivo"])
        for medicao in medicoes:
            problemas = conferir(medicao, regras[medicao["linha"]])
            status = "REPROVADO" if problemas else "APROVADO"
            escritor.writerow([medicao["lote"], status, ";
".join(problemas)])
            print(f"{medicao['lote']}: {status:<9} {';
'.join(problemas)}".rstrip())
    print(f"Resultado gravado em {destino.name}")


# Roda main() só quando o arquivo é executado diretamente (detalhes no
módulo 07).
if __name__ == "__main__":
    main()
```

Resultado ao rodar `python 06-arquivos-e-erros/exemplos/09_conferencia_da_linha.py`:

```
A-0153: APROVADO
A-0154: REPROVADO espessura 27.9 fora de [22.50, 27.50]
A-0155: REPROVADO gramatura 21.2 fora de [21.85, 24.15]; cof 0.33 fora de
[0.15, 0.30]
Resultado gravado em conferencia_linha_a.csv
```

Repare nas decisões: as regras corrompidas **param** o programa (sem regra não há conferência), com `ArquivoInvalido` dizendo a linha do JSON; o CSV é validado na entrada; a função `faixa` esconde a diferença entre "nominal ± %" e "mín/máx"; e o resultado sai num CSV que o pessoal da qualidade abre com duplo clique. O desafio deste módulo leva isso para a pasta inteira, com as quatro linhas de produção.

Resumo

Tarefa	Como
Tratar um erro	`try: ... except ValueError as erro: ...`
Vários tipos	`except (KeyError, ValueError):` ou um `except` para cada
Só se não deu erro / sempre	`else:` / `finally:`
Levantar	`raise ValueError("mensagem com contexto")`
Encadear	`raise ArquivoInvalido("...") from erro`
Relançar o mesmo	`raise` sozinho, dentro do `except`
Exceção própria	`class ArquivoInvalido(ErroDeQualidade): """Docstring."""`
Ler texto	`with open(c, encoding="utf-8") as f: texto = f.read()`
Linha a linha	`for linha in f: linha.strip()`
Gravar / acrescentar	`open(c, "w", ...)` (apaga) / `open(c, "a", ...)`
Caminhos	`Path("dados") / "regras.json"`, `.name`, `.stem`, `.suffix`, `.parent`
Pastas	`mkdir(parents=True, exist_ok=True)`, `iterdir()`, `glob("*.csv")`
Atalhos	`read_text`, `write_text`, `read_bytes` (sempre com `encoding=` no texto)
Pasta do script	`Path(__file__).resolve().parent`
Arquivo do Excel	tente `"utf-8-sig"`; `except UnicodeDecodeError`: `"cp1252"`
Ler CSV BR	`csv.DictReader(f, delimiter=";")` com `open(..., newline="")`
Gravar CSV BR	`csv.DictWriter(f, fieldnames=..., delimiter=";")`, `"utf-8-sig"`, `newline=""`
JSON	`json.load(f)`, `json.dump(d, f, ensure_ascii=False, indent=2)`
JSON quebrado	`except json.JSONDecodeError as erro: erro.lineno`
Gravação segura	grava `nome.tmp` e depois `tmp.replace(destino)`

Você domina este módulo se...

- [ ] Leio um traceback de baixo para cima e digo em que linha do **meu** código está o problema.
- [ ] Explico sem consultar a diferença entre `except`, `else` e `finally`.
- [ ] Sei por que `except:` sozinho é perigoso e capturo só o tipo certo.
- [ ] Crio uma família de exceções próprias e uso `raise ... from erro`.
- [ ] Escolho o modo de abertura certo (`"r"`, `"w"`, `"a"`, `"x"`) e sei qual deles apaga o arquivo.
- [ ] Leio e gravo CSV no padrão do Excel brasileiro sem `KeyError` por causa do BOM e sem linhas em branco.
- [ ] Leio e gravo JSON com acentos legíveis e trato um JSON quebrado com mensagem clara.
- [ ] Processo uma pasta inteira sem que um arquivo ruim derrube os outros.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_ler_numero`	★☆☆	`try/except ValueError`, valor padrão
02	`ex02_conferir_tolerancia`	★☆☆	`raise` com exceção própria e `ValueError`
03	`ex03_ler_linhas`	★☆☆	`with open`, ler linha a linha, `encoding`
04	`ex04_registrar_evento`	★☆☆	modo `"a"`, `mkdir(parents=True)`
05	`ex05_listar_planilhas`	★☆☆	`pathlib`: `iterdir`, `suffix`, `is_file`
06	`ex06_ler_texto_excel`	★★☆	BOM, `UnicodeDecodeError`, fallback cp1252
07	`ex07_ler_csv_br`	★★☆	`csv.DictReader` com `;` e vírgula decimal
08	`ex08_escrever_csv_br`	★★☆	`csv.DictWriter`, `utf-8-sig`, `newline=""`
09	`ex09_hierarquia_excecoes`	★★☆	criar exceções, capturar pela "mãe"
10	`ex10_carregar_config`	★★☆	JSON, `ArquivoInvalido`, `raise ... from`
11	`ex11_depurar_media`	★★☆	leitura de código: prever e consertar
12	`ex12_salvar_json_seguro`	★★★	gravação atômica, limpeza e `raise`
13	`ex13_revisao_ler_medicoes`	★★★	revisão: CSV validado + acentos e números BR (módulos 02, 03 e 05)
14	`ex14_revisao_conferir_pasta`	★★★	revisão: lote de arquivos + funções e contadores (módulos 02 a 05)

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError` e rode, na pasta `curso-python`:

```
python curso.py testar 06 ex01
```

Os testes criam arquivos numa pasta temporária (`tmp_path` do pytest): nada é gravado na sua pasta do curso. Para testar o módulo inteiro: `python curso.py testar 06`.

Desafio do módulo

Conferência de laudos em lote. Um programa que lê os CSVs das quatro linhas de produção em `dados/laudos/` (em encodings diferentes, um com número digitado errado e outro sem uma coluna), aplica as regras de tolerância do JSON, respeita a observação de liberação ("COF: liberado") e gera um CSV para o Excel, um resumo em JSON e um log de erros, sem parar no primeiro problema. Enunciado completo em desafio/README.md.

Quiz

Teste o que ficou: quiz.md.

Para ir além

- Erros e exceções (tutorial oficial): https://docs.python.org/pt-br/3/tutorial/errors.html
- Hierarquia das exceções nativas: https://docs.python.org/pt-br/3/library/exceptions.html#exception-hierarchy
- Leitura e escrita de arquivos: https://docs.python.org/pt-br/3/tutorial/inputoutput.html#reading-and-writing-files
- `pathlib`: https://docs.python.org/pt-br/3/library/pathlib.html
- `csv`: https://docs.python.org/pt-br/3/library/csv.html
- `json`: https://docs.python.org/pt-br/3/library/json.html
- Leitura extra: capítulos 9 e 10 (arquivos e pastas) do livro gratuito *Automate the Boring Stuff with Python*: https://automatetheboringstuff.com/2e/chapter9/
- Leitura extra: o verbete EAFP (o estilo "tentar e tratar") no glossário oficial: https://docs.python.org/pt-br/3/glossary.html#term-EAFP

Enunciados dos exercícios

Exercício 06.01 — Converter sem quebrar

Arquivo: `exercicios/ex01_ler_numero.py`

```
Nível: ★☆☆

Contexto: Uma coluna da planilha deveria ter só números, mas aparecem
células
vazias, "#N/D" do Excel e "25,4 kg" digitado à mão. Um único valor ruim
não
pode derrubar o programa inteiro.

Tarefa: implemente `ler_numero(texto, padrao=None)`:
- converta o número no formato brasileiro ("1.250,5", "25,4", "-0,5",
"980")
  para float;
- se a conversão der ValueError, devolva `padrao` em vez de deixar o erro
  estourar;
- use try/except (não tente validar o texto com if).

Exemplos:
    >>> ler_numero("1.250,5")
    1250.5
    >>> print(ler_numero("#N/D"))
    None
    >>> ler_numero("", padrao=0.0)
    0.0

Dica: a conversão é a do módulo 05. Coloque dentro do try só a linha que
pode
dar erro.

Teste com:  python curso.py testar 06 ex01
```

Exercício 06.02 — Levantar erro fora da tolerância

Arquivo: `exercicios/ex02_conferir_tolerancia.py`

```
Nível: ★☆☆

Contexto: Antes de emitir um certificado, o sistema confere cada medição.
Se
um valor estiver fora da faixa, o certificado NÃO pode sair: em vez de
devolver
um aviso que alguém pode ignorar, a função levanta um erro.
```

```
A exceção `ValorForaDaTolerancia` já está criada no arquivo (seção 5 da
aula).

Tarefa: implemente `conferir_tolerancia(valor, minimo, maximo)`:
- se `minimo` for maior que `maximo`, levante
  ValueError("faixa inválida: mínimo maior que máximo");
- se o valor estiver fora da faixa (os limites contam como dentro),
levante
  ValorForaDaTolerancia com a mensagem
  f"{valor} fora da faixa [{minimo}, {maximo}]";
- se estiver tudo certo, devolva o próprio valor.

Exemplos:
    >>> conferir_tolerancia(25.4, 22.5, 27.5)
    25.4
    >>> conferir_tolerancia(28.0, 22.5, 27.5)
    Traceback (most recent call last):
      ...
    ValorForaDaTolerancia: 28.0 fora da faixa [22.5, 27.5]

Dica: confira primeiro a faixa, depois o valor. Cada problema, um
`raise`.

Teste com:  python curso.py testar 06 ex02
```

Exercício 06.03 — Ler um arquivo de texto

Arquivo: `exercicios/ex03_ler_linhas.py`

```
Nível: ★☆☆

Contexto: A lista de OPs a liberar fica num .txt, uma por linha. O
pessoal
deixa linhas em branco e comentários começando com "#".

Tarefa: implemente `ler_linhas(caminho)`:
- abra o arquivo em UTF-8 usando `with`;
- devolva a lista das linhas sem espaços nas pontas (e sem a quebra de
linha);
- ignore as linhas vazias e as que, depois do strip, começam com "#";
- se o arquivo não existir, deixe o FileNotFoundError acontecer.

Exemplos (arquivo com "# OPs de hoje", "OP 123456", "", "  OP 123457  "):
    >>> ler_linhas(Path("ops.txt"))
    ['OP 123456', 'OP 123457']

Dica: percorrer o arquivo com `for linha in arquivo` entrega uma linha
por vez,
```

```
já com o "\n" no fim.

Teste com:  python curso.py testar 06 ex03
```

Exercício 06.04 — Escrever um registro (log)

Arquivo: `exercicios/ex04_registrar_evento.py`

```
Nível: ★☆☆

Contexto: O robô dos certificados precisa deixar um rastro do que fez: um
arquivo de texto com uma linha por evento. Se o robô parar no meio, o
registro
mostra até onde ele chegou.

Tarefa: implemente `registrar_evento(caminho, momento, mensagem)`:
- acrescente ao FIM do arquivo a linha f"{momento} | {mensagem}", seguida
de
  uma quebra de linha;
- se o arquivo não existir, ele deve ser criado; se as pastas do caminho
não
  existirem, crie-as também;
- nunca apague o que já estava no arquivo;
- use UTF-8.
A função não devolve nada.

Exemplos:
    >>> log = Path("logs") / "robo.log"
    >>> registrar_evento(log, "2025-10-28 08:00", "Início")
    >>> registrar_evento(log, "2025-10-28 08:01", "Certificado da OP
123456 emitido")
    >>> print(log.read_text(encoding="utf-8"), end="")
    2025-10-28 08:00 | Início
    2025-10-28 08:01 | Certificado da OP 123456 emitido

Dica: modo "a" (append). Para as pastas: `caminho.parent.mkdir(...)` com
os
argumentos que evitam erro quando a pasta já existe.

Teste com:  python curso.py testar 06 ex04
```

Exercício 06.05 — Listar as planilhas de uma pasta

Arquivo: `exercicios/ex05_listar_planilhas.py`

```
Nível: ★☆☆
```

```
Contexto: Cada linha de produção salva seus laudos numa pasta
compartilhada.
Lá dentro tem de tudo: CSV, Excel, PDF, subpastas e os arquivos
temporários
que o Excel cria enquanto a planilha está aberta (começam com "~$").

Tarefa: implemente `listar_planilhas(pasta)`, que devolve a lista com os
NOMES
(não os caminhos) dos arquivos da pasta que são planilhas:
- extensão .csv, .xlsx ou .xlsm, sem diferenciar maiúsculas ("LAUDO.CSV"
vale);
- só arquivos (uma subpasta chamada "antigos.csv" não conta);
- ignore os temporários do Excel (nome começando com "~$");
- não entre nas subpastas;
- devolva em ordem alfabética (sorted);
- se a pasta não existir, levante FileNotFoundError com a mensagem
  f"Pasta não encontrada: {pasta}".

Exemplos (pasta com linha_A.csv, linha_B.XLSX, ~$linha_B.xlsx, foto.pdf):
    >>> listar_planilhas(Path("laudos"))
    ['linha_A.csv', 'linha_B.XLSX']

Dica: `pasta.iterdir()` percorre o conteúdo da pasta; cada item é um Path
com
`.name`, `.suffix` e `.is_file()`.

Teste com:  python curso.py testar 06 ex05
```

Exercício 06.06 — Ler arquivo vindo do Excel

Arquivo: `exercicios/ex06_ler_texto_excel.py`

```
Nível: ★★☆

Contexto: Os CSVs chegam de computadores diferentes. Uns foram salvos
como
"CSV UTF-8" (UTF-8 com BOM), outros como "CSV (separado por vírgulas)"
(cp1252),
e alguns vieram de sistemas que gravam UTF-8 sem BOM. O programa precisa
ler
todos e dizer como cada um foi lido.

Tarefa: implemente `ler_texto_excel(caminho)`, que devolve uma tupla
(texto, encoding):
- se o arquivo começa com o BOM (os bytes b"\xef\xbb\xbf"), leia como
  "utf-8-sig" e devolva o encoding "utf-8-sig";
- senão, tente ler como "utf-8" e devolva "utf-8";
- se a leitura em UTF-8 der UnicodeDecodeError, leia como "cp1252" e
devolva
```

```
  "cp1252";
- o texto devolvido nunca contém o BOM;
- se o arquivo não existir, deixe o FileNotFoundError acontecer.

Exemplos:
    >>> Path("a.csv").write_bytes("lote;observação".encode("cp1252"))
    15
    >>> ler_texto_excel(Path("a.csv"))
    ('lote;observação', 'cp1252')

Dica: `caminho.read_bytes()` lê os bytes uma vez só; depois é só usar
`dados.startswith(...)` e `dados.decode(...)` dentro de um try/except.

Teste com:  python curso.py testar 06 ex06
```

Exercício 06.07 — Ler CSV do Excel brasileiro

Arquivo: `exercicios/ex07_ler_csv_br.py`

```
Nível: ★★☆

Contexto: A planilha de medições é exportada pelo Excel como "CSV UTF-8":
separador ";", vírgula decimal e BOM no começo. Você quer cada linha como
um
dicionário, com os números já em float.

    lote;linha;espessura;observacao
    A-0153;A;25,4;COF: liberado
    A-0154;A;;"Selagem; refazer"

Tarefa: implemente `ler_csv_br(caminho, colunas_numericas)`:
- abra com encoding "utf-8-sig" e newline="" e use csv.DictReader com
  delimiter=";";
- tire os espaços das pontas dos nomes das colunas e dos valores;
- nas colunas listadas em `colunas_numericas`, converta o número
brasileiro
  para float; célula vazia vira None;
- as outras colunas continuam como texto;
- devolva a lista de dicionários, um por linha de dados.

Exemplos (com o arquivo acima):
    >>> linhas = ler_csv_br(Path("medicoes.csv"), ["espessura"])
    >>> linhas[0]
    {'lote': 'A-0153', 'linha': 'A', 'espessura': 25.4, 'observacao':
'COF: liberado'}
    >>> linhas[1]["espessura"], linhas[1]["observacao"]
    (None, 'Selagem; refazer')

Dica: monte um dicionário novo para cada linha, percorrendo
```

```
`linha.items()`.
Repare que o csv já cuida do ";" que está entre aspas.

Teste com:  python curso.py testar 06 ex07
```

Exercício 06.08 — Gerar CSV que abre certo no Excel

Arquivo: `exercicios/ex08_escrever_csv_br.py`

```
Nível: ★★☆

Contexto: O resultado da conferência vai para o pessoal da qualidade, que
abre
tudo no Excel com um duplo clique. Para as colunas e os acentos
aparecerem
certos no Excel em português, o arquivo precisa de ";", vírgula decimal e
UTF-8 com BOM.

Tarefa: implemente `escrever_csv_br(caminho, linhas, casas=2)`:
- `linhas` é uma lista de dicionários; o cabeçalho são as chaves do
PRIMEIRO
  dicionário, na mesma ordem;
- grave com csv.DictWriter, delimiter=";", encoding "utf-8-sig" e
newline="";
- valores float viram texto com vírgula decimal e `casas` casas, SEM
separador
  de milhar (1250.5 vira "1250,50");
- None vira célula vazia; os outros valores (str, int) vão como estão;
- devolva quantas linhas de dados foram gravadas;
- se `linhas` estiver vazia, levante ValueError("nada para gravar") SEM
criar
  o arquivo.

Exemplos:
    >>> escrever_csv_br(Path("saida.csv"), [{"lote": "A-0153",
"espessura": 25.4}])
    1
    >>> Path("saida.csv").read_bytes()
    b'\xef\xbb\xbflote;espessura\r\nA-0153;25,40\r\n'

Repare nos 3 bytes do BOM no começo e no "\r\n" (o fim de linha padrão do
csv e do Windows).

Dica: prepare uma CÓPIA de cada dicionário com os valores já formatados e
só
depois chame `writerow`.

Teste com:  python curso.py testar 06 ex08
```

Exercício 06.09 — Suas próprias exceções

Arquivo: `exercicios/ex09_hierarquia_excecoes.py`

```
Nível: ★★☆

Contexto: O sistema de qualidade tem problemas de tipos diferentes: a OP
não
tem especificação cadastrada, ou a medição saiu da faixa. Quem chama
precisa
poder tratar cada um do seu jeito, ou todos de uma vez.

Tarefa 1: crie TRÊS classes de exceção (seção 5 da aula):
- ErroDeQualidade, filha de Exception (a "mãe" de todas);
- EspecificacaoAusente, filha de ErroDeQualidade;
- ValorForaDaTolerancia, filha de ErroDeQualidade.

Tarefa 2: implemente `verificar_medicao(op, ensaio, valor,
especificacoes)`.
`especificacoes` tem o formato {"123456": {"espessura": (22.5, 27.5)}}.
- OP fora de `especificacoes`: levante
  EspecificacaoAusente(f"OP {op} sem especificação");
- ensaio fora da especificação da OP: levante
  EspecificacaoAusente(f"OP {op} sem limite para {ensaio}");
- valor fora da faixa (limites contam como dentro): levante
  ValorForaDaTolerancia(f"OP {op}: {ensaio} = {valor} fora de [{mín},
{máx}]");
- tudo certo: devolva "OK".

Tarefa 3: implemente `resumir_conferencia(medicoes, especificacoes)`.
`medicoes` é uma lista de tuplas (op, ensaio, valor). Confira cada uma
com
`verificar_medicao` e devolva {"aprovadas": N, "problemas": [mensagens]},
em que
cada mensagem é str(erro). Capture SÓ ErroDeQualidade: qualquer outro
erro
(um bug) deve continuar subindo.

Exemplos:
    >>> specs = {"123456": {"espessura": (22.5, 27.5)}}
    >>> verificar_medicao("123456", "espessura", 25.0, specs)
    'OK'
    >>> resumir_conferencia([("123456", "espessura", 25.0), ("999999",
"cof", 0.2)], specs)
    {'aprovadas': 1, 'problemas': ['OP 999999 sem especificação']}

Dica: uma classe de exceção pode ter só a docstring no corpo. E um único
`except ErroDeQualidade as erro:` captura as duas filhas.
```

```
Teste com:  python curso.py testar 06 ex09
```

Exercício 06.10 — Configuração em JSON

Arquivo: `exercicios/ex10_carregar_config.py`

```
Nível: ★★☆

Contexto: As configurações do robô (pasta dos laudos, casas decimais,
texto de
liberação) ficam num arquivo JSON que o próprio usuário edita no Bloco de
Notas. Às vezes o arquivo nem existe; às vezes alguém esquece uma
vírgula.

A exceção `ArquivoInvalido` já está criada no arquivo.

Tarefa 1: implemente `carregar_config(caminho, padrao)`:
- se o arquivo não existir, devolva uma CÓPIA de `padrao`;
- se existir, leia o JSON (UTF-8);
- JSON com erro de sintaxe: levante
  ArquivoInvalido(f"{caminho.name}: JSON inválido (linha
{erro.lineno})"),
  encadeado com `from erro` (o json.JSONDecodeError original);
- JSON que não é um objeto (por exemplo, uma lista): levante
  ArquivoInvalido(f"{caminho.name}: esperado um objeto JSON");
- devolva um dicionário NOVO com as chaves de `padrao` atualizadas pelas
do
  arquivo (o arquivo vence);
- nunca altere o dicionário `padrao` recebido.

Tarefa 2: implemente `salvar_config(caminho, config)`: grave o dicionário
em
JSON, UTF-8, com indent=2 e ensure_ascii=False (acentos legíveis),
criando as
pastas que faltarem.

Exemplos:
    >>> padrao = {"casas": 2, "liberacao": "COF: liberado"}
    >>> carregar_config(Path("nao_existe.json"), padrao)
    {'casas': 2, 'liberacao': 'COF: liberado'}
    >>> salvar_config(Path("config.json"), {"casas": 3})
    >>> carregar_config(Path("config.json"), padrao)
    {'casas': 3, 'liberacao': 'COF: liberado'}

Dica: `json.JSONDecodeError` tem o atributo `lineno`. Para juntar dois
dicionários num novo: `{**padrao, **dados}` ou `dict(padrao)` + `update`.
```

```
Teste com:  python curso.py testar 06 ex10
```

Exercício 06.11 — Depure a média que mente

Arquivo: `exercicios/ex11_depurar_media.py`

```
Nível: ★★☆

Contexto: Esta função calcula a média das espessuras gravadas num .txt
(um
valor por linha). Ela "nunca dá erro"... e por isso ninguém percebeu que
ela
devolve 0 em situações bem diferentes.

    def media_do_arquivo_original(caminho):
        try:
            arquivo = open(caminho)
            valores = [float(linha) for linha in arquivo]
            return sum(valores) / len(valores)
        except:
            return 0

Tarefa, parte 1 (LEIA antes de rodar!): preveja o que a função original
devolve em cada caso e complete `previsoes()`, que devolve uma tupla com
4
respostas (números):
  a) arquivo com as linhas "25.0" e "27.0";
  b) arquivo com as linhas "25,4" e "26,0" (vírgula decimal);
  c) arquivo vazio;
  d) arquivo que não existe.
Depois copie a função para um script, rode os 4 casos e confira.

Tarefa, parte 2: escreva `media_do_arquivo(caminho)`, a versão corrigida:
- abra com `with` e encoding UTF-8;
- ignore as linhas em branco;
- aceite números no formato brasileiro ("25,4", "1.250,5"), com a
conversão
  do módulo 05;
- valor inválido: levante ValueError(f"linha {n}: valor inválido
{texto!r}"),
  em que n é o número da linha no arquivo (a primeira é 1) e texto é o
  conteúdo da linha sem espaços nas pontas; encadeie com `from erro`;
- arquivo sem nenhum valor: levante ValueError("arquivo sem valores");
- arquivo inexistente: NÃO trate; deixe o FileNotFoundError aparecer.

Exemplos (arquivo com "25,4", "", "26,0"):
    >>> media_do_arquivo(Path("espessuras.txt"))
    25.7
```

```
Dica: os defeitos da original são: except "pega-tudo", arquivo nunca
fechado,
sem encoding, sem tratar linha em branco nem vírgula decimal, e o mesmo 0
para
problemas diferentes. Use `enumerate(arquivo, start=1)` para numerar as
linhas.

Teste com:  python curso.py testar 06 ex11
```

Exercício 06.12 — Gravação segura de JSON

Arquivo: `exercicios/ex12_salvar_json_seguro.py`

```
Nível: ★★★

Contexto: O arquivo regras.json guarda as tolerâncias de todas as linhas.
Se o
programa der erro NO MEIO da gravação (ou a energia cair), o arquivo fica
pela
metade e o robô do dia seguinte não funciona. A técnica profissional:
gravar
num arquivo temporário e só no fim trocar pelo definitivo.

Tarefa: implemente `salvar_json_seguro(caminho, dados)`:
1. gere o JSON com indent=2 e ensure_ascii=False e grave-o, em UTF-8, num
   arquivo temporário na MESMA pasta, com o nome do destino + ".tmp"
   (para "regras.json", o temporário é "regras.json.tmp");
2. só depois que tudo deu certo, troque o temporário pelo destino com
   `temporario.replace(caminho)`;
3. se der erro ao gerar o JSON (por exemplo, TypeError porque `dados` tem
um
   set), apague o temporário (se ele existir) e deixe o erro subir. O
arquivo
   de destino antigo tem que continuar exatamente como estava.

Exemplos:
    >>> salvar_json_seguro(Path("regras.json"), {"linha": "A", "cof":
[0.15, 0.3]})
    >>> salvar_json_seguro(Path("regras.json"), {"linhas": {"A", "B"}})
    Traceback (most recent call last):
      ...
    TypeError: Object of type set is not JSON serializable
    >>> Path("regras.json").read_text(encoding="utf-8")   # continua o
antigo
    '{\n  "linha": "A",\n  "cof": [\n    0.15,\n    0.3\n  ]\n}'

Dica: `caminho.with_name(caminho.name + ".tmp")` monta o nome do
temporário.
```

```
Use try/except com `raise` sozinho para relançar o mesmo erro depois de
limpar.

Teste com:  python curso.py testar 06 ex12
```

Exercício 06.13 — Revisão: leitura validada de medições

Arquivo: `exercicios/ex13_revisao_ler_medicoes.py`

```
Nível: ★★★

Contexto: Este é o leitor "de verdade" da planilha de medições exportada
em
CSV. O arquivo pode vir em UTF-8 (com ou sem BOM) ou cp1252, os nomes das
colunas vêm escritos de vários jeitos ("Lote", " LINHA ", "Espessura") e,
de
vez em quando, alguém digita "2O,1" (com a letra O) no lugar de "20,1". O
programa precisa recusar o arquivo com uma mensagem que diga ONDE está o
erro.

Este exercício mistura arquivos e exceções (módulo 06) com números
brasileiros
e acentos (módulo 05), conjuntos e dicionários (módulo 03) e validação
com
decisões (módulo 02).

A exceção `ArquivoInvalido` já está criada no arquivo.

Tarefa: implemente `ler_medicoes(caminho)`, que devolve uma lista de
dicionários {"lote": str, "linha": str, "espessura": float, "gramatura":
float}.
- Encoding: tente UTF-8 com "utf-8-sig"; se der UnicodeDecodeError, use
  "cp1252".
- Separador ";". Use o módulo csv (há campos entre aspas com ";" dentro).
- Nomes de coluna: compare sem espaços nas pontas, em minúsculas e sem
acento.
  Colunas a mais (como "Observação") são ignoradas.
- Arquivo sem nenhuma linha: ArquivoInvalido("arquivo vazio").
- Faltando colunas obrigatórias (lote, linha, espessura, gramatura):
  ArquivoInvalido(f"colunas ausentes: {nomes}"), com os nomes que faltam
em
  ordem alfabética, separados por ", ".
- Linhas totalmente vazias (como ";;;;") são puladas.
- "linha" (de produção) precisa ser A, B, C ou D (aceite minúscula e
espaços;
  devolva maiúscula). Senão:
  ArquivoInvalido(f"linha {n}: linha de produção inválida: {valor!r}").
- espessura e gramatura: número brasileiro. Se não converter:
  ArquivoInvalido(f"linha {n}: valor inválido em {coluna}: {valor!r}"),
```

```
  encadeado com `from erro`.
- n é o número da linha no ARQUIVO: o cabeçalho é a linha 1.
- Os valores de texto voltam sem espaços nas pontas.

Exemplos (arquivo com "Lote;Linha;Espessura;Gramatura" e "A-0153; a
;25,4;23,1"):
    >>> ler_medicoes(Path("medicoes.csv"))
    [{'lote': 'A-0153', 'linha': 'A', 'espessura': 25.4, 'gramatura':
23.1}]

Dica: leia o texto inteiro primeiro (com o fallback de encoding) e depois
use
`csv.reader(texto.splitlines(), delimiter=";")`: o csv aceita qualquer
lista
de linhas. Um conjunto resolve as colunas ausentes: obrigatorias -
cabecalho.
Funções pequenas ajudam muito: ler_texto, normalizar_coluna, numero_br.

Teste com:  python curso.py testar 06 ex13
```

Exercício 06.14 — Revisão: conferir uma pasta inteira sem parar no erro

Arquivo: `exercicios/ex14_revisao_conferir_pasta.py`

```
Nível: ★★★

Contexto: Toda manhã, a pasta de laudos tem um CSV por linha de produção.
O
robô confere a espessura de todos os lotes e gera um único CSV de
resultado
para o Excel. Se UM arquivo estiver com problema, os outros continuam
sendo
processados, e o arquivo com problema é listado para alguém corrigir.

Este exercício mistura arquivos e exceções (módulo 06) com números
brasileiros
(módulo 05), decomposição em funções (módulo 04), dicionários e
contadores
(módulo 03) e decisões (módulo 02).

Tarefa: implemente `conferir_pasta(pasta, minimo, maximo, saida)`:
- processe cada arquivo *.csv da pasta (extensão em minúsculas), em ordem
  alfabética do nome; outros arquivos são ignorados;
- cada CSV tem as colunas "lote" e "espessura", separador ";", número
  brasileiro e encoding UTF-8 (com ou sem BOM: use "utf-8-sig");
- cada lote é "APROVADO" se minimo <= espessura <= maximo, senão
"REPROVADO";
```

```
- grave `saida` como CSV do Excel brasileiro (";", "utf-8-sig",
newline="")
  com o cabeçalho arquivo;lote;espessura;status e a espessura com vírgula
e
  2 casas ("25,40");
- arquivo com problema (falta a coluna lote ou espessura, número inválido
ou
  arquivo que não é UTF-8) é PULADO INTEIRO: nenhuma linha dele vai para
a
  saída, mesmo as que estavam certas. Guarde {nome_do_arquivo: str(erro)}
e
  siga para o próximo;
- devolva {"aprovados": N, "reprovados": N, "com_erro": {nome:
mensagem}}.

Exemplos:
    >>> conferir_pasta(Path("laudos"), 22.5, 27.5, Path("resultado.csv"))
    {'aprovados': 3, 'reprovados': 1, 'com_erro': {'linha_C.csv': ...}}

Dica: escreva uma função que lê UM arquivo inteiro e devolve a lista de
linhas
de resultado (ou levanta erro). Só depois que ela devolveu, junte as
linhas à
lista geral: assim um arquivo com erro na 3ª linha não deixa as 2
primeiras
pela metade na saída. Erros a capturar: KeyError, ValueError (e lembre
que
UnicodeDecodeError é um tipo de ValueError).

Teste com:  python curso.py testar 06 ex14
```

Desafio do módulo 06 — Conferência de laudos em lote

Contexto

Toda manhã, cada linha de produção (A, B, C e D) deixa um CSV exportado do Excel na pasta de laudos. O analista abre um por um, confere as medições contra a especificação e monta à mão a lista do que pode ser liberado. Quando um arquivo vem com problema, a conferência inteira atrasa.

Sua missão: escrever `conferir_laudos.py`, que faz essa conferência sozinho e **não para no primeiro problema**.

```
dados/
  regras_tolerancia.json      regras por linha: espessura e gramatura
(nominal ± %), COF (mín/máx)
```

```
  laudos/
    laudos_linha_A.csv        "CSV UTF-8" (com BOM)
    laudos_linha_B.csv        "CSV (separado por vírgulas)" em cp1252,
coluna "Observação"
    laudos_linha_C.csv        alguém digitou "20,1" (letra O) no lugar de
"20,1"
    laudos_linha_D.csv        exportado sem a coluna "cof"
    leia-me.txt               não é laudo: deve ser ignorado
```

As colunas dos laudos são `lote;linha;espessura;gramatura;cof;observacao` (com variações de maiúsculas e acentos no cabeçalho). A observação pode liberar um ensaio que ficou fora da faixa: `COF: liberado`, `espessura: liberada`.

Requisitos obrigatórios

1. Carregar as regras de `dados/regras_tolerancia.json`. Se o JSON estiver quebrado, **parar** com uma exceção `ArquivoInvalido` dizendo o arquivo e a linha do erro.
2. Processar todos os `.csv` de `dados/laudos/` em ordem alfabética, ignorando os outros arquivos.
3. Ler cada CSV com o encoding certo (UTF-8 com BOM ou cp1252) usando o módulo `csv`, com os nomes de coluna normalizados (sem espaços, minúsculos, sem acento).
4. Validar cada arquivo: colunas obrigatórias (`lote`, `linha`, `espessura`, `gramatura`, `cof`) e números no formato brasileiro. Qualquer problema levanta `ArquivoInvalido` com uma mensagem que diga a linha e a coluna.
5. Arquivo inválido é recusado **inteiro** (nenhum lote dele entra no resultado), o erro é mostrado na tela e o programa segue com os próximos arquivos.
6. Conferir cada lote contra a regra da sua linha de produção:
   - `APROVADO`: os três ensaios dentro da faixa (limites inclusive);
   - `APROVADO COM LIBERAÇÃO`: algum ensaio fora, mas todos os que estão fora foram liberados na observação;
   - `REPROVADO`: algum ensaio fora e não liberado. O motivo (ensaios fora e a faixa) acompanha o status.
7. Gravar em `desafio/saida/`:
   - `resultado.csv` no padrão do Excel brasileiro (`;`, vírgula decimal, `utf-8-sig`, `newline=""`), com `arquivo;lote;linha;espessura;gramatura;cof;status;motivo`;
   - `resumo.json` (legível, com acentos) com arquivos lidos, arquivos recusados e o motivo, total de lotes e a contagem por status, gravado de forma **atômica** (temporário + `replace`);
   - `erros.log`, acrescentando (modo `"a"`) uma linha por arquivo recusado.
8. Usar uma família de exceções própria (`ErroDeQualidade` como mãe de `ArquivoInvalido` e de pelo menos mais uma).

9. Se o `resultado.csv` estiver aberto no Excel (`PermissionError`), mostrar uma mensagem clara ("feche o arquivo no Excel") em vez de só o traceback.

Extras opcionais

- Receber a pasta dos laudos como argumento (`sys.argv`, prévia do módulo 07).
- Gravar também um CSV só com os lotes que podem seguir para emissão de certificado.
- Avisar quando o mesmo lote aparece em dois arquivos.
- Validar também o JSON de regras (toda linha precisa dos três ensaios, e cada regra precisa de `nominal` e `tolerancia_pct` ou de `min` e `max`), com `RegraAusente`.
- Calcular a média de cada ensaio por linha de produção e incluir no `resumo.json`.

Exemplo de execução

Na pasta `curso-python`:

```
python 06-arquivos-e-erros/desafio/conferir_laudos.py
```

Saída da solução de referência:

```
[OK]       laudos_linha_A.csv: 4 lotes
[OK]       laudos_linha_B.csv: 3 lotes
[RECUSADO] laudos_linha_C.csv: linha 3: valor inválido em espessura:
'2O,1'
[RECUSADO] laudos_linha_D.csv: colunas ausentes: cof

Lote    Linha  Status                 Motivo
A-0153  A      APROVADO
A-0154  A      REPROVADO              espessura 27,90 fora de [22,50;
27,50]
A-0155  A      APROVADO COM LIBERAÇÃO  cof 0,33 fora de [0,15; 0,30]
A-0156  A      REPROVADO              gramatura 21,50 fora de [21,85;
24,15]
B-0201  B      APROVADO
B-0202  B      APROVADO COM LIBERAÇÃO  cof 0,27 fora de [0,15; 0,25]
B-0203  B      APROVADO

Lotes conferidos: 7 | APROVADO: 3 | APROVADO COM LIBERAÇÃO: 2 |
REPROVADO: 2
Arquivos gravados em: saida/ (resultado.csv, resumo.json, erros.log)
```

E o `resumo.json` gerado:

```
{
  "arquivos_lidos": 2,
  "arquivos_recusados": {
    "laudos_linha_C.csv": "linha 3: valor inválido em espessura: '20,1'",
    "laudos_linha_D.csv": "colunas ausentes: cof"
  },
  "lotes": 7,
  "por_status": {
    "APROVADO": 3,
    "APROVADO COM LIBERAÇÃO": 2,
    "REPROVADO": 2
  }
}
```

O seu layout na tela pode ser diferente. Os status, as contagens e os arquivos recusados devem ser os mesmos.

Critérios de aceite

- [ ] Roda sem erro e sem pedir nada ao usuário.
- [ ] 7 lotes conferidos: 3 aprovados, 2 aprovados com liberação, 2 reprovados.
- [ ] `laudos_linha_C.csv` recusado com mensagem citando a linha 3 e o valor `'20,1'`; `laudos_linha_D.csv` recusado por falta da coluna `cof`; nenhum lote dos dois no resultado.
- [ ] `leia-me.txt` ignorado.
- [ ] `resultado.csv` abre no Excel em colunas, com acentos e vírgula decimal certos.
- [ ] `resumo.json` legível (acentos sem `\u00e7`) e gravado via arquivo temporário.
- [ ] Rodar duas vezes não duplica o `resultado.csv` nem o `resumo.json` (só o `erros.log` cresce).
- [ ] Nenhum `except:` sozinho; cada `except` captura um tipo específico.
- [ ] Funções pequenas, cada uma com uma responsabilidade (ler regras, ler laudo, conferir lote, gravar saídas).

Dicas de como começar

1. **Comece pelas regras.** Carregue o JSON e escreva `faixa(regras, linha, ensaio)`, que devolve `(mínimo, máximo)` para os dois tipos de regra. Teste no REPL para a linha A: espessura de 22,50 a 27,50; COF de 0,15 a 0,30.
2. **Depois, um laudo.** Reaproveite o exercício 13 (`ler_medicoes`): ele já resolve encoding, cabeçalho e números. Acrescente a coluna `cof` e a `observacao` opcional.

3. **Liberações.** A observação "COF: liberado; espessura: liberada" vira o conjunto `{"cof", "espessura"}`. Normalize (sem acento, minúsculo) e use uma regex por ensaio (módulo 05, "No mundo real").
4. **O lote inteiro de uma vez.** Monte as linhas de resultado de um arquivo numa lista e só junte ao resultado geral depois que o arquivo inteiro deu certo (exercício 14).
5. **Saídas por último.** `csv.DictWriter` para o resultado, gravação atômica para o JSON (exercício 12) e modo `"a"` para o log (exercício 4). Para a data e hora do log, `datetime.now().strftime("%Y-%m-%d %H:%M:%S")` (do módulo `datetime`, que você verá no módulo 07).
6. **Teste o pior caso.** Abra o `resultado.csv` no Excel e rode de novo: a mensagem do `PermissionError` aparece?

A solução de referência está em `solucao_referencia.py`. Abra só depois de tentar.

Quiz — Módulo 06: Arquivos e tratamento de erros

Responda sem rodar o código. Depois confira no gabarito e, na dúvida, teste no REPL.

1. Num traceback, onde está a informação mais importante para começar?

- a) Na primeira linha (`Traceback (most recent call last):`).
- b) Na última linha: o tipo da exceção e a mensagem.
- c) No meio, nas linhas de bibliotecas do Python.
- d) Em nenhum lugar: é preciso usar o depurador.

2. O que este código imprime?

```python
def f(texto):
    try:
        valor = int(texto)
    except ValueError:
        return "A"
    else:
        return "B"
    finally:
        print("C", end=" ")

print(f("7"), f("x"))
```

3. Onde está o problema deste código?

```python
def media(valores):
    try:
        return sum(valores) / len(valroes)
    except:
        return 0
```

4. O que é impresso?

```python
try:
    {"a": 1}["b"]
except LookupError as erro:
    print("pegou", type(erro).__name__, erro)
```

- a) Nada: `KeyError` não é capturado por `LookupError`.
- b) `pegou KeyError 'b'`
- c) `pegou LookupError b`
- d) `pegou KeyError b`

5. Qual é a forma correta de criar uma exceção própria para "valor fora da tolerância", filha de uma base `ErroDeQualidade`?

- a) `ValorForaDaTolerancia = Exception("fora da tolerância")`
- b) `class ValorForaDaTolerancia(ErroDeQualidade): """Medição fora da faixa."""`
- c) `def ValorForaDaTolerancia(): raise ErroDeQualidade`
- d) `class ErroDeQualidade(ValorForaDaTolerancia): pass`

6. Para que serve o `from erro` em `raise ArquivoInvalido("...") from erro`?

7. Depois de rodar este código, o que tem dentro de `log.txt`?

```python
with open("log.txt", "w", encoding="utf-8") as a: a.write("1\n")
with open("log.txt", "a", encoding="utf-8") as a: a.write("2\n")
with open("log.txt", "w", encoding="utf-8") as a: a.write("3\n")
with open("log.txt", "a", encoding="utf-8") as a: a.write("4\n")
```

8. Com `p = Path("laudos") / "2025" / "linha_A.final.csv"`, quanto valem `p.name`, `p.stem` e `p.suffix`?

9. Um CSV salvo pelo Excel como "CSV UTF-8" é lido com `open(caminho, encoding="utf-8")` e `csv.DictReader(..., delimiter=";")`. Na hora de acessar `linha["lote"]`, dá `KeyError: 'lote'`, embora a coluna exista. Qual é a causa mais provável e a correção?

10. O que este código imprime?

```
import csv
print(list(csv.reader(['A-1;"COF: liberado; ok";25,4'], delimiter=";")))
```

11. Por que se usa `newline=""` ao abrir um arquivo para gravar com o módulo `csv`?

- a) Para o arquivo não ter quebras de linha.
- b) Para o `csv` controlar as quebras de linha; sem isso, no Windows aparece uma linha em branco entre os registros.
- c) Para gravar em UTF-8.
- d) Porque o modo `"w"` exige.

12. O que este código imprime?

```
import json
print(json.dumps({"faixa": (1, 2), "ok": None, "x": True}))
```

13. Você vai processar 40 CSVs de laudo de uma pasta. Um deles tem um número digitado errado. O que é mais adequado?

- a) Um `try/except` em volta do `for` inteiro, para parar tudo no primeiro erro.
- b) `except:` sozinho dentro do `for`, ignorando qualquer problema em silêncio.
- c) `try/except` com os tipos esperados **dentro** do `for`, registrando o arquivo e a mensagem, e seguindo com os demais.
- d) Corrigir o arquivo à mão antes de rodar, porque programas não devem tratar erros.

14. Por que gravar primeiro em `regras.json.tmp` e depois fazer `tmp.replace(destino)`?

Gabarito

1. **b).** Leia de baixo para cima: a última linha diz **o que** aconteceu; as de cima, onde. Procure a primeira linha que é do seu código.
2. **`C C B A`.** O `finally` roda antes de cada função devolver seu valor, então os dois "C" aparecem primeiro (durante as chamadas). Depois o `print` mostra os retornos: `"B"` para `"7"` (veio pelo `else`) e `"A"` para `"x"`.
3. **O `except:` pega-tudo esconde o bug.** `valroes` é um erro de digitação (`NameError`), mas a função devolve `0` como se fosse uma média. Capture só o erro esperado (aqui, `ZeroDivisionError` para lista vazia).
4. **b) `pegou KeyError 'b'`.** `KeyError` é filha de `LookupError`, então é capturada. O `type(erro).__name__` mostra o tipo real, e a mensagem de um `KeyError` é a chave com aspas.

5. **b).** Uma exceção é uma classe filha de `Exception` (ou de outra exceção). A d) inverte a família.
6. **Encadear os erros.** O erro original (por exemplo, o `json.JSONDecodeError`) fica guardado em `__cause__`, e o traceback mostra os dois: o erro técnico e o erro com contexto que você criou.
7. **`3\n4\n`.** O terceiro `open` em `"w"` apagou o `1` e o `2`; o último `"a"` acrescentou o `4`.
8. **`'linha_A.final.csv'`, `'linha_A.final'` e `'.csv'`.** O `suffix` é só a última extensão; o `stem` é o nome sem ela.
9. **O BOM.** O "CSV UTF-8" do Excel começa com o BOM, que lido como `"utf-8"` fica grudado no primeiro nome de coluna (`'\ufefflote'`). Abra com `encoding="utf-8-sig"`.
10. **`[['A-1', 'COF: liberado; ok', '25,4']]`.** O `csv` respeita as aspas: o `;` de dentro da observação não separa colunas. Com `split(";")` sairiam 4 pedaços.
11. **b).** O módulo `csv` grava o fim de linha `\r\n` por conta própria. Sem `newline=""`, no Windows o Python ainda troca o `\n` por `\r\n` e fica `\r\r\n`.
12. **`{"faixa": [1, 2], "ok": null, "x": true}`.** Tupla vira lista, `None` vira `null` e `True` vira `true`.
13. **c).** Os laudos são independentes: um arquivo ruim não deve impedir a conferência dos outros 39, mas também não pode sumir em silêncio.
14. **Gravação atômica.** Se der erro (ou o programa cair) no meio da gravação, só o temporário fica estragado; o `regras.json` antigo continua inteiro. O `replace` troca um pelo outro de uma vez.

Módulo 07 — Módulos, pacotes e biblioteca padrão

> **Tempo estimado:** 10 a 14 horas · **Pré-requisitos:** módulos 00 a 06 · **Nível:** iniciante avançando para intermediário

O que você vai aprender

- Importar módulos das quatro formas e escolher a certa para cada caso.
- Criar o seu próprio módulo e o seu próprio pacote (pasta com `__init__.py`).
- Usar `if __name__ == "__main__":` para um arquivo ser script **e** módulo.
- Explicar onde o Python procura módulos (`sys.path`) e resolver `ModuleNotFoundError`.
- Organizar um projeto profissional no **src layout**, com `pyproject.toml`.
- Criar ambiente virtual, instalar com `pip`, gerar `requirements.txt` e usar o **Poetry** que já está no seu repositório.
- Calcular prazos e validades com `datetime` e `timedelta` e converter datas no formato brasileiro.
- Contar, agrupar e nomear dados com `Counter`, `defaultdict`, `namedtuple`, `chain` e `groupby`.
- Calcular média, mediana, desvio padrão e CV% com `statistics`, do jeito que o laboratório usa.
- Usar `math`, `random` (com semente), `os`, `sys` e `time` no dia a dia.
- Escrever um programa de linha de comando com `argparse`, com opções, validação e código de saída.

Por que isso importa

Até aqui todo o seu código cabia num arquivo. Programa de verdade não cabe. O Validador de Certificados vai ter leitura de planilha, regras de tolerância, leitura de e-mail, relatório e robô: cada parte num arquivo, conversando por `import`. Saber organizar isso separa "script que funciona no meu computador" de "sistema que outra pessoa consegue manter".

A outra metade é a **biblioteca padrão**: datas, estatística, contagens, sorteios, linha de comando, tudo já vem com o Python. Quem a conhece escreve menos código e erra menos (e `Counter`, `defaultdict` e `datetime` aparecem o tempo todo

em entrevistas).

E tem o lado prático do mercado: `venv`, `pip`, `requirements.txt`, `pyproject.toml` e Poetry estão em todo projeto Python profissional. Você vai sair deste módulo entendendo o `pyproject.toml` que já existe no seu repositório.

1. Módulo: um arquivo `.py` que outro arquivo usa

Todo arquivo `.py` é um **módulo**. O nome do módulo é o nome do arquivo sem o `.py`. Se você tem `conversoes_br.py`, outro arquivo na mesma pasta pode escrever `import conversoes_br` e usar as funções de lá.

Pense na bancada do laboratório: paquímetro numa gaveta, padrões de calibração em outra. Você não espalha tudo na mesa. Módulos são as gavetas do seu código.

Veja os dois arquivos da pasta `exemplos/`. Primeiro, o módulo:

```python
# exemplos/conversoes_br.py
def numero_br(texto: str) -> float:
    """Converte '1.250,5' em 1250.5."""
    return float(texto.strip().replace(".", "").replace(",", "."))


def formatar_br(valor: float, casas: int = 2) -> str:
    """Converte 1250.5 em '1.250,50'."""
    texto = f"{valor:,.{casas}f}"  # '1,250.50' (padrão americano)
    return texto.replace(",", "X").replace(".", ",").replace("X", ".")
```

Depois, o script que usa o módulo:

```python
# exemplos/02_usando_conversoes.py
import conversoes_br
from conversoes_br import formatar_br

peso_total = conversoes_br.numero_br("1.250,5") +
conversoes_br.numero_br("980,25")
print("Peso total da NF:", formatar_br(peso_total), "kg")
```

Rodando `python exemplos/02_usando_conversoes.py` (na pasta `curso-python`):

```
[conversoes_br] carregado; __name__ = 'conversoes_br'
Peso total da NF: 2.230,75 kg
__name__ deste arquivo: __main__
__name__ do módulo importado: conversoes_br
```

A primeira linha vem de um `print` solto dentro de `conversoes_br.py`. Ela mostra algo importante: **importar um módulo executa o arquivo inteiro**, de cima para baixo. As funções são criadas e qualquer código solto roda.

Separar em módulos permite **reaproveitar** (`numero_br` serve para planilha, e-mail e relatório), **encontrar** bugs e **testar** cada parte isolada.

2. As formas de importar

São quatro formas. Todas funcionam; a escolha é sobre clareza.

```
>>> import math
>>> math.sqrt(2)
1.4142135623730951
>>> import statistics as st
>>> st.mean([25.1, 24.9, 25.3])
25.1
>>> from datetime import date, timedelta
>>> date(2026, 10, 29) + timedelta(days=30)
datetime.date(2026, 11, 28)
>>> from random import Random as GeradorAleatorio
>>> GeradorAleatorio(42).randint(1, 6)
6
```

Forma	Como usa depois	Quando usar
`import math`	`math.sqrt(2)`	Padrão. O prefixo diz de onde veio.
`import statistics as st`	`st.mean(...)`	Apelido **consagrado** (`pandas as pd`).
`from datetime import date`	`date(...)`	Poucos nomes, usados muitas vezes.
`from x import a as b`	`b(...)`	Evitar conflito de nomes.

Existe uma quinta forma, `from modulo import *`, que traz **tudo**. Evite: você perde a noção de onde cada nome veio e pode sobrescrever nomes sem perceber. Depois de `from os import *`, o seu `open("ensaios.csv")` passa a chamar o `open` do `os` (outra função) e dá `TypeError: open() missing required argument 'flags'`.

Ordem dos imports. No topo do arquivo, em três blocos separados por linha em branco: biblioteca padrão (`import csv`), bibliotecas instaladas (`import openpyxl`) e os seus módulos (`from validador.regras import aprovar`). Não precisa decorar: o `ruff check --fix` (ou o `isort`, que está no seu `pyproject.toml`) ordena sozinho.

> 🔮 **Preveja antes de rodar:** existe um arquivo `avisos.py` com estas linhas:

```
print("carregando avisos...")
LIMITE_CV = 5.0
```

E outro arquivo, na mesma pasta, faz:

```
import avisos
import avisos
print(avisos.LIMITE_CV)
```

Quantas vezes aparece "carregando avisos..."?

▼ **Resposta**

Uma vez só. A saída é `carregando avisos...` e depois `5.0`.

Na primeira vez, o Python executa o arquivo e guarda o módulo pronto num "cache" chamado `sys.modules`. Nos imports seguintes, só devolve o que já está lá. Por isso importar o mesmo módulo em dez arquivos não custa dez vezes.

⚠️ **Erro comum:** mudar o módulo com o REPL aberto e estranhar que nada mudou. O módulo já está no cache. Feche e abra o REPL de novo (ou rode o script de novo).

3. `if __name__ == "__main__":` — script e módulo ao mesmo tempo

Todo módulo tem uma variável automática chamada `__name__`:

- quando o arquivo é **rodado diretamente** (`python arquivo.py`), vale `"__main__"`;
- quando o arquivo é **importado**, vale o nome do módulo (`"conversoes_br"`).

Isso permite ter um trecho que só roda quando o arquivo é executado direto. O final de `exemplos/conversoes_br.py` é assim:

```
print(f"[conversoes_br] carregado; __name__ = {__name__!r}")

if __name__ == "__main__":
    # Só roda quando o arquivo é executado diretamente
    print(numero_br("1.250,5"))
    print(formatar_br(1250.5))
```

> 🔮 **Preveja antes de rodar:** o que aparece com `python exemplos/conversoes_br.py`? E o bloco do `if` roda quando o `02_usando_conversoes.py` importa esse módulo?

<details>
<summary>Resposta</summary>

Rodando direto:

```
[conversoes_br] carregado; __name__ = '__main__'
1250.5
1.250,50
```

Importado pelo `02_usando_conversoes.py`, o `print` solto roda (com `__name__ = 'conversoes_br'`), mas o bloco do `if` **não** roda, porque `__name__` não é `"__main__"`. Foi o que vimos na seção 1.
</details>

Serve para um autoteste rápido no fim do módulo e, principalmente, para **programas de linha de comando**: a `main()` fica no módulo (e pode ser testada) e o `if` só a chama quando você roda pelo terminal (seção 15).

> 💡 **Dica:** regra de bolso: funções no corpo do arquivo e **nenhum código solto** fora delas, exceto constantes e o bloco `if __name__ == "__main__":`. Assim, importar o arquivo nunca dispara efeitos inesperados (apagar arquivos, mandar e-mail).

> 🔧 **Modifique:** apague o `print` solto de `conversoes_br.py` e rode de novo o `02_usando_conversoes.py`. O import fica silencioso, como um módulo bem-educado.

4. Onde o Python procura módulos

Quando você escreve `import conversoes_br`, o Python procura `conversoes_br.py` (ou uma pasta `conversoes_br/`) numa lista de pastas, **em ordem**: o `sys.path`. Rode `python exemplos/03_onde_python_procura.py` para ver a sua. No meu computador (Linux):

```
Pastas onde o import procura, em ordem:
  0: /home/user/estudos/curso-python/07-modulos-e-biblioteca-
padrao/exemplos
```

```
  ...
  2: /usr/lib/python3.13
  ...
  5: /usr/local/lib/python3.13/dist-packages
```

No Windows, com o venv ativado, aparecem caminhos como `C:\Users\voce\curso-python\.venv\Lib\site-packages`. Três coisas para entender:

1. **A posição 0 é a pasta do script que você rodou.** Por isso `02_usando_conversoes.py` acha `conversoes_br.py`: estão na mesma pasta.
2. Depois vêm as pastas da **biblioteca padrão** (`math`, `datetime`...).
3. Por último, a **`site-packages`**: onde o `pip install` coloca o que você instala. Cada ambiente virtual tem a sua.

Daí saem os dois problemas mais comuns deste módulo.

Problema 1: `ModuleNotFoundError`. Não achou em lugar nenhum: o pacote não está instalado **neste** Python (venv desativado!), o nome está errado ou o módulo não está na pasta do script.

Problema 2: seu arquivo "esconde" um módulo da biblioteca. Como a pasta do script vem primeiro, um arquivo seu chamado `random.py`, `statistics.py` ou `csv.py` é encontrado **antes** do módulo verdadeiro:

```
  File "C:\Users\voce\testes\random.py", line 3, in <module>
    print(random.randint(1, 6))
          ^^^^^^^^^^^^^^
AttributeError: module 'random' has no attribute 'randint' (consider
renaming
'C:\Users\voce\testes\random.py' since it has the same name as the
standard library
module named 'random' and prevents importing that standard library
module)
```

O Python 3.13 até sugere renomear. No 3.12 a mensagem é mais enigmática: `partially initialized module 'random' has no attribute 'randint' (most likely due to a circular import)`. Mesma causa. Renomeie para `sorteio_bobinas.py` e apague a pasta `__pycache__` ao lado, se existir.

> ⚠️ **Erro comum:** dar ao seu arquivo o nome do assunto que você está estudando. `statistics.py`, `datetime.py`, `pandas.py` e `openpyxl.py` são os campeões.

Rodar como módulo: `python -m`. Você já usou `python -m pip` e `python -m pytest`. O `-m` quer dizer "procure este **módulo** no `sys.path` e rode-o como programa". Com `-m`, a posição 0 do `sys.path` passa a ser a **pasta atual** do terminal, não a pasta do arquivo. Isso importa para pacotes (próxima seção).

> 💡 **Dica:** `python -m pip install ...` garante que o `pip` é o do mesmo Python que vai rodar o seu código. Com `pip install ...` puro, no Windows, às vezes o `pip` é de outra instalação.

5. Pacotes: pastas com `__init__.py`

Quando os módulos crescem, você os agrupa numa **pasta**. Uma pasta de módulos com um arquivo `__init__.py` é um **pacote**. A pasta `exemplos/labqualidade/` é um pacote pequeno, feito para você abrir e ler:

```
exemplos/
├── 04_usando_pacote.py
└── labqualidade/
    ├── __init__.py       # transforma a pasta em pacote; define a
"vitrine"
    ├── __main__.py       # roda com: python -m labqualidade
    ├── conversoes.py     # numero_br, data_br
    ├── estatistica.py    # cv_percentual, resumo
    └── tolerancias.py    # faixa_nominal, dentro_da_faixa
```

Com o pacote, os imports ganham um ponto: `from labqualidade.tolerancias import dentro_da_faixa`. E, graças ao `__init__.py`, também `from labqualidade import faixa_nominal`, direto do pacote.

O `__init__.py` roda uma vez, no primeiro `import labqualidade`. Pode ficar vazio, mas costuma servir de **vitrine**: traz para o nível do pacote as funções que quem usa precisa conhecer.

```python
# exemplos/labqualidade/__init__.py (resumido)
from .conversoes import data_br, numero_br
from .estatistica import cv_percentual, resumo
from .tolerancias import dentro_da_faixa, faixa_nominal

__version__ = "0.1.0"
__all__ = ["cv_percentual", "data_br", "dentro_da_faixa",
"faixa_nominal", "numero_br"]
```

- O **ponto** em `from .conversoes import ...` é um **import relativo**: "o módulo `conversoes` deste mesmo pacote". Dois pontos (`..`) sobem um nível.
- `__all__` lista o que é público: define o que sai num `from labqualidade import *` e documenta a vitrine.
- `__version__` é a convenção para a versão do pacote.

Rode `python exemplos/04_usando_pacote.py` para ver o pacote em uso. O `__main__.py` permite rodá-lo como programa: `python -m labqualidade`, de dentro de `exemplos`.

> ⚠️ **Erro comum:** rodar diretamente um arquivo de dentro do pacote que usa import relativo. O Python não sabe que ele pertence a um pacote: `ImportError: attempted relative import with no known parent package`. Rode pelo pacote (`python -m labqualidade`, de dentro de `exemplos`) ou por um script de fora.

> 🔧 **Modifique:** crie `labqualidade/datas.py` com `dias_entre(inicio, fim)`, exporte-a no `__init__.py` e use-a no `04_usando_pacote.py`.

6. Estrutura de projeto profissional: o src layout

O projeto final vai virar um repositório no seu GitHub. A estrutura que o mercado usa (e que o Poetry cria por padrão) é o **src layout**:

```
validador-certificados/
├── pyproject.toml            # nome, versão, dependências, configurações
├── poetry.lock               # versões exatas instaladas (gerado pelo
Poetry)
├── README.md
├── .gitignore                # .venv/, __pycache__/, saida/ ...
├── src/
│   └── validador/            # o PACOTE (o código de verdade)
│       ├── __init__.py
│       ├── __main__.py       # python -m validador
│       ├── cli.py            # argparse e main()
│       ├── planilha.py       # leitura das 4 abas
│       └── regras.py         # tolerâncias
└── tests/
    └── test_regras.py
```

Por que o pacote fica dentro de `src/`?

- **Obriga a instalar o pacote** (`poetry install` ou `pip install -e .`): os testes rodam contra o pacote **instalado**, como o usuário vai recebê-lo.
- **Evita importar sem querer** a pasta errada só porque o terminal estava nela.

O `pip install -e .` ("e" de *editable*) instala apontando para a sua pasta: você edita o código e a mudança vale na hora. O Poetry faz o mesmo no `poetry install`.

Um `pyproject.toml` mínimo para esse projeto, no formato do Poetry 2:

```
[project]
name = "validador-certificados"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = ["openpyxl (>=3.1,<4)"]

[project.scripts]
validador = "validador.cli:main"   # cria o comando "validador" no
terminal

[tool.poetry]
packages = [{ include = "validador", from = "src" }]

[build-system]
requires = ["poetry-core>=2.0.0,<3.0.0"]
build-backend = "poetry.core.masonry.api"
```

Com `[project.scripts]`, depois do `poetry install` o comando `validador` passa a existir no ambiente e chama a função `main()` de `src/validador/cli.py`.

> 💡 **Dica:** no seu repositório `estudos`, o código está em `src/main.py` (um script, não um pacote) e o `pyproject.toml` tem `package-mode = false`: "só gerencie as dependências, não há pacote para instalar". Para estudos está ótimo. Para o projeto final, você vai querer o pacote.

7. Ambiente virtual e pip a fundo

Você viu o básico no módulo 00. Agora com calma, porque é aqui que mora metade dos "no meu computador funciona".

O problema. Seu computador tem **um** Python e vários projetos. O projeto A precisa de `pandas 2.2`; o B, de `pandas 3.0`. Instalando tudo no mesmo lugar, um quebra o outro. O **ambiente virtual** (venv) é uma pasta com uma cópia leve do Python e uma `site-packages` **só daquele projeto**.

```
cd C:\Users\voce\curso-python
python -m venv .venv                    # cria a pasta .venv
.venv\Scripts\activate                  # ativa (o prompt ganha "(.venv)")
python -m pip install -r requirements.txt
deactivate                              # desativa
```

- Se o PowerShell disser que a execução de scripts está desabilitada, rode uma vez: `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`.
- No Linux/macOS: `python3 -m venv .venv` e `source .venv/bin/activate`.
- Mais de uma versão de Python no Windows? `py -3.12 -m venv .venv` escolhe a versão.

Qual Python está rodando? `python -c "import sys; print(sys.executable)"`. Com o venv ativado, tem de aparecer algo como `...\curso-python\.venv\Scripts\python.exe`. Outro caminho = você instala num Python e roda em outro. No VS Code, escolha o mesmo em "Python: Select Interpreter" (`Ctrl+Shift+P`).

```
python -m pip install openpyxl              # instala a versão mais nova
python -m pip install "pandas>=2.2,<4"      # com restrição de versão (aspas!)
python -m pip install -U openpyxl           # atualiza
python -m pip uninstall openpyxl            # remove
python -m pip list --outdated               # o que tem versão nova
python -m pip show openpyxl                 # versão, dependências, local
python -m pip freeze > requirements.txt     # "foto" exata do ambiente
```

> ⚠️ **Erro comum:** no PowerShell, `pip install pandas>=2.2` sem aspas cria um arquivo chamado `=2.2`! O `>` é redirecionamento no terminal. Use aspas quando houver `<` ou `>`.

Escrita	Significa
`openpyxl==3.1.5`	exatamente essa versão
`openpyxl>=3.1,<4`	da 3.1 até antes da 4 (o mais comum)
`openpyxl~=3.1.2`	"compatível": `>=3.1.2` e `<3.2`

O `requirements.txt` pode ser **escrito à mão**, com faixas (como o do curso: diz o que o projeto **precisa**), ou **gerado com** `pip freeze`, com versões exatas de tudo (uma **foto** do ambiente, boa para um robô em produção).

> 💡 **Dica:** a pasta `.venv` **nunca** vai para o Git (centenas de MB, específica do seu computador). O que vai é a receita: `requirements.txt` ou `pyproject.toml` + `poetry.lock`.

8. Poetry: o gerenciador que já está no seu repositório

O pip instala pacotes. O **Poetry** gerencia o projeto: cria o venv, instala, registra no `pyproject.toml` o que você adicionou, trava as versões exatas no `poetry.lock` e roda comandos dentro do ambiente.

Trecho do `pyproject.toml` da raiz do seu repositório `estudos`:

```toml
[tool.poetry]
name = "estudos"
package-mode = false

[tool.poetry.dependencies]
python = ">=3.10"
black = ">=25.1.0,<26.0.0"
pytest = ">=8.4.1,<9.0.0"
python-dotenv = ">=1.1.1,<2.0.0"
# ... isort, pylint, flake8

[tool.poetry.group.dev.dependencies]

[tool.black]
line-length = 95
```

- `[tool.poetry]`: dados do projeto; `package-mode = false` = só dependências.
- `[tool.poetry.dependencies]`: o que o projeto precisa, com faixas de versão (`python = ">=3.10"` diz quais Pythons servem).
- `[tool.poetry.group.dev.dependencies]`: o grupo de **desenvolvimento**. Está vazio, e `black`, `pylint`, `pytest` e companhia estão nas principais. Funciona, mas o certo é ficarem no `dev`: quem só usa o programa não precisa delas.
- `[tool.black]`: configuração de ferramenta, que agora mora no `pyproject.toml`.

```
poetry install                      # cria o venv e instala tudo do
poetry.lock
poetry add openpyxl                 # adiciona dependência (e atualiza o
lock)
poetry add --group dev pytest ruff  # adiciona no grupo de
desenvolvimento
poetry remove flake8                # remove
poetry show --tree                  # dependências e de quem cada uma
```

```
depende
poetry run pytest                    # roda DENTRO do venv do projeto
poetry env info --path               # onde está o venv
poetry check                         # confere se o pyproject.toml está
válido
```

Para **ativar** o venv do Poetry, o Poetry 2 trocou o antigo `poetry shell` por `poetry env activate`, que **mostra** o comando de ativação. No PowerShell: `Invoke-Expression (poetry env activate)`. E `poetry config virtualenvs.in-project true` faz o venv nascer numa pasta `.venv` dentro do projeto, que o VS Code acha sozinho.

Para arrumar o seu `pyproject.toml`: `poetry remove black isort pylint flake8 pytest` e depois `poetry add --group dev black isort pylint flake8 pytest`. No seu arquivo, o Poetry grava em `[tool.poetry.group.dev.dependencies]` (conferi numa cópia do seu projeto com o Poetry 2.4). Projetos novos usam a seção padrão `[project]` (como na seção 6) e um grupo `[dependency-groups]`; o efeito é o mesmo.

`poetry.lock`. O `pyproject.toml` diz "openpyxl entre 3.1 e 4". O lock diz "openpyxl **3.1.5**, et-xmlfile **2.0.0**", tudo resolvido. Quem clonar e rodar `poetry install` recebe exatamente as mesmas versões. **Faça commit do lock** (o seu já está no repositório).

Ferramenta	O que faz	Quando
`venv` + `pip` + `requirements.txt`	o básico, vem com o Python	scripts, cursos, robôs simples
Poetry	dependências, lock, grupos, build	projetos e portfólio
uv	o mesmo que o Poetry, muito mais rápido	cada vez mais comum no mercado

Saber `venv` + `pip` é obrigatório. Poetry (ou uv) é o próximo passo, com os mesmos conceitos: ambiente isolado, lista de dependências, lock.

9. Datas e prazos com `datetime`

O módulo `datetime` tem três tipos que você vai usar sempre:

- `date`: uma data. Ex.: emissão do laudo.
- `datetime`: data **e** hora. Ex.: início do ensaio de tração.
- `timedelta`: uma **duração**. Ex.: validade de 180 dias.

```
>>> from datetime import date, datetime, timedelta
>>> emissao = date(2026, 10, 29)
>>> emissao.weekday()
3
>>> emissao + timedelta(days=180)
datetime.date(2027, 4, 27)
>>> (date(2027, 4, 27) - emissao).days
180
>>> date(2026, 11, 2) > emissao
True
```

- `weekday()` devolve 0 (segunda) a 6 (domingo); quinta é 3.
- **data + timedelta = data**; **data − data = timedelta** (dias em `.days`).
- `date.today()` e `datetime.now()` mudam a cada execução. Por isso as funções dos exercícios recebem `hoje` como parâmetro.

Texto ↔ data no formato brasileiro. Converta a entrada com `strptime` ("string **p**arse time") e formate a saída com `strftime` ("string **f**ormat time"):

```
>>> from datetime import date, datetime
>>> datetime.strptime("29/10/2026", "%d/%m/%Y").date()
datetime.date(2026, 10, 29)
>>> datetime.strptime("28/10/2026 14:30", "%d/%m/%Y %H:%M")
datetime.datetime(2026, 10, 28, 14, 30)
>>> date(2026, 3, 5).strftime("%d/%m/%Y")
'05/03/2026'
>>> date.fromisoformat("2026-10-29")
datetime.date(2026, 10, 29)
```

Os códigos: `%d` dia, `%m` mês, `%Y` ano com 4 dígitos, `%H:%M` hora e minuto. O formato `2026-10-29` é o **ISO 8601**, padrão internacional (e de bancos de dados), lido direto pelo `fromisoformat`.

> ⚠️ **Erro comum:** usar `%A` para o nome do dia: ele depende do idioma do sistema ("quinta-feira" no seu Windows, "Thursday" no servidor). Use uma lista: `["segunda", "terça", ...][data.weekday()]`.

> ⚠️ **Erro comum:** `from datetime import datetime` e depois `datetime.date(2026, 1, 1)`. Aqui `datetime` é a **classe**, e o erro confunde (`TypeError: descriptor 'date' for 'datetime.datetime' objects doesn't apply to a 'int' object`). Importe os dois tipos: `from datetime import date, datetime`.

`timedelta` aceita `days`, `weeks`, `hours`, `minutes`, `seconds`, mas **não** meses nem anos, porque meses têm tamanhos diferentes.

> 🔮 **Preveja antes de rodar:** qual data sai?
>
> ```
> from datetime import date, timedelta
> print(date(2026, 1, 31) + timedelta(days=30))
> ```

▼ **Resposta**

`2026-03-02`. Janeiro tem 31 dias e fevereiro de 2026 tem 28: 30 dias depois de 31/01 caem em 02/03, e não "no fim de fevereiro". Quem pensa "um mês depois" erra o prazo. Se a regra do cliente for "mesmo dia do mês seguinte", você precisa de uma função própria (ou da biblioteca `python-dateutil`).

Exemplo resolvido: validade do laudo

Problema: o laudo vale 180 dias a partir da emissão. Dada a emissão e a data de hoje, quero saber se ainda vale e quantos dias faltam.

Como eu pensei:

1. "Vale até quando?" é uma conta: emissão + 180 dias. Isso é `timedelta`.
2. "Ainda vale?" é uma comparação: hoje <= vencimento. No dia do vencimento ainda vale (confirme com a qualidade: regra de negócio não se adivinha).
3. "Quantos dias faltam?" é uma subtração de datas; o número está em `.days`.
4. Não uso `date.today()` dentro da função: recebo `hoje` como parâmetro, para testar com qualquer data.

```
>>> from datetime import date, timedelta
>>> def situacao_laudo(emissao, hoje, validade_dias=180):
...     vencimento = emissao + timedelta(days=validade_dias)
...     faltam = (vencimento - hoje).days
...     if faltam < 0:
...         return f"VENCIDO há {-faltam} dias (venceu em
{vencimento:%d/%m/%Y})"
...     return f"válido por mais {faltam} dias (até
{vencimento:%d/%m/%Y})"
...
>>> situacao_laudo(date(2026, 10, 29), date(2027, 4, 20))
'válido por mais 7 dias (até 27/04/2027)'
```

```
>>> situacao_laudo(date(2026, 10, 29), date(2027, 5, 1))
'VENCIDO há 4 dias (venceu em 27/04/2027)'
```

> 🔧 **Modifique:** acrescente o caso "vence hoje" (quando `faltam == 0`) com uma mensagem própria. Teste com `date(2027, 4, 27)`.

Exemplo resolvido: prazo em dias úteis

Problema: o certificado deve sair em até 5 dias úteis depois da NF (segunda a sexta, menos feriados).

Como eu pensei: não existe "somar dias úteis" pronto. Mas eu sei andar um dia (`+ timedelta(days=1)`) e sei perguntar se um dia é útil (`weekday() < 5` e fora dos feriados). Então ando um dia de cada vez e só conto os úteis, até chegar a 5. Primeiro avanço, depois testo: o dia da NF não conta.

```
>>> from datetime import date, timedelta
>>> def somar_dias_uteis(inicio, dias, feriados=None):
...     feriados = feriados or set()
...     atual = inicio
...     contados = 0
...     while contados < dias:
...         atual += timedelta(days=1)
...         if atual.weekday() < 5 and atual not in feriados:
...             contados += 1
...     return atual
...
>>> nf = date(2026, 10, 29)
>>> somar_dias_uteis(nf, 5)
datetime.date(2026, 11, 5)
>>> somar_dias_uteis(nf, 5, {date(2026, 11, 2)})
datetime.date(2026, 11, 6)
```

`feriados=None` com `feriados = feriados or set()` é o jeito seguro de ter um padrão "vazio" sem a armadilha do default mutável (módulo 04). O exercício 09 mostra o que acontece quando a ordem "avança, depois testa" é invertida. O script `exemplos/05_datas_e_prazos.py` junta tudo isso.

10. `collections`: Counter, defaultdict e namedtuple

O módulo `collections` tem versões "turbinadas" de dicionário e tupla para tarefas muito comuns.

`Counter`: contar sem `if`

Você já contou com dicionário e `if chave in contagem` (módulo 03). O `Counter` faz isso numa linha:

```
>>> from collections import Counter
>>> reprovacoes = Counter(["C", "B", "C", "A", "C", "B"])
>>> reprovacoes
Counter({'C': 3, 'B': 2, 'A': 1})
>>> reprovacoes["D"]
0
>>> reprovacoes.most_common(2)
[('C', 3), ('B', 2)]
>>> reprovacoes.update(["D", "C"])
>>> reprovacoes.total()
8
```

Chave ausente vale **0** (sem `KeyError`). `most_common(n)` devolve os `n` mais frequentes; empates ficam na ordem em que apareceram. `update` soma contagens novas; `total()` (Python 3.10+) soma tudo. Com uma expressão geradora (a compreensão sem colchetes, módulo 03), você conta direto de uma lista de dicionários:

```
>>> from collections import Counter
>>> ensaios = [
...     {"linha": "A", "resultado": "aprovado"},
...     {"linha": "C", "resultado": "reprovado"},
...     {"linha": "C", "resultado": "reprovado"},
...     {"linha": "B", "resultado": "reprovado"},
... ]
>>> Counter(e["linha"] for e in ensaios if e["resultado"] == "reprovado")
Counter({'C': 2, 'B': 1})
```

`defaultdict`: agrupar sem testar a chave

O `defaultdict` cria o valor inicial sozinho, chamando a função que você passou:

```
>>> from collections import defaultdict
>>> ops_por_linha = defaultdict(list)
>>> for op, linha in [("OP1", "A"), ("OP2", "C"), ("OP3", "A")]:
...     ops_por_linha[linha].append(op)
...
>>> dict(ops_por_linha)
{'A': ['OP1', 'OP3'], 'C': ['OP2']}
>>> peso_por_nf = defaultdict(float)
>>> for nf, peso in [("123456", 1250.5), ("123457", 980.0), ("123456",
```

```
310.25)]:
...     peso_por_nf[nf] += peso
...
>>> dict(peso_por_nf)
{'123456': 1560.75, '123457': 980.0}
```

Passe o **nome** do tipo, sem parênteses: `defaultdict(list)` (lista vazia), `defaultdict(int)` (zero), `defaultdict(float)` (0.0), `defaultdict(set)`.

🔮 **Preveja antes de rodar:**

```
from collections import defaultdict
grupos = defaultdict(list)
print("Z" in grupos)
if grupos["Z"]:
    print("tem coisa no Z")
print("Z" in grupos)
print(dict(grupos))
```

▼ **Resposta**

`False`, depois `True` e por fim `{'Z': []}`. Só **ler** `grupos["Z"]` já cria a chave com uma lista vazia. Para consultar, use `in` ou `.get()`; e devolva `dict(grupos)` para quem usa a sua função não levar esse susto.

`namedtuple`: tupla com nome nos campos

`m[2]` não diz nada; `m.valor` diz tudo. A `namedtuple` cria um "tipo de tupla" com campos nomeados:

```
>>> from collections import namedtuple
>>> Medicao = namedtuple("Medicao", ["op", "ensaio", "valor"])
>>> m = Medicao("OP245101", "espessura", 25.3)
>>> m
Medicao(op='OP245101', ensaio='espessura', valor=25.3)
>>> m.valor, m[2]
(25.3, 25.3)
>>> m._replace(valor=25.1)
Medicao(op='OP245101', ensaio='espessura', valor=25.1)
```

Continua sendo tupla: imutável, desempacota (`op, ensaio, valor = m`), cabe em `set` e serve de chave de dicionário. Para "mudar" um valor, `_replace` cria **outra** tupla. Quando precisar de validação e métodos, a ferramenta é a `dataclass`, do

módulo 08.

11. `itertools`: chain e groupby

As ferramentas de `itertools` devolvem **iteradores**: objetos que entregam os itens um de cada vez, sob demanda, sem montar a lista toda na memória. Para ver o conteúdo, passe por `list()` ou percorra com `for`.

`chain` emenda sequências; `chain.from_iterable(x)` emenda as sequências **de dentro** de `x` (uma lista de listas, os valores de um dicionário):

```
>>> from itertools import chain
>>> email_1 = [("OP1", "123456", 1250.5)]
>>> email_2 = [("OP2", "123457", 980.0), ("OP3", "123457", 410.0)]
>>> list(chain(email_1, email_2))
[('OP1', '123456', 1250.5), ('OP2', '123457', 980.0), ('OP3', '123457',
410.0)]
>>> abas = {"A": [20.1, 19.8], "B": [25.2], "C": [40.3, 39.9]}
>>> list(chain.from_iterable(abas.values()))
[20.1, 19.8, 25.2, 40.3, 39.9]
```

`groupby` percorre a sequência e junta os itens **vizinhos** com a mesma chave. Para cada grupo, entrega a chave e um iterador com os itens:

```
>>> from itertools import groupby
>>> resultados = ["A", "A", "R", "R", "R", "A"]
>>> [(chave, len(list(grupo))) for chave, grupo in groupby(resultados)]
[('A', 2), ('R', 3), ('A', 1)]
```

🔮 **Preveja antes de rodar:** quantos grupos saem?

```
from itertools import groupby
lotes = ["L02", "L01", "L02"]
print([(lote, len(list(g))) for lote, g in groupby(lotes)])
```

▼ **Resposta**

Três grupos: `[('L02', 1), ('L01', 1), ('L02', 1)]`. O `groupby` **não** procura iguais na lista inteira; ele só junta vizinhos. Para agrupar por lote, **ordene pela mesma chave antes**: `groupby(sorted(lotes))` dá `[('L01', 1), ('L02', 2)]`.

Com `key=`, você escolhe por qual informação agrupar (a mesma ideia do `key=` do `sorted`, módulo 04):

```
>>> from itertools import groupby
>>> bobinas = [("L02", 150.0), ("L01", 120.5), ("L02", 149.5), ("L01",
121.0)]
>>> def lote(bobina):
...     return bobina[0]
...
>>> for nome, grupo in groupby(sorted(bobinas, key=lote), key=lote):
...     pesos = [peso for _, peso in grupo]
...     print(nome, len(pesos), sum(pesos))
...
L01 2 241.5
L02 2 299.5
```

⚠️ **Erro comum:** ler o `grupo` duas vezes. Ele é um iterador: depois de percorrido, fica vazio. Se precisar usar mais de uma vez, guarde numa lista: `itens = list(grupo)`.

💡 **Dica:** para agrupar a lista **inteira**, o `defaultdict(list)` costuma ser mais simples. O `groupby` brilha em sequências que já vêm em ordem: produção do turno, registros por data, resultados seguidos.

Exemplo resolvido: a maior sequência de reprovações

Problema: a máquina da linha C reprovou várias bobinas seguidas? Qual foi a maior sequência de "R"?

Como eu pensei: "sequência seguida" = "vizinhos iguais" = `groupby`. Para cada grupo, guardo o tamanho só se a chave for "R" e pego o maior. Sem nenhum "R", `max` de lista vazia dá erro, então uso `max(..., default=0)`.

```
>>> from itertools import groupby
>>> def maior_sequencia(resultados, alvo="R"):
...     tamanhos = [len(list(g)) for valor, g in groupby(resultados) if
valor == alvo]
...     return max(tamanhos, default=0)
...
>>> maior_sequencia(["A", "R", "R", "A", "R", "R", "R", "A"])
3
>>> maior_sequencia(["A", "A"])
0
```

12. `statistics`: a estatística do laboratório

O módulo `statistics` tem as contas que o laboratório faz todo dia:

```
>>> import statistics
>>> espessuras = [25.1, 24.8, 25.3, 24.9, 25.4, 25.0]
>>> round(statistics.mean(espessuras), 3)
25.083
>>> statistics.median(espessuras)
25.05
>>> round(statistics.stdev(espessuras), 3)
0.232
>>> round(statistics.pstdev(espessuras), 3)
0.211
```

Python	Excel (português)	O que é
`statistics.mean`	`MÉDIA`	média aritmética
`statistics.median`	`MED`	valor do meio (com os dados em ordem)
`statistics.stdev`	`DESVPAD.A` (ou `DESVPAD`)	desvio padrão **amostral** (divide por n − 1)
`statistics.pstdev`	`DESVPAD.P`	desvio padrão **populacional** (divide por n)

Qual desvio usar? No laboratório você mede uma **amostra** (6 pontos da bobina, não a bobina inteira). Então o certo, em geral, é o **amostral**: `stdev`, o mesmo `DESVPAD` padrão do Excel. Se um laudo antigo usava `DESVPAD.P`, os números diferem um pouco; não é erro do Python, é a fórmula.

Coeficiente de variação: `CV% = desvio / média × 100`. Ele compara a uniformidade de coisas de tamanhos diferentes: 0,5 µm de desvio é muito num filme de 12 µm e pouco num de 100 µm. Para as espessuras acima, `statistics.stdev(espessuras) / statistics.mean(espessuras) * 100` dá 0,92%.

🔮 **Preveja antes de rodar:** alguém digitou 52 em vez de 25,2. O que muda mais, a média ou a mediana?

```
import statistics
com_erro = [25.1, 24.8, 25.3, 24.9, 25.4, 25.0, 52.0]
print(round(statistics.mean(com_erro), 2),
statistics.median(com_erro))
```

▼ **Resposta**

`28.93 25.1`. A média pulou de 25,08 para 28,93; a mediana quase não mexeu (de 25,05 para 25,1). A mediana é **robusta** a valores estranhos. Quando média e mediana estão muito diferentes, desconfie de erro de digitação ou de medição.

> ⚠️ **Erro comum:** chamar `stdev` com um valor só. Não existe variação com um ponto: `statistics.StatisticsError: stdev requires at least two data points` (no Python 3.10 a mensagem fala em `variance`). O `StatisticsError` é um tipo de `ValueError`, então `except ValueError` também o captura.

Exemplo resolvido: CV% por linha de produção

Problema: a partir do arquivo `dados/ensaios_linhas.csv`, quero o CV% da espessura de cada linha, para ver qual está instável.

Como eu pensei: decompus em etapas que eu já sei fazer:

1. Ler o CSV com `;` (módulo 06) e converter "20,4" em número (módulo 05).
2. Ficar só com a espessura.
3. Agrupar os valores por linha: `defaultdict(list)`.
4. Para cada linha, em ordem: média, desvio, CV.

```
import csv
import statistics
from collections import defaultdict

valores = defaultdict(list)
with open("dados/ensaios_linhas.csv", encoding="utf-8", newline="") as
arquivo:
    for registro in csv.DictReader(arquivo, delimiter=";"):
        if registro["ensaio"] == "espessura":

valores[registro["linha"]].append(float(registro["valor"].replace(",",
"."))))

for linha in sorted(valores):
    media = statistics.mean(valores[linha])
    cv = statistics.stdev(valores[linha]) / media * 100
    print(f"Linha {linha}: média {media:.2f} µm, CV {cv:.2f}%")
```

Saída (rodando de dentro da pasta do módulo):

```
Linha A: média 19.60 µm, CV 1.99%
Linha B: média 25.38 µm, CV 3.08%
```

```
Linha C: média 41.68 µm, CV 6.44%
Linha D: média 59.48 µm, CV 1.64%
```

A linha C se destaca. Esse é o coração do exemplo "No mundo real" deste módulo.

> 🔧 **Modifique:** troque `"espessura"` por `"cof"`. Os CVs ficam bem maiores. Isso quer dizer que o COF está ruim? Não necessariamente: cada ensaio tem a sua variabilidade natural, e o limite de CV deve ser definido **por ensaio**.

13. `math` e `random`

```
>>> import math
>>> math.ceil(4.2), math.floor(4.8)
(5, 4)
>>> 0.1 + 0.2 == 0.3, math.isclose(0.1 + 0.2, 0.3)
(False, True)
```

- `ceil` arredonda **para cima**; `floor`, **para baixo**. Para "quantas bobinas preciso?" é sempre `ceil`: 4,2 bobinas viram 5.
- `isclose` compara floats com tolerância (o `0.1 + 0.2` do módulo 01). É o princípio do `pytest.approx` dos testes.

Exemplo do dia a dia: quantos metros de filme tem uma bobina? Ela é um anel de filme em volta do tubete; área do anel ÷ espessura = comprimento: `L = π × (D² − d²) / (4 × e)`, com tudo em metros.

```
>>> import math
>>> externo, tubete, espessura = 0.600, 0.152, 25e-6
>>> comprimento = math.pi * (externo**2 - tubete**2) / (4 * espessura)
>>> round(comprimento)
10584
>>> math.ceil(50_000 / comprimento)
5
```

(`25e-6` é notação científica: 25 × 10⁻⁶ m = 25 µm. O `_` em `50_000` só separa os milhares para leitura.)

`random` com semente. O "aleatório" do computador é uma conta: com a mesma **semente**, a sequência de sorteios é sempre a mesma. Ótimo para testes e auditoria ("este foi o sorteio, pode repetir para conferir"). Crie um gerador próprio com `random.Random(semente)` em vez de `random.seed(semente)`, que mexe no gerador global do programa inteiro.

```
>>> import random
>>> gerador = random.Random(7)
>>> gerador.sample(["B01", "B02", "B03", "B04", "B05", "B06", "B07",
"B08"], 3)
['B06', 'B02', 'B04']
>>> gerador.randint(1, 6)
6
>>> gerador.choice("ABCD")
'A'
```

`randint(a, b)` inclui **os dois** extremos; `choice` sorteia um item; `choices(seq, k=n)` sorteia n **com** reposição; `sample(seq, n)`, **sem** reposição; `shuffle(lista)` embaralha a própria lista.

> ⚠️ **Erro comum:** `embaralhadas = random.shuffle(amostras)` guarda `None`. O `shuffle` embaralha a **própria** lista e não devolve nada (como o `list.sort()`). Para ter uma lista nova: `copia = list(amostras)` e depois `random.shuffle(copia)`.

> ⚠️ **Erro comum:** usar `random` para senha ou token. Ele é previsível de propósito. Para segurança, use o módulo `secrets` (`secrets.token_hex(16)`).

14. `os`, `sys` e `time`

`os.environ`: variáveis de ambiente. São configurações que ficam fora do código: pasta dos dados, endereço do servidor, senha do e-mail (senha **nunca** vai no código nem no Git). Leia com `os.environ.get("LAB_PASTA", r"C:\laboratorio")` (o segundo valor é o padrão). Para definir uma antes de rodar o programa:

```
$env:LAB_PASTA = "C:\laboratorio"          # PowerShell (vale até fechar
o terminal)
python exemplos/10_os_sys_time.py
```

No Linux/macOS: `export LAB_PASTA=/home/voce/laboratorio`. O `python-dotenv` do seu `pyproject.toml` lê essas variáveis de um arquivo `.env` (que também fica fora do Git). Para arquivos e pastas, o `os` tem funções antigas (`os.listdir`, `os.path.join`), mas você já aprendeu o jeito moderno: `pathlib` (módulo 06).

Tornando funções testáveis. Uma função que lê `os.environ` lá dentro é difícil de testar. O truque é **receber o ambiente como parâmetro**, com `os.environ` como padrão:

```python
import os
from collections.abc import Mapping


def cv_maximo(ambiente: Mapping[str, str] | None = None) -> float:
    if ambiente is None:
        ambiente = os.environ
    return float(ambiente.get("LAB_CV_MAX", "5.0"))
```

No programa, `cv_maximo()`. No teste, `cv_maximo({"LAB_CV_MAX": "3"})`. O `Mapping` na anotação quer dizer "qualquer coisa que funcione como dicionário só para leitura": o `os.environ` parece um dicionário, mas não é um `dict`. O exercício 11 usa esse padrão.

`sys`: o próprio Python.

```python
import sys

print(sys.executable)  # QUAL Python está rodando (venv!)
print(sys.platform)  # 'win32', 'linux' ou 'darwin' (macOS)
print(sys.argv)  # argumentos: python script.py a b -> ['script.py', 'a',
'b']
print("deu ruim", file=sys.stderr)  # mensagem no canal de erros
sys.exit(1)  # encerra com código de saída 1
```

- **Código de saída**: todo programa termina com um número. `0` = deu certo; outro = algo falhou. O Agendador de Tarefas do Windows mostra esse número em "Resultado da Última Execução". No PowerShell, veja com `$LASTEXITCODE`; no Linux, `echo $?`.
- `sys.stderr` é o canal de **erros**. Separar erro (stderr) de resultado (stdout) permite gravar o relatório num arquivo e ainda ver os erros na tela.
- Tratar `sys.argv` na mão é sofrido: a seção 15 resolve.

`time`: medir e esperar. `time.perf_counter()` é o cronômetro certo para medir duração (`fim - inicio` em segundos); `time.sleep(2)` espera 2 segundos (robôs esperando a tela do TOTVS carregar). Para **datas**, use `datetime`.

O script `exemplos/10_os_sys_time.py` mostra os três juntos. Rode com `python exemplos/10_os_sys_time.py primeiro segundo` e veja o `sys.argv`.

15. `argparse`: programas de linha de comando

O projeto final vai rodar assim:

```
python -m validador valores.xlsx --linha B --tolerancia 7.5 --desde
01/10/2026 -v
```

Tratar isso com `sys.argv` na mão é um pesadelo: ordem, conversão de tipos, valores padrão, ajuda, erro amigável. O `argparse` faz tudo isso, em quatro passos:

```
import argparse

parser = argparse.ArgumentParser(prog="faixa", description="Calcula
nominal ± %.")  # 1
parser.add_argument("nominal", type=float, help="valor nominal")  # 2
parser.add_argument("-p", "--percentual", type=float, default=10.0)  # 2
parser.add_argument("-v", "--verbose", action="store_true")  # 2
args = parser.parse_args(["25", "-p", "5"])  # 3
print(args.nominal, args.percentual, args.verbose)  # 4
```

Saída:

```
25.0 5.0 False
```

1. Cria o parser.
2. Declara cada argumento.
3. Lê os argumentos. **Sem** parâmetro, `parse_args()` lê o `sys.argv` do terminal. **Com** uma lista, lê a lista: é assim que se testa.
4. Usa: cada argumento vira um atributo (`args.percentual`) de um objeto `Namespace` (objetos e atributos: módulo 08).

Declaração	**Uso no terminal**	**Valor em `args`**
`add_argument("planilha")`	`valores.xlsx` (obrigatório, sem `--`)	`'valores.xlsx'`
`add_argument("--linha")`	`--linha B` (opcional)	`'B'` ou `None`
`add_argument("--tol", type=float, default=5.0)`	`--tol 7.5`	`7.5` ou `5.0`
`add_argument("--linha", choices=["A", "B"])`	`--linha E`	erro amigável
`add_argument("-v", "--verbose", action="store_true")`	`-v`	`True` ou `False`
`add_argument("--valores", type=float, nargs="+")`	`--valores 24.1 26`	`[24.1, 26.0]`

Declaração	Uso no terminal	Valor em `args`
`add_argument("--cv-max", type=float)`	`--cv-max 5`	`args.cv_max` (hífen vira `_`)

O `type` pode ser **qualquer função** que receba texto: `type=float`, `type=str.upper` (aceita "b" e transforma em "B" antes de conferir o `choices`) ou uma função sua, como `data_br`, que converte "01/10/2026". Se a sua função levantar `argparse.ArgumentTypeError`, o argparse mostra a mensagem e encerra com código 2.

O `--help` vem de graça: rode `python exemplos/11_cli_argparse.py --help` e veja a ajuda montada a partir dos seus `help=` e `description=`. E um argumento errado vira mensagem clara, com código de saída 2:

```
> python exemplos/11_cli_argparse.py abc
usage: faixa [-h] [-p PERCENTUAL] [--unidade UNIDADE]
             [--valores VALORES [VALORES ...]] [-v]
             nominal
faixa: error: argument nominal: invalid float value: 'abc'
```

As mensagens fixas do argparse saem em inglês, e o desenho do `--help` muda um pouco entre versões do Python. Dentro do `help=`, escreva `%%` para ter um `%` literal.

🔮 **Preveja antes de rodar:**

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--linha", type=str.upper, choices=["A", "B",
"C", "D"])
parser.add_argument("--cv-max", type=float, default=5)
args = parser.parse_args(["--linha", "c"])
print(args.linha, args.cv_max, type(args.cv_max).__name__)
```

▼ **Resposta**

`C 5 int`. O `type=str.upper` roda **antes** do `choices`, então "c" vira "C" e é aceito. Já o `default=5` **não** passa pelo `type`: ficou o inteiro `5` que você escreveu. Para ter float sempre, escreva `default=5.0`.

O molde profissional: `main(argv=None)`

```
import argparse
import sys


def criar_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(prog="faixa")
    parser.add_argument("nominal", type=float)
    parser.add_argument("-p", "--percentual", type=float, default=10.0)
    return parser


def main(argv: list[str] | None = None) -> int:
    args = criar_parser().parse_args(argv)
    margem = args.nominal * args.percentual / 100
    print(f"{args.nominal - margem:.2f} a {args.nominal + margem:.2f}")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

`criar_parser()` separado se testa sozinho. `main(argv=None)` lê o `sys.argv` no terminal e uma lista no teste (`main(["25", "-p", "5"])`, conferindo a saída com `capsys`). O número devolvido vira código de saída via `sys.exit(main())`. E o `if` (seção 3) impede que importar o arquivo no teste dispare o programa.

O `curso.py`, que você usa desde o módulo 00, segue esse molde, com **subcomandos** (`testar`, `progresso`, `modulos`) feitos com `add_subparsers`. Abra e leia: agora você entende cada linha.

Exemplo resolvido: um CLI do zero

Problema: quero `python conferir.py 25 24.1 26 27.9 --percentual 10` mostrando quais valores estão fora de 25 ± 10%, com código de saída 1 se algum estiver fora.

Como eu pensei:

1. **Entradas**: nominal e valores (sempre precisos: posicionais) e percentual (tem padrão: opção com `--`). **Saída**: os valores fora e o código de saída.
2. Escrevo o parser e testo **sem terminal**, passando listas para `parse_args`.
3. Só depois escrevo a regra.

```
>>> import argparse
>>> parser = argparse.ArgumentParser(prog="conferir")
```

```
>>> _ = parser.add_argument("nominal", type=float)
>>> _ = parser.add_argument("valores", type=float, nargs="+")
>>> _ = parser.add_argument("-p", "--percentual", type=float,
default=10.0)
>>> args = parser.parse_args(["25", "24.1", "26", "27.9"])
>>> args
Namespace(nominal=25.0, valores=[24.1, 26.0, 27.9], percentual=10.0)
>>> margem = args.nominal * args.percentual / 100
>>> fora = [v for v in args.valores if not (args.nominal - margem <= v <=
args.nominal + margem)]
>>> fora
[27.9]
>>> codigo_saida = 1 if fora else 0
>>> codigo_saida
1
```

(O `_ =` só esconde o retorno do `add_argument` no REPL.) Versão completa, com `main()`: `exemplos/11_cli_argparse.py`.

> 🔧 **Modifique:** no `11_cli_argparse.py`, acrescente `--saida` com `choices=["texto", "csv"]`. No modo `csv`, imprima `valor;situacao` em cada linha. Rode com `--help` e veja a opção nova aparecer sozinha.

Lendo mensagens de erro deste módulo

`ModuleNotFoundError: No module named 'openpyxl'` — o pacote não está instalado **no Python que está rodando**: ative o venv e rode `python -m pip install openpyxl`. Se o módulo é seu, confira se ele está na pasta **do script** (posição 0 do `sys.path`).

`ImportError: cannot import name 'media' from 'statistics'` — o módulo existe, o nome não (é `mean`). Confira com `dir(statistics)` no REPL ou na documentação.

`AttributeError: module 'random' has no attribute 'randint'` (com "consider renaming" ou "partially initialized module") — seu arquivo tem o nome de um módulo da biblioteca. Renomeie.

`ImportError: attempted relative import with no known parent package` — você rodou diretamente um arquivo que usa `from .algo import ...`. Rode pelo pacote (`python -m pacote`) ou por um script de fora.

`ValueError: time data '2026-10-29' does not match format '%d/%m/%Y'` — o texto não segue o formato. O dado veio em ISO (`ano-mês-dia`) e o formato pedia `dia/mês/ano`.

`statistics.StatisticsError: stdev requires at least two data points` — desvio com menos de 2 valores. Confira o filtro: talvez a linha pedida não tenha dados no período.

`faixa: error: argument --linha: invalid choice: 'E' (choose from 'A', 'B', 'C', 'D')` — não é traceback: é o argparse explicando o que estava errado (código de saída 2). Leia a linha `usage:` acima dela para ver o formato esperado.

Boas práticas

- **Um módulo, um assunto.** Se o nome do arquivo precisa de "e", divida.
- **Imports no topo**, em três blocos, sem `import *`; e **nenhum arquivo seu com nome de módulo** da biblioteca ou de pacote instalado.
- **Todo script termina com** `if __name__ == "__main__":` chamando `main()`.
- **Um venv por projeto**, fora do Git, e dependências registradas (`requirements.txt` ou `pyproject.toml` + `poetry.lock`). Se não está registrado, não existe para quem vai rodar depois.
- **Datas como `date`, não como texto.** Converta na entrada, formate só na saída.
- **Funções recebem `hoje`, `ambiente` e `semente` como parâmetros** em vez de chamarem `date.today()`, `os.environ` e `random` lá dentro. Fica testável.
- **Erros no `stderr` e código de saída significativo** em programas de linha de comando.

No mundo real

O script `exemplos/12_relatorio_linhas.py` é o tipo de ferramenta que você vai rodar toda semana no laboratório: lê o CSV exportado da planilha de ensaios, filtra por ensaio, linha e período, calcula média, desvio e CV% por linha e conta as reprovações. Com `--estrito`, termina com código 1 quando alguma linha passa do CV máximo (assim o Agendador de Tarefas pode avisar alguém).

Ele usa quase tudo do módulo: `argparse` com `type=`, `choices=` e uma função `data_br`; `os.environ` para o CV máximo padrão; `Path(__file__)` para achar o CSV a partir do próprio script; `datetime` para filtrar o período; `defaultdict` e `Counter` para agrupar e contar; `statistics` para as contas; `sys.stderr` e códigos de saída.

Rodando sem opções (na pasta `curso-python`):

```
> python 07-modulos-e-biblioteca-padrao/exemplos/12_relatorio_linhas.py
Ensaio: espessura | período: 01/10/2026 a 03/11/2026
Linha   n    Média  Desvio    CV%  Situação
```

```
A       6    19.60    0.39   1.99  estável
B       6    25.38    0.78   3.08  estável
C       6    41.68    2.68   6.44  ALERTA: CV alto
D       6    59.48    0.97   1.64  estável

Reprovações no período (todos os ensaios):
  Linha C: 4
  Linha B: 2
```

Experimente `--ensaio gramatura --desde 01/10/2026 --ate 15/10/2026` e `--linha c --ensaio cof --cv-max 20`. Abra o arquivo e leia de cima para baixo: pouco mais de 100 linhas, cada função fazendo uma coisa. No projeto final, essas funções vão morar em módulos separados dentro de `src/validador/`, e o mesmo esqueleto (`criar_parser` + `main`) vira o comando `validador`.

No mercado, a "pequena ferramenta de linha de comando que lê um arquivo, calcula e devolve um código de saída" está em toda parte: pipelines de dados, rotinas agendadas, verificações automáticas. Saber escrever uma bem feita é habilidade de entrevista.

Resumo

Assunto	Sintaxe essencial
Importar	`import math` · `import pandas as pd` · `from datetime import date`
Script e módulo	`if __name__ == "__main__":`
Onde procura	`sys.path` (pasta do script primeiro, `site-packages` no fim)
Pacote	pasta + `__init__.py` · `from .modulo import nome` · `python -m pacote`
venv	`python -m venv .venv` · `.venv\Scripts\activate` · `deactivate`
pip	`python -m pip install "x>=1,<2"` · `freeze > requirements.txt` · `install -r`
Poetry	`poetry install` · `poetry add x` · `poetry add --group dev x` · `poetry run x`
Datas	`date(2026, 10, 29)` · `+ timedelta(days=180)` · `(d2 - d1).days` · `weekday()`
Texto ↔ data	`datetime.strptime(t, "%d/%m/%Y").date()` · `d.strftime("%d/%m/%Y")`
Contar	`Counter(seq)` · `.most_common(n)` · chave ausente vale 0
Agrupar	`defaultdict(list)` · `defaultdict(float)` · devolva `dict(...)`

Assunto	Sintaxe essencial
Tupla nomeada	`namedtuple("Medicao", ["op", "valor"])` · `m.valor` · `m._replace(...)`
itertools	`chain.from_iterable(listas)` · `groupby(sorted(x, key=k), key=k)`
Estatística	`mean` · `median` · `stdev` (amostral) · `pstdev` · CV = stdev/mean×100
math / random	`ceil` · `floor` · `isclose` · `random.Random(semente).sample(seq, n)`
os / sys / time	`os.environ.get` · `sys.argv` · `sys.exit(n)` · `sys.stderr` · `time.perf_counter`
argparse	`add_argument(..., type=, default=, choices=, action="store_true")` · `parse_args(argv)`

Você domina este módulo se...

- ☐ Consigo explicar, sem consultar, o que acontece quando o Python executa `import conversoes_br`: onde procura, o que executa e por que o segundo import não executa de novo.
- ☐ Sei o valor de `__name__` quando o arquivo é rodado e quando é importado, e para que serve o `if __name__ == "__main__":`.
- ☐ Monto um pacote com `__init__.py` que exporta funções de dois módulos e o uso de outro script.
- ☐ Crio e ativo um venv no PowerShell, instalo dependências e descubro qual Python está rodando.
- ☐ Leio o `pyproject.toml` do meu repositório e explico cada seção, inclusive `package-mode = false` e o grupo `dev`.
- ☐ Calculo validade e prazo (inclusive em dias úteis) com `date` e `timedelta`, e converto datas brasileiras nos dois sentidos.
- ☐ Escolho entre `Counter`, `defaultdict` e `groupby` para contar ou agrupar, e sei por que o `groupby` precisa de dados ordenados.
- ☐ Escrevo um programa com `argparse` no molde `main(argv=None)` e testo o parser passando uma lista.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_comprimento_bobina`	★☆☆	`math.pi`, `math.ceil`, conversão de unidades, `ValueError`

Nº	Exercício	Nível	O que pratica
02	`ex02_validade_laudo`	★☆☆	`date`, `timedelta`, comparar datas, `.days`
03	`ex03_datas_brasileiras`	★☆☆	`strptime`, `strftime`, `weekday()`
04	`ex04_estatistica_ensaio`	★☆☆	`statistics.mean/median/stdev`, CV%
05	`ex05_contar_reprovacoes`	★☆☆	`Counter`, `most_common`, taxa por linha
06	`ex06_sorteio_amostras`	★★☆	`random.Random(semente)`, `sample`, `shuffle` numa cópia
07	`ex07_agrupar_expedicao`	★★☆	`chain.from_iterable`, `defaultdict(list)` e `(float)`
08	`ex08_sequencias_groupby`	★★☆	`groupby`, ordenar antes de agrupar, `max(default=)`
09	`ex09_corrigir_dias_uteis`	★★☆	**leitura de código**: prever e corrigir 3 defeitos em prazos
10	`ex10_parser_cli`	★★☆	`argparse`: posicional, `choices`, `type` próprio, `store_true`
11	`ex11_cli_relatorio`	★★★	programa completo: CSV, `statistics`, `os.environ`, `stderr`, código de saída
12	`ex12_revisao_expedicao`	★★★	**revisão**: regex e número BR (05) + datas e `defaultdict`
13	`ex13_revisao_relatorio_csv`	★★★	**revisão**: CSV, JSON e erros (06) + agrupar e datas

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError` e rode, na pasta `curso-python`:

```
python curso.py testar 07 ex01     # um exercício
python curso.py testar 07          # o módulo inteiro
```

> 💡 **Dica:** neste módulo, **você** escreve os imports. Os arquivos de exercício só trazem os imports usados nas anotações de tipo. Se aparecer `NameError: name 'statistics' is not defined`, falta o `import statistics` no topo.

Desafio do módulo

Agenda de validade de laudos: um programa de linha de comando que lê `dados/laudos.csv`, separa os laudos em vencidos, a vencer (numa janela de dias) e em dia, mostra um resumo por cliente e devolve um código de saída útil para o Agendador de Tarefas. Junta `datetime`, `Counter`, `argparse`, `os.environ` e `sys`.

Enunciado completo, exemplo de execução e critérios de aceite em desafio/README.md.

Quiz

Teste o que ficou em quiz.md: 13 perguntas com gabarito comentado.

Para ir além

- Tutorial oficial, módulos e pacotes: https://docs.python.org/pt-br/3/tutorial/modules.html
- Ambientes virtuais e pacotes: https://docs.python.org/pt-br/3/tutorial/venv.html
- Tour pela biblioteca padrão: https://docs.python.org/pt-br/3/tutorial/stdlib.html
- `datetime` (a tabela de códigos `%d`, `%m`... fica em "strftime() and strptime() Format Codes"): https://docs.python.org/pt-br/3/library/datetime.html
- `collections`, `itertools` e `statistics`: https://docs.python.org/pt-br/3/library/collections.html, https://docs.python.org/pt-br/3/library/itertools.html e https://docs.python.org/pt-br/3/library/statistics.html
- Tutorial de `argparse`: https://docs.python.org/pt-br/3/howto/argparse.html
- Poetry, guia básico: https://python-poetry.org/docs/basic-usage/
- Leitura extra: "src layout vs flat layout", no guia oficial de empacotamento (https://packaging.python.org/en/latest/discussions/src-layout-vs-flat-layout/).
- Leitura extra: Python Module of the Week, um exemplo comentado para cada módulo da biblioteca padrão (https://pymotw.com/3/).

Enunciados dos exercícios

Exercício 07.01 — Comprimento da bobina e bobinas por pedido

Arquivo: `exercicios/ex01_comprimento_bobina.py`

> Nível: ★☆☆

```
Contexto: A expedição pergunta quantos metros de filme cabem numa bobina
e quantas
bobinas são necessárias para atender um pedido. Dá para estimar só com
geometria:
a bobina é um "anel" de filme enrolado em volta do tubete.

Tarefa:
1. `comprimento_bobina(diametro_externo_mm, diametro_tubete_mm,
espessura_um)` devolve
   o comprimento aproximado do filme em METROS, arredondado com
round(..., 1):
       L = π × (D² − d²) / (4 × e)
   D = diâmetro externo, d = diâmetro do tubete, e = espessura, todos em
METROS
   (mm → m: divida por 1000; µm → m: divida por 1_000_000). Use math.pi.
   Levante ValueError se a espessura for <= 0 ou se o tubete não for
menor que o
   diâmetro externo.
2. `bobinas_necessarias(metros_pedido, metros_por_bobina)` devolve
quantas bobinas
   INTEIRAS atendem o pedido (sempre arredonda para cima: use math.ceil).
   Levante ValueError se metros_por_bobina <= 0.

Exemplos:
    >>> comprimento_bobina(600, 152, 25)
    10583.9
    >>> bobinas_necessarias(50000, 10583.9)
    5
    >>> bobinas_necessarias(20000, 10583.9)
    2

Dica: escreva `import math` no topo. math.ceil(4.01) é 5 e math.ceil(4.0)
é 4.

Teste com:  python curso.py testar 07 ex01
```

Exercício 07.02 — Validade do laudo

Arquivo: `exercicios/ex02_validade_laudo.py`

```
Nível: ★☆☆

Contexto: Todo laudo de qualidade tem validade (no seu laboratório, 180
dias a partir
da emissão). O cliente só aceita material com laudo válido, então a
expedição precisa
saber se o laudo ainda vale e quantos dias faltam.

Tarefa:
```

```
1. `data_vencimento(emissao, dias_validade=180)` devolve a data de
vencimento
   (emissão + dias_validade dias).
2. `laudo_valido(emissao, hoje, dias_validade=180)` devolve True se
`hoje` for
   menor ou igual ao vencimento (no próprio dia do vencimento o laudo
ainda vale).
3. `dias_para_vencer(emissao, hoje, dias_validade=180)` devolve quantos
dias faltam
   até o vencimento (int). Se já venceu, o número é negativo.

Por que `hoje` é parâmetro e não date.today()? Porque assim dá para
testar com
qualquer data. Quem chama a função passa date.today() no programa de
verdade.

Exemplos:
    >>> from datetime import date
    >>> data_vencimento(date(2026, 10, 29))
    datetime.date(2027, 4, 27)
    >>> laudo_valido(date(2026, 10, 29), date(2027, 4, 27))
    True
    >>> dias_para_vencer(date(2026, 10, 29), date(2027, 5, 1))
    -4

Dica: subtrair duas datas dá um timedelta; o atributo .days tem o número
de dias.

Teste com:  python curso.py testar 07 ex02
```

Exercício 07.03 — Datas no formato brasileiro

Arquivo: `exercicios/ex03_datas_brasileiras.py`

```
Nível: ★☆☆

Contexto: Na planilha do laboratório e nos e-mails da expedição as datas
vêm como
texto "29/10/2026" ou "29/10/2026 14:30". O Python precisa de objetos
date/datetime
para fazer contas, e os relatórios precisam do texto de volta.

Tarefa:
1. `ler_data_br(texto)` converte "dd/mm/aaaa" em date. Ignore espaços nas
pontas.
   Texto inválido (ex.: "31/02/2026" ou "2026-10-29") deve gerar
ValueError
   (o strptime já faz isso por você).
2. `formatar_data_br(data)` faz o caminho contrário: date → "dd/mm/aaaa".
```

```
3. `ler_data_hora_br(texto)` converte "dd/mm/aaaa hh:mm" em datetime.
4. `dia_da_semana(data)` devolve o nome do dia em português, em
minúsculas e sem
   "-feira": "segunda", "terça", "quarta", "quinta", "sexta", "sábado",
"domingo".

Exemplos:
    >>> from datetime import date
    >>> ler_data_br(" 29/10/2026 ")
    datetime.date(2026, 10, 29)
    >>> formatar_data_br(date(2026, 3, 5))
    '05/03/2026'
    >>> ler_data_hora_br("29/10/2026 14:30")
    datetime.datetime(2026, 10, 29, 14, 30)
    >>> dia_da_semana(date(2026, 10, 29))
    'quinta'

Dica: datetime.strptime(texto, "%d/%m/%Y") devolve um datetime; o método
.date()
pega só a data. Para o dia da semana, .weekday() devolve 0 (segunda) a 6
(domingo).
Não use "%A": ele depende do idioma do computador.

Teste com:  python curso.py testar 07 ex03
```

Exercício 07.04 — Estatística de um ensaio

Arquivo: `exercicios/ex04_estatistica_ensaio.py`

```
Nível: ★☆☆

Contexto: No ensaio de espessura você mede vários pontos da bobina. O
laudo pede
média, mediana, desvio padrão e o coeficiente de variação (CV%), que diz
se a
espessura está uniforme: CV% = desvio padrão / média × 100.

Tarefa:
1. `coeficiente_variacao(valores)` devolve o CV% com DESVIO AMOSTRAL
   (statistics.stdev, o mesmo DESVPAD.A do Excel), arredondado com
round(..., 2).
   Levante ValueError se houver menos de 2 valores ou se a média for
zero.
2. `resumo_ensaio(valores)` devolve um dicionário com as chaves:
   "n" (int), "media", "mediana", "desvio", "minimo", "maximo"
(arredondados com
   round(..., 3)) e "cv" (arredondado com round(..., 2)).
   Levante ValueError se houver menos de 2 valores.
```

```
Exemplos:
    >>> coeficiente_variacao([25.1, 24.8, 25.3, 24.9, 25.4, 25.0])
    0.92
    >>> resumo_ensaio([10.0, 12.0])["desvio"]
    1.414

Dica: import statistics; use statistics.mean, statistics.median e
statistics.stdev.

Teste com:  python curso.py testar 07 ex04
```

Exercício 07.05 — Reprovações por linha com Counter

Arquivo: `exercicios/ex05_contar_reprovacoes.py`

```
Nível: ★☆☆

Contexto: Toda sexta a coordenação pergunta: "qual linha mais reprovou na
semana?".
Você tem a lista de ensaios do período, cada um como um dicionário:
    {"op": "OP245103", "linha": "C", "ensaio": "espessura", "resultado":
"reprovado"}

Tarefa:
1. `contar_reprovacoes(ensaios)` devolve um Counter com quantas
reprovações cada
   linha teve. Compare o resultado sem diferenciar maiúsculas
("REPROVADO" também
   conta). Linhas sem reprovação não precisam aparecer.
2. `piores_linhas(ensaios, n=2)` devolve a lista das `n` linhas com mais
reprovações,
   como tuplas (linha, quantidade), da maior para a menor (empate: quem
apareceu
   primeiro na lista vem antes, que é o comportamento do most_common).
3. `taxa_reprovacao(ensaios)` devolve um dicionário {linha: % de
reprovação}, com
   round(..., 1), para TODAS as linhas que aparecem nos ensaios
(inclusive 0.0%).

Exemplos:
    >>> ensaios = [
    ...     {"linha": "A", "resultado": "aprovado"},
    ...     {"linha": "C", "resultado": "reprovado"},
    ...     {"linha": "C", "resultado": "REPROVADO"},
    ...     {"linha": "B", "resultado": "reprovado"},
    ... ]
    >>> contar_reprovacoes(ensaios)
    Counter({'C': 2, 'B': 1})
    >>> piores_linhas(ensaios, 1)
```

```
    [('C', 2)]
    >>> taxa_reprovacao(ensaios)
    {'A': 0.0, 'C': 100.0, 'B': 100.0}

Dica: from collections import Counter. Um Counter devolve 0 para chave
ausente.

Teste com:  python curso.py testar 07 ex05
```

Exercício 07.06 — Sorteio de bobinas para auditoria

Arquivo: `exercicios/ex06_sorteio_amostras.py`

```
Nível: ★★☆

Contexto: A auditoria interna pede que as bobinas reensaiadas sejam
escolhidas por
SORTEIO, e que o sorteio possa ser repetido depois (para provar que não
houve
escolha a dedo). Para isso usamos uma semente: mesma semente, mesmo
sorteio.

Tarefa:
1. `sortear_bobinas(bobinas, quantidade, semente=None)` sorteia
`quantidade` bobinas
   SEM repetição, usando EXATAMENTE
`random.Random(semente).sample(bobinas, quantidade)`.
   Levante ValueError se quantidade for negativa ou maior que o número de
bobinas.
2. `sortear_por_linha(bobinas_por_linha, por_linha, semente=None)` recebe
um dicionário
   {linha: [bobinas]} e sorteia `por_linha` bobinas de cada linha. Regras
exatas:
   - crie UM gerador: `gerador = random.Random(semente)`;
   - percorra as linhas em ordem alfabética (sorted);
   - de cada linha sorteie `gerador.sample(lista, min(por_linha,
len(lista)))`;
   - devolva {linha: sorteadas}, com as linhas em ordem alfabética.
3. `embaralhar_ordem(amostras, semente=None)` devolve uma NOVA lista com
as amostras
   embaralhadas (ordem em que vão para o equipamento) usando
   `random.Random(semente).shuffle(...)` numa CÓPIA. A lista original não
muda.

Exemplos:
    >>> sortear_bobinas(["B01", "B02", "B03", "B04", "B05"], 2,
semente=1)
    ['B02', 'B01']
    >>> sortear_bobinas(["B01", "B02"], 3)
```

```
    Traceback (most recent call last):
    ...
    ValueError: Não dá para sortear 3 de 2 bobinas

Dica: shuffle embaralha a própria lista e devolve None. Copie antes:
lista[:] ou
list(lista).

Teste com:  python curso.py testar 07 ex06
```

Exercício 07.07 — Agrupar a expedição por NF

Arquivo: `exercicios/ex07_agrupar_expedicao.py`

```
Nível: ★★☆

Contexto: Cada e-mail da expedição traz uma lista de itens (op, nf,
peso_kg). Um
mesmo dia tem vários e-mails, e uma NF pode ter várias OPs. Para emitir
os
certificados você precisa juntar tudo e agrupar por NF.

Tarefa:
1. `juntar_emails(emails)` recebe uma lista de listas de itens e devolve
UMA lista com
   todos os itens, na ordem (use itertools.chain).
2. `ops_por_nf(itens)` devolve {nf: [ops]} com as OPs na ordem em que
aparecem, sem
   repetir a mesma OP dentro da mesma NF (use defaultdict(list)).
3. `peso_por_nf(itens)` devolve {nf: peso total}, com round(..., 2)
   (use defaultdict(float)).
As funções 2 e 3 devolvem dict comum (converta com dict(...)).

Exemplos:
    >>> email_1 = [("OP245101", "123456", 1250.5)]
    >>> email_2 = [("OP245102", "123456", 980.0), ("OP245103", "123457",
410.25)]
    >>> itens = juntar_emails([email_1, email_2])
    >>> len(itens)
    3
    >>> ops_por_nf(itens)
    {'123456': ['OP245101', 'OP245102'], '123457': ['OP245103']}
    >>> peso_por_nf(itens)
    {'123456': 2230.5, '123457': 410.25}

Dica: chain.from_iterable(lista_de_listas) "achata" uma lista de listas.

Teste com:  python curso.py testar 07 ex07
```

Exercício 07.08 — Sequências de resultados com groupby

Arquivo: `exercicios/ex08_sequencias_groupby.py`

```
Nível: ★★☆

Contexto: Na carta de acompanhamento, cada bobina recebe "A" (aprovada)
ou "R"
(reprovada), na ordem de produção. Uma sequência longa de "R" indica
problema na
máquina. E, no fim do turno, você soma o peso das bobinas de cada lote.

Tarefa:
1. `resumir_sequencia(resultados)` compacta a lista em tuplas (valor,
quantas vezes
   seguidas). ["A", "A", "R", "A"] → [("A", 2), ("R", 1), ("A", 1)].
2. `maior_sequencia(resultados, alvo="R")` devolve o tamanho da MAIOR
sequência
   seguida do valor `alvo` (0 se ele não aparecer).
3. `peso_por_lote(bobinas)` recebe tuplas (lote, peso_kg) FORA de ordem e
devolve uma
   lista de tuplas (lote, peso total com round(..., 2)), com os lotes em
ordem
   alfabética e cada lote aparecendo UMA vez.

Use itertools.groupby nas três.

Exemplos:
    >>> resumir_sequencia(["A", "A", "R", "R", "R", "A"])
    [('A', 2), ('R', 3), ('A', 1)]
    >>> maior_sequencia(["R", "A", "R", "R", "A"])
    2
    >>> peso_por_lote([("L02", 150.0), ("L01", 120.5), ("L02", 149.5)])
    [('L01', 120.5), ('L02', 299.5)]

Dica: groupby só junta itens VIZINHOS. No item 3, ordene antes pela mesma
chave.
O "grupo" é um iterador: para contar, use len(list(grupo)) ou sum(1 for _
in grupo).

Teste com:  python curso.py testar 07 ex08
```

Exercício 07.09 — Leitura de código: o prazo que caía no sábado

Arquivo: `exercicios/ex09_corrigir_dias_uteis.py`

```
Nível: ★★☆
```

````
Contexto: Um colega escreveu funções para calcular o prazo de emissão do
certificado
(N dias úteis depois da NF). O sistema marcou um prazo para um SÁBADO.
Seu trabalho:
ler o código, prever o que ele faz e corrigir os defeitos.

Código do colega (com 3 defeitos):

    from datetime import date, timedelta

    def eh_dia_util(dia, feriados=set()):
        return dia.weekday() <= 5 and dia not in feriados

    def somar_dias_uteis(inicio, dias, feriados=set()):
        atual = inicio
        contados = 0
        while contados < dias:
            if eh_dia_util(atual, feriados):
                contados += 1
            atual = atual + timedelta(days=1)
        return atual

    def dias_uteis_entre(inicio, fim, feriados=set()):
        total = 0
        atual = inicio
        while atual <= fim:
            if eh_dia_util(atual, feriados):
                total += 1
            atual = atual + timedelta(days=1)
        return total

Regras CORRETAS:
- Dia útil: segunda a sexta (weekday() de 0 a 4) e fora da lista de
feriados.
- somar_dias_uteis(inicio, dias, feriados): o próprio dia `inicio` NÃO
conta. Ande
  um dia de cada vez e conte só os dias úteis até completar `dias`. Com
dias=0,
  devolva o próprio `inicio`.
- dias_uteis_entre(inicio, fim, feriados): quantos dias úteis existem
DEPOIS de
  `inicio` até `fim` (inclusive). Se fim <= inicio, devolva 0.
- `feriados` pode ser None (sem feriados). Troque o default `set()` por
None: default
  mutável é uma armadilha (módulo 04), mesmo quando não causa erro aqui.

Tarefa:
1. ANTES de corrigir, preveja: o que a versão do colega devolve para
   `somar_dias_uteis(date(2026, 10, 29), 2)`? (29/10/2026 é uma quinta.)
Escreva sua
````

```
   previsão como retorno de `previsao_versao_com_defeito()`. Depois rode
o código do
   colega para conferir (PRIMM: preveja, rode, investigue).
2. Copie as três funções para baixo, corrija os defeitos e mantenha as
assinaturas.

Exemplos (versão corrigida):
    >>> from datetime import date
    >>> eh_dia_util(date(2026, 10, 31))  # sábado
    False
    >>> somar_dias_uteis(date(2026, 10, 29), 2)
    datetime.date(2026, 11, 2)
    >>> somar_dias_uteis(date(2026, 10, 29), 2, {date(2026, 11, 2)})  #
Finados
    datetime.date(2026, 11, 3)
    >>> dias_uteis_entre(date(2026, 10, 29), date(2026, 11, 5))
    5

Teste com:  python curso.py testar 07 ex09
```

Exercício 07.10 — Argumentos de linha de comando com argparse

Arquivo: `exercicios/ex10_parser_cli.py`

```
Nível: ★★☆

Contexto: O Validador de Certificados vai rodar no terminal (e depois no
Agendador de
Tarefas do Windows), assim:
    python validador.py valores.xlsx --linha B --tolerancia 7.5 --desde
01/10/2026 -v
Antes de validar qualquer coisa, o programa precisa entender esses
argumentos.

Tarefa:
1. `data_br(texto)` converte "dd/mm/aaaa" em date. Se o texto for
inválido, levante
   argparse.ArgumentTypeError com uma mensagem que contenha "dd/mm/aaaa".
   (O argparse transforma esse erro numa mensagem amigável e encerra com
código 2.)
2. `criar_parser()` devolve um argparse.ArgumentParser com
prog="validador" e:
   - `planilha`: argumento posicional obrigatório (texto);
   - `--linha`: uma de A, B, C, D; aceite minúscula (type=str.upper);
padrão None;
   - `--tolerancia`: float, padrão 5.0;
   - `--desde`: data no formato brasileiro (type=data_br), padrão None;
   - `-v` / `--verbose`: liga/desliga (action="store_true").
```

```
Exemplos:
    >>> args = criar_parser().parse_args(["valores.xlsx", "--linha",
"b"])
    >>> args.planilha, args.linha, args.tolerancia, args.desde,
args.verbose
    ('valores.xlsx', 'B', 5.0, None, False)

Dica: nos testes, parse_args recebe uma LISTA de textos, como se fosse o
terminal.
Argumento inválido faz o argparse chamar sys.exit(2): isso é um
SystemExit.

Teste com:  python curso.py testar 07 ex10
```

Exercício 07.11 — Programa de terminal completo: relatório de CV por linha

Arquivo: `exercicios/ex11_cli_relatorio.py`

```
Nível: ★★★

Contexto: Você quer rodar, todo fim de turno,
    python relatorio.py ensaios.csv --ensaio espessura --cv-max 5
e ver se alguma linha está instável. O CV máximo padrão pode vir de uma
variável de
ambiente (LAB_CV_MAX), para cada computador do laboratório ter o seu.

O CSV (separador ";", vírgula decimal, UTF-8) tem as colunas:
    data;op;linha;ensaio;valor
    01/10/2026;OP245101;A;espessura;20,4
(pode ter colunas a mais, como "resultado": ignore-as.)

Tarefa:
1. `ler_registros(caminho)` lê o CSV e devolve uma lista de dicionários
com
   "data" (date), "op" (str), "linha" (str), "ensaio" (str) e "valor"
(float).
   Se o arquivo não existir, deixe o FileNotFoundError acontecer.
2. `criar_parser(ambiente=None)` devolve um ArgumentParser
(prog="relatorio") com:
   - `arquivo`: posicional, type=Path;
   - `--ensaio`: texto, padrão "espessura";
   - `--cv-max`: float; o padrão é float(ambiente["LAB_CV_MAX"]) se essa
chave
     existir, senão 5.0. Se `ambiente` for None, use os.environ.
3. `main(argv=None, ambiente=None)` junta tudo:
   - arquivo inexistente: escreva "Arquivo não encontrado: <arquivo>" no
sys.stderr
```

```
     e devolva 2;
   - nenhum registro do ensaio pedido: escreva "Nenhum registro de
<ensaio>" no
     sys.stderr e devolva 2;
   - para cada linha (em ordem alfabética), imprima EXATAMENTE:
       Linha A: n=3 média=20.10 desvio=0.30 CV=1.49%
     (desvio amostral; com 1 valor só, desvio e CV valem 0.0) e
acrescente
     " ALERTA" no fim quando o CV passar de cv_max (maior estrito);
   - devolva 1 se houve algum ALERTA, senão 0.

Exemplo de execução:
    > python relatorio.py ensaios.csv --cv-max 2
    Linha A: n=3 média=20.10 desvio=0.30 CV=1.49%
    Linha C: n=2 média=42.00 desvio=2.83 CV=6.73% ALERTA

Dica: termine o arquivo com o bloco abaixo; ele não roda nos testes (que
chamam
main direto), só quando o arquivo é executado no terminal:
    if __name__ == "__main__":
        sys.exit(main())

Teste com:  python curso.py testar 07 ex11
```

Exercício 07.12 — Revisão: o e-mail da expedição com datas e agrupamento

Arquivo: `exercicios/ex12_revisao_expedicao.py`

```
Nível: ★★★

Revisão de: módulo 05 (regex e números brasileiros), módulo 03 (listas,
tuplas e
dicionários) e módulo 04 (funções pequenas que se reaproveitam), junto
com datetime
e defaultdict deste módulo.

Contexto: A expedição manda um e-mail assim:

    Bom dia! Segue a programação.
    Data de embarque: 05/11/2026
    OP 245101 - NF 123456 - Peso: 1.250,5 kg
    op 245102 - nf 123456 - peso: 980,00 kg
    OP 245107 - NF 123460 - Peso: 2.010,75 kg
    Att., Expedição

Os certificados precisam estar prontos ANTES do embarque.
```

```
Tarefa:
1. `extrair_itens(texto)` devolve uma lista de tuplas (op, nf, peso_kg),
na ordem do
   e-mail: op no formato "OP245101" (sempre "OP" maiúsculo + dígitos), nf
só com os
   dígitos ("123456") e peso como float (o texto usa formato brasileiro:
"1.250,5").
   Não diferencie maiúsculas/minúsculas e aceite espaços variados em
volta do "-".
2. `extrair_data_embarque(texto)` devolve a date de "Data de embarque:
dd/mm/aaaa"
   (sem diferenciar maiúsculas) ou None se não houver.
3. `prazo_certificado(texto, dias_antes=2)` devolve a data limite para
emitir os
   certificados: embarque − dias_antes dias corridos. Se cair num sábado
ou domingo,
   recue até a sexta-feira anterior. Sem data de embarque, devolva None.
4. `total_por_nf(texto)` devolve {nf: peso total} (round(..., 2)), na
ordem em que as
   NFs aparecem.

Exemplos (com o e-mail acima na variável `email`):
    extrair_itens(email)[1]        -> ("OP245102", "123456", 980.0)
    extrair_data_embarque(email)   -> date(2026, 11, 5)
    prazo_certificado(email)       -> date(2026, 11, 3)
    prazo_certificado(email, 4)    -> date(2026, 10, 30)   (01/11 é
domingo)
    total_por_nf(email)            -> {"123456": 2230.5, "123460":
2010.75}

Dica: monte o padrão do item aos poucos, testando no REPL (como no módulo
05).
re.findall com 3 grupos devolve uma lista de tuplas de 3 textos.

Teste com:  python curso.py testar 07 ex12
```

Exercício 07.13 — Revisão: do CSV do laboratório ao JSON da expedição

Arquivo: `exercicios/ex13_revisao_relatorio_csv.py`

```
Nível: ★★★

Revisão de: módulo 06 (csv com ";", json, pathlib, tratar ValueError),
módulo 05
(números no formato brasileiro) e módulo 03 (dicionários aninhados),
junto com
defaultdict e datetime deste módulo.
```

```
Contexto: A expedição quer um arquivo JSON dizendo, para cada OP, se ela
está
aprovada (todos os ensaios aprovados) ou reprovada, e quais ensaios
reprovaram.
A planilha exportada pelo laboratório às vezes vem com linhas estragadas.

O CSV (UTF-8, separador ";") tem as colunas:
    data;op;linha;ensaio;valor;resultado
    01/10/2026;OP245101;A;espessura;20,4;aprovado

Tarefa:
1. `ler_ensaios_csv(caminho)` devolve uma tupla (registros, ignoradas):
   - registros: lista de dicionários com "data" (date), "op", "linha",
"ensaio",
     "valor" (float; o texto pode ter milhar: "1.250,5") e "resultado"
(minúsculo);
   - linhas com data ou valor inválidos (que geram ValueError na
conversão) NÃO
     entram na lista: só são contadas em `ignoradas`.
2. `resumo_por_op(registros)` devolve um dicionário com as OPs em ordem
alfabética:
       {"OP245101": {"linha": "A", "data": "01/10/2026", "ensaios": 3,
                     "reprovados": [], "situacao": "aprovada"}, ...}
   - "data": a data MAIS ANTIGA dos ensaios da OP, no formato dd/mm/aaaa;
   - "ensaios": quantos registros a OP tem;
   - "reprovados": nomes dos ensaios com resultado "reprovado", em ordem
alfabética;
   - "situacao": "reprovada" se houver algum reprovado, senão "aprovada".
3. `salvar_json(dados, caminho)` grava o dicionário em JSON (UTF-8,
indent=2,
   ensure_ascii=False), criando as pastas que faltarem no caminho.

Exemplo de JSON gerado:
    {
      "OP245103": {
        "linha": "C",
        "data": "05/10/2026",
        "ensaios": 3,
        "reprovados": ["espessura"],
        "situacao": "reprovada"
      }
    }

Dica: agrupe os registros por OP com defaultdict(list) e depois monte o
resumo de
cada grupo. min() funciona com datas.

Teste com:  python curso.py testar 07 ex13
```

Desafio do Módulo 07 — Agenda de validade de laudos

Contexto

Todo laudo de qualidade vence. Hoje alguém do laboratório abre a planilha de laudos, calcula na mão quanto falta para cada um vencer e avisa a expedição. Esquecer um laudo vencido significa material parado no cliente.

Seu desafio: um **programa de linha de comando** que lê a lista de laudos e mostra, numa olhada, o que já venceu e o que vence nos próximos dias. Ele junta quase tudo do módulo: `datetime`, `collections`, `argparse`, `os`, `sys` e organização em funções (ou em um pacote, no extra).

Os dados estão em `../dados/laudos.csv` (UTF-8, separador `;`):

```
op;cliente;produto;emissao;validade_dias
OP244801;Alimentos Bom Sabor;Filme PE 25 µm;02/04/2026;180
OP244930;Café Montanha;Laminado BOPP/PE;10/09/2025;365
...
```

Requisitos obrigatórios

1. Ler o CSV e calcular o **vencimento** de cada laudo (emissão + validade em dias).
2. Aceitar a **data de referência** com `--hoje dd/mm/aaaa`. Sem a opção, usar a data de hoje do computador.
3. Separar os laudos em três grupos:
   - **vencidos**: vencimento antes da data de referência;
   - **a vencer**: vencem em até N dias (`--janela N`, inclusive);
   - **em dia**: o resto.
4. A janela padrão vem da variável de ambiente `LAB_JANELA_DIAS`. Se ela não existir, use 30 dias.
5. Mostrar os vencidos e os a vencer **em ordem de vencimento**, com OP, cliente, data de vencimento e um texto amigável ("amanhã", "em 7 dias", "há 3 dias"). Para os a vencer, mostrar também o dia da semana em português.
6. Mostrar um resumo **por cliente** com quantos vencidos e quantos a vencer (use `Counter`).
7. Arquivo inexistente: mensagem no `stderr` e código de saída 2. Data inválida em `--hoje`: mensagem amigável do argparse (código 2).
8. Com `--estrito`, terminar com código de saída 1 quando houver laudo vencido (assim o Agendador de Tarefas do Windows marca a execução como "falha" e alguém olha).

9. Funções pequenas com docstring, `main(argv=None)` e `if __name__ == "__main__": sys.exit(main())` no final.

Extras opcionais

- `--cliente texto`: filtra pelos clientes cujo nome contém o texto (sem diferenciar maiúsculas).
- `--json caminho`: também grava o resultado em JSON (datas como texto `dd/mm/aaaa`).
- Linhas com data ou validade inválidas são ignoradas com um aviso no `stderr`, sem derrubar o programa.
- Transformar o programa num **pacote** `agenda_laudos/` com `datas.py`, `leitura.py`, `relatorio.py`, `cli.py` e `__main__.py`, e rodar com `python -m agenda_laudos`.
- Avisar quando o vencimento cair num sábado ou domingo (a renovação tem de ser feita antes).

Exemplo de execução

```
> python 07-modulos-e-biblioteca-padrao/desafio/solucao_referencia.py --
hoje 29/10/2026
Agenda de laudos em 29/10/2026 (janela: 30 dias)

VENCIDOS (5)
  OP244930  Café Montanha          venceu em 10/09/2026 (há 49 dias)
  OP244801  Alimentos Bom Sabor    venceu em 29/09/2026 (há 30 dias)
  OP244815  Alimentos Bom Sabor    venceu em 17/10/2026 (há 12 dias)
  OP245066  Pet Nutri              venceu em 26/10/2026 (há 3 dias)
  OP245094  Higiene Total          venceu em 27/10/2026 (há 2 dias)

VENCEM EM ATÉ 30 DIAS (6)
  OP245010  Higiene Total          vence em 30/10/2026, sexta (amanhã)
  OP245041  Laticínios Serra Azul  vence em 30/10/2026, sexta (amanhã)
  OP245058  Alimentos Bom Sabor    vence em 30/10/2026, sexta (amanhã)
  OP244977  Café Montanha          vence em 05/11/2026, quinta (em 7
dias)
  OP245022  Higiene Total          vence em 10/11/2026, terça (em 12
dias)
  OP244902  Laticínios Serra Azul  vence em 11/11/2026, quarta (em 13
dias)

Em dia: 3 laudo(s)

Por cliente (vencidos / a vencer):
  Alimentos Bom Sabor    2 / 1
  Café Montanha          1 / 1
```

```
  Higiene Total          1 / 2
  Laticínios Serra Azul  0 / 2
  Pet Nutri              1 / 0
```

Com filtro e janela menor:

```
> python 07-modulos-e-biblioteca-padrao/desafio/solucao_referencia.py --
hoje 29/10/2026 --janela 7 --cliente café
Agenda de laudos em 29/10/2026 (janela: 7 dias)

VENCIDOS (1)
  OP244930  Café Montanha          venceu em 10/09/2026 (há 49 dias)

VENCEM EM ATÉ 7 DIAS (1)
  OP244977  Café Montanha          vence em 05/11/2026, quinta (em 7
dias)

Em dia: 1 laudo(s)

Por cliente (vencidos / a vencer):
  Café Montanha          1 / 1
```

Para ver o código de saída no PowerShell, rode o programa e depois `$LASTEXITCODE` (no Linux/macOS: `echo $?`).

Critérios de aceite

- ☐ Roda sem argumentos (usa `dados/laudos.csv` e a data de hoje).
- ☐ Com `--hoje 29/10/2026`, mostra 5 vencidos, 6 a vencer e 3 em dia.
- ☐ Os grupos aparecem em ordem de vencimento.
- ☐ `--janela 7` muda a lista "a vencer"; sem `--janela`, vale `LAB_JANELA_DIAS` ou 30.
- ☐ `--hoje 2026-10-29` mostra a mensagem de uso e termina com código 2.
- ☐ Arquivo inexistente: mensagem no `stderr` e código 2, sem traceback.
- ☐ `--estrito` devolve 1 quando há vencidos e 0 quando não há.
- ☐ O código tem `main(argv=None)` e o bloco `if __name__ == "__main__":`.
- ☐ `ruff check` e `ruff format --check` passam.

Dicas para começar

1. **Decomponha antes de digitar.** Escreva no papel as etapas: ler → calcular vencimento → classificar → imprimir. Cada etapa vira uma função.
2. Comece pela função mais simples e teste no REPL: `classificar([...], date(2026, 10, 29), 30)` com dois ou três dicionários feitos à mão.

3. Só depois de as funções funcionarem, escreva o `argparse`. Teste o parser com `criar_parser().parse_args(["--hoje", "29/10/2026"])`, sem precisar do terminal.
4. Para a variável de ambiente, no PowerShell: `$env:LAB_JANELA_DIAS = "15"`; no Linux/macOS: `export LAB_JANELA_DIAS=15`.
5. Travou? Compare com a `solucao_referencia.py` **depois** de tentar. Ela tem um jeito de fazer, não o único.

Quiz do Módulo 07 — Módulos, pacotes e biblioteca padrão

Responda no papel (ou num arquivo) antes de olhar o gabarito. Nas perguntas "o que imprime", tente prever e só depois rode no REPL para conferir.

Perguntas

1. Na pasta do seu script existe um arquivo chamado `statistics.py`, feito por você para testar umas contas. Outro script, na mesma pasta, faz `import statistics` e chama `statistics.mean([1, 2, 3])`. O que acontece?

a) Funciona normalmente: o Python sempre prefere a biblioteca padrão. b) O Python importa o **seu** `statistics.py`, e o `mean` provavelmente não existe nele. c) O Python junta os dois arquivos num só. d) Dá `ModuleNotFoundError`.

2. O que aparece ao rodar `python main.py`?

```
# util.py
print("util:", __name__)
```

```
# main.py
import util

print("main:", __name__)
```

3. Para que serve o arquivo `__init__.py` dentro de uma pasta?

a) É obrigatório em toda pasta do computador que tenha arquivos `.py`. b) Marca a pasta como pacote e roda no primeiro import dele; pode servir de "vitrine". c) Guarda as senhas do projeto. d) Substitui o `requirements.txt`.

4. O que imprime?

```
from datetime import date, timedelta
```

```
print(date(2026, 2, 27) + timedelta(days=2))
```

5. O que imprime?

```
from datetime import date

print((date(2026, 12, 25) - date(2026, 10, 29)).days)
```

6. Onde está o erro? A célula da planilha tem o texto `"2026-10-29"`.

```
from datetime import datetime

emissao = datetime.strptime("2026-10-29", "%d/%m/%Y").date()
```

7. O que imprime?

```
from collections import Counter

print(Counter("abracadabra").most_common(2))
```

8. O que imprime?

```
from itertools import groupby

print([(k, len(list(g))) for k, g in groupby([1, 1, 2, 1])])
```

9. O que imprime?

```
from collections import defaultdict

contagem = defaultdict(int)
contagem["x"] += 1
contagem["y"]
print(len(contagem))
```

10. No laboratório você mede 6 pontos de uma bobina e quer o mesmo desvio padrão que o `DESVPAD.A` do Excel. Qual função usa?

a) `statistics.pstdev` b) `statistics.stdev` c) `statistics.variance` d) `statistics.median`

11. Onde está o erro?

```
import argparse
```

```
parser = argparse.ArgumentParser()
parser.add_argument("--cv-max", type=float, default=5.0)
args = parser.parse_args([])
print(args.cv-max)
```

12. No PowerShell, com o venv ativado, você digita `pip install pandas>=2.2` (sem aspas). O que acontece?

a) Instala o pandas 2.2 ou mais novo. b) Dá erro de sintaxe no pip. c) O `>` vira redirecionamento: o pip instala `pandas` e a saída vai para um arquivo chamado `=2.2`. d) Instala a versão 2.2 exata.

13. O que imprime?

```
import random

print(random.Random(5).randint(1, 100) == random.Random(5).randint(1,
100))
```

Gabarito

1. b. A pasta do script vem **primeiro** no `sys.path`, então o seu `statistics.py` "esconde" o módulo da biblioteca. O resultado é um `AttributeError` (no Python 3.13 a mensagem até sugere renomear o arquivo). Nunca dê a um arquivo seu o nome de um módulo existente.

2.

```
util: util
main: __main__
```

O import executa o `util.py` inteiro, e lá `__name__` vale o nome do módulo. O arquivo rodado diretamente (`main.py`) tem `__name__ == "__main__"`.

3. b. O `__init__.py` transforma a pasta em pacote e roda uma vez, no primeiro import. Pode ficar vazio ou trazer para o nível do pacote as funções públicas (`from .tolerancias import faixa_nominal`).

4. `2026-03-01`. Fevereiro de 2026 tem 28 dias: 27/02 + 1 = 28/02; + 2 = 01/03. O `timedelta` sabe o tamanho de cada mês, você não precisa saber.

5. `57`. São 2 dias até o fim de outubro, 30 de novembro e 25 de dezembro. Subtrair datas dá um `timedelta`; `.days` é o número de dias.

6. O formato não combina com o texto: o dado está em ISO (`ano-mês-dia`) e o formato pede `dia/mês/ano`. Dá `ValueError: time data '2026-10-29' does not match format '%d/%m/%Y'`. Use `"%Y-%m-%d"` ou, mais simples, `date.fromisoformat("2026-10-29")`.

7. `[('a', 5), ('b', 2)]`. O "a" aparece 5 vezes. "b" e "r" empatam com 2; no empate, o `most_common` mantém a ordem em que apareceram, e o "b" veio antes.

8. `[(1, 2), (2, 1), (1, 1)]`. O `groupby` junta só **vizinhos** iguais. O último `1` forma um grupo novo. Para agrupar todos os `1`, ordene antes.

9. `2`. A linha `contagem["y"]`, mesmo só lendo, cria a chave `"y"` com valor `0`. Para consultar sem criar, use `"y" in contagem` ou `contagem.get("y")`.

10. b. `stdev` é o desvio **amostral** (divide por n − 1), igual ao `DESVPAD.A` (e ao `DESVPAD` padrão). `pstdev` é o populacional (`DESVPAD.P`). `variance` é o desvio ao quadrado.

11. O Python lê `args.cv-max` como `args.cv` **menos** `max`. Como não existe `args.cv`, dá `AttributeError: 'Namespace' object has no attribute 'cv'`. O argparse troca o hífen por sublinhado no nome do atributo: use `args.cv_max`.

12. c. No terminal, `>` é redirecionamento de saída. O pip recebe só `pandas` e o texto que ele imprimiria vai para um arquivo `=2.2`. Escreva `python -m pip install "pandas>=2.2"`, com aspas.

13. `True`. Dois geradores com a **mesma semente** produzem a mesma sequência. É isso que torna um sorteio reproduzível (e testável).

Módulo 08 — Orientação a objetos

> **Tempo estimado:** 10 a 14 horas · **Pré-requisitos:** módulos 00 a 07 · **Nível:** intermediário

O que você vai aprender

- Criar uma classe com `__init__`, atributos e métodos, e explicar o que é `self`.
- Diferenciar atributo de instância de atributo de classe (e evitar a armadilha da lista compartilhada).
- Escrever `__str__` e `__repr__` para objetos legíveis no `print` e no REPL.
- Proteger dados com a convenção `_` e com `@property` que valida.
- Criar construtores alternativos com `@classmethod` e utilitários com `@staticmethod`.
- Reaproveitar código com herança e `super()`, e tratar objetos diferentes do mesmo jeito (polimorfismo).
- Escolher entre herança ("é um") e composição ("tem um").
- Escrever classes de dados com `@dataclass` (`field`, `frozen`, `order`, `__post_init__`).
- Definir igualdade (`__eq__` e `__hash__`) e conjuntos fechados de valores com `Enum`.
- Reconhecer quando uma função ou um dicionário resolvem melhor que uma classe.

Por que isso importa

O seu sistema fala de **amostras, tolerâncias, especificações, linhas de produção, pedidos e certificados**. Até aqui, cada uma dessas coisas era um dicionário solto e um punhado de funções espalhadas. Funciona até o dia em que alguém escreve `amostra["valro"]`, ou passa a tolerância da linha B para a OP da linha C.

Orientação a objetos é um jeito de organizar o código **em torno das coisas do problema**: cada coisa vira um tipo (uma classe) que guarda os seus dados e sabe fazer o que lhe cabe. Uma `Tolerancia` sabe dizer se aprova um valor; uma `LinhaDeProducao` sabe qual especificação vale para cada OP; um `Certificado` sabe se pode ser emitido.

No mercado, OO está em toda parte: o `DataFrame` do pandas, o `Path` do `pathlib`, as respostas do `requests` e os modelos do FastAPI (módulo 13) são objetos. Entender classes é entender as bibliotecas que você usa. E saber **quando não usar** classe é o que separa quem aprendeu OO de quem decorou OO.

1. O problema: dados soltos

Lembra do aviso sobre dicionários? Rode `python exemplos/01_dict_vs_classe.py`:

```
amostra = {"codigo": "AM-001", "ensaio": "espessura", "valor": 25.3}
amostra["valro"] = 25.1  # erro de digitação: cria uma chave NOVA, em silêncio
print("Valor usado no laudo:", amostra["valor"])
```

Saída:

```
Valor usado no laudo: 25.3
```

A correção se perdeu e ninguém foi avisado. Com dicionários, nada garante que toda amostra tem as mesmas chaves, nada impede valor negativo, e as funções que trabalham com amostras ficam espalhadas pelo código.

Uma **classe** resolve as três coisas: define quais dados uma amostra tem, pode validar esses dados e junta as funções que operam sobre eles (os **métodos**). Pense na ficha de amostra do laboratório: a **ficha em branco** (com os campos impressos) é a classe; cada **ficha preenchida** é um objeto.

2. Classes e objetos

```
class Amostra:
    def __init__(self, codigo, ensaio, valor):
        self.codigo = codigo
        self.ensaio = ensaio
        self.valor = valor

    def esta_entre(self, minimo, maximo):
        return minimo <= self.valor <= maximo


a = Amostra("AM-001", "espessura", 25.3)
b = Amostra("AM-002", "espessura", 28.1)
print(a.codigo, a.valor, a.esta_entre(22.5, 27.5))
print(b.codigo, b.valor, b.esta_entre(22.5, 27.5))
```

```
print(type(a).__name__, isinstance(a, Amostra))
try:
    print(a.valro)
except AttributeError as erro:
    print("Erro:", erro)
```

Saída:

```
AM-001 25.3 True
AM-002 28.1 False
Amostra True
Erro: 'Amostra' object has no attribute 'valro'
```

O erro de digitação agora **aparece na hora** (sem o `try`, o traceback ainda sugere `Did you mean: 'valor'?`).

Vocabulário (vale para qualquer linguagem):

Termo	O que é	No exemplo
classe	o molde, o tipo	`Amostra`
objeto ou **instância**	uma coisa criada a partir do molde	`a`, `b`
atributo	um dado guardado no objeto	`a.codigo`, `a.valor`
método	uma função que pertence à classe	`a.esta_entre(...)`
instanciar	criar um objeto chamando a classe	`Amostra("AM-001", ...)`

Por convenção, nomes de classe usam **PascalCase** (`LinhaDeProducao`), e o resto continua em `snake_case`. Você já usa objetos desde o módulo 01: `"texto".upper()` é o método `upper` de um objeto do tipo `str`; `Path("dados").exists()`, idem.

> 💡 **Dica:** você já escreveu uma classe no módulo 06: `class ErroDeQualidade(Exception):`. Aquilo era uma classe que **herda** de `Exception`. A seção 9 explica o que isso quer dizer.

3. `__init__` e `self`

`__init__` é o método que o Python chama **logo depois de criar** o objeto, para prepará-lo. É ali que você guarda os dados iniciais. Os dois sublinhados de cada lado indicam um **método especial** ("dunder", de *double underscore*): você não o chama diretamente; o Python chama na hora certa.

E o `self`? É o **próprio objeto** sobre o qual o método está trabalhando. Quando você escreve `a.esta_entre(22.5, 27.5)`, o Python faz, por trás:

```
Amostra.esta_entre(a, 22.5, 27.5)
```

Ou seja: `self` recebe `a`. Por isso todo método começa com `self`, e é por `self` que o método enxerga os atributos (`self.valor`). O nome `self` é convenção, não palavra reservada, mas **sempre** use `self`.

🔮 **Preveja antes de rodar:** o que acontece?

```
class Bobina:
    def __init__(self, lote, peso_kg):
        self.lote = lote
        self.peso_kg = peso_kg

    def resumo():
        return f"{self.lote}: {self.peso_kg} kg"


print(Bobina("L01", 150.0).resumo())
```

▼ **Resposta**

Dá erro: `TypeError: Bobina.resumo() takes 0 positional arguments but 1 was given`. O Python passou o objeto como primeiro argumento (o `self`), mas o método não tinha nenhum parâmetro para recebê-lo. É o erro mais comum de quem está começando com classes. Correção: `def resumo(self):`.

Um `__init__` bem escrito **normaliza e valida** os dados na entrada. Assim, todo objeto que existe já está correto:

```
class Amostra:
    def __init__(self, codigo: str, ensaio: str, valor: float, unidade:
str = "") -> None:
        self.codigo = codigo.strip().upper()
        self.ensaio = ensaio.strip().lower()
        self.valor = float(valor)
        self.unidade = unidade
```

```
a = Amostra(" am-001 ", "Espessura", 25, "µm")
print(a.codigo, a.ensaio, a.valor, a.unidade)
```

Saída:

```
AM-001 espessura 25.0 µm
```

Os parâmetros com padrão (`unidade=""`), as anotações de tipo e as docstrings funcionam exatamente como nas funções do módulo 04. O `__init__` devolve sempre `None` (por isso o `-> None`): quem "devolve" o objeto é a chamada `Amostra(...)`.

4. Métodos e atributos de classe

Métodos podem chamar outros métodos do mesmo objeto, sempre pelo `self`:

```
class Amostra:
    def __init__(self, codigo, valor, unidade=""):
        self.codigo = codigo
        self.valor = valor
        self.unidade = unidade

    def esta_entre(self, minimo, maximo):
        return minimo <= self.valor <= maximo

    def situacao(self, minimo, maximo):
        return "aprovada" if self.esta_entre(minimo, maximo) else
"reprovada"


print(Amostra("AM-001", 25.3).situacao(22.5, 27.5))
```

Saída:

```
aprovada
```

Até aqui, todo atributo foi criado com `self.algo = ...` dentro de um método: é um **atributo de instância**, um por objeto. Um nome criado **no corpo da classe**, fora dos métodos, é um **atributo de classe**: existe **um só**, compartilhado por todos os objetos. Serve para constantes do tipo:

```
>>> class Amostra:
...     laboratorio = "CQ Embalagens"
...     def __init__(self, codigo):
```

```
...         self.codigo = codigo
...
>>> a, b = Amostra("AM-001"), Amostra("AM-002")
>>> a.laboratorio, b.laboratorio, Amostra.laboratorio
('CQ Embalagens', 'CQ Embalagens', 'CQ Embalagens')
```

🔮 **Preveja antes de rodar:** quanto vale cada contador?

```python
class Laudo:
    emitidos = 0

    def __init__(self):
        self.emitidos += 1


a = Laudo()
b = Laudo()
print(a.emitidos, b.emitidos, Laudo.emitidos)
```

▼ **Resposta**

`1 1 0`. O `self.emitidos += 1` é `self.emitidos = self.emitidos + 1`: a **leitura** encontra o atributo da classe (0), mas a **atribuição** cria um atributo **novo** no objeto. Cada laudo ganhou o seu `emitidos = 1`, e o da classe continuou 0. Para contar na classe, escreva `Laudo.emitidos += 1`.

⚠️ **Erro comum:** colocar uma lista no corpo da classe (`bobinas = []`) achando que cada objeto terá a sua. Ela é **uma só**: `self.bobinas.append(...)` não atribui, só altera a lista compartilhada, e todos os objetos enxergam a mudança. Lista, dicionário e conjunto de cada objeto nascem **no** `__init__`: `self.bobinas = []`. O exercício 09 é exatamente esse bug.

5. `__str__` e `__repr__`

Imprima um objeto sem esses métodos e você verá algo como `<__main__.Amostra object at 0x7f3a2c1b9e50>`: o tipo e o endereço na memória. Inútil. Dois métodos especiais resolvem:

- `__str__`: texto **para pessoas**. Usado por `print()`, `str()` e f-strings.

- `__repr__`: texto **para programadores**. Usado no REPL, dentro de listas, no `!r` das f-strings e em logs. Regra de ouro: parecer o código que recria o objeto.

```
class Bobina:
    def __init__(self, lote, numero, peso_kg):
        self.lote = lote
        self.numero = numero
        self.peso_kg = peso_kg

    def __str__(self):
        return f"Bobina {self.lote}-{self.numero:03d} ({self.peso_kg:.1f}
kg)"

    def __repr__(self):
        return f"Bobina(lote={self.lote!r}, numero={self.numero!r},
peso_kg={self.peso_kg!r})"


b = Bobina("L2610", 7, 152.4)
print(b)
print(f"Separada: {b}")
print([b])
print(repr(b))
```

Saída:

```
Bobina L2610-007 (152.4 kg)
Separada: Bobina L2610-007 (152.4 kg)
[Bobina(lote='L2610', numero=7, peso_kg=152.4)]
Bobina(lote='L2610', numero=7, peso_kg=152.4)
```

Repare no `[b]`: dentro de uma lista o Python usa o `__repr__` de cada item. Se você só puder escrever um dos dois, escreva o `__repr__`: sem `__str__`, o `print` usa o `__repr__`.

> 🔧 **Modifique:** apague o `__str__` da `Bobina` e rode de novo. O `print(b)` passa a mostrar o texto do `__repr__`. Depois apague também o `__repr__` e veja o `<__main__.Bobina object at 0x...>` voltar.

6. Encapsulamento: o que é de dentro e o que é de fora

Encapsular é esconder os detalhes internos e oferecer um jeito seguro de usar o objeto. O Python não tem "privado" de verdade; ele usa **convenções**:

Nome	Significa	Exemplo
`valor`	público: use à vontade	`amostra.valor`
`_valor`	interno: "não mexa de fora, pode mudar sem aviso"	`amostra._historico`
`__valor`	interno com nome embaralhado (*name mangling*)	raro; evite

Com `__valor`, o Python renomeia o atributo para `_Amostra__valor`, para evitar conflito em heranças. Não é segurança, e quase nunca é necessário: no dia a dia, um sublinhado basta. O lema da comunidade é "somos todos adultos": o `_` avisa, e quem mexe assume o risco.

7. `@property`: atributo com validação

Problema real: alguém digitou `-25` na espessura. Você quer que **toda** atribuição a `amostra.valor` seja validada, sem obrigar quem usa a trocar `a.valor = 30` por `a.definir_valor(30)`. A `@property` faz um método se parecer com um atributo:

```
class Amostra:
    def __init__(self, codigo, valor):
        self._codigo = codigo.upper()
        self.valor = valor  # já passa pelo setter: validação desde o
início

    @property
    def codigo(self):  # só getter: atributo somente leitura
        return self._codigo

    @property
    def valor(self):  # getter: roda em "a.valor"
        return self._valor

    @valor.setter
    def valor(self, novo):  # setter: roda em "a.valor = ..."
        if novo < 0:
            raise ValueError(f"Valor não pode ser negativo: {novo}")
        self._valor = float(novo)


a = Amostra("am-001", 25.4)
a.valor = 30
print(a.codigo, a.valor)
try:
    a.valor = -5
except ValueError as erro:
    print("ValueError:", erro)
```

```
try:
    a.codigo = "X"
except AttributeError as erro:
    print("AttributeError:", erro)
```

Saída:

```
AM-001 30.0
ValueError: Valor não pode ser negativo: -5
AttributeError: property 'codigo' of 'Amostra' object has no setter
```

Os pontos-chave:

- O dado de verdade mora em `self._valor` (com `_`); a property `valor` é a porta de entrada. Se a property e o atributo tivessem o mesmo nome, o setter chamaria a si mesmo para sempre (`RecursionError`).
- No `__init__`, `self.valor = valor` (sem `_`) passa pelo setter: nenhum objeto nasce inválido.
- Property **sem setter** é somente leitura. (No Python 3.10, a mensagem é só `can't set attribute`.)
- Property também serve para valores **calculados**, sem guardar nada: `@property def em_mils(self): return self._valor / 25.4`.

Exemplo resolvido: peso líquido da bobina

Problema: a balança pesa a bobina **com** o tubete. O certificado precisa do peso líquido. O peso bruto não pode ser menor que o tubete, e ninguém deve conseguir "setar" o líquido na mão.

Como eu pensei: o bruto é um dado que pode ser corrigido (precisa de setter com validação). O tubete é fixo por bobina. O líquido **não é um dado**: é uma conta sobre os outros dois, então é uma property só com getter. Assim ele nunca fica desatualizado.

```
class Bobina:
    def __init__(self, lote, peso_bruto_kg, tubete_kg=2.5):
        self.lote = lote
        self.tubete_kg = tubete_kg
        self.peso_bruto_kg = peso_bruto_kg

    @property
    def peso_bruto_kg(self):
        return self._peso_bruto_kg

    @peso_bruto_kg.setter
```

```
    def peso_bruto_kg(self, novo):
        if novo <= self.tubete_kg:
            raise ValueError(f"Peso bruto {novo} kg não passa do tubete
({self.tubete_kg} kg)")
        self._peso_bruto_kg = float(novo)

    @property
    def peso_liquido_kg(self):
        return round(self._peso_bruto_kg - self.tubete_kg, 2)


b = Bobina("L2610", 154.9)
print(b.peso_liquido_kg)
b.peso_bruto_kg = 160.0
print(b.peso_liquido_kg)
```

Saída:

```
152.4
157.5
```

Repare na ordem do `__init__`: `tubete_kg` é guardado **antes** do peso bruto, porque o setter do bruto usa o tubete.

> 🔧 **Modifique:** tente `b.peso_liquido_kg = 100`. Leia o erro: ele diz que a property não tem setter. É exatamente o que queríamos.

8. `@classmethod` e `@staticmethod`

Tolerâncias chegam de jeitos diferentes: "0,10 a 0,30", "25 ± 10%", números do banco. Um `__init__` cheio de "se o formato for X" fica confuso. A saída são **construtores alternativos**, com `@classmethod`:

- um método comum recebe o **objeto** (`self`);
- um `@classmethod` recebe a **classe** (`cls`) e costuma devolver um objeto novo;
- um `@staticmethod` não recebe nem um nem outro: é uma função comum que mora dentro da classe porque tem a ver com ela.

```
class Tolerancia:
    def __init__(self, minimo, maximo):
        self.minimo = minimo
        self.maximo = maximo

    @classmethod
```

```
    def por_nominal(cls, nominal, percentual):
        margem = nominal * percentual / 100
        return cls(nominal - margem, nominal + margem)

    @classmethod
    def de_texto(cls, texto):  # "0,10 a 0,30"
        minimo, maximo = texto.split(" a ")
        return cls(cls.numero_br(minimo), cls.numero_br(maximo))

    @staticmethod
    def numero_br(texto):
        return float(texto.strip().replace(".", "").replace(",", "."))

    def __repr__(self):
        return f"Tolerancia({self.minimo!r}, {self.maximo!r})"


print(Tolerancia(0.10, 0.30))
print(Tolerancia.por_nominal(25, 10))
print(Tolerancia.de_texto("0,10 a 0,30"))
print(Tolerancia.numero_br("1.250,5"))
```

Saída:

```
Tolerancia(0.1, 0.3)
Tolerancia(22.5, 27.5)
Tolerancia(0.1, 0.3)
1250.5
```

Por que `cls(...)` e não `Tolerancia(...)`? Porque, se alguém criar `class ToleranciaEspessura(Tolerancia)`, o `ToleranciaEspessura.por_nominal(...)` devolve um objeto da **subclasse**. Você já usou classmethods sem saber: `date.today()`, `datetime.strptime(...)` e `dict.fromkeys(...)` são construtores alternativos.

> 💡 **Dica:** está em dúvida se um método deve ser `@staticmethod`? Se ele não usa `self` nem `cls`, provavelmente sim. Se ele também não tem nada a ver com a classe, provavelmente deveria ser uma função comum, fora dela.

9. Herança e `super()`

Espessura, tração e COF são todos **ensaios**: têm nome, valor e unidade e geram uma linha de laudo. Só a regra de aprovação muda. **Herança** permite escrever o que é comum uma vez (na classe-mãe) e só a diferença nas classes-filhas:

```python
class Ensaio:
    def __init__(self, nome, valor, unidade=""):
        self.nome = nome
        self.valor = valor
        self.unidade = unidade

    def aprovado(self):
        raise NotImplementedError("Cada tipo de ensaio define a sua
regra")

    def laudo(self):
        situacao = "APROVADO" if self.aprovado() else "REPROVADO"
        return f"{self.nome}: {self.valor} {self.unidade} -> {situacao}"


class EnsaioMinimo(Ensaio):  # "EnsaioMinimo É UM Ensaio"
    def __init__(self, nome, valor, minimo, unidade=""):
        super().__init__(nome, valor, unidade)  # a mãe cuida do que é
comum
        self.minimo = minimo  # a filha guarda só o que é dela

    def aprovado(self):  # sobrescreve (override) o método da mãe
        return self.valor >= self.minimo


tracao = EnsaioMinimo("tração", 18.0, 20.0, "MPa")
print(tracao.laudo())
print(isinstance(tracao, EnsaioMinimo), isinstance(tracao, Ensaio))
print([classe.__name__ for classe in EnsaioMinimo.__mro__])
```

Saída:

```
tração: 18.0 MPa -> REPROVADO
True True
['EnsaioMinimo', 'Ensaio', 'object']
```

- `class EnsaioMinimo(Ensaio)`: a filha **herda** todos os métodos da mãe (`laudo` veio de graça).
- `super().__init__(...)` chama o `__init__` da mãe. Sem ele, `self.nome` nunca seria criado: a filha que define `__init__` **substitui** o da mãe.
- **Sobrescrever** (*override*): a filha define um método com o mesmo nome; o dela vale.
- O `__mro__` (*method resolution order*) é a ordem em que o Python procura um método: primeiro na filha, depois na mãe, por fim em `object`, a mãe de todas as classes.

> ⚠️ **Erro comum:** esquecer o `super().__init__(...)` na filha. Nada reclama na criação; o erro aparece depois, longe dali: `AttributeError: 'EnsaioMinimo' object has no attribute 'nome'`.

Classe abstrata. Faz sentido criar um `Ensaio("x", 1)` genérico? Não. O módulo `abc` deixa isso explícito:

```
from abc import ABC, abstractmethod

class Ensaio(ABC):
    @abstractmethod
    def aprovado(self) -> bool:
        """Toda filha é obrigada a implementar."""
```

Tentar criar um `Ensaio` dá `TypeError: Can't instantiate abstract class Ensaio without an implementation for abstract method 'aprovado'` (a frase muda um pouco entre versões). Veja `exemplos/05_heranca_polimorfismo.py`.

10. Polimorfismo e "duck typing"

Polimorfismo ("muitas formas") é tratar objetos de tipos diferentes **pela mesma interface**. O método `laudo()` da mãe chama `self.aprovado()`, e cada objeto responde com a sua própria regra. Quem usa não precisa de `if tipo == ...`:

```
class EnsaioFaixa:
    def __init__(self, nome, valor, minimo, maximo):
        self.nome, self.valor, self.minimo, self.maximo = nome, valor,
minimo, maximo

    def aprovado(self):
        return self.minimo <= self.valor <= self.maximo


class EnsaioMaximo:
    def __init__(self, nome, valor, maximo):
        self.nome, self.valor, self.maximo = nome, valor, maximo

    def aprovado(self):
        return self.valor <= self.maximo


ensaios = [EnsaioFaixa("espessura", 28.0, 22.5, 27.5),
EnsaioMaximo("cof", 0.22, 0.30)]
print([e.nome for e in ensaios if not e.aprovado()])
```

Saída:

```
['espessura']
```

Repare: essas duas classes **nem herdam** de uma mãe comum, e funciona. O Python não pergunta o tipo; ele só chama `.aprovado()`. É o **duck typing**: "se anda como pato e grasna como pato, é pato". Você já viu isso: `len()` funciona com lista, texto e dicionário, e `for` funciona com qualquer coisa que se deixe percorrer. A herança ajuda a **garantir** a interface (com `ABC`) e a **reaproveitar** código; o polimorfismo em si vem do duck typing.

> 🔧 **Modifique:** crie `class EnsaioMinimo` com `aprovado()` próprio (valor >= mínimo) e acrescente um objeto dela na lista `ensaios`. A última linha não muda nada e continua funcionando.

11. Composição: "tem um" em vez de "é um"

Herança é para "**é um**": um `EnsaioMinimo` **é um** `Ensaio`. Muita coisa do mundo real é "**tem um**": uma linha de produção **tem** uma especificação padrão; um certificado **tem** resultados de ensaio. Isso é **composição**: um objeto guarda outros objetos como atributos e **delega** o trabalho a eles.

```
LinhaDeProducao "B"
├── fixas: {"cof": Tolerancia(0.10, 0.30)}              (tem tolerâncias
fixas)
├── padrao: Especificacao "Filme PE 25 µm"              (tem uma
especificação padrão)
│           └── tolerancias: {"espessura": Tolerancia(22.5, 27.5)}
└── por OP: {"OP245110": Especificacao "Café Montanha"} (tem
especificações próprias)
```

Exemplo resolvido: a regra da especificação por OP

Problema: cada linha tem tolerâncias fixas e uma especificação padrão. Se a OP tem especificação própria, ela vale no lugar da padrão. Preciso saber, para uma OP, quais tolerâncias aplicar.

Como eu pensei:

1. Primeira tentação: `class LinhaDeProducao(Especificacao)`. Mas a frase "uma linha **é uma** especificação" é falsa, e uma linha precisa de **várias** especificações (a padrão e as das OPs). Herança não serve.

2. Então a linha **guarda** uma especificação padrão e um dicionário `{op: Especificacao}`.
3. "Qual especificação vale para a OP?" é um `.get(op, padrao)`.
4. "Quais tolerâncias aplicar?" são as fixas mais as da especificação, e a da especificação vence: `{**fixas, **especificacao.tolerancias}`.

```
class Especificacao:
    def __init__(self, titulo, tolerancias):
        self.titulo = titulo
        self.tolerancias = dict(tolerancias)  # cópia: protege de
mudanças de fora


class LinhaDeProducao:
    def __init__(self, nome, fixas, padrao):
        self.nome = nome
        self.fixas = dict(fixas)
        self.padrao = padrao  # composição: a linha TEM uma especificação
        self._por_op = {}

    def cadastrar(self, op, especificacao):
        self._por_op[op] = especificacao

    def especificacao_para(self, op):
        return self._por_op.get(op, self.padrao)

    def tolerancias_para(self, op):
        return {**self.fixas, **self.especificacao_para(op).tolerancias}


padrao = Especificacao("Filme PE 25 µm", {"espessura": (22.5, 27.5)})
linha_b = LinhaDeProducao("B", {"cof": (0.10, 0.30)}, padrao)
linha_b.cadastrar("OP245110", Especificacao("Café Montanha",
{"espessura": (24.0, 26.0)}))
print(linha_b.especificacao_para("OP245102").titulo,
linha_b.tolerancias_para("OP245102"))
print(linha_b.especificacao_para("OP245110").titulo,
linha_b.tolerancias_para("OP245110"))
```

Saída:

```
Filme PE 25 µm {'cof': (0.1, 0.3), 'espessura': (22.5, 27.5)}
Café Montanha {'cof': (0.1, 0.3), 'espessura': (24.0, 26.0)}
```

(Para o exemplo ficar curto, as tolerâncias aqui são tuplas; no exercício 11 elas são objetos `Tolerancia`.) Prefira composição na dúvida: ela é mais flexível (dá para trocar a especificação de uma linha em tempo de execução) e não amarra uma

classe à outra. Use herança quando a relação "é um" for verdadeira **e** a filha puder ser usada em qualquer lugar onde a mãe é usada.

12. `dataclasses`: classes de dados sem repetição

Muitas classes só **guardam dados**: o `__init__` copia parâmetros para atributos, o `__repr__` lista os campos, o `__eq__` compara campo a campo. A `@dataclass` escreve tudo isso a partir das anotações de tipo:

```
>>> from dataclasses import dataclass
>>> @dataclass
... class ResultadoEnsaio:
...     ensaio: str
...     valor: float
...     unidade: str = ""
...     aprovado: bool = True
...     def rotulo(self) -> str:
...         return f"{self.ensaio}: {self.valor} ({'ok' if self.aprovado
else 'falhou'})"
...
>>> r = ResultadoEnsaio("cof", 0.32, aprovado=False)
>>> r
ResultadoEnsaio(ensaio='cof', valor=0.32, unidade='', aprovado=False)
>>> r == ResultadoEnsaio("cof", 0.32, "", False)
True
>>> r.rotulo()
'cof: 0.32 (falhou)'
```

As anotações são obrigatórias (é por elas que a dataclass acha os campos) e campos com valor padrão vêm depois dos sem padrão, como nos parâmetros de função. Métodos comuns convivem normalmente com os gerados.

Os recursos que você vai usar

```
>>> from dataclasses import dataclass, field, replace, asdict
>>> from datetime import date
>>> @dataclass
... class Bobina:
...     lote: str
...     peso_kg: float
...     defeitos: list[str] = field(default_factory=list)
...     def __post_init__(self):
...         if self.peso_kg <= 0:
...             raise ValueError(f"Peso inválido: {self.peso_kg}")
...
```

```
>>> b = Bobina("L2610", 152.4)
>>> b.defeitos.append("gel")
>>> Bobina("L2611", 150.0).defeitos
[]
>>> asdict(b)
{'lote': 'L2610', 'peso_kg': 152.4, 'defeitos': ['gel']}
>>> Bobina("L2612", 0)
Traceback (most recent call last):
  ...
ValueError: Peso inválido: 0
>>> @dataclass(frozen=True, order=True)
... class Medicao:
...     data: date
...     linha: str
...     valor: float
...     observacao: str = field(default="", compare=False)
...
>>> m1 = Medicao(date(2026, 10, 2), "A", 20.4)
>>> m2 = Medicao(date(2026, 10, 1), "B", 25.2, "redigitada")
>>> sorted([m1, m2])[0].linha
'B'
>>> m1 == Medicao(date(2026, 10, 2), "A", 20.4, "outra observação")
True
>>> m1.valor = 99
Traceback (most recent call last):
  ...
dataclasses.FrozenInstanceError: cannot assign to field 'valor'
>>> replace(m1, valor=20.1).valor
20.1
>>> len({m1, m1, m2})
2
```

Recurso	Para quê
`field(default_factory=list)`	lista (ou dict) **nova** para cada objeto
`__post_init__`	validar ou ajustar logo depois do `__init__` gerado
`frozen=True`	imutável (atribuir dá `FrozenInstanceError`) e "hashable": vai para `set` e vira chave de `dict`
`order=True`	gera `<`, `>`... comparando os campos **na ordem declarada**
`field(compare=False)`	o campo fica fora de `==`, da ordenação e do hash
`asdict(obj)`	vira dicionário (bom para JSON)
`replace(obj, campo=...)`	cópia com um campo trocado (o jeito de "alterar" um frozen)

🔮 **Preveja antes de rodar:**

```python
from dataclasses import dataclass


@dataclass
class Lote:
    codigo: str
    bobinas: list = []
```

<details>
<summary>Resposta</summary>

O erro acontece **na definição da classe**, antes de criar qualquer lote: `ValueError: mutable default <class 'list'> for field bobinas is not allowed: use default_factory`. A dataclass protege você da armadilha do default mutável (módulo 04 e seção 4). O certo é `bobinas: list = field(default_factory=list)`.
</details>

💡 **Dica:** dataclass, `namedtuple` (módulo 07) ou dicionário? Dicionário para dados passageiros e formatos variáveis (uma linha de CSV recém-lida). `namedtuple` para registros pequenos e imutáveis. Dataclass quando houver validação, métodos, valores padrão ou quando o tipo aparecer em vários lugares do sistema.

13. Igualdade: `is`, `==`, `__eq__` e `__hash__`

- `a is b`: são o **mesmo objeto** na memória?
- `a == b`: têm o **mesmo valor**? Quem decide é o método `__eq__` da classe.

Sem `__eq__`, uma classe comum usa a identidade: dois objetos com os mesmos dados são **diferentes**.

🔮 **Preveja antes de rodar:**

```python
class OrdemProducao:
    def __init__(self, numero):
        self.numero = numero


a = OrdemProducao("245101")
```

```
b = OrdemProducao("245101")
print(a == b, a is b, a == a)
```

▼ **Resposta**

`False False True`. Sem `__eq__`, `==` compara identidade: `a` e `b` são duas fichas diferentes, ainda que preenchidas igual. (A dataclass gera um `__eq__` que compara os campos: com `@dataclass`, o primeiro seria `True`.)

Para definir a igualdade você mesmo:

```
>>> class OrdemProducao:
...     def __init__(self, numero):
...         self.numero = "".join(c for c in numero if c.isdigit())
...     def __eq__(self, outro):
...         if not isinstance(outro, OrdemProducao):
...             return NotImplemented
...         return self.numero == outro.numero
...     def __hash__(self):
...         return hash(self.numero)
...
>>> OrdemProducao("OP-245101") == OrdemProducao("op 245101")
True
>>> OrdemProducao("OP-245101") == "245101"
False
>>> len({OrdemProducao("OP-245101"), OrdemProducao("245101"),
OrdemProducao("245102")})
2
```

Duas regras que pegam todo mundo:

1. **Outro tipo → `return NotImplemented`** (não `False`, não erro). É o jeito de dizer "não sei comparar com isso"; o Python tenta o outro lado e, por fim, responde `False`. Sem esse cuidado, `op == "245101"` quebraria com `AttributeError`.
2. **Definiu `__eq__`, defina `__hash__`.** Ao criar `__eq__`, o Python **apaga** o `__hash__` herdado, e o objeto deixa de entrar em `set` e de ser chave de `dict` (`TypeError: unhashable type`). Objetos iguais precisam ter o mesmo hash: use os mesmos campos do `__eq__`. E só faça isso com campos que não mudam depois.

14. `Enum`: conjuntos fechados de valores

Linha de produção é A, B, C ou D. Situação é liberado, reprovado ou pendente. Guardar isso como texto solto convida ao erro (`"Liberado"`, `"liberada"`, `"LIBERADO "`). Uma `Enum` define o conjunto **fechado** de valores possíveis:

```
>>> from enum import Enum
>>> class Situacao(Enum):
...     LIBERADO = "liberado"
...     REPROVADO = "reprovado"
...     PENDENTE = "pendente"
...
>>> s = Situacao.LIBERADO
>>> s
<Situacao.LIBERADO: 'liberado'>
>>> s.name, s.value
('LIBERADO', 'liberado')
>>> Situacao("reprovado")
<Situacao.REPROVADO: 'reprovado'>
>>> Situacao["PENDENTE"]
<Situacao.PENDENTE: 'pendente'>
>>> [membro.value for membro in Situacao]
['liberado', 'reprovado', 'pendente']
>>> s is Situacao.LIBERADO
True
>>> Situacao("liberada")
Traceback (most recent call last):
  ...
ValueError: 'liberada' is not a valid Situacao
```

- `Situacao("reprovado")` procura pelo **valor** (bom para ler da planilha); `Situacao["PENDENTE"]`, pelo **nome**.
- Membros são únicos: compare com `is`.
- Enum pode ter métodos, `@property` e `@classmethod` como qualquer classe (o exercício 05 cria `Situacao.de_texto(...)` e `situacao.pode_emitir`).
- Quando os valores não importam, `from enum import auto` e `LIBERADO = auto()` numera sozinho.

> ⚠️ **Erro comum:** imprimir o membro direto num relatório. `print(Situacao.LIBERADO)` mostra `Situacao.LIBERADO`. Para o texto, use `.value` (ou `.name`). E cuidado com enums que herdam de `str` (`class Situacao(str, Enum)`): no Python 3.10, `f"{s}"` mostra `liberado`; do 3.11 em diante, mostra `Situacao.LIBERADO`. Usando sempre `.value`, o resultado é o mesmo em qualquer versão.

15. Outros métodos especiais

Os "dunders" ligam a sua classe à sintaxe do Python. Os que você já viu e mais alguns:

Você escreve	O Python chama	Exemplo
`Classe(...)`	`__init__`	criar a fila
`print(obj)`, `f"{obj}"`	`__str__`	texto do certificado
REPL, `[obj]`, `f"{obj!r}"`	`__repr__`	depuração
`a == b`	`__eq__`	OPs iguais
`{obj}`, `dict[obj]`	`__hash__`	OP em conjunto
`a < b`, `sorted(...)`	`__lt__` (e companhia)	ordenar medições
`len(obj)`	`__len__`	quantos pedidos na fila
`for x in obj`	`__iter__`	percorrer a fila
`x in obj`	`__contains__`	"essa NF está na fila?"

```python
class FilaDeEmissao:
    def __init__(self):
        self._pedidos = []

    def adicionar(self, op):
        self._pedidos.append(op)

    def __len__(self):
        return len(self._pedidos)

    def __iter__(self):
        return iter(self._pedidos)  # iter() devolve um iterador da lista

    def __contains__(self, op):
        return op in self._pedidos


fila = FilaDeEmissao()
fila.adicionar("OP245101")
fila.adicionar("OP245118")
print(len(fila), "OP245118" in fila, [op for op in fila])
```

Saída:

```
2 True ['OP245101', 'OP245118']
```

Com três métodos curtos, a fila se comporta como uma coleção do Python. É o que o exercício 13 pede.

16. Quando NÃO usar classes

Classe é ferramenta, não obrigação. Muito código Python profissional é feito de **funções e dicionários**, com poucas classes bem escolhidas. Sinais de que a classe está sobrando:

- **Só tem `__init__` e um método.** Provavelmente é uma função com parâmetros.
- **Não guarda estado** (nenhum `self.algo` é usado). É uma função, ou um módulo de funções (módulo 07).
- **Só agrupa constantes.** Um dicionário, um módulo `config.py` ou uma `Enum`.
- **Só guarda dados, sem regra nenhuma.** Dicionário, `namedtuple` ou `@dataclass`.
- **Hierarquia "para o futuro".** `ClasseBaseAbstrataDeEnsaioGenerico` com uma filha só é complexidade sem retorno. Herança se cria quando a segunda filha aparecer.

```
import statistics


class CalculadoraDeCV:  # exagero: sem estado, só embrulha uma conta
    def calcular(self, valores):
        return statistics.stdev(valores) / statistics.mean(valores) * 100


def cv_percentual(valores):  # simples: faz o mesmo, testa igual
    return statistics.stdev(valores) / statistics.mean(valores) * 100


valores = [25.1, 24.8, 25.3, 24.9, 25.4, 25.0]
print(round(CalculadoraDeCV().calcular(valores), 2),
round(cv_percentual(valores), 2))
```

Saída:

```
0.92 0.92
```

E sinais de que a classe **vale a pena**: os mesmos dados viajam juntos por várias funções (OP + NF + peso + cliente); existem regras que precisam valer sempre (peso positivo, código imutável); várias variações do mesmo conceito (tipos de

ensaio); ou um objeto com estado que muda ao longo do tempo (a fila de emissão). O modelo do Validador tem tudo isso; o cálculo do CV, não.

> 🔧 **Modifique:** abra `exemplos/09_quando_nao_usar_classe.py` e pense em uma classe que você escreveria para o seu trabalho. Ela passa no teste acima? Escreva a versão com funções e compare.

Lendo mensagens de erro deste módulo

`TypeError: Amostra.esta_entre() takes 2 positional arguments but 3 were given` — faltou o `self` na definição do método. O Python sempre passa o objeto como primeiro argumento.

`TypeError: Amostra.__init__() missing 1 required positional argument: 'valor'` — você criou o objeto sem um argumento obrigatório. Confira a assinatura do `__init__`.

`AttributeError: 'Amostra' object has no attribute 'valro'. Did you mean: 'valor'?` — nome errado (o Python até sugere o certo) ou atributo nunca criado. Se o atributo "deveria existir", procure um `super().__init__(...)` esquecido ou um `self.algo = ...` que nunca rodou.

`AttributeError: property 'codigo' of 'Amostra' object has no setter` (no 3.10: `can't set attribute`) — você tentou atribuir a uma property somente leitura. Ou é de propósito (o dado não deve mudar), ou falta o `@codigo.setter`.

`RecursionError: maximum recursion depth exceeded` dentro de uma property — o setter faz `self.valor = ...` em vez de `self._valor = ...` e chama a si mesmo sem fim.

`TypeError: unhashable type: 'OrdemProducao'` — a classe tem `__eq__` mas não tem `__hash__` (ou é uma dataclass sem `frozen=True`). Defina `__hash__` ou use `frozen=True`.

`dataclasses.FrozenInstanceError: cannot assign to field 'valor'` — a dataclass é `frozen`. Use `replace(obj, valor=...)` para criar uma cópia alterada.

`TypeError: non-default argument 'valor' follows default argument` (no 3.13+: `... follows default argument 'unidade'`) — na dataclass, um campo sem padrão veio depois de um com padrão. Reordene os campos.

`ValueError: 'liberada' is not a valid Situacao` — o texto não corresponde a nenhum valor da Enum. Normalize antes (`strip().lower()`) ou trate com `try/except ValueError`.

`TypeError: Can't instantiate abstract class Ensaio ...` — você criou um objeto da classe abstrata, ou a filha esqueceu de implementar um `@abstractmethod`.

Boas práticas

- **Nomes**: classes em `PascalCase` e substantivos (`Tolerancia`, `Certificado`); métodos em `snake_case` e verbos (`aprova`, `emitir`, `salvar`).
- **Objeto nasce válido**: normalize e valide no `__init__` (ou no `__post_init__`).
- **Coleções de cada objeto nascem no `__init__`** (ou com `default_factory`), nunca no corpo da classe.
- **Escreva `__repr__`** em toda classe sua. A depuração agradece.
- **`_` para o que é interno**; `@property` quando precisar validar ou calcular. Não crie getters e setters "por via das dúvidas", como em Java: atributo público simples está ótimo até precisar de regra.
- **Prefira `@dataclass`** para classes que são basicamente dados; `frozen=True` para registros que não devem mudar (medições, resultados).
- **Composição antes de herança.** Herança só para "é um" verdadeiro.
- **Classes pequenas**, cada uma com uma responsabilidade. Se a descrição da classe precisa de "e", talvez sejam duas.
- **Não use classe onde uma função resolve** (seção 16).

No mundo real

O script `exemplos/10_modelo_validador.py` é o esqueleto do Validador de Certificados em objetos. Ele junta tudo do módulo:

```
Linha (Enum)            Situacao (Enum: aprovado / liberado / reprovado)
Tolerancia (dataclass frozen, @classmethod por_nominal, aprova)
Especificacao (dataclass: título, tolerâncias, texto)
LinhaDeProducao ──tem──> fixas + Especificacao padrão + especificações
por OP
Pedido, Medicao (dataclasses frozen: dados do dia)
Resultado ──tem──> Medicao + Situacao
Certificado ──tem──> Pedido + Especificacao + [Resultado]   (@property
aprovado)
avaliar(pedido, linha, medicoes, liberados, emissao) -> Certificado
```

Rodando `python exemplos/10_modelo_validador.py`:

```
CERTIFICADO OP245101 | NF 123456 | Alimentos Bom Sabor | 1250.5 kg
  Especificação: Filme PE 20 µm  (29/10/2026)
  - espessura: 20.4 µm (aprovado)
  - cof: 0.21 (aprovado)
  - tração: 31 MPa (aprovado)
  => PODE EMITIR

CERTIFICADO OP245106 | NF 123457 | Café Montanha | 980 kg
  Especificação: Café Montanha  (29/10/2026)
  - espessura: 26.3 µm (reprovado)
  - cof: 0.28 (aprovado)
  => BLOQUEADO

CERTIFICADO OP245118 | NF 123460 | Pet Nutri | 2010.75 kg
  Especificação: Filme PE 25 µm  (29/10/2026)
  - espessura: 25.1 µm (aprovado)
  - cof: 0.33 (liberado pela qualidade)
  => PODE EMITIR
```

Repare em duas regras de negócio que ficaram **claras** graças aos objetos:

- A OP245106 seria aprovada pelo padrão da linha B (22,5 a 27,5 µm), mas tem especificação própria (24 a 26 µm) e fica bloqueada. A regra mora num lugar só: `LinhaDeProducao.especificacao_para`.
- O COF 0,33 da OP245118 está fora, mas a planilha tinha "COF: liberado". Em vez de um `True`/`False`, o resultado carrega uma `Situacao` com três valores, e o certificado sai com a anotação.

Quando você ligar isso ao SQLite (módulo 11), à planilha (módulo 10) e ao robô (módulo 12), **o modelo não muda**: só muda de onde vêm os dados. É a vantagem de separar as regras (os objetos) da entrada e da saída. No desafio do módulo, você faz exatamente isso lendo os dados de arquivos.

Resumo

Assunto	Sintaxe essencial
Classe e objeto	`class Amostra:` · `a = Amostra(...)` · `isinstance(a, Amostra)`
Inicializar	`def __init__(self, codigo): self.codigo = codigo`
Método	`def aprova(self, valor): ...` · chamada `t.aprova(25.3)`
Atributo de classe	no corpo da classe; um só para todos (cuidado com listas!)
Texto	`__str__` (pessoas, `print`) · `__repr__` (programadores, REPL, listas)

Assunto	Sintaxe essencial
Interno	`self._valor` (convenção)
Property	`@property def valor(self)` · `@valor.setter def valor(self, novo)`
Construtor alternativo	`@classmethod def por_nominal(cls, ...): return cls(...)`
Utilitário	`@staticmethod def numero_br(texto): ...`
Herança	`class Filha(Mae):` · `super().__init__(...)` · sobrescrever método
Abstrata	`class Ensaio(ABC):` · `@abstractmethod`
Composição	`self.padrao = Especificacao(...)` (tem um)
Dataclass	`@dataclass` · `field(default_factory=list)` · `__post_init__`
Dataclass imutável/ordenável	`@dataclass(frozen=True, order=True)` · `field(compare=False)` · `replace`
Igualdade	`__eq__` (outro tipo: `return NotImplemented`) + `__hash__`
Enum	`class Situacao(Enum): LIBERADO = "liberado"` · `Situacao("liberado")` · `.value`
Coleção própria	`__len__` · `__iter__` (`return iter(lista)`) · `__contains__`

Você domina este módulo se...

- [ ] Consigo explicar, sem consultar, o que é `self` e o que o Python faz por trás de `a.esta_entre(1, 2)`.
- [ ] Sei por que `bobinas = []` no corpo da classe é um bug e onde a lista deve nascer.
- [ ] Escrevo `__str__` e `__repr__` e sei qual deles aparece no `print`, no REPL e dentro de uma lista.
- [ ] Protejo um atributo com `@property` e setter que valida, e faço uma property somente leitura.
- [ ] Explico a diferença entre método comum, `@classmethod` e `@staticmethod`, com um exemplo de cada no domínio do laboratório.
- [ ] Decido entre herança e composição usando "é um" / "tem um", e lembro do `super().__init__`.
- [ ] Escrevo uma dataclass com valor padrão mutável correto, `__post_init__`, `frozen` e `order`, e sei o que cada um gera.
- [ ] Reconheço uma classe desnecessária e reescrevo como função.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_classe_amostra`	★☆☆	`class`, `__init__`, `self`, métodos
02	`ex02_bobina_str_repr`	★☆☆	`__str__` e `__repr__`
03	`ex03_tolerancia`	★☆☆	validação no `__init__`, limites opcionais (`None`)
04	`ex04_dataclass_resultado`	★☆☆	`@dataclass`, campos com padrão, `__repr__` / `__eq__` gerados
05	`ex05_enum_situacao`	★☆☆	`Enum`, `@classmethod` e `@property` numa Enum
06	`ex06_amostra_property`	★★☆	`@property`, setter com validação, somente leitura, histórico
07	`ex07_tolerancia_construtores`	★★☆	`@classmethod` (nominal ± %, texto), `@staticmethod`, `cls(...)`
08	`ex08_heranca_ensaios`	★★☆	herança, `super()`, sobrescrita, polimorfismo
09	`ex09_corrigir_lote`	★★☆	**leitura de código**: prever e corrigir lista compartilhada, `super()` e `__eq__` / `__hash__`
10	`ex10_dataclass_medicao`	★★☆	`frozen`, `order`, `field(compare=False)`, hash e `set`
11	`ex11_especificacao_linha`	★★★	composição: `Especificacao` e `LinhaDeProducao` (regra real do projeto)
12	`ex12_revisao_certificado`	★★★	**revisão**: dataclass + número BR (05) + arquivo e exceção (06) + datas (07)
13	`ex13_revisao_fila_emissao`	★★★	**revisão**: regex (05) + exceção (06) + `__len__` / `__iter__` / `__contains__`

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError` e rode, na pasta `curso-python`:

```
python curso.py testar 08 ex01     # um exercício
python curso.py testar 08          # o módulo inteiro
```

> 💡 **Dica:** nos exercícios 04, 10 e 12 o arquivo traz um `__init__` provisório que só levanta erro. Apague-o e transforme a classe numa `@dataclass`, como o enunciado pede.

Desafio do módulo

O Validador de Certificados orientado a objetos: leia as regras das linhas (`regras.json`), os pedidos da expedição e as medições do laboratório, aplique tolerâncias fixas, especificação padrão ou própria da OP e as observações de liberação ("COF: liberado"), e diga quais certificados podem ser emitidos. Junta Enum, dataclasses, `@classmethod`, composição e `@property`.

Enunciado completo, exemplo de execução e critérios de aceite em desafio/README.md.

Quiz

Teste o que ficou em quiz.md: 13 perguntas com gabarito comentado.

Para ir além

- Tutorial oficial, classes: https://docs.python.org/pt-br/3/tutorial/classes.html
- `dataclasses`: https://docs.python.org/pt-br/3/library/dataclasses.html
- `enum` (e o guia "Enum HOWTO"): https://docs.python.org/pt-br/3/library/enum.html e https://docs.python.org/pt-br/3/howto/enum.html
- `abc`, classes abstratas: https://docs.python.org/pt-br/3/library/abc.html
- Métodos especiais (a lista completa dos "dunders"): https://docs.python.org/pt-br/3/reference/datamodel.html#special-method-names
- Leitura extra: Luciano Ramalho, *Python Fluente* (Novatec), capítulos sobre o modelo de dados e classes. Escrito por um brasileiro, é a referência para ir fundo.
- Leitura extra: "Stop Writing Classes", palestra de Jack Diederich (PyCon 2012), sobre a seção 16: https://www.youtube.com/watch?v=o9pEzgHorH0

Enunciados dos exercícios

Exercício 08.01 — Sua primeira classe: Amostra

Arquivo: `exercicios/ex01_classe_amostra.py`

```
Nível: ★☆☆

Contexto: Até agora cada amostra do laboratório era um dicionário
{"codigo": ..., "ensaio": ..., "valor": ...}. Um erro de digitação na
chave
```

```
("valro") passava despercebido. Uma classe junta os dados e o
comportamento da
amostra num lugar só.

Tarefa: complete a classe `Amostra`.
1. `__init__(self, codigo, ensaio, valor, unidade="")` guarda os
atributos:
   - `codigo`: sem espaços nas pontas e em MAIÚSCULAS ("am-001 " -> "AM-
001");
   - `ensaio`: sem espaços nas pontas e em minúsculas;
   - `valor`: sempre float (25 vira 25.0);
   - `unidade`: como veio.
2. `descricao()` devolve "AM-001 | espessura = 25.30 µm" (valor com 2
casas). Sem
   unidade, termina no número: "AM-002 | cof = 0.25".
3. `esta_entre(minimo, maximo)` devolve True se minimo <= valor <=
maximo.

Exemplos:
    >>> a = Amostra(" am-001 ", "Espessura", 25.3, "µm")
    >>> a.codigo, a.ensaio, a.valor
    ('AM-001', 'espessura', 25.3)
    >>> a.descricao()
    'AM-001 | espessura = 25.30 µm'
    >>> a.esta_entre(22.5, 27.5)
    True

Dica: dentro dos métodos, os dados do objeto estão em `self` (self.valor,
...).

Teste com:  python curso.py testar 08 ex01
```

Exercício 08.02 — Bobina com str e repr

Arquivo: `exercicios/ex02_bobina_str_repr.py`

```
Nível: ★☆☆

Contexto: Ao imprimir uma bobina, o Python mostra algo como
<__main__.Bobina object at 0x7f...>, que não ajuda ninguém. Você quer um
texto
amigável no relatório (print) e um texto técnico na depuração (REPL,
listas, logs).

Tarefa: complete a classe `Bobina`.
1. `__init__(self, lote, numero, peso_kg)` guarda `lote` (str), `numero`
(int) e
   `peso_kg` (float).
2. `codigo()` devolve o lote, um hífen e o número com 3 dígitos: "L2610-
```

```
007".
3. `__str__` devolve o texto amigável: "Bobina L2610-007 (152.4 kg)"
(peso com 1 casa).
4. `__repr__` devolve o texto técnico, que parece o código que cria o
objeto:
   "Bobina(lote='L2610', numero=7, peso_kg=152.4)" (use !r nas f-
strings).

Exemplos:
    >>> b = Bobina("L2610", 7, 152.4)
    >>> print(b)
    Bobina L2610-007 (152.4 kg)
    >>> b
    Bobina(lote='L2610', numero=7, peso_kg=152.4)
    >>> [b]
    [Bobina(lote='L2610', numero=7, peso_kg=152.4)]

Dica: f"{numero:03d}" completa com zeros à esquerda; f"{texto!r}" usa o
repr (com aspas).

Teste com:  python curso.py testar 08 ex02
```

Exercício 08.03 — Tolerância com mínimo, máximo ou os dois

Arquivo: `exercicios/ex03_tolerancia.py`

```
Nível: ★☆☆

Contexto: Nem toda tolerância tem os dois limites. Espessura tem faixa
(mín e máx),
tração só tem mínimo ("pelo menos 20 MPa") e COF às vezes só tem máximo.

Tarefa: complete a classe `Tolerancia`.
1. `__init__(self, minimo=None, maximo=None)` guarda `minimo` e `maximo`
(None =
   sem limite daquele lado). Levante ValueError se os dois forem None, ou
se os dois
   existirem e minimo > maximo.
2. `aprova(valor)` devolve True se o valor respeita os limites que
existem
   (limites inclusos).
3. `descricao()` devolve:
   - com os dois limites: "0.10 a 0.30"
   - só mínimo: "mín. 20.00"
   - só máximo: "máx. 0.30"
   (números com 2 casas)

Exemplos:
    >>> Tolerancia(0.10, 0.30).aprova(0.31)
```

```
    False
    >>> Tolerancia(minimo=20).aprova(35.2)
    True
    >>> Tolerancia(maximo=0.3).descricao()
    'máx. 0.30'

Dica: teste "is None" / "is not None" (zero é um limite válido!).

Teste com:  python curso.py testar 08 ex03
```

Exercício 08.04 — Resultado de ensaio com @dataclass

Arquivo: `exercicios/ex04_dataclass_resultado.py`

```
Nível: ★☆☆

Contexto: Uma classe que só guarda dados precisa de __init__, __repr__ e
__eq__,
quase sempre iguais. A @dataclass escreve tudo isso para você a partir da
lista de
campos.

Tarefa: transforme `ResultadoEnsaio` numa dataclass (apague o __init__
provisório)
com os campos, nesta ordem:
    ensaio: str
    valor: float
    unidade: str = ""
    aprovado: bool = True
e escreva o método `rotulo()`, que devolve:
    "espessura: 25.30 µm (aprovado)"   ou   "cof: 0.32 (reprovado)"
(valor com 2 casas; sem unidade, não sobra espaço).

Exemplos:
    >>> r = ResultadoEnsaio("cof", 0.32, aprovado=False)
    >>> r
    ResultadoEnsaio(ensaio='cof', valor=0.32, unidade='', aprovado=False)
    >>> r.rotulo()
    'cof: 0.32 (reprovado)'
    >>> ResultadoEnsaio("cof", 0.32) == ResultadoEnsaio("cof", 0.32)
    True

Dica: from dataclasses import dataclass. Campos com valor padrão vêm
depois dos
campos sem padrão.

Teste com:  python curso.py testar 08 ex04
```

Exercício 08.05 — Situação de liberação com Enum

Arquivo: `exercicios/ex05_enum_situacao.py`

```
Nível: ★☆☆

Contexto: Na planilha, a observação de liberação vem como texto livre:
"COF: liberado", "tração: REPROVADO", "Espessura : pendente ". Guardar a
situação
como texto deixa o programa frágil ("liberado" ≠ "Liberado" ≠
"liberada"). Uma
Enum define o conjunto FECHADO de situações possíveis.

A Enum `Situacao` já está pronta abaixo, com três membros. Tarefa:
1. `Situacao.de_texto(texto)` (classmethod): ignora espaços nas pontas e
   maiúsculas e devolve o membro correspondente ("  LIBERADO " ->
Situacao.LIBERADO).
   Texto desconhecido: levante ValueError com uma mensagem em português.
2. `pode_emitir` (property): True só para Situacao.LIBERADO.
3. `interpretar_observacao(texto)`: separa no PRIMEIRO ":" e devolve uma
tupla
   (ensaio em maiúsculas e sem espaços, Situacao). Sem ":", levante
ValueError.

Exemplos:
    >>> Situacao.de_texto(" Reprovado ")
    <Situacao.REPROVADO: 'reprovado'>
    >>> Situacao.LIBERADO.pode_emitir
    True
    >>> interpretar_observacao("COF: liberado")
    ('COF', <Situacao.LIBERADO: 'liberado'>)

Dica: Situacao("liberado") já procura o membro pelo valor (e levanta
ValueError
se não achar). Dentro do classmethod, use cls(...).

Teste com:  python curso.py testar 08 ex05
```

Exercício 08.06 — Amostra protegida com @property

Arquivo: `exercicios/ex06_amostra_property.py`

```
Nível: ★★☆

Contexto: Alguém digitou -25 na espessura e o certificado saiu com valor
negativo.
Com @property, toda atribuição a `amostra.valor` passa por uma validação,
e o
```

```
código da amostra não pode mais ser trocado depois de criado. Bônus: cada
correção
de valor fica registrada (rastreabilidade, coisa que auditoria adora).

Tarefa: complete a classe `Amostra`.
1. `__init__(self, codigo, valor, unidade="µm")`:
   - `codigo` sem espaços nas pontas e em maiúsculas; vazio ->
ValueError;
   - guarde o código num atributo "interno" `_codigo`;
   - o valor inicial passa pela MESMA validação do setter (abaixo) e NÃO
entra no
     histórico;
   - `unidade` é um atributo comum.
2. `codigo`: property SÓ de leitura (sem setter). `a.codigo = "X"` deve
dar
   AttributeError (o Python faz isso sozinho quando não há setter).
3. `valor`: property com setter que valida:
   - bool ou qualquer coisa que não seja int/float -> TypeError;
   - número negativo -> ValueError;
   - guarda como float; o valor ANTERIOR vai para o fim do histórico.
4. `historico`: property que devolve uma TUPLA com os valores anteriores,
do mais
   antigo para o mais recente (tupla: quem lê não consegue alterar o
original).

Exemplos:
    >>> a = Amostra("am-001", 25.3)
    >>> a.valor = 25.1
    >>> a.valor, a.historico
    (25.1, (25.3,))
    >>> a.valor = -1
    Traceback (most recent call last):
    ...
    ValueError: Valor não pode ser negativo: -1

Dica: True e False são do tipo int no Python! Teste bool ANTES de
int/float:
isinstance(novo, bool).

Teste com:  python curso.py testar 08 ex06
```

Exercício 08.07 — Tolerância com construtores alternativos

Arquivo: `exercicios/ex07_tolerancia_construtores.py`

```
Nível: ★★☆

Contexto: As tolerâncias chegam de jeitos diferentes: faixa fixa da linha
("0,10 a 0,30"), nominal ± percentual da especificação ("25 ± 10%") ou
```

```
números
soltos do banco de regras. Em vez de um __init__ cheio de "se", usamos
@classmethod para criar a mesma Tolerancia a partir de cada formato.

Tarefa: complete a classe `Tolerancia`.
1. `__init__(self, minimo, maximo)`: guarda os dois limites (float).
minimo > maximo
   -> ValueError.
2. `por_nominal(cls, nominal, percentual)` (classmethod): faixa nominal ±
percentual%.
   Percentual negativo -> ValueError. Ex.: (25, 10) -> mínimo 22.5,
máximo 27.5.
3. `converter_numero(texto)` (staticmethod): número no formato brasileiro
-> float.
   "1.250,5" -> 1250.5; "0,10" -> 0.1; " 25 " -> 25.0.
4. `de_texto(cls, texto)` (classmethod): aceita (com espaços variados)
   - "0,10 a 0,30"           -> faixa
   - "25 ± 10%" ou "25 +- 10%" -> nominal ± percentual
   Qualquer outro formato -> ValueError. Use converter_numero e
por_nominal.
5. `aprova(valor)`: True se minimo <= valor <= maximo.
6. `__repr__`: "Tolerancia(minimo=22.5, maximo=27.5)".
Os classmethods devem criar o objeto com `cls(...)`, para funcionar
também em
subclasses.

Exemplos:
    >>> Tolerancia.por_nominal(25, 10)
    Tolerancia(minimo=22.5, maximo=27.5)
    >>> Tolerancia.de_texto("0,10 a 0,30").aprova(0.31)
    False
    >>> Tolerancia.converter_numero("1.250,5")
    1250.5

Dica: re.fullmatch (módulo 05) com grupos resolve o de_texto.

Teste com:  python curso.py testar 08 ex07
```

Exercício 08.08 — Tipos de ensaio com herança e polimorfismo

Arquivo: `exercicios/ex08_heranca_ensaios.py`

```
Nível: ★★☆

Contexto: Todo ensaio tem nome, valor e unidade, e gera uma linha de
laudo. O que
muda é a REGRA de aprovação: espessura tem faixa, tração tem mínimo, COF
tem
máximo. A classe-mãe guarda o que é comum; cada filha define a sua regra.
```

```
Tarefa:
1. `Ensaio.__init__(self, nome, valor, unidade="")` guarda os três
atributos
   (valor como float).
2. `Ensaio.aprovado()` já está pronta: levanta NotImplementedError,
porque cada tipo
   de ensaio define a sua regra. NÃO mexa nela.
3. `Ensaio.laudo()` devolve "espessura: 25.30 µm -> APROVADO" (ou
REPROVADO), valor
   com 2 casas, sem espaço sobrando quando não há unidade ("cof: 0.32 ->
REPROVADO").
   Ela chama self.aprovado(), e cada filha responde do seu jeito
(polimorfismo).
4. As filhas chamam `super().__init__(nome, valor, unidade)` e guardam
seus limites:
   - `EnsaioFaixa(nome, valor, minimo, maximo, unidade="")`: minimo <=
valor <= maximo;
   - `EnsaioMinimo(nome, valor, minimo, unidade="")`: valor >= minimo;
   - `EnsaioMaximo(nome, valor, maximo, unidade="")`: valor <= maximo.
5. `reprovados(ensaios)` devolve os nomes dos ensaios reprovados, na
ordem.

Exemplos:
    >>> EnsaioFaixa("espessura", 25.3, 22.5, 27.5, "µm").laudo()
    'espessura: 25.30 µm -> APROVADO'
    >>> EnsaioMaximo("cof", 0.32, 0.30).laudo()
    'cof: 0.32 -> REPROVADO'
    >>> reprovados([EnsaioMinimo("tração", 18, 20, "MPa"),
EnsaioMaximo("cof", 0.2, 0.3)])
    ['tração']

Teste com:  python curso.py testar 08 ex08
```

Exercício 08.09 — Leitura de código: o lote que pesava demais

Arquivo: `exercicios/ex09_corrigir_lote.py`

```
Nível: ★★☆

Contexto: Um colega modelou os lotes de bobinas. O relatório mostrou o
lote L02
com o peso de TODOS os lotes, o lote de exportação quebrou com
AttributeError e
o programa travou ao colocar lotes num conjunto (set). Leia o código,
preveja e
corrija.

Código do colega (com 3 defeitos):
```

```
    class Lote:
        bobinas = []

        def __init__(self, codigo):
            self.codigo = codigo

        def adicionar(self, peso_kg):
            self.bobinas.append(peso_kg)

        @property
        def peso_total(self):
            return sum(self.bobinas)

        def __eq__(self, outro):
            return self.codigo == outro.codigo


    class LoteExportacao(Lote):
        def __init__(self, codigo, pais):
            self.pais = pais

Comportamento CORRETO esperado:
- cada lote tem a SUA lista de bobinas (atributo de instância criado no
__init__);
- `peso_total` soma só as bobinas daquele lote;
- `LoteExportacao` tem `codigo`, `pais` e a própria lista de bobinas
(chame a mãe);
- dois lotes são iguais quando têm o mesmo código; comparar com algo que
não é
  Lote (ex.: um texto) devolve False sem quebrar (retorne NotImplemented
no __eq__);
- lotes podem ir para um set e ser chave de dicionário: defina __hash__
com base
  no código (quem define __eq__ perde o __hash__ padrão).

Tarefa:
1. ANTES de corrigir, preveja: na versão do colega, depois de
       a = Lote("L01"); b = Lote("L02")
       a.adicionar(100.0); b.adicionar(50.0)
   quanto vale len(b.bobinas)? Devolva sua previsão em
   `previsao_bobinas_do_lote_b()`. Depois rode o código do colega para
conferir.
2. Corrija as duas classes abaixo, mantendo nomes e assinaturas.

Exemplos (versão corrigida):
    >>> a = Lote("L01")
    >>> a.adicionar(100.0)
    >>> Lote("L02").peso_total
    0
```

```
    >>> Lote("L01") == Lote("L01"), Lote("L01") == "L01"
    (True, False)
    >>> len({Lote("L01"), Lote("L01"), Lote("L02")})
    2

Teste com:  python curso.py testar 08 ex09
```

Exercício 08.10 — Medições imutáveis e ordenáveis com dataclass

Arquivo: `exercicios/ex10_dataclass_medicao.py`

```
Nível: ★★☆

Contexto: Medição registrada não se altera (rastreabilidade!). E você
precisa
ordenar medições por data, pegar a última de cada linha e remover
registros
duplicados que vieram da planilha (a mesma medição digitada duas vezes,
às vezes
com observação diferente).

Tarefa:
1. Transforme `Medicao` numa dataclass IMUTÁVEL e ORDENÁVEL
   (`@dataclass(frozen=True, order=True)`) com os campos, nesta ordem:
       data: date
       linha: str
       valor: float
       observacao: str = ""   -> NÃO participa de comparação nem de
igualdade:
                                  use field(default="", compare=False)
   A ordem dos campos define a ordenação: primeiro data, depois linha,
depois valor.
2. `ordenar(medicoes)`: nova lista em ordem (use sorted, que já sabe
comparar).
3. `ultima_por_linha(medicoes)`: {linha: medição mais recente}, com as
linhas em
   ordem alfabética.
4. `sem_duplicadas(medicoes)`: remove repetidas (iguais = mesma data,
linha e
   valor), mantendo a PRIMEIRA ocorrência e a ordem original. Use um set
de
   medições já vistas (frozen=True deixa a dataclass "hashable").

Exemplos:
    >>> from datetime import date
    >>> m = Medicao(date(2026, 10, 1), "A", 20.4, "1ª leitura")
    >>> m == Medicao(date(2026, 10, 1), "A", 20.4, "redigitada")
    True
    >>> m.valor = 99
```

```
    Traceback (most recent call last):
    ...
    dataclasses.FrozenInstanceError: cannot assign to field 'valor'

Teste com:  python curso.py testar 08 ex10
```

Exercício 08.11 — Especificação e linha de produção (composição)

Arquivo: `exercicios/ex11_especificacao_linha.py`

```
Nível: ★★★

Contexto: Esta é a regra central do Validador de Certificados. Cada linha
de
produção (A, B, C, D) tem tolerâncias FIXAS (ex.: COF de 0,10 a 0,30) e
uma
especificação PADRÃO (ex.: espessura 25 µm ± 10%). Algumas OPs têm
especificação
PRÓPRIA (título + texto, pedida pelo cliente), que vale no lugar da
padrão.
Uma linha "TEM UMA" especificação; ela não "É UMA" especificação. Isso é
composição, não herança.

A classe `Tolerancia` já está pronta. Tarefa:
1. `Especificacao(titulo, tolerancias, texto="")`: guarda `titulo`,
`texto` e uma
   CÓPIA do dicionário {ensaio: Tolerancia} em `tolerancias` (mudar o
dicionário
   original depois não pode afetar a especificação).
   - `avaliar(medidas)`: recebe {ensaio: valor} e devolve {ensaio:
aprovado?} para
     cada ensaio DA ESPECIFICAÇÃO, na ordem dela. Ensaio da especificação
que falta
     nas medidas -> KeyError. Medidas a mais são ignoradas.
   - `aprova(medidas)`: True se todos os ensaios foram aprovados.
2. `LinhaDeProducao(nome, fixas, padrao)`: `fixas` é {ensaio: Tolerancia}
(guarde
   uma cópia) e `padrao` é uma Especificacao.
   - `cadastrar(op, especificacao)`: guarda a especificação própria da
OP.
   - `tem_especificacao_propria(op)`: True/False.
   - `especificacao_para(op)`: a própria da OP, ou a padrão da linha.
   - `tolerancias_para(op)`: junta as fixas com as da especificação da OP
(se o mesmo
     ensaio aparecer nas duas, vale a da especificação).
   - `avaliar(op, medidas)`: como Especificacao.avaliar, mas com
tolerancias_para(op)
     (ordem: primeiro as fixas, depois os ensaios novos da
especificação).
```

```
Exemplo:
    >>> padrao = Especificacao("Filme PE 25 µm", {"espessura":
Tolerancia(22.5, 27.5)})
    >>> linha = LinhaDeProducao("B", {"cof": Tolerancia(0.10, 0.30)},
padrao)
    >>> linha.avaliar("OP245102", {"espessura": 25.4, "cof": 0.33})
    {'cof': False, 'espessura': True}
    >>> linha.cadastrar("OP245110", Especificacao("Cliente X", {"cof":
Tolerancia(0.1, 0.4)}))
    >>> linha.avaliar("OP245110", {"cof": 0.33})
    {'cof': True}

Dica: {**a, **b} junta dois dicionários (b vence nos repetidos); dict(x)
copia.

Teste com:  python curso.py testar 08 ex11
```

Exercício 08.12 — Revisão: o certificado de qualidade

Arquivo: `exercicios/ex12_revisao_certificado.py`

```
Nível: ★★★

Revisão de: módulo 05 (números no formato brasileiro e f-strings), módulo
06
(gravar arquivo com pathlib e exceção personalizada) e módulo 07 (datas),
junto com
dataclass, @property e @classmethod deste módulo.

Contexto: O certificado é o produto final do seu sistema. Ele junta OP,
NF,
cliente, peso, data de emissão e os resultados dos ensaios. Só pode ser
emitido se
todos os ensaios estiverem aprovados.

Já estão prontos: `ResultadoEnsaio` (dataclass imutável) e a exceção
`CertificadoReprovadoError`. Tarefa: transforme `Certificado` numa
dataclass com os
campos, nesta ordem:
    op: str
    nf: str
    cliente: str
    peso_kg: float
    emissao: date
    resultados: list[ResultadoEnsaio] = (lista vazia nova para cada
certificado:
                                         field(default_factory=list))
e implemente:
```

```
1. `__post_init__`: op e nf sem espaços nas pontas; vazios -> ValueError;
   peso_kg <= 0 -> ValueError.
2. `numero` (property): "CQ-20261029-OP245101" (data AAAAMMDD + OP).
3. `aprovado` (property): True se houver resultados e todos estiverem
aprovados.
4. `texto()`: o certificado, linha a linha, EXATAMENTE assim (números com
vírgula
   decimal e ponto de milhar, 2 casas; unidade só se houver):
       CERTIFICADO DE QUALIDADE Nº CQ-20261029-OP245101
       Cliente: Alimentos Bom Sabor
       OP: OP245101 | NF: 123456 | Peso: 1.250,50 kg
       Emissão: 29/10/2026
       Resultados:
       - espessura: 25,10 µm (aprovado)
       - cof: 0,22 (aprovado)
       Situação: APROVADO
   (ou "Situação: REPROVADO"; as linhas são unidas com "\n", sem "\n" no
fim)
5. `salvar(pasta)`: grava texto() em pasta/"<numero>.txt" (UTF-8),
criando a pasta
   se preciso, e devolve o Path do arquivo.
6. `emitir(...)` (classmethod, mesmos parâmetros dos campos): sem
resultados ->
   ValueError; algum reprovado -> CertificadoReprovadoError com a
mensagem
   "OP245101 reprovada em: cof, espessura" (nomes em ordem alfabética);
senão
   devolve o certificado.

Dica: f"{1250.5:,.2f}" dá '1,250.50'; troque os separadores (módulo 05).

Teste com:  python curso.py testar 08 ex12
```

Exercício 08.13 — Revisão: a fila de emissão de certificados

Arquivo: `exercicios/ex13_revisao_fila_emissao.py`

```
Nível: ★★★

Revisão de: módulo 05 (regex e número brasileiro), módulo 06 (exceção
personalizada) e módulo 03 (listas e conjuntos), junto com os métodos
especiais
__len__, __iter__ e __contains__ deste módulo.

Contexto: Os e-mails da expedição viram pedidos (OP, NF, peso). Eles
entram numa
FILA: o robô emite um certificado por vez, na ordem de chegada. A mesma
linha de
e-mail às vezes chega duas vezes (alguém respondeu "para todos"), e o
```

```
pedido
repetido não pode entrar de novo.

Já estão prontos: `PedidoExpedicao` (dataclass imutável) e
`FilaVaziaError`.
Tarefa: complete a classe `FilaDeEmissao`.
1. `__init__()`: fila vazia.
2. `adicionar(pedido)`: põe no fim. Pedido repetido (mesma OP e mesma NF)
->
   ValueError.
3. `adicionar_do_email(texto)`: lê as linhas no formato
   "OP 245101 - NF 123456 - Peso: 1.250,5 kg" (sem diferenciar
maiúsculas,
   espaços variados; OP vira "OP245101"; peso em formato brasileiro),
adiciona os
   pedidos que ainda não estão na fila (ignora os repetidos, sem erro) e
devolve
   QUANTOS entraram.
4. `proximo()`: tira e devolve o PRIMEIRO pedido (quem chegou antes sai
antes).
   Fila vazia -> FilaVaziaError.
5. `len(fila)` (__len__): quantos pedidos esperam.
6. `for pedido in fila` (__iter__): percorre na ordem, SEM tirar da fila.
7. `"123456" in fila` (__contains__): True se algum pedido tem essa NF.
8. `peso_total` (property): soma dos pesos, round(..., 2).
9. `__repr__`: "FilaDeEmissao(2 pedidos, 2230.50 kg)".

Exemplo:
    >>> fila = FilaDeEmissao()
    >>> fila.adicionar_do_email("OP 245101 - NF 123456 - Peso: 1.250,5
kg")
    1
    >>> len(fila), "123456" in fila
    (1, True)
    >>> fila.proximo()
    PedidoExpedicao(op='OP245101', nf='123456', peso_kg=1250.5)

Dica: __iter__ pode simplesmente devolver iter(uma_lista). O padrão do
item é o
mesmo do exercício 07.12.

Teste com:  python curso.py testar 08 ex13
```

Desafio do Módulo 08 — O Validador de Certificados orientado a objetos

Contexto

Chegou a hora de modelar o coração do seu projeto com classes. Todo dia o laboratório tem:

- **regras** por linha de produção: tolerâncias fixas (COF, tração) e uma especificação padrão (espessura e gramatura com nominal ± %), além de especificações **próprias** de algumas OPs (título + texto + tolerâncias);
- **pedidos** da expedição: OP, NF, cliente, peso e linha;
- **medições** da planilha, às vezes com uma observação de liberação, como `COF: liberado` ou `COF: pendente`.

O programa decide, para cada pedido, se o certificado **pode ser emitido** ou está **bloqueado**, e por quê. Os arquivos estão em `../dados/`:

```
regras.json     linhas A-D (fixas + padrao) e especificacoes_op
pedidos.csv     op;nf;cliente;peso_kg;linha            (UTF-8, ";",
vírgula decimal)
medicoes.csv    op;ensaio;valor;unidade;observacao
```

Requisitos obrigatórios

1. **Enums** para a linha de produção (A–D) e para a situação de cada ensaio (aprovado, liberado pela qualidade, pendente, reprovado, informativo).
2. **`Tolerancia`** como dataclass imutável, com mínimo e/ou máximo, um `@classmethod` para nominal ± % e outro para criar a partir do dicionário do JSON.
3. **`Especificacao`** (título, texto, tolerâncias) e **`LinhaDeProducao`** por **composição**: a linha tem tolerâncias fixas, uma especificação padrão e as especificações próprias das OPs. As tolerâncias de uma OP = fixas + as da especificação (a da especificação vence se o ensaio se repetir).
4. **Regra da observação**: um ensaio fora da tolerância vira "liberado pela qualidade" se a observação da medição disser `<ensaio>: liberado`, e "pendente" se disser `<ensaio>: pendente`. Ensaio medido sem tolerância é "informativo" e não bloqueia.
5. **`Certificado`** com `numero` (`CQ-AAAAMMDD-OP`), `pode_emitir` (property: há ensaio avaliado e nenhum reprovado ou pendente), a lista de pendências e o texto do certificado com números no formato brasileiro.

6. Linha de comando: `--data dd/mm/aaaa` (padrão: hoje), `--op OPxxxxx` (só uma OP) e `--saida pasta` (grava um `.txt` por certificado emitível). Arquivo inexistente: mensagem no `stderr` e código 2.
7. Nenhuma regra de negócio solta no `main()`: ele só lê argumentos, chama os objetos e imprime.

Extras opcionais

- `__str__` e `__repr__` úteis em todas as classes.
- Um `Validador` que recebe as linhas e as medições e tem um método `certificar(pedido, emissao)`.
- Exportar um resumo em JSON (OP, situação, pendências).
- Escrever testes pytest para `Tolerancia`, `LinhaDeProducao.tolerancias_para` e a regra da observação (o módulo 09 aprofunda).

Exemplo de execução

```
> python 08-orientacao-a-objetos/desafio/solucao_referencia.py --op
OP245118 --data 29/10/2026
CERTIFICADO DE QUALIDADE Nº CQ-20261029-OP245118
Cliente: Pet Nutri
OP: OP245118 | NF: 123460 | Linha: B | Peso: 1.105,00 kg
Especificação: Filme PE 25 µm
  Padrão da linha B
Emissão: 29/10/2026
- espessura: 25,60 µm [tol.: 22,5 a 27,5] -> aprovado
- gramatura: 23,60 g/m² [tol.: 20,7 a 25,3] -> aprovado
- cof: 0,33 [tol.: 0,1 a 0,3] -> liberado pela qualidade
- tração: 26,50 MPa [tol.: mín. 20] -> aprovado
=> PODE EMITIR

Resumo: 1 para emitir, 0 bloqueado(s).
```

Sem `--op`, os seis pedidos são avaliados e o resumo é `3 para emitir, 3 bloqueado(s)`: OP245103 (espessura reprovada), OP245106 (espessura fora da especificação **própria** da OP, embora dentro da padrão da linha) e OP245109 (COF pendente de decisão).

Critérios de aceite

- [ ] Com `--data 29/10/2026`, OP245101, OP245104 e OP245118 podem ser emitidas; OP245103, OP245106 e OP245109 ficam bloqueadas, com o motivo.
- [ ] OP245106 usa a especificação "Café Montanha - embalagem a vácuo"; OP245118 usa a padrão da linha B.

- [ ] O COF 0,33 da OP245118 aparece como "liberado pela qualidade".
- [ ] `--saida` grava `CQ-20261029-OP245118.txt` (e os demais emitíveis) em UTF-8.
- [ ] Pasta de dados inexistente: mensagem clara e código de saída 2, sem traceback.
- [ ] As regras estão nas classes (dá para testar `Tolerancia` e `LinhaDeProducao` sem ler arquivo nenhum).
- [ ] `ruff check` e `ruff format --check` passam.

Dicas para começar

1. **Comece pelo modelo, não pelos arquivos.** Escreva `Tolerancia`, `Especificacao` e `LinhaDeProducao` e teste no REPL com valores digitados à mão.
2. Depois escreva as funções que leem os arquivos e **devolvem objetos**. O resto do programa não deve saber que existe CSV ou JSON.
3. Para a regra da observação, reaproveite o exercício 08.05 (`interpretar_observacao`).
4. Desenhe antes, no papel, quem "tem" quem (composição) e quem "é" quem (herança). Aqui quase tudo é "tem".
5. Compare com a `solucao_referencia.py` só depois de tentar.

Quiz do Módulo 08 — Orientação a objetos

Responda antes de olhar o gabarito. Nas perguntas "o que imprime", preveja primeiro e só depois rode para conferir.

Perguntas

1. Em `amostra.esta_entre(22.5, 27.5)`, o que o parâmetro `self` de `esta_entre` recebe?

a) A classe `Amostra`. b) O objeto `amostra`. c) O número `22.5`. d) Nada: `self` é só um enfeite.

2. O que imprime?

```
class Bobina:
    def __init__(self, lote):
        self.lote = lote

    def __repr__(self):
        return f"Bobina({self.lote!r})"
```

```
b = Bobina("L01")
print(b)
print([b])
```

3. Onde está o erro? Depois de rodar, o lote L02 aparece com o peso do L01.

```
class Lote:
    bobinas = []

    def __init__(self, codigo):
        self.codigo = codigo

    def adicionar(self, peso_kg):
        self.bobinas.append(peso_kg)
```

4. O que imprime?

```
class Amostra:
    def __init__(self, valor):
        self.valor = valor

    @property
    def valor(self):
        return self._valor

    @valor.setter
    def valor(self, novo):
        self._valor = max(novo, 0)


print(Amostra(-5).valor)
```

5. Qual decorador cria um método que recebe a **classe** como primeiro argumento (em geral chamado `cls`) e é muito usado para construtores alternativos, como `Tolerancia.por_nominal(25, 10)`?

a) `@staticmethod` b) `@property` c) `@classmethod` d) `@dataclass`

6. Onde está o erro? `EnsaioMinimo("tração", 18, 20).laudo()` dá `AttributeError: 'EnsaioMinimo' object has no attribute 'nome'`.

```
class Ensaio:
    def __init__(self, nome, valor):
        self.nome = nome
        self.valor = valor

    def laudo(self):
```

```
        return f"{self.nome}: {self.valor}"


class EnsaioMinimo(Ensaio):
    def __init__(self, nome, valor, minimo):
        self.minimo = minimo
```

7. Uma linha de produção usa uma especificação padrão e pode ter especificações próprias por OP. Qual modelagem é a mais adequada?

a) `class LinhaDeProducao(Especificacao)` (herança) b) `class Especificacao(LinhaDeProducao)` (herança) c) `LinhaDeProducao` guarda objetos `Especificacao` em atributos (composição) d) Uma única classe gigante com todos os campos das duas

8. O que imprime?

```
from dataclasses import dataclass


@dataclass
class Pedido:
    op: str
    nf: str


p1 = Pedido("OP1", "100")
p2 = Pedido("OP1", "100")
print(p1 == p2, p1 is p2)
```

9. Onde está o erro? O programa quebra antes de criar qualquer lote.

```
from dataclasses import dataclass


@dataclass
class Lote:
    codigo: str
    bobinas: list = []
```

10. O que imprime?

```
from dataclasses import dataclass


@dataclass(frozen=True, order=True)
class Medicao:
```

```
    linha: str
    valor: float


print(sorted([Medicao("B", 1.0), Medicao("A", 9.0), Medicao("A", 2.0)])
[0])
```

11. O que imprime?

```
from enum import Enum


class Situacao(Enum):
    LIBERADO = "liberado"
    REPROVADO = "reprovado"


print(Situacao("reprovado").name, Situacao.LIBERADO.value)
```

12. O que acontece na última linha?

```
class OrdemProducao:
    def __init__(self, numero):
        self.numero = numero

    def __eq__(self, outro):
        return isinstance(outro, OrdemProducao) and self.numero ==
outro.numero


ops = {OrdemProducao("245101")}
```

13. Em qual destes casos uma **função** resolve melhor que uma classe?

a) Pedidos da expedição (OP, NF, peso, cliente) que passam por várias etapas do sistema. b) Calcular o CV% de uma lista de valores. c) Uma fila de emissão que guarda pedidos e muda ao longo do dia. d) Vários tipos de ensaio com regras de aprovação diferentes.

Gabarito

1. b. O Python transforma `amostra.esta_entre(22.5, 27.5)` em `Amostra.esta_entre(amostra, 22.5, 27.5)`: `self` é o próprio objeto.

2.

```
Bobina('L01')
[Bobina('L01')]
```

Sem `__str__`, o `print` usa o `__repr__`; e dentro de listas o Python sempre usa o `__repr__` de cada item.

3. `bobinas = []` no corpo da classe é um atributo **de classe**: uma lista só, compartilhada por todos os lotes. Crie a lista no `__init__`: `self.bobinas = []`.

4. `0`. O `self.valor = valor` do `__init__` passa pelo setter, que troca valores negativos por 0. É por isso que vale atribuir pela property (e não por `self._valor`) dentro do `__init__`: a regra vale desde o nascimento do objeto.

5. c. `@classmethod` recebe a classe (`cls`) e costuma devolver `cls(...)`. `@staticmethod` não recebe nem objeto nem classe. `@property` transforma método em "atributo". `@dataclass` decora a classe inteira.

6. O `__init__` da filha substitui o da mãe e não chama `super().__init__(nome, valor)`. Então `self.nome` e `self.valor` nunca são criados. Correção: primeira linha do `__init__` da filha: `super().__init__(nome, valor)`.

7. c. A linha **tem** especificações; ela não **é** uma especificação. Relação "tem um" pede composição. Herança (a e b) inverteria o sentido do modelo e não permite várias especificações por linha.

8. `True False`. A dataclass gera um `__eq__` que compara os campos: os pedidos são **iguais**. Mas são dois objetos diferentes na memória, então `is` dá `False`.

9. Padrão mutável num campo de dataclass: `ValueError: mutable default <class 'list'> for field bobinas is not allowed: use default_factory`. Correção: `bobinas: list = field(default_factory=list)` (importando `field` de `dataclasses`).

10. `Medicao(linha='A', valor=2.0)`. Com `order=True`, a comparação segue a ordem dos campos: primeiro `linha` ("A" antes de "B"), e no empate, `valor` (2.0 antes de 9.0).

11. `REPROVADO liberado`. `Situacao("reprovado")` procura pelo valor e devolve o membro `REPROVADO`; `.name` é o nome do membro e `.value` é o valor.

12. `TypeError: unhashable type: 'OrdemProducao'`. Ao definir `__eq__`, a classe perde o `__hash__` padrão e não pode mais ir para um `set` nem virar chave de `dict`. Defina `def __hash__(self): return hash(self.numero)`.

13. b. Calcular o CV não guarda estado nem tem variações: é uma função com parâmetros. Os pedidos (a) viajam juntos pelo sistema (dataclass), a fila (c) tem estado que muda, e os tipos de ensaio (d) são variações de um mesmo conceito.

Módulo 09 — Testes e qualidade de código

> **Tempo estimado:** 12 a 16 horas · **Pré-requisitos:** módulos 00 a 08 · **Nível:** intermediário

O que você vai aprender

- Escrever testes com `pytest` no padrão AAA (preparar, agir, conferir) e **ler uma falha** sem se assustar.
- Usar `pytest.approx` para floats, `pytest.raises` para erros esperados e `@pytest.mark.parametrize` para testar vários casos com uma função só.
- Criar fixtures (inclusive em `conftest.py`) e usar as prontas: `tmp_path`, `capsys`, `caplog` e `monkeypatch`.
- Desenvolver uma função com **TDD**, no ciclo vermelho → verde → refatorar.
- Escrever testes para um código que você não escreveu, escolhendo casos que pegam bugs.
- Medir a cobertura dos testes e entender o que ela **não** garante.
- Usar o `ruff` para achar problemas (lint) e padronizar o código (format).
- Ler type hints e entender o que o `mypy` confere.
- Trocar `print` por `logging`, com níveis, formato e arquivo de log.
- Depurar com `breakpoint()` (pdb) e com o debugger do VS Code.
- Trabalhar com Git como num time: branch, commits pequenos, pull request e revisão.
- Refatorar com segurança: melhorar nomes e funções sem mudar o comportamento.

Por que isso importa

Até aqui você escreveu código que funciona. Agora vai aprender a **provar** que funciona, hoje e daqui a seis meses, depois de dez mudanças. É isso que separa um script de um sistema.

No laboratório você já pensa assim: ninguém libera um laudo sem conferir o instrumento com um padrão de referência. Um teste automatizado é o padrão de referência do código.

Vagas de dados, automação e backend pedem `pytest`, Git, pull request e "código limpo"; um repositório com testes e bons commits vale mais que um certificado. E no projeto final, um erro numa regra de tolerância libera bobina reprovada: os testes são a sua rede de segurança.

1. Por que testar

Você já testa o seu código: roda o script e confere a saída "de olho". Isso é um **teste manual**. Funciona uma vez. O problema é a segunda, a terceira, a vigésima vez.

Um teste automatizado é uma função que chama o seu código com uma entrada conhecida e **confere** se a saída é a esperada. Ele **pega regressões** (você corrige um bug, quebra outra coisa, e o teste antigo avisa), **documenta** (`test_bordas_estao_dentro` explica a regra), **dá coragem para mudar** e **melhora o design**: código difícil de testar costuma ser código mal dividido.

```
          /\        ponta a ponta: o sistema inteiro (robô no TOTVS)
         /  \       poucos, lentos, frágeis
        /----\
       /      \     integração: partes juntas (código + banco SQLite)
      /--------\
     /          \   unidade: uma função isolada (calcular_limites)
    /------------\  muitos, rápidos, baratos
```

Neste módulo o foco é a base da pirâmide: **testes de unidade**.

2. `assert`: a peça mais simples

`assert` confere uma condição. Se for verdadeira, nada acontece. Se for falsa, o Python levanta `AssertionError`. Depois da vírgula vem uma mensagem opcional.

```python
def calcular_limites(nominal, tolerancia_pct):
    margem = nominal * tolerancia_pct / 100
    return nominal - margem, nominal + margem


assert calcular_limites(20.0, 5.0) == (19.0, 21.0)   # silêncio = passou
minimo, maximo = calcular_limites(20.0, 5.0)
assert minimo == 18.0, f"mínimo errado: esperava 18.0, veio {minimo}"
```

```
AssertionError: mínimo errado: esperava 18.0, veio 19.0
```

Mais casos em `exemplos/01_assert_basico.py` (rode na pasta do módulo).

> ⚠️ **Erro comum:** usar `assert` para validar dados no programa de produção. O Python pode rodar com `python -O`, que **desliga todos os asserts**. Para validar entrada, use `if` + `raise ValueError(...)` (módulo 06). `assert` é para testes.

> 🔮 **Preveja antes de rodar:** o que acontece com esta linha no REPL?
>
> ```
> assert 0.1 + 0.2 == 0.3, "a soma não fechou"
> ```

<details>
<summary>Resposta</summary>

Levanta `AssertionError: a soma não fechou`. Em float, `0.1 + 0.2` vale `0.30000000000000004` (módulo 01), então a comparação exata é falsa. Na seção 6 você vê o jeito certo de comparar floats em testes: `pytest.approx`.
</details>

3. pytest: primeiros passos

Dá para testar só com `assert`, mas o **pytest** acrescenta o que falta: encontra os testes sozinho, roda todos e explica **por que** cada falha aconteceu. Ele já está no `requirements.txt` do curso (`python -m pip install pytest`, se precisar).

Regras de descoberta. O pytest procura **arquivos** `test_*.py` (ou `*_test.py`) e, dentro deles, **funções** cujo nome começa com `test` (e classes que começam com `Test`). Seguiu o padrão de nome, virou teste. Não existe registro nem "main".

O mini projeto do módulo

Em `exemplos/mini_projeto_laudo/` há um projeto pequeno, organizado como um real:

```
mini_projeto_laudo/
  pyproject.toml      configuração do pytest e do ruff
  conftest.py         fixtures compartilhadas (seção 8)
  laudo.py            o código: limites, avaliação, média, leitura de
arquivo
  observacao.py       leitura de "COF: liberado" (nasceu com TDD, seção
13)
  tests/
```

```
    test_laudo.py
    test_observacao.py
```

Um trecho de `laudo.py`:

```python
def calcular_limites(nominal: float, tolerancia_pct: float) ->
tuple[float, float]:
    """Devolve (mínimo, máximo) para um valor nominal ± tolerância em
%."""
    if nominal <= 0:
        raise ValueError(f"nominal deve ser positivo, recebi {nominal}")
    if tolerancia_pct < 0:
        raise ValueError(f"tolerância não pode ser negativa, recebi
{tolerancia_pct}")
    margem = nominal * tolerancia_pct / 100
    return nominal - margem, nominal + margem


def avaliar(valor: float, minimo: float, maximo: float) -> str:
    """'APROVADO' se o valor está entre os limites (inclusive), senão
'REPROVADO'."""
    return "APROVADO" if minimo <= valor <= maximo else "REPROVADO"
```

E o primeiro teste, em `tests/test_laudo.py`:

```python
import pytest

from laudo import calcular_limites


def test_limites_com_tolerancia_de_5_porcento():
    # Arrange (preparar): os dados de entrada
    nominal, tolerancia = 20.0, 5.0
    # Act (agir): chama UMA vez o que está sendo testado
    minimo, maximo = calcular_limites(nominal, tolerancia)
    # Assert (conferir): o resultado é o esperado?
    assert minimo == pytest.approx(19.0)
    assert maximo == pytest.approx(21.0)
```

Rodando

Entre na pasta do mini projeto e chame o pytest pelo Python do ambiente virtual (`python -m pytest` garante que é o pytest do seu `.venv`):

```
> cd 09-testes-e-qualidade\exemplos\mini_projeto_laudo
> python -m pytest -v
============================= test session starts
```

```
============================
platform linux -- Python 3.13.16, pytest-9.2.0, pluggy-1.7.0 --
/usr/bin/python
rootdir: /home/user/estudos/curso-python/09-testes-e-
qualidade/exemplos/mini_projeto_laudo
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.15.1
collecting ... collected 16 items

tests/test_laudo.py::test_limites_com_tolerancia_de_5_porcento PASSED
[  6%]
tests/test_laudo.py::test_limites_com_tolerancia_negativa_levanta_erro
PASSED [ 12%]
tests/test_laudo.py::test_avaliar_respeita_as_bordas[19.0-APROVADO]
PASSED [ 18%]
tests/test_laudo.py::test_avaliar_respeita_as_bordas[20.0-APROVADO]
PASSED [ 25%]
tests/test_laudo.py::test_avaliar_respeita_as_bordas[21.0-APROVADO]
PASSED [ 31%]
tests/test_laudo.py::test_avaliar_respeita_as_bordas[18.99-REPROVADO]
PASSED [ 37%]
tests/test_laudo.py::test_avaliar_respeita_as_bordas[21.01-REPROVADO]
PASSED [ 43%]
tests/test_laudo.py::test_media_de_tres_medicoes PASSED
[ 50%]
tests/test_laudo.py::test_media_de_lista_vazia_levanta_erro PASSED
[ 56%]
tests/test_laudo.py::test_ler_medicoes_aceita_virgula_decimal PASSED
[ 62%]
tests/test_laudo.py::test_ler_medicoes_ignora_linhas_vazias PASSED
[ 68%]
tests/test_laudo.py::test_conferir_op_sem_acessar_o_erp PASSED
[ 75%]
tests/test_observacao.py::test_ler_liberacoes[um-liberado] PASSED
[ 81%]
tests/test_observacao.py::test_ler_liberacoes[um-reprovado] PASSED
[ 87%]
tests/test_observacao.py::test_ler_liberacoes[dois-ensaios] PASSED
[ 93%]
tests/test_observacao.py::test_status_desconhecido_levanta_erro PASSED
[100%]

============================= 16 passed in 0.04s
==============================
```

O cabeçalho (`platform`, `rootdir`, `plugins`) muda de computador para computador: no Windows aparece `win32` e um caminho `C:\...`. Com `-q` em vez de `-v`, a saída é um caractere por teste: `.` passou, `F` falhou (um `assert` deu falso), `E` erro (exceção

inesperada fora do `assert`).

Comando	Para quê
`python -m pytest -v`	um teste por linha, com o nome
`python -m pytest -x`	para na **primeira** falha (ótimo no TDD)
`python -m pytest -k bordas`	só testes cujo nome contém "bordas"
`python -m pytest --lf`	só os que falharam na última rodada (*last failed*)
`python -m pytest -s`	mostra os `print` (normalmente o pytest os esconde)

> 💡 **Dica:** o `python curso.py testar 09 ex01` que você usa desde o módulo 00 é um atalho: por baixo, ele chama `python -m pytest -v --tb=short` no arquivo de teste do exercício. Agora você sabe rodar o pytest direto, com qualquer opção.

4. Lendo uma falha do pytest

Uma falha não é uma bronca: é um relatório. Para ver um, coloquei um bug de propósito numa cópia do mini projeto, trocando `<=` por `<` em `avaliar`:

```
> python -m pytest -q --tb=short
..F.F.........
[100%]
================================== FAILURES
==================================
_________________ test_avaliar_respeita_as_bordas[19.0-APROVADO]
_________________
tests/test_laudo.py:35: in test_avaliar_respeita_as_bordas
    assert avaliar(valor, 19.0, 21.0) == esperado
E   AssertionError: assert 'REPROVADO' == 'APROVADO'
E
E     - APROVADO
E     ? ^
E     + REPROVADO
E     ? ^^
_________________ test_avaliar_respeita_as_bordas[21.0-APROVADO]
_________________
tests/test_laudo.py:35: in test_avaliar_respeita_as_bordas
    assert avaliar(valor, 19.0, 21.0) == esperado
E   AssertionError: assert 'REPROVADO' == 'APROVADO'
E
E     - APROVADO
E     ? ^
E     + REPROVADO
```

```
E     ? ^^
========================== short test summary info
============================
FAILED tests/test_laudo.py::test_avaliar_respeita_as_bordas[19.0-
APROVADO] - ...
FAILED tests/test_laudo.py::test_avaliar_respeita_as_bordas[21.0-
APROVADO] - ...
2 failed, 14 passed in 0.34s
```

Leia sempre nesta ordem:

1. **Qual teste?** A faixa `____ test_avaliar_respeita_as_bordas[19.0-APROVADO] ____` traz o nome e o caso.
2. **Onde?** `tests/test_laudo.py:35`, na linha do `assert`.
3. **Obtido x esperado.** `assert 'REPROVADO' == 'APROVADO'`: à esquerda o que o código devolveu, à direita o que o teste esperava.
4. **Padrão.** No resumo, só as bordas (19.0 e 21.0) falharam. Isso já aponta o bug: a comparação nas bordas.

Sem o `--tb=short`, o pytest mostra também o código do teste inteiro e os valores dos parâmetros (`valor = 19.0, esperado = 'APROVADO'`). Com `approx`, mostra a tolerância: um bug que divide a média por `len(valores) - 1` aparece como `assert 30.15 == 20.1 ± 2.0e-05`. O número obtido é uma pista: "que conta dá 30.15?" (`60.3 / 2`, em vez de `60.3 / 3`).

5. Anatomia de um bom teste

O padrão AAA. Todo teste tem três partes: **Arrange** (preparar os dados), **Act** (chamar o código, de preferência uma vez) e **Assert** (conferir). Separe-as com linha em branco ou comentário, como no teste da seção 3.

Nomes que contam a história. O nome aparece no relatório de falha: em vez de `test_1` ou `test_erro`, use `test_media_de_lista_vazia_levanta_erro`.

Regras de ouro. Um comportamento por teste (se o nome precisa de "e", talvez sejam dois). Testes independentes: a ordem não pode importar. Rápidos: sem internet, sem banco real, sem `sleep`. Determinísticos: o mesmo código dá sempre o mesmo resultado.

Como escolher os casos (o pensamento por trás)

Não dá para testar todos os números do mundo. Agrupe as entradas em "famílias" que devem se comportar igual e teste **um representante de cada**, mais as **bordas** entre elas. Para `avaliar(valor, 19.0, 21.0)`:

```
   REPROVADO      |        APROVADO          |     REPROVADO
------------------+--------------------------+------------------
      18.99     19.0          20.0          21.0    21.01
  (logo abaixo) (borda)    (meio)        (borda) (logo acima)
```

Cinco casos cobrem a regra inteira. Bugs adoram as bordas (`<` no lugar de `<=`). Para listas, lembre também: vazia, um elemento, muitos. Para números: zero, negativo, decimal. Para texto: vazio, espaços, acentos.

6. Floats e erros: `pytest.approx` e `pytest.raises`

approx. Nunca compare floats com `==` exato. `pytest.approx` aceita uma pequena diferença (por padrão, um milionésimo do valor). Funciona com listas, tuplas e dicionários, e você escolhe a tolerância com `abs=` (absoluta) ou `rel=` (relativa). Rode `exemplos/02_approx_e_raises.py`: `20.1 == pytest.approx(20.0)` dá `False`, mas `20.1 == pytest.approx(20.0, abs=0.2)` dá `True`.

raises. Quando o comportamento correto é **levantar um erro**, use `pytest.raises` como bloco `with`. O teste passa se o erro acontecer, e falha se não acontecer.

```python
def test_media_de_lista_vazia_levanta_erro():
    with pytest.raises(ValueError, match="vazia"):
        media([])
```

- `match=` confere a mensagem com uma expressão regular (`re.search`, módulo 05). Se a mensagem tiver parênteses ou pontos, use `match=re.escape("valor (kg) inválido")`.
- `with pytest.raises(ValueError) as excinfo:` guarda o erro; depois do bloco, `str(excinfo.value)` é a mensagem.

> ⚠️ **Erro comum:** `pytest.raises(Exception)`. Ele aceita **qualquer** erro, até um `NameError` causado por um bug, e o teste passa por acaso. Sempre use o tipo exato.

> ⚠️ **Erro comum:** colocar várias linhas dentro do `with pytest.raises`. A linha que levanta o erro deve ser a última do bloco; o que vem depois dela nunca roda.

7. Vários casos: `@pytest.mark.parametrize`

Testar cinco valores copiando o teste cinco vezes é repetição. O `parametrize` roda a mesma função para cada linha de uma tabela:

```python
@pytest.mark.parametrize(
    ("valor", "esperado"),
    [
        (19.0, "APROVADO"),  # na borda inferior
        (20.0, "APROVADO"),
        (21.0, "APROVADO"),  # na borda superior
        (18.99, "REPROVADO"),
        (21.01, "REPROVADO"),
    ],
)
def test_avaliar_respeita_as_bordas(valor, esperado):
    assert avaliar(valor, 19.0, 21.0) == esperado
```

Cada linha vira um teste separado, com o caso entre colchetes: `test_avaliar_respeita_as_bordas[19.0-APROVADO]`. Se um caso falha, os outros continuam. Quando os valores são longos ou têm acentos, o nome gerado fica feio (`Tra\xe7\xe3o: reprovado-esperado1`). Dê nomes aos casos com `ids=["um-liberado", "um-reprovado", "dois-ensaios"]` como último argumento do `parametrize` (veja `tests/test_observacao.py` no mini projeto).

🔮 **Preveja antes de rodar:** quantos testes o pytest conta neste arquivo?

```python
@pytest.mark.parametrize("linha", ["A", "B", "C", "D"])
@pytest.mark.parametrize("ensaio", ["espessura", "cof"])
def test_combinacoes(linha, ensaio):
    assert linha in "ABCD"
```

▼ **Resposta**

8. Dois `parametrize` empilhados geram todas as combinações: 4 linhas × 2 ensaios. Saída real: `8 passed in 0.01s`.

🔧 **Modifique:** no mini projeto, acrescente ao `test_avaliar_respeita_as_bordas` o caso `(0.0, "REPROVADO")` e rode `python -m pytest -k bordas -v`. Depois mude o esperado de propósito para `"APROVADO"` e leia a falha.

8. Fixtures: preparando o terreno

Vários testes precisam dos mesmos dados: uma lista de medições, um arquivo, uma conexão. Uma **fixture** é uma função que prepara esse terreno. O teste a pede pelo **nome do parâmetro** e o pytest entrega o valor pronto. Pense na bancada que o técnico arruma antes de cada ensaio: a fixture arruma a bancada, o teste faz o ensaio.

```
@pytest.fixture
def medicoes_espessura() -> list[float]:
    """Três medições de espessura (µm) de uma bobina com nominal de 20 µm."""
    return [20.1, 19.8, 20.4]


def test_media_de_tres_medicoes(medicoes_espessura):
    assert media(medicoes_espessura) == pytest.approx(20.1)
```

- **Uma fixture nova para cada teste** (por padrão). Se um teste altera a lista, o próximo recebe outra, intacta.
- **Fixture pode usar fixture.** No mini projeto, `arquivo_medicoes` pede `tmp_path` e `medicoes_espessura`.
- `conftest.py` guarda as fixtures compartilhadas. O pytest carrega esse arquivo sozinho, e as fixtures dele ficam disponíveis para os testes da pasta, sem `import`.
- **Limpeza com `yield`.** O que vem antes do `yield` prepara; o que vem depois roda no fim do teste, mesmo que ele falhe:

```
@pytest.fixture
def conexao():
    con = sqlite3.connect(":memory:")   # prepara (módulo 11)
    yield con                           # entrega para o teste
    con.close()                         # limpa no final
```

🔮 **Preveja antes de rodar:** com `python -m pytest -s`, quantas vezes aparece "criando bobina"? O segundo teste passa?

```
@pytest.fixture
def bobina():
    print("criando bobina")
    return {"op": "12345", "peso": 250.0}


def test_peso(bobina):
```

```
    bobina["peso"] = 0.0
    assert bobina["peso"] == 0.0


def test_peso_continua_original(bobina):
    assert bobina["peso"] == 250.0
```

▼ **Resposta**

Aparece **duas vezes**, e os dois testes passam (`2 passed`). A fixture roda de novo para cada teste, então o segundo recebe um dicionário novo, com o peso original. É isso que deixa os testes independentes.

9. Arquivos de verdade em pasta temporária: `tmp_path`

Para testar código que lê ou grava arquivos sem sujar seu computador, a fixture pronta `tmp_path` entrega um `Path` (módulo 06) para uma **pasta temporária nova, só deste teste**. O padrão é: criar o arquivo do caso, chamar a função, conferir.

```
def test_ler_medicoes_ignora_linhas_vazias(tmp_path):
    arquivo = tmp_path / "com_buracos.txt"
    arquivo.write_text("20,1\n\n   \n19,8\n", encoding="utf-8")
    assert ler_medicoes(arquivo) == [20.1, 19.8]
```

Quando vários testes criam arquivos, uma *fixture fábrica* evita repetição: em vez de um valor, ela devolve uma **função** que cria arquivos. Veja a fixture `criar_arquivo` no teste do exercício 08 (`testes/test_m09_ex08_ler_medicoes.py`).

10. O que foi impresso e o que foi registrado: `capsys` e `caplog`

capsys. Funções que imprimem também se testam. `capsys.readouterr()` devolve o que saiu no terminal até aquele momento, em `.out` (saída normal) e `.err` (saída de erro).

```
def test_conferir_op_sem_acessar_o_erp(monkeypatch, capsys,
medicoes_espessura):
    monkeypatch.setattr(laudo, "buscar_nominal_no_erp", lambda op: 20.0)

    resultado = conferir_op("12345", medicoes_espessura, 5.0)
```

```
    assert resultado == "APROVADO"
    saida = capsys.readouterr().out
    assert saida == "OP 12345: média 20.10 em [19.00, 21.00] ->
APROVADO\n"
```

Repare no `\n` no fim: o `print` acrescenta a quebra de linha (o `monkeypatch` da primeira linha fica para a seção 11).

🔮 **Preveja antes de rodar:** este teste passa ou falha?

```
def test_capsys_duas_vezes(capsys):
    print("primeira")
    a = capsys.readouterr().out
    print("segunda")
    b = capsys.readouterr().out
    assert (a, b) == ("primeira\n", "segunda\n")
```

▼ **Resposta**

Passa. Cada `readouterr()` devolve o que foi impresso **desde a leitura anterior** e "esvazia" o capturado. Por isso `b` tem só a segunda linha.

caplog. Para código que usa `logging` (seção 17), a fixture `caplog` captura os registros. Cada registro tem nível (`levelname`), nome do logger (`name`) e mensagem (`getMessage()`); `caplog.text` junta tudo num texto só.

```
def test_aviso_vai_para_o_log(caplog):
    logger = logging.getLogger("inspecao")
    logger.warning("OP %s FORA", "12346")
    assert caplog.records[0].levelname == "WARNING"
    assert "12346 FORA" in caplog.text
```

Por padrão o logging só deixa passar WARNING para cima. Para capturar INFO, chame `caplog.set_level(logging.INFO)` no começo do teste.

11. Trocando peças: `monkeypatch`

O `conferir_op` do mini projeto consulta o ERP. Nos testes, não dá para depender do TOTVS: é lento, precisa de rede e senha, e os dados mudam. A fixture `monkeypatch` **troca uma peça durante o teste** e desfaz a troca sozinha no final:

```
monkeypatch.setattr(laudo, "buscar_nominal_no_erp", lambda op: 20.0)
```

Lê-se: "no módulo `laudo`, durante este teste, `buscar_nominal_no_erp` passa a ser uma função que sempre devolve 20.0".

Chamada	Efeito durante o teste
`monkeypatch.setattr(modulo, "nome", novo)`	troca uma função ou atributo
`monkeypatch.setenv("LAB_OPERADOR", "Maria")`	cria ou altera variável de ambiente
`monkeypatch.delenv("LAB_OPERADOR", raising=False)`	apaga a variável (se existir)
`monkeypatch.chdir(tmp_path)`	muda a pasta atual

> ⚠️ **Erro comum:** trocar a peça no lugar errado. `conferir_op` procura o nome `buscar_nominal_no_erp` **dentro do módulo** `laudo` na hora em que roda, por isso trocamos `laudo.buscar_nominal_no_erp`. Regra: troque onde a função é **usada**.

A alternativa elegante: injeção de dependência. Quando é você quem escreve o código, receba a dependência como parâmetro, com o valor real como padrão (funções são valores, módulo 04). O teste passa a função falsa direto, sem `monkeypatch`:

```
def conferir_op(op, valores, tolerancia_pct,
buscar_nominal=buscar_nominal_no_erp):
    nominal = buscar_nominal(op)
    ...


def test_conferir_op_com_erp_falso():
    resultado = conferir_op("12345", [20.1, 19.8, 20.4], 5.0,
buscar_nominal=lambda op: 20.0)
    assert resultado == "APROVADO"
```

> 💡 **Dica:** prefira injeção de dependência no código novo; use `monkeypatch` para código que você não pode mudar e para variáveis de ambiente. Os dois voltam nos módulos 12 e 13 (APIs e robôs testados sem internet).

12. Exemplo resolvido: testes para uma função que você não escreveu

Três exercícios deste módulo invertem o papel: a função existe e **você escreve os testes**. O curso confere se eles aprovam a versão certa e reprovam versões com bug. Veja como eu penso um caso parecido, passo a passo.

O problema: testar `contar_fora(valores, minimo, maximo)`, que conta quantos valores estão fora de `[minimo, maximo]`, com as bordas dentro.

Passo 1, listar as regras. Releio a especificação e anoto uma regra por linha: (a) conta os de fora; (b) bordas estão dentro; (c) lista vazia dá 0.

Passo 2, um caso por regra. Para (a), uma lista com valores dentro e fora, e um resultado que não seja óbvio (2, não 0). Para (b), valores exatamente nas bordas. Para (c), lista vazia.

Passo 3, perguntar "que bug este caso pega?". O das bordas pega quem usou `<=` no lugar de `<`. Se um caso não pega nenhum bug novo, é repetido.

Passo 4, escrever, com mensagens úteis:

```
def testar_contar_fora(contar_fora):
    # Regra: conta os valores fora de [mínimo, máximo]
    assert contar_fora([20.0, 22.5, 18.0], 19.0, 21.0) == 2
    # Bordas: estão DENTRO
    assert contar_fora([19.0, 21.0], 19.0, 21.0) == 0, "bordas devem
estar dentro"
    # Lista vazia: nada fora
    assert contar_fora([], 19.0, 21.0) == 0
```

Passo 5, provar que os testes funcionam. Rodo contra uma versão certa e uma com bug (`valor <= minimo`):

```
versão correta: aprovada
versão com bug: reprovada (bordas devem estar dentro)
```

Um teste que nunca falha não protege nada. Sempre confira que ele **consegue** falhar.

13. TDD passo a passo: vermelho → verde → refatorar

TDD (*Test-Driven Development*) inverte a ordem: o teste vem **antes** do código. O ciclo tem três passos curtos:

```
VERMELHO: teste que falha -> VERDE: código mínimo -> REFATORAR: melhora
com tudo verde
   ^                                                                |
   +------------------------ próximo comportamento --------------------+
```

Você pensa no **uso** antes da implementação, avança em passos pequenos e termina com testes para tudo. Vamos construir `ler_liberacoes`, que lê a observação de liberação da planilha do projeto final: `"COF: liberado; Selagem: reprovado"` deve virar `{"COF": True, "SELAGEM": False}`.

Ciclo 1: o caso mais simples

Vermelho. Em `tests/test_observacao.py`:

```python
from observacao import ler_liberacoes


def test_um_ensaio_liberado():
    assert ler_liberacoes("COF: liberado") == {"COF": True}
```

O arquivo `observacao.py` ainda nem existe. Trecho da saída:

```
> python -m pytest tests/test_observacao.py -q
...
E   ModuleNotFoundError: No module named 'observacao'
...
1 error in 0.16s
```

Vermelho também conta: o teste pediu algo que não existe.

Verde. O código **mínimo** para passar. Mínimo mesmo:

```python
def ler_liberacoes(observacao: str) -> dict[str, bool]:
    return {"COF": True}
```

Resultado: `1 passed`. Parece trapaça, mas prova que o teste está ligado ao código. Quem vai forçar a lógica de verdade é o próximo teste.

Ciclo 2: generalizar

Vermelho. Um caso que o "código falso" não resolve:

```
def test_um_ensaio_reprovado():
    assert ler_liberacoes("Tração: reprovado") == {"TRAÇÃO": False}
```

```
.F
[100%]
...
>       assert ler_liberacoes("Tração: reprovado") == {"TRAÇÃO": False}
E       AssertionError: assert {'COF': True} == {'TRAÇÃO': False}
```

Verde. Agora sim, separar ensaio e status:

```
def ler_liberacoes(observacao: str) -> dict[str, bool]:
    ensaio, status = observacao.split(":")
    return {ensaio.strip().upper(): status.strip() == "liberado"}
```

Resultado: `2 passed`.

Ciclo 3: vários ensaios

Vermelho.

```
def test_varios_ensaios_separados_por_ponto_e_virgula():
    resultado = ler_liberacoes("COF: liberado; Selagem: reprovado")
    assert resultado == {"COF": True, "SELAGEM": False}
```

```
..F
[100%]
...
>       ensaio, status = observacao.split(":")
        ^^^^^^^^^^^^^^
E       ValueError: too many values to unpack (expected 2)

observacao.py:2: ValueError
```

O erro está no **nosso** código: com dois ensaios, o `split(":")` devolve três pedaços.

Verde. Primeiro separar por `;`, depois cada item por `:`. Resultado: `3 passed`.

```
def ler_liberacoes(observacao: str) -> dict[str, bool]:
    resultado = {}
    for item in observacao.split(";"):
```

```
        ensaio, status = item.split(":")
        resultado[ensaio.strip().upper()] = status.strip() == "liberado"
    return resultado
```

Ciclo 4: status desconhecido é erro

Hoje, `"COF: pendente"` vira `{"COF": False}`, como se fosse reprovado. Perigoso. Regra nova: status desconhecido levanta `ValueError`.

Vermelho.

```
def test_status_desconhecido_levanta_erro():
    with pytest.raises(ValueError, match="pendente"):
        ler_liberacoes("COF: pendente")
```

```
...F
[100%]
...
>       with pytest.raises(ValueError, match="pendente"):
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       Failed: DID NOT RAISE ValueError
```

Verde. Dentro do laço, um `if status == "liberado"` / `elif status == "reprovado"` e, no `else`, `raise ValueError(f"status desconhecido: {status!r}")`. Resultado: `4 passed`.

Refatorar

Tudo verde. Agora, **sem mudar o comportamento**, melhoro o código: o `if/elif` vira um dicionário de status válidos, e a leitura de um item ganha uma função com nome. Os três primeiros testes, iguais na forma, viram um `parametrize` (o da seção 7).

```
STATUS_VALIDOS = {"liberado": True, "reprovado": False}


def _ler_item(item: str) -> tuple[str, bool]:
    """Converte 'COF: liberado' em ('COF', True)."""
    ensaio, status = item.split(":")
    status = status.strip()
    if status not in STATUS_VALIDOS:
        raise ValueError(f"status desconhecido: {status!r}")
    return ensaio.strip().upper(), STATUS_VALIDOS[status]
```

```python
def ler_liberacoes(observacao: str) -> dict[str, bool]:
    """Converte 'COF: liberado; Selagem: reprovado' em {'COF': True,
'SELAGEM': False}."""
    return dict(_ler_item(item) for item in observacao.split(";"))
```

Rodando de novo, os 4 testes continuam passando. Este é o `observacao.py` do mini projeto. Repare como os testes deram coragem para reescrever a função inteira.

> 🔧 **Modifique:** o técnico às vezes digita `"COF: LIBERADO"`. Faça um ciclo de TDD: escreva o teste (vermelho), mude uma linha de `_ler_item` (verde) e rode tudo.

> 💡 **Dica:** no TDD, rode com `-x` para parar na primeira falha e trabalhe **um teste por vez**. Nos exercícios, na pasta `curso-python`: `python -m pytest 09-testes-e-qualidade/testes/test_m09_ex08_ler_medicoes.py -x`.

14. Cobertura de testes (conceito)

Cobertura responde: "quais linhas do meu código rodaram durante os testes?". A ferramenta mais usada é o plugin `pytest-cov` (`python -m pip install pytest-cov`). No mini projeto:

```
> python -m pytest --cov=laudo --cov=observacao --cov-report=term-missing
.................
[100%]
================================ tests coverage
================================
_______________ coverage: platform linux, python 3.13.16-final-0
_______________

Name            Stmts   Miss  Cover   Missing
---------------------------------------------
laudo.py           30      2    93%   12, 47
observacao.py       9      0   100%
---------------------------------------------
TOTAL              39      2    95%
16 passed in 0.09s
```

A coluna `Missing` lista as linhas que **nenhum** teste executou:

- Linha 12 de `laudo.py`: o `raise` de "nominal deve ser positivo". Esquecemos de testar nominal zero ou negativo. Lacuna real.

- Linha 47: o `raise ConnectionError` do ERP, que trocamos com `monkeypatch` de propósito. Lacuna aceitável.

> ⚠️ **Erro comum:** perseguir 100% como meta. Cobertura mostra o que **não** foi testado, mas não prova que o resto foi bem testado: um teste sem `assert` "cobre" linhas e não confere nada. Use a cobertura como lanterna, não como nota.

> 🔧 **Modifique:** escreva no mini projeto um teste para `calcular_limites(0.0, 5.0)` com `pytest.raises(ValueError)` e rode a cobertura de novo. A linha 12 sai do `Missing`.

15. ruff: lint e formatação automáticos

Revisar espaço em branco e import esquecido é trabalho de máquina. O **ruff** faz duas coisas, muito rápido:

- `ruff check` (**lint**) procura problemas: import não usado, variável esquecida, armadilhas conhecidas.
- `ruff format` (**formatação**) padroniza espaços, quebras de linha e aspas. Muda a aparência, nunca o comportamento. (`black` e `isort`, que você conhece de nome, fazem parte do que o ruff faz, num programa só.)

O arquivo `ruff.toml`, na raiz do curso, vale para todos os módulos: linhas de até 95 caracteres, código compatível com Python 3.10+ e estas famílias de regras:

Código	Família	Exemplo do que pega
`E`, `W`	estilo (pycodestyle)	dois imports na mesma linha, espaço no fim da linha
`F`	erros prováveis (pyflakes)	import não usado, variável nunca usada
`I`	ordem dos imports (isort)	imports fora da ordem padrão
`B`	armadilhas (bugbear)	lista como valor padrão de parâmetro
`UP`	modernização (pyupgrade)	`"%s" % x` no lugar de f-string

O `exemplos/05_ruff_na_pratica.py` cria um arquivo bagunçado numa pasta temporária e roda o ruff nele. O arquivo:

```python
import os, sys
import json


def conferir(valores, minimo, maximo, avisos=[]):
    aprovados = []
    total = len(valores)
    for i in range(len(valores)):
        v = valores[i]
        if v >= minimo and v <= maximo:
            aprovados.append(v)
        else:
            avisos.append("fora: %s" % v)
    return len(aprovados) == len(valores)
```

E o que o ruff diz:

```
> ruff check --output-format concise conferencia.py
conferencia.py:1:1: E401 [*] Multiple imports on one line
conferencia.py:1:1: I001 [*] Import block is un-sorted or un-formatted
conferencia.py:1:8: F401 [*] `os` imported but unused
conferencia.py:1:12: F401 [*] `sys` imported but unused
conferencia.py:2:8: F401 [*] `json` imported but unused
conferencia.py:5:46: B006 Do not use mutable data structures for argument
defaults
conferencia.py:7:5: F841 Local variable `total` is assigned to but never
used
conferencia.py:13:27: UP031 Use format specifiers instead of percent
format
Found 8 errors.
[*] 5 fixable with the `--fix` option (3 hidden fixes can be enabled with
the `--unsafe-fixes` option).
```

Cada linha traz `arquivo:linha:coluna: CÓDIGO mensagem`. O `[*]` marca o que o ruff conserta sozinho com `ruff check --fix`. Os outros três exigem você: apagar `total`, trocar o `%` por f-string e resolver o `avisos=[]`. (Sem `--output-format concise`, cada problema vem com o trecho do código e uma sugestão.)

> 🔮 **Preveja antes de rodar:** o B006 não é frescura. Lembra do módulo 04? O que este código imprime?
>
> ```python
> def conferir(valor, avisos=[]):
>     if valor > 21.0:
>         avisos.append(f"fora: {valor}")
>     return avisos
> ```

```
print(conferir(22.0))
print(conferir(23.5))
```

▼ **Resposta**

```
['fora: 22.0']
['fora: 22.0', 'fora: 23.5']
```

A lista padrão é criada **uma vez**, quando a função é definida, e é reaproveitada em todas as chamadas: o aviso da primeira OP "vaza" para a segunda. A correção é `avisos=None` e, dentro, `if avisos is None: avisos = []`. O ruff pegou um bug real.

No dia a dia: `ruff check .` lista os problemas da pasta, `ruff check --fix .` conserta os que têm `[*]`, `ruff format --diff .` mostra o que a formatação mudaria, `ruff format .` formata tudo e `ruff format --check .` só confere (útil no CI).

> 💡 **Dica:** no VS Code, instale a extensão **Ruff** (da Astral), escolha-a como formatador padrão de Python e ligue **Format On Save** nas configurações. O código passa a ser formatado sempre que você salva.

16. Type hints e mypy (conceito)

Você lê type hints desde o módulo 01, nos exercícios: `def media(valores: list[float]) -> float`. Eles dizem o que a função espera e o que devolve. O ponto que pouca gente sabe: **o Python não confere os type hints ao rodar.** Em `exemplos/07_type_hints.py`, a função `juntar(a: int, b: int) -> int` recebe `"10"` e `"20"` e devolve `"1020"`, sem reclamar. Quem confere os tipos é uma ferramenta separada, o **mypy**, que lê o código **sem executá-lo**. Num arquivo `pesos.py`, com três problemas que só apareceriam rodando o programa num caso específico:

```
def somar_pesos(pesos: list[float]) -> float:
    return sum(pesos)


def peso_medio(pesos: list[float]) -> float:
    if not pesos:
        return None
```

```
    return somar_pesos(pesos) / len(pesos)


total = somar_pesos(["250,5", "120,0"])
print(total.upper())
```

```
> python -m pip install mypy
> mypy pesos.py
pesos.py:7: error: Incompatible return value type (got "None", expected
"float")  [return-value]
pesos.py:11: error: List item 0 has incompatible type "str"; expected
"float"  [list-item]
pesos.py:11: error: List item 1 has incompatible type "str"; expected
"float"  [list-item]
pesos.py:12: error: "float" has no attribute "upper"  [attr-defined]
Found 4 errors in 1 file (checked 1 source file)
```

A linha 7 é a mais interessante: a anotação promete `float`, mas a função pode devolver `None`. Se for de propósito, a anotação certa é `-> float | None`, e o mypy passa a obrigar quem chama a tratar o `None`.

Nos exercícios "escreva os testes" aparece `Callable[[list[float]], float]` (de `collections.abc`): "uma função que recebe lista de floats e devolve float". A sua função de teste recebe **outra função** como parâmetro.

> 💡 **Dica:** type hints são **graduais**: comece pelas funções públicas. O VS Code (Pylance) já usa as anotações para autocompletar e sublinhar erros enquanto você digita.

17. logging: os prints dos profissionais

O robô do projeto final vai rodar sozinho, de madrugada. Ninguém vai ver o terminal. Se algo der errado, você precisa de um **registro** do que aconteceu, com data, hora e gravidade. É para isso que existe o módulo `logging`.

Nível	Quando usar	Exemplo
`DEBUG`	detalhes para quem está depurando	"lendo linha 37 da aba A"
`INFO`	o normal, marcos do processo	"OP 12345 conferida"
`WARNING`	algo estranho, mas o programa segue	"OP 12346 fora da tolerância"

Nível	Quando usar	Exemplo
`ERROR`	uma operação falhou	"não consegui ler o valor da OP 12347"
`CRITICAL`	o programa não consegue continuar	"planilha de regras não encontrada"

O padrão de uso tem três partes: (1) em cada módulo, um logger com nome, `logger = logging.getLogger(__name__)`; (2) **uma vez**, no início do programa, a configuração com `logging.basicConfig(...)`; (3) no código, `logger.info(...)`, `logger.warning(...)` etc. Trecho de `exemplos/03_logging_inspecao.py`:

```
logging.basicConfig(
    level=logging.INFO,  # DEBUG fica de fora; INFO e acima aparecem
    format="%(asctime)s %(levelname)-8s %(name)s: %(message)s",
    datefmt="%d/%m/%Y %H:%M:%S",
    handlers=[
        logging.StreamHandler(),  # terminal
        logging.FileHandler(ARQUIVO_LOG, mode="w", encoding="utf-8"),  # arquivo
    ],
)
```

```
> python exemplos/03_logging_inspecao.py
29/10/2026 02:43:42 INFO     inspecao: Início da conferência
29/10/2026 02:43:42 INFO     inspecao: OP 12345: 20.10 dentro de [19.00, 21.00]
29/10/2026 02:43:42 WARNING  inspecao: OP 12346: 22.40 FORA de [19.00, 21.00]
29/10/2026 02:43:42 ERROR    inspecao: OP 12347: valor ilegível na planilha
Traceback (most recent call last):
  File ".../exemplos/03_logging_inspecao.py", line 47, in main
    conferir("12347", float("vinte"), 19.0, 21.0)
                      ~~~~~^^^^^^^^^
ValueError: could not convert string to float: 'vinte'
29/10/2026 02:43:42 INFO     inspecao: Fim da conferência

Log gravado em: inspecao.log
```

A data e a hora serão as do seu computador (encurtei o caminho do arquivo). O mesmo conteúdo foi gravado em `exemplos/saida/inspecao.log`, em UTF-8. Repare:

- `logger.exception(...)`, dentro de um `except`, registra no nível ERROR **com o traceback completo**. É o que você vai querer ler no dia seguinte.

- O `logger.debug(...)` do exemplo não apareceu, porque o nível mínimo é INFO. Para investigar um problema, mude para `level=logging.DEBUG` sem tocar no resto do código.
- O exemplo usa o estilo tradicional, `logger.info("OP %s: %.2f", op, valor)`, em que o texto só é montado se a mensagem for aparecer. f-string também funciona.

> ⚠️ **Erro comum:** chamar `basicConfig` dentro de funções ou em vários módulos. Só a primeira chamada vale; as outras são ignoradas em silêncio. Configure uma vez, no `main`.

18. Depuração: `breakpoint()` e o debugger do VS Code

Depurar é investigar, com o método de uma análise de não conformidade: (1) **reproduza** com um caso pequeno, de preferência um teste que falha; (2) **observe** o que entra, o que sai e onde muda; (3) **formule uma hipótese** ("o total está sendo zerado a cada volta"); (4) **teste a hipótese** no REPL ou no depurador; (5) **corrija** e mantenha o teste, que impede a volta do bug.

`print` ajuda a observar, mas o **depurador** é mais poderoso: ele **pausa** o programa numa linha e deixa você ver todas as variáveis e avançar uma linha por vez.

breakpoint() e o pdb

O `exemplos/04_depuracao.py` calcula a média do COF e erra:

```
> python exemplos/04_depuracao.py
Leituras: ['0,32', '0,35', '0,30']
Média calculada: 0.1000
Média esperada:  0.3233  (0,32 + 0,35 + 0,30) / 3
```

Dentro de `media_cof` há um `breakpoint()`, ativado com `--depurar`. Quando o Python chega nele, abre o **pdb**, o depurador que vem com o Python. Trecho de uma sessão real (os comandos digitados vêm depois de `(Pdb)`; encurtei os caminhos):

```
> python exemplos/04_depuracao.py --depurar
> .../exemplos/04_depuracao.py(26)media_cof()
-> breakpoint()  # o programa para aqui e abre o depurador
(Pdb) n
> .../exemplos/04_depuracao.py(27)media_cof()
-> total = 0.0
...
```

```
> .../exemplos/04_depuracao.py(30)media_cof()
-> total = valor  # BUG de propósito: deveria ser total += valor
(Pdb) p valor, total
(0.35, 0.32)
(Pdb) n
> .../exemplos/04_depuracao.py(28)media_cof()
-> for leitura in leituras:
(Pdb) p total
0.35
(Pdb) c
```

Na segunda volta, antes da linha 30, `valor` é 0.35 e `total` é 0.32. Depois dela, o `total` deveria ser `0.32 + 0.35 = 0.67`, mas virou `0.35`: foi **substituído**, não somado. Hipótese confirmada: `total = valor` deveria ser `total += valor`.

Comando pdb	Faz
`n` (next)	executa a linha atual e para na próxima
`s` (step)	entra na função chamada na linha atual
`c` (continue)	segue até o próximo breakpoint (ou o fim)
`p expr` / `pp expr`	mostra o valor de uma expressão (`pp` formata listas e dicionários)
`l` (list) / `w` (where)	mostra o código em volta / a pilha de chamadas
`q` (quit)	encerra o programa

> 💡 **Dica:** `python -m pytest --pdb` abre o pdb **no ponto exato da falha** de um teste. E a variável de ambiente `PYTHONBREAKPOINT=0` desliga todos os `breakpoint()`. Antes do commit, remova os `breakpoint()` esquecidos.

> ⚠️ **Erro comum:** estranhar a linha onde o pdb para. A partir do Python 3.13, ele para **na própria linha** do `breakpoint()`; em versões anteriores, na linha seguinte.

O debugger do VS Code

O VS Code faz o mesmo com cliques:

1. Abra `exemplos/04_depuracao.py`, clique à esquerda do número da linha `total = valor` (ou **F9**): aparece a bolinha vermelha do *breakpoint*.

2. **F5** → **Python Debugger: Current File**. O programa para ali. O painel **Variables** mostra `leitura`, `valor` e `total`; em **Watch**, acrescente expressões como `total + valor`.
3. Avance com **F10** (próxima linha), **F11** (entrar na função), **Shift+F11** (sair dela) e **F5** (continuar). Na **Debug Console**, digite qualquer expressão Python.
4. Botão direito na bolinha → **Edit Breakpoint** cria um *breakpoint condicional* (`leitura == "0,30"`): só para quando a condição é verdadeira.

Para depurar um **teste**, configure o pytest no VS Code (ícone do frasco, **Testing** → **Configure Python Tests** → **pytest** → pasta `curso-python`) e use **Debug Test** no botão direito do teste.

19. Git no dia a dia

No módulo 00 você usou Git sozinho, na branch principal. Num time, ninguém mexe direto na `main`: cada mudança nasce numa **branch**, vira um **pull request** (PR), passa por **revisão** e só então entra na `main`, que fica sempre funcionando.

Fluxo executado num repositório de exemplo (no seu computador, o Git pode responder em português):

```
> git switch -c corrige-borda-tolerancia
Switched to a new branch 'corrige-borda-tolerancia'
```

Você edita o código e roda os testes. Antes do commit, **sempre** confira o que mudou (`git status` lista os arquivos; `git diff` mostra as linhas):

```
> git diff
diff --git a/tolerancia.py b/tolerancia.py
index 3524197..d5f5cb5 100644
--- a/tolerancia.py
+++ b/tolerancia.py
@@ -1,2 +1,2 @@
 def avaliar(valor, minimo, maximo):
-    return "APROVADO" if minimo < valor < maximo else "REPROVADO"
+    return "APROVADO" if minimo <= valor <= maximo else "REPROVADO"
```

Linhas com `-` saíram, com `+` entraram. Agora o commit, com título e corpo (cada `-m` vira um parágrafo):

```
> git add tolerancia.py
> git commit -m "Corrige bordas na avaliação de tolerância" -m "Valores
iguais ao mínimo ou ao máximo estão dentro da especificação. Antes, uma
bobina com espessura exatamente no limite era reprovada."
```

```
[corrige-borda-tolerancia cf2cfbb] Corrige bordas na avaliação de
tolerância
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Por fim, `git push -u origin corrige-borda-tolerancia` manda a branch para o GitHub, que oferece o botão **Compare & pull request**.

Commits pequenos, mensagens boas

Um commit é **uma ideia**: "corrige a borda", e não "corrige a borda, muda o relatório e renomeia três funções". Commits pequenos são fáceis de revisar e de desfazer. Mensagens como `ajustes` ou `commit final agora vai` não dizem nada; `Troca prints de aviso por logging` diz. Título no imperativo ("Corrige", "Adiciona", "Remove"), com até uns 50 caracteres e sem ponto final. Se precisar, deixe uma linha em branco e explique **por que** mudou (o "o quê" o diff já mostra). `git add -p` mostra cada trecho alterado e pergunta se ele entra no commit: é o jeito de separar duas ideias misturadas no mesmo arquivo.

Pull request e revisão de código

A descrição de um PR responde três perguntas:

```
O que muda
- avaliar() passa a considerar as bordas como dentro da tolerância.

Por que
- Bobinas com espessura exatamente no limite estavam sendo reprovadas
(laudo da OP 12345).

Como testar
- python -m pytest -k bordas
```

Revisão de código é outra pessoa lendo o seu PR antes do merge: a segunda conferência do laudo. **Como autor:** PR pequeno, testes passando, ruff limpo, descrição clara; releia o próprio diff antes de pedir revisão. **Como revisor:** o código faz o que a descrição diz? Há teste para o caso novo? Os nomes são claros? Comente o código, nunca a pessoa, e sugira ("que tal extrair isto para uma função?") em vez de ordenar.

> ⚠️ **Erro comum:** fazer commit de senha, token ou planilha com dados de cliente. Uma vez no GitHub, considere vazado: o histórico guarda tudo. Use variáveis de ambiente (como no exercício 04) e um `.gitignore` para arquivos como `.env` e `saida/`.

20. Código limpo e refatoração

Código é lido muito mais vezes do que é escrito, inclusive por você daqui a seis meses. Regras práticas:

- **Nomes que contam a história:** `margem`, `tolerancia_pct`, `media_cof`. Nada de `x`, `d`, `aux2`.
- **Funções pequenas que fazem uma coisa.** Se a descrição tem "e" ("lê o arquivo **e** confere **e** imprime"), divida.
- **Sem números mágicos:** `if valor > 0.35` não diz nada; `if valor > COF_MAXIMO` diz.
- **Sem repetição** (a mesma regra em dois lugares vai divergir um dia) e com **retorno antecipado** (casos especiais no começo, sem `if` dentro de `if`).
- **Separe cálculo de impressão:** uma função que **devolve** o texto do relatório é fácil de testar; uma que só imprime precisa de `capsys`.
- **Comentário explica o porquê**, não o quê: `# a norma conta a borda como aprovada`.

Refatorar é mudar a estrutura sem mudar o comportamento, e só se faz com testes passando antes **e** depois. Diante de código antigo sem testes, o primeiro passo é um **teste de caracterização**: ele registra o que o código faz **hoje**, certo ou errado. O `exemplos/06_refatoracao.py` mostra o antes:

```
def proc(d, t):
    r = []
    for x in d:
        if x[1] == "esp":
            if x[2] >= t["esp"][0] and x[2] <= t["esp"][1]:
                r.append((x[0], "OK"))
            else:
                r.append((x[0], "FORA"))
        elif x[1] == "gram":
            if x[2] >= t["gram"][0] and x[2] <= t["gram"][1]:
                r.append((x[0], "OK"))
            else:
                r.append((x[0], "FORA"))
    return r
```

E o depois, com nomes claros, uma função pequena para a regra que se repetia e type hints (`Medicao` e `Limites` são apelidos de tipo definidos no arquivo):

```
def situacao(valor: float, limites: Limites) -> str:
    minimo, maximo = limites
    return "OK" if minimo <= valor <= maximo else "FORA"
```

```python
def conferir_medicoes(
    medicoes: list[Medicao], tolerancias: dict[str, Limites]
) -> list[tuple[str, str]]:
    return [
        (op, situacao(valor, tolerancias[ensaio]))
        for op, ensaio, valor in medicoes
        if ensaio in tolerancias
    ]
```

```
> python exemplos/06_refatoracao.py
Antes:  [('101', 'OK'), ('102', 'OK'), ('103', 'FORA'), ('104', 'FORA'),
('105', 'OK')]
Depois: [('101', 'OK'), ('102', 'OK'), ('103', 'FORA'), ('104', 'FORA'),
('105', 'OK')]
Mesmo resultado: refatoração segura.
```

Repare: o código antigo **ignora** ensaios sem tolerância (o COF da OP 106 sumiu do resultado). Parece um bug, mas a refatoração **manteve** o comportamento. Refatorar e corrigir são passos separados, em commits separados; misturar os dois é o jeito mais rápido de não saber o que quebrou.

> 🔧 **Modifique:** em `exemplos/06_refatoracao.py`, troque `<=` por `<` em `situacao` e rode. O `assert` de caracterização pega a mudança? Desfaça e acrescente às `MEDICOES` o caso `("107", "esp", 19.0)`: a refatoração continua segura?

Lendo mensagens de erro deste módulo

Mensagem	O que significa	Como resolver
`NotImplementedError: Implemente esta função`	o teste chamou o exercício e achou o `raise`	apague a linha e escreva o código
`ERROR collecting ... ModuleNotFoundError`	o `import` do arquivo de teste falhou; o teste nem começou	confira nome do arquivo, pasta de onde rodou e se o código existe
`fixture 'tmp_pat' not found`	o parâmetro não corresponde a nenhuma fixture	erro de digitação ou fixture num `conftest.py` de outra pasta
`Failed: DID NOT RAISE ValueError`	o código terminou sem o erro esperado	falta o `raise`, ou o caso não é inválido como você pensou

Mensagem	O que significa	Como resolver
`Regex pattern did not match.`	o erro certo aconteceu, mas a mensagem não bate com o `match=`	compare `Expected regex` com `Actual message`
`assert 30.15 == 20.1 ± 2.0e-05`	comparação com `approx` falhou	descubra que conta produz o valor obtido
`Unknown 'parametrise' mark, did you mean 'parametrize'?`	erro de digitação no decorator	é `parametrize`, com z
`no tests ran` (código de saída 5)	o pytest não achou testes	prefixos `test_` / `test` nos nomes, ou pasta errada
ruff `F401`, `F841`	import ou variável sem uso	apague; `I001` se resolve com `ruff check --fix`
mypy `Incompatible return value type`	a função promete um tipo e pode devolver outro	corrija o código ou a anotação (`float | None`)

Dica: o `fixture not found` vem seguido da lista de fixtures disponíveis (`available fixtures: ..., caplog, capsys, ..., tmp_path, ...`). Procure nela o nome certo.

Boas práticas

- Rode os testes **antes** de começar a mexer (para saber que estava verde) e depois de cada mudança pequena.
- Todo bug corrigido ganha um teste que falhava antes da correção. Assim ele não volta.
- Um teste que nunca falhou pode estar testando nada: quebre o código de propósito uma vez e veja o vermelho.
- Teste comportamento, não implementação: confira **o que** a função devolve, não **como** ela faz. Assim você refatora sem reescrever os testes.
- Nada de internet, banco real, data/hora atual ou aleatoriedade sem controle nos testes de unidade: injete a dependência ou use `monkeypatch`.
- `ruff check` e `ruff format` antes de cada commit (melhor ainda: ao salvar no VS Code).
- `logging` para o que o programa faz; `print` só para o resultado que o usuário pediu.
- Uma branch por mudança, commits pequenos, mensagens no imperativo.
- Refatorar e corrigir bug são passos separados.

No mundo real

Situação: o laboratório decidiu que a gramatura da linha B passa a ter tolerância de nominal ± 4% (nominal 22 g/m²), em vez de mínimo e máximo fixos. Você mantém o validador de certificados. O caminho, do pedido ao merge:

1. **Branch:** `git switch -c gramatura-linha-b-percentual`.
2. **Teste primeiro (vermelho)**, com os números da norma. 22 ± 4% dá limites 21,12 e 22,88; as bordas e os vizinhos logo fora são os casos que importam:

```
@pytest.mark.parametrize(
    ("valor", "esperado"),
    [(21.12, "OK"), (22.88, "OK"), (21.11, "FORA"), (22.89, "FORA")],
    ids=["borda-inferior", "borda-superior", "logo-abaixo", "logo-
acima"],
)
def
test_gramatura_linha_b_usa_nominal_22_mais_ou_menos_4_porcento(valor,
esperado):
    regra = buscar_regra("B", "gramatura")
    assert classificar(valor, regra) == esperado
```

3. **Código mínimo (verde)** e **refatoração**, com `python -m pytest -x` a cada passo.
4. **Qualidade, commits e PR:** `ruff check .`, `ruff format .`, `python -m pytest -q`; commits pequenos ("Adiciona regra de tolerância percentual") e um PR com "o que muda / por que / como testar".
5. **Integração contínua (CI):** um robô roda os testes a cada push. No GitHub isso se chama **GitHub Actions**, configurado em `.github/workflows/testes.yml`:

```
name: testes
on: [push, pull_request]
jobs:
  qualidade:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: python -m pip install -r requirements.txt
```

```
      - run: ruff check .
      - run: ruff format --check .
      - run: python -m pytest -q
```

6. **Revisão:** a colega pergunta "e a linha A, que continua com mínimo e máximo fixos, tem teste?". Você acrescenta o teste num commit novo, ela aprova, e o merge acontece.

7. **Uma semana depois**, alguém mexe na leitura da planilha e quebra sem querer o cálculo da linha B. O CI fica vermelho no PR dessa pessoa, **antes** de qualquer certificado errado sair. É para esse dia que os testes existem.

Resumo

Preciso...	Uso
conferir um resultado	`assert obtido == esperado, "mensagem"`
comparar floats	`assert valor == pytest.approx(20.1)`
conferir que levanta erro	`with pytest.raises(ValueError, match="texto"):`
testar vários casos	`@pytest.mark.parametrize(("a", "b"), [(1, 2), ...], ids=[...])`
preparar dados reutilizáveis	`@pytest.fixture` (compartilhada em `conftest.py`)
limpar depois do teste	`yield` dentro da fixture
arquivo temporário	fixture `tmp_path` (um `Path`)
capturar `print` / log	`capsys.readouterr().out` / `caplog.records`, `caplog.text`
trocar função / variável de ambiente	`monkeypatch.setattr`, `setenv`, `delenv`
rodar testes	`python -m pytest` (`-v`, `-q`, `-x`, `-k nome`, `--lf`, `-s`, `--pdb`)
cobertura	`python -m pytest --cov=modulo --cov-report=term-missing`
lint / formatação	`ruff check --fix .` / `ruff format .`
conferir tipos	`mypy arquivo.py`
registrar o que acontece	`logger = logging.getLogger(__name__)`; `logger.info(...)`
pausar e investigar	`breakpoint()` → `n`, `s`, `p var`, `c`, `q`
fluxo Git	`git switch -c nome` → `git diff` → `git add` → `git commit` → `git push` → PR
ciclo TDD	vermelho → verde → refatorar, um teste por vez

Você domina este módulo se...

- ☐ Escrevo um teste no padrão AAA e explico, sem consultar, cada parte da saída de uma falha do pytest.
- ☐ Escolho casos de teste pensando em famílias de entrada e bordas, e sei explicar por que `pytest.raises(Exception)` é uma armadilha.
- ☐ Uso `parametrize`, uma fixture minha e as fixtures `tmp_path`, `capsys` e `monkeypatch` sem olhar a aula.
- ☐ Desenvolvo uma função pequena em TDD, dizendo em qual passo do ciclo estou.
- ☐ Explico a diferença entre `ruff check` e `ruff format`, e entre o que o Python faz com type hints e o que o mypy faz.
- ☐ Troco `print` por `logging` com nível e formato certos, e sei onde chamar `basicConfig`.
- ☐ Acho um bug com `breakpoint()` ou com o debugger do VS Code, inspecionando variáveis linha a linha.
- ☐ Faço uma mudança numa branch, com commits pequenos e mensagens no imperativo, e escrevo a descrição de um pull request.

Exercícios

Este módulo tem três tipos de exercício:

- **Implementar (TDD):** os testes já existem e são a especificação. Leia, rode, veja falhar e faça passar, um por vez.
- **Conserte o bug (leitura de código):** não há `NotImplementedError`; o código já está lá, com defeitos. Leia, preveja o que ele faz, rode os testes e corrija.
- **Escreva os testes:** a função a testar chega **como parâmetro** e você escreve os asserts. O curso roda os seus testes contra uma versão certa (nada pode falhar) e contra versões com bug (cada uma tem de ser reprovada). Veja o exemplo resolvido da seção 12. Para usar `pytest.raises`, escreva `import pytest` no topo do arquivo.

Nº	Exercício	Tipo	Nível	O que pratica
01	`ex01_limites_tolerancia`	implementar (TDD)	★☆☆	ler testes como especificação, `approx`, `raises`
02	`ex02_conserte_numero_br`	conserte o bug	★☆☆	ler código, testar hipótese no REPL

Nº	Exercício	Tipo	Nível	O que pratica
03	`ex03_testes_media`	escreva os testes	★☆☆	escolher casos, `approx`, `raises`
04	`ex04_operador_do_laudo`	implementar (TDD)	★☆☆	variável de ambiente, `monkeypatch`
05	`ex05_log_de_inspecao`	implementar (TDD)	★☆☆	`logging` com níveis, `caplog`
06	`ex06_conserte_pesos_por_nf`	conserte o bug	★★☆	acumulador em dicionário, depuração
07	`ex07_testes_classificar`	escreva os testes	★★☆	bordas, rótulos, casos de erro
08	`ex08_ler_medicoes`	implementar (TDD)	★★☆	`tmp_path`, mensagens de erro úteis
09	`ex09_relatorio_no_terminal`	implementar (TDD)	★★☆	`capsys`, formatação brasileira
10	`ex10_conserte_codigos_bobinas`	conserte o bug	★★☆	off-by-one, validação
11	`ex11_revisao_resumo_por_linha`	revisão: conserte o bug	★★★	4 bugs; dicionários, texto BR, exceções
12	`ex12_revisao_testes_lotes`	revisão: escreva os testes	★★★	listas, fatiamento, mutabilidade

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError` (ou conserte o código, nos exercícios de bug) e rode, na pasta `curso-python`:

```
python curso.py testar 09 ex01
```

Desafio do módulo

Do script legado ao código profissional. Você recebe um script antigo de conferência de medições (funciona, mas não tem testes e é difícil de ler). A missão: protegê-lo com testes de caracterização, refatorar em funções pequenas com type hints, trocar `print` por `logging`, deixar o ruff limpo, acrescentar com TDD a regra de liberação pela observação (`"COF: liberado"`) e entregar tudo numa branch com pull request.

Enunciado completo: desafio/README.md.

Quiz

14 perguntas para revisar o módulo: quiz.md.

Para ir além

- Documentação do pytest (em inglês, com ótimos exemplos): https://docs.pytest.org/
- `logging`, HOWTO oficial: https://docs.python.org/pt-br/3/howto/logging.html
- `pdb`, o depurador do Python: https://docs.python.org/pt-br/3/library/pdb.html
- `typing`, type hints: https://docs.python.org/pt-br/3/library/typing.html
- Ruff (regras e configuração): https://docs.astral.sh/ruff/
- mypy, primeiros passos: https://mypy.readthedocs.io/en/stable/getting_started.html
- Depuração no VS Code: https://code.visualstudio.com/docs/python/debugging
- Livro gratuito *Pro Git*, em português: https://git-scm.com/book/pt-br/v2
- Leituras extras: Kent Beck, *TDD: Desenvolvimento Guiado por Testes*; Robert C. Martin, *Código Limpo* (os capítulos sobre nomes e funções já valem o livro).

Enunciados dos exercícios

Exercício 09.01 — Limites de tolerância (TDD: os testes vêm primeiro)

Arquivo: `exercicios/ex01_limites_tolerancia.py`

```
Nível: ★☆☆

Contexto: a especificação de um filme diz "espessura 20 µm ± 5%". O
laboratório
precisa saber o mínimo e o máximo aceitáveis para conferir cada bobina.

Tarefa: implemente calcular_limites(nominal, tolerancia_pct), que devolve
a tupla
(minimo, maximo) de nominal ± tolerancia_pct %.
  - nominal deve ser maior que zero; senão, levante ValueError.
  - tolerancia_pct não pode ser negativa; senão, levante ValueError com
uma
    mensagem que contenha a palavra "tolerância".
  - tolerância zero é válida: mínimo e máximo ficam iguais ao nominal.

Faça no ciclo do TDD (seção 13 da aula):
  1. VERMELHO: rode os testes e veja todos falharem. Leia o arquivo
```

```
     testes/test_m09_ex01_limites_tolerancia.py: ele é a especificação.
  2. VERDE: escreva o código mais simples que faça os testes passarem.
  3. REFATORAR: melhore nomes e organização, rodando os testes de novo.

Exemplos:
    >>> calcular_limites(20.0, 5.0)
    (19.0, 21.0)
    >>> calcular_limites(25.0, 10.0)
    (22.5, 27.5)

Dica: a margem é nominal * tolerancia_pct / 100.

Teste com:  python curso.py testar 09 ex01
```

Exercício 09.02 — Conserte o bug: número no formato brasileiro

Arquivo: `exercicios/ex02_conserte_numero_br.py`

```
Nível: ★☆☆

Contexto: a planilha do laboratório traz pesos e medidas como texto no
formato
brasileiro ("1.250,50"). Um colega escreveu a função abaixo para
converter esse
texto em float, mas os laudos estão saindo com números absurdos.

Tarefa: este exercício NÃO começa com NotImplementedError. A função já
está
escrita e tem 1 bug. Encontre e corrija.
  1. Antes de rodar, LEIA o código e preveja: o que
converter_numero_br("12,5")
     devolve hoje? Anote seu palpite.
  2. Rode os testes e leia a primeira falha: compare o esperado com o
obtido.
  3. Teste sua hipótese no REPL (python) com um caso pequeno.
  4. Corrija com a menor mudança possível e rode os testes de novo.

Regras da função (o comportamento CORRETO):
  - Ponto é separador de milhar; vírgula é separador decimal.
  - Espaços em volta são ignorados.
  - Texto que não é um número válido levanta ValueError (o próprio
float()
    já faz isso, desde que a conversão esteja certa).

Exemplos (comportamento correto):
    >>> converter_numero_br("1.250,50")
    1250.5
    >>> converter_numero_br(" 0,35 ")
    0.35
```

```
Dica: a ordem das operações importa. Escreva no papel o texto depois de
cada
replace, para "12,5".

Teste com:  python curso.py testar 09 ex02
```

Exercício 09.03 — Escreva os testes: média das medições

Arquivo: `exercicios/ex03_testes_media.py`

```
Nível: ★☆☆

Contexto: até aqui você escreveu funções e o curso escreveu os testes.
Agora o
papel se inverte: a função calcular_media já existe, e VOCÊ escreve os
testes
que garantem que ela está certa. É assim no trabalho: quem testa bem pega
o bug
antes do cliente.

Tarefa: complete testar_calcular_media(calcular_media). Ela recebe como
parâmetro a função a testar (funções são valores, lembra do módulo 04?).
Escreva
asserts que confiram as regras abaixo. Se a função recebida estiver
errada,
algum assert deve falhar. Se estiver certa, nada pode falhar.

Regras de calcular_media(valores) — a especificação:
  - recebe uma lista de números e devolve a média aritmética (float);
  - funciona com valores negativos (ex.: desvios em relação ao nominal);
  - lista vazia levanta ValueError.

Como o curso corrige: chama a sua função com uma versão CORRETA de
calcular_media (nada pode falhar) e com 3 versões COM BUG escondidas nos
testes
do curso (cada uma precisa ser reprovada por algum assert seu).

Exemplos de asserts que você pode usar:
    assert calcular_media([20.0, 21.0]) == pytest.approx(20.5)
    with pytest.raises(ValueError):
        calcular_media([])

Dica: pense nos erros que alguém cometeria ao escrever uma média. Uma
média
"redonda" como [2, 4] -> 3 esconde vários bugs. Para usar pytest.approx e
pytest.raises, escreva "import pytest" no topo deste arquivo.
```

```
Teste com:  python curso.py testar 09 ex03
```

Exercício 09.04 — Operador do laudo (variável de ambiente)

Arquivo: `exercicios/ex04_operador_do_laudo.py`

```
Nível: ★☆☆

Contexto: cada computador do laboratório tem uma variável de ambiente
LAB_OPERADOR com o nome de quem está usando a máquina. O laudo precisa
trazer
esse nome. Os testes deste exercício usam monkeypatch para criar e apagar
a
variável sem mexer no seu computador de verdade (seção 11 da aula).

Tarefa: implemente as duas funções.
  operador_atual():
    - lê a variável de ambiente LAB_OPERADOR (use os.environ.get);
    - remove espaços em volta;
    - se a variável não existir ou estiver vazia (só espaços), devolve
      "não informado".
  assinatura_laudo(op):
    - devolve o texto "OP <op> | operador: <nome>", usando
operador_atual().

Exemplos (com LAB_OPERADOR="  Maria Souza "):
    >>> operador_atual()
    'Maria Souza'
    >>> assinatura_laudo("12345")
    'OP 12345 | operador: Maria Souza'

Exemplo (sem a variável):
    >>> assinatura_laudo("777")
    'OP 777 | operador: não informado'

Dica: os.environ funciona como um dicionário. Não esqueça do "import os"
no
topo do arquivo. Leia os testes para ver como o monkeypatch prepara o
ambiente.

Teste com:  python curso.py testar 09 ex04
```

Exercício 09.05 — Log da inspeção

Arquivo: `exercicios/ex05_log_de_inspecao.py`

```
Nível: ★☆☆

Contexto: o robô que confere as bobinas roda sozinho de madrugada.
Ninguém vê
o terminal, então print() não serve: cada conferência precisa ir para o
log,
com o nível certo, para quem investigar um problema no dia seguinte.

Tarefa: implemente registrar_resultado(logger, op, valor, minimo,
maximo).
  - Se minimo <= valor <= maximo (bordas incluídas): registre no nível
INFO a
    mensagem "OP <op>: <valor> dentro de [<minimo>, <maximo>]" e devolva
True.
  - Senão: registre no nível WARNING a mensagem
    "OP <op>: <valor> FORA de [<minimo>, <maximo>]" e devolva False.
  - Os três números aparecem com 2 casas decimais (formato :.2f).
  - Use o logger recebido no parâmetro (não use print nem logging.info
direto).

Exemplos:
    >>> logger = logging.getLogger("lab")
    >>> registrar_resultado(logger, "123", 25.4, 24.0, 26.0)
    True
    (no log, nível INFO: "OP 123: 25.40 dentro de [24.00, 26.00]")
    >>> registrar_resultado(logger, "124", 27.1, 24.0, 26.0)
    False
    (no log, nível WARNING: "OP 124: 27.10 FORA de [24.00, 26.00]")

Dica: logger.info(...) e logger.warning(...). Os testes usam a fixture
caplog,
que captura os registros de log (seção 17 da aula).

Teste com:  python curso.py testar 09 ex05
```

Exercício 09.06 — Conserte o bug: peso total por NF

Arquivo: `exercicios/ex06_conserte_pesos_por_nf.py`

```
Nível: ★★☆

Contexto: a expedição manda um e-mail por carregamento, com a NF e o
peso. Uma
mesma NF pode sair em vários carregamentos (entrega parcial). O relatório
precisa do peso TOTAL de cada NF, mas os totais estão vindo menores do
que o
real e algumas NFs aparecem duplicadas.
```

```
Tarefa: a função abaixo já está escrita e tem 2 bugs. Encontre e corrija.
  1. Leia o código e preveja: com os pedidos
       [{"nf": "1001", "peso": 250.0}, {"nf": "1001", "peso": 120.5}]
     o que a função devolve hoje? Anote seu palpite antes de rodar.
  2. Rode os testes. Corrija um bug de cada vez e rode de novo.

Regras (o comportamento CORRETO):
  - Recebe uma lista de dicionários com as chaves "nf" (texto) e "peso"
(kg).
  - Devolve um dicionário {nf: peso_total}.
  - O número da NF pode vir com espaços em volta (" 1001 "): é a mesma NF
que
    "1001", e a chave do resultado fica sem espaços.

Exemplo (comportamento correto):
    >>> somar_pesos_por_nf([
    ...     {"nf": "1001", "peso": 250.0},
    ...     {"nf": "1002", "peso": 80.0},
    ...     {"nf": " 1001", "peso": 120.5},
    ... ])
    {'1001': 370.5, '1002': 80.0}

Dica: para cada bug, escreva no papel o dicionário totais depois de cada
volta do for. Ou use breakpoint() dentro do laço e o comando "p totais".

Teste com:  python curso.py testar 09 ex06
```

Exercício 09.07 — Escreva os testes: classificar uma medição

Arquivo: `exercicios/ex07_testes_classificar.py`

```
Nível: ★★☆

Contexto: a função classificar decide se uma medição está ABAIXO, OK ou
ACIMA
da tolerância. Ela vai para o validador de certificados, então um erro
aqui
libera bobina reprovada para o cliente. Sua missão é escrever testes tão
bons
que nenhuma versão com bug passe por eles.

Tarefa: complete testar_classificar(classificar), escrevendo asserts que
confiram TODAS as regras abaixo.

Regras de classificar(valor, minimo, maximo) — a especificação:
  - devolve "ABAIXO" se valor < minimo;
  - devolve "ACIMA" se valor > maximo;
  - devolve "OK" se minimo <= valor <= maximo (as bordas estão DENTRO);
  - levanta ValueError se minimo > maximo (tolerância cadastrada errada).
```

```
Como o curso corrige: chama sua função com a versão CORRETA (nada pode
falhar)
e com 4 versões COM BUG (cada uma precisa ser reprovada por algum assert
seu).

Exemplo de assert:
    assert classificar(20.0, 19.0, 21.0) == "OK"

Dica: bugs adoram morar nas BORDAS e nos casos de erro. Para cada regra,
pense
num valor que a confirma e num valor "no limite". Se quiser organizar
melhor,
crie uma lista de casos (valor, minimo, maximo, esperado) e percorra com
for.
Para pytest.raises, escreva "import pytest" no topo.

Teste com:  python curso.py testar 09 ex07
```

Exercício 09.08 — Ler o arquivo de medições (TDD com tmp_path)

Arquivo: `exercicios/ex08_ler_medicoes.py`

```
Nível: ★★☆

Contexto: o micrômetro do laboratório exporta um arquivo de texto com uma
medição por linha. O técnico às vezes deixa linhas em branco e
comentários
(linhas que começam com #). Quando uma linha está ilegível, o programa
precisa
dizer QUAL linha é, para alguém conferir no equipamento.

Tarefa: implemente ler_medicoes(caminho), que lê o arquivo (UTF-8) e
devolve a
lista de valores (float), na ordem.
  - Aceita vírgula ou ponto decimal: "25,4" e "25.4" viram 25.4.
  - Remove espaços e tabulações em volta de cada linha.
  - Ignora linhas vazias (ou só com espaços) e linhas que começam com
"#".
  - Linha que não é número: levante ValueError com a mensagem
        "linha <n>: valor inválido '<texto>'"
    em que <n> é o número da linha no arquivo, começando em 1 (contando
    também as linhas vazias e os comentários) e <texto> é a linha sem os
    espaços em volta.
  - Arquivo inexistente: deixe o FileNotFoundError acontecer (não trate).

Faça com TDD: rode só este teste parando na primeira falha, faça passar,
rode de novo (seção 13 da aula). Na pasta curso-python:
    python -m pytest 09-testes-e-
```

```
qualidade/testes/test_m09_ex08_ler_medicoes.py -x

Exemplo (arquivo com as linhas "# OP 123", "25,4", "", "25.1"):
    >>> ler_medicoes(Path("medicoes.txt"))
    [25.4, 25.1]

Dica: enumerate(linhas, start=1) dá o número de cada linha. Para gerar o
ValueError com a sua mensagem, use try/except ValueError ao redor do
float()
e levante um novo ValueError (módulo 06).

Teste com:  python curso.py testar 09 ex08
```

Exercício 09.09 — Relatório no terminal (testado com capsys)

Arquivo: `exercicios/ex09_relatorio_no_terminal.py`

```
Nível: ★★☆

Contexto: no fim da conferência, o programa imprime um resumo para o
técnico
conferir na tela antes de emitir os certificados. Funções que só imprimem
também podem (e devem) ser testadas: a fixture capsys captura o que foi
impresso (seção 10 da aula).

Tarefa: implemente imprimir_relatorio(resultados). Cada resultado é uma
tupla
(op, valor, dentro) em que dentro é True (OK) ou False (FORA).
  - Lista vazia: imprima apenas a linha "Nenhuma amostra."
  - Senão, imprima:
      1. o cabeçalho "OP | VALOR | SITUAÇÃO";
      2. uma linha por resultado: "<op> | <valor> | OK" ou "... | FORA",
         com o valor no formato brasileiro, 2 casas decimais e ponto de
milhar
         (25.4 -> "25,40"; 1250.5 -> "1.250,50");
      3. a linha final "Total: <n> amostra(s), <k> fora".
  - A função não devolve nada (só imprime).

Exemplo:
    >>> imprimir_relatorio([("12345", 25.4, True), ("12346", 1250.5,
False)])
    OP | VALOR | SITUAÇÃO
    12345 | 25,40 | OK
    12346 | 1.250,50 | FORA
    Total: 2 amostra(s), 1 fora

Dica: f"{valor:,.2f}" dá "1,250.50" (padrão americano). Troque os
separadores
como no módulo 05, usando um caractere temporário. Uma função auxiliar
```

```
formatar_br(valor) deixa o código mais limpo.

Teste com:  python curso.py testar 09 ex09
```

Exercício 09.10 — Conserte o bug: códigos das bobinas

Arquivo: `exercicios/ex10_conserte_codigos_bobinas.py`

```
Nível: ★★☆

Contexto: cada bobina de uma OP recebe uma etiqueta "<op>-<número>", com
o
número em 3 dígitos começando em 001. A OP 12345 com 3 bobinas gera
12345-001, 12345-002 e 12345-003. A produção reclamou: as etiquetas
começam
em 000 e, às vezes, saem com espaços no meio.

Tarefa: a função abaixo tem 3 bugs. Encontre e corrija.
  1. Leia o código e preveja: o que gerar_codigos_bobinas("12345", 3)
devolve
     hoje? E gerar_codigos_bobinas("12345", 0)? Anote antes de rodar.
  2. Rode os testes e corrija um bug por vez.

Regras (o comportamento CORRETO):
  - Devolve uma lista com `quantidade` códigos, numerados de 1 a
quantidade,
    sempre com 3 dígitos (001, 002, ..., 120).
  - A OP pode vir com espaços em volta (" 4512 "): use a OP sem espaços.
  - quantidade menor que 1 levanta ValueError (não existe OP com 0
bobinas).

Exemplos (comportamento correto):
    >>> gerar_codigos_bobinas("12345", 3)
    ['12345-001', '12345-002', '12345-003']
    >>> gerar_codigos_bobinas(" 4512 ", 1)
    ['4512-001']

Dica: o clássico "erro por um" (off-by-one) mora no range. Teste no REPL:
list(range(3)) e list(range(1, 4)).

Teste com:  python curso.py testar 09 ex10
```

Exercício 09.11 — Revisão: resumo por linha de produção (conserte os bugs)

Arquivo: `exercicios/ex11_revisao_resumo_por_linha.py`

Nível: ★★★

Revisão: mistura dicionários acumuladores (módulo 03), texto e números no formato brasileiro (módulo 05) e exceções (módulo 06) com depuração guiada
por testes (este módulo).

Contexto: a planilha de valores tem registros "linha;valor", como "A;25,4". O
gerente quer, para cada linha de produção (A a D), quantas medições foram feitas, quantas ficaram fora da tolerância e o percentual fora. O código abaixo
foi escrito às pressas e tem 4 bugs.

Tarefa: leia o código com calma, preveja o que ele faz e corrija os 4 bugs.
Sugestão de método (é o que um profissional faz):
  1. Rode os testes. Escolha UMA falha e leia o esperado x obtido.
  2. Formule uma hipótese ("acho que o total está sendo zerado porque...").
  3. Confirme no REPL ou com breakpoint() antes de mudar o código.
  4. Corrija, rode de novo, e só então passe para o próximo bug.

Regras (o comportamento CORRETO):
  - Cada registro é "linha;valor". A linha pode vir em minúsculas ou com espaços (" a "): normalize para maiúsculas sem espaços. O valor usa vírgula decimal ("25,4").
  - A tolerância de cada linha vem em tolerancias[linha] = (minimo, maximo).
    Um valor está FORA se for menor que o mínimo ou maior que o máximo (as bordas estão DENTRO).
  - O resultado é {linha: {"total": n, "fora": k, "percentual_fora": p}}, com
    p = k / n * 100 arredondado para 1 casa decimal.
  - Linha sem tolerância cadastrada levanta ValueError com a mensagem "linha <linha> sem tolerância cadastrada".

Exemplo (comportamento correto):
```
    >>> resumo = resumir_por_linha(
    ...     ["A;25,0", "A;27,0", "B;19,5", "A;25,5"],
    ...     {"A": (24.0, 26.0), "B": (18.0, 21.0)},
    ... )
    >>> resumo["A"]
    {'total': 3, 'fora': 1, 'percentual_fora': 33.3}
    >>> resumo["B"]
    {'total': 1, 'fora': 0, 'percentual_fora': 0.0}
```

Dica: nem todo bug aparece na primeira rodada; alguns só ficam visíveis depois

```
que você corrige outro. É normal: corrija, rode, leia de novo.

Teste com:  python curso.py testar 09 ex11
```

Exercício 09.12 — Revisão: escreva os testes de dividir_em_lotes

Arquivo: `exercicios/ex12_revisao_testes_lotes.py`

```
Nível: ★★★

Revisão: mistura listas, fatiamento e mutabilidade (módulo 03), funções
como
valores (módulo 04) e exceções (módulo 06) com escrita de testes (este
módulo).

Contexto: a expedição monta paletes com no máximo N bobinas. A função
dividir_em_lotes recebe a lista de bobinas e o tamanho do palete e
devolve os
lotes. Ela vai rodar todos os dias no robô da expedição, então seus
testes
precisam ser à prova de bala.

Tarefa: complete testar_dividir_em_lotes(dividir_em_lotes) com asserts
que
confiram TODAS as regras abaixo.

Regras de dividir_em_lotes(itens, tamanho) — a especificação:
  - devolve uma lista de listas, na ordem original, cada uma com
`tamanho`
    itens; o último lote pode ter menos (o que sobrou);
  - nunca devolve lote vazio; lista de itens vazia devolve [];
  - tamanho menor que 1 levanta ValueError;
  - NÃO modifica a lista recebida (quem chamou continua com a lista
intacta).

Exemplos:
    >>> dividir_em_lotes(["B1", "B2", "B3", "B4", "B5"], 2)
    [['B1', 'B2'], ['B3', 'B4'], ['B5']]
    >>> dividir_em_lotes([1, 2, 3, 4], 2)
    [[1, 2], [3, 4]]
    >>> dividir_em_lotes([], 3)
    []

Como o curso corrige: chama sua função com a versão CORRETA (nada pode
falhar)
e com 4 versões COM BUG (cada uma precisa ser reprovada por algum assert
seu).

Dica: liste as regras e, para cada uma, escreva pelo menos um caso que a
```

```
quebraria se estivesse errada. A última regra é a mais esquecida: guarde
uma
cópia da lista ANTES de chamar a função (lista.copy() ou list(lista)) e
compare DEPOIS. Para pytest.raises, escreva "import pytest" no topo.

Teste com:  python curso.py testar 09 ex12
```

Desafio do módulo 09 — Do script legado ao código profissional

Contexto

Todo laboratório tem aquele script que "funciona, mas ninguém mexe". O nosso é o `legado_conferencia.py`: ele lê a planilha de medições (`dados/medicoes_exemplo.csv`), confere cada valor com os limites da linha de produção e imprime um relatório.

Funciona. Mas tem nomes de uma letra, índices mágicos (`x[3]`), uma função gigante, avisos com `print` e nenhum teste. Agora o laboratório pediu uma regra nova:

> Uma medição FORA da tolerância que tenha, na coluna `observacao`, a liberação técnica do mesmo ensaio (por exemplo, `COF: liberado`) deve aparecer como **LIBERADO**, e o resumo deve contar as liberadas.

Mexer nesse script sem testes é pedir para quebrar o que funciona. Sua missão é fazer o que um profissional faria: proteger, refatorar, melhorar e só então acrescentar a regra nova.

Requisitos obrigatórios

1. **Testes de caracterização primeiro.** Antes de mudar qualquer linha, escreva testes que congelam o comportamento atual (rode o legado, guarde a saída e compare).
2. **Refatoração em funções pequenas**, com nomes claros e type hints: ler o CSV, converter número brasileiro, classificar uma medição, resumir por linha, montar o relatório. Regras (limites) separadas da lógica.
3. **`logging` no lugar dos avisos com `print`.** Linha inválida vira `WARNING` no log. Com a opção `--log arquivo.log`, o log também vai para um arquivo.
4. **Regra nova com TDD:** a liberação pela observação. Escreva o teste, veja falhar, implemente, refatore. Cuidado: a liberação de **outro** ensaio não vale (uma observação `Selagem: liberado` não libera o COF).

5. **Pelo menos 12 testes**, usando no total: `parametrize`, uma fixture sua, `tmp_path`, `capsys` ou `caplog`, `pytest.raises` e `pytest.approx`.
6. **`ruff check` e `ruff format --check` sem problemas.**
7. **Git de verdade:** crie um repositório seu (ou uma pasta no seu repositório de estudos), trabalhe numa branch `refatora-conferencia`, faça **pelo menos 4 commits pequenos** com boas mensagens e abra um **pull request** no GitHub com descrição (o que mudou, por quê, como testar).

Extras (opcionais)

- Cobertura acima de 90% com `pytest-cov` (`python -m pip install pytest-cov`).
- `mypy` sem erros no seu código.
- Argumento `--so-fora` que imprime só as medições FORA.
- Um workflow do GitHub Actions que roda `ruff` e `pytest` a cada push (seção "No mundo real" da aula).

Exemplo de execução

Legado (antes), na pasta `09-testes-e-qualidade`:

```
> python desafio/legado_conferencia.py
AVISO: linha 9 ignorada, valor: abc
CONFERÊNCIA DE MEDIÇÕES
OP 12345 | linha A | espessura | 20,30 | OK
OP 12345 | linha A | gramatura | 18,60 | OK
OP 12345 | linha A | cof | 0,38 | FORA
OP 12346 | linha A | espessura | 21,40 | FORA
OP 12347 | linha B | espessura | 24,80 | OK
OP 12347 | linha B | gramatura | 22,10 | OK
OP 12347 | linha B | cof | 0,22 | OK
OP 12349 | linha C | espessura | 30,20 | OK
OP 12349 | linha C | cof | 0,41 | FORA
OP 12350 | linha D | espessura | 15,00 | OK
OP 12350 | linha D | gramatura | 14,20 | OK
OP 12350 | linha D | tracao | 35,00 | SEM REGRA
RESUMO POR LINHA
A: 4 medições, 2 fora (50,0%)
B: 3 medições, 0 fora (0,0%)
C: 2 medições, 1 fora (50,0%)
D: 2 medições, 0 fora (0,0%)
```

Versão nova (depois). O aviso agora vem do logging (a data e a hora serão outras):

```
> python desafio/solucao_referencia.py
2026-10-29 09:52:00,648 WARNING conferencia: linha 9 ignorada, valor: abc
```

```
CONFERÊNCIA DE MEDIÇÕES
OP 12345 | linha A | espessura | 20,30 | OK
OP 12345 | linha A | gramatura | 18,60 | OK
OP 12345 | linha A | cof | 0,38 | LIBERADO
OP 12346 | linha A | espessura | 21,40 | FORA
OP 12347 | linha B | espessura | 24,80 | OK
OP 12347 | linha B | gramatura | 22,10 | OK
OP 12347 | linha B | cof | 0,22 | OK
OP 12349 | linha C | espessura | 30,20 | OK
OP 12349 | linha C | cof | 0,41 | FORA
OP 12350 | linha D | espessura | 15,00 | OK
OP 12350 | linha D | gramatura | 14,20 | OK
OP 12350 | linha D | tracao | 35,00 | SEM REGRA
RESUMO POR LINHA
A: 4 medições, 1 fora (25,0%), 1 liberada(s)
B: 3 medições, 0 fora (0,0%), 0 liberada(s)
C: 2 medições, 1 fora (50,0%), 0 liberada(s)
D: 2 medições, 0 fora (0,0%), 0 liberada(s)
```

Repare: o COF da OP 12345 virou LIBERADO (a observação diz `COF: liberado`), mas o COF da OP 12349 continua FORA (a observação libera a selagem, não o COF).

> 💡 **Dica:** quando a regra nova entrar, um dos seus testes de caracterização vai falhar (a linha do COF da OP 12345 muda de FORA para LIBERADO). Isso é esperado: o requisito mudou. Atualize esse teste **no mesmo commit** da regra nova e explique na mensagem do commit.

Critérios de aceite

- [ ] Existe um teste de caracterização que roda o código antigo e compara a saída.
- [ ] Nenhuma função tem mais de ~20 linhas; nenhum nome de uma letra (fora `i` em laço).
- [ ] Linha com valor inválido gera `WARNING` no log e não derruba o programa.
- [ ] `--log conferencia.log` grava o log num arquivo em UTF-8.
- [ ] Medição FORA com liberação do mesmo ensaio aparece como LIBERADO.
- [ ] Liberação de outro ensaio não libera.
- [ ] O resumo mostra total, fora, percentual e liberadas por linha.
- [ ] `python -m pytest` passa com pelo menos 12 testes.
- [ ] `ruff check .` e `ruff format --check .` sem problemas.
- [ ] Branch, 4+ commits com mensagens no imperativo e pull request aberto.

Dicas para começar

1. Copie `legado_conferencia.py` e a pasta `dados/` para o seu repositório. Faça o primeiro commit **antes** de mudar qualquer coisa: "Adiciona script legado de conferência".
2. Teste de caracterização: rode o legado com `subprocess.run([...], capture_output=True, text=True)` (ou chame a função `f` e use `capsys`) e compare com a saída que você guardou num arquivo `saida_esperada.txt`.
3. Refatore em passos pequenos: extraia UMA função, rode os testes, faça commit. Repita.
4. Faça a função que monta o relatório **devolver texto** em vez de imprimir: fica muito mais fácil de testar.
5. Para a regra nova, comece pelo teste mais simples ("COF: liberado" libera o COF).

Uma solução de referência está em `solucao_referencia.py`, com 22 testes em `test_solucao_referencia.py`. Para rodar os testes (na pasta `curso-python`):

```
python -m pytest 09-testes-e-qualidade/desafio -v
```

Tente fazer sozinho antes de abrir. Depois compare: não existe uma única resposta certa.

Quiz — Módulo 09: Testes e qualidade de código

Responda no papel (ou num arquivo) antes de olhar o gabarito no final.

---

1. Você roda só `python -m pytest`, sem indicar arquivo. Com as regras padrão de descoberta, qual destas funções **não** é executada?

a) `test_media` no arquivo `test_calculos.py` b) `testar_media` no arquivo `test_calculos.py` c) `test_media` no arquivo `calculos.py` d) `test_limites` no arquivo `tests/test_tolerancia.py`

---

2. O que este código imprime?

```
import pytest

print(0.1 + 0.2 == 0.3, 0.1 + 0.2 == pytest.approx(0.3))
```

a) `True True` b) `False False` c) `False True` d) `True False`

---

3. Onde está o problema deste teste?

```
def test_media_de_lista_vazia():
    with pytest.raises(Exception):
        media([])
```

---

4. Qual é a ordem correta do ciclo do TDD?

a) Escrever o código, escrever o teste, refatorar. b) Escrever um teste que falha, escrever o código mínimo para ele passar, refatorar. c) Refatorar, escrever o teste, escrever o código. d) Escrever todos os testes do sistema, depois todo o código.

---

5. Rodando `python -m pytest -s` (o `-s` mostra os prints), em que ordem aparecem as palavras "criando", "testando" e "limpando"?

```
import pytest


@pytest.fixture
def bobina():
    print("criando")
    yield {"peso": 250.0}
    print("limpando")


def test_peso(bobina):
    print("testando")
    assert bobina["peso"] == 250.0
```

---

6. Para que serve a fixture `monkeypatch`?

a) Para corrigir bugs automaticamente. b) Para trocar temporariamente uma função, atributo ou variável de ambiente durante um teste, desfazendo a troca no final. c) Para medir a cobertura dos testes. d) Para rodar os testes em paralelo.

---

7. Depois de executar `print("OK")` e `print("FORA")` dentro de um teste, o que `capsys.readouterr().out` devolve?

---

8. O que este programa mostra no terminal?

```
import logging

logging.basicConfig(level=logging.WARNING, format="%(levelname)s: %
(message)s")
logging.debug("a")
logging.info("b")
logging.warning("c")
logging.error("d")
```

---

9. O `ruff check` avisou `F401 'os' imported but unused`. O que isso significa e qual é a diferença entre `ruff check` e `ruff format`?

---

10. Qual é a melhor mensagem de commit?

a) `ajustes` b) `Corrige borda mínima na classificação de tolerância` c) `commit final agora vai` d) `mudei umas coisas no arquivo de tolerância e também no relatório e no log`

---

11. Seu projeto tem 100% de cobertura de testes. Isso garante que não há bugs?

---

12. Uma função tem a anotação `def somar(a: int, b: int) -> int`. O que acontece ao rodar `somar("10", "20")` com `python`?

a) `TypeError` antes de executar. b) Devolve `30`. c) Devolve `"1020"`: o Python não confere as anotações ao rodar. d) O programa não abre.

---

13. Este código passa ou levanta erro? O que ele imprime?

```
import pytest

with pytest.raises(ValueError, match="linha 1") as excinfo:
    raise ValueError("linha 13: valor inválido")
print("passou:", excinfo.value)
```

---

14. Você está no depurador (pdb), parado dentro de um laço. Qual comando mostra o valor da variável `total`, e qual a diferença entre `n` e `s`?

Gabarito

1. **c.** O pytest só abre arquivos `test_*.py` ou `*_test.py`; `calculos.py` nem é lido. Pegadinha da (b): para **funções**, a regra é o prefixo `test`, sem exigir o sublinhado, então `testar_media` **é** executada. Por isso, nos exercícios "escreva os testes", a função `testar_...` fica num arquivo `exNN_...py`, que o pytest não coleta.
2. **c.** `0.1 + 0.2` vale `0.30000000000000004`, então `==` dá `False`. O `pytest.approx` compara com uma tolerância relativa (1 milionésimo), então dá `True`.
3. `Exception` é genérico demais: o teste passa com **qualquer** erro, até com um `TypeError` ou `NameError` causado por um bug. Use o tipo exato: `pytest.raises(ValueError)` (e, se quiser, `match=` para conferir a mensagem).
4. **b.** Vermelho (teste falhando), verde (código mínimo), refatorar (melhorar com os testes passando). Um teste por vez.
5. criando, testando, limpando. O código antes do `yield` roda antes do teste; o que está depois do `yield` roda no final, mesmo que o teste falhe.
6. **b.** Tudo o que o `monkeypatch` troca é desfeito sozinho no fim do teste, então um teste não "suja" o outro.
7. `"OK\nFORA\n"`: tudo o que foi impresso, com as quebras de linha que o `print` acrescenta.
8. Só `WARNING: c` e `ERROR: d`. Com `level=logging.WARNING`, as mensagens DEBUG e INFO ficam de fora (os níveis, do menor para o maior: DEBUG, INFO, WARNING, ERROR, CRITICAL).
9. F401 é "import não usado": o módulo `os` foi importado e ninguém usa. `ruff check` procura **problemas** (erros prováveis, más práticas, imports não usados); `ruff format` mexe só na **aparência** (espaços, quebras de linha, aspas), sem mudar o comportamento.
10. **b.** Verbo no imperativo, curta, diz o quê e onde. (a) e (c) não dizem nada; (d) mistura vários assuntos, o que indica que deveriam ser commits separados.
11. Não. Cobertura diz quais linhas **rodaram** durante os testes, não se os asserts conferem as coisas certas. Um teste sem assert cobre linhas e não testa nada. Cobertura baixa mostra o que falta testar; cobertura alta não prova ausência de bugs.
12. **c.** Type hints são documentação para pessoas e ferramentas (como o mypy). O Python as ignora ao executar: `"10" + "20"` concatena textos.
13. Passa e imprime `passou: linha 13: valor inválido`. O `match` usa `re.search`, que procura o padrão em **qualquer parte** da mensagem, e "linha 1" está dentro de "linha 13". Para exigir mais precisão, use algo como `match=r"^linha 1:"`.
14. `p total` mostra o valor. `n` (next) executa a linha atual inteira e para na próxima; `s` (step) **entra** na função chamada na linha atual.

Módulo 10 — Análise de dados com pandas, Excel e gráficos

> **Tempo estimado:** 12 a 16 horas · **Pré-requisitos:** módulos 00 a 09 · **Nível:** intermediário

O que você vai aprender

- Criar `Series` e `DataFrame` e explicar a diferença entre os dois.
- Ler CSV do Excel brasileiro (`sep`, `decimal`, `thousands`, `encoding`) sem perder o zero à esquerda da OP, e juntar todas as abas de uma planilha numa tabela só.
- Inspecionar uma base nova e selecionar com `loc`, `iloc` e filtros (`&`, `|`, `~`).
- Criar colunas sem laço `for` e alterar valores com `.loc`, sem atribuição encadeada.
- Limpar texto, números brasileiros, ausentes e duplicatas; converter datas `dd/mm/aaaa`.
- Resumir com `groupby` / `agg`, cruzar com `merge` e montar tabelas dinâmicas.
- Exportar CSV e Excel formatado (cabeçalho, largura, reprovados em vermelho).
- Desenhar barras, linhas, histograma, dispersão e um gráfico de controle (média ± 3σ).
- Responder perguntas de carreira com 133 mil salários reais da área de dados.

Por que isso importa

Você já faz análise de dados todo dia: no Excel. O pandas faz o mesmo trabalho, só que **escrito**. Repetir fica de graça (o relatório do mês que vem é o mesmo script), a escala deixa de ser problema (1 milhão de linhas não assusta) e cada passo da limpeza fica registrado no código, o que vale ouro num laboratório com auditoria.

No mercado, pandas e SQL (módulo 11) aparecem em praticamente toda vaga de analista de dados, engenheiro de dados e automação. E no seu projeto final, é o pandas (ou o openpyxl) que vai ler a planilha de valores com as abas das linhas A, B, C e D.

Lembra do seu script antigo, que lia o `salaries.csv` e contava diretores com um `for`? No fim deste módulo você vai reescrevê-lo em uma linha e responder perguntas bem melhores com o mesmo arquivo.

1. Preparando o terreno

O pandas já está no `requirements.txt` do curso, junto com o `openpyxl` (que o pandas usa para ler e gravar `.xlsx`) e o `matplotlib` (gráficos). Com o ambiente virtual ativado, na pasta `curso-python`:

```
python -m pip install -r requirements.txt
python -c "import pandas, openpyxl, matplotlib;
print(pandas.__version__)"
```

A convenção universal é importar assim:

```
import pandas as pd
```

Versões. Este módulo funciona no pandas 2.2 e no 3.x. Diferenças que você vai ver: no pandas 3, colunas de texto têm o tipo `str` (no 2.2, `object`), e o *copy-on-write* está sempre ligado (a seção 7 explica o que isso muda na prática).

Como rodar os exemplos. Todos ficam em `exemplos/` e acham a pasta `dados/` sozinhos, não importa de onde você rode. Na pasta `curso-python`:

```
python 10-dados-com-pandas/exemplos/02_ler_csv_brasileiro.py
```

Os dados da aula (fictícios, mas com cara de laboratório) estão em `dados/`:

Arquivo	O que é
`medicoes.csv`	70 bobinas medidas: data, OP, linha, produto, espessura, gramatura, COF, peso
`medicoes_sujas.csv`	10 linhas digitadas à mão, com todos os defeitos clássicos
`ops_protheus.csv`	OPs exportadas do ERP, em `cp1252`
`especificacoes.csv`	nominal, tolerância e COF máximo de cada produto
`planilha_linhas.xlsx`	a "planilha de valores": uma aba por linha de produção
`salarios_amostra.csv`	600 linhas sorteadas do `salaries.csv`, para testes rápidos

O script `dados/gerar_dados.py` recria todos eles (rode se apagar algum).

> 💡 **Dica:** para explorar, os notebooks do VS Code (ou o Jupyter) são ótimos: você roda um pedaço por vez. Mas o código de produção mora em arquivos `.py`, com testes.

2. Series e DataFrame

Pense numa aba do Excel. Cada **coluna** é uma `Series`: uma sequência de valores do mesmo tipo, com um rótulo para cada linha (o **índice**). A aba inteira é um `DataFrame`: várias Series lado a lado, compartilhando o mesmo índice.

```
DataFrame
            ┌──────────── Series ────────────┐
 índice      op       linha   espessura_um   peso_kg
   0      024412       A         40.2         1250.5
   1      024412       A         39.1          980.0
   2      024413       B         35.3         1310.2
```

Do arquivo `exemplos/01_series_e_dataframe.py`:

```python
import pandas as pd

espessuras = pd.Series([50.2, 49.1, 51.8, 47.9], name="espessura_um")
print("Média:", espessuras.mean())
print(espessuras - 50)
```

```
Média: 49.75000000000001
0    0.2
1   -0.9
2    1.8
3   -2.1
Name: espessura_um, dtype: float64
```

Repare em `espessuras - 50`: a conta foi feita **em todos os valores de uma vez**, sem `for`. Isso se chama **vetorização**, e é o coração do pandas. (O `49.75000000000001` é o velho problema do float do módulo 01.)

Um DataFrame nasce de um dicionário de listas (cada chave vira uma coluna) ou de uma lista de dicionários (a "tabela" do módulo 03):

```python
df = pd.DataFrame(
    {
        "op": ["024412", "024412", "024413", "024420"],
        "linha": ["A", "A", "B", "C"],
        "espessura_um": [40.2, 39.1, 35.3, 69.8],
        "peso_kg": [1250.5, 980.0, 1310.2, 845.7],
    }
)
print(df.shape, df["peso_kg"].sum())
print(df.dtypes)
```

```
(4, 4) 4386.4
op                  str
linha               str
espessura_um    float64
peso_kg         float64
dtype: object
```

`shape` é `(linhas, colunas)`. `dtypes` mostra o tipo de cada coluna (no pandas 2.2, as colunas de texto aparecem como `object`). `df["peso_kg"]` é uma Series.

> 🔮 **Preveja antes de rodar:** o que este código imprime?

```
s = pd.Series([10, 20, 30])
print(s * 2)
print(s[s > 15])
```

▼ **Resposta**

```
0    20
1    40
2    60
dtype: int64
1    20
2    30
dtype: int64
```

`s * 2` multiplica cada valor. `s > 15` gera uma Series de `True/False`, e `s[...]` com ela dentro devolve só os `True`. Repare que o índice original (1 e 2) é mantido: o pandas não renumera sozinho.

> 🔧 **Modifique:** crie a Series com `index=["b1", "b2", "b3"]` e acesse `s["b2"]`. O índice pode ser texto, como a chave de um dicionário.

3. Lendo CSV brasileiro

O Excel em português salva CSV com **ponto e vírgula** separando colunas, **vírgula** decimal e **ponto** de milhar. O começo do nosso `medicoes.csv`:

```
data;op;linha;produto;bobina;espessura_um;gramatura_gm2;cof;peso_kg;obser
vacao
```

```
03/07/2026;024412;A;PE-40;024412-01;38,1;36,1;0,21;1.558,5;
```

O `read_csv` espera o padrão americano (vírgula separa colunas). Na leitura ingênua, `pd.read_csv(DADOS / "medicoes.csv").shape` dá `(70, 1)`: uma coluna só, com tudo grudado. A leitura correta (de `exemplos/02_ler_csv_brasileiro.py`):

```
df = pd.read_csv(
    DADOS / "medicoes.csv",
    sep=";",  # separador de colunas
    decimal=",",  # vírgula decimal: 47,7 -> 47.7
    thousands=".",  # ponto de milhar: 1.558,5 -> 1558.5
    encoding="utf-8-sig",  # UTF-8 com BOM (o "CSV UTF-8" do Excel)
    dtype={"op": str},  # OP é código, não número: preserva o zero à
esquerda
)
print(df[["op", "bobina", "espessura_um", "cof", "peso_kg"]].head(3))
```

```
       op     bobina  espessura_um   cof  peso_kg
0  024412  024412-01          38.1  0.21   1558.5
1  024412  024412-02          36.4  0.24   1256.6
2  024412  024412-03          39.6  0.27    782.2
```

Cada parâmetro resolve um problema:

Parâmetro	Sem ele...
`sep=";"`	tudo vira uma coluna só
`decimal=","`	`38,1` fica como texto
`thousands="."`	`1.558,5` fica como texto
`encoding`	acentos viram lixo ou dá `UnicodeDecodeError`
`dtype={"op": str}`	`024412` vira o número `24412`

> ⚠️ **Erro comum:** esquecer o `dtype` da OP. O pandas vê só dígitos e converte para número. Resultado do exemplo: `OP sem dtype: 24412 \| OP com dtype: 024412`. Na hora de cruzar com o ERP (que guarda `"024412"`), nada casa. Código (OP, NF, CEP, CNPJ) é **texto**, mesmo que só tenha dígitos.

Encoding. Arquivos antigos do Windows (e muitas exportações de ERP) usam `cp1252`. Ler como UTF-8 quebra no primeiro acento:

```
Erro de encoding: 'utf-8' codec can't decode byte 0xe7 in position 56:
invalid continuation byte
```

O byte `0xe7` é o `ç` de "Torrefação" em cp1252. A correção é dizer o encoding certo:

```
ops = pd.read_csv(DADOS / "ops_protheus.csv", sep=";", decimal=",",
thousands=".",
                  encoding="cp1252", dtype={"op": str})
print(ops[["op", "cliente", "quantidade_kg"]].head(2))
```

```
       op                cliente  quantidade_kg
0  024412  Torrefação Serra Ltda         4500.0
1  024413  Laticínios Serra Azul         3500.0
```

Como o arquivo foi salvo	`encoding=`
Excel: "CSV UTF-8 (delimitado por vírgulas)"	`"utf-8-sig"`
Excel: "CSV (separado por vírgulas)", ERPs antigos	`"cp1252"` (ou `"latin-1"`)
Sistemas modernos, Linux, APIs	`"utf-8"`

> 💡 **Dica:** arquivo grande? Espie antes: `nrows=5` lê só 5 linhas e `usecols=["op", "linha"]` só as colunas que você quer.

> 💡 **Dica:** depois de ler, **sempre** confira `df.dtypes`. Se `espessura_um` aparecer como `str` (ou `object` no 2.2), faltou `decimal=","`, e o `mean()` vai dar `TypeError: Cannot perform reduction 'mean' with string dtype`.

4. Lendo Excel: uma aba, várias, todas

`pd.read_excel` lê `.xlsx` usando o openpyxl por baixo. Nossa planilha de valores tem uma aba por linha de produção, e cada aba tem um **título na linha 1** e o cabeçalho só na linha 2, como muita planilha feita à mão. O parâmetro `header=1` diz "o cabeçalho está na linha de índice 1" (a segunda). De `exemplos/03_ler_excel_varias_abas.py`:

```
PLANILHA = DADOS / "planilha_linhas.xlsx"
aba_a = pd.read_excel(PLANILHA, sheet_name="Linha A", header=1, dtype=
{"OP": str})
print(aba_a.columns.tolist())
```

```
['OP', 'Bobina', 'Data', 'Espessura (µm)', 'Gramatura (g/m²)', 'COF',
'Peso (kg)', 'Observação']
```

Com `sheet_name=None`, o pandas lê **todas** as abas e devolve um **dicionário** `{nome da aba: DataFrame}`:

```
abas = pd.read_excel(PLANILHA, sheet_name=None, header=1, dtype={"OP":
str})
for nome, tabela in abas.items():
    print(f"{nome}: {len(tabela)} linhas")
```

Saída: a Linha A tem 28 linhas; a B, 15; a C, 10; e a D, 17.

Para analisar, queremos **uma** tabela, com uma coluna dizendo de que linha veio cada bobina. `assign` cria a coluna e `pd.concat` empilha as partes:

```
partes = []
for nome, tabela in abas.items():
    partes.append(tabela.assign(linha=nome.removeprefix("Linha ")))
todas = pd.concat(partes, ignore_index=True)
print("Total:", todas.shape)
```

```
Total: (70, 9)
```

`ignore_index=True` renumera 0, 1, 2... (sem ele, cada aba recomeçaria do 0). Por fim, nomes com acento, espaço e unidade são chatos de digitar. Renomeie logo: `todas.rename(columns={"OP": "op", "Espessura (µm)": "espessura_um", ...})`.

As datas já chegam como datas (`datetime64`): o Excel guarda data como data, ao contrário do CSV, que guarda texto.

> ⚠️ **Erro comum:** `header=1` vale para **todas** as abas. Se a planilha tiver uma aba "Instruções" com uma linha só, o `read_excel` falha com `ValueError: Passed header=[1], len of 1, but only 1 lines in file (sheet: Instruções)`. Solução: descubra os nomes antes com `pd.ExcelFile(caminho).sheet_names`, escolha as abas que interessam e passe a lista em `sheet_name=[...]`. É o exercício 05.

5. Inspecionando uma base nova

Antes de qualquer análise, faça o "exame de admissão" da tabela. Cinco comandos (de `exemplos/04_inspecionar_e_selecionar.py`):

```
print(df.shape)
df.info()
```

```
(70, 10)
<class 'pandas.DataFrame'>
RangeIndex: 70 entries, 0 to 69
 #   Column         Non-Null Count  Dtype
 0   data           70 non-null     str
 ...
 8   peso_kg        70 non-null     float64
 9   observacao     3 non-null      str
```

`info()` responde: quantas linhas, quais colunas, de que tipo e **quantos valores não vazios** cada uma tem (`observacao` só tem 3 preenchidos). Depois:

```
print(df[["espessura_um", "cof", "peso_kg"]].describe().round(2))
```

```
       espessura_um    cof  peso_kg
count         70.00  70.00    70.00
mean          52.92   0.27  1216.03
std           16.74   0.06   242.50
min           32.10   0.13   782.20
25%           37.82   0.22  1027.70
50%           48.55   0.27  1220.00
75%           65.18   0.31  1393.88
max           94.40   0.44  1632.50
```

E `df["linha"].value_counts()` conta as bobinas por linha: A 28, D 17, B 15, C 10.

O roteiro, para levar para a vida:

Pergunta	Comando
Qual o tamanho?	`df.shape`
Que colunas e tipos? Tem vazio?	`df.info()`
Como são os números?	`df.describe()`
Quais categorias e quantas de cada?	`df["col"].value_counts()`
Quantos valores diferentes?	`df["col"].nunique()`, `df["col"].unique()`

Olhe o `describe` com olho de laboratório: a espessura vai de 32,1 a 94,4 µm. Faz sentido? Sim, porque há produtos de 35 a 90 µm misturados. Se aparecesse 940, seria vírgula digitada errada. **Desconfiar dos dados é parte do trabalho.**

6. Selecionando linhas e colunas

Quero...	Escreva	Devolve
uma coluna	`df["cof"]`	Series
várias colunas	`df[["op", "cof"]]` (lista dentro)	DataFrame
por rótulo	`df.loc[linhas, colunas]`	depende
por posição	`df.iloc[linhas, colunas]`	depende
por condição	`df[mascara]`	DataFrame

`df.loc[0, "bobina"]` devolve `'024412-01'` (linha de rótulo 0, coluna "bobina"). `df.loc[0:2, ["bobina", "cof"]]` devolve as linhas 0 a 2 dessas duas colunas.

> 🔮 **Preveja antes de rodar:** a tabela tem 4 linhas (índice 0 a 3). Quanto vale cada `len`?

```
df = pd.DataFrame({"bobina": ["a", "b", "c", "d"], "cof": [0.21, 0.38, 0.27, 0.41]})
print(len(df.loc[0:2]), len(df.iloc[0:2]))
```

▼ **Resposta**

```
3 2
```

`loc` trabalha com **rótulos** e inclui o fim (0, 1 e 2). `iloc` trabalha com **posições** e exclui o fim, igual ao fatiamento de listas do módulo 03 (0 e 1).

Filtros booleanos. Uma comparação com uma coluna gera uma **máscara** (Series de `True/False`). Colocada dentro de `df[...]`, ela devolve só as linhas `True`:

```
mascara = df["cof"] > 0.35
print("Bobinas com COF > 0,35:", mascara.sum())
print(df[mascara][["bobina", "linha", "cof", "observacao"]])
```

```
Bobinas com COF > 0,35: 4
       bobina linha   cof     observacao
3   024413-01     B  0.37            NaN
14  024430-03     A  0.38  COF: liberado
```

```
36  024458-02     B  0.44            NaN
58  024483-04     A  0.37  COF: liberado
```

`mascara.sum()` conta os `True` (o Python trata `True` como 1). Para combinar condições, use `&` (e), `|` (ou) e `~` (não), **com parênteses em cada condição**:

```
linha_a_grossa = df[(df["linha"] == "A") & (df["espessura_um"] > 48)]
dentro = df["espessura_um"].between(57.66, 66.34)  # LAM-62: 62 µm ± 7%
print(df[(df["produto"] == "LAM-62") & ~dentro][["bobina",
"espessura_um"]])
```

```
       bobina  espessura_um
28  024447-03          67.5
43  024466-03          57.1
44  024466-04          56.9
```

Outros filtros úteis: `df["linha"].isin(["C", "D"])` ("está na lista?"), `df["observacao"].notna()` ("está preenchido?"), `df["produto"].str.startswith("PE")`. E o `loc` aceita máscara e colunas juntos: `df.loc[mascara, ["bobina", "cof"]]`.

> ⚠️ **Erro comum:** usar `and` / `or` do Python em vez de `&` / `|`: `df[df["linha"] == "A" and df["cof"] > 0.3]` dá `ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(), a.item(), a.any() or a.all().` O `and` quer **um** True ou False; a máscara tem 70.

7. Criando e alterando colunas

Coluna nova = atribuição com uma conta vetorizada (de `exemplos/05_criar_colunas.py`):

```
df["desvio_pct"] = ((df["espessura_um"] - df["nominal_um"]) /
df["nominal_um"] * 100).round(2)
df["status"] = "APROVADA"
df.loc[df["desvio_pct"].abs() > 7, "status"] = "REPROVADA"
print(df[["bobina", "desvio_pct", "status"]])
```

```
      bobina  desvio_pct     status
0  024447-01       -0.32   APROVADA
1  024447-02       -5.00   APROVADA
2  024447-03        8.87  REPROVADA
3  024447-04        0.00   APROVADA
```

O padrão **"valor padrão + exceções"** é o jeito mais legível de criar colunas condicionais: todo mundo começa `APROVADA` e o `.loc[mascara, "coluna"] = valor` troca só onde a condição vale. O exemplo 05 mostra mais quatro ferramentas: `.map(dicionario)` (traduzir códigos), `pd.cut` (faixas: leve, média, pesada), `drop(columns=[...])` e `rename(columns={...})`. `drop`, `rename` e `assign` devolvem uma tabela **nova**: guarde o resultado (`df = df.drop(columns=["x"])`).

Cópias e o copy-on-write. Esta é a armadilha mais famosa do pandas.

> 🔮 **Preveja antes de rodar:** depois deste código, o status da bobina 02 é `"OK"` ou `"FORA"`?

```
df = pd.DataFrame({"bobina": ["01", "02"], "cof": [0.25, 0.41], "status":
["OK", "OK"]})
df[df["cof"] > 0.35]["status"] = "FORA"
print(df)
```

▼ **Resposta**

Continua `"OK"`, e o pandas avisa:

```
ChainedAssignmentError: A value is being set on a copy of a DataFrame
or Series
through chained assignment.
...
  bobina   cof status
0     01  0.25     OK
1     02  0.41     OK
```

São **dois** colchetes seguidos (atribuição encadeada): o primeiro, `df[mascara]`, cria uma tabela que se comporta como cópia; o segundo grava `"FORA"` nessa cópia, que é jogada fora. No pandas 3 isso **nunca** funciona; no 2.2 o aviso se chama `SettingWithCopyWarning` e o resultado também é `"OK"`. O jeito certo é **uma operação só**, com `.loc`:

```
df.loc[df["cof"] > 0.35, "status"] = "FORA"
```

Regras práticas que funcionam nas duas versões:

1. Para alterar valores, use sempre `df.loc[mascara, "coluna"] = valor`.
2. Filtrou e vai alterar o resultado? Faça `.copy()`: `reprovadas = df[df["status"] == "REPROVADA"].copy()`. Assim fica claro que é outra tabela, e o original não muda.

3. Função que recebe um DataFrame não altera o DataFrame de quem chamou: comece com `resultado = df.copy()` (efeito colateral, módulo 04).

8. Tipos, texto, ausentes e duplicatas

Dados digitados à mão chegam assim (`dados/medicoes_sujas.csv`, lido com `dtype=str` para vermos tudo como foi digitado):

```
         data      op linha     bobina espessura_um  peso_kg operador
0  01/09/2026  024470     A  024470-01         51,2  1.120,5   Marcos
1  01/09/2026  024470     a  024470-02         49,8  1.098,0  marcos
2  01/09/2026  024470     A  024470-02         49,8  1.098,0  marcos
3  02/09/2026  024470    A   024470-03          n/d  1.205,3   Marcos
4  04/09/2026  024474     c  024474-01         71,3    845,0    Júlia
5  04/09/2026  024474     C  024474-02          NaN    910,2    JÚLIA
6  04/09/2026  024474     C  024474-03         69,9      NaN    Júlia
7         NaN  024474     C  024474-04         70,1    877,7    Júlia
8  10/09/2026  024479     b  024479-01         35,4  1.310,0      Ana
9  10/09/2026  024479     B  024479-02        34,7   1.288,4      ana
```

Antes de limpar, faça o diagnóstico: `bruto["linha"].unique().tolist()` mostra `['A', ' a', 'A ', 'c', 'C', 'b', 'B']` (sete "linhas" que na verdade são três), e `bruto.isna().sum()` conta os vazios por coluna. `NaN` ("not a number") é como o pandas marca um valor ausente.

Exemplo resolvido: limpar passo a passo (`exemplos/06_limpeza_tipos_ausentes.py`). Primeiro eu listo os problemas, um por coluna. Depois resolvo um por vez, na ordem certa.

Passo 1, texto. Os métodos de string do módulo 05 existem para colunas inteiras, via `.str`:

```
df["linha"] = df["linha"].str.strip().str.upper()  # ['A', 'C', 'B']
df["operador"] = df["operador"].str.strip().str.title()  # ['Marcos',
'Júlia', 'Ana']
```

Passo 2, números brasileiros guardados como texto. A mesma receita do módulo 05: tira o ponto de milhar, troca a vírgula por ponto, converte. `pd.to_numeric` converte a coluna inteira; com `errors="coerce"`, o que não for número (como `"n/d"`) vira `NaN` em vez de erro:

```
df["peso_kg"] = pd.to_numeric(
    df["peso_kg"].str.replace(".", "", regex=False).str.replace(",", ".",
regex=False)
```

```
)
df["espessura_um"] = pd.to_numeric(
    df["espessura_um"].str.strip().str.replace(",", ".", regex=False),
errors="coerce"
)
```

O `regex=False` diz que o `"."` é um ponto de verdade, não o "qualquer caractere" das expressões regulares.

Passo 3, duplicatas. Só agora, com o texto padronizado, as linhas 1 e 2 ficaram idênticas (antes, `" a"` e `"A"` eram diferentes):

```
print(df.duplicated().sum())  # 1: linhas inteiras iguais
print(df.duplicated(subset=["bobina"]).sum())  # 1: mesma bobina
df = df.drop_duplicates(subset=["bobina"], keep="first")
```

Passo 4, ausentes: decida caso a caso. Sem espessura não há o que avaliar: descarte. Peso vazio não impede a análise da espessura: mantenha (aqui o exemplo preenche com 0 para mostrar o `fillna`, mas na vida real é melhor avisar alguém).

```
df = df.dropna(subset=["espessura_um"])
df["peso_kg"] = df["peso_kg"].fillna(0)
df = df.reset_index(drop=True)
```

Para converter quando você tem **certeza** de que tudo converte, existe `df["col"].astype(int)` (ou `float`, `str`).

> ⚠️ **Erro comum:** `pd.Series(["1", "n/d"]).astype(int)` dá `ValueError: invalid literal for int() with base 10: 'n/d'`. Com lixo possível, use `pd.to_numeric(..., errors="coerce")` e depois **conte** quantos viraram `NaN`: lixo demais é sinal de que o problema é outro (separador, encoding).

> 💡 **Dica:** `NaN` não entra nas contas. `pd.Series([1.0, None, 3.0]).mean()` dá `2.0` (média de 1 e 3), não 1.33. Às vezes é o que você quer; às vezes esconde um problema.

Inteiro com vazio vira `float` (2024 vira `2024.0`), porque `NaN` é float. `df["ano"].astype("Int64")` (I maiúsculo) usa o inteiro que aceita ausentes (`<NA>`).

9. Datas

No CSV, data é texto. Para fazer conta com data, converta com `pd.to_datetime` e **diga o formato** (`exemplos/07_datas.py`):

```
df["data"] = pd.to_datetime(df["data"], format="%d/%m/%Y")
print("Depois:", df["data"].dtype, "| exemplo:", df.loc[0, "data"])
```

```
Depois: datetime64[us] | exemplo: 2026-07-03 00:00:00
```

(No pandas 2.2 aparece `datetime64[ns]`: é só a precisão interna, não muda nada para você.)

> 🔮 **Preveja antes de rodar:** que data sai daqui?

```
print(pd.to_datetime(pd.Series(["03/07/2026"]))[0].date())
```

▼ **Resposta**

```
2026-03-07
```

7 de março, não 3 de julho! Sem `format`, o pandas assume o padrão americano (mês/dia/ano). Num laudo, isso é um erro grave e silencioso. Use sempre `format="%d/%m/%Y"` (ou, no mínimo, `dayfirst=True`).

Com a coluna convertida, o acessor `.dt` extrai as partes:

```
df["mes"] = df["data"].dt.month
df["mes_ano"] = df["data"].dt.strftime("%m/%Y")
df["dia_semana"] = df["data"].dt.dayofweek  # 0 = segunda ... 6 = domingo
agosto = df[(df["data"] >= "2026-08-01") & (df["data"] <= "2026-08-31")]
print("Bobinas medidas em agosto:", len(agosto))
```

```
Bobinas medidas em agosto: 19
```

Subtrair datas dá um intervalo (`Timedelta`); `.dt.days` tira o número de dias. O exemplo 07 usa isso para medir quantos dias cada OP levou da emissão à primeira medição.

E para agrupar por mês, `.dt.to_period("M")` transforma cada data no "mês dela":

```
df["periodo"] = df["data"].dt.to_period("M")
print(df.groupby("periodo")["peso_kg"].sum())  # 2026-07 33218.9, 2026-08
22158.0, ...
```

Data inválida (`"31/09/2026"`) com `format` dá `ValueError`. Se preferir marcar e seguir, `errors="coerce"` troca por `NaT` (o `NaN` das datas).

> 💡 **Dica:** para usar o mês como **texto**, prefira `"%Y-%m"` (`"2026-07"`): ordena certo. Com `"%m/%Y"` , `"01/2027"` viria antes de `"12/2026"` .

10. Ordenando

`df.sort_values("peso_kg", ascending=False)` ordena do mais pesado para o mais leve. Para "os 3 maiores", `nlargest` é mais direto:

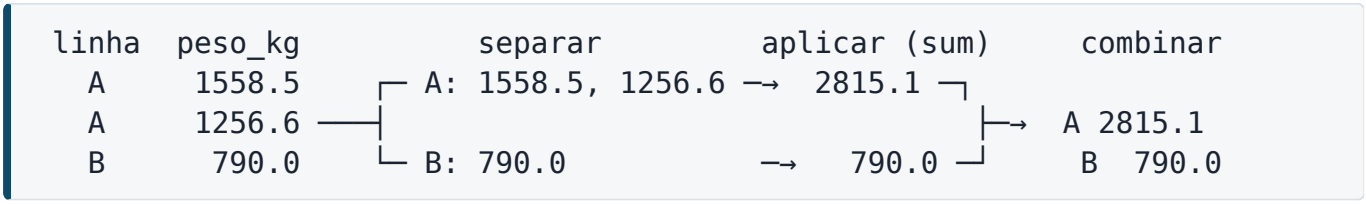
```
print(df.nlargest(3, "cof")[["bobina", "cof"]])
```

```
       bobina   cof
36  024458-02  0.44
14  024430-03  0.38
3   024413-01  0.37
```

Várias chaves com direções diferentes: `df.sort_values(["linha", "cof"], ascending=[True, False])` (linha crescente e, dentro de cada linha, COF decrescente). `sort_index()` ordena pelo índice e `reset_index(drop=True)` renumera depois de ordenar ou filtrar.

11. Agrupando: groupby e agg

`groupby` é a ferramenta que mais economiza seu tempo. A ideia tem nome: **separar, aplicar, combinar**.



De `exemplos/08_groupby_agg.py` :

```
print(df.groupby("linha")["peso_kg"].sum())
```

Saída: uma Series com A 33123.8, B 18543.5, C 12407.7 e D 21046.8. Leia da esquerda para a direita: "agrupe por linha, pegue o peso, some". Para várias estatísticas de uma vez, use `agg` com **agregação nomeada**: `nome_da_coluna_nova=("coluna", "função")`.

```
resumo = df.groupby("linha").agg(
    bobinas=("bobina", "count"),
    ops=("op", "nunique"),
    peso_total_kg=("peso_kg", "sum"),
    cof_medio=("cof", "mean"),
    cof_max=("cof", "max"),
)
print(resumo.round(3))
```

```
       bobinas  ops  peso_total_kg  cof_medio  cof_max
linha
A           28    7        33123.8      0.292     0.38
B           15    4        18543.5      0.309     0.44
C           10    3        12407.7      0.219     0.34
D           17    4        21046.8      0.219     0.28
```

Funções mais usadas: `count`, `size`, `sum`, `mean`, `median`, `min`, `max`, `std`, `nunique`, `first`. A chave do grupo vira o **índice**. Para tê-la como coluna normal (bom para exportar), use `groupby("linha", as_index=False)` ou `.reset_index()` no fim. Duas chaves: `groupby(["linha", "produto"])`.

Exemplo resolvido: "qual linha tem mais COF alto, em %?"

Primeiro eu penso na pergunta para **uma** linha: "das bobinas da linha A, que fração tem COF acima de 0,30?". Isso é `contagem de True / total`, que é exatamente a **média** de uma coluna `True/False`. Então eu crio a coluna e tiro a média por grupo:

```
df["cof_alto"] = df["cof"] > 0.30
print((df.groupby("linha")["cof_alto"].mean() * 100).round(1))
```

```
linha
A    35.7
B    46.7
C    10.0
D     0.0
Name: cof_alto, dtype: float64
```

Repare que a linha D aparece com 0.0. Se eu tivesse feito "filtrar os altos, contar por linha e dividir pelo total", a D sumiria do primeiro resultado e a divisão daria `NaN`. (Esse é um dos defeitos do exercício 07.)

> 🔮 **Preveja antes de rodar:** quais listas saem?

```
t = pd.DataFrame({"linha": ["A", "A", "B"], "obs": ["COF: liberado",
None, None]})
print(t.groupby("linha").size().tolist(), t.groupby("linha")
["obs"].count().tolist())
```

<details>
<summary>Resposta</summary>

```
[2, 1] [1, 0]
```

`size()` conta **linhas** do grupo; `count()` conta valores **não vazios** da coluna.
</details>

transform devolve um valor **por linha** (do tamanho da tabela original), em vez de um por grupo. Serve para comparar cada item com o próprio grupo:

```
df["media_da_op"] = df.groupby("op")["espessura_um"].transform("mean")
df["dif_da_op"] = (df["espessura_um"] - df["media_da_op"]).round(2)
```

> ⚠️ **Erro comum:** `df.groupby("linha").mean()` com colunas de texto na tabela dá `TypeError: dtype 'str' does not support operation 'mean'`. Escolha as colunas (`df.groupby("linha")[["cof", "peso_kg"]].mean()`) ou use `mean(numeric_only=True)`.

12. Juntando tabelas: merge e concat

`merge` é o PROCV (VLOOKUP) do pandas, só que melhor: junta tabelas por uma ou mais colunas-chave. De `exemplos/09_merge.py`, medições + especificação de cada produto:

```
com_spec = medicoes.merge(specs, on="produto", how="left",
validate="many_to_one")
sem_spec = com_spec[com_spec["espessura_nominal"].isna()]
print("Produtos sem especificação:",
sem_spec["produto"].unique().tolist())
```

```
Produtos sem especificação: ['PE-90']
```

O `how` decide o que acontece com quem **não** casa:

```
how="inner"  só o que existe nos dois lados (quem não casou SOME)
how="left"   tudo da esquerda; o que faltar da direita vira NaN
how="right"  tudo da direita
how="outer"  tudo dos dois lados
```

Para conferência de qualidade, quase sempre é `left`: **nenhuma medição pode sumir**. Veja o perigo do `inner` cruzando medições com as OPs do ERP:

```
inner = medicoes.merge(ops[["op", "cliente"]], on="op", how="inner")
print("Medições:", len(medicoes), "| depois do inner com OPs:",
len(inner))
checagem = medicoes.merge(ops[["op", "cliente"]], on="op", how="left",
indicator=True)
print(checagem["_merge"].value_counts())
```

```
Medições: 70 | depois do inner com OPs: 65
_merge
both          65
left_only      5
right_only     0
Name: count, dtype: int64
```

Cinco bobinas sumiram em silêncio: são da OP 024499, medida mas não cadastrada no ERP. O `indicator=True` cria a coluna `_merge`, que mostra de onde veio cada linha. Use-a sempre que estiver em dúvida.

- `validate="many_to_one"` garante que a direita não tem chave repetida. Se a tabela de especificações tiver o mesmo produto duas vezes, o merge para com `MergeError` (um tipo de `ValueError`) em vez de **duplicar** medições sem avisar.
- Colunas com o mesmo nome dos dois lados ganham sufixos `_x` e `_y`. Escolha nomes melhores com `suffixes=("_medido", "_erp")`.
- Chaves com nomes diferentes: `left_on="op", right_on="numero_op"`.

`pd.concat([t1, t2], ignore_index=True)` é outra coisa: **empilha** tabelas com as mesmas colunas (ex.: um CSV por mês, ou as abas da seção 4).

> ⚠️ **Erro comum:** chave com tipos diferentes. Se a OP é texto de um lado e número do outro: `ValueError: You are trying to merge on str and int64 columns for key 'op'.` (no 2.2: `object and int64`). Padronize antes, de preferência como

> texto.

13. Tabela dinâmica: pivot_table

A tabela dinâmica do Excel, em uma chamada (`exemplos/10_pivot_table.py`):

```
df["mes"] = df["data"].dt.strftime("%Y-%m")
tabela = pd.pivot_table(
    df, index="linha", columns="mes", values="peso_kg", aggfunc="sum",
    fill_value=0, margins=True, margins_name="Total",
)
print(tabela.round(0))
```

```
mes    2026-07  2026-08  2026-09    Total
linha
A      14393.0   3376.0  15355.0  33124.0
B      10292.0   3353.0   4898.0  18544.0
C       4274.0   4843.0   3291.0  12408.0
D       4260.0  10586.0   6201.0  21047.0
Total  33219.0  22158.0  29745.0  85122.0
```

Excel (tabela dinâmica)	**`pivot_table`**
Linhas	`index=`
Colunas	`columns=`
Valores	`values=`
Resumir valores por (Soma, Média...)	`aggfunc=`
Mostrar itens sem dados (vazio → 0)	`fill_value=0`
Totais gerais	`margins=True`

Para só **contar** combinações, `pd.crosstab(df["linha"], df["mes"])` é mais curto. E o caminho inverso existe: uma planilha "larga" (um mês por coluna) vira "longa" (uma linha por linha+mês) com `melt`, o formato que o pandas prefere para agrupar e plotar:

`larga.melt(id_vars="linha", var_name="mes", value_name="toneladas")` (veja o exemplo 10).

14. Vetorização x apply x laço for

O seu script antigo contava diretores assim:

```
diretores = 0
for i in df["job_title"]:
    if i == "Director":
        diretores += 1
```

Funciona, mas é Python puro passando por 133 mil valores, um de cada vez. O jeito pandas é uma linha: `(df["job_title"] == "Director").sum()`. O exemplo 11 mede os dois (e o `apply`, que chama uma função Python para cada valor). Na máquina em que esta aula foi escrita, contando os 322 diretores: `for` 34 ms, `apply` 15 ms, vetorizado 7 ms. Com números (dividir o salário por 1000), `apply` levou 26 ms e o vetorizado 0,6 ms: mais de **40 vezes** mais rápido. Os seus números vão ser outros, mas a ordem se mantém.

A regra:

1. **Primeiro** procure a operação vetorizada: aritmética com colunas, comparações, `.str`, `.dt`, `.map(dicionario)`, `.loc` com máscara, `groupby`.
2. Se a lógica for complicada demais para isso, `apply` com uma função sua é aceitável.
3. `for` em linhas de DataFrame (`iterrows`) é o último recurso, para coisas que não são conta (ex.: gerar um certificado por linha).

15. Exportando: CSV e Excel formatado

CSV para o Excel brasileiro abrir certinho (separador, vírgula decimal e BOM):

```
df.to_csv("saida.csv", sep=";", decimal=",", index=False, encoding="utf-8-sig")
```

`index=False` evita uma primeira coluna com 0, 1, 2... que ninguém pediu.

Excel com várias abas: um `pd.ExcelWriter` e vários `to_excel`. E dá para **formatar** com o openpyxl, dentro do mesmo `with`, porque `escritor.sheets[nome]` é a aba do openpyxl. O núcleo de `exemplos/12_exportar_excel_formatado.py`:

```
from openpyxl.styles import Font, PatternFill
from openpyxl.utils import get_column_letter

with pd.ExcelWriter(caminho, engine="openpyxl") as escritor:
```

```
    resumo.to_excel(escritor, sheet_name="Resumo", index=False)
    df.to_excel(escritor, sheet_name="Medições", index=False)

    aba = escritor.sheets["Medições"]
    for celula in aba[1]:  # linha 1 = cabeçalho
        celula.font = Font(bold=True, color="FFFFFF")
        celula.fill = PatternFill("solid", fgColor="1F4E78")
    for i, coluna in enumerate(df.columns, start=1):
        maior = max(len(str(coluna)), *(len(str(v)) for v in df[coluna]))
        aba.column_dimensions[get_column_letter(i)].width = min(maior +
2, 40)
    aba.freeze_panes = "A2"  # congela o cabeçalho
    aba.auto_filter.ref = aba.dimensions  # liga o filtro do Excel

    col_status = df.columns.get_loc("status")  # posição da coluna (0, 1,
2...)
    for linha in aba.iter_rows(min_row=2):
        if linha[col_status].value == "REPROVADA":
            for celula in linha:
                celula.fill = PatternFill("solid", fgColor="FFC7CE")
# ao sair do with, o arquivo é salvo
```

O exemplo completo ainda pinta de amarelo as bobinas com COF fora mas liberadas ("COF: liberado") e formata números (`number_format = "0.000"`). Abra o `saida/relatorio_cof.xlsx` gerado e confira: 62 aprovadas, 5 sem especificação, 2 liberadas e 1 reprovada. O módulo 12 aprofunda o openpyxl.

> 💡 **Dica:** produto sem especificação **não** é aprovado. No exemplo, o `cof_max` dele é `NaN`, e `cof > NaN` dá `False`: sem a linha que marca `SEM ESPECIFICAÇÃO`, essas bobinas sairiam "APROVADAS" sem regra nenhuma. Sempre trate o "não sei" explicitamente.

16. Gráficos com matplotlib

Um gráfico bom responde **uma** pergunta. Escolha o tipo pela pergunta:

Pergunta	Gráfico	Exemplo
Comparar categorias	barras	peso por linha
Evolução no tempo	linhas	peso por semana
Como os valores se espalham?	histograma	espessuras do LAM-62
Duas medidas andam juntas?	dispersão	espessura x gramatura

Pergunta	Gráfico	Exemplo
O processo está estável?	gráfico de controle	seção 17

O matplotlib desenha numa **Figure** (a folha) que contém um ou mais **Axes** (cada gráfico, com seus eixos, título e rótulos). Quase tudo que você configura é método do Axes: `ax.set_title`, `ax.set_xlabel`, `ax.set_ylabel`, `ax.legend`.

Há dois jeitos de desenhar, e eles se misturam bem (`exemplos/13_graficos.py`):

1) Pelo pandas: `Series.plot` e `DataFrame.plot` desenham com o matplotlib e **devolvem o Axes**.

```
peso_t = df.groupby("linha")["peso_kg"].sum() / 1000
ax = peso_t.plot.bar(color="#1F4E78", rot=0)
ax.set_title("Peso medido por linha (jul–set/2026)")
ax.set_xlabel("Linha")
ax.set_ylabel("Peso (t)")
ax.figure.tight_layout()
ax.figure.savefig(SAIDA / "barras_peso_por_linha.png", dpi=120)
print("Barras desenhadas:", len(ax.patches))
```

```
Barras desenhadas: 4
```

2) Pelo pyplot: você cria a figura e o Axes e chama os métodos de desenho.

```
import matplotlib.pyplot as plt

semanal = df.set_index("data").resample("W")["peso_kg"].sum() / 1000
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.plot(semanal.index, semanal.values, marker="o")
ax.set_title("Peso medido por semana")
fig.autofmt_xdate()  # inclina as datas
fig.savefig(SAIDA / "linhas_peso_semanal.png", dpi=120)
```

Os outros dois do exemplo: `ax.hist(valores, bins=6)` (histograma) e `df.plot.scatter(x="espessura_um", y="gramatura_gm2")` (dispersão). A dispersão mostra correlação de 0,99 entre espessura e gramatura (faz sentido: gramatura ≈ espessura × densidade).

Mostrar ou salvar? `fig.savefig("arquivo.png", dpi=120)` grava a imagem (relatório, e-mail, portfólio). `plt.show()` abre uma janela e **para o programa** até você fechá-la. Os exemplos só salvam; acrescente `--mostrar` ao comando para abrir

as janelas. Em scripts automáticos (e nos testes), usa-se `matplotlib.use("Agg")`: desenha só na memória. Feche figuras que não vai mais usar com `plt.close("all")`.

> 🔮 **Preveja antes de rodar:** quantos itens tem `ax.patches`, e qual é o texto de `ax.get_ylabel()`?

```
ax = pd.Series({"A": 3, "B": 5, "C": 2}).plot.bar()
ax.set_title("Bobinas reprovadas")
print(len(ax.patches), repr(ax.get_ylabel()))
```

▼ **Resposta**

```
3 ''
```

Uma barra (um retângulo, um "patch") por categoria. E o rótulo do eixo y é vazio: o pandas não inventa rótulo. Gráfico sem rótulo nos eixos é gráfico pela metade. É isso que os testes dos exercícios 11 e 12 conferem: eles inspecionam o objeto Axes.

> 🔧 **Modifique:** no exemplo 13, troque `plot.bar` por `plot.barh`. Com nomes longos (cargos, clientes), barras horizontais leem muito melhor.

E o Power BI? No mercado, muitos painéis são feitos no Power BI (ou Tableau, Looker Studio). Eles não competem com o pandas, se completam: o Python faz a parte pesada e repetível (ler, limpar, cruzar, conferir regras) e grava uma tabela limpa (Excel, CSV ou um banco); o Power BI conecta nessa tabela e monta o painel interativo para a gerência. Saber os dois aumenta bastante suas chances em vagas de analista. Gráficos em código, como os desta seção, brilham em relatórios automáticos e laudos.

17. Gráfico de controle (média ± 3σ)

No laboratório, a pergunta mais importante não é "esta bobina passou?", e sim "o **processo** está estável?". O gráfico de controle de valores individuais responde:

- **Linha central:** a média.
- **LSC e LIC** (limites superior e inferior de controle): média ± 3 desvios-padrão (σ).

- Ponto fora dos limites indica **causa especial** (troca de matéria-prima, ajuste de máquina, erro de medição): vale investigar.

Exemplo resolvido (`exemplos/14_grafico_controle.py`). Eu quero vigiar a espessura do LAM-62. Primeiro eu separo os dados e ponho na ordem em que as bobinas foram medidas (gráfico de controle é sobre **tempo**). Depois calculo os limites. Por fim, desenho pontos, média, limites e, para comparar, os limites de **especificação** (62 µm ± 7%):

```
lam = df[df["produto"] == "LAM-62"].sort_values(["data",
"bobina"]).reset_index(drop=True)
valores = lam["espessura_um"]
media = valores.mean()
sigma = valores.std()  # desvio-padrão amostral, igual ao DESVPAD do
Excel
lsc, lic = media + 3 * sigma, media - 3 * sigma

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(valores.index + 1, valores, marker="o", label="Espessura")
ax.axhline(media, color="green", label=f"Média {media:.1f}")
ax.axhline(lsc, color="red", linestyle="--", label=f"LSC {lsc:.1f}")
ax.axhline(lic, color="red", linestyle="--", label=f"LIC {lic:.1f}")
ax.axhline(62 * 1.07, color="orange", linestyle=":", label="LSE")
ax.axhline(62 * 0.93, color="orange", linestyle=":", label="LIE")
```

```
Média 61.54 | σ 3.22 | LIC 51.86 | LSC 71.21
Pontos fora de CONTROLE: 0
Pontos fora de ESPECIFICAÇÃO: [67.5, 56.9, 57.1]
Linhas no gráfico: 6
```

Leia com calma, porque essa é uma lição de qualidade de verdade:

- **Limites de controle** são a "voz do processo": calculados dos dados.
- **Limites de especificação** são a "voz do cliente": vêm do contrato.

Aqui o processo está **sob controle** (nenhum ponto fora de 51,9 a 71,2) e mesmo assim 3 bobinas saíram **fora de especificação**. Ou seja: a variação é "normal" para esse processo, mas grande demais para a tolerância do cliente. Não adianta caçar causa especial; é preciso reduzir a variação (ou rever a especificação). Isso se chama **capacidade** do processo (índices Cp e Cpk, veja "Para ir além").

> ⚠️ **Erro comum:** calcular os limites com os mesmos dados que você quer vigiar, com poucos pontos. Um ponto muito fora puxa a média e infla o σ. Com 10 pontos, é matematicamente impossível um deles ficar a mais de 3σ da

> média calculada com ele mesmo. O certo: calcule os limites num período de **referência** estável (20 a 25 pontos, no mínimo) e use-os para vigiar as bobinas **novas**. É o que o exercício 12 faz.

> 💡 **Dica:** o gráfico de valores individuais "de livro" (I-MR) estima o σ pela média das amplitudes móveis ($\bar{MR}$ / 1,128), que é menos sensível a tendências. A versão com `std()` é uma boa primeira aproximação e é a que muitos laboratórios usam no Excel.

18. Laboratório guiado: o mercado de dados em 133 mil salários

Hora de usar tudo num arquivo real e grande: `../arquivos/salaries.csv` (a partir da pasta `curso-python`), com salários de profissionais de dados de 2020 a 2025. Colunas: `work_year`, `experience_level` (EN júnior, MI pleno, SE sênior, EX executivo), `employment_type` (FT tempo integral...), `job_title`, `salary_in_usd`, `employee_residence`, `remote_ratio` (0, 50, 100), `company_location`, `company_size`.

Rode `python 10-dados-com-pandas/exemplos/15_laboratorio_salarios.py` e acompanhe. (Sem o arquivo grande, ele usa a amostra de 600 linhas em `dados/` e os números mudam.)

Pergunta 0: posso confiar nesta base?

```
(133349, 11)
Anos: {2020.0: 75, 2021.0: 216, 2022.0: 1660, 2023.0: 8521, 2024.0:
62240, 2025.0: 60627}
work_year vazio: 10
Linhas idênticas a outra: 68722
```

Três achados antes de qualquer conta: 2020 e 2021 têm poucos registros; 10 linhas sem ano (por isso `work_year` veio como float e o script converte para `Int64`); e **68 mil linhas idênticas a outra**. Duplicata? Talvez não: sem um identificador da pessoa, duas pessoas com mesmo cargo, nível, país e salário geram linhas iguais. Decisão consciente: manter, e registrar essa decisão. Em análise de dados, **escrever as premissas** é metade do trabalho.

Pergunta 1: quanto ganha cada nível? (só tempo integral, em mil USD/ano)

```
NIVEIS = {"EN": "Júnior", "MI": "Pleno", "SE": "Sênior", "EX":
"Executivo"}
```

```
df = df[df["employment_type"] == "FT"].copy()
df["nivel"] = pd.Categorical(
    df["experience_level"].map(NIVEIS), categories=list(NIVEIS.values()),
ordered=True
)
df["salario_mil_usd"] = df["salary_in_usd"] / 1000
df.groupby("nivel", observed=True)["salario_mil_usd"].agg(["count",
"median", "mean"])
```

```
           count  median   mean
nivel
Júnior     12106    87.5   99.9
Pleno      40203   130.0  143.4
Sênior     77057   161.3  173.0
Executivo   3197   190.0  202.0
```

O `pd.Categorical(..., ordered=True)` faz os níveis aparecerem na ordem da carreira, e não em ordem alfabética. Repare que a **média é sempre maior que a mediana**: poucos salários muito altos puxam a média para cima. Para salário, use a mediana ("metade ganha menos que isso").

Pergunta 2: quais cargos aparecem mais?

São 387 cargos diferentes. Os quatro primeiros do `value_counts()`: Data Scientist (17.244), Software Engineer (14.955), Data Engineer (14.794) e Data Analyst (12.209).

Pergunta 3: cargo x nível (mediana, mil USD), com `pivot_table`:

```
nivel               Júnior  Pleno  Sênior  Executivo
job_title
Analytics Engineer    98.9  130.0   153.0      204.0
Data Analyst          81.9   90.0   120.0      114.4
Data Engineer         84.0  120.0   153.1      194.5
Data Scientist        97.0  129.0   160.0      189.4
```

Leitura de carreira: o analista de dados é a porta de entrada mais comum, mas o salário cresce menos com a senioridade. Engenharia de dados e analytics engineering (SQL + Python + pipelines, exatamente a trilha deste curso) crescem mais.

Pergunta 4: trabalho remoto paga menos?

```
            count  median
regime
Híbrido       304    69.6
```

```
Presencial  104802   148.0
Remoto       27457   145.0
```

Parece que híbrido paga a metade. Mas **cuidado**: híbrido tem só 304 registros, de países e anos variados. Para comparar de forma justa, fixe o que puder: só empresas dos EUA, só 2025, e compare **dentro de cada nível**:

```
regime     Híbrido  Presencial  Remoto
nivel
Júnior        26.3        92.9    88.6
Pleno        101.5       138.2   129.9
Sênior         NaN       168.4   160.0
Executivo      NaN       190.0   194.9
Registros híbridos nessa fatia: 3
```

Agora a conclusão honesta: remoto paga um pouco menos que presencial (uns 5%) nos níveis júnior a sênior; sobre híbrido **não dá para concluir nada** (3 registros). Isso é pensar como analista: desconfiar da primeira resposta, controlar variáveis e olhar o tamanho da amostra.

Pergunta 5: e quem mora no Brasil?

São só 85 registros: 5 júnior (mediana 40 mil USD), 32 pleno (57), 45 sênior (66,7) e 3 executivo (110). Provavelmente gente que trabalha para fora (salário em dólar). Serve como pista, não como tabela salarial do Brasil.

O script ainda mostra a evolução por ano e salva `saida/salario_por_nivel.png`, um bom começo para um projeto de portfólio.

> 🔧 **Modifique:** escolha **o seu** cargo-alvo (ex.: `"Data Analyst"` ou `"Data Engineer"`) e calcule a mediana por nível e por `company_size` (S, M, L). Depois faça um gráfico de barras com o resultado.

Lendo mensagens de erro deste módulo

Mensagem	O que significa e como corrigir
`KeyError: 'Espessura'`	A coluna não tem esse nome. `print(df.columns.tolist())` e compare acento, maiúscula, espaço e unidade. Renomeie logo após ler.

Mensagem	O que significa e como corrigir
`UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe7`	O arquivo não é UTF-8. Tente `encoding="cp1252"`.
`ValueError: The truth value of a Series is ambiguous`	`and`, `or`, `not` ou `if` com uma Series. Use `&`, `|`, `~` com parênteses.
`TypeError: Cannot perform reduction 'mean' with string dtype`	A coluna é texto: faltou `decimal` / `thousands` ou há lixo. Use `pd.to_numeric(..., errors="coerce")`.
`ValueError: You are trying to merge on str and int64 columns`	A chave tem tipos diferentes nas duas tabelas. Padronize (de preferência como texto).
`ChainedAssignmentError` (3.x) ou `SettingWithCopyWarning` (2.2)	Você fez `df[...][...] = valor`. Use `df.loc[mascara, "coluna"] = valor`.
`ValueError: day is out of range for month` (ou `doesn't match format`)	Data inexistente ou em outro formato. Corrija na origem ou use `errors="coerce"`.
`PermissionError: [Errno 13] Permission denied: '...xlsx'`	O arquivo está aberto no Excel (Windows bloqueia). Feche-o e rode de novo.
`ImportError: Missing optional dependency 'openpyxl'`	Instale: `python -m pip install openpyxl` (com o venv ativado).

Para ler o traceback inteiro: comece pela **última linha** (o tipo do erro) e suba até a primeira linha que aponta para um arquivo **seu**. As linhas de dentro do pandas quase nunca são o problema.

Boas práticas

- **Leia com os tipos certos** (`sep`, `decimal`, `thousands`, `encoding`, `dtype`) e confira `df.dtypes` logo depois.
- **Renomeie colunas para `snake_case`** sem acento logo após ler.
- **Não altere a tabela de quem chamou:** `df.copy()` no começo da função.
- **Altere valores com `.loc`**, numa operação só. Nada de `df[...][...] = ...`.
- **Prefira operações vetorizadas**; `apply` e `for` só quando não houver alternativa.
- **`merge` com `how="left"`, `validate=` e, na dúvida, `indicator=True`.** Conte as linhas antes e depois.
- **Trate o "não sei" explicitamente:** sem especificação não é aprovado nem reprovado.
- **Datas sempre com `format=`.**

- **Separe leitura, regras e saída em funções** (como no "No mundo real" abaixo). As regras viram funções testáveis com DataFrames pequenos, como nos exercícios.
- **Gráfico com título e rótulos nos eixos**, sempre. Uma pergunta por gráfico.

No mundo real

Juntando tudo, `exemplos/16_conferencia_planilha.py` faz uma versão em pandas do núcleo do seu projeto final: lê as 4 abas da planilha de valores, descobre o produto de cada OP no CSV do ERP, cruza com as especificações, confere espessura e gramatura (nominal ± tolerância) e COF (máximo, aceitando a observação "COF: liberado"), e marca como **pendente** o que não dá para decidir. O coração é uma função pura:

```
def conferir(medicoes: pd.DataFrame, specs: pd.DataFrame) ->
pd.DataFrame:
    """Acrescenta as colunas 'status' e 'motivo' a cada bobina."""
    df = medicoes.merge(specs, on="produto", how="left")
    folga = df["espessura_nominal"] * df["tolerancia_pct"] / 100
    esp_fora = ~df["espessura_um"].between(
        df["espessura_nominal"] - folga, df["espessura_nominal"] + folga
    )
    # ... gram_fora é igual, com a gramatura ...
    cof_fora = df["cof"] > df["cof_max"]
    cof_liberado = df["observacao"].fillna("").str.contains("cof:
liberado", case=False)

    df["motivo"] = ""
    df.loc[esp_fora, "motivo"] += "espessura; "
    df.loc[gram_fora, "motivo"] += "gramatura; "
    df.loc[cof_fora & ~cof_liberado, "motivo"] += "COF; "
    df["motivo"] = df["motivo"].str.rstrip("; ")

    df["status"] = "APROVADA"
    df.loc[df["motivo"] != "", "status"] = "REPROVADA"
    # ... e por último as PENDÊNCIAS: produto sem especificação, OP sem
cadastro
    return df
```

Resultado: 54 aprovadas, 10 pendentes e 6 reprovadas. As reprovadas:

```
linha    bobina    status                    motivo
    B 024433-01 REPROVADA                 gramatura
    B 024458-02 REPROVADA                       COF
    C 024451-02 REPROVADA      espessura; gramatura
```

```
    D 024447-03 REPROVADA      espessura; gramatura
    D 024466-03 REPROVADA                 espessura
    D 024466-04 REPROVADA      espessura; gramatura
```

Repare: a bobina 024430-03 tem COF 0,38 (máximo 0,35), mas está aprovada porque a planilha diz "COF: liberado". E nenhuma bobina "some" ou é aprovada sem regra. No módulo 14, essa mesma lógica volta com regras vindas do banco SQLite (módulo 11) e testes para cada caso.

Resumo

Tarefa	Sintaxe
CSV brasileiro	`pd.read_csv(c, sep=";", decimal=",", thousands=".", encoding="utf-8-sig", dtype={"op": str})`
Excel, todas as abas	`pd.read_excel(c, sheet_name=None, header=1)` → dict
Nomes das abas	`pd.ExcelFile(c).sheet_names`
Empilhar	`pd.concat([t1, t2], ignore_index=True)`
Inspecionar	`df.shape`, `df.info()`, `df.describe()`, `df["c"].value_counts()`
Colunas	`df["c"]`, `df[["a", "b"]]`
Rótulo / posição	`df.loc[linhas, colunas]`, `df.iloc[i, j]`
Filtro	`df[(df["a"] > 1) & (df["b"].isin(["x", "y"]))]`, `~`, `between`
Alterar valores	`df.loc[mascara, "c"] = valor`
Texto	`df["c"].str.strip().str.upper()`, `.str.contains(...)`
Números com lixo	`pd.to_numeric(s, errors="coerce")`
Ausentes	`isna().sum()`, `dropna(subset=[...])`, `fillna(v)`
Duplicatas	`duplicated(subset=[...])`, `drop_duplicates(subset=[...])`
Datas	`pd.to_datetime(s, format="%d/%m/%Y")`, `.dt.month`, `.dt.strftime("%Y-%m")`
Ordenar	`sort_values(["a", "b"], ascending=[True, False])`, `nlargest(n, "c")`
Agrupar	`df.groupby("g", as_index=False).agg(novo=("c", "sum"))`
Por linha do grupo	`df.groupby("g")["c"].transform("mean")`

Tarefa	Sintaxe
Juntar	`a.merge(b, on="k", how="left", validate="many_to_one", indicator=True)`
Tabela dinâmica	`df.pivot_table(index=, columns=, values=, aggfunc=, fill_value=0)`
Exportar	`to_csv(..., sep=";", decimal=",", index=False, encoding="utf-8-sig")`, `pd.ExcelWriter`
Gráfico rápido	`ax = serie.plot.bar()`; `ax.set_title(...)`; `ax.figure.savefig("x.png")`
Gráfico pyplot	`fig, ax = plt.subplots()`; `ax.plot`, `ax.hist`, `ax.scatter`, `ax.axhline`

Você domina este módulo se...

- ▢ Consigo ler um CSV do Excel brasileiro e um CSV em cp1252 sem consultar, e explicar para que serve cada parâmetro.
- ▢ Consigo explicar a diferença entre `loc` e `iloc`, e entre `size()` e `count()`.
- ▢ Consigo filtrar com duas condições sem cair no erro do `and`.
- ▢ Consigo explicar por que `df[mascara]["col"] = valor` não funciona e escrever a versão certa.
- ▢ Consigo limpar uma coluna de números brasileiros com lixo e dizer quantos valores viraram ausentes.
- ▢ Consigo resumir uma tabela com `groupby` / `agg` e cruzar duas tabelas com `merge`, sabendo dizer o que acontece com quem não casa.
- ▢ Consigo escolher o gráfico certo para uma pergunta e salvá-lo em PNG com título e rótulos.
- ▢ Consigo explicar a diferença entre limite de controle e limite de especificação.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_primeiro_dataframe`	★☆☆	criar DataFrame, `sum`, `idxmax`, `loc`
02	`ex02_ler_csv_brasileiro`	★☆☆	`read_csv` com `sep`, `decimal`, `thousands`, `encoding`, `dtype`
03	`ex03_filtrar_fora_da_faixa`	★☆☆	máscaras, `|`, `isin`, `reset_index`
04	`ex04_nova_coluna_desvio`	★☆☆	colunas calculadas, `.loc`, não alterar o original

Nº	Exercício	Nível	O que pratica
05	`ex05_ler_abas_excel`	★★☆	`read_excel` com várias abas, `concat`, `rename`
06	`ex06_limpar_medicoes`	★★☆	`.str`, `to_numeric`, `dropna`, `drop_duplicates`
07	`ex07_corrija_o_relatorio`	★★☆	**leitura de código e depuração**: 4 defeitos clássicos
08	`ex08_resumo_por_linha`	★★☆	`groupby` + agregação nomeada, `round`
09	`ex09_conferir_especificacao`	★★★	`merge` left com `validate`, regras de status
10	`ex10_tabela_por_mes`	★★☆	`to_datetime` com formato, `pivot_table`
11	`ex11_grafico_barras`	★☆☆	gráfico de barras, título e rótulos, salvar PNG
12	`ex12_grafico_controle`	★★★	média ± 3σ, pontos fora, `axhline`, `scatter`
13	`ex13_revisao_emails_expedicao`	★★☆	**revisão**: regex e números BR (mód. 05) + pandas
14	`ex14_revisao_relatorio_excel`	★★★	**revisão**: dataclass (mód. 08) + pathlib (mód. 06) + Excel formatado

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError`, e rode, na pasta `curso-python`:

```
python curso.py testar 10 ex01
```

Os testes usam DataFrames pequenos e arquivos temporários (nunca o arquivo grande). Nos gráficos, usam o backend "Agg" e inspecionam o Axes (título, rótulos, barras, linhas).

Desafio do módulo

Relatório mensal da qualidade: um programa de linha de comando que lê as medições, as OPs do ERP e as especificações, filtra um mês (`--mes 2026-08`), confere cada bobina e gera um Excel formatado com as abas Resumo, Não conformidades e Medições, mais um gráfico de controle em PNG. Enunciado completo em desafio/README.md.

Quiz

Teste o que ficou com o quiz do módulo (14 perguntas, gabarito no final).

Para ir além

- Guia oficial "10 minutes to pandas": https://pandas.pydata.org/docs/user_guide/10min.html
- pandas para quem vem do Excel ("Comparison with spreadsheets"): https://pandas.pydata.org/docs/getting_started/comparison/comparison_with_spreadsheets.html
- Copy-on-Write no pandas (o que mudou no 3.0): https://pandas.pydata.org/docs/user_guide/copy_on_write.html
- Tutoriais do matplotlib (comece por "Quick start guide"): https://matplotlib.org/stable/tutorials/index.html
- Formatação de planilhas com openpyxl: https://openpyxl.readthedocs.io/en/stable/styles.html
- Livro gratuito do criador do pandas, *Python for Data Analysis*, 3ª edição: https://wesmckinney.com/book/
- Controle estatístico de processo (gráficos de controle, Cp e Cpk), no manual do NIST: https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc32.htm

Enunciados dos exercícios

Exercício 10.01 — Primeiro DataFrame

Arquivo: `exercicios/ex01_primeiro_dataframe.py`

```
Nível: ★☆☆

Contexto: a expedição mandou duas listas soltas: o código de cada bobina
e o
peso dela. Você quer transformar isso numa tabela para fazer contas.

Tarefa: implemente três funções.
  1. criar_tabela_bobinas(bobinas, pesos): devolve um DataFrame com as
colunas
     "bobina" e "peso_kg", nessa ordem. Se as listas tiverem tamanhos
     diferentes, levante ValueError.
  2. peso_total(tabela): soma da coluna "peso_kg" como float. Tabela
vazia -> 0.0.
  3. bobina_mais_pesada(tabela): o código (texto) da bobina de maior
peso. Se
     houver empate, vale a que aparece primeiro.

Exemplos:
    >>> tabela = criar_tabela_bobinas(["024412-01", "024412-02"],
[1250.5, 980.0])
```

```
    >>> tabela
          bobina  peso_kg
    0  024412-01   1250.5
    1  024412-02    980.0
    >>> peso_total(tabela)
    2230.5
    >>> bobina_mais_pesada(tabela)
    '024412-01'

Dica: pd.DataFrame({"coluna": lista, ...}); a Series tem .sum() e
.idxmax()
(o rótulo da linha do maior valor), e .loc[rótulo, "coluna"] pega um
valor.

Teste com:  python curso.py testar 10 ex01
```

Exercício 10.02 — Ler CSV brasileiro

Arquivo: `exercicios/ex02_ler_csv_brasileiro.py`

```
Nível: ★☆☆

Contexto: o laboratório salva as medições pelo Excel em português (CSV
com ponto
e vírgula, vírgula decimal e ponto de milhar). As OPs vêm do ERP, num CSV
antigo em cp1252. Ler errado faz o 024412 virar 24412 e o 1.558,5 virar
texto.

Tarefa: implemente duas funções.
  1. ler_medicoes(caminho): lê um CSV com separador ";", vírgula decimal,
     ponto de milhar e encoding "utf-8-sig". A coluna "op" deve continuar
texto
     (com os zeros à esquerda). Devolve o DataFrame.
  2. ler_ops_erp(caminho): lê o CSV do ERP: separador ";", encoding
"cp1252",
     vírgula decimal, ponto de milhar e a coluna "op" como texto.

Exemplo de arquivo de medições:
    data;op;bobina;espessura_um;peso_kg
    03/07/2026;024412;024412-01;38,1;1.558,5

Exemplos:
    >>> df = ler_medicoes("medicoes.csv")
    >>> df.loc[0, "op"], df.loc[0, "espessura_um"], df.loc[0, "peso_kg"]
    ('024412', 38.1, 1558.5)

Dica: pd.read_csv tem os parâmetros sep, decimal, thousands, encoding e
dtype.
```

```
Teste com:  python curso.py testar 10 ex02
```

Exercício 10.03 — Filtrar o que está fora da faixa

Arquivo: `exercicios/ex03_filtrar_fora_da_faixa.py`

```
Nível: ★☆☆

Contexto: antes de liberar uma série, o laboratório quer ver só as
bobinas com
alguma medida fora da faixa, e às vezes só as de algumas linhas.

Tarefa: implemente duas funções. Nenhuma pode alterar o DataFrame
recebido.
  1. fora_da_faixa(df, coluna, minimo, maximo): devolve as linhas em que
o valor
     da coluna é MENOR que o mínimo ou MAIOR que o máximo (os limites em
si estão
     dentro da faixa). Mantenha a ordem original e renumere o índice (0,
1, 2...).
  2. filtrar_linhas(df, linhas): devolve as linhas cuja coluna "linha"
está na
     lista recebida (ex.: ["A", "C"]), na ordem original e com índice 0,
1, 2...

Exemplos:
    >>> df
          bobina linha  espessura_um
    0  024412-01     A          38.1
    1  024412-02     A          44.5
    2  024413-01     B          35.0
    >>> fora_da_faixa(df, "espessura_um", 36.0, 44.0)
          bobina linha  espessura_um
    0  024412-02     A          44.5
    >>> filtrar_linhas(df, ["B"])
          bobina linha  espessura_um
    0  024413-01     B          35.0

Dica: máscara booleana com | (ou) e parênteses, ou ~ com .between();
.isin()
para "está na lista"; .reset_index(drop=True) para renumerar.

Teste com:  python curso.py testar 10 ex03
```

Exercício 10.04 — Colunas calculadas: desvio e status

Arquivo: `exercicios/ex04_nova_coluna_desvio.py`

```
Nível: ★☆☆

Contexto: a espessura nominal do produto é 62 µm, com tolerância de ±7%.
Você
quer ver, bobina a bobina, quanto cada uma desviou do nominal e se
passou.

Tarefa: implemente duas funções. As duas devolvem um DataFrame NOVO e não
podem alterar o DataFrame recebido.
  1. adicionar_desvio(df, nominal): acrescenta as colunas
       "desvio_um"  = espessura_um - nominal              (2 casas
decimais)
       "desvio_pct" = (espessura_um - nominal) / nominal * 100  (2 casas)
  2. classificar(df, tolerancia_pct): recebe uma tabela que já tem
     "desvio_pct" e acrescenta "status": "APROVADA" se o desvio, em valor
     absoluto, for <= tolerancia_pct; senão "REPROVADA".

Exemplos:
    >>> df = pd.DataFrame({"bobina": ["01", "02"], "espessura_um": [61.8,
67.5]})
    >>> com_desvio = adicionar_desvio(df, 62.0)
    >>> com_desvio
      bobina  espessura_um  desvio_um  desvio_pct
    0     01          61.8      -0.20       -0.32
    1     02          67.5       5.50        8.87
    >>> classificar(com_desvio, 7.0)["status"].tolist()
    ['APROVADA', 'REPROVADA']

Dica: comece com resultado = df.copy(); contas com colunas são
vetorizadas;
.round(2) e .abs() funcionam na coluna inteira; para o status, crie a
coluna com
um valor e troque com .loc[mascara, "status"] = "...".

Teste com:  python curso.py testar 10 ex04
```

Exercício 10.05 — Todas as abas numa tabela só

Arquivo: `exercicios/ex05_ler_abas_excel.py`

```
Nível: ★★☆

Contexto: a planilha de valores do laboratório tem uma aba por linha de
produção
```

```
("Linha A", "Linha B", ...). Às vezes alguém acrescenta outras abas, como
"Instruções". Cada aba de linha tem um TÍTULO na primeira linha da
planilha e o
cabeçalho só na segunda:

    linha 1:  Controle de qualidade — Linha A
    linha 2:  OP | Bobina | Espessura (µm)
    linha 3:  024412 | 024412-01 | 38.1
    ...

Tarefa: implemente ler_todas_as_abas(caminho), que devolve UMA tabela
com:
  - só as abas cujo nome começa com "Linha " (as outras são ignoradas);
  - as colunas, nesta ordem: "linha", "op", "bobina", "espessura_um";
  - "linha" = a letra do nome da aba ("Linha C" -> "C");
  - "op" como texto (preserve zeros à esquerda: "024412");
  - as linhas na ordem das abas e, dentro da aba, na ordem da planilha;
  - índice 0, 1, 2, ...
Se nenhuma aba começar com "Linha ", levante ValueError.

Exemplo:
    >>> ler_todas_as_abas("planilha.xlsx")
      linha      op     bobina  espessura_um
    0     A  024412  024412-01          38.1
    1     A  024412  024412-02          36.4
    2     B  024413  024413-01          32.9

Dica: pd.ExcelFile(caminho).sheet_names dá a lista de nomes das abas.
pd.read_excel(caminho, sheet_name=[nomes], header=1, dtype={"OP": str})
devolve um dicionário {nome da aba: DataFrame}. Cuidado: header=1 numa
aba de
uma linha só (como "Instruções") dá erro; escolha as abas ANTES de ler.
Junte as partes com pd.concat.

Teste com:  python curso.py testar 10 ex05
```

Exercício 10.06 — Limpar medições digitadas à mão

Arquivo: `exercicios/ex06_limpar_medicoes.py`

```
Nível: ★★☆

Contexto: uma planilha preenchida por várias pessoas chegou bagunçada:
linha
escrita como " a" ou "B ", números com vírgula guardados como texto,
"n/d" no
lugar da medida e a mesma bobina lançada duas vezes.

Tarefa: implemente limpar_medicoes(bruto). A tabela recebida tem as
```

```
colunas
"op", "linha", "bobina", "espessura_um" e "peso_kg", TODAS como texto
(foi lida
com dtype=str, então células vazias chegam como ausentes). Devolva uma
tabela
NOVA, com as mesmas colunas, aplicando as regras nesta ordem:
  1. "linha": sem espaços nas pontas e em maiúscula (" a" -> "A").
  2. "espessura_um": texto com vírgula decimal vira número ("49,8" ->
49.8,
     " 34,7 " -> 34.7). O que não for número ("n/d", vazio) vira ausente
(NaN).
  3. "peso_kg": formato brasileiro com milhar ("1.120,5" -> 1120.5);
vazio fica
     ausente (NaN), sem descartar a linha.
  4. Descarte as linhas sem espessura (não dá para avaliar sem medida).
  5. Se a mesma "bobina" aparecer mais de uma vez, fique só com a
primeira.
  6. Índice renumerado: 0, 1, 2, ...
Não altere a tabela recebida.

Exemplo:
    >>> bruto
           op linha     bobina espessura_um  peso_kg
    0  024470     A  024470-01         51,2  1.120,5
    1  024470    a   024470-02          n/d  1.098,0
    >>> limpar_medicoes(bruto)
           op linha     bobina  espessura_um  peso_kg
    0  024470     A  024470-01          51.2   1120.5

Dica: .str.strip(), .str.upper(), .str.replace(".", "", regex=False),
pd.to_numeric(..., errors="coerce"), .dropna(subset=[...]),
.drop_duplicates(subset=[...]).

Teste com:  python curso.py testar 10 ex06
```

Exercício 10.07 — Corrija o relatório (leitura de código e depuração)

Arquivo: `exercicios/ex07_corrija_o_relatorio.py`

```
Nível: ★★☆

Contexto: um colega escreveu a função abaixo para dizer, por linha de
produção,
qual porcentagem das bobinas teve COF acima do máximo. O relatório saiu
com
números estranhos e o gerente desconfiou. Leia o código com atenção ANTES
de
```

```
rodar qualquer coisa:

    def percentual_cof_alto(df, cof_max):
        df["status"] = "OK"
        df[df["cof"] > cof_max]["status"] = "ALTO"               # (1)
        altas = df[df["status"] == "ALTO"]
        contagem = altas.groupby("linha")["bobina"].count()
        total = df.groupby("linha")["bobina"].count()
        percentual = (contagem / total * 100).round(1)            # (2)
        return percentual.sort_values()                           # (3)

Este código tem QUATRO defeitos:
  - um que faz a coluna "status" nunca virar "ALTO";
  - um que deixa linhas sem nenhuma bobina alta com NaN em vez de 0.0;
  - um na ordenação (o gerente quer as piores linhas primeiro);
  - um efeito colateral: a função altera o DataFrame de quem chamou.

Tarefa: escreva a versão correta de percentual_cof_alto(df, cof_max), que
devolve uma Series:
  - índice = as linhas de produção (todas as que aparecem em df), nome
"linha";
  - valores = % de bobinas com cof > cof_max (estritamente maior), 1 casa
decimal;
    linha sem bobina alta -> 0.0;
  - nome da Series: "pct_cof_alto";
  - ordem: do maior percentual para o menor; empate -> ordem alfabética
da linha;
  - sem alterar o df recebido.

Exemplo:
    >>> df
      linha bobina   cof
    0     A     a1  0.38
    1     A     a2  0.30
    2     B     b1  0.25
    >>> percentual_cof_alto(df, 0.35)
    linha
    A    50.0
    B     0.0
    Name: pct_cof_alto, dtype: float64

Dica: para cada defeito, escreva primeiro (num comentário) o que acontece
e por
quê. Lembre que a média de uma coluna True/False é a proporção de True.

Teste com:  python curso.py testar 10 ex07
```

Exercício 10.08 — Resumo por linha de produção

Arquivo: `exercicios/ex08_resumo_por_linha.py`

```
Nível: ★★☆

Contexto: na reunião mensal, a qualidade apresenta um quadro por linha:
quantas
bobinas foram medidas, o peso total, a espessura média e a variação dela.

Tarefa: implemente resumo_por_linha(df). A tabela recebida tem as colunas
"linha", "bobina", "espessura_um", "peso_kg" e "cof". Devolva um
DataFrame com
UMA linha por linha de produção, em ordem alfabética, índice 0, 1, 2... e
as
colunas, nesta ordem:
  - "linha"
  - "bobinas":          quantidade de bobinas
  - "peso_total_kg":    soma de peso_kg, 1 casa decimal
  - "espessura_media":  média de espessura_um, 2 casas
  - "espessura_desvio": desvio-padrão amostral de espessura_um, 2 casas
                        (linha com uma bobina só fica com NaN, e tudo
bem)
  - "cof_max":          maior COF

Exemplo:
    >>> resumo_por_linha(df)
      linha  bobinas  peso_total_kg  espessura_media  espessura_desvio
cof_max
    0     A        3         3330.5            39.37              1.17
0.27
    1     B        1         1310.2            35.30               NaN
0.37

Dica: df.groupby("linha", as_index=False).agg(nome=("coluna", "função"),
...)
e depois .round({...}) com um dicionário {coluna: casas}.

Teste com:  python curso.py testar 10 ex08
```

Exercício 10.09 — Conferir medições contra a especificação (merge)

Arquivo: `exercicios/ex09_conferir_especificacao.py`

```
Nível: ★★★
```

```
Contexto: as medições dizem o produto de cada bobina; a tabela de
especificações diz o nominal e a tolerância (%) de cada produto. Conferir
uma
contra a outra é um PROCV no Excel. No pandas, é um merge.

Tarefa: implemente conferir_espessura(medicoes, specs).
  medicoes: colunas "bobina", "produto", "espessura_um" (pode ter
outras).
  specs:    colunas "produto", "nominal_um", "tolerancia_pct".
Devolva uma tabela NOVA com todas as colunas de medicoes, na mesma ordem
de
linhas, índice 0, 1, 2..., mais três colunas no fim:
  - "minimo" = nominal_um - nominal_um * tolerancia_pct / 100 (2 casas)
  - "maximo" = nominal_um + nominal_um * tolerancia_pct / 100 (2 casas)
  - "status":
      "SEM ESPECIFICAÇÃO" se o produto não existe em specs (minimo e
maximo
                          ficam ausentes, NaN);
      "APROVADA"  se minimo <= espessura_um <= maximo;
      "REPROVADA" nos outros casos.
As colunas "nominal_um" e "tolerancia_pct" NÃO devem aparecer no
resultado.
Se specs tiver o mesmo produto em duas linhas, levante ValueError (a
regra
ficaria ambígua).

Exemplo:
    >>> conferir_espessura(medicoes, specs)
      bobina produto  espessura_um  minimo  maximo             status
    0     01  LAM-62          67.5   57.66   66.34          REPROVADA
    1     02   PE-40          38.1   36.00   44.00           APROVADA
    2     03   PE-90          94.4     NaN     NaN  SEM ESPECIFICAÇÃO

Dica: merge(..., how="left", validate="many_to_one") mantém todas as
medições e
reclama (com um erro que é ValueError) se a especificação estiver
duplicada.

Teste com:  python curso.py testar 10 ex09
```

Exercício 10.10 — Datas e tabela dinâmica: peso por linha e mês

Arquivo: `exercicios/ex10_tabela_por_mes.py`

```
Nível: ★★☆

Contexto: o PCP pediu uma tabela com o peso medido por linha (nas linhas
da
tabela) e por mês (nas colunas), igual à tabela dinâmica que fazem no
```

```
Excel.

Tarefa: implemente duas funções.
  1. adicionar_mes(df): a coluna "data" tem texto no formato dd/mm/aaaa
     ("03/07/2026"). Devolva uma tabela NOVA com a coluna "mes" no
formato
     "AAAA-MM" ("2026-07"). Data inválida ou vazia -> ValueError.
  2. peso_por_linha_e_mes(df): recebe uma tabela com "linha", "data" e
     "peso_kg" e devolve a tabela dinâmica:
       - índice: as linhas de produção (nome do índice: "linha"), em
ordem;
       - colunas: os meses "AAAA-MM" em ordem (nome das colunas: "mes");
       - valores: soma do peso_kg, 1 casa decimal; combinação sem
         produção = 0.0.

Exemplo:
    >>> df
      linha        data  peso_kg
    0     A  03/07/2026   1558.5
    1     A  15/08/2026    980.0
    2     B  20/07/2026   1310.2
    >>> peso_por_linha_e_mes(df)
    mes    2026-07  2026-08
    linha
    A       1558.5    980.0
    B       1310.2      0.0

Dica: pd.to_datetime(..., format="%d/%m/%Y") levanta ValueError sozinho
quando
a data não bate com o formato; .dt.strftime("%Y-%m") formata; pivot_table
tem
index, columns, values, aggfunc e fill_value.

Teste com:  python curso.py testar 10 ex10
```

Exercício 10.11 — Gráfico de barras: peso por linha

Arquivo: `exercicios/ex11_grafico_barras.py`

```
Nível: ★☆☆

Contexto: o gerente da produção quer, no relatório mensal, um gráfico
simples
com o peso total medido em cada linha de produção, em toneladas.

Tarefa: implemente duas funções.
  1. grafico_peso_por_linha(df): a tabela tem as colunas "linha" e
"peso_kg".
     Desenhe um gráfico de BARRAS com uma barra por linha de produção (em
```

```
ordem
     alfabética), altura = soma do peso em TONELADAS (kg / 1000).
Configure:
       título:  "Peso medido por linha"
       eixo x:  "Linha"
       eixo y:  "Peso (t)"
     Devolva o objeto Axes (o "gráfico" do matplotlib).
  2. salvar_png(ax, caminho): salva a figura do Axes em PNG no caminho
dado,
     criando a pasta se ela não existir. Devolve o caminho como Path.

Exemplo:
    >>> ax = grafico_peso_por_linha(df)
    >>> ax.get_title()
    'Peso medido por linha'
    >>> [barra.get_height() for barra in ax.patches]
    [33.1238, 18.5435, 12.4077, 21.0468]
    >>> salvar_png(ax, Path("saida/peso.png"))
    PosixPath('saida/peso.png')      (no Windows aparece WindowsPath)

Dica: (df.groupby("linha")["peso_kg"].sum() / 1000).plot.bar() devolve o
Axes;
ax.set_title, ax.set_xlabel, ax.set_ylabel; ax.figure.savefig(caminho).

Teste com:  python curso.py testar 10 ex11
```

Exercício 10.12 — Gráfico de controle (média ± 3σ)

Arquivo: `exercicios/ex12_grafico_controle.py`

```
Nível: ★★★

Contexto: o laboratório quer acompanhar a espessura de um produto bobina
a
bobina. Os limites de controle são calculados num período de REFERÊNCIA,
em que
o processo estava estável, e depois usados para vigiar as bobinas novas:
    média = média da referência
    σ     = desvio-padrão amostral da referência
    LIC   = média - 3σ   (limite inferior de controle)
    LSC   = média + 3σ   (limite superior de controle)
Um ponto fora de [LIC, LSC] indica causa especial: vale investigar.

Tarefa: implemente três funções.
  1. limites_de_controle(referencia): recebe uma Series de números e
devolve o
     dicionário {"media": ..., "lic": ..., "lsc": ...} (floats, sem
arredondar).
     Com menos de 2 valores, levante ValueError (não dá para calcular σ).
```

```
  2. pontos_fora_de_controle(valores, limites): devolve a lista das
POSIÇÕES
     (0, 1, 2...) dos valores estritamente abaixo do LIC ou acima do LSC.
  3. grafico_controle(valores, limites, titulo): desenha e devolve o Axes
com:
       - os valores como linha com marcadores, no eixo x as amostras 1,
2, 3...;
       - três linhas horizontais (ax.axhline): média, LIC e LSC;
       - os pontos fora de controle destacados com ax.scatter (se não
houver
         nenhum, não chame o scatter);
       - título = titulo; eixo x "Amostra"; eixo y "Valor".
     O gráfico deve ter exatamente 4 linhas (a dos valores + 3
horizontais).

Exemplos:
    >>> ref = pd.Series([61.8, 62.3, 61.5, 62.0, 62.4])
    >>> limites = limites_de_controle(ref)
    >>> {k: round(v, 2) for k, v in limites.items()}
    {'media': 62.0, 'lic': 60.9, 'lsc': 63.1}
    >>> pontos_fora_de_controle(pd.Series([62.1, 63.5, 61.9, 60.0]),
limites)
    [1, 3]

Dica: .std() do pandas já é o amostral; posições: range(len(valores)) ou
.reset_index(drop=True) e o índice; fig, ax = plt.subplots().

Teste com:  python curso.py testar 10 ex12
```

Exercício 10.13 — Revisão: e-mails da expedição viram tabela

Arquivo: `exercicios/ex13_revisao_emails_expedicao.py`

```
Nível: ★★☆

Revisão intercalada: expressões regulares e números brasileiros (módulo
05),
listas e dicionários (módulo 03) e pandas (este módulo).

Contexto: a expedição pede certificados por e-mail. Cada e-mail tem, em
algum
lugar do texto, linhas assim:

    OP: 24518
    NF: 123456
    Peso: 1.250,5 kg

Uma mesma OP pode sair em várias notas fiscais (cargas parciais).
```

```
Tarefa: implemente três funções.
  1. extrair_pedido(texto): devolve {"op": int, "nf": str, "peso_kg":
float}
     ou None se faltar qualquer um dos três. Aceite maiúsculas ou
minúsculas
     ("op:", "Peso:") e espaços variados depois dos dois-pontos. O peso
vem no
     formato brasileiro ("1.250,5" -> 1250.5; "980" -> 980.0).
  2. pedidos_para_tabela(emails): recebe uma lista de textos e devolve um
     DataFrame com as colunas "op", "nf", "peso_kg", um pedido por linha,
na
     ordem dos e-mails, ignorando os incompletos. Sem pedidos válidos:
DataFrame
     vazio, mas com essas três colunas.
  3. peso_por_op(pedidos): recebe a tabela do item 2 e devolve um
DataFrame com
     as colunas "op", "notas" (quantas NFs) e "peso_total_kg" (soma, 1
casa),
     uma linha por OP, em ordem crescente de OP, índice 0, 1, 2...

Exemplos:
    >>> extrair_pedido("Bom dia!\nOP: 24518\nNF: 123456\nPeso: 1.250,5
kg")
    {'op': 24518, 'nf': '123456', 'peso_kg': 1250.5}
    >>> extrair_pedido("OP: 24518 (NF ainda não emitida)") is None
    True

Dica: re.search(r"OP:\s*(\d+)", texto, re.IGNORECASE); para o peso,
capture
[\d.,]+ e converta tirando o ponto e trocando a vírgula. Monte uma lista
de
dicionários e passe para pd.DataFrame(lista, columns=[...]).

Teste com:  python curso.py testar 10 ex13
```

Exercício 10.14 — Revisão: relatório Excel formatado a partir de objetos

Arquivo: `exercicios/ex14_revisao_relatorio_excel.py`

```
Nível: ★★★

Revisão intercalada: dataclasses (módulo 08), pathlib e exceções (módulo
06) e
exportação para Excel com formatação (este módulo).

Contexto: o seu sistema de conferência produz uma lista de objetos
Resultado
```

```
(a dataclass abaixo, já pronta). O gerente quer receber isso numa
planilha
"apresentável": cabeçalho destacado, colunas largas o bastante, cabeçalho
congelado e as bobinas reprovadas pintadas de vermelho claro.

Tarefa: implemente duas funções.
  1. resultados_para_dataframe(resultados): devolve um DataFrame com uma
linha
     por objeto e as colunas na ordem dos campos da dataclass ("op",
"bobina",
     "espessura_um", "status"). Lista vazia -> DataFrame vazio com essas
colunas.
  2. exportar_relatorio(resultados, caminho): grava um .xlsx com UMA aba
chamada
     "Resultados" (sem a coluna de índice do pandas) e:
       - cabeçalho (linha 1) em negrito;
       - painel congelado abaixo do cabeçalho (freeze_panes = "A2");
       - largura de cada coluna >= tamanho do texto do cabeçalho + 2;
       - linhas com status "REPROVADA": TODAS as células da linha com
         preenchimento sólido na cor "FFC7CE"; as outras linhas sem
preenchimento.
     Crie as pastas do caminho se não existirem. Lista vazia ->
ValueError
     (não faz sentido mandar relatório vazio). Devolva o caminho como
Path.

Exemplo:
    >>> r = [Resultado("024447", "024447-03", 67.5, "REPROVADA"),
    ...      Resultado("024447", "024447-04", 62.0, "APROVADA")]
    >>> exportar_relatorio(r, Path("saida/relatorio.xlsx"))
    PosixPath('saida/relatorio.xlsx')      (no Windows aparece
WindowsPath)

Dica: dataclasses.asdict e dataclasses.fields; pd.ExcelWriter(caminho,
engine="openpyxl") e, dentro do with, escritor.sheets["Resultados"] dá a
aba
do openpyxl; Font(bold=True), PatternFill("solid", fgColor="FFC7CE"),
aba.column_dimensions["A"].width, get_column_letter(1) == "A".

Teste com:  python curso.py testar 10 ex14
```

Desafio do módulo 10 — Relatório mensal da qualidade

Contexto

Todo começo de mês, alguém do laboratório passa uma manhã montando o mesmo relatório: filtra as medições do mês na planilha, confere bobina por bobina contra as especificações, conta aprovadas e reprovadas por linha, pinta de vermelho o que está fora e cola um gráfico. Você vai transformar essa manhã em **um comando**.

O que o programa faz

Lê três arquivos de `10-dados-com-pandas/dados/`:

- `medicoes.csv` (CSV brasileiro, UTF-8 com BOM);
- `ops_protheus.csv` (exportação do ERP, `cp1252`), para saber o cliente de cada OP;
- `especificacoes.csv`, com nominal, tolerância (%) e COF máximo de cada produto.

Filtra um mês, confere cada bobina e gera, na pasta `saida/`, um Excel formatado e dois gráficos em PNG.

Requisitos obrigatórios

1. Linha de comando com `argparse` (módulo 07):
   - `--mes AAAA-MM` (opcional; sem ele, use o último mês que aparece nos dados);
   - `--produto` (produto do gráfico de controle; padrão `LAM-62`);
   - `--dados` e `--saida` (pastas; padrões: `dados/` e `saida/` do módulo).
2. Leitura correta dos três arquivos (separador, decimal, milhar, encoding, OP como texto).
3. Conferência de cada bobina:
   - espessura e gramatura dentro de nominal ± tolerância (%), limites inclusivos;
   - COF até o máximo, **exceto** se a observação disser "COF: liberado" (em qualquer combinação de maiúsculas);
   - status `APROVADA`, `REPROVADA` ou `PENDENTE`, e uma coluna `motivo` (ex.: `"espessura; gramatura"`);
   - produto sem especificação ou OP sem cadastro no ERP é `PENDENTE`, nunca aprovado.
4. Excel `relatorio_qualidade_AAAA-MM.xlsx` com três abas:
   - **Resumo**: por linha de produção, bobinas, aprovadas, reprovadas, pendentes, peso total (kg) e % de aprovadas, com uma linha de **Total** no fim;
   - **Não conformidades**: só as bobinas reprovadas ou pendentes;

    - **Medições**: todas as bobinas do mês, com status e motivo.
    - Em todas: cabeçalho destacado, largura de coluna ajustada, painel congelado. Linhas reprovadas em vermelho claro (`FFC7CE`) e pendentes em amarelo (`FFEB9C`).
5. `aprovacao_AAAA-MM.png`: barras com o % de aprovadas por linha, com título e rótulos.
6. `controle_AAAA-MM.png`: gráfico de controle da espessura do `--produto` no mês, com média e limites ± 3σ calculados nos **meses anteriores** (referência). Se não houver histórico, calcule no próprio mês e avise.
7. Mês sem medições ou arquivo inexistente: mensagem clara em português e código de saída 1 (sem traceback).

Extras opcionais

- Aba "Por cliente" com bobinas e peso por cliente (use o `cliente` do ERP).
- Inserir os PNG dentro do Excel (`openpyxl.drawing.image.Image`).
- Opção `--todas` para gerar um relatório por mês, de uma vez.
- Testes com pytest para `conferir` e `resumir`, usando DataFrames pequenos.
- Repetir a análise com a planilha `planilha_linhas.xlsx` (4 abas) em vez do CSV.

Exemplo de execução

```
> python 10-dados-com-pandas/desafio/solucao_referencia.py --mes 2026-08
Relatório de 2026-08: 19 bobinas
linha  bobinas  aprovadas  reprovadas  pendentes  peso_total_kg
pct_aprovadas
    A        3          3           0          0         3376.1
100.0
    B        3          2           1          0         3353.2
66.7
    C        4          3           1          0         4843.0
75.0
    D        9          6           3          0        10585.7
66.7
Total       19         14           5          0        22158.0
73.7
LAM-62: média 61.17 µm, LIC 50.68, LSC 71.66, 0 ponto(s) fora de controle
Arquivos em ...\10-dados-com-pandas\saida: relatorio_qualidade_2026-
08.xlsx, aprovacao_2026-08.png, controle_2026-08.png
```

```
> python 10-dados-com-pandas/desafio/solucao_referencia.py --mes 2025-01
Erro: não há medições em 2025-01. Meses disponíveis: 2026-07, 2026-08,
2026-09
```

Critérios de aceite

- [ ] Roda sem argumentos e gera os três arquivos em `saida/`.
- [ ] `--mes 2026-08` mostra 19 bobinas: 14 aprovadas e 5 reprovadas.
- [ ] `--mes 2026-09` mostra 10 pendentes na linha A (produto PE-90 sem especificação e OP 024499 sem cadastro no ERP).
- [ ] A bobina 024430-03 (COF 0,38, "COF: liberado") aparece como aprovada em julho.
- [ ] O Excel abre sem aviso de reparo, com as três abas formatadas e cores corretas.
- [ ] Os gráficos têm título e rótulos nos dois eixos.
- [ ] Mês inexistente dá mensagem amigável e código de saída 1.
- [ ] `ruff check` e `ruff format --check` passam no seu arquivo.

Dicas de como começar

1. Comece pela **leitura**: uma função `ler_dados(pasta)` que devolve uma tabela só (medições + cliente + especificação, com dois `merge` `how="left"`). Imprima `df.dtypes` e `df.shape` para conferir.
2. Escreva `conferir(df)` como no "No mundo real" da aula: máscaras para cada regra, coluna `motivo` montada com `.loc`, e as pendências por último.
3. `pd.crosstab(df["linha"], df["status"])` dá quase o resumo inteiro de graça.
4. Só depois cuide do Excel e dos gráficos. Reaproveite o exemplo 12 (formatação) e o exemplo 14 (gráfico de controle).
5. Deixe o `main()` curto: ler argumentos, chamar as funções, imprimir o resumo.

A solução de referência está em `solucao_referencia.py`. Tente antes de abrir!

Quiz — Módulo 10: pandas, Excel e gráficos

Responda sem rodar. Depois confira no gabarito (e, na dúvida, rode no REPL). Em todas as perguntas, considere `import pandas as pd`.

1. Um CSV salvo pelo Excel brasileiro tem a linha `024412;38,1;1.558,5`. Quais parâmetros do `read_csv` fazem `1.558,5` virar o número `1558.5`?

- a) `decimal=","`
- b) `decimal=","` e `thousands="."`
- c) `sep=";"`
- d) `dtype=float`

2. O que este código imprime?

```
s = pd.Series([3, 1, 2])
print(s[s > 1].tolist())
```

3. `df` tem 5 linhas, com índice 0, 1, 2, 3, 4. O que este código imprime?

```
print(len(df.iloc[1:3]), len(df.loc[1:3]))
```

4. Onde está o erro?

```
fora = df[df["linha"] == "A" and df["cof"] > 0.30]
```

5. Depois deste código, a coluna `status` da tabela `df` muda? Por quê? Como escrever do jeito certo?

```
df[df["cof"] > 0.35]["status"] = "FORA"
```

6. O que este código imprime?

```
print(pd.Series([True, False, True, True]).mean())
```

7. O que este código imprime?

```
t = pd.DataFrame({"linha": ["A", "B", "A"], "cof": [0.2, None, 0.4]})
print(t.groupby("linha").size().tolist(), t.groupby("linha")
["cof"].count().tolist())
```

8. Você quer conferir medições contra especificações e **nenhuma medição pode sumir**, mesmo sem especificação. Qual `how` usar no `merge`?

- a) `"inner"`
- b) `"left"` (medições à esquerda)
- c) `"right"` (medições à esquerda)
- d) tanto faz

9. O que este código imprime?

```
a = pd.DataFrame({"op": ["1", "2", "3"]})
b = pd.DataFrame({"op": ["1", "3"], "cliente": ["x", "y"]})
print(len(a.merge(b, on="op")), len(a.merge(b, on="op", how="left")))
```

10. `pd.to_datetime(pd.Series(["03/07/2026"]))` (sem `format`) dá qual data?

- a) 3 de julho de 2026

- b) 7 de março de 2026
- c) erro
- d) depende do Windows estar em português

11. Onde está o problema? O arquivo abre no Excel brasileiro com tudo numa coluna só e os decimais com ponto.

```
df.to_csv("relatorio.csv")
```

12. O que este código imprime?

```
s = pd.Series(["1,5", "n/d", "2"]).str.replace(",", ".")
print(pd.to_numeric(s, errors="coerce").isna().sum())
```

13. Qual gráfico responde melhor "a espessura da linha D está estável ao longo do mês?"

- a) pizza
- b) histograma
- c) gráfico de controle (valores na ordem de produção, média e limites ± 3σ)
- d) barras com a média de cada mês

14. No gráfico de controle do LAM-62, nenhum ponto saiu dos limites de controle, mas 3 bobinas ficaram fora da especificação (62 µm ± 7%). O que isso significa?

- a) O gráfico está errado: se há bobinas fora, o processo está fora de controle.
- b) O processo está estável, mas a variação natural dele é grande demais para a tolerância do cliente.
- c) As 3 bobinas são causas especiais e basta investigá-las.
- d) Os limites de controle deveriam ser iguais aos de especificação.

Gabarito

1. **b.** `thousands="."` remove o ponto de milhar e `decimal=","` trata a vírgula como decimal. Só com `decimal`, o `1.558,5` continua texto.
2. **`[3, 2]`.** A máscara `s > 1` é `[True, False, True]`; o filtro mantém a ordem.
3. **`2 3`.** `iloc` usa posições e exclui o fim (1 e 2); `loc` usa rótulos e inclui o fim (1, 2 e 3).
4. **`and` não funciona com Series** (dá `ValueError: The truth value of a Series is ambiguous`). Use `&` e parênteses: `df[(df["linha"] == "A") & (df["cof"] > 0.30)]`.

5. **Não muda.** É atribuição encadeada: `df[mascara]` gera uma tabela que se comporta como cópia, e o valor é gravado nela. O certo: `df.loc[df["cof"] > 0.35, "status"] = "FORA"`.
6. **`0.75`.** True vale 1 e False vale 0: a média é a proporção de True (3 de 4).
7. **`[2, 1]` `[2, 0]`.** `size()` conta linhas (A tem 2, B tem 1); `count()` conta valores não vazios de `cof` (o de B é `None`).
8. **b.** `how="left"` mantém todas as linhas da tabela da esquerda; quem não casa fica com `NaN` nas colunas da direita. Com `inner`, as medições sem especificação sumiriam.
9. **`2` `3`.** O `inner` só mantém as OPs "1" e "3"; o `left` mantém as três (a "2" fica com `cliente` vazio).
10. **b.** Sem `format`, o pandas assume mês/dia/ano. Use sempre `format="%d/%m/%Y"`.
11. **Faltam os parâmetros brasileiros.** O certo é `df.to_csv("relatorio.csv", sep=";", decimal=",", index=False, encoding="utf-8-sig")` (o `index=False` ainda evita a coluna 0, 1, 2...).
12. **`1`.** `"1,5"` vira `1.5`, `"2"` vira `2.0` e `"n/d"` vira `NaN` (por causa de `errors="coerce"`): um ausente.
13. **c.** Estabilidade ao longo do tempo é exatamente a pergunta do gráfico de controle. O histograma mostra a distribuição, mas perde a ordem no tempo.
14. **b.** Limites de controle são a "voz do processo"; os de especificação, a "voz do cliente". Processo sob controle com peças fora de especificação indica **capacidade** insuficiente: é preciso reduzir a variação (ou renegociar a tolerância), não caçar causa especial.

Módulo 11 — Bancos de dados e SQL (SQLite)

> **Tempo estimado:** 10 a 14 horas · **Pré-requisitos:** módulos 00 a 10 · **Nível:** intermediário

O que você vai aprender

- Explicar tabela, linha, coluna, chave primária e chave estrangeira com exemplos do laboratório.
- Conectar num banco SQLite com `sqlite3`, executar comandos e ler resultados (`fetchone`, `fetchall`, `for`).
- Criar tabelas com tipos e restrições (`NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`, `REFERENCES`) e entender o `IntegrityError`.
- Inserir dados **sempre** com parâmetros (`?` e `:nome`), um por vez ou com `executemany`.
- Escrever `SELECT` com `WHERE`, `ORDER BY`, `LIMIT`, `DISTINCT`, `LIKE`, `IS NULL` e `CASE`.
- Resumir com `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, `GROUP BY` e `HAVING`.
- Juntar tabelas com `INNER JOIN` e `LEFT JOIN` e achar "quem não tem par".
- Usar subconsultas (`IN`, `EXISTS`, escalar) e CTEs (`WITH`).
- Alterar e apagar com `UPDATE` / `DELETE` sem desastre, e agrupar alterações numa transação (`commit`, `rollback`, `with`).
- Demonstrar um ataque de SQL injection e escrever a versão segura.
- Modelar o banco de regras do projeto final (linhas, especificações, tolerâncias).
- Levar o resultado de uma consulta para o pandas com `read_sql`.

Por que isso importa

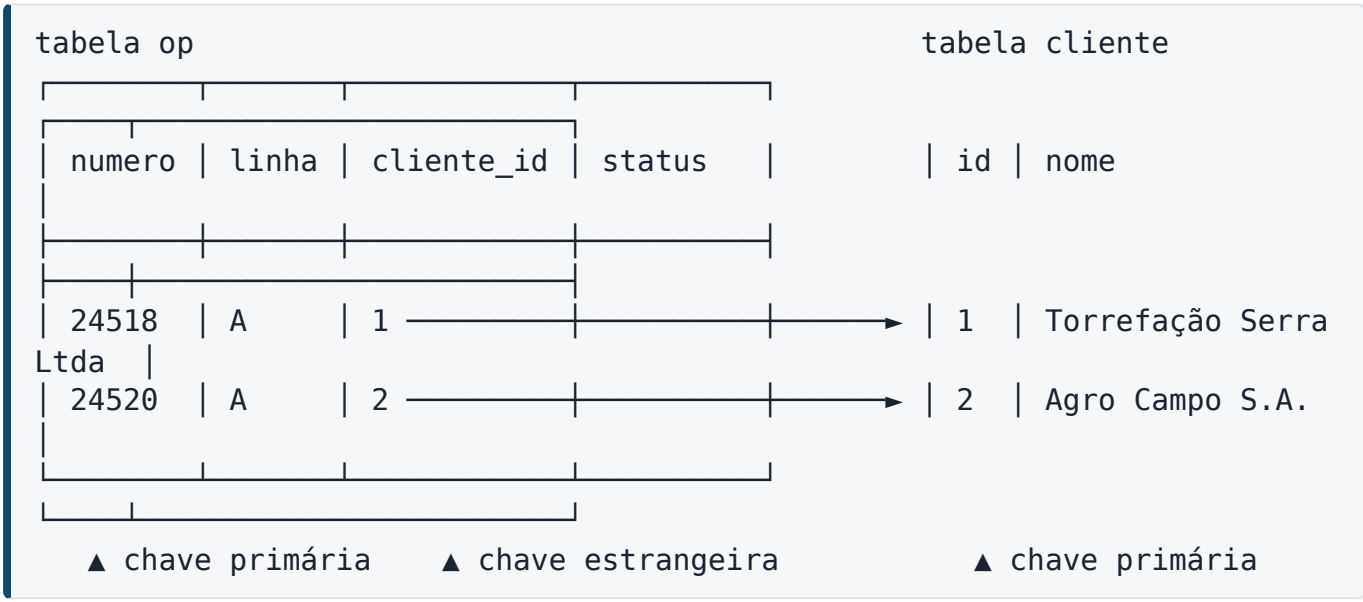
Planilha é ótima para uma pessoa. Quando vários programas e várias pessoas precisam dos mesmos dados, com regras que não podem ser quebradas, entra o **banco de dados**. O ERP TOTVS Protheus que você usa todo dia é, por baixo, um banco de dados relacional enorme.

SQL é a habilidade técnica mais pedida em vagas de dados: analista, engenheiro, BI, analytics engineering. Ela aparece até em vagas de automação e backend. E é uma das mais estáveis: o SQL que você aprende hoje é praticamente o mesmo de 30 anos atrás e vale em SQLite, PostgreSQL, SQL Server, Oracle e BigQuery.

No projeto final (módulo 14), as regras de conferência (padrão de cada linha, especificações de cliente e tolerâncias) ficam num banco SQLite. Na seção 15 você vai entender cada decisão daquele banco.

1. O que é um banco de dados relacional

Um banco relacional guarda dados em **tabelas**. Cada tabela é parecida com uma aba do Excel, com uma diferença importante: as colunas têm **tipo** e **regras**, e o banco obriga todo mundo a respeitá-las.



- **Linha** (registro): uma OP, um cliente, um ensaio.
- **Coluna** (campo): uma informação de todas as linhas, com um tipo.
- **Chave primária** (*primary key*, PK): a coluna que identifica cada linha sem ambiguidade. Não repete e não fica vazia. Na `op`, é o `numero`.
- **Chave estrangeira** (*foreign key*, FK): uma coluna que aponta para a PK de outra tabela. O `cliente_id` da OP aponta para o `id` do cliente. O banco recusa uma OP de um cliente que não existe.

Por que não guardar o **nome** do cliente direto na OP? Porque, se a Torrefação Serra mudar de razão social, você teria que corrigir em 500 OPs (e uma ia escapar). Com a FK, o nome mora num lugar só. Essa ideia (cada informação num lugar, ligada por chaves) se chama **normalização**, e é o "relacional" do nome.

SQL (*Structured Query Language*) é a linguagem para conversar com o banco. Você diz **o que** quer ("as OPs da linha A com mais de 4 t"), e o banco descobre **como** buscar.

SQLite é um banco inteiro num arquivo só (`.db`), sem servidor para instalar, e já vem com o Python (módulo `sqlite3`). Está dentro do seu celular, do navegador e de milhares de programas. Para olhar um `.db` com mouse, instale o **DB Browser for SQLite** (gratuito). Bancos "de servidor", como PostgreSQL e SQL Server, usam o mesmo SQL; muda a forma de conectar.

2. Primeira conexão com sqlite3

O ciclo é sempre: **conectar → executar → (commit) → fechar**. De `exemplos/01_primeira_conexao.py`:

```
import sqlite3

conexao = sqlite3.connect(":memory:")  # banco só na memória; para arquivo: "lab.db"
conexao.execute("CREATE TABLE bobina (id INTEGER PRIMARY KEY, op INTEGER, peso_kg REAL)")
conexao.execute("INSERT INTO bobina (op, peso_kg) VALUES (24518, 1250.5)")
conexao.execute("INSERT INTO bobina (op, peso_kg) VALUES (24518, 980.0)")
conexao.execute("INSERT INTO bobina (op, peso_kg) VALUES (24519, 1100.0)")
conexao.commit()  # grava de verdade

cursor = conexao.execute("SELECT id, op, peso_kg FROM bobina")
print(cursor.fetchall())
conexao.close()
```

```
[(1, 24518, 1250.5), (2, 24518, 980.0), (3, 24519, 1100.0)]
```

- `sqlite3.connect(caminho)` abre o arquivo (e **cria** um vazio se ele não existir). `":memory:"` é um banco temporário, ótimo para testar.
- `conexao.execute(sql)` roda um comando e devolve um **cursor**: um "marcador" que percorre o resultado.
- Cada linha do resultado chega como **tupla**, na ordem das colunas do `SELECT`.
- `commit()` confirma as alterações. Sem ele, outro programa não enxerga os dados, e eles se perdem se o programa terminar.
- O `id INTEGER PRIMARY KEY` foi numerado sozinho: 1, 2, 3.

> 🔮 **Preveja antes de rodar:** o resultado tem duas linhas da OP 24518. O que os três `print` mostram?

```
cursor = conexao.execute("SELECT op, peso_kg FROM bobina WHERE op =
24518")
print(cursor.fetchone())
print(cursor.fetchone())
print(cursor.fetchone())
```

▼ **Resposta**

```
(24518, 1250.5)
(24518, 980.0)
None
```

`fetchone()` devolve a **próxima** linha e avança o marcador. Quando as linhas acabam, devolve `None`. Por isso `fetchone()` é o jeito de perguntar "existe?" (se veio `None`, não existe).

O cursor também pode ser percorrido com `for`, sem carregar tudo na memória de uma vez:

```
for op, peso in conexao.execute("SELECT op, peso_kg FROM bobina"):
    print(op, peso)
```

> 💡 **Dica:** no Windows, `sqlite3.connect("lab.db")` cria o arquivo na pasta **de onde você rodou o comando**, não na pasta do script. Use caminhos com `pathlib` a partir de `Path(__file__)`, como no módulo 06.

3. Criando tabelas: tipos e restrições

```
CREATE TABLE nome_da_tabela (
    coluna1 TIPO RESTRIÇÕES,
    coluna2 TIPO RESTRIÇÕES,
    ...
);
```

Tipos do SQLite: `INTEGER` (inteiro), `REAL` (decimal), `TEXT` (texto), `BLOB` (bytes) e o valor especial `NULL` (ausente). Não há tipo de data: guarde datas como `TEXT` no formato ISO `'AAAA-MM-DD'`, que ordena e compara certo.

As **restrições** são regras de negócio que o banco passa a garantir, venha o dado de onde vier (do seu script, de outro programa, de alguém no DB Browser). De `exemplos/02_criar_tabelas_e_restricoes.py`:

```
conexao.execute("PRAGMA foreign_keys = ON")  # no SQLite, FK só vale com isto!
conexao.executescript(
    """
    CREATE TABLE cliente (
        id   INTEGER PRIMARY KEY,
        nome TEXT NOT NULL UNIQUE
    );
    CREATE TABLE op (
        numero        INTEGER PRIMARY KEY,
        cliente_id    INTEGER NOT NULL REFERENCES cliente (id),
        quantidade_kg REAL    NOT NULL CHECK (quantidade_kg > 0),
        status        TEXT    NOT NULL DEFAULT 'ABERTA'
    );
    """
)
```

Restrição	Significa
`PRIMARY KEY`	identifica a linha: não repete
`NOT NULL`	obrigatório
`UNIQUE`	não repete (ex.: dois clientes com o mesmo nome)
`CHECK (condição)`	a condição precisa ser verdadeira
`DEFAULT valor`	valor usado quando o INSERT não informa a coluna
`REFERENCES tabela (coluna)`	chave estrangeira: o valor precisa existir lá

Tentando quebrar cada regra, o banco responde com `sqlite3.IntegrityError`:

```
NOT NULL     -> IntegrityError: NOT NULL constraint failed: cliente.nome
UNIQUE       -> IntegrityError: UNIQUE constraint failed: cliente.nome
CHECK        -> IntegrityError: CHECK constraint failed: quantidade_kg > 0
FOREIGN KEY  -> IntegrityError: FOREIGN KEY constraint failed
PRIMARY KEY  -> IntegrityError: UNIQUE constraint failed: op.numero
```

`executescript` roda vários comandos separados por `;` (o `execute` só aceita um). `CREATE TABLE IF NOT EXISTS` não dá erro se a tabela já existir, e `DROP TABLE IF EXISTS nome` apaga a tabela. Para ver as colunas de uma tabela: `PRAGMA table_info(op)`.

> ⚠️ **Erro comum:** esquecer o `PRAGMA foreign_keys = ON`. Por compatibilidade antiga, o SQLite vem com as chaves estrangeiras **desligadas**, e a FK vira enfeite. Ligue em **toda** conexão (ele não fica salvo no arquivo). Nossos scripts `.sql` já começam com ele.

> 🔮 **Preveja antes de rodar:** a coluna é `REAL`. O que o SQLite faz com `'12abc'`?

```
conexao.execute("CREATE TABLE teste (valor REAL)")
conexao.execute("INSERT INTO teste VALUES ('12abc'), ('38.5'), (40)")
print(conexao.execute("SELECT valor, typeof(valor) FROM
teste").fetchall())
```

▼ **Resposta**

```
[('12abc', 'text'), (38.5, 'real'), (40.0, 'real')]
```

Ele **aceita** e guarda como texto! O SQLite tem tipagem flexível: o tipo da coluna é uma "preferência" (ele converte `'38.5'` e `40` para real), não uma regra. Para o tipo ser regra de verdade, crie a tabela com `STRICT` no fim (SQLite 3.37+): `CREATE TABLE teste (valor REAL) STRICT` recusa o `'12abc'` com `IntegrityError: cannot store TEXT value in REAL column teste_estrito.valor`. Outra opção é um `CHECK`. Bancos como PostgreSQL já são rígidos por padrão.

4. Inserindo dados com parâmetros

```
INSERT INTO tabela (coluna1, coluna2) VALUES (valor1, valor2);
```

No Python, os valores **nunca** são colados no texto do SQL. Eles vão separados, como **parâmetros**. O `sqlite3` aceita dois estilos (`exemplos/03_inserir_com_parametros.py`):

```
# Posicional: "?" e uma tupla, na mesma ordem
cursor = conexao.execute(
    "INSERT INTO expedicao (op, nf, peso_kg) VALUES (?, ?, ?)", (24518,
"123456", 1250.5)
)
print("id gerado:", cursor.lastrowid)
```

```
# Nomeado: ":nome" e um dicionário
pedido = {"op": 24541, "nf": "123465", "peso": 2000.25}
conexao.execute("INSERT INTO expedicao (op, nf, peso_kg) VALUES (:op,
:nf, :peso)", pedido)

# Muitas linhas: executemany com uma lista de tuplas
cargas = [(24520, "123458", 2400.0), (24521, "123459", 2950.0), (24518,
"123470", 1830.0)]
cursor = conexao.executemany("INSERT INTO expedicao (op, nf, peso_kg)
VALUES (?, ?, ?)", cargas)
print("linhas inseridas:", cursor.rowcount)
conexao.commit()
```

```
id gerado: 1
linhas inseridas: 3
```

Por que parâmetros? Três motivos, do menos ao mais importante:

1. **Comodidade:** o `sqlite3` cuida das aspas, dos números, do `None` (vira `NULL`).
2. **Texto com apóstrofo:** `"Plásticos D'Ávila"` colado no SQL quebra o comando; como parâmetro, funciona.
3. **Segurança:** sem parâmetros, o texto do usuário vira **código** SQL. Isso é o SQL injection, que a seção 13 mostra na prática.

> ⚠️ **Erro comum:** um parâmetro só, sem vírgula. `conexao.execute(sql, 24518)` dá `ProgrammingError: parameters are of unsupported type`, e `(24518)` sem vírgula é só um número entre parênteses. O certo é `(24518,)`. Quantidade errada de valores dá `Incorrect number of bindings supplied. The current statement uses 2, and there are 1 supplied.`

`cursor.lastrowid` é o id que acabou de ser gerado (útil para inserir os "filhos" logo em seguida). `cursor.rowcount` é quantas linhas o comando afetou.

5. O banco do laboratório

A partir daqui, os exemplos usam um banco com cara de laboratório (dados fictícios), definido em `dados/laboratorio.sql`:

```
linha (codigo PK, descricao)
  ▲
  | linha
op (numero PK, linha FK, cliente_id FK, produto, especificacao, emissao,
  |  quantidade_kg, status)              |
```

```
   │                                    ▼
   │                              cliente (id PK, nome, cidade, uf)
   ├─── ensaio (id PK, op FK, bobina, caracteristica, valor, resultado,
data)
   └─── expedicao (id PK, op FK, nf, peso_kg, data)
```

São 4 linhas, 6 clientes, 12 OPs (outubro/2026), 60 ensaios (espessura, gramatura e COF de 2 bobinas por OP; um ainda sem valor) e 8 cargas expedidas. O `status` da OP é `ABERTA`, `LIBERADA`, `BLOQUEADA` ou `CERTIFICADA`; o `resultado` do ensaio é `OK` ou `FORA`.

Para abrir no DB Browser, gere os arquivos `.db` (na pasta `curso-python`):

```
python 11-banco-de-dados-e-sql/dados/montar_banco.py
```

Os exemplos montam o banco na memória com `abrir_banco_exemplo()` (cada exemplo começa do zero) e imprimem os resultados como tabela com `mostrar(conexao, sql)`, ambas em `exemplos/utilitarios.py`. Leia esse arquivo: ele usa `cursor.description`, que traz os nomes das colunas do resultado.

> 💡 **Dica:** escreva as palavras do SQL em MAIÚSCULAS (`SELECT`, `WHERE`) e os nomes em minúsculas. O banco não liga, mas fica muito mais fácil de ler.

6. SELECT: escolher, filtrar, ordenar

A forma geral, na ordem em que se **escreve**:

```sql
SELECT colunas
FROM tabela
WHERE condição
ORDER BY colunas
LIMIT n;
```

De `exemplos/04_select_where_order.py`:

```sql
SELECT numero, produto, quantidade_kg / 1000 AS toneladas
FROM op
WHERE linha = 'A' AND quantidade_kg >= 4500
ORDER BY quantidade_kg DESC
```

```
numero | produto                     | toneladas
-------+-----------------------------+----------
```

```
24521  | Filme PE 60 µm sacaria      | 6
24520  | Filme PE 50 µm branco       | 5.2
24518  | Filme PE 40 µm transparente | 4.5
(3 linhas)
```

- No `SELECT` cabem contas e **apelidos** (`AS toneladas`). `SELECT *` traz todas as colunas (bom para explorar, ruim em programa: se a tabela ganhar uma coluna, seu código quebra).
- Texto vai entre **aspas simples** (`'A'`). Aspas duplas, no SQL, são para nomes.
- `ORDER BY coluna DESC` ordena decrescente (`ASC`, o padrão, é crescente). Várias colunas: `ORDER BY emissao, numero`.
- `LIMIT 3` pega só as 3 primeiras linhas.

Os operadores do `WHERE`:

Operador	Exemplo
`=`, `<>` (diferente), `<`, `<=`, `>`, `>=`	`status <> 'CERTIFICADA'`
`AND`, `OR`, `NOT`	`linha = 'A' AND quantidade_kg > 4000`
`BETWEEN a AND b` (inclui as pontas)	`emissao BETWEEN '2026-10-21' AND '2026-10-22'`
`IN (lista)`	`status IN ('LIBERADA', 'CERTIFICADA')`
`LIKE` (`%` = qualquer coisa, `_` = um caractere)	`produto LIKE '%PE 35%'`
`IS NULL`, `IS NOT NULL`	`valor IS NULL`

```sql
SELECT numero, emissao, status
FROM op
WHERE emissao BETWEEN '2026-10-21' AND '2026-10-22'
  AND status IN ('LIBERADA', 'CERTIFICADA')
ORDER BY emissao, numero
```

```
numero | emissao    | status
-------+------------+------------
24520  | 2026-10-21 | LIBERADA
24521  | 2026-10-21 | LIBERADA
24528  | 2026-10-22 | CERTIFICADA
(3 linhas)
```

`SELECT DISTINCT status FROM op` mostra cada status uma vez só. E `LIKE` no SQLite não diferencia maiúsculas de minúsculas (para letras sem acento).

> 🔮 **Preveja antes de rodar:** existe um ensaio com `valor` NULL (o COF da bobina 2 da OP 24534). Quantas linhas esta consulta devolve?

```
SELECT op, bobina FROM ensaio WHERE valor = NULL
```

<details>
<summary>▼ Resposta</summary>

```
op | bobina
---+-------
(0 linhas)
```

Nenhuma. `NULL` significa "desconhecido", e "desconhecido = desconhecido" também é desconhecido, nunca verdadeiro. Para testar ausência, use `valor IS NULL` (que acha o ensaio pendente da OP 24534). Esse erro roda sem mensagem nenhuma: o resultado simplesmente vem vazio.
</details>

> ⚠️ **Erro comum:** `AND` vem antes de `OR`. `uf = 'SP' OR uf = 'MG' AND cidade = 'Varginha'` significa `uf = 'SP' OR (uf = 'MG' AND cidade = 'Varginha')`. Na dúvida, **parênteses**.

CASE é o `if` do SQL, dentro do `SELECT`:

```
SELECT numero, especificacao,
       CASE WHEN especificacao IS NULL THEN 'padrão da linha'
            ELSE 'especificação do cliente' END AS regra
FROM op
WHERE linha = 'A'
```

```
numero | especificacao | regra
-------+---------------+-------------------------
24518  | NULL          | padrão da linha
24519  | NULL          | padrão da linha
24520  | NULL          | padrão da linha
24521  | ESP-0533      | especificação do cliente
(4 linhas)
```

7. Agregação: GROUP BY e HAVING

Funções de agregação transformam muitas linhas em um número: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` (`exemplos/05_agregacao_group_by.py`):

```sql
SELECT COUNT(*) AS ops, SUM(quantidade_kg) AS total_kg,
       ROUND(AVG(quantidade_kg), 1) AS media_kg,
       MIN(emissao) AS primeira, MAX(emissao) AS ultima
FROM op
```

```
ops | total_kg | media_kg | primeira   | ultima
----+----------+----------+------------+-----------
12  | 45200    | 3766.7   | 2026-10-20 | 2026-10-27
(1 linha)
```

🔮 **Preveja antes de rodar:** há 20 ensaios de COF, e um deles está sem valor. O que sai?

```sql
SELECT COUNT(*) AS linhas, COUNT(valor) AS medidos
FROM ensaio
WHERE caracteristica = 'cof'
```

▼ **Resposta**

```
linhas | medidos
-------+--------
20     | 19
(1 linha)
```

`COUNT(*)` conta **linhas**; `COUNT(coluna)` conta valores **não nulos**. O mesmo vale para `AVG`, `SUM`, `MIN` e `MAX`: todos ignoram NULL. (É a mesma ideia do `size()` x `count()` do pandas.)

GROUP BY faz uma conta por grupo, como o `groupby` do pandas:

```sql
SELECT linha, COUNT(*) AS ops, SUM(quantidade_kg) AS total_kg
FROM op
GROUP BY linha
ORDER BY total_kg DESC
```

```
linha | ops | total_kg
------+-----+---------
A     | 4   | 18700
D     | 3   | 11900
B     | 3   | 7900
C     | 2   | 6700
(4 linhas)
```

Regra prática: no `SELECT` de uma consulta agrupada, só entram as colunas do `GROUP BY` e funções de agregação. Um truque útil: `SUM(CASE WHEN resultado = 'FORA' THEN 1 ELSE 0 END)` conta quantas linhas atendem à condição (no SQLite, até `SUM(resultado = 'FORA')` funciona, porque comparação vale 1 ou 0).

WHERE x HAVING. O `WHERE` filtra **linhas** antes de agrupar; o `HAVING` filtra **grupos** depois:

```sql
SELECT op, COUNT(*) AS cargas, SUM(peso_kg) AS peso_total
FROM expedicao
WHERE data >= '2026-10-26'
GROUP BY op
HAVING COUNT(*) >= 2
ORDER BY op
```

```
op    | cargas | peso_total
------+--------+-----------
24518 | 2      | 3080.5
24541 | 2      | 3750.25
(2 linhas)
```

Para entender o porquê, veja a ordem em que o banco **executa** (diferente da ordem em que escrevemos):

```
1. FROM / JOIN    de onde vêm as linhas
2. WHERE          filtra linhas
3. GROUP BY       forma os grupos
4. HAVING         filtra grupos
5. SELECT         calcula as colunas e apelidos
6. ORDER BY       ordena
7. LIMIT          corta
```

Por isso `WHERE COUNT(*) >= 2` dá erro (`misuse of aggregate function COUNT()`): no passo 2, os grupos ainda não existem.

8. JOINs: juntando tabelas

A OP tem o `cliente_id`; o nome do cliente está em `cliente`. O `JOIN` junta as duas pela condição do `ON` (`exemplos/06_joins.py`):

```sql
SELECT o.numero, c.nome AS cliente, o.produto
FROM op AS o
JOIN cliente AS c ON c.id = o.cliente_id
```

```sql
WHERE o.linha = 'D'
ORDER BY o.numero
```

```
numero | cliente                | produto
-------+------------------------+-----------------------
24541  | Torrefação Serra Ltda | Laminado PET/PE 62 µm
24546  | Laticínios Serra Azul | Laminado BOPP/PE 55 µm
24551  | Agro Campo S.A.       | Laminado PET/PE 62 µm
(3 linhas)
```

`op AS o` dá um apelido curto à tabela; `o.numero` diz de qual tabela vem a coluna (obrigatório quando o nome existe nas duas, senão dá `ambiguous column name`).

```
INNER JOIN (ou só JOIN)     LEFT JOIN
 cliente     op              cliente     op
  ┌──┐     ┌──┐              ┌──┐     ┌──┐
  │ ▒│▇▇▇▇▇│▒ │              │▇▇│▇▇▇▇▇│▒ │
  └──┘     └──┘              └──┘     └──┘
 só o que casa              todos da esquerda; sem par,
 dos dois lados             colunas da direita = NULL
```

O mesmo `merge` do pandas (módulo 10): `how="inner"` e `how="left"`. Com `LEFT JOIN`, os clientes de MG aparecem todos, mesmo o Café do Vale, que não tem OP:

```sql
SELECT c.nome, o.numero
FROM cliente AS c
LEFT JOIN op AS o ON o.cliente_id = c.id
WHERE c.uf = 'MG'
ORDER BY c.nome, o.numero
```

```
nome                  | numero
----------------------+-------
Café do Vale          | NULL
Laticínios Serra Azul | 24527
Laticínios Serra Azul | 24546
(3 linhas)
```

"Quem não tem" é uma pergunta muito comum (OP sem ensaio, cliente sem pedido). A receita é `LEFT JOIN` + `IS NULL` numa coluna da direita (o "anti-join"):

```sql
SELECT o.numero, o.status
FROM op AS o
LEFT JOIN ensaio AS e ON e.op = o.numero
WHERE e.id IS NULL
```

```
numero | status
-------+-------
24550  | ABERTA
24551  | ABERTA
(2 linhas)
```

Dá para encadear vários JOINs e agregar no fim. Peso expedido por cliente (expedição → OP → cliente):

```sql
SELECT c.nome AS cliente, COUNT(x.id) AS cargas, SUM(x.peso_kg) AS
peso_kg
FROM expedicao AS x
JOIN op AS o ON o.numero = x.op
JOIN cliente AS c ON c.id = o.cliente_id
GROUP BY c.nome
ORDER BY peso_kg DESC
```

```
cliente                | cargas | peso_kg
-----------------------+--------+--------
Torrefação Serra Ltda  | 4      | 6830.75
Agro Campo S.A.        | 2      | 5350
Higiene Bem-Estar S.A. | 2      | 2750.5
(3 linhas)
```

🔮 **Preveja antes de rodar:** queremos os clientes de MG e, ao lado, as OPs deles da linha B (ou NULL). O Café do Vale aparece?

```sql
SELECT c.nome, o.numero
FROM cliente AS c
LEFT JOIN op AS o ON o.cliente_id = c.id
WHERE c.uf = 'MG' AND o.linha = 'B'
```

▼ **Resposta**

```
nome                   | numero
-----------------------+-------
Laticínios Serra Azul | 24527
(1 linha)
```

Não aparece. O `WHERE` roda **depois** do `LEFT JOIN`: na linha do Café do Vale, `o.linha` é NULL, e `NULL = 'B'` não é verdadeiro. O `WHERE` jogou fora justamente quem o `LEFT JOIN` tinha guardado. A condição sobre a tabela da direita vai no `ON`:

```
LEFT JOIN op AS o ON o.cliente_id = c.id AND o.linha = 'B'
WHERE c.uf = 'MG'
```

Agora sai `Laticínios Serra Azul | 24527` e `Café do Vale | NULL`. É um dos defeitos do exercício 10.

9. Subconsultas e CTEs

Uma **subconsulta** é um `SELECT` dentro de outro. Leia de dentro para fora (`exemplos/07_subconsultas_e_cte.py`).

Escalar (devolve um valor): OPs acima da média.

```
SELECT numero, quantidade_kg
FROM op
WHERE quantidade_kg > (SELECT AVG(quantidade_kg) FROM op)
ORDER BY quantidade_kg DESC
```

```
numero | quantidade_kg
-------+--------------
24521  | 6000
24520  | 5200
24518  | 4500
24551  | 4100
24541  | 4000
24546  | 3800
(6 linhas)
```

Com IN (devolve uma lista): OPs com algum ensaio FORA.

```
SELECT numero, status
FROM op
WHERE numero IN (SELECT op FROM ensaio WHERE resultado = 'FORA')
```

```
numero | status
-------+----------
24519  | BLOQUEADA
24527  | BLOQUEADA
24533  | BLOQUEADA
24546  | BLOQUEADA
(4 linhas)
```

Com EXISTS / NOT EXISTS (correlacionada: usa a linha de fora). Clientes sem OP:

```sql
SELECT nome
FROM cliente AS c
WHERE NOT EXISTS (SELECT 1 FROM op AS o WHERE o.cliente_id = c.id)
```

Resultado: só o `Café do Vale`.

> ⚠️ **Erro comum:** `WHERE id NOT IN (SELECT cliente_id FROM op)` parece igual, mas se a subconsulta trouxer **um** NULL, o `NOT IN` não devolve **nada** (de novo o "desconhecido"). Prefira `NOT EXISTS`.

CTE (`WITH`) dá nome a um passo intermediário. Consultas grandes ficam legíveis, de cima para baixo, como funções pequenas em Python:

```sql
WITH situacao AS (
    SELECT op,
           SUM(resultado = 'FORA') AS fora,
           SUM(valor IS NULL) AS pendentes
    FROM ensaio
    GROUP BY op
)
SELECT o.numero, o.status, s.fora, s.pendentes
FROM op AS o
JOIN situacao AS s ON s.op = o.numero
WHERE s.fora > 0 OR s.pendentes > 0
ORDER BY o.numero
```

```
numero | status    | fora | pendentes
-------+-----------+------+----------
24519  | BLOQUEADA | 1    | 0
24527  | BLOQUEADA | 1    | 0
24533  | BLOQUEADA | 2    | 0
24534  | ABERTA    | 0    | 1
24546  | BLOQUEADA | 2    | 0
(5 linhas)
```

CTEs aparecem em praticamente todo teste técnico de SQL para vagas de dados. Treine.

> 🔧 **Modifique:** na CTE acima, acrescente `COUNT(*) AS ensaios` e mostre também as OPs sem problema nenhum (tire o `WHERE`). Depois troque o `JOIN situacao` por `LEFT JOIN`: quais OPs aparecem a mais, e com que valores?

10. UPDATE e DELETE

```
UPDATE tabela SET coluna = valor WHERE condição;
DELETE FROM tabela WHERE condição;
```

São os comandos mais perigosos do SQL: **sem `WHERE`, valem para a tabela inteira.** `DELETE FROM ensaio` apaga todos os ensaios, sem perguntar.

O hábito que evita desastre: rode antes um `SELECT` com o **mesmo** `WHERE` e confira o que vai mudar (`exemplos/08_update_delete_transacoes.py`):

```
alvo = "WHERE status = 'ABERTA' AND numero IN (SELECT op FROM ensaio)"
print(banco.execute(f"SELECT numero FROM op {alvo}").fetchall())
cursor = banco.execute(f"UPDATE op SET status = 'LIBERADA' {alvo}")
print("UPDATE alterou", cursor.rowcount, "linha(s)")
```

```
[(24534,)]
UPDATE alterou 1 linha(s)
```

(O f-string aqui é aceitável porque `alvo` é texto **nosso**, fixo no código, nunca vindo do usuário. Valores do usuário vão como parâmetro, sempre.)

`cursor.rowcount` diz quantas linhas o `UPDATE`/`DELETE` afetou. Se você esperava 1 e veio 500, ainda dá para desistir com `rollback()`, antes do `commit()`.

11. Transações: commit, rollback e with

Uma **transação** agrupa comandos que só fazem sentido juntos: ou todos acontecem, ou nenhum. Exemplo do laboratório: certificar uma carga é **gravar a expedição E mudar o status da OP**. Metade disso no banco é um problema real (OP certificada sem carga, ou o contrário).

- `conexao.commit()` confirma tudo desde o último commit.
- `conexao.rollback()` desfaz tudo desde o último commit.
- `with conexao:` faz os dois por você: **commit** se o bloco terminar sem erro, **rollback** se der exceção (e a exceção continua subindo).

> 🔮 **Preveja antes de rodar:** o terceiro comando falha no `CHECK (peso_kg > 0)`. Depois do `except`, qual é o status da OP 24541 (era `LIBERADA`) e quantas cargas com NF `123480` existem?

```
try:
    with banco:
        banco.execute(
            "INSERT INTO expedicao (op, nf, peso_kg, data) VALUES (?, ?,
?, ?)",
            (24541, "123480", 250.0, "2026-10-30"),
        )
        banco.execute("UPDATE op SET status = 'CERTIFICADA' WHERE numero
= ?", (24541,))
        banco.execute(
            "INSERT INTO expedicao (op, nf, peso_kg, data) VALUES (?, ?,
?, ?)",
            (24541, "123481", -1, "2026-10-30"),
        )
except sqlite3.IntegrityError as erro:
    print("Erro, nada foi gravado:", erro)
print(banco.execute("SELECT status FROM op WHERE numero =
24541").fetchone())
print(banco.execute("SELECT COUNT(*) FROM expedicao WHERE nf =
'123480'").fetchone())
```

▼ **Resposta**

```
Erro, nada foi gravado: CHECK constraint failed: peso_kg > 0
('LIBERADA',)
(0,)
```

O `with` desfez **também** os dois comandos que tinham dado certo. É exatamente o "tudo ou nada" que queremos.

⚠️ **Erro comum:** achar que `with conexao:` fecha a conexão, como o `with open(...)` fecha o arquivo. **Não fecha.** Ele só controla a transação. Para fechar automaticamente, use `contextlib.closing`:

```
from contextlib import closing
with closing(sqlite3.connect("lab.db")) as conexao:
    with conexao:
        ...
```

💡 **Dica:** no Python 3.12+ existe o parâmetro `autocommit` em `sqlite3.connect`, que deixa o controle de transações mais explícito. O comportamento padrão (o usado aqui) continua igual e funciona em todas as versões do curso.

Transação também é desempenho. Mil `INSERT` com um `commit` cada são lentos (cada commit grava no disco). Mil `INSERT` num `executemany` dentro de um `with` são quase instantâneos.

12. Linhas com nome: row_factory

Tuplas obrigam a lembrar a posição de cada coluna (`linha[3]` é o quê?). Com `row_factory = sqlite3.Row`, cada linha aceita acesso por **nome** (`exemplos/10_row_factory_e_with.py`):

```
conexao.row_factory = sqlite3.Row
linha = conexao.execute("SELECT numero, produto FROM op WHERE numero =
24541").fetchone()
print(linha["produto"], "|", linha[0], "|", linha.keys())
ops = [dict(r) for r in conexao.execute("SELECT numero, status FROM op
LIMIT 2")]
print(ops)
```

```
Laminado PET/PE 62 µm | 24541 | ['numero', 'produto']
[{'numero': 24518, 'status': 'LIBERADA'}, {'numero': 24519, 'status':
'BLOQUEADA'}]
```

`dict(linha)` vira um dicionário: ótimo para montar dataclasses (módulo 08), JSON ou DataFrames. A `row_factory` vale para a conexão inteira; numa função que recebe a conexão de outra pessoa, prefira não mudá-la (use `cursor.description` para pegar os nomes, como no `utilitarios.py`).

13. SQL injection na prática

Esta é a falha de segurança mais famosa da web, e ainda derruba sistemas todo ano. Ela acontece quando o programa **cola** o texto do usuário dentro do SQL. Veja `exemplos/09_sql_injection.py`:

```
def buscar_cliente_inseguro(conexao, nome):
    """NUNCA faça assim: o texto do usuário vira parte do comando SQL."""
    sql = f"SELECT id, nome, cidade FROM cliente WHERE nome = '{nome}'"
    return conexao.execute(sql).fetchall()
```

Com um nome normal, funciona. Agora veja o que o programa monta com outras entradas:

```
2) Um nome com apóstrofo já quebra a versão insegura:
  SQL montado: SELECT id, nome, cidade FROM cliente WHERE nome =
'Plásticos D'Ávila'
  OperationalError: near "Ávila": syntax error
3) Ataque: ' OR '1'='1  -> devolve TODOS os clientes
  SQL montado: SELECT id, nome, cidade FROM cliente WHERE nome = 'x' OR
'1'='1'
  6 clientes vazaram
4) Ataque com UNION: lê outra tabela inteira
  SQL montado: SELECT id, nome, cidade FROM cliente WHERE nome = 'x'
UNION SELECT numero, produto, status FROM op --'
  12 linhas de OUTRA tabela
```

Leia o ataque 3 devagar: o apóstrofo do usuário **fechou** o texto, e o resto virou código: `OR '1'='1'` é sempre verdadeiro. No ataque 4, o `UNION` grudou uma consulta à tabela `op` e o `--` transformou o apóstrofo final em comentário. Numa tela de login, o ataque 3 entra sem senha; com `UNION`, dá para ler a tabela de usuários.

A correção é a mesma da seção 4: **parâmetros**.

```python
def buscar_cliente_seguro(conexao, nome):
    sql = "SELECT id, nome, cidade FROM cliente WHERE nome = ?"
    return conexao.execute(sql, (nome,)).fetchall()
```

```
5) Versão segura com os mesmos textos:
  'Biscoitos Aurora'                                        -> [(3,
'Biscoitos Aurora', 'Marília')]
  "Plásticos D'Ávila"                                       -> []
  "x' OR '1'='1"                                            -> []
  "x' UNION SELECT numero, produto, status FROM op --"      -> []
```

Com parâmetro, o banco recebe o comando e o valor **separados**: o valor nunca é interpretado como SQL, por mais aspas e palavras-chave que tenha. Ele só procura um cliente com aquele nome estranho (e não acha).

E nomes de coluna? Parâmetros substituem **valores**, nunca nomes de tabela ou coluna. Se o usuário escolhe a ordenação, confira o nome numa **lista permitida** antes de montar o texto:

```python
COLUNAS_PERMITIDAS = {"numero", "emissao", "quantidade_kg"}

def ops_ordenadas(conexao, coluna):
    if coluna not in COLUNAS_PERMITIDAS:
        raise ValueError(f"ordenação não permitida: {coluna!r}")
```

```
    sql = f"SELECT numero FROM op ORDER BY {coluna} LIMIT 3"  # seguro: coluna conferida
    return [numero for (numero,) in conexao.execute(sql)]
```

```
  [24527, 24550, 24528]
  ValueError: ordenação não permitida: 'numero; DROP TABLE op'
```

> 💡 **Dica:** o `execute` do `sqlite3` recusa dois comandos de uma vez (`ProgrammingError: You can only execute one statement at a time.`), o que barra o clássico `'; DROP TABLE op; --`. Mas não conte com isso: o `executescript` aceita vários, outros bancos aceitam, e os ataques 3 e 4 funcionam com um comando só. A defesa é **sempre** parâmetro.

14. Índices

Buscar a OP 24518 numa tabela sem índice obriga o banco a ler **todas** as linhas (um *scan*). Um **índice** é como o sumário de um livro: uma estrutura ordenada que leva direto às linhas certas. A chave primária já tem índice; para outras colunas muito usadas em `WHERE` e `JOIN`, crie você (`exemplos/11_indices.py`, com 300 mil ensaios):

```
conexao.execute("CREATE INDEX idx_ensaio_op ON ensaio (op)")
```

```
Plano sem índice: [(3, 0, 0, 'SCAN ensaio')]
200 buscas sem índice: 2914 ms
Plano com índice: [(3, 0, 0, 'SEARCH ensaio USING COVERING INDEX
idx_ensaio_op (op=?)')]
200 buscas com índice: 0.5 ms
```

`EXPLAIN QUERY PLAN` antes de uma consulta mostra o plano: `SCAN` (lê tudo) ou `SEARCH ... USING INDEX` (vai direto). Os tempos variam por máquina, mas a diferença de milhares de vezes é real. O preço: o índice ocupa espaço e deixa `INSERT`/`UPDATE` um pouco mais lentos. Crie índices para as colunas que você **consulta** muito, não para todas.

15. Modelando o banco de regras do projeto final

Hora de projetar o banco do Validador de Certificados. Os requisitos, do mundo real:

1. Cada linha (A, B, C, D) tem um **padrão**: tolerâncias que valem para toda OP dela.
2. Alguns clientes têm **especificação própria** (título + texto que vai no certificado), que muda **algumas** tolerâncias. O resto continua vindo do padrão da linha.
3. Tolerância pode ser **fixa** (mínimo e/ou máximo: COF de 0,15 a 0,35; tração mínima 20 MPa) ou **nominal ± %** (espessura e gramatura, sobre o nominal que vem da planilha de cada OP).

Exemplo resolvido: do requisito às tabelas. Primeiro eu listo os "substantivos": linha, especificação, tolerância. Cada um vira uma tabela. Depois penso nas relações: uma especificação tem várias tolerâncias (uma por característica), então a tolerância aponta para a especificação. E o padrão da linha? Em vez de uma tabela a mais, ele é **uma especificação como as outras**, só que com a coluna `linha` preenchida. Assim a tabela de tolerâncias serve para os dois casos. O resultado, idêntico ao do projeto final (`dados/regras.sql`):

```
CREATE TABLE linha (
    codigo    TEXT PRIMARY KEY CHECK (codigo IN ('A', 'B', 'C', 'D')),
    descricao TEXT NOT NULL
);

CREATE TABLE especificacao (
    titulo TEXT PRIMARY KEY,                -- 'PADRAO-A', 'ESP-0533'
    texto  TEXT NOT NULL DEFAULT '',        -- vai no certificado
    linha  TEXT REFERENCES linha (codigo)   -- preenchida = PADRÃO da
linha
);

-- Cada linha tem no máximo UM padrão
CREATE UNIQUE INDEX um_padrao_por_linha
    ON especificacao (linha) WHERE linha IS NOT NULL;

CREATE TABLE tolerancia (
    titulo         TEXT NOT NULL REFERENCES especificacao (titulo),
    caracteristica TEXT NOT NULL,           -- espessura, gramatura, cof,
tracao, selagem
    tipo           TEXT NOT NULL CHECK (tipo IN ('fixa', 'nominal')),
    minimo         REAL,                    -- tipo 'fixa'
    maximo         REAL,                    -- tipo 'fixa'
    percentual     REAL,                    -- tipo 'nominal'
    PRIMARY KEY (titulo, caracteristica)
);
```

```
linha (codigo PK) ◄──── especificacao (titulo PK, texto, linha FK ou
NULL)
```

```
                                   ▲
                                   | titulo
                          tolerancia (titulo FK + caracteristica = PK
composta,
                                     tipo, minimo, maximo, percentual)
```

Cada decisão, com o porquê:

- `PRIMARY KEY (titulo, caracteristica)` (chave composta): uma especificação não pode ter duas regras de COF.
- **Índice único parcial** (`WHERE linha IS NOT NULL`): no máximo um padrão por linha, mas quantas especificações de cliente quiser (essas têm `linha` NULL).
- `CHECK (tipo IN (...))`: um erro de digitação (`'nomial'`) é recusado na hora.
- **Não guardamos o nominal**: ele muda a cada OP e vem da planilha. O banco guarda só o percentual. Guardar o que muda junto com o que não muda é a origem de muita inconsistência.

As restrições funcionando (`exemplos/12_banco_de_regras.py`):

```
Segundo padrão para a linha A -> UNIQUE constraint failed:
especificacao.linha
Tipo inválido -> CHECK constraint failed: tipo IN ('fixa', 'nominal')
```

A consulta que importa: quais tolerâncias valem para uma OP da linha A com a especificação ESP-0533? As da especificação, mais as do padrão que a especificação **não** redefine:

```sql
SELECT caracteristica, tipo, minimo, maximo, percentual, titulo AS origem
FROM tolerancia
WHERE titulo = :titulo
UNION ALL
SELECT t.caracteristica, t.tipo, t.minimo, t.maximo, t.percentual,
t.titulo
FROM tolerancia AS t
JOIN especificacao AS e ON e.titulo = t.titulo
WHERE e.linha = :linha
  AND t.caracteristica NOT IN (
      SELECT caracteristica FROM tolerancia WHERE titulo = :titulo
  )
ORDER BY caracteristica
```

```
caracteristica | tipo    | minimo | maximo | percentual | origem
---------------+---------+--------+--------+------------+---------
cof            | fixa    | 0.15   | 0.35   | NULL       | PADRAO-A
espessura      | nominal | NULL   | NULL   | 5          | ESP-0533
gramatura      | nominal | NULL   | NULL   | 10         | PADRAO-A
```

```
selagem         | fixa    | 8      | NULL   | NULL        | PADRAO-A
tracao          | fixa    | 28     | NULL   | NULL        | ESP-0533
(5 linhas)
```

`UNION ALL` empilha os resultados de duas consultas com as mesmas colunas (como o `pd.concat`). Os parâmetros nomeados (`:titulo`, `:linha`) deixam reaproveitar o mesmo valor em vários lugares, passando um dicionário: `{"linha": "A", "titulo": "ESP-0533"}`.

Por fim, a tolerância nominal vira uma faixa no Python, com o nominal da planilha (60 µm ± 5% = 57 a 63 µm). Essa parte é uma função pura, sem banco: fácil de testar. É o exercício 12, e o coração das regras do módulo 14.

> 🔧 **Modifique:** rode a consulta acima para a linha D com a ESP-0412 (o laminado de café). Quais características vêm da especificação e quais do padrão?

16. SQL e pandas: read_sql e to_sql

O trabalho de dados real costuma ser: **SQL** filtra, junta e agrega no banco (onde os dados estão), e o **pandas** recebe a tabela pronta para análise, gráficos e Excel (`exemplos/13_pandas_read_sql.py`):

```python
import pandas as pd

consulta = """
    SELECT o.linha, e.caracteristica, e.valor
    FROM ensaio AS e
    JOIN op AS o ON o.numero = e.op
    WHERE e.caracteristica = ? AND e.valor IS NOT NULL
"""
df = pd.read_sql(consulta, banco, params=("espessura",))
print(df.groupby("linha")["valor"].agg(["count", "mean"]).round(2))
```

```
       count   mean
linha
A          8  48.60
B          4  34.40
C          4  70.62
D          4  58.05
```

Os parâmetros continuam valendo (`params=`). O caminho inverso, `df.to_sql("meta", banco, index=False, if_exists="replace")`, grava um DataFrame como tabela (útil para carregar uma planilha no banco). Para outros bancos (PostgreSQL, SQL Server), o pandas usa a biblioteca SQLAlchemy para conectar; o SQL é o mesmo.

Lendo mensagens de erro deste módulo

Mensagem	O que significa e como corrigir
`sqlite3.OperationalError: no such table: ensaios`	Nome da tabela errado, ou o banco aberto não é o que você pensa (o `connect` criou um vazio num caminho errado). Confira o caminho e `SELECT name FROM sqlite_master`.
`sqlite3.OperationalError: no such column: numro`	Coluna com nome errado (ou de outra tabela sem o `JOIN`). Veja `PRAGMA table_info(tabela)`.
`sqlite3.OperationalError: near "Ávila": syntax error`	SQL montado com f-string e um apóstrofo no meio. Use parâmetros.
`sqlite3.OperationalError: misuse of aggregate function COUNT()`	Agregação no `WHERE`. Use `HAVING` (depois do `GROUP BY`).
`sqlite3.OperationalError: ambiguous column name: id`	A coluna existe nas duas tabelas do JOIN. Use o apelido: `o.id`.
`sqlite3.IntegrityError: UNIQUE constraint failed: op.numero`	Chave repetida. Confira se o registro já existe (ou use `UPDATE`).
`sqlite3.IntegrityError: FOREIGN KEY constraint failed`	O valor da FK não existe na tabela de origem (ex.: cliente 99).
`sqlite3.IntegrityError: CHECK constraint failed: peso_kg > 0`	O dado quebra uma regra do banco. Corrija o dado, não a regra.
`sqlite3.ProgrammingError: Incorrect number of bindings supplied`	Quantidade de `?` diferente da de valores. Um valor só: `(valor,)`.
`sqlite3.ProgrammingError: You can only execute one statement at a time.`	Dois comandos num `execute`. Use dois `execute` (ou `executescript`, só com SQL seu).
`sqlite3.ProgrammingError: Cannot operate on a closed database.`	A conexão já foi fechada. Abra de novo, ou reorganize o `with closing(...)`.
`sqlite3.OperationalError: database is locked`	Outro programa (ou o DB Browser com alterações não gravadas) está escrevendo no arquivo. Grave/feche o outro e tente de novo.

Lembre: a última linha do traceback diz o tipo do erro; suba até a primeira linha que aponta para um arquivo **seu**.

Boas práticas

- **Valores sempre como parâmetros** (`?` ou `:nome`). Nomes de coluna, só de lista permitida.
- `PRAGMA foreign_keys = ON` em toda conexão SQLite.
- **Regras no banco:** `NOT NULL`, `UNIQUE`, `CHECK`, FK. O banco é a última linha de defesa contra dado ruim.
- **Datas em texto ISO** (`'AAAA-MM-DD'`), que ordenam e comparam certo.
- `SELECT` **antes de** `UPDATE` / `DELETE`, com o mesmo `WHERE`; confira o `rowcount`.
- **Operações relacionadas numa transação** (`with conexao:`) e conexão fechada no fim (`closing`).
- **Nada de** `SELECT *` **em programa:** liste as colunas.
- `ORDER BY` **sempre que a ordem importar.** Sem ele, a ordem não é garantida.
- **Scripts** `.sql` **versionados no Git** para criar o banco (como `dados/regras.sql`): o banco vira algo reprodutível.
- **Funções pequenas que recebem a conexão** (como nos exercícios): fáceis de testar com `sqlite3.connect(":memory:")`.

No mundo real

Toda manhã a expedição pergunta: "quais cargas já podem receber certificado, e por que as outras não?". Com SQL, a resposta é **uma consulta** (`exemplos/14_fila_de_certificados.py`), que junta quatro tabelas, resume os ensaios numa CTE e explica a situação de cada carga com `CASE`:

```
WITH situacao_op AS (                        -- passo 1: um resumo por OP
    SELECT op,
           COUNT(*)                  AS ensaios,
           SUM(resultado = 'FORA')   AS fora,
           SUM(valor IS NULL)        AS pendentes
    FROM ensaio
    GROUP BY op
)
SELECT x.op, x.nf, x.peso_kg, c.nome AS cliente,     -- passo 2: cada
carga
       CASE
           WHEN o.status = 'CERTIFICADA'      THEN 'já certificada'
           WHEN s.ensaios IS NULL             THEN 'sem ensaios'
           WHEN s.fora > 0                    THEN 'ensaio fora'
           WHEN s.pendentes > 0               THEN 'ensaio pendente'
```

```
           WHEN o.status <> 'LIBERADA'        THEN 'OP ' ||
LOWER(o.status)
           ELSE 'EMITIR'
       END AS situacao
FROM expedicao AS x
JOIN op AS o ON o.numero = x.op
JOIN cliente AS c ON c.id = o.cliente_id
LEFT JOIN situacao_op AS s ON s.op = x.op
WHERE x.data BETWEEN :inicio AND :fim
ORDER BY situacao = 'EMITIR' DESC, x.op, x.nf
```

O exemplo acrescenta duas cargas pedidas por e-mail (OPs 24533 e 24534) e roda a semana de 25 a 31/10:

```
op    | nf     | peso_kg | cliente                | situacao
------+--------+---------+------------------------+---------------
24518 | 123456 | 1250.5  | Torrefação Serra Ltda  | EMITIR
24518 | 123470 | 1830    | Torrefação Serra Ltda  | EMITIR
24520 | 123458 | 2400    | Agro Campo S.A.        | EMITIR
24521 | 123459 | 2950    | Agro Campo S.A.        | EMITIR
24541 | 123465 | 2000.25 | Torrefação Serra Ltda  | EMITIR
24541 | 123466 | 1750    | Torrefação Serra Ltda  | EMITIR
24528 | 123452 | 1300    | Higiene Bem-Estar S.A. | já certificada
24528 | 123453 | 1450.5  | Higiene Bem-Estar S.A. | já certificada
24533 | 123490 | 1500    | Biscoitos Aurora       | ensaio fora
24534 | 123491 | 800     | Torrefação Serra Ltda  | ensaio pendente
(10 linhas)
```

Repare no `LEFT JOIN` com a CTE (para não perder carga de OP sem ensaio), no `||` (junta textos no SQL) e no `ORDER BY` com uma comparação (as cargas a emitir primeiro). No exercício 14 você faz o passo seguinte: emitir os certificados dessa fila, numerados, numa transação.

Resumo

Tarefa	Sintaxe
Conectar / fechar	`con = sqlite3.connect("lab.db")`, `con.close()`, `closing(...)`
Executar	`cur = con.execute(sql, params)`, `con.executemany(sql, lista)`
Ler	`cur.fetchone()`, `cur.fetchall()`, `for linha in cur:`
Gravar / desfazer	`con.commit()`, `con.rollback()`, `with con:`

Tarefa	Sintaxe
Parâmetros	`"... WHERE op = ?", (op,)` ou `"... WHERE op = :op", {"op": op}`
Criar tabela	`CREATE TABLE IF NOT EXISTS t (id INTEGER PRIMARY KEY, c TEXT NOT NULL, ...)`
Restrições	`NOT NULL`, `UNIQUE`, `CHECK (...)`, `DEFAULT`, `REFERENCES t (c)`
Inserir	`INSERT INTO t (a, b) VALUES (?, ?)`; `cur.lastrowid`
Consultar	`SELECT a, b FROM t WHERE ... ORDER BY a DESC LIMIT 10`
Filtros	`=`, `<>`, `BETWEEN`, `IN (...)`, `LIKE '%x%'`, `IS NULL`, `AND` / `OR`
Agregar	`COUNT(*)`, `COUNT(c)`, `SUM`, `AVG`, `MIN`, `MAX`, `GROUP BY`, `HAVING`
Juntar	`FROM a JOIN b ON b.x = a.y`, `LEFT JOIN`, anti-join com `IS NULL`
Subconsultas	`WHERE x > (SELECT AVG...)`, `IN (SELECT ...)`, `NOT EXISTS (...)`
CTE	`WITH nome AS (SELECT ...) SELECT ... FROM nome`
Alterar / apagar	`UPDATE t SET c = ? WHERE ...`, `DELETE FROM t WHERE ...`; `cur.rowcount`
Condicional	`CASE WHEN ... THEN ... ELSE ... END`
Índice	`CREATE INDEX idx ON t (c)`; `EXPLAIN QUERY PLAN ...`
Linhas com nome	`con.row_factory = sqlite3.Row`; `dict(linha)`
pandas	`pd.read_sql(sql, con, params=(...))`, `df.to_sql("t", con, index=False)`

Você domina este módulo se...

- [ ] Consigo explicar chave primária e chave estrangeira com um exemplo do laboratório, sem consultar.
- [ ] Consigo criar uma tabela com `NOT NULL`, `UNIQUE`, `CHECK` e FK, e dizer qual erro cada uma provoca.
- [ ] Consigo explicar por que `WHERE valor = NULL` não acha nada e qual a forma certa.
- [ ] Consigo explicar a diferença entre `WHERE` e `HAVING` pela ordem de execução.
- [ ] Consigo escrever um `LEFT JOIN` que acha "quem não tem" e explicar por que uma condição no `WHERE` pode desfazer o `LEFT`.
- [ ] Consigo demonstrar um SQL injection com `' OR '1'='1` e corrigir com parâmetros.
- [ ] Consigo agrupar comandos numa transação com `with conexao:` e sei que ele não fecha a conexão.

- [ ] Consigo desenhar o banco de regras do projeto final e justificar cada tabela.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_criar_tabela_bobinas`	★☆☆	`CREATE TABLE` com PK, `NOT NULL`, `CHECK`, `UNIQUE`
02	`ex02_inserir_bobina`	★☆☆	`INSERT` com `?`, `lastrowid`, `commit`
03	`ex03_inserir_varias`	★☆☆	`executemany` numa transação (tudo ou nada)
04	`ex04_buscar_por_op`	★☆☆	`SELECT` com `WHERE` e `ORDER BY`, `fetchone`
05	`ex05_filtros_where`	★☆☆	`BETWEEN`, `IS NULL`, `LIKE`, `DISTINCT`
06	`ex06_agregacoes`	★★☆	`COUNT`, `AVG`, `MIN`, `MAX`, `COALESCE`, filtro opcional
07	`ex07_group_by_having`	★★☆	`GROUP BY`, `HAVING`, ordenação com empate
08	`ex08_joins`	★★☆	`JOIN`, `LEFT JOIN`, anti-join, JOIN encadeado
09	`ex09_subconsulta`	★★☆	subconsulta escalar, `IN`, `NOT EXISTS`
10	`ex10_corrija_as_consultas`	★★☆	**leitura de código e depuração**: 3 armadilhas clássicas
11	`ex11_busca_segura`	★★★	filtros dinâmicos sem SQL injection, lista permitida
12	`ex12_banco_de_regras`	★★★	banco de regras do projeto: padrão + especificação, nominal ± %
13	`ex13_revisao_importar_csv`	★★☆	**revisão**: CSV (mód. 06), números BR (mód. 05), datas (mód. 07) + INSERT
14	`ex14_revisao_emitir_certificados`	★★★	**revisão**: dataclass (mód. 08), datas (mód. 07) + transação

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError`, e rode, na pasta `curso-python`:

```
python curso.py testar 11 ex01
```

Cada teste cria um banco novo **na memória** (`sqlite3.connect(":memory:")`), monta as tabelas (a partir de `dados/laboratorio.sql` ou `dados/regras.sql`, quando o enunciado diz "banco do laboratório" ou "banco de regras") e passa a conexão para a sua função. Não altere os arquivos `.sql` da pasta `dados/`: os testes dependem deles.

Desafio do módulo

Ferramenta do banco de regras: um programa de linha de comando com subcomandos para criar o banco de regras, listar as especificações, mostrar as tolerâncias efetivas de uma OP (com as faixas calculadas a partir do nominal) e conferir valores medidos, devolvendo APROVADA ou REPROVADA com os motivos. Enunciado completo em desafio/README.md.

Quiz

Teste o que ficou com o quiz do módulo (14 perguntas, gabarito no final).

Para ir além

- Documentação oficial do `sqlite3` em português: https://docs.python.org/pt-br/3/library/sqlite3.html
- SQL do SQLite, comando por comando: https://www.sqlite.org/lang.html
- Tipagem flexível e tabelas STRICT no SQLite: https://www.sqlite.org/stricttables.html
- SQL injection explicado pela OWASP: https://owasp.org/www-community/attacks/SQL_Injection
- DB Browser for SQLite (para ver e editar os `.db`): https://sqlitebrowser.org
- Para treinar SQL com exercícios corrigidos na hora: SQLBolt (https://sqlbolt.com) e, depois, funções de janela (`ROW_NUMBER`, `LAG`), o próximo passo pedido em vagas de dados: https://www.sqlite.org/windowfunctions.html

Enunciados dos exercícios

Exercício 11.01 — Criar a tabela de bobinas

Arquivo: `exercicios/ex01_criar_tabela_bobinas.py`

```
Nível: ★☆☆

Contexto: o laboratório vai registrar cada bobina pesada num banco
SQLite, em
vez de uma planilha. Antes de inserir qualquer coisa, a tabela precisa
existir,
com regras que impeçam dados absurdos.

Tarefa: implemente criar_tabela_bobinas(conexao), que cria a tabela
"bobina"
com as colunas, nesta ordem:
    id       inteiro, chave primária (o SQLite numera sozinho)
    op       inteiro, obrigatório (NOT NULL)
    numero   inteiro, obrigatório (o número da bobina dentro da OP)
    peso_kg  real, obrigatório e maior que zero (CHECK)
e com a regra: não pode haver duas bobinas com o mesmo par (op, numero).
Chamar a função duas vezes não pode dar erro (use IF NOT EXISTS). Grave
com
commit.

Exemplo:
    >>> conexao = sqlite3.connect(":memory:")
    >>> criar_tabela_bobinas(conexao)
    >>> conexao.execute("INSERT INTO bobina (op, numero, peso_kg) VALUES
(24518, 1, 1250.5)")
    >>> conexao.execute("INSERT INTO bobina (op, numero, peso_kg) VALUES
(24518, 1, 980)")
    Traceback (most recent call last):
    sqlite3.IntegrityError: UNIQUE constraint failed: bobina.op,
bobina.numero

Dica: CREATE TABLE IF NOT EXISTS nome (coluna TIPO RESTRIÇÕES, ...,
UNIQUE (a, b)).

Teste com:  python curso.py testar 11 ex01
```

Exercício 11.02 — Inserir com parâmetros

Arquivo: `exercicios/ex02_inserir_bobina.py`

```
Nível: ★☆☆

Contexto: a balança da expedição manda OP, número da bobina e peso. Seu
programa grava cada leitura no banco.

Tarefa: a tabela já existe:
    CREATE TABLE bobina (
        id INTEGER PRIMARY KEY, op INTEGER NOT NULL, numero INTEGER NOT
```

```
NULL,
        peso_kg REAL NOT NULL CHECK (peso_kg > 0), UNIQUE (op, numero))
Implemente duas funções:
  1. inserir_bobina(conexao, op, numero, peso_kg): insere a bobina usando
     PARÂMETROS (?), faz commit e devolve o id gerado. Se o banco recusar
     (peso <= 0, bobina repetida), deixe o sqlite3.IntegrityError subir.
  2. contar_bobinas(conexao, op): quantas bobinas a OP tem (0 se
nenhuma).

Exemplos:
    >>> inserir_bobina(conexao, 24518, 1, 1250.5)
    1
    >>> inserir_bobina(conexao, 24518, 2, 980.0)
    2
    >>> contar_bobinas(conexao, 24518)
    2

Dica: cursor = conexao.execute(sql, (a, b, c)); cursor.lastrowid. Para um
valor
só: (op,) com vírgula. fetchone() devolve uma tupla: pegue o [0].

Teste com:  python curso.py testar 11 ex02
```

Exercício 11.03 — Inserir vários ensaios de uma vez (tudo ou nada)

Arquivo: `exercicios/ex03_inserir_varias.py`

```
Nível: ★☆☆

Contexto: ao terminar uma OP, o laboratório lança todos os ensaios de uma
vez.
Se UM deles estiver errado, nenhum deve ser gravado: lote pela metade no
banco
é pior do que lote nenhum.

Tarefa: a tabela já existe:
    CREATE TABLE ensaio (
        id INTEGER PRIMARY KEY, op INTEGER NOT NULL, bobina INTEGER NOT
NULL,
        caracteristica TEXT NOT NULL, valor REAL)
Implemente inserir_ensaios(conexao, ensaios), em que ensaios é uma lista
de
tuplas (op, bobina, caracteristica, valor). Insira todas numa transação
só
(executemany dentro de "with conexao:") e devolva quantas foram
inseridas.
Lista vazia -> 0. Se alguma for recusada pelo banco, o
sqlite3.IntegrityError
deve subir e NENHUMA linha do lote pode ficar gravada.
```

```
Exemplo:
    >>> inserir_ensaios(conexao, [(24518, 1, "espessura", 38.8),
    ...                           (24518, 1, "cof", 0.31)])
    2

Dica: "with conexao:" faz commit no fim do bloco, ou rollback se der
erro.

Teste com:  python curso.py testar 11 ex03
```

Exercício 11.04 — Buscar os ensaios de uma OP

Arquivo: `exercicios/ex04_buscar_por_op.py`

```
Nível: ★☆☆

Contexto: na hora de emitir o certificado, o sistema precisa listar os
ensaios
da OP, numa ordem previsível, e saber se a OP existe.

Tarefa: use o banco do laboratório (tabelas op e ensaio, veja a aula ou
dados/laboratorio.sql). Implemente duas funções, sempre com parâmetros
(?):
  1. ensaios_da_op(conexao, op): lista de tuplas (bobina, caracteristica,
valor)
     da OP, ordenada por bobina e, dentro da bobina, por caracteristica
(ordem
     alfabética). OP sem ensaios (ou inexistente) -> lista vazia.
  2. existe_op(conexao, numero): True se a OP está na tabela op, senão
False.

Exemplos:
    >>> ensaios_da_op(conexao, 24534)[:2]
    [(1, 'cof', 0.23), (1, 'espessura', 69.8)]
    >>> existe_op(conexao, 24534), existe_op(conexao, 99999)
    (True, False)

Dica: ORDER BY a, b. fetchone() devolve None quando não há linha.

Teste com:  python curso.py testar 11 ex04
```

Exercício 11.05 — Filtros com WHERE

Arquivo: `exercicios/ex05_filtros_where.py`

```
Nível: ★☆☆

Contexto: o PCP pede listas rápidas: as OPs de uma linha num período, os
ensaios que ainda não foram medidos e os produtos que contêm um trecho no
nome.

Tarefa: use o banco do laboratório. Implemente três funções (com
parâmetros):
  1. ops_da_linha_no_periodo(conexao, linha, inicio, fim): números das
OPs da
     linha com emissão entre inicio e fim (datas 'AAAA-MM-DD',
inclusive),
     ordenados por emissão e depois por número.
  2. ensaios_pendentes(conexao): tuplas (op, bobina, caracteristica) dos
ensaios
     sem valor, ordenadas por op, bobina, caracteristica.
  3. buscar_produtos(conexao, trecho): nomes de produto, SEM repetição e
em
     ordem alfabética, que contêm o trecho (maiúsculas e minúsculas tanto
faz).

Exemplos:
    >>> ops_da_linha_no_periodo(conexao, "A", "2026-10-20", "2026-10-20")
    [24518, 24519]
    >>> ensaios_pendentes(conexao)
    [(24534, 2, 'cof')]
    >>> buscar_produtos(conexao, "laminado")
    ['Laminado BOPP/PE 55 µm', 'Laminado PET/PE 62 µm']

Dica: BETWEEN, IS NULL (nunca "= NULL"), LIKE com '%' || ? || '%',
DISTINCT.

Teste com:  python curso.py testar 11 ex05
```

Exercício 11.06 — Funções de agregação

Arquivo: `exercicios/ex06_agregacoes.py`

```
Nível: ★★☆

Contexto: o gerente da qualidade quer números rápidos: quantas medições
de uma
característica existem, a média, o mínimo e o máximo; e o peso já
expedido.

Tarefa: use o banco do laboratório. Implemente duas funções:
  1. resumo_caracteristica(conexao, caracteristica): devolve o dicionário
       {"n": quantidade de valores medidos (ignore os NULL),
```

```
        "media": média com 2 casas (arredonde no SQL),
        "minimo": menor valor, "maximo": maior valor}
     Característica sem medições -> {"n": 0, "media": None, "minimo":
None,
     "maximo": None}.
  2. peso_expedido(conexao, op=None): soma do peso_kg da tabela
expedicao; se op
     for informada, só dessa OP. Sem nenhuma carga -> 0.0 (nunca None).

Exemplos:
    >>> resumo_caracteristica(conexao, "cof")
    {'n': 19, 'media': 0.25, 'minimo': 0.15, 'maximo': 0.43}
    >>> peso_expedido(conexao, 24518)
    3080.5
    >>> peso_expedido(conexao, 24550)
    0.0

Dica: COUNT(coluna) ignora NULL; ROUND(AVG(x), 2); SUM de nada dá NULL,
mas
COALESCE(SUM(x), 0) ou TOTAL(x) dão 0. Uma consulta com agregação e sem
GROUP
BY devolve sempre UMA linha.

Teste com:  python curso.py testar 11 ex06
```

Exercício 11.07 — GROUP BY e HAVING

Arquivo: `exercicios/ex07_group_by_having.py`

```
Nível: ★★☆

Contexto: na reunião de qualidade, duas perguntas: "quais características
mais
reprovam?" e "quais OPs saíram em várias cargas?".

Tarefa: use o banco do laboratório. Implemente duas funções:
  1. fora_por_caracteristica(conexao): tuplas (caracteristica,
quantidade) com
     quantos ensaios deram resultado 'FORA' em cada característica. Só
entram
     as que têm pelo menos 1. Ordem: quantidade decrescente; empate,
ordem
     alfabética.
  2. ops_com_varias_cargas(conexao, minimo): tuplas (op, cargas,
peso_total) da
     tabela expedicao, só das OPs com pelo menos `minimo` cargas,
ordenadas por op.

Exemplos:
```

```
    >>> fora_por_caracteristica(conexao)
    [('espessura', 3), ('cof', 2), ('gramatura', 1)]
    >>> ops_com_varias_cargas(conexao, 2)[0]
    (24518, 2, 3080.5)

Dica: WHERE filtra linhas ANTES de agrupar; HAVING filtra grupos DEPOIS.

Teste com:  python curso.py testar 11 ex07
```

Exercício 11.08 — JOINs: juntando OPs, clientes e expedição

Arquivo: `exercicios/ex08_joins.py`

```
Nível: ★★☆

Contexto: as informações estão espalhadas: a OP tem o cliente_id, o nome
do
cliente está na tabela cliente e as cargas estão na expedicao. Relatório
bom
junta tudo.

Tarefa: use o banco do laboratório. Implemente três funções:
  1. relatorio_ops(conexao, linha): lista de dicionários
     {"numero": ..., "cliente": nome do cliente, "produto": ...} das OPs
da
     linha, ordenada por número.
  2. ops_sem_ensaio(conexao): números das OPs que não têm nenhum ensaio,
     em ordem crescente.
  3. peso_por_cliente(conexao): tuplas (nome do cliente, peso total
expedido)
     de TODOS os clientes (quem não expediu nada aparece com 0.0),
ordenadas
     por peso decrescente e, no empate, por nome.

Exemplos:
    >>> relatorio_ops(conexao, "D")[0]
    {'numero': 24541, 'cliente': 'Torrefação Serra Ltda', 'produto':
'Laminado PET/PE 62 µm'}
    >>> ops_sem_ensaio(conexao)
    [24550, 24551]
    >>> peso_por_cliente(conexao)[:2]
    [('Torrefação Serra Ltda', 6830.75), ('Agro Campo S.A.', 5350.0)]

Dica: JOIN ... ON; LEFT JOIN + "IS NULL" acha quem não tem par; dá para
encadear dois LEFT JOIN (cliente -> op -> expedicao); COALESCE(SUM(...),
0).

Teste com:  python curso.py testar 11 ex08
```

Exercício 11.09 — Subconsultas

Arquivo: `exercicios/ex09_subconsulta.py`

```
Nível: ★★☆

Contexto: algumas perguntas dependem de outra pergunta: "quais OPs são
maiores
que a média?" precisa primeiro da média; "quais OPs tiveram reprovação?"
precisa primeiro da lista de ensaios FORA.

Tarefa: use o banco do laboratório. Resolva com subconsultas (SELECT
dentro de
SELECT). Implemente três funções:
  1. ops_acima_da_media(conexao): tuplas (numero, quantidade_kg) das OPs
com
     quantidade acima da média de todas as OPs, da maior para a menor.
  2. ops_com_reprovacao(conexao): números das OPs com pelo menos um
ensaio
     'FORA', em ordem crescente.
  3. clientes_sem_op(conexao): nomes dos clientes sem nenhuma OP, em
ordem
     alfabética.

Exemplos:
    >>> ops_acima_da_media(conexao)[:2]
    [(24521, 6000.0), (24520, 5200.0)]
    >>> ops_com_reprovacao(conexao)
    [24519, 24527, 24533, 24546]
    >>> clientes_sem_op(conexao)
    ['Café do Vale']

Dica: WHERE x > (SELECT AVG(...) ...); WHERE numero IN (SELECT ...);
WHERE NOT EXISTS (SELECT 1 FROM ... WHERE ...).

Teste com:  python curso.py testar 11 ex09
```

Exercício 11.10 — Corrija as consultas (leitura de código e depuração)

Arquivo: `exercicios/ex10_corrija_as_consultas.py`

```
Nível: ★★☆

Contexto: um colega escreveu três consultas para o painel do laboratório.
Todas
RODAM sem erro, mas devolvem resultados errados. Leia cada uma e descubra
```

```
o
defeito antes de rodar qualquer coisa.

    # (1) Ensaios ainda sem valor
    SELECT op, bobina, caracteristica
    FROM ensaio
    WHERE valor = NULL
    ORDER BY op, bobina, caracteristica

    # (2) Quantos ensaios cada OP tem (inclusive as que não têm nenhum)
    SELECT o.numero, COUNT(*)
    FROM op AS o
    LEFT JOIN ensaio AS e ON e.op = o.numero
    GROUP BY o.numero
    ORDER BY o.numero

    # (3) TODOS os clientes, cada um com as suas OPs de uma linha (ou
None)
    SELECT c.nome, o.numero
    FROM cliente AS c
    LEFT JOIN op AS o ON o.cliente_id = c.id
    WHERE o.linha = ?
    ORDER BY c.nome, o.numero

Tarefa: escreva as versões corretas (use o banco do laboratório):
  1. ensaios_sem_valor(conexao) -> lista de (op, bobina, caracteristica).
  2. ensaios_por_op(conexao) -> lista de (numero, quantidade) de TODAS as
OPs;
     OP sem ensaio tem quantidade 0.
  3. clientes_e_ops_da_linha(conexao, linha) -> lista de (nome, numero)
com
     TODOS os clientes; quem não tem OP na linha aparece uma vez, com
None.
     Ordem: nome e depois número.

Exemplos:
    >>> ensaios_sem_valor(conexao)
    [(24534, 2, 'cof')]
    >>> ensaios_por_op(conexao)[-1]
    (24551, 0)
    >>> clientes_e_ops_da_linha(conexao, "B")[:2]
    [('Agro Campo S.A.', None), ('Biscoitos Aurora', None)]

Dica: (1) como se compara com NULL? (2) o que COUNT(*) conta numa linha
que o
LEFT JOIN completou com NULL? (3) o que o WHERE faz com as linhas em que
o.linha é NULL? Onde mais dá para pôr essa condição?

Teste com:  python curso.py testar 11 ex10
```

Exercício 11.11 — Busca com filtros, à prova de SQL injection

Arquivo: `exercicios/ex11_busca_segura.py`

```
Nível: ★★★

Contexto: a tela de consulta de OPs tem campos opcionais (linha, produto,
status, especificação) e um seletor "ordenar por". O que o usuário digita
NÃO
pode virar código SQL. Um colega sugeriu montar tudo com f-string... não!

Tarefa: implemente buscar_ops(conexao, filtros, ordenar_por="numero"),
que
devolve a lista dos NÚMEROS das OPs (tabela op do banco do laboratório):
  - filtros é um dicionário {coluna: valor}. Colunas permitidas: "linha",
    "produto", "status", "especificacao". Cada filtro é uma igualdade, e
todos
    se combinam com AND. Dicionário vazio -> todas as OPs.
  - Os VALORES entram sempre como parâmetros (?), nunca colados no SQL.
  - Coluna de filtro fora da lista -> ValueError.
  - ordenar_por: "numero", "emissao" ou "quantidade_kg" (crescente; no
empate,
    por número). Qualquer outro -> ValueError.

Exemplos:
    >>> buscar_ops(conexao, {"linha": "A", "status": "LIBERADA"})
    [24518, 24520, 24521]
    >>> buscar_ops(conexao, {"linha": "B"}, ordenar_por="quantidade_kg")
    [24527, 24550, 24528]
    >>> buscar_ops(conexao, {"linha": "A' OR '1'='1"})
    []
    >>> buscar_ops(conexao, {}, ordenar_por="numero; DROP TABLE op")
    Traceback (most recent call last):
    ValueError: ...

Dica: NOMES de coluna não podem ser parâmetros (só valores). Por isso
confira
os nomes numa lista permitida e monte só essa parte do texto; os valores
vão
numa lista de parâmetros, na mesma ordem dos "?".

Teste com:  python curso.py testar 11 ex11
```

Exercício 11.12 — O banco de regras do projeto final

Arquivo: `exercicios/ex12_banco_de_regras.py`

```
Nível: ★★★

Contexto: no Validador de Certificados (módulo 14), as regras ficam num
banco
SQLite com três tabelas (veja dados/regras.sql e a seção da aula):
    linha(codigo, descricao)
    especificacao(titulo, texto, linha)  -- linha preenchida = PADRÃO da
linha;
                                          -- linha NULL = especificação de
cliente
    tolerancia(titulo, caracteristica, tipo, minimo, maximo, percentual)
                                          -- tipo 'fixa' usa
minimo/maximo;
                                          -- tipo 'nominal' usa percentual
A OP traz a linha e, às vezes, o título de uma especificação de cliente.
As
tolerâncias que valem são as do padrão da linha, trocadas pelas da
especificação do cliente nas características que ela define.

Tarefa: implemente três funções.
  1. especificacao_da_op(conexao, linha, titulo=None): devolve (titulo,
texto).
     Título vazio ou None -> o padrão da linha. Título informado -> essa
     especificação, comparando sem diferenciar maiúsculas e ignorando
espaços
     nas pontas (" esp-0533 " acha "ESP-0533"). Não encontrou (título
     desconhecido ou linha sem padrão) -> LookupError.
  2. tolerancias_efetivas(conexao, linha, titulo=None): dicionário
     {caracteristica: {"tipo", "minimo", "maximo", "percentual",
"origem"}}, em
     que "origem" é o título de onde a tolerância veio. Começa pelas do
padrão
     da linha e sobrescreve com as da especificação do cliente (se
houver).
     Erros como na função 1.
  3. limites(tolerancia, nominal=None): devolve (minimo, maximo).
     tipo 'fixa' -> (minimo, maximo) da própria tolerância (podem ser
None).
     tipo 'nominal' -> nominal ± percentual%, arredondado em 4 casas;
nominal
     None ou <= 0 -> ValueError.

Exemplos:
    >>> especificacao_da_op(conexao, "A")
    ('PADRAO-A', 'Padrão da linha A (filmes PE monocamada).')
    >>> efetivas = tolerancias_efetivas(conexao, "A", "ESP-0533")
    >>> efetivas["espessura"]
    {'tipo': 'nominal', 'minimo': None, 'maximo': None, 'percentual':
5.0,
```

```
     'origem': 'ESP-0533'}
    >>> limites(efetivas["espessura"], nominal=60)
    (57.0, 63.0)
    >>> limites(efetivas["cof"])
    (0.15, 0.35)

Dica: UPPER(TRIM(titulo)) = UPPER(TRIM(?)). Para as efetivas, duas
consultas
(padrão e cliente) e um dicionário que a segunda atualiza já resolvem; a
aula
mostra também a versão numa consulta só, com UNION ALL.

Teste com:  python curso.py testar 11 ex12
```

Exercício 11.13 — Revisão: importar a planilha da expedição para o banco

Arquivo: `exercicios/ex13_revisao_importar_csv.py`

```
Nível: ★★☆

Revisão intercalada: CSV com ponto e vírgula e exceções (módulo 06),
números
brasileiros (módulo 05), datas com datetime (módulo 07), conjuntos
(módulo 03)
e INSERT em transação (este módulo).

Contexto: a expedição mantém um CSV (salvo pelo Excel) com as cargas do
dia.
Você vai importar esse arquivo para a tabela expedicao do banco do
laboratório:
    expedicao(id, op, nf, peso_kg, data)   -- data no formato 'AAAA-MM-
DD'

Formato do arquivo (UTF-8 com BOM, separador ";", cabeçalho na linha 1):
    op;nf;peso;data
    24518;123480;1.250,5;30/10/2026

Tarefa: implemente importar_expedicao(conexao, caminho), que devolve a
tupla
(quantidade_inserida, linhas_com_erro). Uma linha do arquivo é válida
quando:
  - op é um número inteiro de uma OP que EXISTE na tabela op;
  - nf não está vazia (guarde como texto, sem espaços nas pontas);
  - peso, no formato brasileiro ("1.250,5"), é maior que zero;
  - data é uma data válida no formato dd/mm/aaaa (grave como 'AAAA-MM-
DD').
Linhas inválidas são puladas e o NÚMERO da linha no arquivo vai para a
```

```
lista
de erros (o cabeçalho é a linha 1, então o primeiro dado é a linha 2). As
válidas são gravadas todas juntas, numa transação.

Exemplo: com o arquivo acima mais a linha "99999;123481;100,0;30/10/2026"
    >>> importar_expedicao(conexao, "expedicao.csv")
    (1, [3])

Dica: csv.DictReader(arquivo, delimiter=";"); enumerate(leitor, start=2);
datetime.strptime(texto, "%d/%m/%Y").date().isoformat(); pegue os números
das
OPs uma vez só, num set, antes do laço.

Teste com:  python curso.py testar 11 ex13
```

Exercício 11.14 — Revisão: emitir certificados numa transação

Arquivo: `exercicios/ex14_revisao_emitir_certificados.py`

```
Nível: ★★★

Revisão intercalada: dataclass (módulo 08), datas (módulo 07), coleções e
funções (módulos 03 e 04) e transações com sqlite3 (este módulo).

Contexto: cada carga da expedição precisa de um certificado de qualidade.
O
banco do laboratório ganhou uma tabela nova:
    CREATE TABLE certificado (
        id           INTEGER PRIMARY KEY,
        numero       TEXT NOT NULL UNIQUE,               -- 'CQ-2026-
0001'
        expedicao_id INTEGER NOT NULL UNIQUE REFERENCES expedicao (id),
        emitido_em   TEXT NOT NULL                       -- 'AAAA-MM-DD'
    )

Tarefa: implemente emitir_certificados(conexao, data), que emite os
certificados pendentes e devolve a lista de Certificado (dataclass
abaixo, já
pronta). Uma carga (linha de expedicao) recebe certificado quando:
  - ainda não tem certificado;
  - a OP dela está com status 'LIBERADA';
  - a OP tem pelo menos um ensaio e NENHUM ensaio com resultado 'FORA' ou
    com valor NULL.
Regras:
  - cargas processadas em ordem de expedicao.id;
  - número "CQ-AAAA-NNNN", com AAAA = ano de `data` e NNNN = sequência de
4
    dígitos que CONTINUA a partir do maior número já existente daquele
ano
```

```
    (sem nenhum no ano, começa em 0001);
  - emitido_em = data no formato 'AAAA-MM-DD';
  - tudo numa transação: se qualquer INSERT falhar, nenhum certificado
fica
    gravado e o erro sobe (sqlite3.IntegrityError, por exemplo).
Rodar de novo logo depois não emite nada (devolve lista vazia).

Exemplo (banco do laboratório):
    >>> emitidos = emitir_certificados(conexao, date(2026, 10, 30))
    >>> len(emitidos), emitidos[0]
    (6, Certificado(numero='CQ-2026-0001', op=24518, nf='123456',
peso_kg=1250.5,
                    emitido_em='2026-10-30'))

Dica: um SELECT com JOIN e NOT EXISTS acha as cargas; para a sequência,
SELECT MAX(numero) ... WHERE numero LIKE 'CQ-2026-%' e int(numero[-4:]);
f"{n:04d}" escreve 7 como "0007".

Teste com:  python curso.py testar 11 ex14
```

Desafio do módulo 11 — Ferramenta do banco de regras

Contexto

No projeto final, as regras de conferência (padrão de cada linha, especificações de cliente e tolerâncias) moram num banco SQLite. Hoje, quem quer saber "qual é a faixa de espessura da OP da linha A com a ESP-0533?" abre o DB Browser e escreve SQL na mão. Você vai criar uma ferramenta de linha de comando que responde isso, e confere valores medidos, em um comando.

Requisitos obrigatórios

1. Linha de comando com `argparse` e subcomandos (`add_subparsers`, módulo 07), com a opção geral `--banco` (padrão: `11-banco-de-dados-e-sql/saida/regras.db`).
2. `criar`: cria o banco a partir de `dados/regras.sql`. Se o arquivo já existir, recusa, a não ser que venha `--forcar`.
3. `listar`: mostra cada especificação, a linha (ou `-` para especificação de cliente) e quantas tolerâncias ela tem. Padrões primeiro.
4. `regras --linha X [--especificacao TITULO] [--nominal carac=valor ...]`: mostra as tolerâncias **efetivas** (padrão da linha + especificação do cliente por cima), com a origem de cada uma. Para as do tipo nominal, calcula a faixa se o nominal for informado; senão mostra "nominal ± N%".

5. `conferir --linha X [--especificacao TITULO] --valor carac=valor ... [--nominal ...]`: imprime `APROVADA` ou `REPROVADA` com os motivos. Código de saída 0 se aprovada e 2 se reprovada (útil para outros scripts).
6. Todo valor vindo do usuário entra nas consultas como **parâmetro** (`?` ou `:nome`).
7. Título da especificação sem diferenciar maiúsculas e sem espaços nas pontas.
8. Erros (banco inexistente, especificação desconhecida, `nome=valor` mal escrito): mensagem clara em português e código de saída 1, sem traceback. Atenção: `sqlite3.connect("caminho_errado.db")` **cria** um banco vazio sem avisar; confira se o arquivo existe antes.
9. Sem argumentos, roda uma demonstração num banco na memória.

Extras opcionais

- `adicionar-especificacao --titulo ESP-0700 --texto "..."` e `adicionar-tolerancia --titulo ESP-0700 --caracteristica cof --tipo fixa --minimo 0.1 --maximo 0.2`, com as restrições do banco protegendo os dados (trate o `IntegrityError`).
- `exportar --saida regras.xlsx`: todas as tolerâncias numa planilha (pandas `read_sql` + `to_excel`).
- Aceitar números com vírgula (`cof=0,28`).
- Testes com pytest para `tolerancias_efetivas` e `conferir`, usando um banco `:memory:`.

Exemplo de execução

```
> python 11-banco-de-dados-e-sql/desafio/solucao_referencia.py criar
Banco criado: ...\11-banco-de-dados-e-sql\saida\regras.db

> python 11-banco-de-dados-e-sql/desafio/solucao_referencia.py regras --
linha A --especificacao esp-0533 --nominal espessura=60 --nominal
gramatura=55,5
Linha A, especificação ESP-0533
característica  tipo       mínimo   máximo  origem
cof             fixa         0,15     0,35  PADRAO-A
espessura       nominal        57       63  ESP-0533
gramatura       nominal     49,95    61,05  PADRAO-A
selagem         fixa            8        -  PADRAO-A
tracao          fixa           28        -  ESP-0533

> python 11-banco-de-dados-e-sql/desafio/solucao_referencia.py conferir -
-linha D --valor cof=0.31 --valor espessura=60 --nominal espessura=62
REPROVADA
  - cof 0.31 acima do máximo 0.3

> python 11-banco-de-dados-e-sql/desafio/solucao_referencia.py regras --
```

```
linha B --especificacao ESP-9
Erro: especificação 'ESP-9' não cadastrada
```

Critérios de aceite

- [ ] `criar` gera o banco; rodar de novo sem `--forcar` dá erro amigável.
- [ ] `listar` mostra as 7 especificações (4 padrões e 3 de cliente).
- [ ] `regras --linha A --especificacao ESP-0533 --nominal espessura=60` mostra a espessura de 57 a 63 vinda da ESP-0533 e o COF de 0,15 a 0,35 vindo do PADRAO-A.
- [ ] `conferir` devolve código de saída 0 (aprovada) ou 2 (reprovada).
- [ ] Nenhuma consulta monta SQL com f-string a partir de texto do usuário.
- [ ] Banco inexistente não é criado sem querer.
- [ ] `ruff check` e `ruff format --check` passam no seu arquivo.

Dicas de como começar

1. Comece pelas funções de banco, sem CLI: `especificacao(conexao, linha, titulo)` e `tolerancias_efetivas(conexao, linha, titulo)`. Teste no REPL com `sqlite3.connect(":memory:")` e `executescript` do `regras.sql`.
2. A consulta das tolerâncias efetivas está na seção 15 da aula (UNION ALL + NOT IN).
3. `limites(...)` é uma função pura: fácil de testar sem banco.
4. Só depois monte o `argparse`. Use `action="append"` para aceitar `--valor` várias vezes.
5. Para fechar a conexão sempre: `with closing(sqlite3.connect(...)) as conexao:`.

A solução de referência está em `solucao_referencia.py`. Tente antes de abrir!

Quiz — Módulo 11: bancos de dados e SQL

Responda sem rodar. Depois confira no gabarito. As perguntas com SQL usam o banco do laboratório da aula (`dados/laboratorio.sql`): 12 OPs, 6 clientes, 60 ensaios.

1. Para que serve a chave estrangeira `cliente_id INTEGER REFERENCES cliente (id)` na tabela `op`?

- a) Deixa a busca por cliente mais rápida.
- b) Impede gravar uma OP com um cliente que não existe (com `PRAGMA foreign_keys = ON`).
- c) Cria o cliente automaticamente.

- d) Faz o `id` do cliente ser único.

2. O que esta consulta devolve?

```
SELECT NULL = NULL, NULL IS NULL, 1 + NULL;
```

3. Quantos clientes esta consulta conta? (Clientes de SP: Torrefação Serra, em Franca, e Biscoitos Aurora, em Marília. De MG: Laticínios Serra Azul, em Juiz de Fora, e Café do Vale, em Varginha.)

```
SELECT COUNT(*) FROM cliente WHERE uf = 'SP' OR uf = 'MG' AND cidade =
'Varginha';
```

4. Onde está o erro?

```
SELECT linha, COUNT(*) FROM op WHERE COUNT(*) > 2 GROUP BY linha;
```

5. O que `conexao.execute(sql, 24518)` provoca, e como corrigir?

```
sql = "SELECT * FROM op WHERE numero = ?"
conexao.execute(sql, 24518)
```

6. A tabela `op` tem 12 linhas e só 3 têm `especificacao` preenchida. O que sai?

```
SELECT COUNT(*), COUNT(especificacao) FROM op;
```

7. Qual a diferença entre `INNER JOIN` e `LEFT JOIN` entre `cliente` (esquerda) e `op` (direita)?

8. Por que este código é perigoso? Mostre um valor de `nome` que vaza todos os clientes.

```
sql = f"SELECT * FROM cliente WHERE nome = '{nome}'"
conexao.execute(sql)
```

9. Você precisa deixar o usuário escolher a coluna de ordenação. Por que `conexao.execute("SELECT * FROM op ORDER BY ?", (coluna,))` não resolve, e o que fazer?

10. O que acontece com os dados?

```
with conexao:
    conexao.execute("UPDATE op SET status = 'CERTIFICADA' WHERE numero =
```

```
24541")
    conexao.execute("INSERT INTO expedicao (op, nf, peso_kg, data) "
                    "VALUES (24541, '1', -5, '2026-10-30')")  # CHECK
(peso_kg > 0)
```

- a) O UPDATE fica gravado e o INSERT não.
- b) Nada fica gravado: o `with` faz rollback e a exceção sobe.
- c) Os dois ficam gravados.
- d) A conexão é fechada.

11. Depois de um bloco `with conexao:`, a conexão está fechada?

12. O que esta consulta faz e qual "padrão" ela usa?

```sql
SELECT o.numero
FROM op AS o
LEFT JOIN ensaio AS e ON e.op = o.numero
WHERE e.id IS NULL;
```

13. No banco de regras do projeto final, por que existe o índice abaixo?

```sql
CREATE UNIQUE INDEX um_padrao_por_linha ON especificacao (linha) WHERE
linha IS NOT NULL;
```

14. `sqlite3.connect("regras.bd")` (você errou a extensão; o arquivo certo é `regras.db`). O que acontece?

- a) `FileNotFoundError`.
- b) O SQLite cria um banco novo e vazio com esse nome, sem avisar.
- c) Abre o `regras.db` mesmo assim.
- d) `sqlite3.OperationalError` na hora.

Gabarito

1. **b.** A FK garante a integridade: `cliente_id` precisa existir em `cliente.id`. No SQLite, só vale com `PRAGMA foreign_keys = ON` em cada conexão.
2. **`(None, 1, None)`.** Comparar com NULL dá NULL ("desconhecido"); só `IS NULL` dá verdadeiro (1). Conta com NULL também dá NULL.
3. **3.** O `AND` vem antes do `OR`, então a condição é `uf = 'SP' OR (uf = 'MG' AND cidade = 'Varginha')`: os 2 de SP mais o Café do Vale. Use parênteses para não depender da precedência.

4. **Agregação no WHERE.** O WHERE roda antes do GROUP BY e não enxerga `COUNT(*)` (erro `misuse of aggregate function COUNT()`). Use `HAVING COUNT(*) > 2` depois do `GROUP BY`.
5. **`ProgrammingError: parameters are of unsupported type`.** Os parâmetros vão numa sequência: `conexao.execute(sql, (24518,))`, com a vírgula.
6. **`12 | 3`.** `COUNT(*)` conta linhas; `COUNT(coluna)` conta valores não nulos.
7. **INNER** só traz clientes que têm OP (e as OPs). **LEFT** traz todos os clientes; os sem OP aparecem uma vez, com as colunas de `op` em NULL (o Café do Vale, por exemplo).
8. **SQL injection.** O texto do usuário vira código SQL. Com `nome = "x' OR '1'='1"` a condição fica `nome = 'x' OR '1'='1'`, sempre verdadeira. Correto: `conexao.execute("SELECT * FROM cliente WHERE nome = ?", (nome,))`.
9. **Parâmetros só substituem VALORES**, nunca nomes de coluna ou tabela. Com `?`, o banco ordena por um texto constante (não pela coluna). Confira o nome numa lista permitida (`if coluna not in {"numero", "emissao"}: raise ValueError`) e só então monte o texto.
10. **b.** O `with conexao:` é uma transação: erro dentro do bloco desfaz tudo (rollback) e a exceção (`IntegrityError`) continua subindo.
11. **Não.** O `with conexao:` controla a transação (commit/rollback), mas não fecha. Feche com `conexao.close()` ou use `with contextlib.closing(sqlite3.connect(...))`.
12. **Lista as OPs sem nenhum ensaio** (24550 e 24551). É o "anti-join": LEFT JOIN e depois ficar só com quem não achou par (`IS NULL` numa coluna da direita).
13. **Para garantir no máximo UM padrão por linha.** É um índice único *parcial*: vale só para as linhas com `linha` preenchida (as especificações de cliente têm `linha` NULL e podem ser muitas).
14. **b.** O `connect` cria o arquivo se ele não existe. A primeira consulta depois dá `no such table`, longe da causa real. Por isso, confira `Path(caminho).is_file()` antes, ou abra em modo somente leitura (`?mode=ro` com `uri=True`).

Módulo 12 — Automação e integração

> **Tempo estimado:** 14 a 18 horas · **Pré-requisitos:** módulos 00 a 11 · **Nível:** intermediário

O que você vai aprender

- Explicar o que acontece numa requisição HTTP: método, URL, cabeçalhos, corpo e status.
- Consultar uma API com `requests` usando `params`, `json`, `timeout`, `raise_for_status()` e `Session`.
- Tratar falhas de rede (`Timeout`, `ConnectionError`, `HTTPError`) e repetir com espera crescente.
- Testar código que usa a rede **sem internet**, com injeção de dependência e `monkeypatch`.
- Organizar centenas de arquivos com `pathlib` e `shutil`, primeiro em modo ensaio.
- Gerar um relatório Excel com estilos, largura de coluna, filtro, painel congelado e formatação condicional usando `openpyxl`.
- Montar e salvar um e-mail com anexo (`email.message`) e ler arquivos `.eml`.
- Escolher o caminho certo para ler o Outlook: clássico (`win32com`), Novo Outlook, Microsoft Graph ou Power Automate.
- Agendar um script no Agendador de Tarefas do Windows.
- Escrever um robô web com Playwright no Edge: seletores robustos, esperas, prints de tela, log, modo ensaio e idempotência.

Por que isso importa

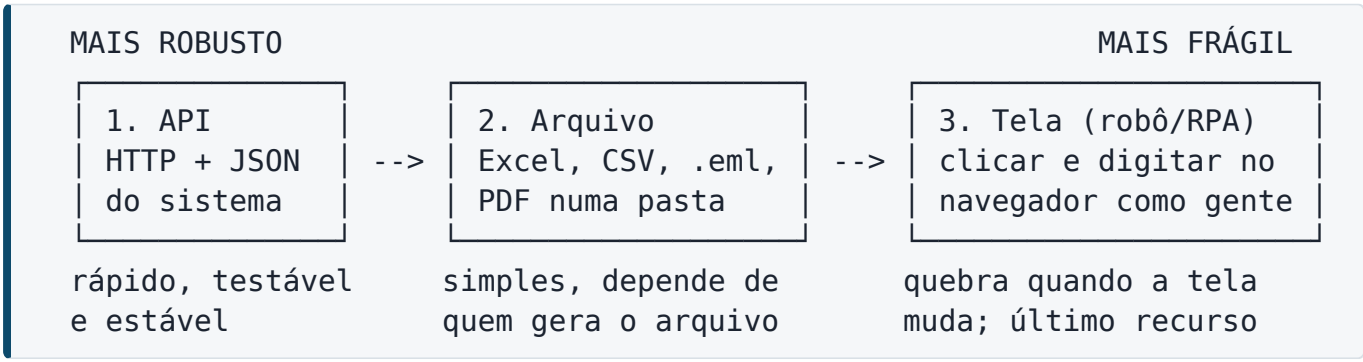
Todo dia alguém no laboratório copia número de uma tela para outra. Abre o e-mail da expedição, procura a OP, abre o Protheus, digita, confere, imprime, responde. São minutos por pedido, horas por semana, e um erro de digitação vira certificado errado.

Automação é a área de Python que mais rápido gera valor dentro de uma empresa. Também é uma porta de entrada no mercado: vagas de "analista de automação", "RPA" e "engenheiro de dados" pedem exatamente APIs, arquivos, Excel, e-mail e robôs.

Este módulo é a ponte do projeto final com o mundo: o núcleo (regras, tolerâncias, fila) está nos módulos anteriores. Aqui você aprende a **ler** pedidos de e-mail, **gerar** relatórios e **operar** a tela do ERP com um robô que não faz besteira.

1. O mapa da automação: API, arquivo ou tela?

Antes de escrever código, pergunte **por onde** os dados entram e saem. Existem três caminhos, do melhor para o pior:



- **API**: o sistema oferece uma "porta" para programas. É o caminho profissional.
- **Arquivo**: alguém (ou outro programa) gera um arquivo; seu script lê ou escreve.
- **Tela**: o robô imita uma pessoa. Funciona com qualquer sistema, mas é frágil.

> 💡 **Dica:** antes de fazer um robô para o TOTVS Protheus, pergunte à TI se existe integração por API REST ou outra rotina de importação. O Protheus tem recursos de integração; quem decide se estão liberados é a TI. Se a resposta for "não", o robô de tela é a saída, e vamos fazê-lo direito.

2. HTTP em 10 minutos

HTTP é a "língua" que navegadores e programas usam para conversar com servidores. É como o balcão do laboratório: alguém faz um **pedido** e recebe uma **resposta**.

Um pedido (requisição) tem quatro partes:

```
GET /tolerancias?linha=A HTTP/1.1          <- MÉTODO, CAMINHO (com
parâmetros) e versão
Host: api.lab.local                        <- CABEÇALHOS: informações
sobre o pedido
Accept: application/json
Authorization: Bearer abc123
                                           <- linha em branco
```

```
(corpo vazio: GET não leva corpo)          <- CORPO: os dados enviados
(POST, PUT...)
```

E a resposta volta assim:

```
HTTP/1.1 200 OK                            <- STATUS: deu certo?
Content-Type: application/json; charset=utf-8
Content-Length: 120

{"linha": "A", "itens": [{"ensaio": "espessura", "minimo": 18.0,
"maximo": 22.0}]}
```

Métodos

Método	Para quê	Exemplo
`GET`	Ler, sem mudar nada	consultar tolerâncias da linha A
`POST`	Criar algo novo	registrar uma medição
`PUT`	Substituir algo inteiro	trocar a regra de COF da linha B
`PATCH`	Mudar parte de algo	corrigir só o valor de uma medição
`DELETE`	Apagar	excluir uma medição lançada errada

`GET`, `PUT` e `DELETE` são **idempotentes**: repetir o mesmo pedido deixa o servidor no mesmo estado. `POST` não: dois `POST` iguais criam duas medições. Guarde essa palavra; ela volta na seção 12.

Códigos de status

O primeiro dígito diz a família:

Família	Significado	Os mais comuns
2xx	Sucesso	`200 OK`, `201 Created`, `204 No Content`
3xx	Redirecionamento	`301`, `302` (vá para outro endereço)
4xx	Erro **seu** (cliente)	`400` pedido malformado, `401` sem login, `403` sem permissão, `404` não existe, `422` dados inválidos, `429` devagar!
5xx	Erro **do servidor**	`500` erro interno, `502`/`504` problema no caminho, `503` fora do ar

JSON

JSON é o formato de texto mais usado em APIs. Ele mapeia quase direto para Python: objeto `{}` vira `dict`, lista `[]` vira `list`, `true/false/null` viram `True/False/None`. Você já usou o módulo `json` no módulo 06.

3. requests: conversando com APIs

O `requests` é a biblioteca mais usada para HTTP em Python. Já está no `requirements.txt` do curso. Se precisar: `python -m pip install requests`.

Para treinar sem depender da internet, o exemplo 01 sobe uma pequena "API de treino" no seu próprio computador (endereço `127.0.0.1`, que significa "esta máquina"). O trecho que importa é este:

```
resposta = requests.get(f"{base}/tolerancias", params={"linha": "A"},
timeout=5)
print("URL montada:", resposta.url)
print("Status:", resposta.status_code, resposta.reason)
print("Content-Type:", resposta.headers["Content-Type"])
dados = resposta.json()  # texto JSON -> dict do Python
```

Rode `python exemplos/01_http_basico.py`:

```
1) GET com parâmetros de consulta
   URL montada: <base>/tolerancias?linha=A
   Status: 200 OK
   Content-Type: application/json; charset=utf-8
   espessura   18.0 a 22.0
   cof          0.2 a 0.3
2) Um pedido que dá errado (404)
   Status: 404 | ok? False
   Corpo: {'erro': "Linha 'Z' não encontrada"}
   raise_for_status levantou HTTPError 404
3) POST enviando JSON
   Status: 201 Created
   Corpo: {'id': 1, 'op': '123456', 'ensaio': 'cof', 'valor': 0.25}
```

(`<base>` é o endereço do servidor de treino; a porta muda a cada execução.)

O que cada peça faz:

- `params={"linha": "A"}` monta a query string `?linha=A` e codifica acentos e espaços.
- `timeout=5` desiste depois de 5 segundos sem resposta. **Sempre** passe timeout.

- `resposta.status_code` é o número; `resposta.ok` é `True` para status abaixo de 400.
- `resposta.json()` converte o corpo JSON em `dict` / `list`.
- `resposta.raise_for_status()` levanta `requests.HTTPError` se o status for 4xx ou 5xx.
- `requests.post(url, json=dados)` envia `dados` como JSON (e põe o cabeçalho certo).

> ⚠️ **Erro comum:** achar que um 404 ou 500 levanta exceção sozinho. Não levanta! Para o `requests`, receber uma resposta é sucesso, mesmo que a resposta diga "erro". Chame `raise_for_status()` logo depois de cada chamada.

> 🔮 **Preveja antes de rodar:** que URL o `requests` monta aqui?
>
> ```
> import requests
>
> pedido = requests.Request(
>     "GET", "https://api.lab.local/medicoes",
>     params={"linha": "A", "op": None, "ensaio": "tração"},
> ).prepare()
> print(pedido.url)
> ```

▼ **Resposta**

```
https://api.lab.local/medicoes?linha=A&ensaio=tra%C3%A7%C3%A3o
```

Parâmetros com valor `None` são ignorados, e o `ç` / `ã` viram códigos `%XX`. O `Request(...).prepare()` monta o pedido sem enviar: ótimo truque para conferir a URL no REPL sem internet.

> 🔧 **Modifique:** no exemplo 01, peça a linha `"B"` e acrescente um segundo parâmetro, `"ensaio": "espessura"`. Veja a URL mudar. O servidor de treino ignora o parâmetro extra; uma API real poderia filtrar por ele.

Sessão: configurar uma vez, usar sempre

Quando você faz várias chamadas para a mesma API, use `requests.Session()`. Ela guarda cabeçalhos (como o token de acesso) e reaproveita a conexão, o que é mais rápido.

```
with requests.Session() as sessao:
    sessao.headers.update(
        {"User-Agent": "robo-laboratorio/1.0", "Authorization": "Bearer
abc123"}
    )
    for rota in ["/status", "/linhas"]:
        print(sessao.get(base + rota, timeout=5).json())
```

Saída (parte do exemplo 02):

```
1) Sessão: cabeçalhos configurados uma vez, usados em todas as chamadas
   {'rota': '/status', 'user_agent': 'robo-laboratorio/1.0',
'autorizacao': 'Bearer abc123'}
   {'rota': '/linhas', 'user_agent': 'robo-laboratorio/1.0',
'autorizacao': 'Bearer abc123'}
```

> ⚠️ **Erro comum:** escrever o token ou a senha no código. Ele vai parar no Git e no e-mail de alguém. Leia de uma variável de ambiente: `token = os.environ["API_LAB_TOKEN"]`. No PowerShell, para a sessão atual: `$env:API_LAB_TOKEN = "abc123"`.

Tente você

Usando `requests.Request(...).prepare()`, descubra que URL sai de `params={"ops": ["123456", "123789"], "limite": 10}`.

<details>
<summary>Resposta</summary>

`?ops=123456&ops=123789&limite=10`. Uma lista vira a mesma chave repetida.

</details>

4. Falhas de rede: timeout, erros e novas tentativas

A rede da fábrica cai. O servidor reinicia na hora do almoço. O Wi-Fi do laboratório some. Um robô profissional espera por isso.

As exceções do `requests` formam uma família. Todas herdam de `RequestException`:

```
RequestException
 ├── ConnectionError        não conseguiu nem conversar com o servidor
 │    └── ConnectTimeout    (também é Timeout) demorou para conectar
 ├── Timeout
 │    └── ReadTimeout       conectou, mas a resposta não veio a tempo
```

```
  ├── HTTPError              raise_for_status() viu um 4xx ou 5xx
  └── JSONDecodeError        .json() recebeu algo que não é JSON
```

O exemplo 02 provoca as duas primeiras de propósito. Rode `python exemplos/02_sessao_e_falhas.py` e veja o final:

```
2) Timeout: o servidor demora 2 s, mas só aceitamos esperar 0,5 s
   ReadTimeout depois de 0.5 s
3) Ninguém escutando na porta: ConnectionError
   ConnectionError
4) A família de exceções do requests
   Timeout é RequestException? True
   HTTPError é RequestException? True
```

🔮 **Preveja antes de rodar:** o que este laço imprime para cada exceção?

```python
import requests

for erro in [requests.ReadTimeout("x"), requests.ConnectTimeout("x"),
requests.HTTPError("x")]:
    try:
        raise erro
    except requests.ConnectionError:
        print("conexão")
    except requests.Timeout:
        print("timeout")
    except requests.RequestException:
        print("genérico")
```

▼ **Resposta**

```
timeout
conexão
genérico
```

O Python testa os `except` de cima para baixo e entra no **primeiro** que combina. `ReadTimeout` é só `Timeout`. `ConnectTimeout` é `ConnectionError` **e** `Timeout`, e o primeiro `except` já o pega. `HTTPError` só combina com a mãe de todas.

Exemplo resolvido: tentar de novo com espera fixa

Problema: a consulta às vezes falha por queda de rede. Quero tentar até 3 vezes.

Como pensei:

1. Preciso repetir, então é um laço. Sei quantas vezes: `for` com `range`.
2. Dentro do laço, tento a chamada. Deu certo? `return` e acabou.
3. Deu `ConnectionError`? Aviso, espero e deixo o laço tentar de novo.
4. Se foi a **última** tentativa, não adianta esperar: relanço o erro com `raise`.
5. Para não amarrar a função a uma consulta específica, recebo a consulta como **função** (módulo 04: funções são valores).

```
import time
import requests

def tentar_varias_vezes(funcao, tentativas=3, espera=2.0):
    for numero in range(1, tentativas + 1):
        try:
            return funcao()
        except requests.ConnectionError as erro:
            print(f"tentativa {numero} falhou: {erro}")
            if numero == tentativas:
                raise
            time.sleep(espera)

# Uma consulta "de mentira" que falha duas vezes e depois funciona:
falhas = [requests.ConnectionError("rede caiu"),
requests.ConnectionError("rede caiu")]

def consultar():
    if falhas:
        raise falhas.pop()
    return {"linha": "A", "itens": []}

print(tentar_varias_vezes(consultar, espera=0.1))
```

Saída:

```
tentativa 1 falhou: rede caiu
tentativa 2 falhou: rede caiu
{'linha': 'A', 'itens': []}
```

No exercício 04 você vai evoluir esta ideia: espera que **dobra** a cada falha (*backoff exponencial*), `Timeout` também repetido, e um `dormir` injetável para o teste não esperar de verdade.

> 💡 **Dica:** nunca repita um `HTTPError` 400, 401, 403 ou 404. Repetir não conserta um pedido errado, uma senha errada ou um endereço que não existe.

> Só vale repetir falhas passageiras: rede, timeout, 429, 500, 502, 503 e 504.

5. Testar sem internet: injeção de dependência e monkeypatch

Teste que depende da internet é teste ruim: fica lento, falha quando a API cai e não consegue simular erros. Existem duas técnicas para isolar a rede.

Técnica 1 — injeção de dependência. A função **recebe** a sessão como parâmetro. No programa real, você passa `requests.Session()`. No teste, um objeto falso (um *dublê*).

```
def buscar_feriados(ano: int, sessao: requests.Session | None = None) ->
list[str]:
    if sessao is None:
        sessao = requests.Session()
    resposta = sessao.get(URL_FERIADOS.format(ano=ano), timeout=10)
    resposta.raise_for_status()
    feriados = []
    for item in resposta.json():
        _, mes, dia = item["date"].split("-")
        feriados.append(f"{dia}/{mes} - {item['name']}")
    return feriados
```

O dublê só precisa ter os métodos que a função usa. Python não confere o tipo: se tem `.get()`, serve (isso se chama *duck typing*: "se anda como pato...").

```
class RespostaFalsa:
    def __init__(self, dados, status_code=200):
        self.dados = dados
        self.status_code = status_code

    def json(self):
        return self.dados

    def raise_for_status(self):
        if self.status_code >= 400:
            raise requests.HTTPError(f"{self.status_code} Error")


class SessaoFalsa:
    def __init__(self, resposta):
        self.resposta = resposta
        self.chamadas = []          # o teste pode conferir o que foi
pedido
```

```
    def get(self, url, **kwargs):
        self.chamadas.append((url, kwargs))
        return self.resposta


def test_formata_os_feriados():
    dados = [{"date": "2026-04-21", "name": "Tiradentes"}]
    sessao = SessaoFalsa(RespostaFalsa(dados))
    assert buscar_feriados(2026, sessao) == ["21/04 - Tiradentes"]
```

Técnica 2 — monkeypatch. Quando a função chama `requests.get` direto, o teste troca `requests.get` por uma função falsa, só durante o teste. O pytest desfaz a troca no fim.

```
def test_cidade_do_cep_com_monkeypatch(monkeypatch):
    def get_falso(url, timeout):
        assert url == "https://viacep.com.br/ws/01001000/json/"
        return RespostaFalsa({"localidade": "São Paulo", "uf": "SP"})

    monkeypatch.setattr(requests, "get", get_falso)
    assert cidade_do_cep("01001-000") == "São Paulo/SP"
```

O exemplo 03 tem as duas técnicas completas. Rode `python exemplos/03_cliente_testavel.py`:

```
exemplos/03_cliente_testavel.py::test_formata_os_feriados PASSED
[ 20%]
exemplos/03_cliente_testavel.py::test_monta_a_url_e_usa_timeout PASSED
[ 40%]
exemplos/03_cliente_testavel.py::test_erro_http_nao_e_engolido PASSED
[ 60%]
exemplos/03_cliente_testavel.py::test_cidade_do_cep_com_monkeypatch
PASSED [ 80%]
exemplos/03_cliente_testavel.py::test_cep_inexistente PASSED
[100%]

============================== 5 passed in 0.03s
==============================
```

Qual usar? Prefira **injeção**: a dependência fica visível na assinatura e o teste não precisa saber como a função foi escrita por dentro. Use `monkeypatch` para código que você não pode (ou não quer) mudar.

> 💡 **Dica:** com internet, experimente em casa as APIs públicas do exemplo 03, sem teste: `requests.get("https://viacep.com.br/ws/01001000/json/", timeout=10).json()` devolve um dicionário com `'localidade': 'São Paulo'` e `'uf': 'SP'`, entre outros.

6. Arquivos e pastas em lote (pathlib + shutil)

Você já usou `pathlib` no módulo 06. Agora o foco é **lote**: centenas de arquivos de uma vez. No REPL do Windows:

```
>>> from pathlib import Path
>>> arquivo = Path("C:/laboratorio/laudos") / "laudo_123456_B_2026-03-10.pdf"
>>> arquivo
WindowsPath('C:/laboratorio/laudos/laudo_123456_B_2026-03-10.pdf')
>>> print(arquivo)
C:\laboratorio\laudos\laudo_123456_B_2026-03-10.pdf
>>> arquivo.name, arquivo.stem, arquivo.suffix
('laudo_123456_B_2026-03-10.pdf', 'laudo_123456_B_2026-03-10', '.pdf')
>>> arquivo.parent.name
'laudos'
>>> arquivo.with_suffix(".xlsx").name
'laudo_123456_B_2026-03-10.xlsx'
```

(No Linux/macOS aparece `PosixPath` e as barras são `/`.)

Para listar e mover:

O que fazer	Código
Itens da pasta (1 nível)	`pasta.iterdir()`
Por padrão	`pasta.glob("*.pdf")`
Também nas subpastas	`pasta.rglob("*.pdf")`
Criar pasta (e as de cima)	`pasta.mkdir(parents=True, exist_ok=True)`
Copiar mantendo a data	`shutil.copy2(origem, destino)`
Mover ou renomear	`shutil.move(origem, destino)`
Copiar uma árvore	`shutil.copytree(origem, destino)`
Compactar uma pasta	`shutil.make_archive("backup", "zip", pasta)`
Apagar uma árvore (cuidado!)	`shutil.rmtree(pasta)`

⚠️ **Erro comum:** confiar que `shutil.move` avisa quando o destino já existe. Ele pode **sobrescrever** o arquivo sem perguntar (depende do sistema). Antes de mover, verifique `destino.exists()`.

🔮 **Preveja antes de rodar:** em que ordem saem estes nomes?

```
print(sorted(["laudo_b.pdf", "Laudo_a.pdf", "laudo_A.pdf", "10.pdf",
"9.pdf"]))
```

▼ **Resposta**

```
['10.pdf', '9.pdf', 'Laudo_a.pdf', 'laudo_A.pdf', 'laudo_b.pdf']
```

`sorted` compara texto caractere por caractere pela tabela Unicode: dígitos vêm antes de maiúsculas, que vêm antes de minúsculas. Por isso `"10"` vem antes de `"9"` (o `1` é menor que o `9`) e `"Laudo"` antes de `"laudo"`. Para ordem "humana" de letras use `sorted(nomes, key=str.lower)`; para números, nomeie os arquivos com zeros à esquerda (`009`, `010`).

Exemplo resolvido: organizar laudos por linha

Problema: a pasta `entrada` tem laudos como `laudo_123456_A.pdf` misturados com outros arquivos. Quero cada laudo em `laudos/linha_A/`, `laudos/linha_B/`...

Como pensei:

1. O nome do arquivo tem tudo que preciso. Uma regex (módulo 05) separa OP e linha.
2. Mover arquivos é perigoso. Então separo em duas funções: `planejar` (decide, não mexe em nada) e `executar` (mexe, ou só mostra, conforme o modo).
3. O modo **ensaio** mostra o plano. Só depois de conferir eu rodo de verdade.
4. Nunca sobrescrevo: se o destino existe, pulo e aviso.

```
PADRAO_LAUDO = re.compile(r"laudo_(\d{6})_([A-D])\.pdf", re.IGNORECASE)

def planejar(origem: Path, destino: Path) -> list[tuple[Path, Path]]:
    plano = []
    for arquivo in sorted(origem.iterdir()):
        achado = PADRAO_LAUDO.fullmatch(arquivo.name)
        if arquivo.is_file() and achado:
```

```
            linha = achado.group(2).upper()
            plano.append((arquivo, destino / f"linha_{linha}" /
arquivo.name))
    return plano

def executar(plano: list[tuple[Path, Path]], ensaio: bool) -> None:
    for origem, destino in plano:
        if ensaio:
            print(f"   [ENSAIO] {origem.name} -> {destino.parent.name}/")
            continue
        if destino.exists():
            print(f"   [PULADO] {destino.name} já existe no destino")
            continue
        destino.parent.mkdir(parents=True, exist_ok=True)
        shutil.move(origem, destino)
        print(f"   [MOVIDO] {origem.name} -> {destino.parent.name}/")
```

O exemplo 04 cria uma pasta temporária com arquivos de mentira e roda tudo. Saída de `python exemplos/04_arquivos_em_lote.py`:

```
1) Modo ensaio (nada é movido):
   [ENSAIO] LAUDO_124001_b.PDF -> linha_B/
   [ENSAIO] laudo_123456_A.pdf -> linha_A/
   [ENSAIO] laudo_123789_B.pdf -> linha_B/
   [ENSAIO] laudo_124050_D.pdf -> linha_D/
2) Agora de verdade:
   [MOVIDO] LAUDO_124001_b.PDF -> linha_B/
   [MOVIDO] laudo_123456_A.pdf -> linha_A/
   [MOVIDO] laudo_123789_B.pdf -> linha_B/
   [MOVIDO] laudo_124050_D.pdf -> linha_D/
3) Como ficou:
   entrada/anotacoes.txt
   entrada/~$relatorio.xlsx
   laudos/linha_A/laudo_123456_A.pdf
   laudos/linha_B/LAUDO_124001_b.PDF
   laudos/linha_B/laudo_123789_B.pdf
   laudos/linha_D/laudo_124050_D.pdf
```

Repare: o `~$relatorio.xlsx` ficou para trás. É o arquivo de trava que o Excel cria enquanto uma planilha está aberta. Robôs devem ignorá-lo (exercício 05).

> 🔧 **Modifique:** no exemplo 04, mude o padrão para aceitar só linhas `A` e `B` (`[AB]`) e rode de novo. Quais arquivos ficam em `entrada`? Depois crie um laudo repetido dentro de `criar_bagunca` já no destino e veja o `[PULADO]` aparecer.

7. Relatórios Excel com openpyxl

No módulo 10 você exportou DataFrames com `to_excel`. Para controlar a **aparência** (cores, larguras, filtros), usamos o `openpyxl` direto. O vocabulário:

```
Workbook (o arquivo .xlsx)
 └── Worksheet (uma aba: wb.active, wb["Medições"],
wb.create_sheet("Resumo"))
      └── Cell (uma célula: ws["A1"], ws.cell(row=1, column=1))
           ├── .value          o conteúdo
           ├── .font           Font(bold=True, color="FFFFFF")
           ├── .fill           PatternFill("solid", start_color="1F4E78")
           ├── .alignment      Alignment(horizontal="center")
           └── .number_format  "0.00", "#,##0.00", "dd/mm/yyyy"
```

O exemplo 05 monta um relatório completo em seis passos. Os pontos-chave:

```
wb = Workbook()
ws = wb.active
ws.title = "Medições"

ws.append(CABECALHO)                          # 1) uma linha inteira por
vez
for m in MEDICOES:
    ws.append([m["op"], m["linha"], m["ensaio"], m["valor"], m["min"],
m["max"], status(m)])

for celula in ws[1]:                          # 2) ws[1] = a linha 1
    celula.font = Font(bold=True, color="FFFFFF")
    celula.fill = PatternFill("solid", start_color="1F4E78")

for linha in ws.iter_rows(min_row=2, min_col=4, max_col=6):
    for celula in linha:                      # 3) formato numérico
        celula.number_format = "0.00"

for indice, coluna in enumerate(ws.iter_cols(), start=1):      # 4)
larguras
    maior = max(len(str(c.value)) for c in coluna if c.value is not None)
    ws.column_dimensions[get_column_letter(indice)].width = maior + 2

ws.freeze_panes = "A2"                        # 5) congela o cabeçalho
ws.auto_filter.ref = ws.dimensions            #    e liga o filtro

ws.conditional_formatting.add(                # 6) REPROVADO em vermelho
    f"G2:G{ws.max_row}",
    CellIsRule(operator="equal", formula=['"REPROVADO"'],
               font=Font(color="9C0006"),
               fill=PatternFill("solid", start_color="FFC7CE",
```

```
end_color="FFC7CE")),
)
wb.save(caminho)
```

Rode `python exemplos/05_relatorio_excel.py`. Ele gera o arquivo e depois o **reabre** para conferir (é assim que os testes dos exercícios fazem):

```
Gerado: relatorio_medicoes.xlsx
Planilha: Medições | dimensões: A1:G6
Cabeçalho negrito? True | cor: 001F4E78
Largura da coluna C: 11.0
Congelado em: A2 | filtro: A1:G6
Regra condicional: G2:G6 equal ['"REPROVADO"']
OPs com reprovação: ['123789', '124001']
```

O arquivo fica em `exemplos/saida/relatorio_medicoes.xlsx`. Abra no Excel e teste o filtro. Repare: a cor `1F4E78` volta como `001F4E78` (o openpyxl acrescenta o canal de transparência na frente).

Formatação condicional ou pintar a célula direto? Pintar direto (`celula.fill = ...`) é fixo. A formatação condicional é uma **regra** que o Excel reavalia: se alguém mudar o texto para `APROVADO`, a cor some sozinha. Para relatórios que as pessoas editam, prefira a regra.

> ⚠️ **Erro comum:** `formula=["REPROVADO"]` sem as aspas internas. O Excel entende `REPROVADO` como nome de uma célula ou intervalo e a regra nunca dispara. O certo é `formula=['"REPROVADO"']`: aspas simples do Python por fora, duplas do Excel por dentro.

> 🔮 **Preveja antes de rodar:** quanto vale `ws.max_row` em cada `print`?
>
> ```
> from openpyxl import Workbook
>
> ws = Workbook().active
> ws.append(["OP", "Valor"])
> ws.append(["123456", 20.4])
> print(ws.max_row)
> ws.cell(row=10, column=1)
> print(ws.max_row, ws.dimensions)
> ```

▼ **Resposta**

```
2
10 A1:B10
```

Só **olhar** uma célula com `ws.cell(...)` (ou `ws["A10"]`) já a cria, vazia. Com isso a planilha "cresce" e o `max_row` passa a ser 10. Cuidado ao usar `ws.max_row` para saber onde estão os dados: acesse células só onde precisa escrever.

Lendo uma planilha existente:

```
from openpyxl import load_workbook

wb = load_workbook("resultados.xlsx", data_only=True)   # data_only:
valor, não a fórmula
ws = wb["Linha A"]
for op, espessura, cof in ws.iter_rows(min_row=2, max_col=3,
values_only=True):
    print(op, espessura, cof)
```

💡 **Dica:** para análise (filtros, médias, groupby), leia com pandas (módulo 10). Para escrever com estilo, use openpyxl. Dá para juntar: `df.to_excel(...)` e depois `load_workbook` para formatar.

🔧 **Modifique:** no exemplo 05, acrescente uma segunda regra condicional que pinte `APROVADO` de verde (`fill` `C6EFCE`, fonte `006100`). Depois crie uma aba "Resumo" com `wb.create_sheet("Resumo")` e escreva nela quantas medições foram reprovadas.

8. E-mail: montar, salvar e ler mensagens

Um e-mail é um **arquivo de texto** com cabeçalhos (`From`, `To`, `Subject`, `Date`...) e um corpo, que pode ter várias partes (texto, HTML, anexos). O formato se chama MIME, e a biblioteca padrão do Python sabe montar e ler.

Montar (sem enviar)

```
from email.message import EmailMessage

msg = EmailMessage()
msg["From"] = "Laboratório de Qualidade <laboratorio@embalagens.example>"
msg["To"] = "Expedição <expedicao@embalagens.example>"
```

```
msg["Subject"] = f"Certificado de qualidade - OP {op} / NF {nf}"
msg.set_content(f"Olá,\n\nSegue o certificado de qualidade da OP {op} (NF
{nf}).\n")
msg.add_attachment(pdf.read_bytes(), maintype="application",
subtype="pdf",
                   filename=pdf.name)
caminho_eml.write_bytes(bytes(msg))     # salva no formato padrão .eml
```

O exemplo 06 faz isso (com uma versão HTML opcional) e mostra a estrutura. Saída de `python exemplos/06_email_montar.py`:

```
Assunto: Certificado de qualidade - OP 123456 / NF 98765
Para: Expedição <expedicao@embalagens.example>
Estrutura da mensagem:
   multipart/mixed
   multipart/alternative
   text/plain
   text/html
   application/pdf certificado_123456.pdf
Primeira linha do texto: Olá,
Anexos: ['certificado_123456.pdf']
Salvo em: certificado_123456.eml
```

Dê dois cliques no `.eml` gerado em `exemplos/saida/`: o Outlook abre a mensagem pronta. Esse é um ótimo **modo ensaio** para e-mails: o robô monta, uma pessoa confere.

🔮 **Preveja antes de rodar:** o que acontece na segunda atribuição?

```
from email.message import EmailMessage

msg = EmailMessage()
msg["Subject"] = "Certificado OP 123456"
msg["Subject"] = "Certificado OP 999999"
```

▼ **Resposta**

```
ValueError: There may be at most 1 Subject headers in a message
```

Cabeçalhos de e-mail **não** funcionam como um dicionário comum: atribuir de novo acrescentaria um segundo `Subject`, e o `EmailMessage` recusa. Para trocar, apague antes: `del msg["Subject"]` e depois atribua.

Enviar: smtplib (conceito)

Enviar depende de um servidor SMTP liberado pela TI. O código tem esta forma:

```
import os
import smtplib

with smtplib.SMTP("smtp.empresa.example", 587, timeout=30) as smtp:
    smtp.starttls()                                    # conexão
criptografada
    smtp.login(os.environ["SMTP_USUARIO"], os.environ["SMTP_SENHA"])
    smtp.send_message(msg)
```

Em empresas com Microsoft 365 é comum o envio por usuário e senha estar **bloqueado** por segurança. Não tente contornar: converse com a TI (as saídas oficiais são o Microsoft Graph, a seção 9, ou um servidor de envio interno).

O mesmo vale para **ler** por IMAP (`imaplib`): no Microsoft 365 o acesso exige OAuth (login moderno), não só senha. Na prática, para ler o Outlook corporativo, veja a próxima seção.

Ler arquivos .eml

Um `.eml` salvo vira bytes, e os bytes viram uma mensagem:

```
from email import policy
from email.parser import BytesParser

msg = BytesParser(policy=policy.default).parsebytes(caminho.read_bytes())
corpo = msg.get_body(preferencelist=("plain", "html"))   # prefere texto
puro
texto = corpo.get_content()
```

Use **sempre** `policy=policy.default`: ela decodifica os acentos dos cabeçalhos (`=?utf-8?q?Expedi=C3=A7=C3=A3o?=` vira `Expedição`) e oferece `get_body` e `get_content`. Sem ela, você cai na API antiga, bem mais trabalhosa.

Exemplo resolvido: de onde vem cada pedido?

Problema: na pasta `dados/emails/` há 5 pedidos da expedição. Quero ver, para cada um, o remetente, a data, o assunto e a OP.

Como pensei:

1. Ler cada `.eml` com `BytesParser` (acima).

2. O remetente vem como `Nome <endereco>`. `email.utils.parseaddr` separa as partes.
3. A data vem como texto (`Mon, 09 Mar 2026 08:12:00 -0300`). `email.utils.parsedate_to_datetime` devolve um `datetime` (módulo 07).
4. Alguns e-mails só têm HTML (`OP: <b>124001</b>`). Minha regex aceita um `<b>` opcional, só para este exemplo; no exercício 11 você vai remover as tags.

```
for caminho in sorted(PASTA_EMAILS.glob("*.eml")):
    msg = ler_eml(caminho)
    nome, endereco = parseaddr(msg["From"])
    quando = parsedate_to_datetime(msg["Date"])
    corpo = msg.get_body(preferencelist=("plain", "html"))
    texto = corpo.get_content()
    op = re.search(r"OP:?\s*(?:<b>)?(\d{6})", texto)
    print(caminho.name)
    print("   de:", endereco, "| em:", quando.strftime("%d/%m/%Y %H:%M"))
    print("   assunto:", msg["Subject"])
    print("   corpo:", corpo.get_content_type(), "| OP:", op.group(1) if
op else "?")
```

Saída de `python exemplos/07_email_ler.py` (os dois primeiros e o terceiro, só HTML):

```
pedido_001.eml
   de: expedicao@embalagens.example | em: 09/03/2026 08:12
   assunto: Pedido de certificado - OP 123456
   corpo: text/plain | OP: 123456
pedido_002.eml
   de: expedicao@embalagens.example | em: 09/03/2026 09:40
   assunto: Certificado urgente — OP 123789
   corpo: text/plain | OP: 123789
pedido_003.eml
   de: expedicao@embalagens.example | em: 09/03/2026 10:05
   assunto: Pedido de certificado - OP 124001
   corpo: text/html | OP: 124001
```

O `pedido_002` tem texto **e** HTML (`multipart/alternative`); o `get_body` escolheu o texto puro, como pedimos.

9. Outlook no Windows: clássico, novo e Microsoft Graph

Na empresa, os pedidos chegam no Outlook, não numa pasta de `.eml`. Existem dois "Outlooks" no Windows hoje, e isso muda tudo:

	Outlook clássico	Novo Outlook
Como reconhecer	Faz parte do Office; tem menu **Arquivo > Opções**	App novo, parecido com o Outlook na web
Automação local (COM/`win32com`)	**Sim**	**Não**
Macros VBA	Sim	Não
Caminho para Python	`win32com` (só Windows)	Microsoft Graph ou Power Automate

Outlook clássico com `win32com`. O pacote `pywin32` (`pip install pywin32`) deixa o Python "pilotar" o Outlook aberto na sua máquina. O exemplo 08 lê a caixa de entrada:

```
import win32com.client

outlook = win32com.client.Dispatch("Outlook.Application")
caixa = outlook.GetNamespace("MAPI").GetDefaultFolder(6)   # 6 = Caixa de Entrada
itens = caixa.Items
itens.Sort("[ReceivedTime]", True)                          # mais recentes primeiro
for msg in itens.Restrict("[ReceivedTime] >= '09/03/2026 00:00'"):
    if msg.Class == 43 and "certificado" in msg.Subject.lower():   # 43 = e-mail
        print(msg.ReceivedTime, msg.Subject)
        print(msg.Body[:200])                                       # texto puro
```

Outras ações úteis: `anexo.SaveAsFile(caminho)` para cada item de `msg.Attachments`, `msg.Move(pasta)` para arquivar e, para responder, `novo = outlook.CreateItem(0)` seguido de `novo.Display()`. O `Display()` **abre a janela** com o e-mail pronto para você conferir e clicar em Enviar: é o modo ensaio do Outlook. `novo.Send()` envia direto.

> ⚠️ **Erro comum:** `msg.SenderEmailAddress` de um colega da mesma empresa às vezes volta como `/O=EXCHANGELABS/OU=...` em vez do e-mail. Para remetentes internos use `msg.Sender.GetExchangeUser().PrimarySmtpAddress`.

O exemplo 08 só funciona no Windows com o Outlook clássico. Em outros casos ele explica isso e termina.

Novo Outlook: Microsoft Graph. O Graph é a API oficial da Microsoft para e-mail, calendário, OneDrive e Teams. É HTTP + JSON, exatamente o que você aprendeu nas seções 2 a 4. A parte difícil não é o código, é a **permissão**: a TI precisa registrar um aplicativo no Microsoft Entra ID e liberar a permissão `Mail.Read`. Com isso, o login é feito pela biblioteca `msal`, que entrega um token. A chamada em si tem esta cara (conceito, não roda sem essa configuração):

```
resposta = requests.get(
    "https://graph.microsoft.com/v1.0/me/mailFolders/inbox/messages",
    headers={"Authorization": f"Bearer {token}"},
    params={"$select": "subject,from,receivedDateTime,body", "$top": 25,
            "$orderby": "receivedDateTime desc"},
    timeout=30,
)
resposta.raise_for_status()
for mensagem in resposta.json()["value"]:
    print(mensagem["receivedDateTime"], mensagem["subject"])
```

Sem TI disponível? Power Automate. Um fluxo de nuvem "Quando um novo email chegar" pode filtrar pelo assunto e salvar a mensagem (há ação para exportar no formato `.eml`) numa pasta do OneDrive ou SharePoint. O OneDrive sincroniza a pasta no seu computador e o Python lê os `.eml`, como na seção 8. Simples e sem `win32com`.

> 💡 **Dica:** separe **de onde vem** o e-mail de **o que fazer** com ele. Escreva o processamento recebendo bytes de `.eml` (exercícios 11 e 14). Hoje os bytes vêm de uma pasta; amanhã, do Graph. O resto do programa não muda.

10. Agendamento: o Agendador de Tarefas do Windows

Um robô que você precisa lembrar de rodar não é um robô. O Windows já tem um agendador.

Passo 1 — um lançador `.bat`. O exemplo `exemplos/11_rodar_robo.bat` mostra o modelo:

```
@echo off
set PASTA=C:\robos\curso-python
set SCRIPT=12-automacao-e-integracao\exemplos\09_robo_certificados.py
cd /d "%PASTA%"
if not exist logs mkdir logs
".venv\Scripts\python.exe" "%SCRIPT%" >> "logs\agendador.log" 2>&1
exit /b %ERRORLEVEL%
```

Três detalhes que evitam 90% dos problemas:

1. `cd /d` entra na pasta do projeto. O Agendador começa em `C:\Windows\System32`, e caminhos relativos como `dados\ops.csv` quebrariam.
2. Chama o `python.exe` **do venv** direto. Não precisa "ativar" nada.
3. `>> logs\agendador.log 2>&1` guarda tudo que o script imprimiria, inclusive erros.

Passo 2 — criar a tarefa. Pelo menu Iniciar, abra "Agendador de Tarefas":

1. **Criar Tarefa...** (a versão completa, não a "Básica").
2. Aba **Geral**: nome `Robo Certificados`. Se o robô abre o navegador visível, marque "Executar somente quando o usuário estiver conectado".
3. Aba **Disparadores**: Novo > Semanalmente, de segunda a sexta, 07:30.
4. Aba **Ações**: Novo > Iniciar um programa > o caminho do `.bat`. Em "Iniciar em", a pasta do projeto.
5. Aba **Configurações**: "Interromper a tarefa se for executada por mais de" 1 hora.

Ou, no PowerShell, a mesma tarefa em uma linha (e comandos para testar e remover):

```
schtasks /Create /TN "Robo Certificados" /TR "C:\robos\curso-python\12-
automacao-e-integracao\exemplos\11_rodar_robo.bat" /SC WEEKLY /D
MON,TUE,WED,THU,FRI /ST 07:30
schtasks /Run /TN "Robo Certificados"
schtasks /Query /TN "Robo Certificados" /V /FO LIST
schtasks /Delete /TN "Robo Certificados" /F
```

Na coluna "Resultado da Última Execução", `0x0` é sucesso e `0x1` significa que o script terminou com `sys.exit(1)`. Por isso nossos scripts devolvem 1 quando algo falha: o Agendador (e você) ficam sabendo.

No Linux/macOS o equivalente é o `cron`: `30 7 * * 1-5 cd /home/voce/curso-python && .venv/bin/python script.py >> logs/cron.log 2>&1`.

11. Automação web com Playwright

O **Playwright** controla um navegador de verdade: abre páginas, preenche campos, clica, espera e tira prints. A Microsoft mantém o projeto, e ele funciona muito bem com o Edge.

Por que Playwright e não "gravar cliques na tela" (como o `pyautogui`)? Porque ele encontra os elementos **pelo significado** (o campo chamado "OP", o botão "Confirmar"), não pela posição do mouse. Se a janela mudar de tamanho, o robô continua funcionando.

Instalação (uma vez)

Com o ambiente virtual ativado:

```
pip install playwright
```

Pronto. Não é preciso baixar navegador: usamos o **Microsoft Edge já instalado** no Windows, com `channel="msedge"`. (Em Linux/macOS sem Edge, rode `python -m playwright install chromium` e lance sem o `channel`.)

O primeiro robô

O simulador `dados/simulador_certificados.html` é uma página de **treino** que imita a tela de emissão de um ERP (OP, NF, peso, linha, observação, Confirmar e Cancelar). Ela até demora um pouco para responder, como um sistema real.

```
from pathlib import Path

from playwright.sync_api import expect, sync_playwright

pagina_treino = Path("12-automacao-e-
integracao/dados/simulador_certificados.html")

with sync_playwright() as p:
    navegador = p.chromium.launch(channel="msedge", headless=False,
slow_mo=300)
    page = navegador.new_page()
    page.goto(pagina_treino.resolve().as_uri())

    page.get_by_label("OP", exact=True).fill("123456")
    page.get_by_label("NF", exact=True).fill("98765")
    page.get_by_label("Peso (kg)").fill("1.250,50")
    page.get_by_label("Linha").select_option("B")
    page.screenshot(path="preenchido.png")

    page.get_by_role("button", name="Cancelar").click()   # ensaio: NÃO
confirma
    expect(page.get_by_role("status")).to_have_text("Operação cancelada")
    navegador.close()
```

As peças:

- `sync_playwright()` liga o Playwright; o `with` desliga no fim.
- `p.chromium.launch(channel="msedge", ...)`: o Edge é um navegador da família Chromium. `headless=False` mostra a janela; `slow_mo=300` desacelera cada ação em 300 ms para você acompanhar com os olhos.

- `page` é a aba. `goto` abre o endereço (`as_uri()` transforma o caminho do arquivo em `file:///C:/...`).
- `get_by_label(...)` encontra o campo pelo **rótulo** visível; `fill` digita.
- `expect(...).to_have_text(...)` **espera** até a mensagem aparecer (ou falha depois do tempo limite).

Seletores: como o robô acha as coisas

Use os localizadores na ordem desta tabela. Os de cima imitam como uma **pessoa** acha o elemento e sobrevivem a mudanças no visual da página:

Prioridade	Localizador	Quando usar
1	`page.get_by_role("button", name="Confirmar")`	botões, links, caixas, abas
2	`page.get_by_label("Peso (kg)")`	campos com rótulo
3	`page.get_by_placeholder("Digite a OP")`	campos sem rótulo, com texto de dica
4	`page.get_by_text("Certificado gerado")`	textos na tela
5	`page.locator("#peso")` (CSS)	quando nada acima funciona
6	XPath (`page.locator("xpath=//div[3]/input")`)	evite: quebra fácil

`exact=True` exige o texto inteiro (sem ele, `get_by_label("OP")` também acharia um rótulo "OP de origem"). Para descobrir os nomes, aperte **F12** no Edge e inspecione o elemento, ou deixe o Playwright gravar por você:

```
python -m playwright codegen --channel msedge https://endereco-da-tela
```

O **codegen** abre o Edge e uma janela que vai escrevendo o código Python enquanto você clica. Use como ponto de partida, nunca como código final.

> 🔮 **Preveja antes de rodar:** a tela tem os botões "Confirmar" e "Cancelar". O que acontece com esta linha?
>
> ```
> page.get_by_role("button").click()
> ```

▼ **Resposta**

O Playwright recusa e levanta um erro (texto real):

```
Error: Locator.click: Error: strict mode violation:
get_by_role("button") resolved to 2 elements:
    1) <button type="button" id="confirmar">Confirmar</button> aka
get_by_role("button", name="Confirmar")
    2) <button type="button" id="cancelar">Cancelar</button> aka
get_by_role("button", name="Cancelar")
```

É o **modo estrito**: uma ação nunca "chuta" entre vários elementos. Ótimo para um robô que emite certificados! A mensagem ainda sugere o seletor certo para cada um.

Esperas: nunca `time.sleep`

A maior fonte de robôs instáveis é esperar "no chute". O Playwright tem esperas embutidas:

- **Toda ação espera sozinha.** `click()` aguarda o botão existir, estar visível, habilitado e parado. `fill()` aguarda o campo aceitar texto.
- **`expect(...)` espera uma condição** e tenta de novo até o limite: `to_be_visible()`, `to_have_text(...)`, `to_contain_text(...)`, `to_have_value(...)`, `to_be_empty()`. Aceita regex: `to_have_text(re.compile(r"Certificado \d+|Erro"))`.
- **Navegação:** `page.wait_for_url("**/certificados/**")` espera o endereço mudar.
- **Tempo limite padrão:** `page.set_default_timeout(10_000)` (10 s). Para um passo lento, `timeout=30_000` só nele.

```
time.sleep(5)  ->  rápido demais num dia lento, lento demais num dia
normal
expect(...)    ->  espera exatamente o necessário, e falha com mensagem
clara
```

Prints, trace e outros recursos

- `page.screenshot(path="tela.png", full_page=True)`: a página inteira.
- `page.get_by_role("table").screenshot(path="tabela.png")`: só um elemento.
- **Trace**: grava tudo (prints, cliques, rede) num `.zip` que você "assiste" depois. `contexto.tracing.start(screenshots=True, snapshots=True)` no início, `contexto.tracing.stop(path="trace.zip")` no fim, e `python -m playwright show-trace trace.zip` para ver.

- Elementos dentro de **iframe**: `page.frame_locator("iframe").get_by_label("OP")`.
- **Janelas de confirmação** do navegador (`alert` / `confirm`): `page.on("dialog", lambda dialogo: dialogo.accept())`.
- **Login guardado**: `p.chromium.launch_persistent_context("perfil_robo", channel="msedge", headless=False)` usa uma pasta de perfil própria e mantém cookies entre execuções (útil com login único da empresa).

E o Protheus?

O TOTVS Protheus roda no navegador pelo **WebApp**. Do ponto de vista do robô, é uma página web como o simulador, com algumas particularidades:

- **Telas antes da rotina:** programa inicial e ambiente, login, empresa/filial, data base e só então o menu. Cada uma é um passo do robô, com um `expect` confirmando que a tela certa abriu.
- **Componentes próprios:** a tela é desenhada com componentes web da TOTVS, muitas vezes dentro de *shadow DOM* (uma "subpágina" encapsulada). Os localizadores `get_by_role`, `get_by_label`, `get_by_text` e CSS do Playwright atravessam shadow DOM aberto; XPath não atravessa. Mais um motivo para fugir do XPath.
- **Lentidão variável:** use `expect` com tempo limite generoso nos passos que dependem do servidor, e confira a mensagem de retorno de cada operação.
- **Ambiente de teste:** peça à TI um ambiente de homologação (cópia do Protheus para testes). O robô aprende lá; em produção, só depois de muitos ensaios.
- **Credenciais:** usuário e senha do robô ficam em variáveis de ambiente (`os.environ["PROTHEUS_USUARIO"]`), nunca no código ou no Git. Combine com a TI se o robô terá um usuário próprio, com só as permissões necessárias.

O caminho é: treinar no simulador, gravar o fluxo real com `codegen` no ambiente de teste, trocar os seletores gravados por `get_by_role` / `get_by_label`, e só então encaixar no esqueleto do robô da próxima seção.

12. O robô de certificados: modo ensaio, log, prints e idempotência

Um robô de verdade não é só "preencher e clicar". Ele precisa de cinco proteções:

```
 pedidos ──> [validar dados] ──> [já emitido?] ──> [plano de ações] ──>
[executar na tela]
                  | inválido          | sim               |
|
```

```
                  v                    v                    | ensaio:
| falhou:
             log + pendência     log + pula                | Cancelar
| print + log
                                                           v
v
                                                    registro (CSV) <──
emitido: anota
```

1. **Modo ensaio por padrão:** faz tudo, menos o passo irreversível. Emitir de verdade exige pedir explicitamente (`--confirmar`).
2. **Log em arquivo:** o robô roda sozinho às 7h; o log é o único jeito de saber o que aconteceu. Use `logging`, não `print` (módulo 09).
3. **Prints de tela:** um antes de confirmar e um em cada erro. Evidência para você e para a auditoria da qualidade.
4. **Idempotência:** antes de agir, pergunte a um registro se aquilo já foi feito. Depois de agir, anote **na hora**. Rodar de novo não duplica nada.
5. **Falhas isoladas:** um pedido com problema vira log + print e o robô segue para o próximo. No fim, o código de saída diz se houve falha.

Exemplo resolvido: separar o plano da execução

Problema: quero testar a lógica do robô sem abrir o navegador.

Como pensei: a parte que **decide** (o que preencher, em que ordem, se confirma ou cancela) não precisa da tela. Então escrevo uma função que devolve o plano como dados:

```python
def planejar(pedido: dict, ensaio: bool) -> list[tuple[str, str, str]]:
    return [
        ("preencher", "OP", pedido["op"]),
        ("preencher", "NF", pedido["nf"]),
        ("preencher", "Peso (kg)", formatar_peso(pedido["peso_kg"])),
        ("selecionar", "Linha", pedido["linha"]),
        ("print", f"antes_de_confirmar_{pedido['op']}", ""),
        ("clicar", "Cancelar" if ensaio else "Confirmar", ""),
    ]
```

E uma função **burra** que executa qualquer plano, com uma linha por tipo de ação:

```python
def executar(page, acoes, pasta_prints):
    for acao, alvo, valor in acoes:
        if acao == "preencher":
            page.get_by_label(alvo, exact=True).fill(valor)
        elif acao == "selecionar":
```

```
            page.get_by_label(alvo, exact=True).select_option(valor)
        elif acao == "clicar":
            page.get_by_role("button", name=alvo, exact=True).click()
        elif acao == "print":
            page.screenshot(path=pasta_prints / f"{alvo}.png")
```

Agora o teste mais importante do robô ("em ensaio, nunca clica em Confirmar") roda em milissegundos, sem navegador. O exercício 12 é exatamente isso, com o código de um colega cheio de defeitos para você encontrar.

O robô completo

O `exemplos/09_robo_certificados.py` junta tudo: validação antes da tela, registro `emitidos.csv`, log diário, pasta de prints por execução, `try/except` por pedido e `expect` esperando sucesso **ou** erro da tela. Ele tem um pedido com OP errada de propósito. Rode no Windows (depois do `pip install playwright`):

```
python 12-automacao-e-integracao/exemplos/09_robo_certificados.py
```

O Edge abre, preenche cada pedido e cancela. No terminal (os horários variam):

```
02:44:20 INFO    Início. Modo: ENSAIO
02:44:29 INFO    [ENSAIO] OP 123456 preenchida e cancelada
02:44:29 INFO    [ENSAIO] OP 123789 preenchida e cancelada
02:44:29 ERROR   OP 12400 com dados inválidos: OP inválida: '12400'
02:44:29 INFO    [ENSAIO] OP 124050 preenchida e cancelada
02:44:29 INFO    Fim. {'emitidos': 0, 'ensaiados': 3, 'pulados': 0,
'falhas': 1}
```

Com `--confirmar`, ele emite no simulador e anota no registro. Rodando **de novo**:

```
02:44:42 INFO    Início. Modo: CONFIRMAR
02:44:43 INFO    OP 123456 / NF 98765 já emitida antes: pulando
02:44:43 INFO    OP 123789 / NF 98766 já emitida antes: pulando
02:44:43 ERROR   OP 12400 com dados inválidos: OP inválida: '12400'
02:44:43 INFO    OP 124050 / NF 98771 já emitida antes: pulando
02:44:43 INFO    Fim. {'emitidos': 0, 'ensaiados': 0, 'pulados': 3,
'falhas': 1}
```

Isso é idempotência na prática. Os prints ficam em `exemplos/saida/robo/prints_.../`. Se o Playwright não estiver instalado, o script só explica como instalar e termina.

> 🔧 **Modifique:** no robô, troque a NF do segundo pedido por `"98x66"` e rode com `--confirmar --headless`. A tela recusa ("Erro: NF inválida"); veja no log o traceback, o print `123789_ERRO.png` e o robô seguindo para o próximo pedido. Depois acrescente a validação da NF em `problemas_do_pedido`, para o erro ser pego **antes** da tela.

O exemplo 10 mostra a mesma ideia de idempotência sem navegador, usando o `Message-ID` de cada e-mail. Saída de `python exemplos/10_log_e_idempotencia.py`:

```
--- 1ª rodada ---
INFO  Novo pedido: pedido_001.eml | Pedido de certificado - OP 123456
INFO  Novo pedido: pedido_002.eml | Certificado urgente — OP 123789
INFO  Novo pedido: pedido_003.eml | Pedido de certificado - OP 124001
INFO  Novo pedido: pedido_004.eml | Pedido de certificado - OP 124050
INFO  Novo pedido: pedido_005.eml | Pedido de certificado - OP 125310
novos: 5
--- 2ª rodada (nada mudou na pasta) ---
DEBUG pedido_001.eml já processado, pulando
DEBUG pedido_002.eml já processado, pulando
DEBUG pedido_003.eml já processado, pulando
DEBUG pedido_004.eml já processado, pulando
DEBUG pedido_005.eml já processado, pulando
novos: 0
--- o arquivo de log tem 10 linhas; a primeira é: ---
INFO  Novo pedido: pedido_001.eml | Pedido de certificado - OP 123456
```

Repare no `salvar_vistos`: ele grava num arquivo temporário e depois **troca** pelo definitivo (`temporario.replace(caminho)`). Se o computador desligar no meio, o registro antigo continua inteiro. Robôs gostam de arquivos que nunca ficam pela metade.

Lendo mensagens de erro deste módulo

Não conectou:

```
requests.exceptions.ConnectionError: HTTPConnectionPool(host='127.0.0.1',
port=8766):
Max retries exceeded with url: /tolerancias (Caused by
NewConnectionError(...
Failed to establish a new connection: [Errno 111] Connection refused"))
```

Ninguém atendeu no endereço. No Windows aparece `[WinError 10061]`. Confira o endereço e a porta, se o servidor está no ar e se a VPN está ligada. Com `NameResolutionError`, o nome do servidor nem foi encontrado (erro de digitação ou rede interna fora do ar).

Demorou demais:

```
requests.exceptions.ReadTimeout: HTTPConnectionPool(host='127.0.0.1',
port=8765):
Read timed out. (read timeout=0.5)
```

Conectou, mas a resposta não veio a tempo. Aumente o `timeout` se a API é lenta mesmo, ou trate com novas tentativas (seção 4).

O servidor respondeu com erro:

```
requests.exceptions.HTTPError: 404 Client Error: Not Found for url:
http://127.0.0.1:8765/x
```

Veio do `raise_for_status()`. Leia o número: 4xx é pedido seu errado (endereço, parâmetro, login); 5xx é problema do lado de lá. `erro.response.text` costuma explicar.

Não era JSON:

```
requests.exceptions.JSONDecodeError: Expecting value: line 1 column 1
(char 0)
```

Você chamou `.json()` e veio HTML ou texto. Clássico: a API redirecionou para uma página de login, ou devolveu uma página de erro. Imprima `resposta.status_code` e `resposta.text[:300]` antes de qualquer coisa.

Certificado SSL (rede corporativa): `SSLError ... certificate verify failed: unable to get local issuer certificate`. O proxy da empresa usa um certificado próprio. Peça à TI o arquivo da autoridade certificadora e use `verify="caminho/empresa.pem"`. Nunca "resolva" com `verify=False` em produção: isso desliga a segurança da conexão.

Planilha aberta no Excel: `PermissionError: [Errno 13] Permission denied: 'relatorio.xlsx'`. No Windows, o Excel trava o arquivo aberto. Feche a planilha ou salve com outro nome (exercício 06).

Formato antigo: `InvalidFileException: openpyxl does not support the old .xls file format, please use xlrd to read this file, or convert it to the more recent .xlsx file format.` Abra no Excel e salve como `.xlsx`.

Edge não encontrado:

```
Error: BrowserType.launch: Chromium distribution 'msedge' is not found at
/opt/microsoft/msedge/msedge
Run "playwright install msedge"
```

Este texto veio de um Linux sem Edge. No Windows, verifique se o Edge está instalado; em último caso rode `python -m playwright install msedge` (pede administrador) ou use `python -m playwright install chromium` e lance sem `channel`.

Elemento não apareceu:

```
playwright._impl._errors.TimeoutError: Locator.click: Timeout 3000ms
exceeded.
Call log:
  - waiting for get_by_role("button", name="Gravar")
```

O robô esperou um botão "Gravar" que não existe (o nome real é "Confirmar") ou que ainda não apareceu. O *call log* mostra exatamente qual seletor falhou. Confira o nome no F12 ou no `codegen`.

A tela mostrou outra coisa:

```
AssertionError: Locator expected to have text 'Certificado'
Actual value:
Call log:
  - Expect "to_have_text" get_by_role("status") with timeout 2000ms
```

Veio de um `expect`. O "Actual value" mostra o que estava na tela. Aqui, nada: o robô não chegou a clicar em Confirmar.

Outlook: `pywintypes.com_error: (-2147221005, 'Invalid class string', None, None)` (em português a mensagem vem traduzida). O Windows não encontrou o Outlook clássico: não está instalado ou a máquina só tem o Novo Outlook (seção 9).

Boas práticas

- **API > arquivo > tela.** Robô de tela é o último recurso, não o primeiro.
- **Timeout em toda chamada HTTP** e `raise_for_status()` logo depois.

- **Segredos fora do código:** tokens e senhas em variáveis de ambiente. Nada de senha no Git, no log ou no print de tela.
- **Funções puras no núcleo** (planejar, validar, formatar) e efeitos colaterais (rede, disco, tela) nas bordas. O núcleo se testa sem internet e sem navegador.
- **Injete dependências** (sessão, função de emitir, função de dormir) para testar.
- **Modo ensaio como padrão** em qualquer coisa que mova, apague, envie ou emita.
- **Nunca sobrescreva** evidências: confira `exists()` ou gere um nome novo.
- **Idempotência:** registre o que foi feito e consulte antes de agir.
- **Log em arquivo** com data e hora, e código de saída 0/1 para o Agendador.
- **Caminhos a partir do script** (`Path(__file__).resolve().parent`), não da pasta atual.
- **Seletores por papel e rótulo** no Playwright; `expect` em vez de `sleep`.
- **Comece pequeno:** um pedido, em ensaio, com você olhando. Depois dez. Depois o dia.

No mundo real

Uma rotina realista no laboratório, juntando as peças deste módulo:

```
07:25  Power Automate salva os e-mails "Pedido de certificado" como .eml
na pasta
       OneDrive\Robo\entrada (sincronizada no PC do laboratório).
07:30  O Agendador roda rodar_robo.bat:
         1. lê os .eml novos (Message-ID ainda não visto);
         2. extrai OP, NF e peso; e-mail incompleto vira pendência;
         3. confere a OP no banco de regras (módulo 11) e escolhe a
especificação;
         4. robô Playwright no Edge: preenche a tela de certificados,
tira print,
            CONFIRMA (este passo só foi ligado depois de 2 semanas de
ensaio);
         5. anota cada emissão no registro (idempotência);
         6. gera fila_AAAA-MM-DD.xlsx com abas Fila e Pendências;
         7. monta o e-mail-resumo com a planilha e abre no Outlook
(Display) para
            o analista conferir e enviar.
07:45  O analista lê o resumo, resolve as 2 pendências na mão e envia.
```

O ganho não é só tempo. É **rastreabilidade**: para cada certificado existe log, print da tela antes da confirmação e registro de quando foi emitido. Numa auditoria de cliente, isso vale ouro.

Repare também no que **não** é automático: o envio do e-mail e as pendências. Bons robôs deixam com as pessoas as decisões que pedem julgamento.

O desafio deste módulo é construir as etapas 1, 2, 3, 5, 6 e 7 (com um emissor simulado no lugar da tela).

Resumo

Assunto	Essencial
GET com parâmetros	`requests.get(url, params={...}, timeout=10)`
POST com JSON	`requests.post(url, json=dados, timeout=10)`
Conferir status	`resposta.raise_for_status()`; `resposta.status_code`; `resposta.ok`
Corpo	`resposta.json()`, `resposta.text`, `resposta.headers["Content-Type"]`
Sessão	`with requests.Session() as s: s.headers.update({...})`
Exceções	`RequestException` > `ConnectionError`, `Timeout`, `HTTPError`
Testar sem rede	receber `sessao` por parâmetro; `monkeypatch.setattr(requests, "get", falso)`
Arquivos em lote	`iterdir`, `glob`, `rglob`, `mkdir(parents=True, exist_ok=True)`
Mover/copiar	`shutil.move`, `shutil.copy2` (confira `exists()` antes)
Excel	`Workbook`, `ws.append`, `Font`, `PatternFill`, `number_format`, `column_dimensions[...].width`
Excel (extras)	`freeze_panes = "A2"`, `auto_filter.ref`, `conditional_formatting.add(..., CellIsRule(...))`
Montar e-mail	`EmailMessage()`, `msg["Subject"]`, `set_content`, `add_attachment`, `bytes(msg)`
Ler .eml	`BytesParser(policy=policy.default).parsebytes(b)`, `get_body(preferencelist=...)`
Outlook clássico	`win32com.client.Dispatch("Outlook.Application")` (só Windows)
Novo Outlook	Microsoft Graph (TI registra o app) ou Power Automate salvando `.eml`
Agendar	`.bat` com `cd /d` + `python.exe` do venv; `schtasks /Create ...`
Playwright	`sync_playwright()`, `p.chromium.launch(channel="msedge", headless=False)`
Localizar	`get_by_role(..., name=...)`, `get_by_label(...)`, `exact=True`

Assunto	Essencial
Esperar	ações esperam sozinhas; `expect(loc).to_have_text(...)`; sem `sleep`
Evidências	`page.screenshot(path=...)`, trace, `logging` em arquivo
Robô seguro	ensaio por padrão, validação antes da tela, registro idempotente, falha isolada

Você domina este módulo se...

- ☐ Explico sem consultar o que são método, status, cabeçalho e corpo numa requisição HTTP, e o que significam 200, 201, 404, 422 e 503.
- ☐ Escrevo uma chamada `requests` com `params`, `timeout` e `raise_for_status()` e sei qual exceção aparece quando a rede cai, quando demora e quando dá 404.
- ☐ Testo uma função que usa a rede sem internet, com um dublê injetado ou `monkeypatch`.
- ☐ Organizo uma pasta de arquivos em lote com modo ensaio e sem sobrescrever nada.
- ☐ Gero uma planilha com cabeçalho estilizado, filtro e formatação condicional, e confiro o resultado reabrindo com `load_workbook`.
- ☐ Monto um e-mail com anexo, salvo como `.eml` e leio de volta OP, NF e peso.
- ☐ Sei explicar por que o `win32com` não funciona no Novo Outlook e quais são as alternativas.
- ☐ Escrevo um robô Playwright que usa `get_by_role`/`get_by_label`, espera com `expect`, tira prints, roda em ensaio por padrão e não emite nada em dobro.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_classificar_status`	★☆☆	famílias de status HTTP, quando repetir
02	`ex02_montar_url`	★☆☆	juntar URL sem barra dupla, `urlencode`
03	`ex03_buscar_tolerancias`	★★☆	`requests` com sessão injetada, `timeout`, `raise_for_status`
04	`ex04_com_retentativas`	★★★	backoff exponencial, exceções de rede, `dormir` injetável
05	`ex05_listar_planilhas`	★☆☆	`pathlib`, extensões, arquivos `~$` do Excel
06	`ex06_nome_disponivel`	★☆☆	nunca sobrescrever: `nome (2).pdf`
07	`ex07_organizar_laudos`	★★☆	regex no nome, `shutil.move`, modo ensaio

Nº	Exercício	Nível	O que pratica
08	`ex08_larguras_colunas`	★☆☆	lógica de largura de coluna (função pura)
09	`ex09_relatorio_excel`	★★★	`openpyxl`: estilos, formato, filtro, formatação condicional
10	`ex10_montar_email`	★★☆	`EmailMessage`, anexo, `mimetypes`
11	`ex11_ler_pedido_eml`	★★☆	ler `.eml`, HTML, regex, número brasileiro
12	`ex12_corrigir_plano_robo`	★★☆	leitura de código: achar os defeitos do plano do robô
13	`ex13_revisao_fila_idempotente`	★★★	revisão: CSV, funções como valores, exceções (módulos 03, 04, 06)
14	`ex14_revisao_resumo_por_dia`	★★☆	revisão: datas, regex, acumuladores, formato BR (módulos 03, 05, 07)

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError` e rode, na pasta `curso-python`:

```
python curso.py testar 12 ex01
```

Para testar o módulo inteiro: `python curso.py testar 12`. Nenhum exercício usa internet nem navegador: rede, tela e relógio são sempre dublês nos testes.

Desafio do módulo

Robô da expedição (modo ensaio). Um programa de linha de comando que lê os pedidos de certificado em `.eml`, confere as OPs no cadastro, monta a fila com plano de ações, "emite" com registro idempotente, gera o relatório Excel com abas Fila e Pendências e salva o e-mail-resumo para o supervisor. Enunciado completo em desafio/README.md.

Quiz

Teste o que ficou: quiz.md.

Para ir além

- `urllib.parse` (montar e desmontar URLs): https://docs.python.org/pt-br/3/library/urllib.parse.html

- `pathlib`: https://docs.python.org/pt-br/3/library/pathlib.html
- `shutil`: https://docs.python.org/pt-br/3/library/shutil.html
- `email` (exemplos de mensagens): https://docs.python.org/pt-br/3/library/email.examples.html
- `logging` (guia básico): https://docs.python.org/pt-br/3/howto/logging.html
- requests: https://requests.readthedocs.io/
- openpyxl (estilos e formatação condicional): https://openpyxl.readthedocs.io/
- Playwright para Python (comece por "Locators" e "Auto-waiting"): https://playwright.dev/python/docs/intro
- Microsoft Graph, API de e-mail: https://learn.microsoft.com/graph/api/resources/mail-api-overview

Enunciados dos exercícios

Exercício 12.01 — Classificar códigos de status HTTP

Arquivo: `exercicios/ex01_classificar_status.py`

```
Nível: ★☆☆

Contexto: o robô consulta uma API e precisa decidir o que fazer com cada
resposta: seguir em frente, desistir ou tentar de novo mais tarde.

Tarefa: implemente duas funções.

1. classificar_status(codigo) devolve a família do código:
   - 100 a 199 -> "informativo"
   - 200 a 299 -> "sucesso"
   - 300 a 399 -> "redirecionamento"
   - 400 a 499 -> "erro do cliente"
   - 500 a 599 -> "erro do servidor"
   Qualquer outro número levanta ValueError com a mensagem
   "Código de status inválido: <codigo>".

2. vale_tentar_de_novo(codigo) devolve True só para os códigos que
costumam
   ser passageiros: 408, 429, 500, 502, 503 e 504. Para os outros, False.

Exemplos:
    >>> classificar_status(201)
    'sucesso'
    >>> classificar_status(404)
    'erro do cliente'
    >>> vale_tentar_de_novo(503)
    True
```

```
    >>> vale_tentar_de_novo(404)
    False

Dica: a divisão inteira 404 // 100 dá 4.

Teste com:  python curso.py testar 12 ex01
```

Exercício 12.02 — Montar a URL de uma consulta

Arquivo: `exercicios/ex02_montar_url.py`

```
Nível: ★☆☆

Contexto: o robô chama várias rotas da mesma API. Juntar pedaços de URL
"na
mão" gera erros clássicos: barra dobrada (api//medicoes), barra faltando
(apimedicoes) e acentos que quebram o endereço.

Tarefa: implemente montar_url(base, caminho, params=None).
- Junte base e caminho com exatamente UMA barra entre eles, não importa
se a
  base termina com "/" ou se o caminho começa com "/".
- params é opcional. Ignore as chaves cujo valor é None. As que sobrarem
  viram a "query string" (?chave=valor&...) na ordem do dicionário,
  codificadas com urllib.parse.urlencode (espaço vira "+", acentos viram
%XX).
- Se não sobrar nenhum parâmetro, a URL não leva "?".

Exemplos:
    >>> montar_url("https://api.lab.local/", "/medicoes", {"linha": "A",
"limite": 10})
    'https://api.lab.local/medicoes?linha=A&limite=10'
    >>> montar_url("https://api.lab.local", "tolerancias")
    'https://api.lab.local/tolerancias'
    >>> montar_url("https://api.lab.local", "busca", {"texto": "filme
PE", "op": None})
    'https://api.lab.local/busca?texto=filme+PE'

Dica: str.rstrip("/") e str.lstrip("/") removem barras só de uma ponta.
Para codificar: from urllib.parse import urlencode.

Teste com:  python curso.py testar 12 ex02
```

Exercício 12.03 — Buscar tolerâncias numa API (com sessão injetada)

Arquivo: `exercicios/ex03_buscar_tolerancias.py`

```
Nível: ★★☆

Contexto: a TI publicou uma API interna com as tolerâncias de cada linha.
Seu
código vai consultá-la, mas os testes não podem depender da rede. Por
isso a
função RECEBE a sessão HTTP como parâmetro: no programa real você passa
um
requests.Session(); nos testes, uma sessão falsa que só finge responder.

A API responde a  GET <base_url>/tolerancias?linha=A  com um JSON assim:
    {"linha": "A",
     "itens": [{"ensaio": "espessura", "minimo": 18.0, "maximo": 22.0},
               {"ensaio": "cof", "minimo": 0.2, "maximo": 0.3}]}

Tarefa: implemente buscar_tolerancias(sessao, base_url, linha,
timeout=10).
- A linha aceita "a" ou "A"; converta para maiúscula. Se não for A, B, C
ou D,
  levante ValueError ANTES de qualquer chamada à rede.
- Chame sessao.get(f"{base_url}/tolerancias", params={"linha": <linha>},
  timeout=timeout). Sempre passe timeout!
- Chame resposta.raise_for_status() (erros HTTP NÃO devem ser engolidos).
- Devolva um dicionário {ensaio: (minimo, maximo)} montado a partir de
"itens".

Exemplos:
    >>> buscar_tolerancias(requests.Session(), "http://api.lab.local",
"a")
    {'espessura': (18.0, 22.0), 'cof': (0.2, 0.3)}

Dica: resposta.json() já devolve dicionários e listas do Python.

Teste com:  python curso.py testar 12 ex03
```

Exercício 12.04 — Tentar de novo com espera crescente (backoff)

Arquivo: `exercicios/ex04_com_retentativas.py`

```
Nível: ★★★

Contexto: a rede da fábrica oscila. Uma consulta que falha às 7h02
costuma
funcionar às 7h03. Um robô profissional não desiste na primeira falha,
mas
também não martela o servidor: espera um pouco, e cada vez um pouco mais.

Tarefa: implemente com_retentativas(funcao, tentativas=3, espera=1.0,
```

```
dormir=time.sleep).
- Chame funcao() (sem argumentos) e devolva o que ela devolver.
- Se ela levantar requests.ConnectionError ou requests.Timeout, espere e
tente
  de novo, até completar `tentativas` chamadas no total.
- As esperas dobram: espera, espera*2, espera*4, ... Use dormir(segundos)
para
  esperar (os testes trocam dormir por uma função que só anota os
valores,
  para não esperar de verdade).
- Não espere depois da ÚLTIMA tentativa: se todas falharem, levante de
novo a
  última exceção (use `raise` sozinho ou `raise erro`).
- Qualquer outra exceção (por exemplo requests.HTTPError ou ValueError)
sobe
  na hora, sem nova tentativa: repetir não resolve.
- tentativas menor que 1 -> ValueError.

Exemplos:
    Uma função que falha 2 vezes com ConnectionError e depois devolve
"ok":
    >>> esperas = []
    >>> com_retentativas(funcao, tentativas=3, espera=1.0,
dormir=esperas.append)
    'ok'
    >>> esperas
    [1.0, 2.0]

Dica: um for tentativa in range(tentativas) com try/except dentro.
Pergunte
"esta é a última tentativa?" antes de dormir.

Teste com:  python curso.py testar 12 ex04
```

Exercício 12.05 — Listar as planilhas de uma pasta

Arquivo: `exercicios/ex05_listar_planilhas.py`

```
Nível: ★☆☆

Contexto: toda manhã o robô precisa processar as planilhas de resultados
que o
pessoal salva na pasta de rede. Só que a pasta também tem PDFs,
anotações,
subpastas e os famosos arquivos "~$Resultados.xlsx", que o Excel cria
enquanto a planilha está aberta (não são planilhas de verdade!).

Tarefa: implemente listar_planilhas(pasta).
- Devolva os NOMES (str) dos arquivos .xlsx e .xlsm que estão diretamente
```

```
na
  pasta (não entre nas subpastas). A extensão vale em maiúsculas ou
  minúsculas (.XLSX também conta).
- Ignore arquivos cujo nome começa com "~$".
- Ignore pastas, mesmo que o nome termine em .xlsx.
- Devolva a lista em ordem alfabética, sem diferenciar maiúsculas de
  minúsculas (use key=str.lower no sorted).
- Se a pasta não existir, levante FileNotFoundError.

Exemplos (pasta com: Linha_B.xlsx, linha_a.xlsx, ~$linha_a.xlsx,
laudo.pdf):
    >>> listar_planilhas(Path("resultados"))
    ['linha_a.xlsx', 'Linha_B.xlsx']

Dica: Path.iterdir(), .is_file(), .suffix e .name. Lembre de .lower().

Teste com:  python curso.py testar 12 ex05
```

Exercício 12.06 — Nunca sobrescrever um arquivo

Arquivo: `exercicios/ex06_nome_disponivel.py`

```
Nível: ★☆☆

Contexto: o robô salva o PDF de cada certificado numa pasta. Se já
existir um
"certificado_123456.pdf" (de uma emissão anterior), gravar por cima
apagaria
uma evidência. A saída é fazer como o Windows: "certificado_123456
(2).pdf".

Tarefa: implemente nome_disponivel(pasta, nome).
- Se pasta / nome não existe, devolva esse caminho.
- Se existe, tente "<nome sem extensão> (2)<extensão>", depois " (3)", e
assim
  por diante, e devolva o primeiro caminho livre.
- A função NÃO cria nenhum arquivo: só escolhe o nome.

Exemplos (a pasta já tem relatorio.xlsx e relatorio (2).xlsx):
    >>> nome_disponivel(Path("saida"), "relatorio.xlsx")
    PosixPath('saida/relatorio (3).xlsx')      # no Windows:
WindowsPath(...)
    >>> nome_disponivel(Path("saida"), "novo.xlsx")
    PosixPath('saida/novo.xlsx')

Dica: Path("certificado.pdf").stem é "certificado" e .suffix é ".pdf".

Teste com:  python curso.py testar 12 ex06
```

Exercício 12.07 — Organizar laudos em pastas (com modo ensaio)

Arquivo: `exercicios/ex07_organizar_laudos.py`

```
Nível: ★★☆

Contexto: a pasta "Entrada" acumula centenas de PDFs de laudos com nomes
padronizados, por exemplo  laudo_123456_B_2026-03-10.pdf  (OP, linha e
data).
Você vai arrumar tudo em  destino/linha_B/2026-03/laudo_123456_B_2026-03-
10.pdf.
Como mover arquivos em lote é perigoso, o padrão é rodar primeiro em
ENSAIO.

Tarefa: implemente organizar_laudos(origem, destino, ensaio=True).
- Considere só arquivos (não pastas) da origem cujo nome segue EXATAMENTE
o
  padrão  laudo_<6 dígitos>_<linha A a D>_<AAAA-MM-DD>.pdf  (tudo
minúsculo,
  linha maiúscula). Os demais arquivos ficam onde estão.
- O destino de cada laudo é  destino / f"linha_{linha}" / "AAAA-MM" /
nome.
- Devolva uma lista de tuplas (nome_do_arquivo, caminho_relativo), em
ordem
  alfabética pelo nome. caminho_relativo é o destino relativo à pasta
destino,
  com barras "/" (ex.: "linha_B/2026-03/laudo_123456_B_2026-03-10.pdf").
- ensaio=True: só devolve o plano. Não cria pastas nem move nada.
- ensaio=False: cria as pastas que faltarem e move os arquivos.
- Nunca sobrescreva: se o arquivo já existir no destino, deixe o original
na
  origem e NÃO inclua esse arquivo na lista devolvida.

Exemplos:
    >>> organizar_laudos(Path("entrada"), Path("laudos"))
    [('laudo_123456_B_2026-03-10.pdf', 'linha_B/2026-
03/laudo_123456_B_2026-03-10.pdf')]

Dica: re.fullmatch com grupos; shutil.move(origem, destino);
Path.relative_to(...).as_posix() dá o caminho relativo com "/".

Teste com:  python curso.py testar 12 ex07
```

Exercício 12.08 — Calcular a largura das colunas de um relatório

Arquivo: `exercicios/ex08_larguras_colunas.py`

```
Nível: ★☆☆

Contexto: planilha gerada por robô com colunas estreitas (aquele "#####"
no
lugar do número) passa impressão de desleixo. Antes de mexer no openpyxl,
vamos resolver a parte de lógica: qual largura cada coluna precisa ter.

Tarefa: implemente calcular_larguras(linhas, minimo=8, maximo=50).
- `linhas` é uma lista de linhas; cada linha é uma lista de valores
  (a primeira costuma ser o cabeçalho).
- Para cada coluna, a largura é o tamanho do MAIOR texto da coluna
  (len(str(valor))) mais 2 de folga.
- None conta como texto vazio ("").
- Linhas podem ter tamanhos diferentes: a quantidade de colunas é a da
linha
  mais comprida; células que faltam contam como vazias.
- Limite cada largura entre `minimo` e `maximo`.
- Sem linhas, devolva [].

Exemplos:
    >>> calcular_larguras([["OP", "Ensaio"], ["123456", "Espessura"]])
    [8, 11]
    >>> calcular_larguras([["Observação"], ["x" * 80]])
    [50]

Dica: monte primeiro uma lista de zeros com o número de colunas e vá
atualizando o máximo de cada posição. max() e min() ajudam a limitar.

Teste com:  python curso.py testar 12 ex08
```

Exercício 12.09 — Relatório Excel de medições, pronto para o gerente

Arquivo: `exercicios/ex09_relatorio_excel.py`

```
Nível: ★★★

Contexto: todo fim de turno alguém copia as medições para uma planilha,
pinta
os reprovados de vermelho e manda por e-mail. Vamos automatizar a
planilha.

Cada medição chega como dicionário:
    {"op": "123456", "linha": "A", "ensaio": "Espessura",
     "valor": 20.4, "minimo": 18.0, "maximo": 22.0}

Tarefa: implemente gerar_relatorio(medicoes, caminho) com openpyxl.
1. Uma planilha (aba) chamada "Medições".
```

```
2. Linha 1 = cabeçalho:
   ["OP", "Linha", "Ensaio", "Valor", "Mínimo", "Máximo", "Status"]
   em negrito, fonte branca ("FFFFFF") e fundo sólido azul "1F4E78".
3. Uma linha por medição, na ordem recebida. Status = "APROVADO" se
   minimo <= valor <= maximo, senão "REPROVADO".
4. Colunas D, E e F (Valor, Mínimo, Máximo) das linhas de dados com
   number_format "0.00".
5. Largura de cada coluna = maior texto da coluna (cabeçalho incluído) +
2.
6. Cabeçalho congelado (freeze_panes = "A2") e filtro ligado em toda a
   tabela (auto_filter.ref = "A1:G<última linha>").
7. Formatação condicional na coluna Status (G2:G<última linha>): regra
   CellIsRule com operator="equal" e formula=['"REPROVADO"'], fonte
   "9C0006" e fundo "FFC7CE". Se não houver medições, pule este item.
8. Crie a pasta de `caminho` se ela não existir, salve e devolva
`caminho`.

Exemplos:
    >>> gerar_relatorio(medicoes, Path("saida/turno.xlsx"))
    PosixPath('saida/turno.xlsx')         # no Windows: WindowsPath(...)

Dica: siga o exemplo 05 da aula, mas escreva com as suas palavras.
Imports úteis: Workbook (openpyxl), Font, PatternFill (openpyxl.styles),
CellIsRule (openpyxl.formatting.rule), get_column_letter
(openpyxl.utils).

Teste com:  python curso.py testar 12 ex09
```

Exercício 12.10 — Montar o e-mail do certificado (sem enviar)

Arquivo: `exercicios/ex10_montar_email.py`

```
Nível: ★★☆

Contexto: depois de emitir o certificado, o laboratório responde à
expedição
com o PDF anexado. Montar a mensagem em Python permite salvá-la como .eml
(para conferir no Outlook) ou entregá-la a um servidor SMTP depois.

Tarefa: implemente montar_email_certificado(remetente, destinatarios, op,
nf, anexo=None).
- Devolva um email.message.EmailMessage com:
  - From = remetente
  - To = os destinatários separados por ", "
  - Subject = "Certificado de qualidade - OP <op> / NF <nf>"
  - corpo em texto puro EXATAMENTE assim (com quebra de linha no fim):
        Olá,

        Segue o certificado de qualidade referente à OP <op> (NF <nf>).
```

```
        Atenciosamente,
        Laboratório de Qualidade
- Se `anexo` (um Path) for informado, anexe o conteúdo do arquivo com o
nome
  original (filename) e o tipo descoberto por mimetypes.guess_type
  (.pdf -> application/pdf). Tipo desconhecido -> application/octet-
stream.
- Lista de destinatários vazia -> ValueError.
- A função NÃO envia nada.

Exemplos:
    >>> msg = montar_email_certificado("lab@x.example",
["exp@x.example"], "123456", "98765")
    >>> msg["Subject"]
    'Certificado de qualidade - OP 123456 / NF 98765'

Dica: msg.set_content(texto) para o corpo; msg.add_attachment(bytes,
maintype=..., subtype=..., filename=...) para o anexo.

Teste com:  python curso.py testar 12 ex10
```

Exercício 12.11 — Ler o pedido da expedição num arquivo .eml

Arquivo: `exercicios/ex11_ler_pedido_eml.py`

```
Nível: ★★☆

Contexto: os pedidos de certificado chegam por e-mail. Salvos como .eml
(ou
exportados por uma regra/fluxo do Outlook), eles viram BYTES que o Python
lê.
Alguns vêm em texto puro, outros só em HTML, e alguns vêm incompletos.

Tarefa: implemente ler_pedido(conteudo), onde `conteudo` são os bytes do
.eml.
- Leia com BytesParser(policy=policy.default).parsebytes(conteudo).
- Pegue o corpo com msg.get_body(preferencelist=("plain", "html")). Se o
  corpo escolhido for HTML (get_content_type() == "text/html"), troque
cada
  tag <...> por um espaço antes de procurar os campos.
- Procure no texto (maiúsculas ou minúsculas):
    "OP: 123456"      -> op = "123456"   (exatamente 6 dígitos)
    "NF: 98765"       -> nf = "98765"    (1 ou mais dígitos)
    "Peso: 1.250,5 kg" -> peso_kg = 1250.5 (formato brasileiro -> float)
  Os dois-pontos são opcionais e pode haver espaços entre as partes.
- Devolva o dicionário:
    {"remetente": <só o endereço, sem o nome>, "assunto": <assunto>,
     "op": ..., "nf": ..., "peso_kg": ...}
```

```
- Se faltar OP, NF ou peso, levante ValueError com a mensagem
  "Campo não encontrado: OP" (ou NF, ou peso), checando nessa ordem.

Exemplos (o arquivo dados/emails/pedido_001.eml):
    >>> ler_pedido(Path("dados/emails/pedido_001.eml").read_bytes())
    {'remetente': 'expedicao@embalagens.example',
     'assunto': 'Pedido de certificado - OP 123456',
     'op': '123456', 'nf': '98765', 'peso_kg': 1250.5}

Dica: email.utils.parseaddr("Nome <a@b.c>") devolve ("Nome", "a@b.c").
Para o peso: tire os pontos de milhar e troque a vírgula por ponto
(módulo 05).

Teste com:  python curso.py testar 12 ex11
```

Exercício 12.12 — Leitura de código: conserte o plano do robô

Arquivo: `exercicios/ex12_corrigir_plano_robo.py`

```
Nível: ★★☆

Contexto: um colega separou o robô em duas partes: uma função que PLANEJA
as
ações (lista de tuplas, sem navegador) e outra que as EXECUTA com o
Playwright.
Ótima ideia: o plano pode ser testado sem abrir o Edge. O problema é que
o
plano dele tem defeitos, e um deles é grave: CONFIRMA a emissão no modo
ensaio.

O código do colega (NÃO copie sem ler; ele tem pelo menos 5 defeitos):

    def planejar_certificado(pedido, ensaio=True, acoes=[]):
        if len(pedido["op"]) != 6:
            raise ValueError("OP inválida")
        acoes.append(("preencher", "OP", pedido["op"]))
        acoes.append(("preencher", "NF", pedido["nf"]))
        acoes.append(("preencher", "Peso (kg)", f"
{pedido['peso_kg']:,.2f}"))
        acoes.append(("selecionar", "Linha", pedido["linha"]))
        if pedido["observacao"]:
            acoes.append(("preencher", "Observação",
pedido["observacao"]))
        acoes.append(("print", f"antes_de_confirmar_{pedido['op']}", ""))
        if ensaio:
            acoes.append(("clicar", "Confirmar", ""))
        else:
            acoes.append(("clicar", "Cancelar", ""))
        return acoes
```

```
Passo 1 (leitura): ANTES de rodar qualquer coisa, leia o código e anote
no
papel o que ele devolve para o pedido do exemplo abaixo, chamado DUAS
vezes
seguidas. Depois confira copiando o código num arquivo de rascunho.

Passo 2 (conserto): escreva abaixo a versão correta, que segue estas
regras:
- A OP deve ter exatamente 6 caracteres e todos dígitos; senão,
ValueError.
- peso_kg deve ser maior que zero; senão, ValueError.
- As ações, nesta ordem:
    ("preencher", "OP", op)
    ("preencher", "NF", nf)
    ("preencher", "Peso (kg)", <peso no formato brasileiro, 2 casas:
"1.250,50">)
    ("selecionar", "Linha", linha)
    ("preencher", "Observação", obs)   <- só se o pedido TIVER a chave
                                          "observacao" com texto não
vazio
    ("print", f"antes_de_confirmar_{op}", "")
    e por último ("clicar", "Cancelar", "") quando ensaio=True,
    ou ("clicar", "Confirmar", "") quando ensaio=False.
- Cada chamada devolve uma lista NOVA (chamadas não podem se misturar).

Exemplos:
    >>> pedido = {"op": "123456", "nf": "98765", "peso_kg": 1250.5,
"linha": "B"}
    >>> planejar_certificado(pedido)
    [('preencher', 'OP', '123456'), ('preencher', 'NF', '98765'),
     ('preencher', 'Peso (kg)', '1.250,50'), ('selecionar', 'Linha',
'B'),
     ('print', 'antes_de_confirmar_123456', ''), ('clicar', 'Cancelar',
'')]

Dica: módulo 04 (a armadilha do valor padrão mutável) e módulo 05
(números no
formato brasileiro). dict.get() devolve None quando a chave não existe.

Teste com:  python curso.py testar 12 ex12
```

Exercício 12.13 — Revisão: fila de emissão idempotente

Arquivo: `exercicios/ex13_revisao_fila_idempotente.py`

```
Nível: ★★★

Revisão de: funções como valores (módulo 04), CSV com ";" e tratamento de
```

```
erros (módulo 06), conjuntos e dicionários (módulo 03) + robôs (este
módulo).

Contexto: o Agendador roda o robô às 7h e às 13h. Se a fila da manhã
ainda
estiver na pasta à tarde, o robô NÃO pode emitir os mesmos certificados
de
novo. E se um pedido falhar (a tela travou), os outros precisam
continuar.
A solução é um "livro de registro": um CSV com tudo o que já foi emitido.

Tarefa: implemente processar_fila(pedidos, emitir, registro).
- `pedidos`: lista de dicionários com pelo menos "op" e "nf".
- `emitir`: uma FUNÇÃO que recebe um pedido e devolve o número do
certificado
  (str), ou levanta uma exceção se algo der errado. Nos testes ela é
falsa; no
  programa real, ela chama o robô.
- `registro`: caminho de um CSV com delimitador ";" e cabeçalho
  op;nf;certificado  (encoding utf-8). Se não existir, crie-o com o
cabeçalho.
- Para cada pedido, na ordem:
  - se o par (op, nf) já está no registro -> não chame emitir; conte
"pulados";
  - senão chame emitir(pedido):
    - deu certo -> acrescente a linha op;nf;certificado no registro NA
HORA
      (não só no fim: se o PC desligar, o que já foi feito está anotado)
e
      conte "emitidos";
    - levantou qualquer Exception -> conte "falhas", não registre, siga
em frente.
- Pedidos repetidos dentro da mesma fila também só são emitidos uma vez.
- Devolva {"emitidos": n, "pulados": n, "falhas": n}.

Exemplos:
    >>> processar_fila(pedidos, emitir, Path("emitidos.csv"))   # 1ª vez
    {'emitidos': 3, 'pulados': 0, 'falhas': 0}
    >>> processar_fila(pedidos, emitir, Path("emitidos.csv"))   # 2ª vez
    {'emitidos': 0, 'pulados': 3, 'falhas': 0}

Dica: leia o registro uma vez para um set de tuplas (op, nf) e vá
acrescentando a esse set. Para gravar uma linha, abra com modo "a" e
newline="" e use csv.writer(arquivo, delimiter=";").

Teste com:  python curso.py testar 12 ex13
```

Exercício 12.14 — Revisão: resumo diário dos pedidos por e-mail

Arquivo: `exercicios/ex14_revisao_resumo_por_dia.py`

```
Nível: ★★☆

Revisão de: números brasileiros e regex (módulo 05), datas e acumuladores
com
dicionário (módulos 03 e 07) + leitura de e-mails (este módulo).

Contexto: o supervisor da expedição quer saber, por dia, quantos pedidos
de
certificado chegaram e quanto peso eles somam. Você tem os e-mails em
bytes.

Tarefa: implemente resumir_por_dia(emails).
- Cada item de `emails` são os bytes de um .eml. Leia com
  BytesParser(policy=policy.default).
- A data de cada pedido vem do cabeçalho "Date": converta com
  email.utils.parsedate_to_datetime e use só a data (.date()), no fuso do
  próprio cabeçalho (não converta fusos).
- O peso vem do corpo em texto puro (msg.get_body(preferencelist=
("plain",)));
  procure "Peso: 1.250,5 kg" (dois-pontos opcional, maiúsculas ou
minúsculas)
  e converta o número brasileiro para float.
- E-mails sem peso no texto puro (ou sem texto puro) são IGNORADOS.
- Agrupe por data: quantidade de pedidos e soma dos pesos.
- Devolva uma lista de textos, do dia mais antigo para o mais recente,
assim:
      "09/03/2026: 3 pedidos, 4.540,75 kg"
      "10/03/2026: 1 pedido, 640,00 kg"
  ("pedido" no singular quando for 1; peso com 2 casas, no formato
brasileiro).
- Lista vazia de e-mails (ou nenhum válido) -> [].

Exemplos:
    >>> resumir_por_dia([eml1, eml2, eml3])
    ['09/03/2026: 2 pedidos, 2.230,75 kg', '10/03/2026: 1 pedido,
2.310,00 kg']

Dica: um dicionário {data: [quantidade, soma]} ou dois defaultdict.
Objetos
date podem ser ordenados com sorted() e formatados com
strftime("%d/%m/%Y").

Teste com:  python curso.py testar 12 ex14
```

Desafio do módulo 12 — Robô da expedição (modo ensaio)

Contexto

Toda manhã chegam pedidos de certificado da expedição por e-mail. Hoje alguém lê um por um, confere a OP no sistema, vê se ela tem especificação de cliente, emite o certificado e avisa o supervisor. Você vai automatizar a parte chata **com segurança**: o robô roda em **modo ensaio** por padrão, não emite nada em dobro e deixa rastro de tudo.

Os e-mails de exemplo estão em `../dados/emails/` (5 arquivos `.eml`, um deles sem NF e outro com uma OP que não existe no cadastro). O cadastro de OPs é `../dados/ops.csv` (separado por `;`):

```
op;linha;produto;especificacao_titulo;especificacao_texto
123456;B;Filme PE 40 µm;Espec. Cliente Alfa;COF 0,20 a 0,30 e espessura
40 µm ± 5%
123789;A;Filme PP 20 µm;;
...
```

Especificação em branco significa "use o padrão da linha".

Requisitos obrigatórios

1. Programa de linha de comando (`argparse`) com as opções `--emails` (pasta), `--ops` (CSV), `--saida` (pasta) e `--confirmar`. Sem `--confirmar`, é **ensaio**.
2. Ler todos os `.eml` da pasta e extrair OP (6 dígitos), NF e peso (formato brasileiro). Corpo em texto puro ou só em HTML.
3. E-mail sem algum campo, ou com OP fora do cadastro, vira **pendência** com o motivo. Uma pendência nunca derruba o programa.
4. Para cada OP válida: linha, produto e especificação (título + texto, ou `Padrão da linha X`).
5. Montar o **plano de ações** do robô de cada pedido. Em ensaio, o plano termina em `Cancelar`; com `--confirmar`, em `Confirmar`.
6. Emissão **idempotente**: um registro `emitidos.csv` na pasta de saída. Rodar duas vezes com `--confirmar` não emite nada de novo. Em ensaio, nada é registrado.
7. Uma emissão que falha é registrada no log e o robô segue para o próximo pedido.
8. Relatório Excel `fila_AAAA-MM-DD.xlsx` com duas abas, `Fila` e `Pendências`, cabeçalho estilizado, larguras ajustadas, filtro, painel congelado e `FALHA` em vermelho (formatação condicional).

9. E-mail-resumo `resumo_AAAA-MM-DD.eml` para o supervisor, com a planilha anexada. **Não envie**: só salve o arquivo.
10. Log em arquivo (`robo_expedicao.log`) e no terminal.
11. Código de saída: `0` se tudo certo, `1` se algum pedido falhou.

Para a emissão, use um **emissor simulado** (uma função ou classe que devolve `000001`, `000002`...). Ele tem a mesma "cara" do robô real: recebe um pedido e devolve o número do certificado.

Extras opcionais

- Ligar o robô Playwright de verdade (`exemplos/09_robo_certificados.py`) no lugar do emissor simulado, rodando contra o simulador `dados/simulador_certificados.html`.
- Mover os `.eml` processados para `processados/` e os com pendência para `revisar/`, sem sobrescrever nada.
- Opção `--desde AAAA-MM-DD` que ignora e-mails mais antigos (cabeçalho `Date`).
- Testes com `pytest` para `ler_email`, `montar_fila` e `processar` (use `tmp_path`).
- Agendar no Agendador de Tarefas do Windows com um `.bat` (aula, seção 10).

Exemplo de execução

```
> python 12-automacao-e-integracao/desafio/solucao_referencia.py
09:54:24 INFO    Início (ENSAIO)
09:54:24 INFO    3 pedido(s) na fila, 2 pendência(s)
09:54:24 WARNING Pendência em pedido_004.eml: NF não encontrado no e-mail
09:54:24 WARNING Pendência em pedido_005.eml: OP 125310 sem cadastro
09:54:24 INFO    OP 123456: plano com 7 ações, termina em Cancelar
09:54:24 INFO    OP 123789: plano com 7 ações, termina em Cancelar
09:54:24 INFO    OP 124001: plano com 7 ações, termina em Cancelar
09:54:24 INFO    Gerados: fila_2026-10-29.xlsx e resumo_2026-10-29.eml

Olá,

Resumo da fila de certificados de 29/10/2026 (MODO ENSAIO: nada foi
emitido).

Pedidos na fila: 3 (4.540,75 kg)
  - OP 123456 / NF 98765: ENSAIO
  - OP 123789 / NF 98766: ENSAIO
  - OP 124001 / NF 98770: ENSAIO

Pendências: 2
  - pedido_004.eml: NF não encontrado no e-mail
  - pedido_005.eml: OP 125310 sem cadastro
```

```
Planilha completa em anexo.

Robô da expedição
```

Com `--confirmar`, os pedidos aparecem como `EMITIDO 000001`, `EMITIDO 000002`... Na segunda execução com `--confirmar`, todos aparecem como `JÁ EMITIDO`.

Critérios de aceite

- [ ] Roda sem argumentos usando os dados de exemplo e termina com código 0.
- [ ] Os 3 pedidos válidos entram na fila; os 2 problemáticos viram pendências com motivo.
- [ ] Em ensaio, nenhum plano contém `Confirmar` e `emitidos.csv` não é criado.
- [ ] Duas execuções seguidas com `--confirmar` geram 3 linhas no registro, não 6.
- [ ] A planilha abre no Excel com as abas `Fila` e `Pendências` formatadas.
- [ ] O `.eml` abre no Outlook com o texto do resumo e a planilha anexada.
- [ ] O log em arquivo conta a história da execução (início, pendências, emissões, fim).
- [ ] Nenhuma senha, e-mail real ou caminho pessoal está escrito no código.
- [ ] `ruff check` e `ruff format --check` passam.

Dicas de como começar

1. Comece pelo **núcleo sem efeitos colaterais**: uma função que recebe os bytes de um `.eml` e devolve um pedido (você já fez isso no exercício 11).
2. Depois, `montar_fila(pasta, ops)` devolvendo `(fila, pendencias)`. Teste no REPL.
3. O plano de ações é o exercício 12; a fila idempotente é o exercício 13.
4. Só no fim junte as saídas (Excel, e-mail, log) e o `argparse`.
5. Use uma `dataclass Pedido` (módulo 08) para não carregar dicionários soltos por aí.
6. Rode em ensaio dezenas de vezes. Só use `--confirmar` quando o ensaio estiver perfeito.

A solução de referência está em `solucao_referencia.py`. Tente antes de abrir.

Quiz — Módulo 12: Automação e integração

Responda sem consultar a aula. Depois confira no gabarito, no fim da página.

---

1. A API do laboratório recebeu um `POST /medicoes` e gravou a medição nova. Qual código de status é o mais adequado na resposta?

a) 200 b) 201 c) 204 d) 302

---

2. O que este código imprime?

```
import requests

pedido = requests.Request(
    "GET", "https://api.lab.local/bobinas", params={"linha": ["A", "B"],
"limite": 5}
).prepare()
print(pedido.url)
```

---

3. Onde está o problema deste trecho de um robô que roda sozinho de madrugada?

```
resposta = requests.get("https://api.lab.local/tolerancias", params=
{"linha": "A"})
dados = resposta.json()
```

---

4. O servidor respondeu `500 Internal Server Error`. O que este código imprime?

```
try:
    resposta = requests.get(url, timeout=10)
    print("deu certo:", resposta.status_code)
except requests.RequestException:
    print("deu erro")
```

a) `deu erro` b) `deu certo: 500` c) nada, o programa trava d) `deu certo: 200`

---

5. Por que a função abaixo é fácil de testar sem internet?

```
def buscar_tolerancias(sessao, base_url, linha, timeout=10):
    resposta = sessao.get(f"{base_url}/tolerancias", params={"linha":
linha}, timeout=timeout)
    ...
```

---

6. O que este código imprime?

```
from pathlib import Path

p = Path("laudo_123456_B.PDF")
print(p.suffix, p.suffix.lower() in {".pdf"}, p.stem)
```

---

7. Você roda `shutil.move(origem, destino)` e já existe um arquivo com o mesmo nome em `destino`. O que pode acontecer, e como o robô deveria se proteger?

---

8. O que este código imprime?

```
from email.message import EmailMessage

msg = EmailMessage()
msg["Subject"] = "teste"
msg.set_content("Olá")
print(msg.get_content_type())
msg.add_attachment(b"%PDF", maintype="application", subtype="pdf",
filename="c.pdf")
print(msg.get_content_type(), len(list(msg.iter_attachments())))
```

---

9. Sua empresa migrou todo mundo para o **Novo Outlook**. O script com `win32com.client.Dispatch("Outlook.Application")` parou de achar os e-mails. Qual alternativa NÃO resolve?

a) Microsoft Graph com um aplicativo registrado pela TI. b) Um fluxo do Power Automate que salva os e-mails numa pasta do OneDrive. c) Instalar o pywin32 de novo. d) Voltar a usar o Outlook clássico naquela máquina, se a empresa permitir.

---

10. Por que `time.sleep(5)` depois de clicar em "Confirmar" é uma má ideia num robô Playwright? O que usar no lugar?

---

11. A tela tem os botões "Confirmar" e "Cancelar". O que acontece com `page.get_by_role("button").click()`?

a) Clica no primeiro botão. b) Clica nos dois. c) Levanta um erro de *strict mode violation*. d) Não faz nada e segue.

---

12. O robô rodou às 7h e emitiu 10 certificados. Às 13h o Agendador roda de novo com os mesmos e-mails na pasta. O que significa dizer que o robô é **idempotente**?

---

13. O script funciona quando você roda no terminal, mas no Agendador de Tarefas falha com `FileNotFoundError: 'dados/ops.csv'`. Qual a causa mais provável e duas formas de corrigir?

---

14. O que é o "modo ensaio" de um robô e por que ele deve ser o padrão?

Gabarito

▼ **Ver respostas**

1. b) `201 Created`. `200` é o "OK" genérico; `204` é sucesso sem corpo (comum em DELETE); `302` é redirecionamento.

2. `https://api.lab.local/bobinas?linha=A&linha=B&limite=5`. Uma lista vira a mesma chave repetida, que é o jeito padrão de mandar vários valores.

3. Falta `timeout=`. Sem ele, se a API não responder, o robô fica esperando para sempre. Também falta `resposta.raise_for_status()` antes do `.json()`: um 404 ou 500 passaria em silêncio.

4. b) `deu certo: 500`. Para o requests, receber uma resposta (mesmo de erro) não é exceção. Só `raise_for_status()` transforma 4xx/5xx em `HTTPError`.

5. Porque a sessão é **injetada**: o teste passa um objeto falso com um método `get` que devolve uma resposta pronta. Nada sai do computador. É injeção de dependência; a outra técnica é `monkeypatch`.

6. `.PDF True laudo_123456_B`. `.suffix` mantém as maiúsculas do nome; por isso comparamos com `.lower()`.

7. O arquivo existente pode ser **sobrescrito sem aviso** (a documentação do `shutil.move` diz que isso depende do sistema). Antes de mover, verifique `destino.exists()` e pule, ou escolha outro nome (`relatorio (2).xlsx`, exercício 06).

8. `text/plain`, depois `multipart/mixed 1`. Ao anexar, a mensagem vira "multipart" sozinha: o texto vira uma parte e o PDF, outra.

9. c). O Novo Outlook não tem a interface COM que o `win32com` usa. Reinstalar o pywin32 não muda isso.

10. Às vezes 5 s é pouco (o ERP estava lento e o robô segue errado), às vezes é demais (robô lento à toa). Use as esperas do Playwright: as ações já esperam o elemento ficar pronto, e `expect(...).to_have_text(...)` espera até a condição ser verdadeira, com limite de tempo.

11. c). Locators do Playwright são "estritos": se a busca encontrar mais de um elemento para uma ação, ele recusa em vez de chutar. Seja específico: `get_by_role("button", name="Confirmar")`.

12. Rodar de novo não muda o resultado: nenhum certificado é emitido em dobro. O robô consulta um registro (CSV, banco) do que já fez antes de agir, e anota logo depois de agir.

13. O Agendador começa em outra pasta (normalmente `C:\Windows\System32`), e o caminho relativo `dados/ops.csv` é procurado lá. Correções: no `.bat`, entrar na pasta do projeto (`cd /d C:\robos\...`) ou preencher "Iniciar em"; no código, montar caminhos a partir do script: `Path(__file__).resolve().parent / "dados" / "ops.csv"`.

14. É o modo em que o robô faz tudo (lê, valida, preenche, tira print), menos o passo irreversível (Confirmar). Deve ser o padrão para que um esquecimento (rodar sem pensar) nunca emita nada; emitir de verdade exige um pedido explícito (`--confirmar`).

Módulo 13 — APIs web com FastAPI

> **Tempo estimado:** 12 a 16 horas · **Pré-requisitos:** módulos 00 a 12 · **Nível:** intermediário

O que você vai aprender

- Explicar o que é uma API REST: recursos, métodos, códigos de status e JSON.
- Criar uma API com FastAPI, rodar com `uvicorn` e usar a documentação automática `/docs`.
- Testar rotas com `TestClient`, sem subir servidor.
- Receber parâmetros de caminho e de consulta, com tipos e validação (`Query`, `Enum`).
- Receber e validar o corpo de uma requisição com Pydantic v2 (`BaseModel`, `Field`, `Literal`, `field_validator`, `model_validator`) e controlar o que sai (modelo de resposta).
- Responder com o código de status certo e levantar `HTTPException`.
- Organizar a API em módulos com `APIRouter`.
- Usar dependências (`Depends`) para chave de acesso e para a conexão com o banco, e trocá-las nos testes com `dependency_overrides`.
- Guardar os dados em SQLite com uma dependência que abre e fecha a conexão.
- Consumir a sua API com `requests`, como um robô ou outro sistema faria.

Por que isso importa

No módulo 12 você foi **cliente**: chamou APIs dos outros. Agora você vai ser o **servidor**: a regra de tolerância do laboratório vira um serviço que qualquer programa pode consultar. O robô de certificados pergunta "a OP 123456 está liberada?", o Power BI puxa as medições, a TI integra com o ERP. Todos usando a **mesma** regra, num lugar só.

FastAPI é um dos frameworks web mais usados em Python, especialmente em dados, automação e aprendizado de máquina. Vagas de backend Python citam FastAPI com frequência. E ele ensina boas práticas de graça: tipos, validação, documentação e testes.

No projeto final, a API é uma das extensões: o núcleo do Validador de Certificados (módulos anteriores) ganha uma "porta" HTTP.

1. O que é uma API REST

Uma API web é um programa que fica esperando pedidos HTTP e responde, quase sempre, com JSON. REST é um **estilo** de organizar esses pedidos. A ideia central: o endereço diz **qual recurso** e o método diz **o que fazer** com ele.

```
  MÉTODO   CAMINHO                    SIGNIFICADO
  GET      /linhas/B/tolerancias      ler as tolerâncias da linha B
  POST     /medicoes                  registrar uma medição nova
  GET      /medicoes/15               ler a medição 15
  PUT      /regras/B/espessura        criar ou trocar a regra de
espessura da linha B
  PATCH    /ocorrencias/3/encerrar    mudar parte de uma ocorrência
  DELETE   /amostras/7                apagar a amostra 7
```

Convenções que você vai ver em toda API bem feita:

- Caminhos com **substantivos no plural** (`/medicoes`), não verbos (`/registrarMedicao`).
- O **método** carrega a ação. `GET` nunca altera nada.
- O **status** conta o resultado: 200 deu certo, 201 criou, 404 não existe, 422 dados inválidos (a tabela completa está no módulo 12, seção 2).
- Erros também respondem JSON, com uma mensagem útil.

Uma analogia: a API é o **balcão do laboratório**. Quem chega não entra na sala dos equipamentos. Faz um pedido no formato combinado ("quero o laudo da OP 123456") e recebe a resposta no formato combinado. O que acontece lá dentro (planilha, banco, cálculo) pode mudar sem que o cliente perceba.

2. A primeira API

Instale (já está no `requirements.txt` do curso):

```
python -m pip install fastapi uvicorn httpx
```

- `fastapi`: o framework.
- `uvicorn`: o servidor que recebe as conexões HTTP e entrega ao FastAPI.
- `httpx`: usado pelo `TestClient` nos testes.

O menor programa útil:

```
from fastapi import FastAPI

```

```python
app = FastAPI(title="API do Laboratório", version="0.1.0")


@app.get("/")
def raiz() -> dict:
    return {"mensagem": "API do laboratório no ar"}


@app.get("/saude")
def saude() -> dict:
    return {"status": "ok"}
```

Leia assim: `@app.get("/saude")` **registra** a função logo abaixo. Quando chegar um `GET /saude`, o FastAPI chama `saude()` e converte o dicionário devolvido em JSON. O `@` é um **decorador**: um jeito curto de dizer "passe esta função para o app guardar".

Para subir o servidor, na pasta do arquivo:

```
uvicorn 01_primeira_api:app --reload
```

`01_primeira_api` é o nome do arquivo (sem `.py`), `app` é a variável com o `FastAPI()` e `--reload` reinicia sozinho quando você salva. Saída típica:

```
INFO:     Started server process [880]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:39330 - "GET / HTTP/1.1" 200 OK
```

Abra `http://127.0.0.1:8000/saude` no navegador e veja o JSON. Depois abra `http://127.0.0.1:8000/docs`: uma página interativa com todas as rotas, gerada automaticamente. Clique numa rota, em **Try it out** e em **Execute**. Para parar o servidor: `Ctrl+C` no terminal.

> 💡 **Dica:** os exemplos deste módulo têm dois modos. `python exemplos/01_primeira_api.py` faz uma **demonstração** sem servidor; com `--servir`, sobem o uvicorn em `http://127.0.0.1:8000`.

> 🔮 **Preveja antes de rodar:** além de `GET /` e `GET /saude`, o exemplo 01 faz `GET /nada` e `POST /`. Que status cada um recebe?

▼ **Resposta**

Saída de `python exemplos/01_primeira_api.py`:

```
GET  /       -> 200 {'mensagem': 'API do laboratório no ar'}
GET  /saude  -> 200 {'status': 'ok'}
GET  /nada   -> 404 {'detail': 'Not Found'}
POST /       -> 405 {'detail': 'Method Not Allowed'}
Título na documentação: API do Laboratório 0.1.0
Rotas documentadas: ['/', '/saude']
```

`/nada` não existe: 404. O caminho `/` existe, mas só para `GET`: 405, "método não permitido". Esses dois erros o FastAPI gera sozinho.

🔧 **Modifique:** acrescente ao exemplo 01 uma rota `GET /versao` que devolve `{"versao": "0.1.0"}`. Rode com `--servir` e confira no `/docs` que ela apareceu.

💡 **Dica:** você vai ver `async def` em muitos tutoriais. Use `def` comum, como neste curso, sempre que a rota usar bibliotecas "normais" (SQLite, arquivos, pandas). O FastAPI roda essas funções em threads e tudo funciona. `async def` só faz sentido com bibliotecas assíncronas, assunto para depois do curso.

3. Testando com TestClient

Subir o servidor e clicar no `/docs` é ótimo para explorar, mas não para conferir 30 casos toda vez que você muda algo. O `TestClient` é um "navegador de mentira": conversa com o `app` direto, sem rede e sem servidor.

```python
from fastapi.testclient import TestClient

cliente = TestClient(app)
resposta = cliente.get("/saude")
print(resposta.status_code, resposta.json())
```

```
200 {'status': 'ok'}
```

A resposta tem a mesma "cara" da resposta do `requests` (módulo 12): `.status_code`, `.json()`, `.text`, `.headers`. Para enviar dados: `cliente.post("/medicoes", json={...})`, `cliente.get("/bobinas", params={"linha": "C"})`, `cliente.get("/eu", headers={"X-Chave": "lab-123"})`.

Exemplo resolvido: o teste de uma rota

Problema: quero garantir que `GET /saude` responde `{"status": "ok"}`.

Como pensei: todo teste tem três partes (módulo 09, padrão AAA):

1. **Arrange (preparar):** criar o cliente.
2. **Act (agir):** fazer a requisição.
3. **Assert (conferir):** primeiro o status, depois o corpo. Conferir o status antes dá uma mensagem de erro muito mais clara quando a rota nem existe.

```
from fastapi.testclient import TestClient

from main import app

cliente = TestClient(app)


def test_saude():
    resposta = cliente.get("/saude")                # agir
    assert resposta.status_code == 200              # conferir o status
    assert resposta.json() == {"status": "ok"}      # conferir o corpo
```

É exatamente o molde dos testes dos exercícios deste módulo. Abra `testes/test_m13_ex01_ola_api.py` e compare.

4. Parâmetros de caminho

Um pedaço do endereço pode ser variável: `/bobinas/3`, `/linhas/B/tolerancias`. Você marca o pedaço com chaves e recebe um parâmetro com o **mesmo nome**:

```
@app.get("/bobinas/{numero}")
def ver_bobina(numero: int) -> dict:
    for bobina in BOBINAS:
        if bobina["numero"] == numero:
            return bobina
    return {"aviso": f"bobina {numero} não existe (no exemplo 04 isso
vira um 404)"}
```

O type hint `int` não é enfeite. O FastAPI usa o tipo para **converter** (`"3"` vira `3`) e para **validar** (`"tres"` é recusado com 422, sem nem chamar a sua função).

> 🔮 **Preveja antes de rodar:** e se você esquecer o tipo?

```python
@app.get("/dobro/{numero}")
def dobro(numero):
    return {"dobro": numero * 2}

print(TestClient(app).get("/dobro/21").json())
```

▼ **Resposta**

```
{'dobro': '2121'}
```

Sem tipo, tudo que vem do endereço é **texto**. E `"21" * 2` repete o texto (módulo 01). Com `numero: int`, a resposta seria `{'dobro': 42}`.

Para aceitar só alguns valores, use um `Enum` (módulo 08). O `/docs` até mostra uma lista de escolha:

```python
class Linha(str, Enum):
    A = "A"
    B = "B"
    C = "C"
    D = "D"


@app.get("/linhas/{linha}/tolerancias")
def tolerancias_da_linha(linha: Linha) -> dict:
    return {"linha": linha.value, "tolerancias":
TOLERANCIAS[linha.value]}
```

Herdar de `str` faz cada membro do Enum se comportar como texto (e sair como texto no JSON). Rode `python exemplos/02_parametros.py`. As primeiras linhas:

```
GET /linhas/B/tolerancias
    200 {'linha': 'B', 'tolerancias': {'espessura': [38.0, 42.0],
'gramatura': [36.5, 39.5]}}
GET /linhas/E/tolerancias
    422 {'detail': [{'type': 'enum', 'loc': ['path', 'linha'], 'msg':
"Input should be 'A', 'B', 'C' or 'D'", 'input': 'E', 'ctx': {'expected':
"'A', 'B', 'C' or 'D'"}}]}
GET /bobinas/3
    200 {'numero': 3, 'op': '123789', 'linha': 'B', 'peso_kg': 640.0}
GET /bobinas/tres
    422 {'detail': [{'type': 'int_parsing', 'loc': ['path', 'numero'],
```

```
'msg': 'Input should be a valid integer, unable to parse string as an
integer', 'input': 'tres'}]}
```

Aprenda a ler o 422: `loc` diz **onde** (`path`, `query` ou `body`, e o nome do campo), `msg` diz **o quê**, `input` mostra o valor recebido.

> 💡 **Dica:** uma alternativa mais leve ao `Enum` é `typing.Literal["A", "B", "C", "D"]`: mesmo efeito, sem criar classe. Usamos os dois neste módulo.

5. Parâmetros de consulta

Parâmetro da função que **não** aparece no caminho vira parâmetro de consulta (a parte depois do `?`). Com valor padrão, é opcional:

```python
@app.get("/bobinas")
def listar_bobinas(
    linha: Linha | None = None,                          # opcional
    peso_min: Annotated[float, Query(ge=0)] = 0,         # ge = maior ou
igual
    limite: Annotated[int, Query(ge=1, le=100)] = 10,    # entre 1 e 100
) -> list[dict]:
    resultado = [
        b for b in BOBINAS
        if (linha is None or b["linha"] == linha.value) and b["peso_kg"]
>= peso_min
    ]
    return resultado[:limite]
```

`Annotated[tipo, Query(...)]` junta o tipo com regras extras. As mais usadas: `ge`, `gt`, `le`, `lt` para números; `min_length`, `max_length` e `pattern` para textos. O resto do exemplo 02:

```
GET /bobinas?linha=C
    200 [{'numero': 4, 'op': '124001', 'linha': 'C', 'peso_kg': 1210.0},
{'numero': 5, 'op': '124001', 'linha': 'C', 'peso_kg': 1100.0}]
GET /bobinas?peso_min=500&limite=2
    200 [{'numero': 3, 'op': '123789', 'linha': 'B', 'peso_kg': 640.0},
{'numero': 4, 'op': '124001', 'linha': 'C', 'peso_kg': 1210.0}]
GET /bobinas?limite=0
    422 {'detail': [{'type': 'greater_than_equal', 'loc': ['query',
'limite'], 'msg': 'Input should be greater than or equal to 1', 'input':
'0', 'ctx': {'ge': 1}}]}
```

⚠️ **Erro comum:** declarar `linha: str` sem valor padrão achando que é opcional. Sem padrão, o parâmetro é **obrigatório**: chamar `/bobinas` sem `?linha=` dá 422. Para opcional: `linha: str | None = None`.

🔮 **Preveja antes de rodar:** o que acontece com o parâmetro `extra`?

```
@app.get("/x/{linha}")
def x(linha: str, limite: int = 10):
    return {"linha": linha, "limite": limite}

print(TestClient(app).get("/x/A?limite=3&extra=1").json())
```

▼ **Resposta**

```
{'linha': 'A', 'limite': 3}
```

Parâmetros de consulta que a função não declara são simplesmente ignorados. Não dá erro, o que também significa que um erro de digitação (`?limte=3`) passa em silêncio e o padrão 10 é usado. Teste os nomes!

🔧 **Modifique:** acrescente ao exemplo 02 um filtro opcional `op` (texto) em `/bobinas`. Teste `/bobinas?op=124001` e `/bobinas?op=124001&limite=1`.

6. Corpo com Pydantic

Dados maiores (uma medição, um pedido de certificado) vão no **corpo** da requisição, em JSON. Você descreve o formato com uma classe Pydantic. O FastAPI entende: parâmetro cujo tipo é um `BaseModel` vem do corpo.

```
from typing import Literal

from pydantic import BaseModel, Field, field_validator


class Medicao(BaseModel):
    op: str = Field(pattern=r"^\d{6}$", examples=["123456"])
    linha: Literal["A", "B", "C", "D"]
    ensaio: str = Field(min_length=2)
    valor: float = Field(gt=0)
```

```
    unidade: str = "µm"                          # com padrão: opcional

    @field_validator("ensaio")
    @classmethod
    def padronizar_ensaio(cls, valor: str) -> str:
        return valor.strip().lower()             # " Espessura " ->
"espessura"


@app.post("/medicoes/avaliar")
def avaliar(medicao: Medicao) -> Resultado:
    ...
```

O que cada peça faz:

- **Tipo** (`str`, `float`, `Literal[...]`): o Pydantic converte e valida.
- **`Field(...)`**: regras extras (`gt`, `ge`, `min_length`, `pattern`) e metadados (`examples`, que aparecem no `/docs`).
- **Valor padrão**: torna o campo opcional.
- **`@field_validator("campo")`**: sua própria regra para um campo. Pode transformar o valor (devolve o novo) ou recusar (levanta `ValueError`). Por padrão roda **depois** da conversão de tipo; com `mode="before"`, recebe o dado cru (útil para aceitar `"1.250,5"`, exercício 12).
- **`@model_validator(mode="after")`**: regra que envolve **vários** campos ao mesmo tempo, como "mínimo não pode ser maior que máximo". Recebe o objeto pronto (`self`) e deve terminar com `return self`.

O modelo funciona sozinho, fora da API. Isso é ótimo para testar e para usar no resto do projeto (o robô do módulo 12 poderia validar os pedidos com ele). Saída da primeira parte de `python exemplos/03_corpo_pydantic.py`:

```
1) O modelo funciona sozinho, sem API:
   Medicao(op='123456', linha='A', ensaio='espessura', valor=20.4,
unidade='µm')
   model_dump(): {'op': '123456', 'linha': 'A', 'ensaio': 'espessura',
'valor': 20.4, 'unidade': 'µm'}
   ValidationError com 3 erros:
    - ('op',) String should match pattern '^\d{6}$'
    - ('linha',) Input should be 'A', 'B', 'C' or 'D'
    - ('valor',) Input should be greater than 0
```

Repare: `valor="20.4"` (texto) virou `20.4`, e `"  Espessura "` virou `"espessura"`. E o Pydantic lista **todos** os erros de uma vez, não só o primeiro.

> 🔮 **Preveja antes de rodar:** o que sai deste `model_dump()`?

```python
class M(BaseModel):
    op: str
    valor: float = Field(gt=0)

print(M(op="123456", valor=1, cor="azul").model_dump())
```

▼ **Resposta**

```
{'op': '123456', 'valor': 1.0}
```

Campos que o modelo não conhece (`cor`) são ignorados, e o `1` inteiro vira `1.0` porque o campo é `float`. Se quiser **recusar** campos desconhecidos, acrescente `model_config = ConfigDict(extra="forbid")` na classe.

Modelo de resposta. O tipo de retorno da rota (`-> Resultado`) é o **contrato de saída**: o FastAPI valida o que sai e **remove** o que não está no modelo. No exemplo 03, a função devolve um campo `uso_interno` que nunca chega ao cliente:

```
2) Pela API:
   200 {'op': '123456', 'ensaio': 'cof', 'valor': 0.31, 'minimo': 0.2,
'maximo': 0.3, 'aprovado': False}
   422 [['body', 'ensaio'], ['body', 'valor']]
```

Isso protege você de vazar sem querer um campo interno (uma senha, um custo). O segundo pedido mandou só `op` e `linha`: o 422 aponta os dois campos obrigatórios que faltaram.

> 💡 **Dica:** o Pydantic v2 usa `model_dump()` e `model_validate()`. Tutoriais antigos (Pydantic v1) usam `.dict()` e `parse_obj()`; se você encontrar isso, traduza.

7. Códigos de status e HTTPException

Por padrão, toda resposta de sucesso é 200. Para dizer outra coisa:

Situação	Status	Como
Criou algo novo	201	`@app.post(..., status_code=201)`
Apagou, nada a devolver	204	`@app.delete(..., status_code=204)` e a função devolve `None`

Situação	Status	Como
Não existe	404	`raise HTTPException(status_code=404, detail="...")`
Conflita com o estado atual	409	`raise HTTPException(status_code=409, detail="...")`
Sem chave / chave inválida	401	idem
Sem permissão	403	idem
Dados inválidos	422	automático (Pydantic)

O módulo `fastapi.status` tem nomes para os números (`status.HTTP_201_CREATED`, `status.HTTP_404_NOT_FOUND`), que deixam o código mais legível.

`HTTPException` é uma exceção: `raise` interrompe a função na hora, em qualquer profundidade, e o FastAPI monta a resposta `{"detail": "..."}` com o status escolhido.

🔮 **Preveja antes de rodar:** um colega escreveu `return` em vez de `raise`. Que status o cliente recebe para `/linhas/Z`?

```python
@app.get("/linhas/{codigo}")
def ver(codigo: str):
    if codigo != "A":
        return HTTPException(status_code=404, detail="Linha não encontrada")
    return {"codigo": "A"}
```

▼ **Resposta**

```
200 {"status_code":404,"detail":"Linha não encontrada","headers":null}
```

200! Com `return`, a exceção vira um objeto comum, que o FastAPI converte em JSON e devolve como sucesso. O cliente que só olha o status acha que deu certo. Exceção se **levanta** (`raise`), não se devolve.

Exemplo resolvido: um cadastro com os status certos

Problema: cadastro de amostras em memória: criar, ver, listar e apagar, cada operação com o status que conta o que aconteceu.

Como pensei:

1. Separei o que o cliente **envia** (`AmostraEntrada`: código, OP, descrição) do que a API **devolve** (`Amostra`: tudo isso mais o `id`). Herança (módulo 08) evita repetir.
2. Três rotas precisam de "ache a amostra ou responda 404". Virou uma função auxiliar, `buscar_ou_404`, que levanta a `HTTPException`. A exceção "sobe" pela rota que chamou.
3. Código repetido não é erro de dados, é **conflito** com o que já existe: 409.
4. O novo id é o maior existente + 1 (`max(AMOSTRAS, default=0) + 1`; o `default` evita erro com o dicionário vazio).

```
def buscar_ou_404(id_amostra: int) -> Amostra:
    if id_amostra not in AMOSTRAS:
        raise HTTPException(status.HTTP_404_NOT_FOUND, detail=f"Amostra
{id_amostra} não existe")
    return AMOSTRAS[id_amostra]


@app.post("/amostras", status_code=status.HTTP_201_CREATED)
def criar_amostra(entrada: AmostraEntrada) -> Amostra:
    if any(a.codigo == entrada.codigo for a in AMOSTRAS.values()):
        raise HTTPException(
            status.HTTP_409_CONFLICT, detail=f"Já existe amostra com
código {entrada.codigo}"
        )
    novo_id = max(AMOSTRAS, default=0) + 1
    amostra = Amostra(id=novo_id, **entrada.model_dump())
    AMOSTRAS[novo_id] = amostra
    return amostra


@app.delete("/amostras/{id_amostra}",
status_code=status.HTTP_204_NO_CONTENT)
def apagar_amostra(id_amostra: int) -> None:
    buscar_ou_404(id_amostra)
    del AMOSTRAS[id_amostra]
```

Saída de `python exemplos/04_status_e_erros.py`:

```
POST   /amostras    -> 201 {'codigo': 'AM-001', 'op': '123456',
'descricao': 'Bobina 3', 'id': 1}
POST   /amostras    -> 201 {'codigo': 'AM-002', 'op': '123789',
'descricao': '', 'id': 2}
POST   /amostras    -> 409 {'detail': 'Já existe amostra com código AM-
001'}
GET    /amostras/2  -> 200 {'codigo': 'AM-002', 'op': '123789',
'descricao': '', 'id': 2}
```

```
DELETE /amostras/2  -> 204 (sem corpo)
DELETE /amostras/2  -> 404 {'detail': 'Amostra 2 não existe'}
GET    /amostras    -> 200 [{'codigo': 'AM-001', 'op': '123456',
'descricao': 'Bobina 3', 'id': 1}]
```

🔧 **Modifique:** acrescente uma rota `PATCH /amostras/{id_amostra}/descricao` que recebe `{"descricao": "..."}` e troca só a descrição. Que status ela deve devolver quando a amostra não existe? (O exercício 07 pratica `PATCH` com outra regra de negócio.)

⚠️ **Erro comum:** guardar dados num dicionário global e esperar que eles fiquem lá. Eles **somem** quando o servidor reinicia (e o `--reload` reinicia a cada vez que você salva). Para dados de verdade, banco: seção 10.

8. Organizando em módulos

Uma API real tem dezenas de rotas. Tudo num arquivo vira bagunça. A ferramenta é o `APIRouter`: um "mini-app" por assunto, que depois é plugado no app principal.

```python
# rotas/tolerancias.py
from fastapi import APIRouter

router = APIRouter(prefix="/linhas", tags=["tolerâncias"])


@router.get("/{linha}/tolerancias", summary="Tolerâncias de uma linha")
def listar_tolerancias(linha: Linha, con: Conexao) -> list[Tolerancia]:
    ...
```

```python
# main.py
from fastapi import FastAPI

from .rotas import medicoes, tolerancias

app = FastAPI(title="API do Laboratório de Qualidade", version="1.0.0")
app.include_router(tolerancias.router)
app.include_router(medicoes.router)
```

`prefix` é colado na frente de todas as rotas do router; `tags` agrupa as rotas no `/docs`. O exemplo 07 tem uma API completa organizada assim:

```
exemplos/api_laboratorio/
  __init__.py
  main.py            cria o app e inclui os routers
  banco.py           conexão SQLite, tabelas e dados iniciais
  modelos.py         modelos Pydantic (o que entra e o que sai)
  regras.py          lógica pura de tolerância (sem FastAPI, sem banco)
  rotas/
    __init__.py
    tolerancias.py   GET /linhas/{linha}/tolerancias
    medicoes.py      POST /medicoes e GET /ops/{op}/situacao
```

Repare em `regras.py`: a regra de negócio (faixa ou nominal ± %) não sabe que existe FastAPI nem banco. É a mesma ideia do robô do módulo 12: o núcleo puro no meio, as "bordas" (HTTP, banco) em volta. Assim a regra pode ser testada e reaproveitada.

Dentro do pacote, os imports são **relativos** (`from ..banco import Conexao`: dois pontos = "o pacote de cima"). Para subir o servidor, de dentro de `exemplos`:

```
uvicorn api_laboratorio.main:app --reload
```

> ⚠️ **Erro comum:** declarar `@app.get("/linha/{codigo}")` **antes** de `@app.get("/linha/especial")`. As rotas são testadas na ordem em que foram declaradas, e `{codigo}` aceita qualquer texto, inclusive `especial`. Declare as rotas fixas antes das rotas com parâmetro.

9. Dependências

Várias rotas precisam da mesma coisa antes de começar: conferir a chave de acesso, abrir a conexão com o banco, carregar as regras. Repetir esse código em cada rota é pedir para esquecer em uma. A solução do FastAPI é a **injeção de dependência** com `Depends`, a mesma ideia do módulo 12 ("receba a sessão como parâmetro"), só que automática.

```
CHAVES = {"lab-123": "Ana (laboratório)", "exp-456": "Carla (expedição)"}


def usuario_atual(x_chave: Annotated[str | None, Header()] = None) ->
str:
    if x_chave not in CHAVES:
        raise HTTPException(status_code=401, detail="Chave de acesso
inválida ou ausente")
    return CHAVES[x_chave]
```

```
def obter_tolerancias() -> dict:
    return TOLERANCIAS


Usuario = Annotated[str, Depends(usuario_atual)]
Tolerancias = Annotated[dict, Depends(obter_tolerancias)]


@app.get("/tolerancias/{linha}")
def ver_tolerancias(linha: str, usuario: Usuario, tolerancias:
Tolerancias) -> dict:
    ...
```

Como funciona: antes de chamar `ver_tolerancias`, o FastAPI chama `usuario_atual` e `obter_tolerancias` e passa os resultados. Se `usuario_atual` levantar 401, a rota nem roda. `Header()` lê um cabeçalho HTTP; o nome `x_chave` vira o cabeçalho `x-chave` (o FastAPI troca `_` por `-`).

O melhor vem nos testes: `app.dependency_overrides` troca uma dependência por outra sem tocar nas rotas. Saída de `python exemplos/05_dependencias.py`:

```
sem chave     -> {'detail': 'Chave de acesso inválida ou ausente'}
chave errada  -> 401
chave certa   -> {'usuario': 'Carla (expedição)'}
tolerâncias A -> {'linha': 'A', 'consultado_por': 'Ana (laboratório)',
'tolerancias': {'espessura': [18.0, 22.0]}}
com override  -> {'espessura': [19.0, 21.0]}
```

A última linha veio de:

```
app.dependency_overrides[obter_tolerancias] = lambda: {"A": {"espessura":
[19.0, 21.0]}}
...
app.dependency_overrides.clear()   # sempre desfaça no fim
```

> ⚠️ **Erro comum:** confundir 401 com 403. **401**: "não sei quem você é" (sem chave ou chave inválida). **403**: "sei quem você é, mas você não pode fazer isso" (chave só de leitura tentando alterar regra). O exercício 09 pratica os dois.

> 💡 **Dica:** chave no cabeçalho é um bom começo para uma API interna. Para usuários de verdade, o FastAPI tem suporte a OAuth2 e tokens JWT (veja "Security" na documentação). E chaves nunca ficam no código: leia de variável

> de ambiente.

10. Persistência com SQLite

Para os dados sobreviverem ao reinício, banco. Você já sabe SQLite (módulo 11). A pergunta é: **como a rota recebe a conexão?** Resposta: uma dependência com `yield`.

Exemplo resolvido: a conexão como dependência

Como pensei:

1. Cada requisição deve ter a **sua** conexão: abrir no começo, fechar no fim, mesmo se a rota der erro. Isso é um `try/finally` (módulo 06).
2. Uma dependência com `yield` faz exatamente isso: o código **antes** do `yield` roda antes da rota; o `finally` roda **depois** que a resposta foi montada.
3. O FastAPI pode preparar a dependência numa thread e rodar a rota em outra. O `sqlite3`, por segurança, recusa usar a conexão em outra thread. `check_same_thread=False` desliga essa trava, o que é seguro aqui porque cada requisição tem a sua conexão.
4. `row_factory = sqlite3.Row` faz as linhas funcionarem como dicionário (`linha["valor"]`), o que facilita montar os modelos.
5. Um apelido `Conexao` deixa as assinaturas das rotas curtas.

```
def obter_conexao():
    CAMINHO_BANCO.parent.mkdir(parents=True, exist_ok=True)
    con = sqlite3.connect(CAMINHO_BANCO, check_same_thread=False)
    con.row_factory = sqlite3.Row
    try:
        criar_tabela(con)
        yield con
    finally:
        con.close()


Conexao = Annotated[sqlite3.Connection, Depends(obter_conexao)]


@app.post("/medicoes", status_code=status.HTTP_201_CREATED)
def criar_medicao(entrada: MedicaoEntrada, con: Conexao) -> Medicao:
    with con:  # transação: commit se tudo der certo, rollback se der
erro
        cursor = con.execute(
            "INSERT INTO medicoes (op, ensaio, valor) VALUES (?, ?, ?)",
```

```
            (entrada.op, entrada.ensaio, entrada.valor),
        )
    return Medicao(id=cursor.lastrowid, **entrada.model_dump())
```

Saída de `python exemplos/06_sqlite.py`:

```
POST -> 201 {'op': '123456', 'ensaio': 'espessura', 'valor': 20.4, 'id':
1}
POST -> 201 {'op': '123456', 'ensaio': 'cof', 'valor': 0.27, 'id': 2}
POST -> 201 {'op': '123789', 'ensaio': 'espessura', 'valor': 39.8, 'id':
3}
GET /medicoes?op=123456 -> [{'op': '123456', 'ensaio': 'espessura',
'valor': 20.4, 'id': 1}, {'op': '123456', 'ensaio': 'cof', 'valor': 0.27,
'id': 2}]
Depois do reinício, /medicoes/3 -> {'op': '123789', 'ensaio':
'espessura', 'valor': 39.8, 'id': 3}
GET /medicoes/99 -> 404
```

O "reinício" é um segundo `TestClient`: os dados continuam, porque estão no arquivo `exemplos/saida/medicoes.db`. Nos testes, você troca `obter_conexao` por um banco **em memória** com `dependency_overrides`; cada teste começa limpo e nada é gravado em disco (exercício 11).

> ⚠️ **Erro comum:** montar o SQL com f-string: `f"SELECT * FROM medicoes WHERE op = '{op}'"`. Numa API isso é ainda mais grave que num script, porque **qualquer um** na rede pode mandar um `op` malicioso (SQL injection, módulo 11). Sempre `?` e a tupla de parâmetros.

> 🔧 **Modifique:** acrescente ao exemplo 06 a rota `DELETE /medicoes/{id_medicao}` (204, ou 404 se não existir). Dica: `cursor.rowcount` diz quantas linhas o `DELETE` apagou.

> 💡 **Dica:** quando o projeto crescer, existem bibliotecas que mapeiam tabelas para classes (SQLAlchemy, SQLModel, este último do mesmo autor do FastAPI). Para começar e para entender o que elas fazem por baixo, `sqlite3` puro é o melhor professor.

11. Documentação automática

O FastAPI gera, a partir do seu código, um documento padrão chamado **OpenAPI** (`/openapi.json`) e duas telas em cima dele:

- `/docs` (Swagger UI): interativa, com **Try it out**.
- `/redoc`: só leitura, boa para imprimir ou mandar para alguém.

Tudo que você escreve vira documentação:

No código	No /docs
`FastAPI(title=..., version=..., description=...)`	cabeçalho da página
`tags=["medições"]` (rota ou router)	grupos de rotas
`summary="Registra e avalia uma medição"`	título da rota
docstring da função da rota	descrição da rota
tipos e `Field(examples=[...])` dos modelos	esquema e exemplo do corpo
tipo de retorno	esquema da resposta

Saída do final de `python exemplos/07_api_organizada.py`:

```
Rotas: /linhas/{linha}/tolerancias, /medicoes, /ops/{op}/situacao, /saude
```

> 💡 **Dica:** mande o link do `/docs` para quem vai usar a sua API (a TI, o pessoal do BI). É documentação que nunca fica desatualizada, porque **é** o código.

12. Rodando de verdade: uvicorn e um cliente requests

Juntando com o módulo 12: o exemplo 08 sobe a API do laboratório num uvicorn **de verdade** (numa thread, numa porta livre) e conversa com ela por HTTP com `requests`, como o robô de certificados faria:

```
servidor = uvicorn.Server(uvicorn.Config(app, host="127.0.0.1",
port=porta, log_level="warning"))
threading.Thread(target=servidor.run, daemon=True).start()
...
resposta = sessao.post(f"{base}/medicoes", json=medicao, timeout=5)
resposta.raise_for_status()
```

Saída de `python exemplos/08_cliente_requests.py`:

```
saúde: 200 {'status': 'ok'}
medição: 201 aprovado = False
situação: {'op': '124001', 'medicoes': 1, 'reprovadas': 1, 'liberada':
False}
```

```
pedido inválido: 422 campos com problema: ['op', 'linha', 'ensaio',
'valor']
```

Detalhes para quando for usar no trabalho:

- `uvicorn main:app --reload` é para **desenvolver**. Em uso contínuo, rode sem `--reload`.
- `--host 127.0.0.1` (padrão) só aceita conexões do próprio computador. Para colegas na rede acessarem, `--host 0.0.0.0` e a porta liberada no firewall do Windows. Antes de fazer isso, converse com a TI: uma API exposta na rede precisa de chave de acesso e de alguém responsável por ela.
- `--port 8080` troca a porta, se a 8000 estiver ocupada.
- Com `pip install "fastapi[standard]"`, você ganha o comando `fastapi dev main.py`, um atalho para o uvicorn com recarga.

Lendo mensagens de erro deste módulo

422 Unprocessable Content (na resposta, não no terminal):

```
{'detail': [{'type': 'greater_than_equal', 'loc': ['query', 'limite'],
 'msg': 'Input should be greater than or equal to 1', 'input': '0',
'ctx': {'ge': 1}}]}
```

O pedido não passou na validação. `loc` diz onde (`path`, `query`, `body` + campo), `msg` diz o quê, `input` mostra o valor recebido. Corrija o pedido, não a API (a não ser que a regra esteja errada).

404 `{'detail': 'Not Found'}` (sem a sua mensagem): não existe rota para esse caminho. Erro de digitação, barra a mais, ou o router não foi incluído (`include_router`). Se a resposta tem a **sua** mensagem (`'Linha Z não encontrada'`), a rota existe e foi a sua lógica que respondeu.

405 `{'detail': 'Method Not Allowed'}`: o caminho existe, mas não para esse método (`POST` numa rota `GET`). Confira o decorador.

500 Internal Server Error: uma exceção que ninguém tratou estourou dentro da rota. O cliente vê só "Internal Server Error"; o traceback de verdade aparece **no terminal do uvicorn**. Leia de baixo para cima, como no módulo 00. No `TestClient`, a exceção original aparece direto no teste.

Resposta fora do modelo:

```
fastapi.exceptions.ResponseValidationError: 1 validation error:
  {'type': 'missing', 'loc': ('response', 'aprovado'), 'msg': 'Field
required', 'input': {'op': '123456'}}
```

A rota devolveu algo que não bate com o tipo de retorno (faltou `aprovado`). O cliente recebe 500. O erro é seu: complete o que a função devolve.

uvicorn não acha o app:

```
ERROR:    Error loading ASGI app. Could not import module "main".
ERROR:    Error loading ASGI app. Attribute "aplicativo" not found in
module "01_primeira_api".
```

No primeiro, você está na pasta errada ou o arquivo tem outro nome. No segundo, o nome depois dos `:` não é o da variável (`app`).

Porta ocupada:

```
ERROR:    [Errno 98] error while attempting to bind on address
('127.0.0.1', 8000): [errno 98] address already in use
```

(No Windows o número é `10048`.) Outro uvicorn ainda está rodando, talvez em outro terminal. Pare-o com `Ctrl+C` ou use `--port 8001`.

SQLite e threads: `sqlite3.ProgrammingError: SQLite objects created in a thread can only be used in that same thread.` Faltou `check_same_thread=False` (seção 10).

Sem httpx: `RuntimeError: The starlette.testclient module requires the httpx package to be installed.` Rode `python -m pip install httpx` (ou o `requirements.txt`).

Boas práticas

- **Substantivos no plural nos caminhos**, método HTTP para a ação, status que conta o resultado.
- **Valide na entrada** com Pydantic (tipos, `Field`, validadores) e **controle a saída** com o tipo de retorno. Nunca confie no que o cliente mandou.
- **`raise HTTPException`**, nunca `return`. E mensagens de `detail` que ajudam quem chamou a corrigir o pedido.
- **Lógica de negócio fora das rotas**, em funções puras (`regras.py`). A rota só traduz HTTP para Python e de volta.
- **Uma conexão por requisição**, via dependência com `yield`. SQL sempre com `?`.

- **Teste com `TestClient`**, conferindo o status antes do corpo. Troque banco e configurações com `dependency_overrides`.
- **Configuração por variável de ambiente** (caminho do banco, chaves), não no código.
- **Rotas fixas antes das rotas com parâmetro** no mesmo prefixo.
- **Documente enquanto escreve:** `title`, `version`, `tags`, `summary`, docstrings.
- **Não exponha na rede sem combinar com a TI.**

No mundo real

O exemplo 07 é um retrato pequeno, mas fiel, de uma API de laboratório. Rode `python exemplos/07_api_organizada.py`:

```
GET /linhas/B/tolerancias
   espessura  nominal  38.0 a 42.0
   gramatura  nominal  36.48 a 39.52
POST /medicoes
   Espessura   41.2 -> 201 True
   gramatura   39.9 -> 201 False
   cof         0.25 -> 404 Sem tolerância para linha B / ensaio cof
GET /ops/123789/situacao -> {'op': '123789', 'medicoes': 2, 'reprovadas':
1, 'liberada': False}
GET /ops/999999/situacao -> 404
Rotas: /linhas/{linha}/tolerancias, /medicoes, /ops/{op}/situacao, /saude
```

Repare em três decisões de "mundo real":

1. **Tolerância nominal ± %** guardada como nominal e percentual, e convertida em mínimo/máximo na hora (40 ± 5% → 38,0 a 42,0). Se a especificação mudar o percentual, muda um número no banco.
2. **`404` para ensaio sem tolerância**, em vez de aprovar ou reprovar no escuro. Medição sem regra é um problema de cadastro, e a API diz isso claramente.
3. **A situação da OP** junta todas as medições: uma reprovação basta para não liberar. É a pergunta que o robô do módulo 12 faria antes de emitir o certificado.

Como isso se encaixa na empresa:

```
  operador (tablet) ──POST /medicoes──┐
  robô de certificados ──GET /ops/{op}/situacao──┤
  Power BI / Excel ──GET (relatórios)──┤──> API do laboratório ──> SQLite
(ou o banco da TI)
  ERP (integração da TI) ──────────────┘
```

Para produção, o caminho natural é a TI hospedar a API num servidor, trocar o SQLite pelo banco corporativo (o código das rotas quase não muda, porque o acesso ao banco está isolado em `banco.py`) e colocar autenticação de verdade. Mas a **regra**, que é o que você entende melhor que ninguém, já está pronta, testada e documentada.

O desafio deste módulo leva essa API até a emissão de certificados.

Resumo

Assunto	Essencial
Criar o app	`app = FastAPI(title=..., version=...)`
Rota	`@app.get("/caminho")`, `@app.post`, `@app.put`, `@app.patch`, `@app.delete`
Subir	`uvicorn main:app --reload` → `http://127.0.0.1:8000/docs`
Testar	`cliente = TestClient(app)`; `cliente.get(...)`, `.post(..., json=...)`
Caminho	`@app.get("/bobinas/{numero}")` + `def f(numero: int)`
Valores fixos	`class Linha(str, Enum)` ou `Literal["A", "B"]`
Consulta	parâmetro fora do caminho; opcional com `= None`; `Annotated[int, Query(ge=1)] = 10`
Corpo	`class Medicao(BaseModel)` + `def f(medicao: Medicao)`
Regras de campo	`Field(gt=0, min_length=2, pattern=r"...")`, `@field_validator`
Regra entre campos	`@model_validator(mode="after")` … `return self`
Converter	`modelo.model_dump()`, `Modelo.model_validate(dict)`
Saída	tipo de retorno `-> Resultado` (valida e filtra)
Status	`status_code=201` no decorador; 204 devolve `None`
Erro	`raise HTTPException(status_code=404, detail="...")`
Organizar	`router = APIRouter(prefix=..., tags=[...])`; `app.include_router(router)`
Dependência	`Annotated[str, Depends(funcao)]`; `Header()` para cabeçalhos
Banco	dependência com `yield` + `finally: con.close()`; `check_same_thread=False`
Trocar no teste	`app.dependency_overrides[original] = substituta`; `.clear()` no fim

Você domina este módulo se...

- ☐ Explico sem consultar a diferença entre parâmetro de caminho, de consulta e corpo, e sei como o FastAPI decide qual é qual.
- ☐ Escolho o status certo para criar (201), apagar (204), não encontrado (404), conflito (409), sem chave (401), sem permissão (403) e dados inválidos (422).
- ☐ Escrevo um modelo Pydantic com `Field`, `Literal` e um validador, e leio um 422 apontando o campo com problema.
- ☐ Testo uma rota com `TestClient`, conferindo status e corpo.
- ☐ Uso `Depends` para chave de acesso e para a conexão, e troco uma dependência no teste com `dependency_overrides`.
- ☐ Guardo dados em SQLite com uma dependência com `yield`, usando sempre `?` no SQL.
- ☐ Organizo uma API em `main.py`, `modelos.py`, `banco.py` e `rotas/` com `APIRouter`.
- ☐ Subo a API com uvicorn, uso o `/docs` e consumo a API com `requests`.

Exercícios

Nº	Exercício	Nível	O que pratica
01	`ex01_ola_api`	★☆☆	primeiras rotas GET, JSON, 405
02	`ex02_parametros_caminho`	★☆☆	parâmetros de caminho com tipo, 422 automático
03	`ex03_linhas_404`	★☆☆	`HTTPException` 404, normalizar entrada
04	`ex04_parametros_consulta`	★★☆	filtros opcionais, `Annotated` + `Query(ge, le)`
05	`ex05_modelos_pydantic`	★☆☆	`BaseModel`, `Field`, `Literal`, `ValidationError`
06	`ex06_validar_medicao`	★★☆	corpo + modelo de resposta + função pura
07	`ex07_ocorrencias`	★★☆	CRUD com 201, 404, 409, 204 e `PATCH`
08	`ex08_router_e_docs`	★☆☆	`APIRouter`, prefixo, tags, resumos, `/openapi.json`
09	`ex09_dependencias`	★★☆	`Depends`, `Header`, 401 x 403, `dependency_overrides`
10	`ex10_leitura_prever_status`	★★☆	leitura de código: prever os status da API de um colega

Nº	Exercício	Nível	O que pratica
11	`ex11_revisao_estatisticas_sqlite`	★★★	revisão: SQLite, `statistics`, agrupamento (módulos 03, 07, 11)
12	`ex12_revisao_regras_tolerancia`	★★★	revisão: Enum, número BR, validadores (módulos 04, 05, 08)

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código e rode, na pasta `curso-python`:

```
python curso.py testar 13 ex01
```

Nos exercícios de API, o arquivo já cria `app = FastAPI()`: você escreve as rotas (e os modelos e funções que o enunciado pedir, no lugar de `raise NotImplementedError`). Para o módulo inteiro: `python curso.py testar 13`.

Desafio do módulo

API de Certificados do laboratório. Regras de tolerância (faixa e nominal ± %) em SQLite, medições avaliadas na hora, situação da OP, emissão de certificado só para OP liberada (e sem duplicar), chave de acesso para alterar regras e `/docs` organizado. Enunciado completo em desafio/README.md.

Quiz

Teste o que ficou: quiz.md.

Para ir além

- `sqlite3` (documentação oficial): https://docs.python.org/pt-br/3/library/sqlite3.html
- `enum`: https://docs.python.org/pt-br/3/library/enum.html
- `typing` (`Annotated`, `Literal`): https://docs.python.org/pt-br/3/library/typing.html
- Tutorial oficial do FastAPI (tem versão em português): https://fastapi.tiangolo.com/pt/tutorial/
- FastAPI, "Bigger Applications" (organizar em módulos): https://fastapi.tiangolo.com/tutorial/bigger-applications/
- Pydantic v2 (modelos e validadores): https://docs.pydantic.dev/latest/concepts/models/

- Leitura extra: os códigos de status HTTP explicados pela MDN: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status

Enunciados dos exercícios

Exercício 13.01 — Olá, API!

Arquivo: `exercicios/ex01_ola_api.py`

```
Nível: ★☆☆

Contexto: antes de qualquer regra de negócio, toda API tem rotas simples
que
dizem "estou no ar" e "qual versão está rodando". O monitoramento da TI e
outros sistemas consultam essas rotas o dia todo.

Tarefa: no `app` abaixo, crie três rotas GET:
- "/"        devolve {"mensagem": "API do laboratório no ar"}
- "/saude"   devolve {"status": "ok"}
- "/versao"  devolve {"versao": VERSAO}   (use a constante, não escreva
"1.0.0")

Exemplos:
    >>> from fastapi.testclient import TestClient
    >>> cliente = TestClient(app)
    >>> cliente.get("/saude").json()
    {'status': 'ok'}
    >>> cliente.get("/versao").status_code
    200

Dica: cada rota é uma função comum com o decorador @app.get("/caminho")
em
cima. A função devolve um dicionário; o FastAPI transforma em JSON.

Teste com:  python curso.py testar 13 ex01
```

Exercício 13.02 — Parâmetros de caminho

Arquivo: `exercicios/ex02_parametros_caminho.py`

```
Nível: ★☆☆

Contexto: um pedaço do endereço pode ser variável, como o número da
bobina em
/bobinas/42. O FastAPI lê esse pedaço, converte para o tipo que você
declarar
```

```
e recusa (com 422) o que não servir.

Tarefa: no `app` abaixo, crie duas rotas GET:
- "/dobro/{numero}"    numero é int. Devolva {"numero": numero, "dobro":
numero * 2}.
                       Um número inválido ("abc") deve dar 422 (o FastAPI
faz isso
                       sozinho se você declarar o tipo).
- "/saudacao/{nome}"   devolva {"mensagem": "Olá, <nome>!"}

Exemplos:
    >>> cliente.get("/dobro/21").json()
    {'numero': 21, 'dobro': 42}
    >>> cliente.get("/saudacao/Carla").json()
    {'mensagem': 'Olá, Carla!'}
    >>> cliente.get("/dobro/abc").status_code
    422

Dica: o nome entre chaves no caminho tem de ser IGUAL ao nome do
parâmetro
da função: @app.get("/dobro/{numero}") -> def dobro(numero: int).

Teste com:  python curso.py testar 13 ex02
```

Exercício 13.03 — Linhas de produção e o erro 404

Arquivo: `exercicios/ex03_linhas_404.py`

```
Nível: ★☆☆

Contexto: a API do laboratório precisa informar os dados de cada linha de
produção. Pedir uma linha que não existe não é "erro do servidor": é um
404,
com uma mensagem clara para quem chamou.

Tarefa: usando o dicionário LINHAS, crie no `app`:
- GET "/linhas"           devolve a lista de códigos em ordem: ["A", "B",
"C", "D"]
- GET "/linhas/{codigo}"  devolve {"codigo": <código>, "nome": ...,
"produto": ...}
  - aceite minúsculas: /linhas/b mostra a linha B (com "codigo": "B");
  - se a linha não existir, levante HTTPException com status 404 e
    detail "Linha X não encontrada" (X em maiúscula).

Exemplos:
    >>> cliente.get("/linhas/b").json()
    {'codigo': 'B', 'nome': 'Extrusora B', 'produto': 'Filme PE 40 µm'}
    >>> cliente.get("/linhas/z").json()
    {'detail': 'Linha Z não encontrada'}
```

```
Dica: from fastapi import HTTPException;
raise HTTPException(status_code=404, detail="...").

Teste com:  python curso.py testar 13 ex03
```

Exercício 13.04 — Filtros com parâmetros de consulta

Arquivo: `exercicios/ex04_parametros_consulta.py`

```
Nível: ★★☆

Contexto: a expedição quer listar bobinas com filtros: só da linha C, só
as
liberadas, só acima de 500 kg... Filtros opcionais são parâmetros de
consulta
(a parte depois do "?": /bobinas?linha=C&status=liberada).

Tarefa: usando a lista BOBINAS, crie GET "/bobinas" com estes parâmetros
de
consulta, todos opcionais:
- linha: str ou None (padrão None). Compare sem diferenciar maiúsculas.
- status: str ou None (padrão None). Ex.: "liberada", "retida".
- peso_min: float, padrão 0, não pode ser negativo (ge=0).
- limite: int, padrão 10, entre 1 e 50 (ge=1, le=50).
Devolva as bobinas que passam em TODOS os filtros informados, na ordem
original da lista, cortando em `limite` itens. Valores fora das regras
(limite=0, peso_min=-1, limite=abc) devem dar 422.

Exemplos:
    >>> cliente.get("/bobinas", params={"linha": "c", "status":
"liberada"}).json()
    [{'numero': 7, 'op': '124001', 'linha': 'C', 'peso_kg': 1210.0,
'status': 'liberada'}]
    >>> len(cliente.get("/bobinas").json())
    10

Dica: from typing import Annotated; from fastapi import Query;
limite: Annotated[int, Query(ge=1, le=50)] = 10

Teste com:  python curso.py testar 13 ex04
```

Exercício 13.05 — Modelo Pydantic de uma medição

Arquivo: `exercicios/ex05_modelos_pydantic.py`

```
Nível: ★☆☆

Contexto: antes de aceitar uma medição na API, queremos garantir que ela
faz
sentido: OP com 6 dígitos, linha que existe, valor positivo. Um modelo
Pydantic descreve essas regras uma vez e valida tudo sozinho.

Tarefa:
1. Complete a classe Medicao (herda de BaseModel) com os campos:
   - op: str com exatamente 6 dígitos (Field(pattern=r"^\d{6}$"))
   - linha: só "A", "B", "C" ou "D" (Literal["A", "B", "C", "D"])
   - ensaio: str com pelo menos 2 caracteres (Field(min_length=2))
   - valor: float maior que zero (Field(gt=0))
   - unidade: str, padrão "µm"
   - observacao: str ou None, padrão None
2. Implemente resumo(medicao), que devolve um texto assim:
       "OP 123456 | linha A | espessura: 20.4 µm"
   (use o valor como o Python mostra o float, sem formatar).

Exemplos:
    >>> m = Medicao(op="123456", linha="A", ensaio="espessura",
valor="20.4")
    >>> m.valor
    20.4
    >>> resumo(m)
    'OP 123456 | linha A | espessura: 20.4 µm'
    >>> Medicao(op="12345", linha="A", ensaio="cof", valor=0.2)
    Traceback (most recent call last):
    ...
    pydantic_core._pydantic_core.ValidationError: 1 validation error for
Medicao

Dica: from typing import Literal; from pydantic import Field. Um campo
com
valor padrão (unidade: str = "µm") vira opcional.

Teste com:  python curso.py testar 13 ex05
```

Exercício 13.06 — Rota que valida uma medição contra a tolerância

Arquivo: `exercicios/ex06_validar_medicao.py`

```
Nível: ★★☆

Contexto: o operador mede a espessura na linha e quer saber na hora:
passou?
E quanto falta para reprovar? A API recebe a medição no CORPO do POST
(JSON) e devolve o resultado.
```

```
Tarefa:
1. avaliar(valor, minimo, maximo) devolve (aprovado, margem):
   - aprovado: minimo <= valor <= maximo
   - margem: distância até o limite MAIS PRÓXIMO, ou seja
     min(valor - minimo, maximo - valor), arredondada com round(..., 4).
     Fica negativa quando o valor está fora.
2. Complete os modelos:
   - PedidoValidacao: linha (str), ensaio (str), valor (float, ge=0)
   - ResultadoValidacao: linha (str), ensaio (str), valor, minimo,
maximo,
     margem (floats) e aprovado (bool)
3. Crie POST "/validar" que recebe um PedidoValidacao e devolve um
   ResultadoValidacao (use-o como tipo de retorno da função):
   - padronize linha para MAIÚSCULA e ensaio para minúscula sem espaços
nas
     pontas antes de procurar em TOLERANCIAS;
   - se não houver tolerância, HTTPException 404 com detail
     "Tolerância não cadastrada: linha X / ensaio Y" (já padronizados).

Exemplos:
    >>> avaliar(20.4, 18.0, 22.0)
    (True, 1.6)
    >>> cliente.post("/validar", json={"linha": "a", "ensaio": " COF ",
"valor": 0.31}).json()
    {'linha': 'A', 'ensaio': 'cof', 'valor': 0.31, 'minimo': 0.2,
'maximo': 0.3,
     'margem': -0.01, 'aprovado': False}

Dica: o tipo de retorno (-> ResultadoValidacao) vira o "modelo de
resposta": o
FastAPI valida e documenta o que sai.

Teste com:  python curso.py testar 13 ex06
```

Exercício 13.07 — Ocorrências de qualidade: os códigos de status certos

Arquivo: `exercicios/ex07_ocorrencias.py`

```
Nível: ★★☆

Contexto: o laboratório registra ocorrências (não conformidades) por OP:
"bolhas no filme", "COF acima do especificado"... Cada operação da API
deve
responder com o código de status que conta o que aconteceu: 201 criou,
404 não existe, 409 conflito com o estado atual, 204 apagou (sem corpo).
```

```
Tarefa: guarde as ocorrências no dicionário OCORRENCIAS ({id: dict}).
1. Complete OcorrenciaEntrada: op (6 dígitos), descricao (mínimo 5
caracteres)
   e gravidade ("baixa", "media" ou "alta").
2. Crie as rotas:
   - POST /ocorrencias -> 201. Guarda e devolve
     {"id": <novo>, "op": ..., "descricao": ..., "gravidade": ...,
"status": "aberta"}.
     O novo id é o maior id existente + 1 (o primeiro é 1).
     Mesma op E mesma descricao de uma ocorrência já cadastrada -> 409
com
     detail "Ocorrência repetida".
   - GET /ocorrencias -> 200, lista de todas, em ordem de id. Parâmetro
de
     consulta opcional `status` ("aberta" ou "encerrada") filtra a lista.
   - GET /ocorrencias/{id} -> 200 com a ocorrência, ou 404 com detail
     "Ocorrência não encontrada".
   - PATCH /ocorrencias/{id}/encerrar -> 200 com a ocorrência já com
     "status": "encerrada". Se já estava encerrada -> 409 com detail
     "Ocorrência já encerrada". Se não existe -> 404 (mesma mensagem
acima).
   - DELETE /ocorrencias/{id} -> 204 sem corpo, ou 404.

Exemplos:
    >>> cliente.post("/ocorrencias", json={"op": "123456", "descricao":
"Bolhas no filme",
    ...                                    "gravidade":
"alta"}).status_code
    201
    >>> cliente.patch("/ocorrencias/1/encerrar").json()["status"]
    'encerrada'

Dica: @app.post("/ocorrencias", status_code=201); @app.patch(...);
@app.delete(..., status_code=204) com a função devolvendo None. Uma
função
auxiliar buscar_ou_404(id) evita repetir o mesmo if em três rotas.

Teste com:  python curso.py testar 13 ex07
```

Exercício 13.08 — APIRouter e documentação automática

Arquivo: `exercicios/ex08_router_e_docs.py`

```
Nível: ★☆☆

Contexto: quando a API cresce, cada assunto ganha seu próprio arquivo com
um
APIRouter (ensaios, medições, certificados...). O app principal só
"pluga"
```

```
os routers. De quebra, prefixo, tags e resumos deixam o /docs organizado
para quem vai usar a API (a TI, o pessoal do BI, você daqui a 6 meses).

Tarefa (tudo neste arquivo, para simplificar):
1. Crie o app com título "API do Laboratório" e versão "1.0.0".
2. Crie `router = APIRouter(prefix="/ensaios", tags=["ensaios"])` com:
   - GET "/"         -> lista dos nomes de ENSAIOS em ordem alfabética,
                        com summary="Lista os ensaios"
   - GET "/{nome}"   -> {"nome": nome, "unidade": ..., "norma": ...},
                        com summary="Detalhes de um ensaio".
                        Nome inexistente -> 404 com detail "Ensaio X não
cadastrado".
3. Inclua o router no app (app.include_router).

Exemplos:
    >>> cliente.get("/ensaios/cof").json()
    {'nome': 'cof', 'unidade': 'adimensional', 'norma': 'ASTM D1894'}
    >>> cliente.get("/openapi.json").json()["info"]["title"]
    'API do Laboratório'

Dica: from fastapi import APIRouter, HTTPException. Com o servidor no ar,
abra /docs e veja as rotas agrupadas pela tag "ensaios".

Teste com:  python curso.py testar 13 ex08
```

Exercício 13.09 — Dependências: chave de acesso, perfil e regras injetadas

Arquivo: `exercicios/ex09_dependencias.py`

```
Nível: ★★☆

Contexto: a API de regras vai ser usada por dois públicos: o laboratório
(pode ler e alterar regras) e o pessoal do BI (só lê). Cada um recebe uma
chave, enviada no cabeçalho X-Chave. Verificar a chave em toda rota seria
repetição; com Depends, a verificação vira uma peça reaproveitável.

Tarefa:
1. perfil_atual(x_chave): devolve o perfil da chave em CHAVES ("escrita"
ou
   "leitura"). Chave ausente ou desconhecida -> HTTPException 401 com
detail
   "Chave de acesso inválida ou ausente". (A assinatura já está pronta: o
   Header() faz o FastAPI ler o cabeçalho X-Chave.)
2. obter_regras(): devolve o dicionário REGRAS.
3. Complete o modelo Faixa: minimo (float) e maximo (float).
4. Rotas, usando Depends(perfil_atual) e Depends(obter_regras):
   - GET /regras/{linha}: qualquer chave válida. Devolve
```

```
     {"linha": linha, "regras": <dict de ensaios da linha>}, ou 404 com
detail
     "Linha X sem regras".
   - PUT /regras/{linha}/{ensaio}: corpo Faixa. Só perfil "escrita";
perfil
     "leitura" -> 403 com detail "Sem permissão de escrita". Grava
     [minimo, maximo] no dicionário de regras (crie a linha se não
existir) e
     devolve {"linha": ..., "ensaio": ..., "minimo": ..., "maximo": ...}.
   Use SEMPRE o dicionário recebido de obter_regras (não REGRAS direto):
os
   testes trocam essa dependência com app.dependency_overrides.

Exemplos:
    >>> cliente.get("/regras/A", headers={"X-Chave": "bi-789"}).json()
    {'linha': 'A', 'regras': {'espessura': [18.0, 22.0], 'cof': [0.2,
0.3]}}
    >>> cliente.get("/regras/A").status_code
    401

Dica: from fastapi import Depends, HTTPException.
def rota(perfil: Annotated[str, Depends(perfil_atual)], ...). 401 = "não
sei
quem você é"; 403 = "sei quem você é, mas você não pode".

Teste com:  python curso.py testar 13 ex09
```

Exercício 13.10 — Leitura de código: preveja as respostas da API do colega

Arquivo: `exercicios/ex10_leitura_prever_status.py`

```
Nível: ★★☆

Contexto: um colega escreveu a API abaixo (ela está pronta, logo depois
deste
enunciado) e pediu sua revisão. Antes de rodar qualquer coisa, LEIA o
código
e preveja o que cada requisição devolve. É assim que se pega defeito em
revisão de código: pensando, não chutando.

Tarefa:
1. Sem rodar, preencha prever_status() com o código de status que você
   espera para cada requisição (as chaves do dicionário já estão
escritas).
2. Preencha prever_json_tolerancias_a() com o JSON (como dict do Python)
que
   GET /tolerancias/A devolve. Atenção aos tipos!
```

```
3. Só então confira: num arquivo de rascunho, faça
       from fastapi.testclient import TestClient
       cliente = TestClient(app, raise_server_exceptions=False)
   e envie as requisições. Errou alguma? Descubra por quê antes de
corrigir.
   (raise_server_exceptions=False faz o TestClient devolver 500 em vez de
   estourar a exceção do servidor no seu terminal.)
4. Não altere a API do colega: os testes conferem as suas previsões
contra
   ela.

Exemplos:
    >>> prever_status()["1: GET /tolerancias/A"]
    200

Dica: pense em cada etapa: existe rota para esse caminho? E para esse
método?
Os dados passam na validação do Pydantic (lembre que ele converte tipos)?
O
código da função pode levantar alguma exceção que ninguém trata?

Teste com:  python curso.py testar 13 ex10
```

Exercício 13.11 — Revisão: API de medições com SQLite e estatística

Arquivo: `exercicios/ex11_revisao_estatisticas_sqlite.py`

```
Nível: ★★★

Revisão de: SQLite com parâmetros "?" e row_factory (módulo 11),
statistics
(módulo 07), dicionários e agrupamento (módulo 03) + FastAPI (este
módulo).

Contexto: o laboratório quer lançar medições pela API e ver, por OP, o
resumo
estatístico de cada ensaio (n, média, desvio padrão, mínimo e máximo),
como
no relatório de série. Os dados precisam sobreviver ao reinício: SQLite.

Tarefa:
1. criar_tabela(con): cria (se não existir) a tabela
       medicoes (id INTEGER PRIMARY KEY AUTOINCREMENT, op TEXT NOT NULL,
                 ensaio TEXT NOT NULL, valor REAL NOT NULL)
2. obter_conexao(): dependência com yield. Conecta em CAMINHO_BANCO com
   check_same_thread=False, define row_factory = sqlite3.Row, chama
   criar_tabela, entrega a conexão (yield) e a FECHA no fim (finally).
```

```
3. MedicaoEntrada: op (6 dígitos), ensaio (mínimo 2 caracteres), valor (>
0).
4. Rotas (recebendo a conexão por Depends(obter_conexao)):
   - POST /medicoes -> 201, grava e devolve {"id", "op", "ensaio",
"valor"}.
   - GET /ops/{op}/resumo -> {"op": op, "ensaios": {...}}, um item por
ensaio
     em ordem alfabética:
         {"n": 3, "media": 20.1, "desvio": 0.3, "minimo": 19.8, "maximo":
20.4}
     media e desvio arredondados com round(..., 3). desvio =
statistics.stdev
     quando n >= 2, e 0.0 quando n == 1.
     OP sem medições -> 404 com detail "OP X sem medições".

Exemplos:
    >>> cliente.post("/medicoes", json={"op": "123456", "ensaio": "cof",
"valor": 0.25}).json()
    {'id': 1, 'op': '123456', 'ensaio': 'cof', 'valor': 0.25}
    >>> cliente.get("/ops/123456/resumo").json()
    {'op': '123456', 'ensaios': {'cof': {'n': 1, 'media': 0.25, 'desvio':
0.0,
     'minimo': 0.25, 'maximo': 0.25}}}

Dica: nos testes, obter_conexao é trocada por uma conexão em memória
(app.dependency_overrides). Por isso as rotas devem usar SÓ a conexão que
recebem. Para o resumo: SELECT ensaio, valor ... WHERE op = ? ORDER BY
ensaio,
depois agrupe os valores num dicionário {ensaio: [valores]}.

Teste com:  python curso.py testar 13 ex11
```

Exercício 13.12 — Revisão: regras de tolerância (faixa e nominal ± %) na API

Arquivo: `exercicios/ex12_revisao_regras_tolerancia.py`

```
Nível: ★★★

Revisão de: Enum (módulo 08), números no formato brasileiro (módulo 05),
funções puras (módulo 04) + validação avançada com Pydantic (este
módulo).

Contexto: o projeto final precisa de dois tipos de tolerância. Por FAIXA
(COF de 0,20 a 0,30) e por NOMINAL ± % (espessura 40 µm ± 5%, ou seja, 38
a
42). E as medições às vezes chegam como texto copiado do Excel:
"1.250,5".
```

```
Tarefa:
1. TipoTolerancia: um Enum de texto (class TipoTolerancia(str, Enum)) com
   FAIXA = "faixa" e NOMINAL = "nominal".
2. numero_br(valor): int/float -> float. Texto COM vírgula é brasileiro
   ("1.250,5" -> 1250.5; "20,5" -> 20.5). Texto SEM vírgula vale como
número
   Python ("20.5" -> 20.5). Texto inválido -> ValueError.
3. Regra (BaseModel): tipo (TipoTolerancia) e minimo, maximo, nominal,
   percentual (float ou None, padrão None). Um
@model_validator(mode="after")
   exige:
   - faixa: minimo e maximo preenchidos e minimo <= maximo;
   - nominal: nominal > 0 e 0 < percentual <= 100 (ambos preenchidos).
   Senão, levante ValueError (o Pydantic transforma em ValidationError /
422).
4. limites(regra) -> (minimo, maximo). Para nominal:
   (round(nominal - folga, 4), round(nominal + folga, 4)), folga =
nominal * percentual / 100.
5. ItemLote: ensaio (str) e valor (float), com @field_validator("valor",
   mode="before") que usa numero_br (texto inválido vira 422).
   Lote: linha (Literal "A" a "D") e medicoes (list[ItemLote], ao menos 1
item).
6. Rotas (guarde as regras em REGRAS[(linha, ensaio)]; ensaio sempre em
minúsculas):
   - PUT /regras/{linha}/{ensaio}, corpo Regra -> 200 com
     {"linha", "ensaio", "tipo", "minimo", "maximo"} (limites já
calculados).
     Linha fora de A-D -> 422 (use Literal no parâmetro de caminho).
   - GET /regras/{linha}/{ensaio} -> o mesmo formato, ou 404 com detail
     "Regra não cadastrada".
   - POST /lotes/validar, corpo Lote -> {"linha": ..., "aprovado": bool,
     "itens": [{"ensaio", "valor", "minimo", "maximo", "aprovado"}, ...],
     "sem_regra": [ensaios sem regra, na ordem do lote]}.
     Itens sem regra não entram em "itens". O lote só é aprovado se todos
os
     itens forem aprovados E "sem_regra" estiver vazio.

Exemplos:
    >>> numero_br("1.250,5")
    1250.5
    >>> cliente.put("/regras/B/espessura", json={"tipo": "nominal",
"nominal": 40,
    ...                                           "percentual": 5}).json()
    {'linha': 'B', 'ensaio': 'espessura', 'tipo': 'nominal', 'minimo':
38.0, 'maximo': 42.0}

Dica: from pydantic import model_validator, field_validator. No validador
"after", use self.tipo, self.minimo... e termine com `return self`.
```

> Teste com: python curso.py testar 13 ex12

Desafio do módulo 13 — API de Certificados do laboratório

Contexto

Hoje a decisão "posso emitir o certificado desta OP?" mora na cabeça de quem olha a planilha. Você vai transformar essa decisão numa **API**: um serviço que guarda as regras de tolerância, recebe as medições, diz se a OP está liberada e só então registra o certificado. O robô do módulo 12, uma planilha de BI ou um sistema da TI podem consultar essa API, todos com a mesma regra.

Requisitos obrigatórios

1. FastAPI + SQLite, com o banco num arquivo (caminho configurável por variável de ambiente) e a conexão entregue às rotas por uma dependência com `yield`.
2. Tabelas `regras`, `medicoes` e `certificados`. Regras iniciais para as linhas A a D, com os dois tipos: **faixa** (mínimo/máximo) e **nominal ± %**.
3. `GET /linhas/{linha}/regras`: regras da linha, já convertidas em mínimo e máximo. Linha fora de A–D responde 422.
4. `PUT /linhas/{linha}/regras/{ensaio}`: cria ou troca uma regra. Exige o cabeçalho `X-Chave` com perfil de escrita (401 sem chave válida; 403 com chave só de leitura). Regra inconsistente (mínimo > máximo, percentual zero...) responde 422.
5. `POST /medicoes`: valida o corpo com Pydantic, avalia contra a regra, grava e responde 201 com `aprovado`. Sem regra para o ensaio: 404.
6. `GET /ops/{op}/situacao`: total de medições, reprovadas e se a OP está liberada.
7. `POST /certificados` com OP, NF e peso: 201 se a OP está liberada; 409 se não tem medições, se tem reprovação, ou se o par OP/NF já tem certificado (idempotência).
8. `GET /certificados` com filtro opcional `?op=`.
9. Título, versão, tags e resumos para o `/docs` ficar organizado.
10. Rodar com uvicorn e ter um modo de demonstração que não precisa de servidor.

Extras opcionais

- Testes com `TestClient` e banco em memória via `app.dependency_overrides`.
- Organizar em pacote (`main.py`, `banco.py`, `modelos.py`, `rotas/`), como o exemplo 07.

- `GET /ops/{op}/resumo` com média e desvio padrão por ensaio (exercício 11).
- Aceitar valores no formato brasileiro (`"41,2"`) nas medições (exercício 12).
- Gerar o PDF do certificado e devolvê-lo numa rota (`FileResponse`).
- Ligar o robô do módulo 12 à API: ele consulta `/ops/{op}/situacao` antes de emitir.

Exemplo de execução

```
> python 13-apis-com-fastapi/desafio/solucao_referencia.py
Regras da linha B:
   espessura  38.0 a 42.0
   gramatura  36.48 a 39.52
medição 123789 espessura   41.2 -> aprovado=True
medição 123789 gramatura   38.1 -> aprovado=True
medição 123456 espessura   43.0 -> aprovado=False
POST /certificados (OP liberada)   201 {'op': '123789', 'nf': '98766',
'peso_kg': 980.25, 'id': 1}
POST /certificados (de novo)       409 Certificado já emitido
POST /certificados (reprovada)     409 {'detail': 'OP 123456 com 1
medição(ões) reprovada(s)'}
PUT regra com chave de leitura     403 {'detail': 'Sem permissão de
escrita'}
PUT regra com chave de escrita     200 {'linha': 'B', 'ensaio': 'cof',
'tipo': 'faixa', 'minimo': 0.2, 'maximo': 0.3}
Certificados emitidos: 1
```

Com `--servir`, abra `http://127.0.0.1:8000/docs` e faça o mesmo roteiro pelo navegador.

Critérios de aceite

- [ ] A demonstração roda sem servidor e termina sem erro.
- [ ] Com `--servir`, o `/docs` mostra as rotas agrupadas por tag, com resumos.
- [ ] Espessura da linha B (40 ± 5%) aparece como 38,0 a 42,0.
- [ ] Uma OP com qualquer medição reprovada nunca recebe certificado.
- [ ] Pedir o mesmo certificado duas vezes não cria dois registros.
- [ ] Alterar regra sem chave dá 401; com chave de leitura dá 403.
- [ ] Todo SQL usa parâmetros `?` (nenhuma f-string montando SQL).
- [ ] Os dados continuam lá depois de parar e subir o servidor de novo.
- [ ] `ruff check` e `ruff format --check` passam.

Dicas de como começar

1. Comece pelo banco: `ESQUEMA`, regras iniciais e a dependência `obter_conexao`. Teste no REPL que as tabelas são criadas.
2. Escreva a função pura `limites(...)` e teste com faixa e com nominal ± %.
3. Uma rota por vez, sempre com o `TestClient` num script de rascunho: regras → medições → situação → certificados.
4. Deixe a autenticação (`X-Chave`) por último: é uma dependência que você pluga na rota de escrita.
5. Para a idempotência, a restrição `UNIQUE (op, nf)` no banco é a última linha de defesa; a consulta antes do `INSERT` dá a mensagem amigável.

A solução de referência está em `solucao_referencia.py`. Tente antes de abrir.

Quiz — Módulo 13: APIs com FastAPI

Responda sem consultar a aula. Depois confira no gabarito, no fim da página.

---

1. Numa API REST, qual combinação de método e caminho é a mais adequada para **registrar** uma medição nova?

a) `GET /medicoes/nova` b) `POST /medicoes` c) `PUT /medicao` d) `DELETE /medicoes`

---

2. O que estas duas requisições devolvem?

```
@app.get("/itens/{n}")
def item(n: int):
    return {"n": n}

@app.get("/busca")
def busca(q: str, limite: int = 5):
    return {"q": q, "limite": limite}

cliente = TestClient(app)
print(cliente.get("/itens/07").json(), cliente.get("/busca?
q=cof").json())
print(cliente.get("/busca").status_code)
```

---

3. Como o FastAPI decide se um parâmetro da função vem do **caminho**, da **consulta** (query string) ou do **corpo**?

---

4. Qual a diferença entre um 404 e um 422 numa API FastAPI? Dê um exemplo de cada no contexto do laboratório.

---

5. O que este código imprime?

```python
from pydantic import BaseModel, Field

class M(BaseModel):
    valor: float = Field(gt=0)
    nome: str = "x"

print(M(valor="3").model_dump())
```

---

6. Uma rota levanta `KeyError` sem tratamento. Qual status o cliente recebe, e como transformar isso numa resposta útil?

a) 404, automático b) 422 c) 500; levantar `HTTPException(404, ...)` quando a chave não existir d) 200 com `null`

---

7. O que este código imprime?

```python
@app.get("/linha/{codigo}")
def ver(codigo: str):
    return {"codigo": codigo}

@app.get("/linha/especial")
def especial():
    return {"especial": True}

print(TestClient(app).get("/linha/especial").json())
```

---

8. Qual a diferença entre 401 e 403?

---

9. Para que serve `app.dependency_overrides` nos testes?

---

10. Por que usamos `sqlite3.connect(..., check_same_thread=False)` numa dependência do FastAPI, e por que cada requisição deve abrir e fechar a sua conexão?

---

11. Onde está o problema?

```python
@app.get("/medicoes")
def listar(con: Conexao, op: str):
    return con.execute(f"SELECT * FROM medicoes WHERE op =
'{op}'").fetchall()
```

---

12. Qual comando sobe a API do arquivo `main.py` (com `app = FastAPI()` dentro) e recarrega sozinho quando você salva o arquivo? Em que endereço fica a documentação?

---

13. A rota é `@app.delete("/amostras/{id}", status_code=204)`. O que a função deve devolver, e o que o cliente recebe no corpo?

Gabarito

▼ **Ver respostas**

1. b) `POST /medicoes`. Em REST o caminho nomeia o recurso (substantivo, no plural) e o método diz a ação. `GET` não deve criar nada.

2. `{'n': 7} {'q': 'cof', 'limite': 5}` e depois `422`. O `"07"` vira o inteiro 7; `limite` usa o padrão 5; sem `q` (obrigatório, sem valor padrão) a validação falha.

3. Pelo nome e pelo tipo: se o nome aparece entre chaves no caminho, é de caminho; se o tipo é um modelo Pydantic (`BaseModel`), é o corpo; os demais (tipos simples) são de consulta. `Query()`, `Path()`, `Header()` e `Body()` deixam isso explícito.

4. 404: o recurso não existe (`GET /linhas/Z`, OP sem medições). 422: o pedido chegou com dados inválidos (`valor` negativo, OP com 5 dígitos, `limite=abc`). O 422 o FastAPI gera sozinho a partir dos tipos; o 404 de "não achei" é você quem levanta.

5. `{'valor': 3.0, 'nome': 'x'}`. No modo padrão, o Pydantic converte o texto `"3"` em `3.0` e preenche o valor padrão de `nome`.

6. c). Exceção não tratada vira `500 Internal Server Error`, que não explica nada ao cliente. Procure com `.get()` (ou `in`) e levante `HTTPException(404, detail=...)`.

7. `{'codigo': 'especial'}`. As rotas são testadas **na ordem em que foram declaradas**, e `/linha/{codigo}` aceita qualquer texto. Declare as rotas fixas antes das rotas com parâmetro.

8. 401: "não sei quem você é" (chave ausente ou inválida). 403: "sei quem você é, mas você não tem permissão" (chave de leitura tentando alterar uma regra).

9. Para trocar uma dependência por outra durante o teste, sem mexer nas rotas: por exemplo, a conexão com o banco de verdade por um banco em memória, ou as regras reais por regras de teste. Lembre de limpar (`app.dependency_overrides.clear()`) no fim.

10. O FastAPI pode preparar a dependência numa thread e executar a rota em outra; sem `check_same_thread=False`, o `sqlite3` recusa. Abrir e fechar por requisição (dependência com `yield` e `finally`) evita compartilhar uma conexão entre requisições simultâneas e garante que ela é fechada mesmo quando a rota dá erro.

11. SQL montado com f-string: **SQL injection**. Um `op` como `' OR '1'='1` devolve a tabela inteira (ou pior). Use parâmetros: `con.execute("SELECT * FROM medicoes WHERE op = ?", (op,))`.

12. `uvicorn main:app --reload`, rodado na pasta do `main.py`. A documentação interativa fica em `http://127.0.0.1:8000/docs` (e a alternativa em `/redoc`).

13. A função devolve `None` (ou nada), e o cliente recebe o status 204 **sem corpo**. 204 significa "deu certo e não há conteúdo para mostrar".

Módulo 14 — Projeto final: Validador de Certificados

> **Tempo estimado:** 15 a 25 horas · **Pré-requisitos:** módulos 00 a 13 (08, 09, 11 e 12 são essenciais; 13 para a extensão da API) · **Nível:** intermediário (projeto integrador)

O que você vai aprender

- Transformar uma descrição de trabalho ("leio a planilha, confiro, emito no TOTVS") numa lista de requisitos e numa arquitetura em camadas.
- Dizer, para cada pedaço de código, em qual camada ele mora e do que ele pode depender.
- Modelar o domínio com dataclasses que se recusam a existir com dados inválidos.
- Carregar regras de qualidade de um banco SQLite aberto só para leitura.
- Implementar tolerância fixa (mín/máx) e nominal ± % sem cair na armadilha do float.
- Ler uma planilha de 4 abas com openpyxl, sendo tolerante no formato e rígido no conteúdo.
- Escolher a especificação da OP (cliente ou padrão da linha) sem nunca chutar.
- Extrair OP, NF e peso dos e-mails da expedição e separar os fora do padrão.
- Montar a fila de emissão, emitir em **modo ensaio** com idempotência, e gerar o relatório da série.
- Juntar tudo numa CLI com argparse, logging e códigos de saída.
- Montar um projeto profissional (src layout, pyproject, testes, GitHub Actions) e publicá-lo como portfólio.
- Planejar as extensões: robô Playwright no Edge, leitura do Outlook e API FastAPI.

Por que isso importa

Este é o projeto que você descreveu no começo do curso. Não é exercício de livro: é a automação que vai tirar horas de digitação da sua semana e evitar um certificado emitido com valor fora da tolerância.

Para o mercado, ele vale ainda mais. Um recrutador de dados, automação ou backend vê um sistema com regras de negócio reais, banco de dados, leitura de Excel, testes automatizados, CI no GitHub e decisões de projeto explicadas. Isso conta mais que dez certificados de curso. Na entrevista, você conta a história: "tinha um processo manual, construí isso, e funciona assim".

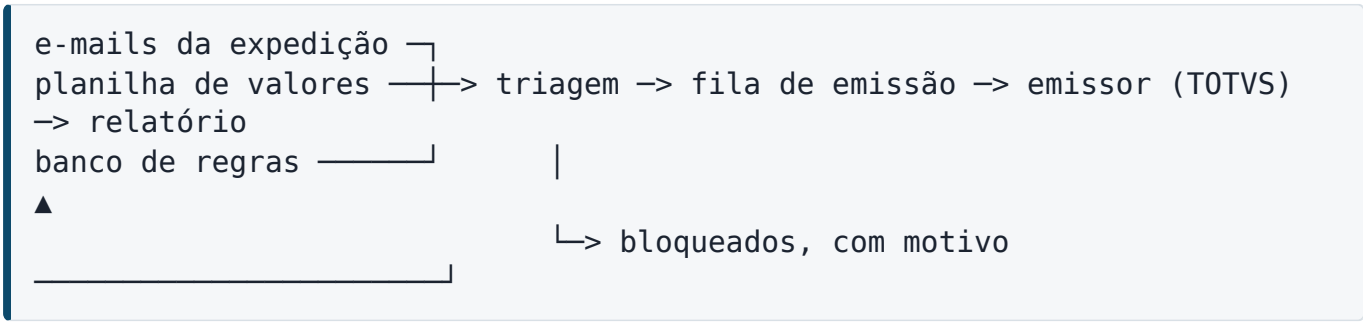
Neste módulo você vai **construir** o sistema em 14 etapas testadas e, no fim, encontrar a montagem completa na pasta `projeto-completo/`, pronta para virar o seu repositório.

1. O problema, em uma página

Antes de qualquer código, escreva o problema em português. Se não cabe numa página, ainda não está entendido.

Hoje (manual): a expedição manda e-mail pedindo certificado ("OP 24518, NF 123456, 1.250,5 kg"). Alguém abre a planilha de valores, acha a OP na aba da linha certa, confere cada característica contra a especificação do cliente (ou o padrão da linha), olha se o COF foi liberado, e digita o certificado no TOTVS.

Amanhã (com o sistema):



O que a versão 1 faz:

1. Lê as regras (especificações e tolerâncias) de um banco SQLite.
2. Lê a planilha de valores (4 abas: Linha A, B, C e D).
3. Lê os pedidos dos e-mails (OP, NF e peso) e separa os fora do padrão.
4. Para cada pedido: acha a OP, confere a linha, escolhe a especificação, valida as tolerâncias e a liberação do COF.
5. Emite os certificados das OPs aprovadas e liberadas (por enquanto, em ensaio).
6. Grava um relatório da série: emitidos, bloqueados e o motivo de cada bloqueio.

O que a versão 1 NÃO faz (e está tudo bem): não clica no TOTVS de verdade, não lê o Outlook direto, não tem tela. Escopo pequeno e terminado vence escopo grande pela metade.

Exemplo resolvido: dos verbos às funções

Primeiro eu pego a descrição do processo e sublinho os **substantivos** e os **verbos**. Substantivos viram modelos (dados). Verbos viram funções (ações).

```
"A expedição manda o PEDIDO (OP, NF, peso). Eu acho a OP na PLANILHA,
confiro cada
CARACTERÍSTICA contra a ESPECIFICAÇÃO e a TOLERÂNCIA, vejo a OBSERVAÇÃO
de liberação
e emito o CERTIFICADO."

substantivos -> Pedido, Medicao (linha da planilha), Especificacao,
Tolerancia,
                ResultadoValidacao, ItemFila (o pedido depois da triagem)
verbos       -> ler_email, ler_planilha, escolher_especificacao,
validar_medicao,
                esta_liberado, montar_fila, emitir_fila,
escrever_relatorio
```

Depois eu pergunto, para cada verbo: "isso precisa de arquivo, rede ou tela?". Se não precisa, é regra pura e vai para o núcleo. Se precisa, é borda. É assim que nasce a arquitetura da próxima seção.

> 💡 **Dica:** escreva essa lista num papel antes de abrir o VS Code. Dez minutos aqui economizam um fim de semana de refatoração.

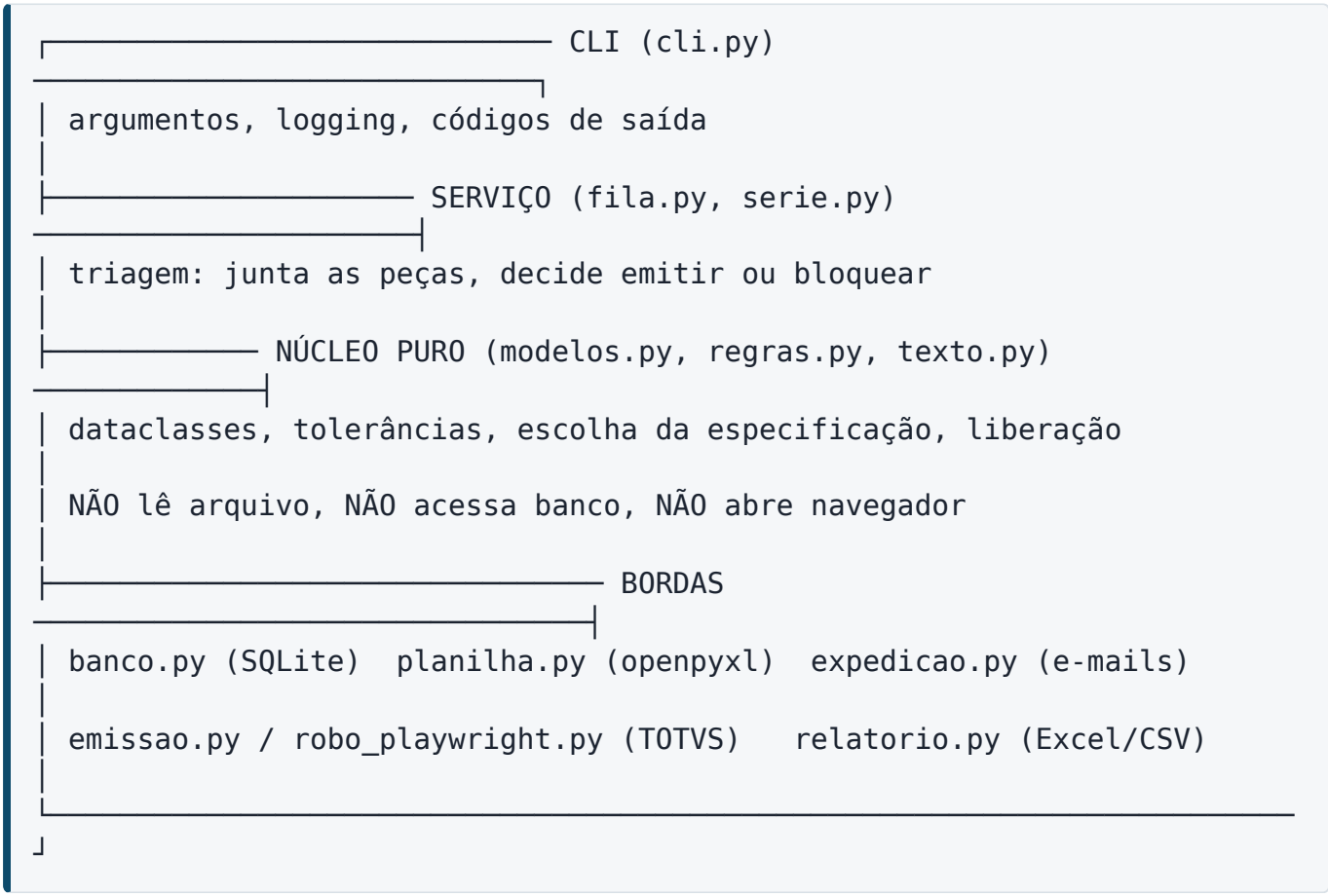
Tente você: na frase "o supervisor recebe um aviso quando alguma OP fica bloqueada", quais são os substantivos e os verbos? Viraria modelo, função, ou os dois?

<details>
<summary>Resposta</summary>

Substantivos: supervisor, aviso, OP. Verbos: receber (o aviso), ficar bloqueada. "Aviso" pode virar um modelo pequeno (destinatário, assunto, texto) e uma função `montar_aviso(bloqueados) -> Aviso` (regra pura). Enviar o e-mail é borda (precisa de rede): outra função, `enviar_aviso(aviso)`, que fica isolada como o emissor do TOTVS.
</details>

2. Arquitetura em camadas

Pense no laboratório. O técnico **mede**. A norma **decide** se passou. O supervisor **assina**. Ninguém mistura os papéis: o técnico não muda a norma porque a bobina está quase boa. O sistema segue a mesma divisão.



Regra de dependência: as dependências apontam para dentro. As bordas e o serviço importam o núcleo; o núcleo não importa ninguém de fora. Consequência prática: se amanhã a planilha virar um banco, ou o TOTVS ganhar uma API, só a borda muda. As regras, que são o que vale dinheiro, ficam intactas.

O exemplo `exemplos/01_nucleo_e_casca.py` mostra a ideia em 30 linhas:

```python
def verificar_espessura(valor: float, nominal: float, percentual: float)
-> str | None:
    """None se passou; senão, o motivo. Não sabe de onde vieram os
números."""
    folga = nominal * percentual / 100
    minimo, maximo = round(nominal - folga, 4), round(nominal + folga, 4)
    if minimo <= valor <= maximo:
        return None
    return (
        f"espessura {valor:g} fora de {nominal:g} ± {percentual:g}%
```

```
({minimo:g} a {maximo:g})"
    )
```

```
python exemplos/01_nucleo_e_casca.py
OP 24518: OK
OP 24519: espessura 44.8 fora de 40 ± 10% (36 a 44)
OP 24520: OK
núcleo testado sem abrir arquivo nenhum
```

> 🔧 **Modifique:** troque o `10` da chamada em `main()` por `12` e rode de novo. A OP 24519 passa? Calcule o novo máximo de cabeça antes de rodar (40 + 12% de 40).

Camada	Arquivo no projeto	Etapas (exercícios)
Núcleo: modelos	`modelos.py`	1
Borda: banco de regras	`banco.py`	2
Núcleo: regras	`regras.py`	3, 4, 5, 9
Núcleo: texto	`texto.py`	6
Borda: planilha	`planilha.py`	7, 8
Borda: e-mails	`expedicao.py`	10
Serviço: triagem	`fila.py`	11
Borda: emissão (TOTVS)	`emissao.py`	12
Borda: relatório	`relatorio.py`	13
Entrada: CLI	`cli.py`	14

Tente você: em qual camada fica uma função que decide se a OP pode ter certificado **sem** o COF liberado quando o cliente é "Biscoitos Aurora"?

▼ **Resposta**

No núcleo (regras). É uma regra de negócio: recebe dados (cliente, observação) e devolve uma decisão. Onde o nome do cliente é lido (planilha ou banco) é borda; a decisão em si não depende de arquivo nenhum.

3. Decisões de projeto (e por quê)

Projetos profissionais registram as decisões, não só o código. Daqui a seis meses, você (ou quem herdar o sistema) vai perguntar "por que é assim?". Estas são as do Validador:

1. **Regra devolve motivo, não True/False.** `verificar_fixa` devolve `None` (passou) ou `"cof 0.38 acima do máximo 0.35"`. O relatório precisa do porquê; um booleano joga a informação fora.
2. **Nunca chutar.** Título de especificação desconhecido, linha divergente da aba, OP repetida na planilha, e-mail com duas OPs: tudo vira **pendência** com motivo. Uma pendência custa um minuto do técnico. Um certificado errado custa um cliente.
3. **Especificação do cliente herda do padrão da linha.** O cliente lista só o que muda (ex.: COF de 0,18 a 0,25). O resto vem do padrão. Menos cadastro, menos erro.
4. **Limites inclusivos e arredondados.** Valor igual ao limite passa. Os limites são arredondados em 4 casas para o float não reprovar 33,3 num mínimo de 33,3.
5. **Modo ensaio é o padrão.** Emitir de verdade exige pedir (`--emitir`). O erro mais caro é emitir errado; o padrão seguro protege contra o esquecimento.
6. **Idempotência por (OP, NF).** Rodar a série duas vezes não emite duas vezes. A chave inclui a NF porque a mesma OP pode sair em entregas parciais.
7. **Robô isolado atrás de uma interface.** O sistema conversa com um `Emissor` (`emitir(item) -> comprovante`). Hoje existe o de ensaio; o robô Playwright vai implementar a mesma interface.
8. **Uma falha não derruba a série.** Se a tela do TOTVS travar numa OP, ela vira "falha" e a próxima segue. Mas bug (erro inesperado) sobe e para tudo.
9. **Regras no SQLite, não numa planilha de regras.** O banco tem tipos, chave primária e `CHECK` (tipo só pode ser `fixa` ou `nominal`; uma linha tem no máximo um padrão). Uma planilha aceita qualquer coisa em qualquer célula. O custo: editar exige um programa como o DB Browser for SQLite.
10. **Sem pandas no núcleo.** openpyxl lê a planilha linha a linha e o resto é Python puro. Menos dependências, instalação mais leve no PC da fábrica. pandas continua ótimo para **analisar** os dados depois (módulo 10).

> ⚠️ **Erro comum:** tomar a decisão certa e não escrever o porquê. O próximo que mexer (talvez você) "simplifica" e reintroduz o bug que a decisão evitava.

4. Os dados de exemplo

Tudo o que você precisa para construir e testar está em `dados/`. São dados **fictícios**, mas com os casos difíceis da vida real.

```
dados/
  gerar_planilha.py      cria planilha_valores.xlsx (4 abas, 16 OPs)
  gerar_banco_regras.py  cria regras.db a partir de schema_regras.sql
  schema_regras.sql      as tabelas linha, especificacao e tolerancia
  planilha_valores.xlsx
  regras.db
  emails/                8 e-mails da expedição (.txt), um deles em
cp1252
```

Para recriar os arquivos (Windows, na pasta `curso-python`):

```
python 14-projeto-final\dados\gerar_planilha.py
python 14-projeto-final\dados\gerar_banco_regras.py
```

No Linux/macOS, troque `\` por `/`.

A planilha. Uma aba por linha, cabeçalho na linha 1, uma OP por linha:

```
OP | Data | Linha | Produto | Especificação | Espessura nominal (µm) |
Espessura (µm) |
Gramatura nominal (g/m²) | Gramatura (g/m²) | COF | Tração (MPa) |
Selagem (N/15mm) |
Observação
```

O banco. Padrões de linha têm a coluna `linha` preenchida; especificações de cliente deixam `linha` vazia e listam só o que muda:

```
CREATE TABLE IF NOT EXISTS tolerancia (
    titulo         TEXT NOT NULL REFERENCES especificacao (titulo),
    caracteristica TEXT NOT NULL,
    tipo           TEXT NOT NULL CHECK (tipo IN ('fixa', 'nominal')),
    minimo         REAL,
    maximo         REAL,
    percentual     REAL,
    PRIMARY KEY (titulo, caracteristica)
);
```

```
python exemplos/05_consultar_regras.py
Padrões de linha:
  linha A: PADRAO-A
```

```
  linha B: PADRAO-B
  linha C: PADRAO-C
  linha D: PADRAO-D

ESP-0412: Torrefação Serra - embalagem de café 250 g. COF interno de 0,18
a 0,25; selagem mínima 12 N/15mm.
  cof        fixa     mín=0.18 máx=0.25 %=None
  selagem    fixa     mín=12.0 máx=None %=None
```

Os e-mails. Cada um demonstra um caso:

Arquivo	OP	O que demonstra	Resultado esperado
`email_01_op24518.txt`	24518	pedido limpo, linha A	emitir
`email_02_op24519.txt`	24519	espessura 44,8 em 40 ± 10%	bloqueado (reprovada)
`email_03_op24527.txt`	24527	"COF: aguardando 24h"	bloqueado (sem liberação)
`email_04_op24541.txt`	24541	ESP-0412, "NF-e nº 123.789"	emitir
`email_05_op24546.txt`	24546	especificação ESP-9999	bloqueado (pendência)
`email_06_op24533.txt`	24533	aba B, linha digitada C, arquivo cp1252	bloqueado (pendência)
`email_07_sem_nf.txt`	24555	"a NF sai amanhã"	fora do padrão
`email_08_duas_ops.txt`	24520 e 24521	duas OPs no mesmo e-mail	fora do padrão

> 💡 **Dica:** quando for usar no trabalho, monte primeiro um conjunto de exemplo como este, com um caso de cada tipo. Ele vira a base dos seus testes e da sua confiança.

5. Roteiro de construção

A ordem das etapas não é aleatória: é **de dentro para fora**. Primeiro o núcleo (fácil de testar, sem arquivo), depois as bordas, por último a orquestração. Cada etapa é um exercício com testes.

Fase	Etapas	Por que nesta ordem
1. Domínio	1 modelos	todo o resto troca essas "fichas"
2. Regras	2 banco, 3 fixa, 4 nominal, 5 validar OP, 6 liberação	o coração do sistema; puro e testável
3. Entradas	7 planilha, 8 linha, 9 especificação, 10 e-mails	traduzem o mundo para os modelos
4. Saídas	11 fila, 12 emissor, 13 relatório, 14 CLI	juntam tudo e entregam o resultado

Como trabalhar cada etapa:

1. Leia o enunciado (docstring do topo) inteiro. Duas vezes.
2. Olhe as **peças prontas** no arquivo: são as etapas anteriores, já resolvidas, para cada etapa funcionar sozinha. Você pode fazer a etapa 11 antes da 7, e um erro na etapa 3 não quebra a 5.
3. Experimente no REPL antes de escrever a função (`python`, cole a peça, teste).
4. Escreva, rode `python curso.py testar 14 exNN`, leia a falha, corrija.
5. Compare com `solucoes/` só depois de passar. Os comentários de lá explicam o porquê.

> ⚠️ **Erro comum:** querer fazer as 14 etapas num dia só. Faça uma ou duas por sessão. Cada uma tem uma ideia nova; dê tempo para ela assentar.

6. Domínio e regras (etapas 1 a 6)

6.1 Modelos que se defendem (etapa 1)

Uma dataclass com `__post_init__` valida os dados no nascimento. Um objeto inválido nunca chega a existir, então ninguém lá na frente precisa desconfiar dele.

Exemplo resolvido: a ficha da bobina

Quero uma ficha de bobina que recuse lote vazio, largura ou peso zerados. Primeiro penso nos campos (lote, largura, peso). Depois, nas regras: "o que tornaria esta ficha um absurdo?". Por fim, `frozen=True`, porque uma bobina pesada não muda de peso.

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class Bobina:
    lote: str
    largura_mm: float
    peso_kg: float

    def __post_init__(self) -> None:
        if not self.lote.strip():
            raise ValueError("bobina sem lote")
        if self.largura_mm <= 0 or self.peso_kg <= 0:
            raise ValueError(f"bobina {self.lote}: largura e peso devem
ser > 0")


print(Bobina("L2026-118", 640, 412.5))
try:
    Bobina("L2026-119", 640, 0)
except ValueError as erro:
    print("recusada:", erro)
```

```
Bobina(lote='L2026-118', largura_mm=640, peso_kg=412.5)
recusada: bobina L2026-119: largura e peso devem ser > 0
```

Na etapa 1 você faz o mesmo para `Tolerancia` e `Pedido`, e escreve a propriedade `ResultadoValidacao.aprovado`. Por que propriedade, e não um campo `aprovado: bool`? Porque um campo pode contradizer os motivos (`aprovado=True` com `motivos=["cof alto"]`). Calculado a partir dos motivos, ele nunca mente.

O exemplo `exemplos/02_modelos_com_dataclasses.py` mostra ainda `frozen`, `replace` e a junção de dicionários que você vai usar adiante.

6.2 O banco de regras (etapa 2)

A etapa 2 lê as tabelas `especificacao` e `tolerancia` e devolve `{titulo: Especificacao}`. O resto do sistema nunca vê SQL.

Exemplo resolvido: as linhas de produção

Mesmo raciocínio, numa tabela mais simples. Quero `{codigo: descricao}`. Uma consulta, um laço que desempacota cada linha, e fecho a conexão. (Rode na pasta `14-projeto-final`: o caminho do banco é relativo a ela.)

```
import sqlite3

conexao = sqlite3.connect("dados/regras.db")
consulta = "SELECT codigo, descricao FROM linha ORDER BY codigo"
linhas = {codigo: descricao for codigo, descricao in
conexao.execute(consulta)}
conexao.close()
print(linhas)
```

```
{'A': 'Extrusora 1 - filmes PE monocamada', 'B': 'Extrusora 2 - filmes PE
finos', 'C': 'Coextrusora 3 camadas', 'D': 'Laminadora - PET/PE e
BOPP/PE'}
```

Para as especificações são duas consultas: uma cria as fichas, a outra pendura cada tolerância na ficha do seu título. Repare que `linha` NULL chega como `None` e precisa virar `""`.

> ⚠️ **Erro comum:** `sqlite3.connect("regras_errado.db")` não dá erro: **cria** um banco vazio. O erro só aparece depois, como `no such table`. No projeto completo, o banco é aberto com `?mode=ro` (somente leitura), e um caminho errado vira erro na hora, com a mensagem certa.

6.3 Tolerância fixa e nominal (etapas 3 e 4)

Duas regras, mesma convenção: devolvem `None` ou o motivo. A fixa compara com mínimo e máximo (um deles pode faltar: tração só tem mínimo). A nominal calcula a faixa a partir do valor nominal da OP: 40 µm ± 10% vira 36 a 44.

Os números do motivo usam o formato `:g`, que corta zeros inúteis:

```
print(f"{0.30:g} | {8.0:g} | {44.8:g} | {65.565:g}")
```

```
0.3 | 8 | 44.8 | 65.565
```

> 🔮 **Preveja antes de rodar:** a gramatura nominal é 37,0 g/m² com ± 10%. A medida foi 33,3, exatamente o limite. Passa?
>
> ```
> minimo = 37.0 * (1 - 10 / 100)
> print(minimo)
> print(minimo <= 33.3)
> ```

▼ **Resposta**

```
33.300000000000004
False
```

Reprovaria uma bobina boa. É o mesmo problema do `0.1 + 0.2` do módulo 01: o float não representa 0,9 exatamente. A correção da etapa 4 é arredondar os limites com `round(..., 4)`. Veja `exemplos/03_float_nos_limites.py`.

6.4 Validar a OP inteira (etapa 5)

Quem manda é a **especificação**: o laço percorre as tolerâncias que ela exige, não os valores medidos. Assim, "faltou medir a selagem" vira motivo, e uma medição extra que ninguém pediu é ignorada. E o laço não para no primeiro problema: junta todos, porque o técnico quer corrigir tudo de uma vez.

```
para cada tolerância da especificação:
    sem valor medido?          -> motivo "<car> sem valor medido"
    tipo fixa?                 -> verificar_fixa
    tipo nominal sem nominal?  -> motivo "<car> sem valor nominal"
    tipo nominal?              -> verificar_nominal
    a regra devolveu texto?    -> entra nos motivos
```

6.5 A observação de liberação (etapa 6)

A etapa 6 é de **leitura de código**. Você recebe a versão que um estagiário escreveu:

```python
def esta_liberado_v1(observacao):
    return "liberado" in observacao.lower()
```

Antes de consertar, você prevê o que ela faz em quatro casos reais. Depois escreve a versão correta: normalizar o texto (minúsculas, sem acento, espaços simples) e procurar a marca "cof" seguida de "liberado" com uma regex.

🔮 **Preveja antes de rodar:** qual destes casa com o padrão?

```python
import re

padrao = r"\bcof\s*[:-]?\s*liberad[oa]\b"
for texto in ["cof: nao liberado", "cof - liberada", "cof liberado?
```

```
nao"]:
    print(texto, "->", re.search(padrao, texto) is not None)
```

▼ **Resposta**

```
cof: nao liberado -> False
cof - liberada -> True
cof liberado? nao -> True
```

No primeiro, depois de `cof:` e do espaço vem `nao`, e o padrão exige `liberad` ali. O terceiro mostra um limite: regex não entende frase. "COF liberado? não" é raro o bastante para aceitarmos o risco, mas vale um teste documentando o comportamento.

🔧 **Modifique:** a qualidade decidiu que "COF: OK" também libera. Mude o padrão para `liberad[oa]|ok` dentro de um grupo e escreva um teste para "COF: ok" e outro para "COF: ok?" antes de mudar o código.

7. Entradas (etapas 7 a 10)

7.1 Ler a planilha (etapa 7)

Regra de ouro das entradas: **olhe o arquivo real antes de escrever o leitor**.

```
python exemplos/04_espiar_planilha.py
abas: ['Linha A', 'Linha B', 'Linha C', 'Linha D']
cabeçalho: ('OP', 'Data', 'Linha', 'Produto', 'Especificação') ...
  'OP'                        -> 24518 (int)
  'Data'                      -> datetime.datetime(2026, 10, 20, 0, 0)
(datetime)
  'Linha'                     -> 'A' (str)
  'Produto'                   -> 'Filme PE 40 µm transparente' (str)
  'Especificação'             -> None (NoneType)
  'Espessura nominal (µm)'    -> 40 (int)
  'Espessura (µm)'            -> 41.2 (float)
Linha A: 5 OPs [24518, 24519, 24520, 24521, 24522]
Linha B: 4 OPs [24527, 24528, 24533, 24534]
Linha C: 4 OPs [24541, 24542, 24546, 24547]
Linha D: 3 OPs [24555, 24556, 24560]
```

Três descobertas que mudam o código: a OP vem como `int` (o pedido do e-mail traz `str`); célula vazia vem `None`; o cabeçalho tem acento e unidade entre parênteses.

Exemplo resolvido: OP e COF da aba Linha C, pelo nome da coluna

Primeiro penso: "não quero depender da posição da coluna; alguém pode inserir uma coluna nova amanhã". Então monto um dicionário `{nome normalizado: índice}` a partir do cabeçalho. Depois percorro as linhas com `enumerate(..., start=2)`, porque o cabeçalho é a linha 1 do Excel. (Também na pasta `14-projeto-final`.)

```
import unicodedata

from openpyxl import load_workbook


def normalizar(texto):
    decomposto = unicodedata.normalize("NFKD", str(texto or ""))
    sem_acento = "".join(c for c in decomposto if not
unicodedata.combining(c))
    return " ".join(sem_acento.lower().split())


pasta_trabalho = load_workbook("dados/planilha_valores.xlsx",
read_only=True, data_only=True)
aba = pasta_trabalho["Linha C"]
linhas = aba.iter_rows(values_only=True)
colunas = {normalizar(nome): i for i, nome in enumerate(next(linhas))}
for numero, linha in enumerate(linhas, start=2):
    print(numero, linha[colunas["op"]], linha[colunas["especificacao"]],
linha[colunas["cof"]])
pasta_trabalho.close()
```

```
2 24541 ESP-0412 0.21
3 24542 ESP-0412 0.27
4 24546 ESP-9999 0.2
5 24547 ESP-0618 0.16
```

A etapa 7 vai além: tira a unidade do cabeçalho (`"Espessura (µm)"` vira `"espessura"`), converte número digitado como texto (`"41,2"`), pula linhas vazias, recusa planilha sem coluna obrigatória (`PlanilhaInvalida`) e **fecha** o arquivo num `finally`.

> ⚠️ **Erro comum:** esquecer o `close()` com `read_only=True`. No Windows, o arquivo fica preso: o laboratório não consegue salvar a planilha enquanto o programa roda.

7.2 A linha bate com a aba? (etapa 8)

Cada aba é de uma linha, mas cada OP também tem a coluna "Linha". Se divergem, alguém copiou a OP para a aba errada. O sistema não escolhe: o padrão da linha muda as tolerâncias. Vira pendência: `linha informada C diferente da aba Linha B`.

7.3 Escolher a especificação (etapa 9)

Título vazio, padrão da linha. Título conhecido, padrão **mais** as tolerâncias do cliente por cima. Título desconhecido, `EspecificacaoDesconhecida`. A junção usa `{**padrao, **cliente}`.

> 🔮 **Preveja antes de rodar:**
>
> ```
> padrao = {"cof": 0.35, "selagem": 8}
> cliente = {"cof": 0.25}
> print({**padrao, **cliente})
> print({**cliente, **padrao})
> ```

▼ **Resposta**

```
{'cof': 0.25, 'selagem': 8}
{'cof': 0.35, 'selagem': 8}
```

Quem vem depois vence. Na ordem errada, o padrão da linha apagaria a exigência do cliente, e a OP 24542 (COF 0,27; cliente pede no máximo 0,25) seria aprovada.

7.4 Os e-mails da expedição (etapa 10)

Os padrões de regex já vêm prontos do módulo 05. O trabalho da etapa é **decidir**: tirar repetições, juntar todos os problemas e nunca escolher entre duas OPs.

> 🔮 **Preveja antes de rodar:** o assunto e o corpo citam a OP.

```
import re

RE_OP = re.compile(r"\bOP\b\s*(?:n[º°o]\.?\s*)?[:#-]?\s*(\d{4,6})\b",
re.IGNORECASE)
texto = "Assunto: Certificado - OP 24518\n\nOP: 24518\nNF: 123456"
achados = RE_OP.findall(texto)
print(achados)
print(sorted(set(achados)))
```

▼ **Resposta**

```
['24518', '24518']
['24518']
```

`findall` devolve todas as ocorrências. O `set` junta as iguais: a mesma OP duas vezes é uma OP só. Duas OPs **diferentes** continuam duas, e o e-mail fica fora do padrão.

Rodando os padrões sobre os 8 e-mails de exemplo:

```
python exemplos/06_emails_da_expedicao.py
email_01_op24518.txt     OP=['24518'] NF=['123456'] peso=['1.250,5']
email_02_op24519.txt     OP=['24519'] NF=['123457'] peso=['980,0']
email_03_op24527.txt     OP=['24527'] NF=['123460'] peso=['2.040']
email_04_op24541.txt     OP=['24541'] NF=['123789'] peso=['2.310,75']
email_05_op24546.txt     OP=['24546'] NF=['123790'] peso=['1.100']
  (email_06_op24533.txt não é UTF-8: lendo como cp1252)
email_06_op24533.txt     OP=['24533'] NF=['123801'] peso=['860,4']
email_07_sem_nf.txt      OP=['24555'] NF=[] peso=[]
email_08_duas_ops.txt    OP=['24520', '24521'] NF=['123805'] peso=
['3.200']
```

O e-mail 06 foi salvo no Windows em cp1252. Ler com plano B (`UnicodeDecodeError` → tenta cp1252) é a lição do módulo 05 aplicada. E o e-mail 07 mostra por que a regex de peso exige a palavra "peso": "(1.500 kg)" no meio da frase não é um campo confiável.

8. Saídas e orquestração (etapas 11 a 14)

8.1 A triagem (etapa 11)

É a camada de serviço: não lê arquivo, só chama as peças na ordem certa.

```
pedido -> OP existe? uma vez só? ──não──> bloqueado ("OP não encontrada"
/ "aparece 2 vezes")
            │sim
            ▼
          linha bate com a aba? ───não──> bloqueado (linha divergente)
            │sim
            ▼
          especificação conhecida? ─não─> bloqueado (especificação não
cadastrada)
            │sim
            ▼
          valida tolerâncias + liberação (junta TODOS os motivos)
            │
            ├── nenhum motivo ──> fila ("emitir")
            └── algum motivo ──> bloqueado (com os motivos)
```

As três primeiras checagens **param** no primeiro problema: sem saber qual OP ou qual especificação, não há o que validar. A última junta tudo. Uma função auxiliar com **retornos antecipados** deixa cada caso em uma linha.

> 💡 **Dica:** antes do laço, monte um índice `{op: [medições]}`. Cada pedido acha a sua OP direto, sem varrer a planilha inteira. E o caso "OP repetida" aparece de graça: é a lista com dois itens.

8.2 O emissor e o modo ensaio (etapa 12)

O sistema não sabe o que é TOTVS. Ele conhece uma interface:

```python
class Emissor:
    """Interface: todo emissor tem `ensaio` e `emitir(item) ->
comprovante`."""

    ensaio: bool = False

    def emitir(self, item: ItemFila) -> str:
        raise NotImplementedError
```

O `EmissorTotvs` de hoje só tem o modo ensaio: registra o que faria e devolve `"ENSAIO-24518-123456"`. A função `emitir_fila` cuida do resto: idempotência por (OP, NF), falha de uma OP vira "falha" e a série segue, bug sobe. O exemplo `exemplos/07_logging_e_modo_ensaio.py` mostra o padrão em miniatura:

```
python exemplos/07_logging_e_modo_ensaio.py
INFO     ENSAIO: emitiria certificado da OP 24518 / NF 123456
```

```
INFO     ENSAIO: emitiria certificado da OP 24541 / NF 123789
WARNING  OP 24518 / NF 123456 repetida: ignorada
```

🔮 **Preveja antes de rodar:** `replace` copia o item trocando a situação. Depois mexemos na lista do ORIGINAL.

```python
from dataclasses import dataclass, field, replace


@dataclass
class ItemFila:
    op: str
    situacao: str
    motivos: list[str] = field(default_factory=list)


original = ItemFila("24518", "emitir")
copia = replace(original, situacao="falha")
original.motivos.append("tela não respondeu")
print(copia.situacao, copia.motivos)
```

▼ **Resposta**

```
falha ['tela não respondeu']
```

`replace` faz cópia **rasa**: a cópia aponta para a mesma lista. Por isso a etapa 12 monta uma lista nova (`item.motivos + ["..."]`) em vez de dar `append` no item recebido. Mexer no que você recebeu é efeito colateral escondido.

8.3 O relatório da série (etapa 13)

CSV para o Excel brasileiro: separador `;`, vírgula decimal (`f"{1250.5:.2f}".replace(".", ",")` dá `1250,50`) e encoding `utf-8-sig`. Sem o BOM do `-sig`, o Excel mostra "EspecificaÃ§Ã£o" no cabeçalho. E `newline=""` no `open`, senão aparece uma linha em branco entre os registros no Windows.

O projeto completo grava também um Excel formatado, com três abas (Série, E-mails fora do padrão, Resumo) e um gráfico de barras nativo na aba Resumo.

8.4 A CLI (etapa 14)

A CLI é fina: lê argumentos, confere se os arquivos existem, chama quem faz o trabalho e mostra o resumo. Quem faz o trabalho chega por parâmetro (`executar`). Nos testes, entra um dublê que só anota a chamada; no projeto completo, a série real. Isso se chama **injeção de dependência**, e é o que deixa a CLI testável sem planilha, banco ou TOTVS.

O **código de saída** (`return 0` ou `return 1`) é a forma de um programa avisar outro programa. O Agendador de Tarefas do Windows e o GitHub Actions leem esse número: diferente de zero significa "precisa de gente".

> 🔧 **Modifique:** depois de passar na etapa 14, acrescente a opção `--quieto`, que não imprime o resumo (útil no agendador). Escreva o teste primeiro.

9. Montagem final: o projeto-completo

A pasta `projeto-completo/` é o sistema montado como um projeto independente: copie a pasta para fora do curso e ela funciona sozinha.

```
projeto-completo/
  pyproject.toml          metadados, dependências, comando "validador",
pytest e ruff
  README.md               o que é, como instalar e usar (a vitrine do
repositório)
  .gitignore              saida/, .venv/, .env e bancos que não são de
exemplo
  .github/workflows/      testes.yml: lint + testes + série de exemplo a
cada push
  sql/schema.sql          schema do banco de regras
  dados/                  planilha, banco e e-mails de exemplo
(fictícios)
  src/validador_certificados/
    modelos.py  texto.py  regras.py              <- núcleo puro
    banco.py  planilha.py  expedicao.py          <- bordas de entrada
    fila.py  serie.py                            <- serviço
    emissao.py  robo_playwright.py  historico.py  relatorio.py   <-
bordas de saída
    cli.py  __main__.py                          <- entrada
  tests/                  77 testes: unidade e ponta a ponta sobre dados/
```

Por que `src/`? Com o pacote dentro de `src`, os testes só enxergam o código depois de instalado. Parece burocracia, mas garante que o que você testa é o que o usuário instala, e não um arquivo solto que por acaso estava na pasta.

Os trechos principais do `pyproject.toml`:

```toml
[project]
name = "validador-certificados"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = ["openpyxl>=3.1"]

[project.optional-dependencies]
dev = ["pytest>=8.0", "ruff>=0.6"]
robo = ["playwright>=1.40"]

[project.scripts]
validador = "validador_certificados.cli:main"
```

`[project.scripts]` cria o comando `validador` no ambiente virtual. Os extras separam o que é obrigatório (openpyxl) do que é só para desenvolver (`dev`) ou para o robô (`robo`): o PC da expedição não precisa de pytest.

Instalar e rodar (PowerShell, dentro de `projeto-completo`):

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -e ".[dev]"
validador --planilha dados/planilha_valores.xlsx --regras dados/regras.db
--emails dados/emails
```

No Linux/macOS, ative com `source .venv/bin/activate`. A saída abaixo é do Linux (no Windows, os caminhos aparecem com `\`), e o carimbo de data e hora muda a cada execução:

```
INFO     série iniciada em modo ENSAIO
INFO     regras: 7 especificações carregadas de dados/regras.db
INFO     planilha: 16 OPs lidas de dados/planilha_valores.xlsx
INFO     expedição: 8 e-mails lidos de dados/emails
WARNING  e-mail fora do padrão: email_07_sem_nf.txt (NF não encontrada;
peso não encontrado)
WARNING  e-mail fora do padrão: email_08_duas_ops.txt (mais de uma OP no
e-mail (24520, 24521))
INFO     triagem: 2 para emitir, 4 bloqueados
WARNING  OP 24519 bloqueada: espessura 44.8 fora de 40 ± 10% (36 a 44)
WARNING  OP 24527 bloqueada: COF sem liberação na observação
WARNING  OP 24546 bloqueada: especificação 'ESP-9999' não cadastrada
WARNING  OP 24533 bloqueada: linha informada C diferente da aba Linha B
INFO     ENSAIO: emitiria certificado: OP 24518 | NF 123456 | 1250.50 kg
| PADRAO-A
INFO     ENSAIO: emitiria certificado: OP 24541 | NF 123789 | 2310.75 kg
```

```
| ESP-0412
INFO     relatórios gravados: relatorio_2026-10-29_024641.xlsx e
relatorio_2026-10-29_024641.csv
Série processada em modo ENSAIO (nada foi enviado ao TOTVS)
  bloqueado                 4
  ensaio                    2
  e-mails fora do padrão    2
Relatório: saida/relatorio_2026-10-29_024641.xlsx
```

Sem instalar nada, `python exemplos/08_serie_de_ponta_a_ponta.py` (na pasta `14-projeto-final`) roda a mesma série sobre os dados do módulo.

Logging em dois destinos. O terminal mostra INFO (ou DEBUG com `-v`); o arquivo `saida/validador.log` guarda sempre DEBUG, com data e hora. Quando alguém perguntar "por que a OP 24527 não saiu na terça?", a resposta está no log. Cada módulo cria o seu `logger = logging.getLogger(__name__)`, e só a CLI configura os destinos.

Testes e qualidade (dentro de `projeto-completo`, com o ambiente ativado):

```
python -m pytest
ruff check .
ruff format --check .
```

```
.....................................................................
[ 93%]
.....
[100%]
77 passed in 0.44s
```

O teste mais valioso é o de ponta a ponta (`tests/test_cli.py`): roda a CLI sobre `dados/` numa pasta temporária e confere o resumo, os arquivos gerados e que o ensaio não gravou histórico. Se ele passa, o sistema inteiro funciona.

10. O robô do TOTVS, isolado

O robô é a parte mais frágil: depende de uma tela que muda sem aviso. Por isso ele fica **atrás da interface** `Emissor`, e o resto do sistema nem sabe que ele existe.

O arquivo `src/validador_certificados/robo_playwright.py` é um **esqueleto** comentado (não testado automaticamente): a classe `EmissorTotvsPlaywright` com login, emissão, print de tela e tradução de erro. O coração dele:

```python
def emitir(self, item: ItemFila) -> str:
    from playwright.sync_api import TimeoutError as TempoEsgotado

    pagina = self._pagina
    try:
        # AJUSTE: caminho até a rotina de certificado de qualidade
        pagina.get_by_role("menuitem", name="Certificado de
Qualidade").click()
        pagina.get_by_label("Ordem de Produção").fill(item.op)
        pagina.get_by_label("Nota Fiscal").fill(item.nf)
        pagina.get_by_label("Peso").fill(f"
{item.peso_kg:.2f}".replace(".", ","))
        pagina.get_by_role("button", name="Confirmar").click()
        comprovante = pagina.get_by_test_id("numero-
certificado").inner_text()
    except TempoEsgotado as erro:
        self._print_de_tela(f"falha_{item.op}_{item.nf}")
        raise ErroEmissao(f"tela não respondeu: {erro}") from erro
    self._print_de_tela(f"ok_{item.op}_{item.nf}")
    return comprovante
```

Roteiro para implementar quando tiver acesso à tela:

1. `python -m pip install -e ".[robo]"`. O Edge já vem no Windows; o Playwright o usa com `channel="msedge"`.
2. Grave seus cliques uma vez: `playwright codegen --channel msedge <endereço do TOTVS>`. O codegen mostra os seletores de cada campo.
3. Troque os seletores marcados com `AJUSTE`. Prefira `get_by_role`, `get_by_label` e `get_by_text`. Se a tela estiver dentro de um iframe, use `frame_locator`.
4. Usuário e senha em variáveis de ambiente (`TOTVS_USUARIO`, `TOTVS_SENHA`) ou num `.env` lido com python-dotenv. **Nunca** no código, nunca no Git.
5. Rode em **homologação** (base de teste) com o navegador visível. Só depois em produção, e por algumas semanas comparando com a emissão manual.
6. Na CLI, troque `EmissorTotvs(ensaio=False)` pelo novo emissor quando `--emitir` for passado. Nada mais no sistema muda: esse é o prêmio da interface.

> ⚠️ **Erro comum:** usar `time.sleep(5)` para "esperar a tela". Ora é pouco (falha), ora é demais (lento). O Playwright espera sozinho o elemento ficar pronto; ajuste só o tempo máximo (`set_default_timeout`).

11. Do curso para o trabalho

1. **Troque os dados.** Copie a planilha real para uma pasta local e rode com `-v`. Se faltar coluna, a mensagem diz qual. Se o cabeçalho tiver outro nome, ajuste `chave_coluna` ou a planilha (combine com o laboratório).
2. **Cadastre as regras.** Abra `regras.db` no DB Browser for SQLite e cadastre os padrões das 4 linhas e as especificações de cliente. Comece pelos clientes com mais pedidos.
3. **Rode em paralelo.** Por duas semanas, faça a emissão manual como sempre e rode o sistema em ensaio. Compare o relatório com o que você decidiu. Cada diferença é um bug ou uma regra que faltou.
4. **Agende.** Um arquivo `rodar_serie.bat` e o Agendador de Tarefas do Windows (módulo 12) rodam a série toda manhã:

   ```
   @echo off
   cd /d C:\automacoes\validador
   call .venv\Scripts\activate
   validador --planilha "\\servidor\qualidade\valores.xlsx" --regras
   dados\regras.db --emails C:\automacoes\emails --saida saida
   ```

5. **Guarde o histórico.** A pasta `saida/` acumula relatórios e log. Faça backup dela junto com o `regras.db`.

> 💡 **Dica:** mostre o relatório de ensaio para o seu gestor antes de pedir acesso ao TOTVS para o robô. Um relatório certo por duas semanas é o melhor argumento.

12. Portfólio no GitHub

O `projeto-completo/` foi feito para virar o seu repositório. Passo a passo:

1. **Repositório próprio.** Copie a pasta para fora do curso (ex.: `C:\projetos\validador-certificados`) e, dentro dela:

   ```
   git init
   git add .
   git commit -m "Versão inicial do Validador de Certificados"
   ```

Crie o repositório vazio no site do GitHub (sem README) e siga as instruções de `git remote add origin ...` e `git push -u origin main` que ele mostra.

2. **README com prints.** Rode a série, tire prints (Windows: `Win+Shift+S`) do terminal e da aba "Série" do relatório, salve em `docs/img/` e cite no README: `!Relatório da série`. Um recrutador decide em 30 segundos; a imagem faz isso por você.

3. **Testes no GitHub Actions.** O arquivo `.github/workflows/testes.yml` já roda lint, testes e a série de exemplo no Windows e no Ubuntu a cada push. Depois do primeiro push, veja a aba **Actions**. Acrescente o selo no topo do README: `![testes](https://github.com/SEU_USUARIO/validador-certificados/actions/workflows/testes.yml/badge.svg)`.

4. **Só dados fictícios.** Nunca suba a planilha real, clientes, OPs ou o banco de produção. O `.gitignore` já barra `saida/` e `.env`. Na dúvida, não suba.

5. **Commits que contam a história.** Um commit por etapa ("Regra de tolerância nominal com limites arredondados") mostra como você pensa. Use branches para as extensões (módulo 09).

6. **Divulgue.** Fixe o repositório no seu perfil e escreva um post curto no LinkedIn: o problema, a solução, um print, o link.

13. Extensões

13.1 Robô Playwright no TOTVS. Seção 10. É a extensão que mais economiza tempo e a que exige mais cuidado.

13.2 Ler os e-mails direto do Outlook. Hoje a série lê `.txt`. No Outlook clássico do Windows, dá para ler uma pasta da caixa com `pywin32` (módulo 12, só Windows):

```
import win32com.client  # python -m pip install pywin32

outlook =
win32com.client.Dispatch("Outlook.Application").GetNamespace("MAPI")
caixa = outlook.GetDefaultFolder(6)  # 6 = Caixa de Entrada
pasta = caixa.Folders.Item("Certificados")
emails = {}
for mensagem in pasta.Items:
    if mensagem.UnRead:
        emails[f"{mensagem.EntryID[-12:]}.txt"] = f"
{mensagem.Subject}\n{mensagem.Body}"
```

O dicionário `{nome: texto}` é exatamente o que `separar_emails` recebe. Só a borda muda; a regra de e-mail continua a mesma. (No Outlook novo e no Microsoft 365, o caminho é a API Microsoft Graph, que pede autorização da TI.)

13.3 API com FastAPI. É o desafio deste módulo: outra porta de entrada para o mesmo núcleo, para o PCP consultar se uma OP pode ter certificado.

13.4 Outras ideias. Aviso por e-mail ao supervisor quando houver bloqueio; um gráfico de controle das espessuras por linha com matplotlib (módulo 10); guardar cada série num banco para medir o tempo entre o pedido e a emissão.

Lendo mensagens de erro deste módulo

Pacote não encontrado. Você rodou os testes ou o comando sem instalar o projeto, ou com o ambiente virtual errado ativado.

```
    from validador_certificados.cli import main
ModuleNotFoundError: No module named 'validador_certificados'
```

Correção: ative o `.venv` e rode `python -m pip install -e ".[dev]"` na pasta do projeto.

Banco errado. O caminho do banco estava errado e o `sqlite3.connect` criou um banco vazio.

```
    conexao.execute("SELECT titulo FROM especificacao")
    ~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
sqlite3.OperationalError: no such table: especificacao
```

Correção: confira o caminho; apague o `.db` vazio criado por engano; abra com `?mode=ro`.

Formato de planilha. openpyxl só lê `.xlsx` (e parentes). Um `.xls` antigo ou um `.csv` dão:

```
openpyxl.utils.exceptions.InvalidFileException: openpyxl does not support
.csv file format, please check you can open it with Excel first.
Supported formats are: .xlsx,.xlsm,.xltx,.xltm
```

Correção: abra no Excel e salve como `.xlsx`.

Coluna faltando. O erro é nosso, de propósito, e diz a aba e as colunas:

```
validador_certificados.modelos.PlanilhaInvalida: aba 'Linha B': faltam
colunas: cof, observacao
```

Correção: alguém renomeou ou apagou a coluna. Repare que o cabeçalho "Obs" não vale como "Observação".

Regra quebrada no banco. A `Tolerancia` recusa nascer:

```
    raise ValueError(f"{self.caracteristica}: mínimo maior que o máximo")
ValueError: cof: mínimo maior que o máximo
```

Correção: o cadastro inverteu mínimo e máximo. O sistema para em vez de validar com regra errada.

Argumento faltando. O argparse explica sozinho e sai com código 2:

```
usage: validador [-h] --planilha PLANILHA --regras REGRAS --emails EMAILS
                 [--saida SAIDA] [--emitir] [-v] [--version]
validador: error: the following arguments are required: --regras, --
emails
```

Arquivo aberto no Excel (Windows). Gravar um relatório com o mesmo nome de um que está aberto no Excel dá `PermissionError: [Errno 13] Permission denied`. A CLI trata como erro esperado (código 1). Correção: feche o arquivo; por isso cada série grava um relatório com data e hora no nome.

Boas práticas

- **Núcleo puro, bordas finas.** Se uma regra precisa de `open()`, ela está no lugar errado.
- **Nunca chutar.** Dado ambíguo vira pendência com motivo legível.
- **Modo seguro por padrão.** Ação perigosa exige pedido explícito (`--emitir`).
- **Idempotência.** Rodar duas vezes dá o mesmo resultado que rodar uma.
- **Capture só o erro esperado.** `except ErroEmissao`, nunca `except Exception` para esconder.
- **Log com contexto.** "OP 24527 bloqueada: COF sem liberação" ajuda; "erro" não.
- **Dados de exemplo versionados**, gerados por script, com um caso de cada tipo.
- **Teste de ponta a ponta** sobre esses dados, rodando no CI a cada push.
- **Segredos fora do código**: variável de ambiente ou `.env` no `.gitignore`.
- **Decisões escritas**: o porquê vale tanto quanto o como.

No mundo real

Sexta-feira, 16h, expedição cheia. Chegaram 8 e-mails pedindo certificado. Você roda:

```
validador --planilha dados/planilha_valores.xlsx --regras dados/regras.db
--emails dados/emails
```

Em um segundo, o resumo: 2 para emitir, 4 bloqueados, 2 e-mails fora do padrão. Você abre o relatório e decide, linha a linha:

Item	O sistema disse	O que você faz
OP 24518 e 24541	ensaio (seria emitido)	emite no TOTVS (ou deixa o robô emitir, no futuro)
OP 24519	espessura 44,8 fora de 40 ± 10%	avisa a produção; não há certificado para OP reprovada
OP 24527	COF sem liberação	pergunta ao laboratório quando sai o ensaio de 24 h
OP 24546	especificação ESP-9999 não cadastrada	confere o pedido do cliente e cadastra (ou corrige a planilha)
OP 24533	linha C digitada na aba B	corrige a planilha com quem lançou
e-mail 07	NF e peso não encontrados	responde à expedição pedindo a NF
e-mail 08	duas OPs no mesmo e-mail	pede um e-mail por OP (ou implementa a extensão)

O que levava uma hora de conferência virou dez minutos de **decisão**. Repare: o sistema não decidiu nada que não devia. Tudo o que era ambíguo voltou para uma pessoa, com o motivo escrito.

É essa história que você conta na entrevista.

Resumo

Tema	Essência
Camadas	núcleo puro (modelos, regras) ← serviço (fila) ← bordas (banco, planilha, e-mail, TOTVS, relatório) ← CLI
Modelo que se defende	`@dataclass` + `__post_init__` levantando `ValueError`

Tema	Essência
Regra	devolve `None` (passou) ou o motivo (`str`)
Tolerância nominal	`round(nominal ∓ nominal * pct / 100, 4)`, limites inclusivos
Especificação	vazio → padrão; conhecida → `{**padrao, **cliente}`; desconhecida → exceção
Liberação	normalizar + `re.search(r"\bcof\s*[:-]?\s*liberad[oa]\b", texto)`
Planilha	`load_workbook(..., read_only=True, data_only=True)`, cabeçalho → dict, `close()`
E-mails	`findall` + `set` + `sorted`; dois valores diferentes = fora do padrão
Emissão	interface `Emissor`; ensaio por padrão; chave `(OP, NF)`; `except ErroEmissao`
Relatório	CSV `;` + `utf-8-sig` + `newline=""`; Excel com openpyxl
CLI	argparse, `executar` injetado, código de saída 0/1
Projeto	src layout, `pip install -e ".[dev]"`, pytest, ruff, GitHub Actions

Você domina este módulo se...

- [ ] Desenho as camadas do Validador de memória e explico a regra de dependência.
- [ ] Explico, sem consultar, por que as regras devolvem motivo em vez de booleano.
- [ ] Mostro com um exemplo por que os limites da tolerância nominal são arredondados.
- [ ] Explico a diferença entre `{**padrao, **cliente}` e `{**cliente, **padrao}` e qual a consequência de trocar.
- [ ] Dou três exemplos de "nunca chutar" no sistema e digo o que acontece em cada um.
- [ ] Explico por que a chave de idempotência é (OP, NF) e por que capturo só `ErroEmissao`.
- [ ] Instalo o projeto-completo num ambiente virtual novo, rodo os testes e a série de exemplo sem olhar o README.
- [ ] Sei o que mudar (e o que NÃO mudar) para trocar o emissor de ensaio pelo robô.

Exercícios

Cada exercício é uma etapa do sistema, em ordem de construção. Juntos, eles já são a revisão integrada do curso: cada etapa usa assuntos de vários módulos anteriores.

Nº	Exercício	Nível	O que pratica
01	`ex01_modelos`	★☆☆	dataclasses, `__post_init__`, `@property` (módulo 08)
02	`ex02_carregar_regras`	★★☆	sqlite3, consultas, montar objetos (módulos 08 e 11)
03	`ex03_tolerancia_fixa`	★☆☆	regra pura, `None` × motivo, `:g` (módulos 01, 02 e 04)
04	`ex04_tolerancia_nominal`	★☆☆	float no limite, `round`, tupla (módulos 01 e 03)
05	`ex05_validar_op`	★★☆	laço sobre a especificação, juntar motivos (módulos 02 a 04)
06	`ex06_liberacao`	★★☆	leitura de código: prever e consertar; unicodedata, regex (módulo 05)
07	`ex07_ler_planilha`	★★★	openpyxl, cabeçalho → dict, conversões, `finally` (módulos 06 e 10)
08	`ex08_conferir_linha`	★☆☆	`re.fullmatch`, mensagens de pendência (módulo 05)
09	`ex09_escolher_especificacao`	★★☆	`{**a, **b}`, exceção personalizada, "nunca chutar" (módulos 03 e 06)
10	`ex10_emails_expedicao`	★★☆	regex, número BR, juntar problemas (módulos 05 e 06)
11	`ex11_fila_emissao`	★★★	camada de serviço, índice, retornos antecipados (módulos 03, 04 e 06)
12	`ex12_emissor_ensaio`	★★★	interface, herança, idempotência, `replace` (módulos 08 e 12)
13	`ex13_relatorio_serie`	★☆☆	`Counter`, CSV no padrão Excel BR (módulos 06 e 07)
14	`ex14_cli`	★★☆	argparse, injeção de dependência, código de saída (módulos 07 e 09)

Como fazer: abra `exercicios/exNN_...py`, leia o enunciado, escreva o código no lugar de `raise NotImplementedError` e rode, na pasta `curso-python`:

```
python curso.py testar 14 ex01
```

Para testar o módulo inteiro: `python curso.py testar 14`.

Desafio do módulo

A API do Validador (extensão FastAPI). Uma API com `/saude`, `/especificacoes`, `/especificacoes/{titulo}` e `POST /validar`, que importa o núcleo do `projeto-completo` sem reescrever regra nenhuma, com testes no `TestClient`. Enunciado completo em desafio/README.md.

Quiz

Teste o que ficou: quiz.md.

Para ir além

- dataclasses: https://docs.python.org/pt-br/3/library/dataclasses.html
- argparse (tutorial): https://docs.python.org/pt-br/3/howto/argparse.html
- logging (tutorial): https://docs.python.org/pt-br/3/howto/logging.html
- sqlite3: https://docs.python.org/pt-br/3/library/sqlite3.html
- Empacotamento com pyproject.toml (guia oficial, em inglês): https://packaging.python.org/en/latest/guides/writing-pyproject-toml/
- Playwright para Python, gerador de código (em inglês): https://playwright.dev/python/docs/codegen
- GitHub Actions para Python (em inglês): https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-python
- Leitura extra: "Arquitetura Hexagonal" (Ports and Adapters), de Alistair Cockburn: o nome formal da ideia "núcleo puro e bordas" deste módulo.

Enunciados dos exercícios

Exercício 14.01 — Modelos de dados do sistema

Arquivo: `exercicios/ex01_modelos.py`

```
Nível: ★☆☆

Contexto: Todo o Validador de Certificados conversa por meio de "fichas":
Tolerancia,
Especificacao, Medicao, Pedido, ResultadoValidacao e ItemFila. Os campos
já estão
declarados abaixo. Leia com calma: eles aparecem em todas as etapas
seguintes. Falta o
comportamento que impede uma ficha inválida de existir.

Tarefa: implemente três coisas.
1. Tolerancia.__post_init__: levante ValueError quando
   - tipo não for "fixa" nem "nominal";
   - tipo "fixa" vier sem mínimo E sem máximo (os dois None);
   - tipo "fixa" tiver mínimo maior que o máximo (quando os dois
existem);
   - tipo "nominal" tiver percentual None, zero ou negativo.
2. Pedido.__post_init__: levante ValueError quando op ou nf estiverem
vazios (ou só
   com espaços), ou quando peso_kg for zero ou negativo.
3. ResultadoValidacao.aprovado (propriedade): True quando a lista de
motivos está
   vazia; False quando há pelo menos um motivo.

Exemplos:
    >>> Tolerancia("cof", "fixa", minimo=0.15, maximo=0.35)
    Tolerancia(caracteristica='cof', tipo='fixa', minimo=0.15,
maximo=0.35, percentual=None)
    >>> Tolerancia("cof", "faixa")
    Traceback (most recent call last):
    ValueError: tipo de tolerância inválido: 'faixa'
    >>> ResultadoValidacao("24518", "PADRAO-A").aprovado
    True
    >>> ResultadoValidacao("24519", "PADRAO-A", ["espessura
alta"]).aprovado
    False

Dica: no __post_init__ os campos já estão preenchidos (self.tipo,
self.minimo...).
Uma propriedade é um método com @property em cima: lê-se sem parênteses.

Teste com:  python curso.py testar 14 ex01
```

Exercício 14.02 — Carregar as regras do banco SQLite

Arquivo: `exercicios/ex02_carregar_regras.py`

```
Nível: ★★☆

Contexto: As especificações e tolerâncias ficam num banco SQLite (schema
em SCHEMA,
logo abaixo). Padrões de linha têm a coluna `linha` preenchida
('A'..'D');
especificações de cliente têm `linha` NULL. O resto do sistema não quer
saber de SQL:
quer um dicionário de objetos Especificacao prontos.

Tarefa: implemente `carregar_especificacoes(conexao)`.
- Devolva {titulo: Especificacao} com TODAS as linhas da tabela
especificacao.
- `linha` NULL no banco vira "" no objeto.
- Cada Especificacao recebe em `tolerancias` um dicionário
  {caracteristica: Tolerancia} com as linhas da tabela tolerancia daquele
título
  (minimo, maximo e percentual NULL viram None).
- Especificação sem nenhuma tolerância fica com o dicionário vazio.
- Banco sem especificações -> {}.
- Não feche a conexão: ela é de quem chamou.
- Se o banco tiver uma tolerância sem sentido (ex.: 'fixa' sem mínimo e
sem máximo),
  deixe o ValueError da Tolerancia subir: regra quebrada não pode ser
ignorada.

Exemplos:
    >>> especificacoes = carregar_especificacoes(conexao)
    >>> sorted(especificacoes)
    ['ESP-0412', 'PADRAO-A']
    >>> especificacoes["PADRAO-A"].linha, especificacoes["ESP-
0412"].linha
    ('A', '')
    >>> especificacoes["ESP-0412"].tolerancias["cof"]
    Tolerancia(caracteristica='cof', tipo='fixa', minimo=0.18,
maximo=0.25, percentual=None)

Dica: duas consultas resolvem: primeiro SELECT nas especificações (monte
o dicionário),
depois SELECT nas tolerâncias (pendure cada uma na especificação do seu
título).
`for a, b, c in conexao.execute(sql):` desempacota cada linha.

Teste com:  python curso.py testar 14 ex02
```

Exercício 14.03 — Regra de tolerância fixa (mínimo/máximo)

Arquivo: `exercicios/ex03_tolerancia_fixa.py`

```
Nível: ★☆☆

Contexto: COF, tração e selagem têm limites fixos. Às vezes só um lado
existe
(tração: só mínimo). O relatório precisa dizer POR QUE a OP reprovou,
então a regra
não devolve True/False: devolve None (passou) ou um texto com o motivo.

Tarefa: implemente `verificar_fixa(caracteristica, valor, minimo,
maximo)`.
- minimo ou maximo None significa "sem limite daquele lado".
- Os limites são inclusivos: valor igual ao mínimo ou ao máximo passa.
- Abaixo do mínimo: devolva "<caracteristica> <valor> abaixo do mínimo
<minimo>".
- Acima do máximo: devolva "<caracteristica> <valor> acima do máximo
<maximo>".
- Dentro: devolva None.
- Formate os números com :g (0.30 vira 0.3; 8.0 vira 8).

Exemplos:
    >>> verificar_fixa("cof", 0.22, 0.15, 0.35) is None
    True
    >>> verificar_fixa("cof", 0.38, 0.15, 0.35)
    'cof 0.38 acima do máximo 0.35'
    >>> verificar_fixa("selagem", 7.6, 8.0, None)
    'selagem 7.6 abaixo do mínimo 8'
    >>> verificar_fixa("tracao", 999.0, 20.0, None) is None
    True

Dica: teste "minimo is not None" ANTES de comparar: None < 0.15 dá
TypeError.

Teste com:  python curso.py testar 14 ex03
```

Exercício 14.04 — Regra nominal ± % (espessura e gramatura)

Arquivo: `exercicios/ex04_tolerancia_nominal.py`

```
Nível: ★☆☆

Contexto: Espessura e gramatura não têm limite fixo: a tolerância é um
percentual
sobre o valor nominal da OP (ex.: 40 µm ± 10% aceita de 36 a 44). Cuidado
com o
float: 37 * (1 - 10/100) dá 33.300000000000004, e uma gramatura medida de
33.3
(exatamente no limite) seria reprovada injustamente.
```

```
Tarefa: implemente duas funções.
1. `limites_nominal(nominal, percentual)` devolve a tupla (minimo,
maximo) =
   nominal ∓ nominal * percentual / 100, cada limite arredondado com
round(x, 4).
   Levante ValueError se nominal ou percentual forem zero ou negativos.
2. `verificar_nominal(caracteristica, valor, nominal, percentual)` usa os
limites
   (inclusivos) e devolve None se passou, ou o texto
   "<caracteristica> <valor> fora de <nominal> ± <percentual>% (<minimo>
a <maximo>)"
   com todos os números formatados com :g.

Exemplos:
    >>> limites_nominal(40, 10)
    (36.0, 44.0)
    >>> limites_nominal(37, 10)
    (33.3, 40.7)
    >>> verificar_nominal("gramatura", 33.3, 37, 10) is None
    True
    >>> verificar_nominal("espessura", 44.8, 40, 10)
    'espessura 44.8 fora de 40 ± 10% (36 a 44)'

Dica: em verificar_nominal, reaproveite limites_nominal e desempacote a
tupla.
Uma comparação encadeada (a <= x <= b) lê como na matemática.

Teste com:  python curso.py testar 14 ex04
```

Exercício 14.05 — Validar uma OP inteira

Arquivo: `exercicios/ex05_validar_op.py`

```
Nível: ★★☆

Contexto: Você já tem as regras fixa e nominal (peças prontas abaixo).
Agora é
juntar: para cada tolerância da especificação, pegar o valor medido da
OP, aplicar a
regra certa e acumular TODOS os motivos de reprovação. Quem corrige a OP
quer ver
todos os problemas de uma vez, não um por vez.

Tarefa: implemente `validar_medicao(medicao, especificacao)` e devolva um
ResultadoValidacao(op=medicao.op, especificacao=especificacao.titulo,
motivos=[...]).
Para cada Tolerancia em especificacao.tolerancias (na ordem do
dicionário):
- valor = medicao.valores.get(caracteristica). Se não houver valor:
```

```
  motivo "<caracteristica> sem valor medido" e passe para a próxima.
- tipo "fixa": use verificar_fixa(caracteristica, valor, minimo, maximo).
- tipo "nominal": o nominal vem de medicao.nominais. Se não houver:
  motivo "<caracteristica> sem valor nominal". Se houver: use
verificar_nominal.
- Se a regra devolver um texto, ele entra na lista de motivos.
Características medidas que a especificação não exige são ignoradas.

Exemplos:
    >>> resultado = validar_medicao(medicao_24519, padrao_a)
    >>> resultado.motivos
    ['espessura 44.8 fora de 40 ± 10% (36 a 44)']
    >>> resultado.aprovado
    False

Dica: o laço é "for tolerancia in especificacao.tolerancias.values():".
Use continue
para pular para a próxima característica depois de anotar um motivo.

Teste com:  python curso.py testar 14 ex05
```

Exercício 14.06 — Observação de liberação: leia, preveja e conserte

Arquivo: `exercicios/ex06_liberacao.py`

```
Nível: ★★☆

Contexto: Um estagiário escreveu a primeira versão da checagem de
liberação do COF:

    def esta_liberado_v1(observacao):
        return "liberado" in observacao.lower()

Ela foi para produção e deu problema: OP sem liberação entrou na fila.
Leia o código
com atenção antes de continuar.

Tarefa 1 (leitura de código): implemente `prever_v1()` devolvendo uma
lista com o
que esta_liberado_v1 devolveria para estas observações, nesta ordem.
Responda SEM
RODAR primeiro; depois confira.
    1. "COF: não liberado - refazer ensaio"
    2. "Liberado para corte; COF pendente"
    3. "COF: líberado"      (o técnico digitou um acento sem querer)
    4. None                 (célula vazia na planilha)
Use True ou False. Quando a função quebraria, use o NOME da exceção como
```

```
texto
(ex.: "TypeError").

Tarefa 2: implemente `normalizar(texto)`: minúsculas, sem acento e com
espaços
simples (sem espaço nas pontas e um só espaço entre palavras). None vira
"".

Tarefa 3: implemente a versão correta, `esta_liberado(observacao)`. Ela
devolve True
só se o texto normalizado tiver a palavra "cof" seguida de "liberado" ou
"liberada"
(podendo haver ":" ou "-" e espaços no meio). "cof: nao liberado",
"aguardando",
"liberado" sem "cof", texto vazio e None devolvem False.

Exemplos:
    >>> normalizar("  COF:   Não  Liberado ")
    'cof: nao liberado'
    >>> esta_liberado("COF - LIBERADO")
    True
    >>> esta_liberado("Cof: Liberado. Bobina 2 com rebarba")
    True
    >>> esta_liberado("COF: não liberado - refazer ensaio")
    False

Dica: unicodedata.normalize("NFKD", texto) separa a letra do acento ("í"
vira "i" +
acento); descarte os caracteres c em que unicodedata.combining(c) é
verdadeiro.
Para a marca, use re.search com: \b + cof + \s* + [:-]? + \s* +
liberad[oa] + \b.

Teste com:  python curso.py testar 14 ex06
```

Exercício 14.07 — Ler a planilha de valores (4 abas) com openpyxl

Arquivo: `exercicios/ex07_ler_planilha.py`

```
Nível: ★★★

Contexto: O laboratório preenche um .xlsx com uma aba por linha de
produção
("Linha A" ... "Linha D"), cabeçalho na linha 1 e uma OP por linha. Gente
digita
cabeçalho de jeitos diferentes ("Espessura (µm)", "ESPESSURA (um)") e às
vezes digita
número como texto ("41,2"). O programa deve ser tolerante no FORMATO e
rígido no
```

```
CONTEÚDO.

Tarefa: implemente três funções.
1. `chave_coluna(cabecalho)`: apague o que estiver entre parênteses
(unidade),
   normalize (peça pronta `normalizar`) e troque espaços por "_".
   "Espessura nominal (µm)" -> "espessura_nominal"; None -> "".
2. `numero_celula(valor)`: int ou float -> float; texto com vírgula ou
ponto
   ("41,2", " 41.2 ") -> float; None, "" ou texto que não é número
("n/d") -> None.
   (bool é subclasse de int: True/False também devolvem None.)
3. `ler_planilha(caminho)` devolve list[Medicao], na ordem das abas e das
linhas:
   - abra com load_workbook(caminho, read_only=True, data_only=True) e
FECHE no
     final (try/finally com .close());
   - leia só as abas cujo nome normalizado começa com "linha";
   - a 1ª linha é o cabeçalho: monte {chave_coluna(nome): índice da
coluna};
   - se faltar alguma chave de OBRIGATORIAS, levante PlanilhaInvalida com
uma
     mensagem que cite a aba e as colunas faltando;
   - pule linhas sem OP; para as demais crie Medicao com: op e linha via
     texto_celula; aba = nome da aba; linha_excel = número da linha no
Excel
     (o cabeçalho é a 1, então a 1ª OP é a 2); valores = {característica:
número}
     só das CARACTERISTICAS preenchidas; nominais = {"espessura": ...,
"gramatura": ...}
     (colunas de NOMINAIS) só se preenchidos; especificacao, observacao e
produto
     via texto_celula ("" quando vazio ou quando a coluna não existe).

Exemplos:
    >>> chave_coluna("Tração (MPa)")
    'tracao'
    >>> numero_celula("41,2")
    41.2
    >>> medicoes = ler_planilha(Path("dados/planilha_valores.xlsx"))
    >>> len(medicoes), medicoes[0].op, medicoes[0].aba,
medicoes[0].linha_excel
    (16, '24518', 'Linha A', 2)

Dica: aba.iter_rows(values_only=True) devolve tuplas de valores; chame
next() uma vez
para tirar o cabeçalho e use enumerate(..., start=2) no resto. Montar um
dict por
linha ({chave: valor}) deixa o resto do código bem legível.
```

```
Teste com:  python curso.py testar 14 ex07
```

Exercício 14.08 — A linha digitada bate com a aba?

Arquivo: `exercicios/ex08_conferir_linha.py`

```
Nível: ★☆☆

Contexto: Cada aba da planilha é de uma linha de produção, mas cada OP
também tem a
coluna "Linha". Quando alguém copia uma OP para a aba errada, as duas
discordam. O
sistema não pode escolher uma delas: o padrão da linha muda as
tolerâncias. Vira
pendência.

Tarefa: implemente duas funções.
1. `normalizar_linha(texto)`: aceita "A", " a ", "Linha A", "LINHA  b" e
devolve a
   letra maiúscula ("A".."D"). Qualquer outra coisa ("Linha E", "AB", "",
None,
   "Resumo") devolve None.
2. `conferir_linha(nome_aba, linha_informada)` devolve None se estiver
tudo certo, ou:
   - "aba '<nome_aba>' não corresponde a uma linha de produção" se a aba
não for
     de linha;
   - "linha não informada ou inválida: '<texto>'" se a linha informada
não for
     reconhecida (None conta como texto vazio: '');
   - "linha informada <X> diferente da aba <nome_aba>" se as letras forem
diferentes.

Exemplos:
    >>> normalizar_linha(" linha  b ")
    'B'
    >>> print(normalizar_linha("Linha E"))
    None
    >>> print(conferir_linha("Linha B", "b"))
    None
    >>> conferir_linha("Linha B", "C")
    'linha informada C diferente da aba Linha B'

Dica: depois de strip() e upper(), re.fullmatch(r"(?:LINHA\s*)?([A-D])",
texto)
resolve normalizar_linha: o grupo 1 é a letra. Para as aspas da mensagem,
{texto!r}.
```

```
Teste com:  python curso.py testar 14 ex08
```

Exercício 14.09 — Escolher a especificação da OP (sem chutar)

Arquivo: `exercicios/ex09_escolher_especificacao.py`

```
Nível: ★★☆

Contexto: Na planilha, a coluna "Especificação" traz o título da
especificação do
cliente (ex.: "ESP-0412") ou fica vazia. Vazia = vale o padrão da linha.
A
especificação do cliente lista só o que muda: o resto vem do padrão da
linha. Título
que não existe no banco é pendência: usar o padrão "porque é parecido"
pode liberar
um certificado errado.

Tarefa: implemente duas funções.
1. `chave_titulo(titulo)`: a forma "canônica" do título, para comparar:
sem espaços
   nas pontas, espaços repetidos viram um só, tudo em maiúsculas.
2. `escolher_especificacao(titulo, linha, especificacoes)`:
- `especificacoes` é {titulo: Especificacao}. O padrão da linha é a
especificação
  cujo campo `linha` é igual a `linha` ("A".."D"). Se a linha não tiver
padrão,
  levante EspecificacaoDesconhecida("linha <linha> sem especificação
padrão
  cadastrada").
- Título None, vazio ou só espaços: devolva o padrão da linha.
- Título informado: compare as chaves (chave_titulo) do título e dos
títulos do
  dicionário (" esp-0412 " encontra "ESP-0412"). Se não existir, levante
  EspecificacaoDesconhecida("especificação '<titulo sem espaços nas
pontas>' não
  cadastrada"). NUNCA devolva o padrão nesse caso.
- Título encontrado: devolva uma NOVA Especificacao com titulo, texto e
linha da
  especificação encontrada e tolerancias = as do padrão, sobrescritas
pelas da
  especificação do cliente. Não altere os objetos originais.

Exemplos:
    >>> chave_titulo("  esp-0412 ")
    'ESP-0412'
    >>> escolher_especificacao("", "A", especificacoes).titulo
    'PADRAO-A'
```

```
    >>> esp = escolher_especificacao(" esp-0412 ", "C", especificacoes)
    >>> esp.titulo, esp.tolerancias["cof"].maximo,
esp.tolerancias["espessura"].percentual
    ('ESP-0412', 0.25, 8.0)
    >>> escolher_especificacao("ESP-9999", "A", especificacoes)
    Traceback (most recent call last):
    EspecificacaoDesconhecida: especificação 'ESP-9999' não cadastrada

Dica: {**a, **b} cria um dicionário novo com tudo de a e, por cima, tudo
de b.
Para achar o padrão: next((e for e in especificacoes.values() if ...),
None).

Teste com:  python curso.py testar 14 ex09
```

Exercício 14.10 — Ler os pedidos da expedição (OP, NF e peso)

Arquivo: `exercicios/ex10_emails_expedicao.py`

```
Nível: ★★☆

Contexto: A expedição pede certificado por e-mail, cada pessoa de um
jeito: "OP:
24518", "OP nº 24541", "NF 123457", "NF-e nº 123.789", "Nota fiscal:
123460",
"Peso: 1.250,5 kg", "Peso líquido: 2.040 kg". Um e-mail = um pedido. Se
faltar algo,
ou se houver dois valores diferentes para o mesmo campo (duas OPs, por
exemplo), o
e-mail fica "fora do padrão": o sistema não escolhe por você.

Tarefa: implemente três funções.
1. `numero_br(texto)`: "1.250,5" -> 1250.5; "2.040" -> 2040.0; "980,0" ->
980.0.
2. `ler_email(texto, origem="")` devolve Pedido(op, nf, peso_kg, origem).
Os padrões
   RE_OP, RE_NF e RE_PESO já estão prontos (vêm do módulo 05). Com eles:
   - NF: tire os pontos ("123.789" -> "123789"), inclusive um ponto final
de frase;
   - peso: converta com numero_br;
   - valores repetidos iguais contam uma vez só (a OP costuma estar no
assunto E
     no corpo; "1.500 kg" e "1500 kg" são o mesmo peso);
   - junte TODOS os problemas, nesta ordem, e levante PedidoInvalido com
eles
     separados por "; ":
       "OP não encontrada" / "mais de uma OP no e-mail (24520, 24521)"
       "NF não encontrada" / "mais de uma NF no e-mail (1, 2)"
       "peso não encontrado" / "mais de um peso no e-mail (1500, 1700)"
```

```
       "peso deve ser maior que zero"
     (nas listas, valores em ordem crescente; pesos formatados com :g).
3. `separar_emails(emails)` recebe {nome_do_arquivo: texto} e devolve
   (lista de Pedido com origem=nome, {nome: mensagem do PedidoInvalido}).

Exemplos:
    >>> ler_email("OP: 24518\nNF: 123456\nPeso: 1.250,5 kg")
    Pedido(op='24518', nf='123456', peso_kg=1250.5, origem='')
    >>> ler_email("A carga da OP 24555 (1.500 kg) está pronta. A NF sai
amanhã.")
    Traceback (most recent call last):
    PedidoInvalido: NF não encontrada; peso não encontrado

Dica: RE_OP.findall(texto) devolve a lista dos grupos ("24518", ...);
set() tira
as repetições e sorted() põe em ordem. Teste os padrões no REPL antes de
escrever o
resto: é assim que se descobre o que eles devolvem.

Teste com:  python curso.py testar 14 ex10
```

Exercício 14.11 — Montar a fila de emissão (a triagem)

Arquivo: `exercicios/ex11_fila_emissao.py`

```
Nível: ★★★

Contexto: Chegou a hora de juntar as peças. Para cada pedido da
expedição, o sistema
decide: vai para a fila de emissão (OP liberada e aprovada) ou fica
bloqueado, com o
motivo. Esta é a camada de serviço: não lê arquivo nenhum, só usa as
peças prontas
das etapas anteriores (já estão no arquivo, antes da sua parte).

Tarefa: implemente `montar_fila(pedidos, medicoes, especificacoes)` e
devolva a tupla
(fila, bloqueados): duas listas de ItemFila, cada uma na ordem dos
pedidos. Para cada
pedido, nesta ordem (as 3 primeiras checagens PARAM no primeiro
problema):
  1. Procure a OP do pedido nas medições (medicao.op == pedido.op).
     Nenhuma -> bloqueado ["OP não encontrada na planilha"].
     Mais de uma -> bloqueado ["OP aparece <n> vezes na planilha"].
  2. conferir_linha(medicao.aba, medicao.linha): se devolver texto ->
bloqueado [texto].
  3. escolher_especificacao(medicao.especificacao,
normalizar_linha(medicao.aba),
     especificacoes): se levantar EspecificacaoDesconhecida -> bloqueado
```

```
[str(erro)].
  4. validar_medicao(medicao, especificacao): os motivos entram na lista.
  5. Se NÃO esta_liberado(medicao.observacao): acrescente SEM_LIBERACAO.
  Se a lista de motivos dos passos 4-5 estiver vazia -> ItemFila com
situacao
  "emitir"; senão, situacao "bloqueado" com os motivos.
Todo ItemFila leva op, nf e peso_kg do pedido. O campo especificacao
recebe o título
da especificação escolhida (fica "" quando o bloqueio acontece antes do
passo 3).

Exemplos:
    >>> fila, bloqueados = montar_fila(pedidos, medicoes, especificacoes)
    >>> [(i.op, i.situacao) for i in fila]
    [('24518', 'emitir')]
    >>> bloqueados[0].motivos
    ['espessura 44.8 fora de 40 ± 10% (36 a 44)']

Dica: monte antes um índice {op: [medições]} com dict.setdefault(op,
[]).append(m):
cada pedido acha suas medições direto, sem varrer a lista inteira. A
função
auxiliar triar(pedido, encontradas, especificacoes) já está declarada:
decidir UM
pedido nela, com retornos antecipados, deixa montar_fila bem curta.
(Opcional: se
preferir fazer tudo em montar_fila, pode apagar triar.)

Teste com:  python curso.py testar 14 ex11
```

Exercício 14.12 — Emissor do TOTVS em modo ensaio (e a emissão da fila)

Arquivo: `exercicios/ex12_emissor_ensaio.py`

```
Nível: ★★★

Contexto: O robô do TOTVS ainda não existe (vai ser Playwright no Edge).
Mas o resto
do sistema já pode ser construído contra uma INTERFACE: a classe Emissor
(pronta
abaixo), com `ensaio` e `emitir(item) -> comprovante`. Hoje o
EmissorTotvs só tem o
modo ensaio: registra o que faria. Quando o robô ficar pronto, nada fora
dele muda.

Tarefa: implemente três coisas.
1. EmissorTotvs.__init__(ensaio=True): se ensaio for False, levante
```

```
   EmissaoIndisponivel (o modo real ainda não existe). Senão, guarde
   self.ensaio = True e self.registro = [] (lista vazia).
2. EmissorTotvs.emitir(item): acrescente ao registro o texto
   "emitiria certificado: OP <op> | NF <nf> | <peso com 2 casas> kg |
<especificacao>"
   e devolva o comprovante "ENSAIO-<op>-<nf>".
3. emitir_fila(fila, emissor, ja_emitidas=None) devolve uma lista NOVA de
ItemFila,
   um para cada item da fila, na mesma ordem:
   - item com situacao diferente de "emitir": volta como está;
   - (op, nf) em ja_emitidas, ou já emitido antes NESTA mesma fila:
situacao
     "ignorado" e motivo "certificado já emitido para esta OP/NF"
acrescentado
     (o emissor NÃO é chamado);
   - emissor.emitir(item) deu certo: situacao "ensaio" se emissor.ensaio,
senão
     "emitido"; comprovante = o que emitir devolveu;
   - emissor.emitir levantou ErroEmissao: situacao "falha" e motivo
     "falha na emissão: <mensagem do erro>" acrescentado; a série
CONTINUA;
   - qualquer outra exceção é bug: deixe subir.
   Não altere os itens recebidos nem o conjunto ja_emitidas (trabalhe com
cópias).

Exemplos:
    >>> emissor = EmissorTotvs()
    >>> resultado = emitir_fila([item_24518], emissor)
    >>> resultado[0].situacao, resultado[0].comprovante
    ('ensaio', 'ENSAIO-24518-123456')
    >>> emissor.registro
    ['emitiria certificado: OP 24518 | NF 123456 | 1250.50 kg | PADRAO-
A']

Dica: dataclasses.replace(item, situacao="falha", motivos=novos) cria uma
CÓPIA do
item com esses campos trocados. set(ja_emitidas or ()) cria um conjunto
novo (vazio
quando ja_emitidas for None).

Teste com:  python curso.py testar 14 ex12
```

Exercício 14.13 — Relatório da série (CSV para o Excel brasileiro)

Arquivo: `exercicios/ex13_relatorio_serie.py`

```
Nível: ★☆☆

Contexto: No fim da série, a qualidade quer abrir um arquivo no Excel e
```

```
ver cada
pedido: o que foi (ou seria) emitido, o que ficou bloqueado e por quê. O
Excel em
português espera ";" como separador e vírgula decimal, e só reconhece
acentos de um
CSV em UTF-8 se o arquivo tiver BOM (encoding "utf-8-sig").

Tarefa: implemente duas funções.
1. `resumir(itens)` devolve {situacao: quantidade}, com as chaves em
ordem
   alfabética. Lista vazia -> {}.
2. `escrever_relatorio_csv(itens, caminho)` grava o CSV e devolve o
caminho:
   - encoding "utf-8-sig", delimitador ";", newline="" no open;
   - crie a pasta do arquivo se ela não existir;
   - cabeçalho: COLUNAS (pronto abaixo);
   - uma linha por item, na ordem recebida: op, nf, peso com 2 casas e
vírgula
     decimal ("1250,50"), especificacao, situacao em MAIÚSCULAS, motivos
unidos por
     " | " e comprovante.

Exemplos:
    >>> resumir(itens)
    {'bloqueado': 4, 'ensaio': 2}
    >>> caminho = escrever_relatorio_csv(itens, Path("saida/serie.csv"))
    >>> print(caminho.read_text(encoding="utf-8-sig").splitlines()[1])
    24518;123456;1250,50;PADRAO-A;ENSAIO;;ENSAIO-24518-123456

Dica: collections.Counter conta para você; dict(sorted(contagem.items()))
ordena.
csv.writer(arquivo, delimiter=";") e writerow([...]) escrevem cada linha.

Teste com:  python curso.py testar 14 ex13
```

Exercício 14.14 — A linha de comando (argparse) que junta tudo

Arquivo: `exercicios/ex14_cli.py`

```
Nível: ★★☆

Contexto: Na expedição de sexta-feira ninguém quer abrir o VS Code. O
sistema roda
com um comando:
    validador --planilha valores.xlsx --regras regras.db --emails
pasta_emails
A CLI é fina: lê argumentos, confere se os arquivos existem, chama quem
faz o
trabalho e mostra o resumo. Quem faz o trabalho chega pelo parâmetro
```

```
`executar`
(injeção de dependência): nos testes, um dublê; no projeto completo, a
série real.

Tarefa: implemente duas funções.
1. `criar_parser()` devolve um argparse.ArgumentParser com
prog="validador" e:
   --planilha, --regras e --emails: obrigatórios, type=Path;
   --saida: type=Path, padrão Path("saida");
   --emitir: flag (store_true), padrão False;
   -v/--verbose: flag (store_true), padrão False.
2. `main(argv=None, executar=executar_demo)` devolve o código de saída
(int):
   a) args = criar_parser().parse_args(argv);
   b) confira, nesta ordem: planilha é arquivo, regras é arquivo, emails
é pasta.
      No primeiro que falhar, imprima em sys.stderr
      "Erro: planilha não encontrada: <caminho>" (ou "banco de regras não
      encontrado: ..." ou "pasta de e-mails não encontrada: ...") e
devolva 1,
      sem chamar executar;
   c) resumo = executar(planilha=..., regras=..., emails=..., saida=...,
      ensaio=not args.emitir)   (tudo por nome);
   d) imprima "Modo: ENSAIO" (ou "Modo: REAL" com --emitir) e depois uma
linha
      "<situacao>: <quantidade>" para cada item do resumo, em ordem
alfabética;
   e) devolva 1 se o resumo tiver "falha" maior que zero; senão, 0.

Exemplos:
    >>> main(["--planilha", "valores.xlsx", "--regras", "regras.db",
    ...       "--emails", "emails"], executar=lambda **kw: {"ensaio": 2,
"bloqueado": 4})
    Modo: ENSAIO
    bloqueado: 4
    ensaio: 2
    0

Dica: Path.is_file() e Path.is_dir() já dizem se existe E se é do tipo
certo.
print(..., file=sys.stderr) escreve na saída de erro. Rode
"python exercicios/ex14_cli.py --help" para ver sua CLI.

Teste com:  python curso.py testar 14 ex14
```

Desafio do módulo 14 — A API do Validador (extensão FastAPI)

Contexto

O Validador já roda pela linha de comando. Agora o PCP quer consultar, por um sistema deles, se uma OP pode ter certificado. E a supervisão quer ver as tolerâncias de uma especificação sem abrir o banco. A resposta profissional é uma **API**: outra porta de entrada para o **mesmo núcleo**.

A regra de ouro do desafio: a API **não reescreve regra nenhuma**. Ela importa o pacote `validador_certificados` (do `projeto-completo/`) e só traduz HTTP para objetos do domínio e de volta. Se você se pegar copiando a regra de tolerância para dentro da API, pare: algo está errado na arquitetura.

Requisitos obrigatórios

1. Uma função `criar_app(caminho_regras)` que carrega as especificações do banco **uma vez** e devolve o `FastAPI`.
2. `GET /saude` → `{"status": "ok", "especificacoes": <quantidade>}`.
3. `GET /especificacoes` → lista com título, texto, linha e tolerâncias de cada especificação. Aceita `?linha=A` para filtrar.
4. `GET /especificacoes/{titulo}` → uma especificação (sem diferenciar maiúsculas). Título inexistente → **404** com a mensagem no `detail`.
5. `POST /validar` com um corpo Pydantic: `op` (só dígitos, 4 a 6), `linha` (`A` a `D`), `especificacao` (opcional), `valores`, `nominais` e `observacao`. Responde `op`, `especificacao` usada, `aprovado`, `liberado`, `pode_emitir` e `motivos`.
   - Especificação desconhecida → **422** com a mensagem do núcleo.
   - Corpo inválido (linha "E", OP com letra) → **422** automático do Pydantic.
6. Testes com `TestClient` cobrindo cada rota e cada código de status.
7. A documentação automática em `/docs` mostra exemplos de corpo (`examples=`).

Extras opcionais

- `POST /serie`: recebe a planilha (upload `.xlsx`) e uma lista de pedidos em JSON e devolve a triagem completa (fila + bloqueados). Dica: `UploadFile` e um arquivo temporário.
- Autenticação simples por token no cabeçalho (`X-Token`) com uma dependência.
- Guardar cada validação numa tabela `consulta` (histórico de quem perguntou o quê).

- Rodar a API como serviço no servidor da fábrica (ver "Para ir além" do módulo 13).

Exemplo de execução

A solução de referência faz uma demonstração com o `TestClient` (sem abrir servidor):

```
python 14-projeto-final/desafio/solucao_referencia.py
GET  /saude                       -> 200 {'status': 'ok',
'especificacoes': 7}
GET  /especificacoes/esp-0412     -> 200 {'titulo': 'ESP-0412',
'tolerancias': 2}
GET  /especificacoes/ESP-9999     -> 404 {'detail': "especificação 'ESP-
9999' não cadastrada"}
POST /validar                     -> 200 {'op': '24518', 'especificacao':
'PADRAO-A', 'aprovado': True, 'liberado': True, 'pode_emitir': True,
'motivos': []}
POST /validar                     -> 200 {'op': '24519', 'especificacao':
'PADRAO-A', 'aprovado': False, 'liberado': True, 'pode_emitir': False,
'motivos': ['espessura 44.8 fora de 40 ± 10% (36 a 44)']}
POST /validar                     -> 422 {'detail': "especificação 'ESP-
9999' não cadastrada"}
POST /validar                     -> 422 {'erro_de_validacao_no_campo':
'linha'}
```

Para subir o servidor de verdade e testar no navegador:

```
python 14-projeto-final/desafio/solucao_referencia.py --servir
```

Abra `http://127.0.0.1:8000/docs`, clique em `POST /validar`, "Try it out", e mande o exemplo que já aparece preenchido.

Critérios de aceite

- [ ] Nenhuma regra de negócio dentro da API: tudo vem de `validador_certificados`.
- [ ] O banco é aberto só para leitura e carregado uma vez, na criação do app.
- [ ] `GET /especificacoes/esp-0412` funciona (maiúsculas não importam).
- [ ] Título inexistente → 404; especificação desconhecida no `/validar` → 422.
- [ ] Corpo inválido → 422 sem você escrever um `if` sequer (Pydantic faz).
- [ ] `pode_emitir` só é `true` quando aprovado **e** liberado.
- [ ] Pelo menos 8 testes com `TestClient`, todos passando, sem servidor no ar.
- [ ] `ruff check` e `ruff format --check` limpos.
- [ ] `/docs` mostra exemplos de corpo prontos para testar.

Dicas de como começar

1. Comece pelo `/saude` e por um teste dele. Só depois avance.
2. Crie os modelos Pydantic olhando para os dataclasses: `ToleranciaSaida` tem os mesmos campos de `Tolerancia`. `ToleranciaSaida(**vars(tolerancia))` converte.
3. No `/validar`, monte uma `Medicao` com `aba=f"Linha {linha}"` e reaproveite `escolher_especificacao`, `validar_medicao` e `esta_liberado`. São 15 linhas.
4. Transforme `EspecificacaoDesconhecida` em `HTTPException(422, detail=str(erro))`. É a camada de entrada traduzindo um erro do domínio para a língua do HTTP.
5. Para importar o pacote sem instalar, veja como a solução de referência mexe no `sys.path`. No seu repositório próprio, prefira `pip install -e .` e coloque a API em `src/validador_certificados/api.py`.

Quiz — Módulo 14: Projeto final

Responda sem rodar. Depois confira no gabarito, no fim da página.

1. Em qual camada fica a função que decide se a espessura 44,8 µm está dentro de 40 ± 10%?

- a) Na CLI, porque é ela que recebe os argumentos.
- b) Na leitura da planilha, porque é de lá que o valor vem.
- c) No núcleo de regras, que não lê nem escreve arquivo nenhum.
- d) No emissor, porque é ele que decide se emite.

2. O que este código imprime?

```
print(37 * (1 - 10 / 100), round(37 * (1 - 10 / 100), 4))
```

3. O que este código imprime?

```
padrao = {"cof": 0.35, "selagem": 8}
cliente = {"cof": 0.25}
print({**padrao, **cliente})
print({**cliente, **padrao})
```

4. Qual das observações abaixo é considerada liberada pelo padrão `r"\bcof\s*[:-]?\s*liberad[oa]\b"` (aplicado ao texto já normalizado)?

- a) `cof: nao liberado`
- b) `liberado para corte; cof pendente`
- c) `cof - liberada`

- d) nenhuma das três

5. Por que a especificação de título desconhecido (`ESP-9999`) vira pendência em vez de usar o padrão da linha?

- a) Porque o padrão da linha é mais rígido e reprovaria a OP.
- b) Porque usar uma regra "parecida" pode emitir um certificado com tolerâncias erradas; o sistema não chuta.
- c) Porque o SQLite não aceita títulos desconhecidos.
- d) Porque a especificação do cliente sempre tem todas as características.

6. Onde está o erro? O programa abaixo deveria listar as especificações, mas dá `sqlite3.OperationalError: no such table: especificacao`, e um arquivo novo aparece na pasta.

```
import sqlite3

conexao = sqlite3.connect("regras_errado.db")  # o nome certo é regras.db
for linha in conexao.execute("SELECT titulo FROM especificacao"):
    print(linha)
```

7. O que este código imprime?

```
from dataclasses import dataclass, field, replace


@dataclass
class ItemFila:
    op: str
    situacao: str
    motivos: list[str] = field(default_factory=list)


a = ItemFila("24518", "emitir")
b = replace(a, situacao="ensaio")
b.motivos.append("x")
print(a.situacao, b.situacao, a.motivos)
```

8. A chave de idempotência da emissão é `(OP, NF)`, e não só a OP. Por quê?

- a) Porque a NF é mais curta.
- b) Porque a mesma OP pode sair em duas notas (entregas parciais), e cada nota precisa do seu certificado.
- c) Porque o TOTVS exige a NF em maiúsculas.
- d) Tanto faz: OP sozinha daria o mesmo resultado.

9. Em `emitir_fila`, por que capturar só `ErroEmissao` e não `except Exception`?

- a) `except Exception` não funciona dentro de laço.
- b) Para que um bug (ex.: `AttributeError` num seletor) apareça e pare a série, em vez de virar silenciosamente "falha de tela" em todas as OPs.
- c) Porque `ErroEmissao` é mais rápido.
- d) Porque o Playwright só levanta `ErroEmissao`.

10. O que este código imprime?

```
from dataclasses import dataclass, field


@dataclass
class ResultadoValidacao:
    motivos: list[str] = field(default_factory=list)

    @property
    def aprovado(self) -> bool:
        return not self.motivos


r = ResultadoValidacao()
print(r.aprovado)
r.motivos.append("cof 0.38 acima do máximo 0.35")
print(r.aprovado)
```

11. A CLI devolve código de saída `1` quando houve falha de emissão. Para que isso serve?

- a) Para o Python liberar memória.
- b) Para o Agendador de Tarefas (ou o GitHub Actions) saber que a execução precisa de atenção humana, sem ninguém ler o texto da tela.
- c) Para o Excel abrir o relatório automaticamente.
- d) Não serve para nada; é só costume.

12. Você clonou o projeto, rodou `python -m pytest` e recebeu `ModuleNotFoundError: No module named 'validador_certificados'`. Qual é a correção mais profissional?

- a) Copiar todos os arquivos de `src/validador_certificados` para a pasta `tests`.
- b) Ativar o ambiente virtual e rodar `python -m pip install -e ".[dev]"`.
- c) Renomear `src` para `validador_certificados`.
- d) Apagar o `pyproject.toml`.

13. Qual destes arquivos **não** deve ir para o repositório público do portfólio?

- a) `tests/test_regras.py`
- b) `dados/planilha_valores.xlsx` com dados fictícios
- c) a planilha real do laboratório com OPs e clientes da empresa
- d) `.github/workflows/testes.yml`

14. O modo ensaio é o padrão da CLI e emitir de verdade exige `--emitir`. Qual é a razão principal?

- a) Ensaio é mais rápido.
- b) O erro mais caro é emitir certificado errado; o padrão seguro protege contra o esquecimento de quem roda.
- c) O argparse não aceita flags com valor padrão `True`.
- d) O TOTVS bloqueia robôs que rodam sem flag.

Gabarito

1. **c.** Regra de negócio fica no núcleo puro: recebe números, devolve resultado. Assim se testa sem planilha e serve igual para a CLI, para a API e para o robô.
2. **`33.300000000000004 33.3`.** O float não representa 0,9 exatamente. Por isso os limites são arredondados com `round(..., 4)`: sem isso, 33,3 seria reprovado.
3. **`{'cof': 0.25, 'selagem': 8}`** e depois **`{'cof': 0.35, 'selagem': 8}`**. No `{**a, **b}`, quem vem depois vence. A ordem `{**padrao, **cliente}` é a que faz o cliente sobrescrever o padrão.
4. **c.** Em (a), depois de `cof:` vem `nao`, e o padrão exige `liberad` logo depois dos separadores. Em (b), `liberado` aparece antes de `cof`, e o padrão exige `cof` primeiro.
5. **b.** "Nunca chutar": uma pendência custa um minuto do técnico; um certificado errado custa uma reclamação de cliente.
6. **`sqlite3.connect` cria o arquivo se ele não existe.** O banco novo está vazio, por isso "no such table". Abra só para leitura (`?mode=ro` com `uri=True`) ou confira `Path(...).is_file()` antes: o erro passa a dizer o que realmente aconteceu.
7. **`emitir ensaio ['x']`.** `replace` faz cópia **rasa**: o novo objeto aponta para a MESMA lista `motivos`. É por isso que `emitir_fila` cria uma lista nova (`item.motivos + [...]`) em vez de dar `append`.
8. **b.** Entregas parciais existem. Com a OP sozinha, a segunda nota ficaria sem certificado.
9. **b.** Erro esperado (uma tela que não respondeu) é tratado e a série continua. Erro inesperado é bug: esconder um bug só adia e espalha o problema.
10. **`True`** e depois **`False`**. A propriedade é calculada na hora a partir dos motivos: não existe um `aprovado` guardado que possa ficar desatualizado.

11. **b.** Código de saída é a forma padrão de um programa dizer "deu certo" (0) ou "precisa de alguém" (diferente de 0) para outro programa.
12. **b.** Com o src layout, o pacote precisa ser instalado. O `-e` (editável) faz o Python enxergar o código da pasta `src` sem copiar nada.
13. **c.** Dado real de cliente e de produção é da empresa. No portfólio, só dados fictícios, gerados por script.
14. **b.** Um padrão seguro protege contra o esquecimento. Para fazer a ação perigosa, é preciso pedir explicitamente.

Guia de carreira: do laboratório de qualidade à programação

> Escrito para quem trabalha em laboratório de qualidade numa indústria de embalagens plásticas e quer trabalhar com programação. Números e rankings conferidos em **outubro de 2026**. Mercado muda: confira as fontes antes de decisões grandes.

Neste guia, cada afirmação vem marcada:

- 📊 **Fato:** dado com fonte (link no texto e na lista de fontes no fim).
- 💬 **Opinião:** minha leitura do mercado e conselho prático. Pode e deve ser questionada.

Quando não achei fonte confiável para um número, **não dei número**.

Resumo em 1 minuto

- **Fique com Python.** Para dados e automação, que são os seus objetivos, ele é a escolha certa. Some **SQL** a ele desde já.
- **O caminho mais rápido é o híbrido "qualidade + dados"** dentro da própria indústria. Você já conhece o processo, os dados e as pessoas. Isso vale muito.
- **Prova vale mais que certificado.** Uma automação real funcionando no laboratório, com horas economizadas medidas, abre mais portas que dez cursos concluídos.
- **Portfólio no GitHub** com os projetos do curso + **LinkedIn** contando essa história.
- No fim deste guia há um **plano de 6 meses** e um **de 12 meses**.

---

1. Python vale a pena ou devo mudar de linguagem?

1.1 O que os dados dizem

📊 **Fato:** na pesquisa do Stack Overflow de 2026 (mais de 30 mil participantes de 169 países), as linguagens mais usadas por todos os respondentes foram JavaScript (62,0%), SQL (58,4%), HTML/CSS (58,2%) e Python (58,0%). Depois vêm Bash/Shell

(51,3%), TypeScript (43,8%), Java (27,6%) e C# (27,1%). (Stack Overflow Developer Survey 2026, Technology; anúncio dos resultados)

📊 **Fato:** na edição anterior, o uso de Python tinha subido 7 pontos percentuais de 2024 para 2025, um dos maiores saltos da pesquisa. (Stack Overflow Developer Survey 2025, Technology)

📊 **Fato:** Python segue em 1º lugar no índice TIOBE de outubro de 2026. O TIOBE mede popularidade por buscas na internet, não vagas de emprego. (TechRepublic sobre o TIOBE de outubro de 2026; índice oficial)

📊 **Fato:** no relatório Octoverse 2025 do GitHub, o TypeScript passou Python e JavaScript como linguagem mais usada **no GitHub** (agosto de 2025). O mesmo relatório aponta Python como líder em projetos de IA e ciência de dados. (GitHub Octoverse 2025)

📊 **Fato:** no Brasil, entre profissionais de dados, Python e SQL aparecem como as ferramentas mais usadas nas análises da pesquisa State of Data Brazil, feita pela comunidade Data Hackers com a Bain & Company. (State of Data Brazil 2025-2026, Bain; dados abertos da pesquisa)

📊 **Fato:** o relatório da Brasscom sobre o mercado de trabalho de tecnologia aponta que, entre 2019 e 2024, a demanda foi de 665.403 profissionais, enquanto 464.569 pessoas se formaram em cursos da área entre 2018 e 2023. (Brasscom, Perspectivas do Mercado de Trabalho do Macrossetor TIC)

💬 **Opinião:** a falta de profissionais é real nas vagas experientes. Na vaga de entrada (júnior), a disputa ficou mais dura em 2025 e 2026, em parte porque ferramentas de IA fazem tarefas simples que antes iam para iniciantes. Não encontrei uma estatística oficial e confiável sobre vagas júnior no Brasil, então trate isso como tendência, não como número. A consequência prática: **quem chega com conhecimento de um negócio real (como você chega com a indústria) se destaca de quem só fez cursos.**

1.2 Comparação honesta das linguagens

Linguagem	Onde é usada	Para você
Python	Dados, automação, IA, scripts, backend (FastAPI, Django)	💬 Melhor encaixe. Curva de aprendizado mais suave, e é a língua dos dados.
JavaScript / TypeScript	Web (obrigatória no navegador), backend com Node.js	💬 Muitas vagas, mas muita concorrência de iniciantes. Só faz sentido se você quiser criar sites e telas.

Linguagem	Onde é usada	Para você
Java	Sistemas corporativos, bancos, grandes empresas	💬 Mercado sólido no Brasil para backend corporativo. Mais verbosa, mais distante de dados.
C# (.NET)	Sistemas corporativos no ecossistema Microsoft, jogos (Unity)	💬 Boa opção em empresas "100% Microsoft". Mesma observação do Java sobre dados.
SQL	Bancos de dados: **todo** trabalho com dados	💬 Não é alternativa ao Python: é **complemento obrigatório**. Comece já.

Um nicho que pouca gente lembra: o TOTVS Protheus, que você usa no trabalho, é customizado com linguagens próprias da TOTVS, o AdvPL e o TLPP. (TDN TOTVS: TLPP x AdvPL) 💬 **Opinião:** é um mercado real de desenvolvedor de ERP no Brasil, mas pequeno e preso a um fornecedor. Vale como curiosidade e como ponte com a TI da sua empresa, não como primeira linguagem.

1.3 Recomendação

💬 Opinião, com convicção: não mude de linguagem. Fique com Python e some SQL.

1. Seus objetivos (dados e automação) são o território do Python.
2. O projeto que você quer construir (validar planilhas, ler e-mails, gerar certificados) é Python puro: pandas, openpyxl, Playwright.
3. A segunda linguagem é muito mais fácil que a primeira. Se um dia uma vaga pedir Java ou C#, com a base deste curso você aprende em semanas, não em anos.
4. Trocar de linguagem no começo é a forma mais comum de nunca sair do começo.

Quando reconsiderar: se você descobrir que gosta mesmo é de construir telas e sites, aí JavaScript/TypeScript entra. Mas descubra isso **depois** de terminar o curso.

---

2. As trilhas possíveis

2.1 Híbrido "qualidade + dados" (a transição mais rápida)

Cargos com esse perfil (os nomes variam por empresa): analista de qualidade com foco em dados, analista de melhoria contínua, analista de processos, analista de dados industriais, analista de BI da área industrial.

No dia a dia: automatizar relatórios do laboratório, montar painéis de CEP (gráficos de controle), cruzar dados do ERP com resultados de ensaio, investigar a causa de reclamações com dados, calcular capacidade de processo (Cp, Cpk).

Por que é mais rápida (💬 opinião): você não precisa convencer um recrutador desconhecido. Você já está dentro, conhece o processo, fala a língua da produção e sabe onde estão as dores. Um analista de dados vindo de fora leva meses para aprender o que você já sabe sobre espessura, gramatura, COF e selagem.

O que exige além do curso: Excel avançado, Power BI, SQL, estatística aplicada à qualidade (CEP, capacidade, análise do sistema de medição).

2.2 Analista de Dados / BI

No dia a dia: responder perguntas de negócio com dados. Consultas SQL, limpeza de dados, painéis no Power BI, relatórios, apresentações.

O que exige: SQL (a habilidade número um), Excel, Power BI, Python com pandas, estatística básica, comunicação ("contar a história" dos números).

📊 **Fato:** a Robert Half publica, no Guia Salarial 2026, faixas mensais brutas para Analista de BI: pleno de R$ 7.650 a R$ 12.850 e sênior de R$ 10.900 a R$ 18.250 (no nível júnior, a mesma fonte indica R$ 5.600 a R$ 9.000). (Robert Half: Analista de BI Pleno; Analista de BI Sênior)

⚠️ Leia com cuidado: consultorias de recrutamento como a Robert Half trabalham mais com empresas médias e grandes, e os valores variam muito por cidade e porte da empresa. Em indústrias do interior, espere valores menores. Sites como o Glassdoor trazem relatos de profissionais, mas muitas vezes com amostras pequenas (às vezes menos de 20 salários), e os números oscilam bastante de uma página para outra.

2.3 Engenheiro de Dados

No dia a dia: construir e manter os "encanamentos" de dados (pipelines): levar dados do ERP, de sensores e de planilhas para um banco ou data lake, limpos e no horário.

O que exige: SQL avançado, Python, modelagem de dados, uma nuvem (Azure, AWS ou Google Cloud), Docker, ferramentas como Spark, Airflow ou dbt, Git no dia a dia.

📊 **Fato:** na pesquisa State of Data Brazil 2025-2026, 43,5% dos engenheiros de dados e de machine learning declararam ganhar acima de R$ 12 mil por mês, contra 11,6% dos analistas de dados. Entre os respondentes de nível sênior, 52,2% estavam acima de R$ 12 mil. A amostra é de quem escolheu responder, não um censo. (relatório State of Data Brazil 2025-2026, PDF)

💬 **Opinião:** raramente é a primeira vaga. O caminho comum é analista de dados → engenheiro de dados, depois de 1 a 3 anos lidando com dados de verdade.

2.4 Desenvolvedor Backend Python

No dia a dia: criar e manter sistemas: APIs, regras de negócio, integrações com bancos de dados e outros sistemas.

O que exige: Python sólido, FastAPI ou Django, SQL, testes automatizados, Git com branches e pull requests, HTTP, Docker, noções de nuvem e Linux.

📊 **Fato:** o Guia Salarial 2026 da Robert Half traz, para Desenvolvedor(a) Back-End Júnior, faixa nacional de R$ 6.050 a R$ 8.750 (mediana de R$ 6.850). Por cidade, a variação é grande: São Paulo de R$ 5.750 a R$ 8.300, Rio de Janeiro de R$ 7.050 a R$ 10.200. (Robert Half: Back-End Júnior; São Paulo; Rio de Janeiro)

💬 **Opinião:** no Brasil, boa parte das vagas de backend pede Java, C# ou JavaScript/TypeScript. Vagas de backend **em Python** existem, mas são menos numerosas que as de dados em Python. É uma trilha ótima, só que mais competitiva para quem vem de outra área.

2.5 Automação / RPA

No dia a dia: robôs que fazem tarefas repetitivas: baixar relatórios, preencher sistemas, mandar e-mails, consolidar planilhas. Exatamente o que o seu projeto final faz com o TOTVS.

O que exige: Python para scripts, automação de navegador (Playwright, Selenium), APIs, Excel e Outlook, boas práticas de robô (log, modo ensaio, tratamento de falhas). Muitas empresas também usam ferramentas próprias de RPA (como Power Automate ou UiPath).

💬 **Opinião:** é a trilha mais fácil de **provar** dentro da empresa, porque o resultado é medido em horas economizadas. Combina muito bem com o híbrido "qualidade + dados". Não encontrei uma fonte salarial confiável e específica para essa trilha no Brasil.

2.6 Quadro comparativo

Trilha	Distância a partir do curso (💬)	O que mais falta	Primeira vaga típica
Qualidade + dados	Curta	Power BI, SQL, estatística de qualidade	Transição interna
Analista de Dados/BI	Curta a média	SQL todo dia, Power BI	Analista júnior, assistente de dados
Automação/RPA	Curta a média	Projetos de automação provados	Analista de automação júnior
Backend Python	Média	Docker, nuvem, mais projetos de API	Desenvolvedor júnior
Engenheiro de Dados	Longa	Nuvem, pipelines, modelagem	Geralmente depois de ser analista

---

3. O que estudar além deste curso

Habilidade	Por quê	Para quais trilhas	Quanto (💬)
SQL	Está em praticamente toda vaga de dados	Todas	Desde já, 3 vezes por semana, exercícios curtos
Excel avançado	Tabela dinâmica, PROCX, Power Query: a língua da empresa	Híbrido, dados	O suficiente para não depender de ninguém
Power BI	Ferramenta de painel mais pedida no Brasil (💬)	Híbrido, dados	1 a 2 meses de prática com dados reais anonimizados
Estatística	CEP, capacidade, média × mediana, desvio, testes de hipótese	Híbrido, dados	Você já tem a base do laboratório: formalize
Git	Todo time de tecnologia usa	Todas	O curso cobre (módulos 00 e 09)
Docker	Rodar o mesmo sistema em qualquer máquina	Backend, eng. de dados	Depois do curso, 2 a 4 semanas

Habilidade	Por quê	Para quais trilhas	Quanto (💬)
Nuvem	Onde os sistemas e os dados vivem hoje	Backend, eng. de dados	Uma nuvem só, o básico, depois do curso
Inglês técnico	Documentação, erros e respostas estão em inglês	Todas	Leitura: um pouco todo dia

📊 **Fato:** a certificação de Power BI da Microsoft (exame PL-300) cobre preparação de dados com Power Query, modelagem, DAX, visualização e gestão de relatórios, e a Microsoft oferece trilhas de estudo e um simulado gratuitos. (guia de estudo do PL-300)

💬 **Opinião:** certificação ajuda a passar por filtros de RH, mas não substitui portfólio. Faça a do Power BI só se mirar vagas de dados e depois de já usar a ferramenta de verdade.

Links para estudar cada item: recursos.md.

---

4. Portfólio com os projetos do curso

O `PLANO-DE-ESTUDOS.md` define 4 marcos. Cada um vira um repositório próprio no GitHub:

Marco	Projeto	O que mostra para um recrutador
1	Desafio do módulo 04	Lógica, funções, código organizado
2	Desafio do módulo 08	Orientação a objetos, uma ferramenta útil
3	Desafio do módulo 11 + laboratório de salários do módulo 10	SQL, pandas, análise de dados: o "kit" de vaga de dados
4	Validador de Certificados (módulo 14)	Sistema completo, com testes, de um problema real da indústria

Como transformar um projeto do curso em projeto de portfólio:

1. **Repositório próprio**, com nome claro (`validador-certificados` , não `projeto-final`).

2. **README que vende o projeto**: o problema (em 2 frases), o que o programa faz, como rodar (comandos copiáveis), um print ou GIF, como rodar os testes, o que você aprendeu.
3. **Testes rodando** e, se possível, rodando sozinhos a cada push (GitHub Actions, módulo 14).
4. **Dados fictícios.** Nunca dados reais da empresa, nem anonimizados "mais ou menos".
5. **Commits com boas mensagens.** Recrutador técnico olha o histórico.

Sobre o `salaries.csv` do seu repositório: ele traz salários de profissões de dados em dólar, de vários países. 💬 É ótimo para praticar e para portfólio, mas não use como referência de salário no Brasil.

Ideias de projeto extra com o seu domínio (💬 diferenciam muito):

- Painel de CEP de espessura com limites de controle, a partir de um CSV.
- Comparador de laudos: lê dois arquivos de resultados e destaca diferenças fora da tolerância.
- API de tolerâncias: consulta a especificação de um produto por código (módulo 13).
- Robô em "modo ensaio" que preenche um formulário de teste no navegador (módulo 12).

---

5. GitHub e LinkedIn

5.1 GitHub

- **Foto e nome reais**, bio de uma linha ("Qualidade industrial → dados e automação com Python").
- **README de perfil** (um repositório com o mesmo nome do seu usuário): quem você é, o que está estudando, links para os 4 projetos.
- **Fixe (pin) os 4 melhores repositórios.** O resto pode ficar, mas não é vitrine.
- 💬 Os "quadradinhos verdes" de atividade ajudam a mostrar constância, mas qualidade dos projetos pesa mais. Não faça commits vazios só para pintar o gráfico.
- README em português serve para vagas no Brasil. Um parágrafo em inglês no topo é um bônus.

5.2 LinkedIn

- **Título:** o que você é + para onde vai. Exemplo: "Laboratório de Qualidade | Python, SQL e Power BI | Dados na indústria de embalagens".
- **Sobre:** sua história em 5 linhas. Do laboratório para os dados, o que já automatizou, o que procura.
- **Projetos:** cada marco do portfólio com link para o GitHub e o resultado.
- **Publique de vez em quando** (💬 uma vez a cada 2 semanas já basta): um aprendizado, um antes/depois de uma automação (sem dados da empresa), um gráfico do seu projeto.
- **Conecte-se** com pessoas de dados e qualidade da sua região e do seu setor, e com recrutadores de tecnologia.
- **Honestidade sempre:** escreva "estudando" no que você está estudando. Exagero aparece na primeira entrevista técnica.

---

6. Como conseguir a primeira oportunidade

6.1 Transição interna (💬 a melhor aposta)

1. **Escolha uma dor real e pequena** do laboratório: o relatório semanal que leva 2 horas, a planilha consolidada à mão, a conferência de tolerâncias.
2. **Automatize** com o que o curso já ensinou (a partir do módulo 06 dá para muita coisa).
3. **Meça**: quanto tempo levava, quanto leva agora, quantos erros evitou. "Economiza 6 horas por mês" convence qualquer gestor.
4. **Converse com a TI antes de colocar em uso**: acesso a sistemas, segurança, LGPD e onde o código vai ficar. Isso mostra maturidade e evita problemas.
5. **Apresente ao gestor** o antes/depois e proponha: "posso dedicar X horas por semana a isso?".
6. **Aproxime-se** de quem já trabalha com dados ou TI na empresa (BI, engenharia de processos, TI). Pergunte como eles trabalham. Ofereça ajuda.
7. **Fique de olho no recrutamento interno.** Muitas vagas de analista são preenchidas por gente de dentro que já provou valor.

O seu projeto final (o Validador de Certificados) foi pensado para isso: rodar primeiro em **modo ensaio**, sem gravar nada no TOTVS, comparando com o processo manual.

6.2 Estágio e trainee

Estágio exige estar matriculado em curso técnico ou superior. 💬 Se você ainda não tem graduação, um tecnólogo (Análise e Desenvolvimento de Sistemas, Ciência de Dados, Gestão da Produção com ênfase em dados) abre a porta do estágio e passa pelo filtro de RH de indústrias grandes. Não é obrigatório para trabalhar com programação, mas ajuda, principalmente em empresas tradicionais.

6.3 Freelas e projetos pequenos

💬 Automatizar planilhas e relatórios para pequenos negócios (de conhecidos, de fornecedores) dá experiência e história para contar. Cuidados: escopo por escrito, começar pequeno, não usar o tempo nem os dados do seu emprego, e não cobrar barato demais só para "pegar experiência".

6.4 Busca externa

- Onde: LinkedIn e plataformas de recrutamento usadas pelas empresas da sua região.
- Termos de busca: "analista de dados júnior", "analista de BI júnior", "assistente de dados", "analista de melhoria contínua", "analista de automação", "desenvolvedor Python júnior", "RPA".
- 💬 Candidate-se mesmo sem cumprir 100% dos requisitos. A lista da vaga costuma ser um desejo, não um filtro rígido.
- 💬 Indústrias do seu setor (embalagens, plásticos, alimentos, bens de consumo) valorizam quem entende chão de fábrica. Comece por elas.

---

7. Como se preparar para entrevistas técnicas

7.1 O formato mais comum (💬)

1. **Triagem com RH:** sua história, motivação, pretensão salarial.
2. **Teste técnico:** online ou para fazer em casa. Para dados: SQL, um caso em Excel ou Power BI, às vezes pandas. Para desenvolvimento: um exercício de lógica ou uma pequena API.
3. **Entrevista técnica:** você explica um projeto seu e resolve um problema pensando em voz alta.
4. **Conversa com o gestor:** como você trabalha, como lida com erro e prazo.

7.2 Como treinar

- **SQL:** `SELECT`, `WHERE`, `GROUP BY`, `HAVING`, `JOIN`, subconsultas. Um exercício por dia nas plataformas de recursos.md.
- **Python:** listas, dicionários, funções, ler CSV, pandas (`groupby`, `merge`, filtros). Os exercícios do curso são um ótimo treino: refaça os de revisão sem olhar.
- **Seu projeto em 2 minutos:** problema → o que você fez → resultado (com número). Treine em voz alta.
- **Histórias no formato STAR** (Situação, Tarefa, Ação, Resultado). Exemplo: "o relatório de liberação levava 2 horas (S); eu precisava reduzir erros de digitação (T); escrevi um script em Python que lê a planilha e confere as tolerâncias (A); hoje leva 5 minutos e não tivemos mais erro de transcrição (R)".
- **Estatística de conversa:** média × mediana, desvio padrão, o que é um outlier, o que um gráfico de controle mostra. Você vive isso no laboratório: use a seu favor.
- **Simulado com IA:** peça para a IA ser o entrevistador, fazer uma pergunta por vez e avaliar suas respostas no fim.

7.3 Na hora

- Pense em voz alta. O entrevistador avalia o raciocínio, não só a resposta.
- Não sabe? Diga o que sabe e como descobriria. "Não sei, mas eu procuraria na documentação do pandas por..." vale mais que inventar.
- Faça perguntas sobre o time, os dados e as ferramentas. Mostra interesse real.

---

8. Plano de 6 meses e plano de 12 meses

💬 Os dois planos são sugestões, não garantias. Eles seguem o `PLANO-DE-ESTUDOS.md`, que leva cerca de 31 semanas com 1 hora por dia.

8.1 Plano de 6 meses: transição híbrida "qualidade + dados"

Ritmo: **1h30 por dia, 5 dias por semana** (1h de curso + 30 min de SQL ou Power BI). Objetivo: assumir tarefas de dados no seu próprio trabalho e ter os 3 primeiros marcos do portfólio.

Mês	Curso	Além do curso	Entrega do mês
1	Módulos 00 a 02	SQL básico (SELECT, WHERE) 3x por semana	Ambiente pronto, LinkedIn atualizado, `DIARIO.md` começado

Mês	Curso	Além do curso	Entrega do mês
2	Módulos 03 e 04	SQL com GROUP BY; Power BI: primeiros painéis	Marco 1 no GitHub; lista de 3 tarefas do laboratório para automatizar
3	Módulos 05 e 06	Power BI com dados fictícios do laboratório	Primeira automação real (ler planilha/CSV e gerar resumo), com tempo economizado medido
4	Módulos 07 e 08	Estatística de qualidade (CEP, Cp/Cpk)	Marco 2; apresentação da automação ao gestor
5	Módulos 09 e 10	SQL com JOIN; Excel avançado	Painel de CEP em Python ou Power BI; conversa com a TI sobre acesso e governança
6	Módulo 11	Revisão de SQL para entrevista	Marco 3; proposta formal de horas dedicadas a dados, ou primeiras candidaturas internas

Ao fim dos 6 meses o curso ainda não acabou (faltam os módulos 12 a 14). Tudo bem: o objetivo aqui é a **primeira oportunidade**, não o diploma do curso.

8.2 Plano de 12 meses: curso completo + trilha escolhida

Ritmo: **1 hora por dia, 5 dias por semana**.

Período	Foco	Entrega
Meses 1 a 7	Curso completo, seguindo o `PLANO-DE-ESTUDOS.md`; SQL 2x por semana a partir do mês 2	Os 4 marcos no GitHub; ao menos uma automação em uso no trabalho
Mês 8	Escolher a trilha (dados ou automação/backend) com base no que você mais gostou	Validador rodando em modo ensaio no laboratório; LinkedIn reescrito para a trilha
Meses 9 e 10	**Dados:** Power BI a fundo, estatística, SQL avançado (PL-300 opcional). **Automação/backend:** Docker, publicar a API do módulo 13 numa nuvem, mais um robô	Um projeto extra completo da trilha
Meses 11 e 12	Candidaturas internas e externas, simulados de entrevista, ajustes no portfólio	Entrevistas acontecendo; meta: primeira posição na área

8.3 Sinais de que você está no caminho

- Você resolve exercícios novos sem abrir a solução na maioria das vezes.
- Alguém no trabalho já te procurou para "fazer aquela planilha automática".
- Você consegue explicar o seu Validador em 2 minutos para alguém de fora da área.
- Você tem pelo menos um número para contar: horas economizadas, erros evitados.

Fontes

Consultadas em outubro de 2026.

- Stack Overflow Developer Survey 2026, Technology: https://survey.stackoverflow.co/2026/technology
- Stack Overflow, anúncio dos resultados de 2026: https://stackoverflow.blog/2026/10/06/the-results-of-the-2026-developer-survey-are-here/
- Stack Overflow Developer Survey 2025, Technology: https://survey.stackoverflow.co/2025/technology
- TechRepublic, TIOBE Index de outubro de 2026: https://www.techrepublic.com/article/news-tiobe-october-2026-java-closes-cpp-gap/
- Índice TIOBE (página oficial): https://www.tiobe.com/tiobe-index/
- GitHub Octoverse 2025: https://github.blog/2025-10-28-octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/
- Bain & Company e Data Hackers, State of Data Brazil 2025-2026: https://www.bain.com/pt-br/insights/state-of-data-2025/ e relatório em PDF: https://www.bain.com/contentassets/aaff7ea7a43944dbac97232329c3c1b6/bain company-e-data-hackers_state-of-data-2025-2026.pdf
- Data Hackers, dados abertos do State of Data Brazil: https://github.com/data-hackers-br/state-of-data-brazil
- Brasscom, Perspectivas do Mercado de Trabalho do Macrossetor TIC: https://brasscom.org.br/wp-content/uploads/2025/02/Relatorio-Perspectivas-do-Mercado-de-Trabalho.pdf
- Robert Half, Guia Salarial 2026, páginas de cargo: https://www.roberthalf.com/br/pt/vagas-detalhes/analista-de-bi-pleno, https://www.roberthalf.com/br/pt/vagas-detalhes/analista-de-bi-senior, https://www.roberthalf.com/br/pt/vagas-detalhes/desenvolvedora-back-end-junior (e as variações por cidade citadas no texto)

- Glassdoor, salários de Desenvolvedor Python Júnior: https://www.glassdoor.com.br/Sal%C3%A1rios/desenvolvedor-python-junior-sal%C3%A1rio-SRCH_KO0,27.htm
- Microsoft Learn, guia de estudo do exame PL-300: https://learn.microsoft.com/pt-br/credentials/certifications/resources/study-guides/pl-300
- TOTVS TDN, TLPP x AdvPL: https://tdn.totvs.com/display/tec/TLPP+x+AdvPL

Recursos complementares

> Todos os links foram conferidos em **outubro de 2026**. A preferência é por material **gratuito** e, quando existe, **em português**. Sites mudam: se um link quebrar, pesquise pelo nome do recurso.

Não tente fazer tudo. Este curso já é a sua trilha principal. Use estes recursos para três coisas: ver um assunto explicado de outro jeito, praticar mais, e estudar o que fica fora do curso (SQL a fundo, Power BI, estatística, Docker).

Qual recurso usar em cada fase do curso

Fase do curso	Para ver de outro jeito	Para praticar mais
Módulos 00 a 04	Curso em Vídeo (Python, Mundos 1 e 2); Pense em Python	Lista de exercícios da Python Brasil; beecrowd (categoria iniciante)
Módulos 05 a 08	Pense em Python; Curso em Vídeo (Mundo 3)	Exercism (trilha de Python)
Módulo 09	Pro Git (capítulos 2 e 3)	Seus próprios projetos, com testes
Módulo 10	Python for Data Analysis; Kaggle Learn (Pandas)	Kaggle Learn (exercícios de Pandas)
Módulo 11	Kaggle Learn (Intro to SQL)	SQLBolt
Módulo 12	Automate the Boring Stuff with Python	Automatize algo real do seu trabalho
Módulo 13	Documentação do FastAPI em português	Uma API para um projeto seu
Depois do curso	Python Fluente; Docker; Power BI (Microsoft Learn)	Projetos de portfólio

---

1. Documentação oficial

- **Tutorial oficial do Python, em português**. Feito para quem já programa um pouco. Leia a partir do módulo 03 ou 04 deste curso.

- **Documentação do Python, em português**. Referência da linguagem e da biblioteca padrão. Aprenda a consultá-la: é o que profissionais fazem.
- **Configurar o ambiente Python no Windows (Microsoft Learn)**. Complementa o módulo 00.
- **Pro Git, em português**. O livro oficial do Git, gratuito. Capítulos 1 a 3 cobrem tudo o que você precisa no começo.
- **Documentação do FastAPI, em português**. Excelente tutorial passo a passo. A tradução não está completa: algumas páginas aparecem em inglês.
- **Documentação do pandas** (inglês). Para o módulo 10 em diante.
- **Docker: Get started** (inglês). Para depois do curso, nas trilhas de backend e engenharia de dados.

2. Livros

Gratuitos (versão online oficial)

- **Pense em Python**, de Allen B. Downey. Tradução da 2ª edição de *Think Python*, com texto cedido pela Editora Novatec. Didático, ótimo para quem está começando. (A 3ª edição em português é vendida pela Novatec.)
- **Python Fluente, 2ª edição**, de Luciano Ramalho, autor brasileiro. Um dos livros de Python mais respeitados do mundo, publicado online em português pelo próprio autor. É para **depois** do curso: o público é quem já programa e quer dominar o Python moderno.
- **Automate the Boring Stuff with Python**, de Al Sweigart (inglês). Livre para ler online. Automação de planilhas, PDFs, e-mails e navegador: casa direto com o módulo 12.
- **Python for Data Analysis, 3ª edição**, de Wes McKinney, o criador do pandas (inglês). Versão online aberta. A referência para o módulo 10.
- **Pro Git** (veja a seção 1).

3. Cursos em vídeo gratuitos

- **Curso em Vídeo: Python 3**, com Gustavo Guanabara. Dividido em Mundo 1, 2 e 3, com muitos exercícios. Gratuito no site e no YouTube (o certificado é pago). Linguagem muito acessível para iniciantes. O mesmo canal tem um curso de banco de dados com MySQL, útil como reforço do módulo 11.
- **Fundação Bradesco, Escola Virtual**. Cursos online gratuitos com certificado, entre eles "Linguagem de Programação Python – Básico". Bom para ter certificados de base no currículo.

- **CS50's Introduction to Programming with Python (Harvard)** (inglês). Aulas e exercícios gratuitos (há também a opção de certificado pago pelo edX). Excelente qualidade, para quem já lê inglês.
- **Kaggle Learn** (inglês). Microcursos gratuitos e práticos: Python, Pandas, Intro to SQL, visualização de dados.
- **Microsoft Learn: Introdução ao Power BI**. Trilha gratuita e oficial, em português. Depois dela, siga as trilhas de preparação de dados e de relatórios do mesmo site.

4. Plataformas de exercícios

- **Lista de exercícios da Python Brasil**. Clássica, em português, organizada por assunto (estrutura sequencial, decisão, repetição, listas, funções, strings, arquivos). Perfeita para os módulos 01 a 06.
- **beecrowd** (antigo URI Online Judge). Juiz online brasileiro com milhares de problemas. Comece pela categoria "Iniciante".
- **Exercism: trilha de Python** (inglês). Gratuito, com análise automática do seu código e mentoria opcional de voluntários.
- **SQLBolt** (inglês). Lições interativas de SQL no navegador, sem instalar nada e sem cadastro. O melhor jeito de começar SQL.
- **Python Tutor**. Não é lista de exercícios, mas ajuda muito: cole um código curto e veja, passo a passo, o que acontece com cada variável.

5. Comunidades brasileiras

- **Python Brasil**. Portal da comunidade brasileira de Python: guias, grupos regionais e eventos, incluindo a conferência anual Python Brasil.
- **Data Hackers**. Comunidade de dados e IA que se apresenta como a maior do Brasil. Newsletter, podcast, um Slack gratuito para membros e a pesquisa anual State of Data Brazil.
- **PyLadies Brasil**. Comunidade que incentiva a participação de mulheres em Python, com grupos em várias cidades.
- **Stack Overflow em Português**. Perguntas e respostas de programação em português. Antes de perguntar, pesquise; ao perguntar, use o modelo da seção 13.7 da aula do módulo 00.

💡 Grupos regionais de Python (os "GruPy" e meetups) aparecem no portal da Python Brasil. Ir a um encontro presencial é a forma mais rápida de conhecer gente que trabalha na área.

6. Dados, BI, Excel e estatística

- **Microsoft Learn: Automatizar a limpeza de dados com o Power Query**. Power Query no Excel: o mesmo motor de transformação do Power BI.
- **Guia de estudo do exame PL-300 (Power BI)**. O que a certificação de Power BI cobra, com links para trilhas e um simulado gratuito.
- **Khan Academy: Estatística e probabilidade**. Aulas gratuitas de estatística, do básico (média, mediana, desvio) a testes de hipótese. Confira o idioma no seletor do site: parte do conteúdo está em português.
- **State of Data Brazil** (Data Hackers + Bain). Relatório gratuito sobre o mercado de dados no Brasil: cargos, ferramentas e salários.

7. Mapas de carreira

- **roadmap.sh**. Mapas visuais do que estudar em cada carreira. Veja os de Data Analyst, Python e Backend. Não se assuste com o tamanho: ninguém sabe tudo aquilo para a primeira vaga.

Cuidados

- **PDFs "grátis" de livros pagos** em sites desconhecidos costumam ser pirataria e, às vezes, vírus. Use as versões oficiais listadas aqui.
- **Cursos pagos** podem valer a pena, mas só compre depois de esgotar os gratuitos e saber exatamente o que falta. Desconfie de promessa de emprego garantido.
- **Certificado não é portfólio.** Recrutador técnico quer ver código funcionando no GitHub.